GUÍA MLOPS

DE CERO A SENIOR / STAFF

VERSION 5.0 — PORTFOLIO EDITION

ÍNDICE

GUÍA MLOPS V5.0

PORTFOLIO EDITION

📚 Guía MLOps — Portfolio Edition

De Python Básico a Senior/Staff en MLOps

Programa completo de 8 semanas para construir el portafolio ML-MLOps-Portfolio desde cero.

🎯 ¿Qué Lograrás?

Al completar esta guía serás capaz de:

Habilidad	Nivel	Evidencia en el Portafolio
Código Python profesional	Senior	Type hints, Pydantic, SOLID en los 3 proyectos
Pipelines ML reproducibles	Senior	sklearn Pipeline unificado, sin data leakage
Testing & CI/CD	Senior	80%+ coverage, GitHub Actions, matrix testing
APIs de producción	Senior	FastAPI con validación, Docker multi-stage
Observabilidad	Staff	Prometheus, logging estructurado, drift detection
Pasar entrevistas técnicas	Staff	Simulacros completos, speech de 5-7 min

🧭 Cómo Usar Esta Guía

Perfil de Entrada

- Python básico (funciones, clases, módulos)
- Git elemental (clone, commit, push)
- Comodidad con la terminal

Método de Estudio

```
┌──────────────────────────────────────────────────────────────────────┐
│                        CICLO DE APRENDIZAJE                          │
├──────────────────────────────────────────────────────────────────────┤
│                                                                      │
│   1. LEER          2. REPLICAR         3. PRACTICAR      4. VALIDAR  │
│   ────────         ───────────         ────────────      ──────────  │
│   El módulo        En uno de los       Ejercicios de     Comparar con │
│   teórico          3 proyectos         EJERCICIOS.md     el portafolio │
│                                                                      │
│   💡 Cada módulo tiene sección "📦 Cómo se usó en el Portafolio" que  │
│      referencia código real de BankChurn, CarVision o TelecomAI.     │
│                                                                      │
└──────────────────────────────────────────────────────────────────────┘
```

Rutas de Aprendizaje

Si eres...	Ruta recomendada
Principiante	Seguir módulos 01-23 en orden, 8 semanas
DS con experiencia	Saltar a Fase 2 (módulo 07), revisar Fase 1 como referencia
Preparando entrevista	Ir directo a módulos 20-23 + Simulacros

📊 Roadmap Visual

```
╔═══════════════════════════════════════════════════════════════════════╗
╔═══════════════════════════════════════════════════════════════════════╗
╔═══════════════════════════════════════════════════════════════════════╗
```

```
[04] Entornos Reproducibles        [10] Experiment Tracking
[05] Git Profesional
[06] Versionado de Datos

FASE 3: MLOps CORE (Semanas 5-6)   FASE 4: PRODUCCIÓN (Semana 7)

[11] Testing para ML
[12] CI/CD con GitHub Actions
[13] Docker Avanzado               [17] Estrategias de Despliegue
[14] FastAPI para ML               [18] Infraestructura como Código
[15] Streamlit Dashboards
[16] Observabilidad

FASE 5: ESPECIALIZACIÓN (Semana 8)

[19] Documentación (Model Cards)
[20] Proyecto Integrador           🎯 PORTAFOLIO COMPLETO
[21] Glosario MLOps                   3 proyectos production-ready
[22] Checklist Profesional            CI/CD ≥80% coverage
[23] Recursos y Referencias           Listo para entrevistas
```

📖 Índice de Módulos

FASE 1: Fundamentos de Ingeniería (Semanas 1-2)

Objetivo: Establecer las bases de código profesional que usarás en todo el portafolio.

#	Módulo	Qué Aprenderás	Tiempo
01	Python Moderno	Type hints, Pydantic, dataclasses, SOLID	4 h
02	Diseño de Sistemas ML	ML Canvas, C4 Model, ADRs, arquitectura	4 h
03	Estructura de Proyecto	src/ layout, pyproject.toml, Makefile	3 h
04	Entornos Reproducibles	venv, Poetry, requirements, lockfiles	4 h
05	Git Profesional	Conventional Commits, pre-commit, branching	4 h
06	Versionado de Datos	DVC, pipelines de datos, remote storage	4 h

📦 **Aplicación en el Portafolio**: Todo el código de `common_utils/`, `pyproject.toml` y `.pre-commit-config.yaml`.

🎤 **Checkpoint Junior**: Al completar esta fase, practica con SIMULACRO_ENTREVISTA_JUNIOR.md

FASE 2: ML Engineering (Semanas 3-4)

Objetivo: Dominar el core de Machine Learning profesional: pipelines reproducibles y experimentos rastreables.

#	Módulo	Qué Aprenderás	Tiempo
07	sklearn Pipelines	Pipeline, ColumnTransformer, Custom Transformers	5 h
08	Ingeniería de Features	Data leakage, FeatureEngineer class, validación	4 h
09	Training Profesional	Clase Trainer, cross-validation, métricas	5 h
10	Experiment Tracking	MLflow tracking, Model Registry, signatures	4 h

📦 **Aplicación en el Portafolio**:
- `BankChurn-Predictor/src/bankchurn/pipeline.py` → Pipeline unificado
- `CarVision-Market-Intelligence/src/carvision/features.py` → FeatureEngineer class
- `mlruns/` en cada proyecto → Experimentos MLflow

FASE 3: MLOps Core (Semanas 5-6)

Objetivo: Implementar las prácticas que distinguen un proyecto ML profesional: testing, CI/CD, APIs y observabilidad.

#	Módulo	Qué Aprenderás	Tiempo
11	Testing para ML	Pirámide de testing, fixtures, 80%+ coverage	6 h
12	CI/CD con GitHub Actions	Matrix testing, coverage gates, security scanning	5 h
13	Docker Avanzado	Multi-stage builds, non-root, docker-compose	4 h
14	FastAPI para ML	Schemas Pydantic, /predict, /health, error handling	4 h
15	Streamlit Dashboards	Caching, tabs, visualizaciones, consumo de API	3 h
16	Observabilidad	Logging estructurado, Prometheus, drift detection	4 h

📦 **Aplicación en el Portafolio**:
- `tests/` en cada proyecto → 80%+ coverage
- `.github/workflows/ci-mlops.yml` → Pipeline CI/CD real
- `app/fastapi_app.py` → API de predicción
- `app/streamlit_app.py` → Dashboard interactivo

🎤 **Checkpoint Mid**: Al completar esta fase, practica con SIMULACRO_ENTREVISTA_MID.md

FASE 4: Producción (Semana 7)

Objetivo: Entender estrategias de despliegue e infraestructura como código.

#	Módulo	Qué Aprenderás	Tiempo
17	Estrategias de Despliegue	Lambda vs ECS vs K8s, blue-green, canary	4 h
18	Infraestructura como Código	Terraform basics, Kubernetes intro	3 h

📦 **Aplicación en el Portafolio**:
- `infra/terraform/` → Templates Terraform
- `k8s/` → Manifests Kubernetes
- `docker-compose.demo.yml` → Orquestación local

FASE 5: Especialización y Maestría (Semana 8)

Objetivo: Documentación profesional, validación del portafolio y preparación para entrevistas.

#	Módulo	Qué Aprenderás	Tiempo
19	Documentación ML	Model Cards, Dataset Cards, MkDocs	3 h
20	Proyecto Integrador	Rúbrica 100 puntos, checklist final	4 h
21	Glosario MLOps	100+ términos esenciales	Referencia
22	Checklist Profesional	Verificación pre-deploy, auditoría	2 h
23	Recursos y Referencias	Libros, cursos, papers, comunidades	Referencia

📦 **Aplicación en el Portafolio**:
- `docs/` en cada proyecto → Model Cards y READMEs profesionales
- `RUNBOOK.md` → Guía de operaciones

🎤 **Checkpoint Senior**: Al completar toda la guía, practica con SIMULACRO_ENTREVISTA_SENIOR_PARTE1.md + PARTE2

📚 Material Complementario

Práctica y Evaluación

Recurso	Descripción	Cuándo Usarlo
Ejercicios	Problemas prácticos por módulo	Después de cada módulo
Soluciones	Respuestas detalladas	Para verificar o desbloquear
Rúbrica de Evaluación	Criterios 100 puntos	Al finalizar un proyecto

Preparación de Entrevistas

Nivel	Recurso	Descripción	Cuándo Usarlo
🟢	Simulacro Junior	50 preguntas fundamentales	Semana 4 - 5
🟡	Simulacro Mid	60 preguntas técnicas	Semana 6 - 7
🔴	Simulacro Senior Parte 1	50+ preguntas avanzadas	Semana 8
🔴	Simulacro Senior Parte 2	System design, casos prácticos	Semana 8

Recurso Complementario	Descripción
Speech de Portafolio	Guión completo 5 - 7 min
Talking Points	Puntos clave concisos

Recursos Adicionales

Recurso	Descripción
SYLLABUS	Programa detallado semana a semana
Plan de Estudios	Cronograma día a día
Decisiones Técnicas	ADRs: por qué cada herramienta
Plantillas	Templates reutilizables
Guía Audiovisual	Cómo crear demos y videos
Guía de Mantenimiento	Operaciones y runbooks
📺 **Recursos por Módulo**	**Videos YouTube, Coursera y documentación por módulo**

🏗️ Los 3 Proyectos del Portafolio

Esta guía te prepara para construir estos 3 proyectos production-ready:

1. BankChurn-Predictor

```
📁 BankChurn-Predictor/
├── src/bankchurn/          # Código fuente
│   ├── config.py           # Configuración Pydantic
│   ├── pipeline.py         # Pipeline sklearn unificado
│   └── trainer.py          # Clase de entrenamiento
├── app/                    # APIs
│   ├── fastapi_app.py
│   └── streamlit_app.py
├── tests/                  # 79%+ coverage
└── Dockerfile              # Multi-stage, non-root
```

- **Problema**: Clasificación binaria (churn/no-churn)
- **Técnicas**: RandomForest, class weighting, SimpleImputer

- **Módulos clave**: 07, 09, 11, 14

2. CarVision-Market-Intelligence

```
📁 CarVision-Market-Intelligence/
├── src/carvision/
│   ├── features.py         # FeatureEngineer centralizado
│   ├── data.py             # clean_data parameterizado
│   └── pipeline.py
├── app/
│   └── streamlit_app.py    # Dashboard principal
├── tests/                  # 97% coverage
└── configs/config.yaml
```

- **Problema**: Regresión (predicción de precios de autos)
- **Técnicas**: Custom FeatureEngineer, RandomForest
- **Módulos clave**: 08, 15, 11

3. TelecomAI-Customer-Intelligence

```
📁 TelecomAI-Customer-Intelligence/
├── src/telecomai/
│   ├── data.py
│   └── training.py
├── app/
│   ├── fastapi_app.py
│   └── example_load.py
├── tests/                  # 97% coverage
└── docs/
```

- **Problema**: Clasificación multiclase (segmentación de clientes)
- **Técnicas**: LogisticRegression, GradientBoosting
- **Módulos clave**: 09, 10, 12

⚡ Quick Start

```
# 1. Clonar el portafolio
git clone https://github.com/DuqueOM/ML-MLOps-Portfolio.git
cd ML-MLOps-Portfolio

# 2. Empezar con BankChurn (proyecto base)
cd BankChurn-Predictor
pip install -e ".[dev]"

# 3. Ejecutar el flujo completo
make train          # Entrena el modelo
make test           # Ejecuta tests (79%+ coverage)
make serve          # Inicia API en localhost:8000

# 4. Verificar que todo funciona
curl http://localhost:8000/health
```

📈 Tiempo Estimado

Fase	Módulos	Horas	Semanas
Fundamentos	01-06	23h	2
ML Engineering	07-10	18h	2
MLOps Core	11-16	26h	2
Producción	17-18	7h	1
Especialización	19-23	12h	1
TOTAL	23 módulos	**~86h**	8 **semanas**

Dedicación sugerida: 10-12 horas/semana

✅ Convenciones de la Guía

Símbolo	Significado
💡	Tip o consejo práctico
⚠️	Advertencia importante
❌	Anti-patrón o error común
✅	Buena práctica recomendada
🔧	Ejercicio práctico
📝	Nota o aclaración
🎯	Objetivo de aprendizaje
📦	Cómo se usó en el portafolio

🚀 ¡Empieza Ahora!

Módulo 1 → Python Moderno para MLOps

Tiempo estimado para completar la guía: 8 semanas a ritmo moderado

Última actualización: Diciembre 2024

SYLLABUS

GUÍA MLOPS V5.0

PORTFOLIO EDITION

📅 SYLLABUS — Guía MLOps (Portfolio Edition)

Programa completo de 8 semanas para construir el portafolio ML-MLOps-Portfolio desde cero

📌 **Navegación**: Este documento complementa el Índice Principal (00_INDICE.md) con detalles de macro-módulos y progresión 0 → Senior/Staff. Para la estructura módulo por módulo, consulta el índice.

🎯 Objetivo del Programa

Al completar este programa serás capaz de:

- ✅ **Reproducir** 100% de los artefactos clave del portafolio (modelos, APIs, dashboards)
- ✅ Implementar **CI/CD profesional** con 80%+ coverage
- ✅ Diseñar **arquitecturas ML production-ready**
- ✅ **Pasar entrevistas técnicas** nivel Senior/Staff
- ✅ Crear **Model Cards y Dataset Cards** completos
- ✅ Implementar **observabilidad y monitoreo** básico

📊 Estructura del Programa (12 Módulos)

Módulo	Nombre	Duración	Mini-Proyecto
00	Introducción	0.5 días	Setup inicial
01	Python Moderno	2 días	Librería `utils/`
02	Ingeniería de Datos	4 días	ETL reproducible
03	Feature Engineering	3 días	Transformadores `.pkl`
04	Modelado	6 días	Scripts de entrenamiento
05	MLflow & DVC	3 días	Tracking local
06	Despliegue API	3 días	FastAPI `/predict`
07	Dashboard	2 días	Streamlit app
08	CI/CD & Testing	3 días	GitHub Actions
09	Model & Dataset Cards	1.5 días	Documentación ML
10	Observabilidad	2 días	Logging + alertas
11	Mantenimiento & Auditoría	2 días	Runbooks

Tiempo total estimado: 32 días (~ 6 - 8 semanas a ritmo moderado)

🧭 Ruta 0 → Senior/Staff (macro-módulos)

Esta ruta agrupa los 23 módulos de la guía en 11 **macro-módulos** que siguen el plan "0 → Senior/Staff MLOps" que definiste. No reemplaza la numeración actual (01–23), sino que ofrece una vista de alto nivel basada en madurez.

Macro-Módulo	Nombre	Objetivo principal	Módulos relacionados
00	Entorno, Herramientas y Flujo de Trabajo	Poder ejecutar los 3 proyectos del portafolio en tu máquina	00_INDICE, 03_ESTRUCTURA_PROYECTO, 04_ENTORNOS, 05_GIT_PROFESIONAL, 17_DESPLIEGUE, PLAN_ESTUDIOS, QUICK_START del repo
01			01_PYTHON_MODERNO, 03_ESTRUCTURA_PROYECTO, common_utils/ ★

Macro-Módulo	Nombre	Objetivo principal	Módulos relacionados
	Python Fundamentos para Producción	Pasar de Python junior a escribir código pythonico y mantenible	
02	Fundamentos de Data Science y ML	Tener bases sólidas de DS/ML antes de entrar a MLOps	07_SKLEARN_PIPELINES, 08_INGENIERIA_FEATURES, 09_TRAINING_PROFESIONAL, notebooks de los proyectos
03	Ingeniería de Datos Aplicada a ML	Preparar datos como en una empresa, pensando en ML downstream	06_VERSIONADO_DATOS, 08_INGENIERIA_FEATURES, partes de TelecomAI-Customer-Intelligence
04	Fundamentos de MLOps	Entender reproducibilidad, versionado y artefactos	06_VERSIONADO_DATOS, 10_EXPERIMENT_TRACKING, DECISIONES_TECH, 22_CHECKLIST
05	Pipelines + CI/CD	Construir pipelines reales con tests y gates de calidad	07_SKLEARN_PIPELINES, 11_TESTING_ML, 12_CI_CD, workflows de .github/workflows/
06	Model Deployment	Desplegar modelos con nivel Senior (APIs, Docker, serverless)	13_DOCKER, 14_FASTAPI, 17_DESPLIEGUE, docker-compose.demo.yml
07	Monitoring, Observabilidad y Alertas	Diferenciarte como Senior mediante observabilidad real	16_OBSERVABILIDAD, 22_CHECKLIST, dashboards de Grafana
08	Infraestructura y Nube	Operar como engineer: IaC, redes y cloud basics	17_DESPLIEGUE, 18_INFRAESTRUCTURA, infra/terraform/, k8s/
09	Escalado y Sistemas Distribuidos	Pensar en batch/streaming y K8s para ML a gran escala	18_INFRAESTRUCTURA, partes avanzadas de 17_DESPLIEGUE, tests/load/*
10	Seguridad, Gobernanza y Cumplimiento	Tratar el portafolio como un sistema empresarial	19_DOCUMENTACION, 12_CI_CD (gitleaks, security), .gitleaks.toml, RUNBOOK.md
11	Arquitectura MLOps Senior/Staff	Ver el sistema completo: multi-model, observabilidad y gobierno	20_PROYECTO_INTEGRADOR, 21_GLOSARIO, 23_RECURSOS, DECISIONES_TECH, RUNBOOK.md

Guion resumido por macro-módulo

MÓDULO 00 — Entorno, Herramientas y Flujo de Trabajo
Objetivo: garantizar que puedas ejecutar los 3 proyectos (BankChurn, CarVision, TelecomAI).
Incluye: Conda/pipx/uv, Docker + Docker Compose, Git + branching, VS Code + DevContainers, Makefiles, estructura estándar ML/MLOps.
Práctica en este repo: seguir `00_INDICE.md`, `PLAN_ESTUDIOS.md` y el QUICK_START de la raíz hasta ejecutar BankChurn end-to-end.

MÓDULO 01 — Python Fundamentos para Producción
Objetivo: llevar de Python junior a código pythonico y mantenible.
Incluye: POO aplicada a ML, tipado estático (mypy), logging profesional, manejo de errores, estructura de paquetes.
Práctica en este repo: trabajar `01_PYTHON_MODERNO.md` y refactorizar utilidades en `common_utils/` y el código de BankChurn.

MÓDULO 02 — Fundamentos de Data Science y ML
Objetivo: construir bases sólidas de DS/ML antes de MLOps.
Incluye: exploración, limpieza, feature engineering, validación cruzada, overfitting/underfitting.
Práctica en este repo: rehacer el pipeline de features y validación de BankChurn apoyándote en `07_SKLEARN_PIPELINES.md`, `08_INGENIERIA_FEATURES.md` y `09_TRAINING_PROFESIONAL.md`.

MÓDULO 03 — Ingeniería de Datos Aplicada a ML
Objetivo: preparar datos como en una empresa, pensando en su uso en modelos.
Incluye: ETL/ELT, orquestación ligera, data quality, feature stores.
Práctica en este repo: usar `06_VERSIONADO_DATOS.md` y `08_INGENIERIA_FEATURES.md` para montar un mini feature store inspirado en TelecomAI.

MÓDULO 04 — Fundamentos de MLOps
Objetivo: introducir el mindset MLOps (reproducibilidad, versionado, artefactos).
Incluye: versionado de datos y modelos, ML metadata, experiment tracking, artefactos.
Práctica en este repo: integrar MLflow y DVC a BankChurn siguiendo `06_VERSIONADO_DATOS.md`, `10_EXPERIMENT_TRACKING.md` y `DECISIONES_TECH.md`.

MÓDULO 05 — Pipelines + CI/CD
Objetivo: crear pipelines reales con CI/CD enterprise-like.
Incluye: GitHub Actions, testing, coverage, code-quality gates.
Práctica en este repo: combinar `07_SKLEARN_PIPELINES.md`, `11_TESTING_ML.md` y `12_CI_CD.md` para obtener un pipeline completo para los 3 proyectos usando los workflows reales del repositorio.

MÓDULO 06 — Model Deployment
Objetivo: desplegar con nivel Senior.
Incluye: APIs con FastAPI, dockerización, serverless, patrones de model serving.
Práctica en este repo: usar `13_DOCKER.md`, `14_FASTAPI.md` y `17_DESPLIEGUE.md` para desplegar CarVision en contenedor + endpoint (local y/o cloud).

MÓDULO 07 — Monitoring, Observabilidad y Alertas
Objetivo: incorporar observabilidad que diferencie un junior de un senior.
Incluye: concept vs data drift, monitoreo de features/predicciones, logging estructurado, Prometheus + Grafana.
Práctica en este repo: seguir `16_OBSERVABILIDAD.md` para instrumentar BankChurn con métricas y paneles, apoyándote en los manifiestos de `k8s/` y las reglas de `infra/`.

MÓDULO 08 — Infraestructura y Nube
Objetivo: operar como un engineer en cloud.
Incluye: IaC (Terraform), fundamentos AWS/GCP, redes y seguridad básica.
Práctica en este repo: partir de `17_DESPLIEGUE.md` y `18_INFRAESTRUCTURA.md` para desplegar un stack MLOps básico en cloud (o simularlo localmente con los manifests y Terraform).

MÓDULO 09 — Escalado y Sistemas Distribuidos
Objetivo: pensar en batch/streaming y K8s para producción masiva.
Incluye: batch vs streaming, Kubernetes, automatización avanzada.
Práctica en este repo: usar las secciones avanzadas de `18_INFRAESTRUCTURA.md`, los manifests en `k8s/` y los tests de carga en `tests/load/` como base para diseñar un despliegue escalable de CarVision.

MÓDULO 10 — Seguridad, Gobernanza y Cumplimiento
Objetivo: llevar la senioridad al plano empresarial.
Incluye: políticas, roles, seguridad de repos (secrets, escaneo), Model Cards y ética.
Práctica en este repo: combinar `19_DOCUMENTACION.md`, la configuración de `12_CI_CD.md` (gitleaks, security scanning) y `.gitleaks.toml` para definir políticas mínimas y completar Model Cards para los 3 proyectos.

MÓDULO 11 — Arquitectura MLOps Senior/Staff
Objetivo: tener visión completa de sistemas reales (multi-model, gobierno, observabilidad a gran escala).
Incluye: arquitecturas event-driven, multi-model governance, patrones de observabilidad.
Práctica en este repo: usar `20_PROYECTO_INTEGRADOR.md`, `DECISIONES_TECH.md` y `RUNBOOK.md` para diseñar y documentar una arquitectura MLOps completa que integre los 3 proyectos.

Puedes usar:

- Esta sección para entender el **mapa mental 0 → Senior/Staff**.
- El índice de 23 módulos (`00_INDICE.md`) y el plan por semanas para avanzar **paso a paso**.

📚 Detalle por Módulo

Módulo 00 — Introducción (0.5 días)

Contenido	Entregable
Objetivos del curso	✅ Entender el roadmap
Cómo leer la guía	✅ Setup de herramientas

Contenido	Entregable
Requerimientos mínimos	✅ Entorno listo
Mapa guía → repo	✅ Comprensión de estructura

Output: Entorno de desarrollo listo, comprensión clara del objetivo final.

Módulo 01 — Python Moderno (2 días)

Contenido	Entregable
Type hints y tipado estático	Código tipado
Dataclasses y Pydantic	Config validado
OOP y SOLID básico	Clases bien diseñadas
Estructura de paquete	`utils/` funcional

Mini-Proyecto: Crear librería `utils/` con `config.py` (Pydantic) y `mathops.py` (funciones tipadas).

Validar: `make check-01`

Módulo 02 — Ingeniería de Datos (4 días)

Contenido	Entregable
Lectura/escritura de datos	Loaders reutilizables
Validación con schemas	Contratos de datos
Transformaciones básicas	ETL reproducible
Tests de integridad	Datos validados

Mini-Proyecto: ETL que produce CSV/Parquet reproducible + tests de integridad.

Validar: `make check-02`

Módulo 03 — Feature Engineering (3 días)

Contenido	Entregable
Pipelines serializables	Pipeline persistido
Custom encoders	Transformadores `.pkl`
Prevención de data leakage	Código seguro
Persistencia de artefactos	Artefactos reutilizables

Mini-Proyecto: `FeatureEngineer` class con transformadores serializados.

Validar: `make check-03`

Módulo 04 — Modelado (6 días)

Contenido	Entregable
Pipelines sklearn completos	Pipeline unificado
Validación temporal/cruzada	CV implementado
Hyperparameter tuning	Búsqueda de hiperparámetros
Experimentación reproducible	Scripts de entrenamiento

Mini-Proyecto: Scripts que generan modelos y reportes en `outputs/`.

Validar: `make check-04`

Módulo 05 — MLflow & DVC (3 días)

Contenido	Entregable
MLflow server local	`mlflow ui` funcionando
Tracking de experimentos	Métricas registradas
DVC init y pipelines	`dvc.yaml` configurado
Versionado de artefactos	Datos versionados

Mini-Proyecto: `mlruns/` y `dvc/` que emulan el flujo del repo.

Validar: `make check-05`

Módulo 06 — Despliegue API (3 días)

Contenido	Entregable
FastAPI básico	API funcional
Schemas Pydantic	Request/Response tipados
Tests de integración	Tests pasando
Dockerfile	Contenedor listo

Mini-Proyecto: API local con endpoint `/predict` funcional.

Validar: `make check-06`

Módulo 07 — Dashboard (2 días)

Contenido	Entregable
Streamlit básico	App funcionando
Consumo de API	Integración con backend
Caching y optimización	Performance aceptable
Ejemplo desplegable	Ready to deploy

Mini-Proyecto: Dashboard Streamlit que consume la API local.

Validar: `make check-07`

Módulo 08 — CI/CD & Testing (3 días)

Contenido	Entregable
GitHub Actions	Workflow configurado
Matrix testing	Tests multi-versión
Coverage reports	80%+ coverage
Security scanning	gitleaks local

Mini-Proyecto: `ci_template.yml` funcional, simulación local con `act`.

Validar: `make check-08`

Módulo 09 — Model & Dataset Cards (1.5 días)

Contenido	Entregable
Plantilla Model Card	Template relleno
Plantilla Dataset Card	Template relleno
Buenas prácticas de documentación	Docs completos
Ejemplos del portafolio	Cards reales

Mini-Proyecto: Model Card y Dataset Card completados para un mini-proyecto.

Validar: `make check-09`

Módulo 10 — Observabilidad & Monitoring (2 días)

Contenido	Entregable
Logging estructurado	Logs configurados
Métricas básicas	Latencia, error rate
Simulación de alertas	Scripts de alerta
Drift detection básico	Checks implementados

Mini-Proyecto: Sistema con logging estructurado y scripts de alerta.

Validar: `make check-10`

Módulo 11 — Mantenimiento & Auditoría (2 días)

Contenido	Entregable
Playbooks de mantenimiento	Runbooks documentados
Tests de regresión	Regression tests
Actualización de dependencias	Proceso documentado
Calendario de revisiones	Plan de mantenimiento

Mini-Proyecto: MAINTENANCE_GUIDE.md y scripts de validación.

Validar: `make check-11`

📊 Rúbrica de Evaluación (100 puntos por módulo)

Criterio	Puntos	Descripción
Funcionalidad	40	Pasa tests mínimos, produce outputs esperados
Calidad del código	20	Linters, type hints, modularidad
Documentación	15	README, Model/Dataset Cards
Reproducibilidad	15	Instrucciones make, lockfile, ejecución local
Tests y cobertura	10	Pruebas unitarias/integración mínimas

Nota mínima aprobatoria: 70/100 por módulo

📈 Progreso Sugerido

```
Semana 1:   Módulos 00-01 (Fundamentos Python)
Semana 2:   Módulos 02-03 (Datos y Features)
```

```
Semana 3:   Módulo 04 (Modelado completo)
Semana 4:   Módulos 05-06 (Tracking + API)
Semana 5:   Módulos 07-08 (Dashboard + CI/CD)
Semana 6:   Módulos 09-11 (Docs + Mantenimiento)
```

🎤 Preparación para Entrevistas

La guía incluye simulacros de entrevista adaptados a cada nivel de experiencia:

Nivel	Simulacro	Preguntas	Cuándo Usar
🟢 Junior	SIMULACRO_ENTREVISTA_JUNIOR.md	50	Semanas 1-4
🟡 Mid	SIMULACRO_ENTREVISTA_MID.md	60	Semanas 5-6
🔴 Senior	SIMULACRO_ENTREVISTA_SENIOR_PARTE1.md + PARTE2	115	Semanas 7-8

Progresión recomendada:
1. **Junior**: Python básico, ML fundamentos, Git, estructura de proyecto
2. **Mid**: Pipelines, testing, CI/CD, Docker, APIs
3. **Senior**: System design, arquitectura, liderazgo, trade-offs

Material complementario:
- Speech de Portafolio — Guión de 5-7 min
- Talking Points — Puntos clave concisos

✅ Prerrequisitos

- **Python** 3.10+ instalado
- **Git** básico (clone, commit, push)
- **Línea de comandos** básica (bash/zsh)
- **Cuenta GitHub** activa
- **Editor/IDE** (VS Code recomendado)
- 8 **GB RAM** mínimo, 16 GB recomendado

🛠️ Cómo usar la guía

1. **Clonar** el repositorio guía
2. **Ejecutar** `make setup` para preparar el entorno
3. **Seguir** cada módulo en orden
4. **Completar** el mini-proyecto de cada módulo
5. **Validar** con `make check-XX` correspondiente
6. **Revisar** soluciones en `solutions/` si necesitas ayuda

📦 Entregables Finales

Al completar la guía tendrás:

- [ ] Portafolio ML reproducido localmente
- [ ] 3 proyectos con CI/CD funcionando
- [ ] Model Cards y Dataset Cards completos
- [ ] APIs y dashboards desplegables
- [ ] Sistema de observabilidad básico
- [ ] Runbooks de mantenimiento

¡Empieza ahora! → 00_INDICE.md

PLAN DE ESTUDIOS

GUÍA MLOPS V5.0

PORTFOLIO EDITION

📅 Plan de Estudios — 8 Semanas

Roadmap detallado para completar el portafolio MLOps

📊 Vista General

```
┌──────────────────────────────────────────────────────────────────┐
│                    PLAN DE 8 SEMANAS                             │
├──────────────────────────────────────────────────────────────────┤
│                                                                  │
│  Semana 1-2:  FUNDAMENTOS                                        │
│               Python moderno, estructura, Git, entornos          │
│                                                                  │
│  Semana 3-4:  ML ENGINEERING                                     │
│               Pipelines sklearn, feature engineering, MLflow     │
│                                                                  │
│  Semana 5-6:  MLOps CORE                                         │
│               Testing, CI/CD, Docker, APIs                       │
│                                                                  │
│  Semana 7:    PRODUCCIÓN                                         │
│               Deploy, observabilidad, infraestructura            │
│                                                                  │
│  Semana 8:    PROYECTO FINAL                                     │
│               Documentación, demo, preparación entrevistas       │
│                                                                  │
└──────────────────────────────────────────────────────────────────┘
```

Consejo didáctico: En cada módulo, antes de marcar el checkpoint como completado, revisa también la sección de **"Errores habituales y cómo depurarlos"** para consolidar patrones de debugging.

Ruta 0 → Senior/Staff: Usa la sección **"Ruta 0 → Senior/Staff (macro-módulos)"** del SYLLABUS como mapa de alto nivel, y este plan de 8 semanas como cronograma concreto.

📚 Semana 1: Python Moderno + Estructura

Objetivos

- [ ] Dominar type hints y Pydantic
- [ ] Crear estructura src/ layout
- [ ] Configurar pyproject.toml

Actividades Diarias

Día	Actividad	Tiempo	Entregable
1	Leer 01_PYTHON_MODERNO	2 h	Notas
2	Ejercicios type hints	3 h	Código tipado
3	Leer 02_DISENO_SISTEMAS	2 h	ML Canvas
4	Leer 03_ESTRUCTURA_PROYECTO	2 h	Estructura base
5	Crear proyecto BankChurn base	3 h	Repo inicial

Recursos

- Pydantic Docs
- Python Type Hints Cheat Sheet

📚 Semana 2: Git + Entornos + DVC

Objetivos

- [ ] Configurar pre-commit hooks
- [ ] Dominar Conventional Commits
- [ ] Inicializar DVC

Actividades Diarias

Día	Actividad	Tiempo	Entregable
1	Leer 04_ENTORNOS	2 h	requirements.txt
2	Leer 05_GIT_PROFESIONAL	2 h	pre-commit.yaml
3	Configurar pre-commit en proyecto	2 h	Hooks funcionando
4	Leer 06_VERSIONADO_DATOS	2 h	DVC init
5	Versionar datos de BankChurn	3 h	.dvc files

📚 Semana 3: Pipelines sklearn

Objetivos

- [ ] Crear Pipeline unificado
- [ ] Implementar ColumnTransformer
- [ ] Crear Custom Transformer

Actividades Diarias

Día	Actividad	Tiempo	Entregable
1 - 2	Leer 07_SKLEARN_PIPELINES	4 h	Pipeline básico
3	Implementar ColumnTransformer	3 h	Preprocessing
4 - 5	Leer 08_INGENIERIA_FEATURES	4 h	FeatureEngineer class

Proyecto

Crear `src/bankchurn/training.py` con pipeline completo.

📚 Semana 4: Training + MLflow

Objetivos

- [ ] Crear clase Trainer profesional
- [ ] Implementar cross-validation
- [ ] Integrar MLflow tracking

Actividades Diarias

Día	Actividad	Tiempo	Entregable
1 - 2	Leer 09_TRAINING_PROFESIONAL	4 h	ChurnTrainer class
3 - 5	Leer 10_EXPERIMENT_TRACKING	6 h	MLflow integrado

Proyecto

Entrenar modelo con métricas en MLflow UI.

📚 Semana 5: Testing

Objetivos

- [ ] Escribir tests unitarios
- [ ] Alcanzar 80% coverage
- [ ] Crear fixtures reutilizables

Actividades Diarias

Día	Actividad	Tiempo	Entregable
1 - 2	Leer 11_TESTING_ML	4 h	conftest.py

Día	Actividad	Tiempo	Entregable
3 - 5	Escribir tests para BankChurn	6 h	80% coverage

📚 Semana 6: CI/CD + Docker + APIs

Objetivos

- [ ] Crear GitHub Actions workflow
- [ ] Dockerfile multi-stage
- [ ] API FastAPI funcional

Actividades Diarias

Día	Actividad	Tiempo	Entregable
1	Leer 12_CI_CD	2 h	ci.yml
2	Leer 13_DOCKER	2 h	Dockerfile
3 - 4	Leer 14_FASTAPI	4 h	/predict endpoint
5	Leer 15_STREAMLIT	2 h	Dashboard básico

📚 Semana 7: Producción

Objetivos

- [ ] Implementar logging estructurado
- [ ] Entender estrategias de deploy
- [ ] Conocer Terraform/K8s básico

Actividades Diarias

Día	Actividad	Tiempo	Entregable
1 - 2	Leer 16_OBSERVABILIDAD	4 h	Logging JSON
3 - 4	Leer 17_DESPLIEGUE	4 h	Estrategia elegida
5	Leer 18_INFRAESTRUCTURA	2 h	Conceptos IaC

📚 Semana 8: Proyecto Final

Objetivos

- [ ] Completar documentación
- [ ] Pasar rúbrica de evaluación
- [ ] Preparar para entrevistas

Actividades Diarias

Día	Actividad	Tiempo	Entregable
1	Leer 19_DOCUMENTACION	2 h	Model Card
2 - 3	20_PROYECTO_INTEGRADOR	6 h	Self-assessment
4	Revisar 21_GLOSARIO	2 h	Términos dominados
5	Simulacros de entrevista	3 h	Preparación lista

⏱️ Tiempo Total Estimado

Componente	Horas
Lectura de módulos	40h
Ejercicios prácticos	30h
Proyecto BankChurn	20h
Proyectos adicionales	20h
TOTAL	**110h**

Dedicación sugerida: 15-20 horas/semana

✅ Checklist de Finalización

- [ ] 3 proyectos funcionando (BankChurn, CarVision, TelecomAI)
- [ ] CI/CD pasando en los 3
- [ ] Coverage ≥ 80% en todos
- [ ] APIs dockerizadas
- [ ] READMEs profesionales
- [ ] Model Cards completos

PYTHON MODERNO

GUÍA MLOPS V5.0

PORTFOLIO EDITION

01. Python Moderno para MLOps

🎯 Objetivo del Módulo

Transformar tu código de "funciona en un notebook" a "pasa code review en una empresa FAANG".

En este portafolio aplicarás estos patrones sobre `common_utils/` y el código de los tres proyectos (BankChurn-Predictor, CarVision-Market-Intelligence, TelecomAI-Customer-Intelligence), para que tu Python sea consistente en todo el stack.

```
╔═══════════════════════════════════════════════════════════════════╗
║                                                                   ║
║   ANTES (Data Scientist típico)          DESPUÉS (MLOps Engineer)  ║
║   ─────────────────────────────          ────────────────────────  ║
║   • Un archivo gigante                   • Paquete instalable      ║
║   • Sin tipos                            • Type hints en todo      ║
║   • Config hardcodeada                   • Pydantic validation     ║
║   • "Funciona en mi máquina"             • Funciona en cualquier máquina ║
║                                                                   ║
╚═══════════════════════════════════════════════════════════════════╝
```

📋 Contenido

1. Type Hints: Tu Contrato con el Futuro
2. Pydantic: Validación Automática
3. src/ Layout: Estructura Profesional
4. Principios SOLID para ML
5. Ejercicios Prácticos

1.1 Type Hints: Tu Contrato con el Futuro

La Analogía del Restaurante

```
╔═══════════════════════════════════════════════════════════════════╗
║  🍽️ IMAGINA UN RESTAURANTE:                                       ║
║                                                                   ║
║  SIN MENÚ (código sin tipos):                                     ║
║  - "Tráeme algo de comer"                                         ║
║  - El chef improvisa                                              ║
║  - El cliente no sabe qué esperar                                 ║
║  - Resultado: sorpresas (bugs)                                    ║
║                                                                   ║
║  CON MENÚ (código con tipos):                                     ║
║  - "Quiero el plato #5: Pasta Carbonara"                           ║
║  - El chef sabe exactamente qué preparar                           ║
║  - El cliente sabe qué recibirá                                    ║
║  - Resultado: consistencia                                         ║
║                                                                   ║
║  TYPE HINTS = El menú de tu código                                 ║
╚═══════════════════════════════════════════════════════════════════╝
```

Código Real del Portafolio: Sin Tipos vs Con Tipos

```
# ❌ ANTES: ¿Qué recibe? ¿Qué retorna?
# (Esto es lo que encontrarías en un notebook)

def prepare_features(df, num_cols, cat_cols, target):
    X = df.drop(columns=[target])
    y = df[target]

    preprocessor = ColumnTransformer([
        ('num', StandardScaler(), num_cols),
        ('cat', OneHotEncoder(), cat_cols)
    ])

    X_transformed = preprocessor.fit_transform(X)
    return X_transformed, y, preprocessor
```

```
# ✅ DESPUÉS: Código real de BankChurn-Predictor/src/bankchurn/training.py

from __future__ import annotations  # Permite usar tipos modernos en Python 3.9+
```

```
from pathlib import Path
from typing import List, Tuple

import numpy as np
import pandas as pd
from numpy.typing import NDArray
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, StandardScaler

def prepare_features(
    df: pd.DataFrame,
    num_cols: List[str],
    cat_cols: List[str],
    target: str
) -> Tuple[NDArray[np.float64], pd.Series, ColumnTransformer]:
    """Prepara features para entrenamiento.

    Parameters
    ----------
    df : pd.DataFrame
        DataFrame con datos crudos.
    num_cols : List[str]
        Nombres de columnas numéricas.
    cat_cols : List[str]
        Nombres de columnas categóricas.
    target : str
        Nombre de la columna objetivo.

    Returns
    -------
    Tuple[NDArray, pd.Series, ColumnTransformer]
        Features transformadas, target, y preprocessor fitted.
    """
    X = df.drop(columns=[target])
    y = df[target]

    preprocessor = ColumnTransformer([
        ('num', StandardScaler(), num_cols),
        ('cat', OneHotEncoder(handle_unknown='ignore'), cat_cols)
    ])

    X_transformed = preprocessor.fit_transform(X)
    return X_transformed, y, preprocessor
```

Los Tipos Esenciales para ML

```
# ═══════════════════════════════════════════════════════════════════════
# TIPOS BÁSICOS - Los usarás constantemente
# ═══════════════════════════════════════════════════════════════════════

from typing import (
    List,       # Lista de elementos: List[str] = ["a", "b"]
    Dict,       # Diccionario: Dict[str, float] = {"acc": 0.95}
    Tuple,      # Tupla fija: Tuple[int, int] = (100, 10)
    Optional,   # Puede ser None: Optional[Path] = None
    Union,      # Múltiples tipos: Union[str, List[str]]
    Any,        # Cualquier tipo (evitar si posible)
    Literal,    # Valores específicos: Literal["train", "eval"]
)
from pathlib import Path

# Ejemplos del portafolio real:

# BankChurn: features son listas de strings
features: List[str] = ["CreditScore", "Age", "Balance"]

# CarVision: métricas son diccionario string->float
metrics: Dict[str, float] = {"rmse": 4794.27, "r2": 0.77}

# TelecomAI: puede recibir path o None
model_path: Optional[Path] = None

# ═══════════════════════════════════════════════════════════════════════
```

```python
# TIPOS PARA ML - Específicos de Machine Learning
# ═══════════════════════════════════════════════════════════════════════════

import pandas as pd
import numpy as np
from numpy.typing import NDArray
from sklearn.base import BaseEstimator
from sklearn.pipeline import Pipeline

# DataFrame de pandas
def load_data(path: Path) -> pd.DataFrame:
    return pd.read_csv(path)

# Array NumPy tipado
def predict_proba(X: NDArray[np.float64]) -> NDArray[np.float64]:
    return model.predict_proba(X)[:, 1]

# Modelo sklearn
def train_model(X: NDArray, y: NDArray) -> BaseEstimator:
    model = RandomForestClassifier()
    model.fit(X, y)
    return model

# ═══════════════════════════════════════════════════════════════════════════
# TIPOS AVANZADOS - Para código más robusto
# ═══════════════════════════════════════════════════════════════════════════

from typing import TypedDict, Literal

# TypedDict: diccionarios con estructura conocida
class MetricsDict(TypedDict):
    accuracy: float
    precision: float
    recall: float
    f1: float
    roc_auc: float

# Literal: solo valores específicos permitidos
ModelType = Literal["random_forest", "logistic", "gradient_boosting"]

def build_model(model_type: ModelType, seed: int) -> BaseEstimator:
    """
    mypy SABE que model_type solo puede ser estos 3 valores.
    Si escribes build_model("xgboost", 42), mypy dará error.
    """
    if model_type == "random_forest":
        return RandomForestClassifier(random_state=seed)
    elif model_type == "logistic":
        return LogisticRegression(random_state=seed)
    else:  # gradient_boosting
        return GradientBoostingClassifier(random_state=seed)
```

Configurar mypy

Añade esto a tu `pyproject.toml`:

```toml
# pyproject.toml - Configuración de mypy
[tool.mypy]
python_version = "3.11"
warn_return_any = true
warn_unused_ignores = true
disallow_untyped_defs = true      # ← Fuerza tipos en todas las funciones
ignore_missing_imports = true     # ← Para librerías sin stubs

# Ignorar librerías de ML que no tienen stubs completos
[[tool.mypy.overrides]]
module = [
    "sklearn.*",
    "pandas.*",
    "numpy.*",
    "mlflow.*",
    "joblib.*",
]
ignore_missing_imports = true
```

Ejecutar:

```
mypy src/  # Verifica tipos en todo el código
```

1.2 Pydantic: Validación Automática

La Analogía del Guardia de Seguridad

```
╔═══════════════════════════════════════════════════════════════╗
║  🛡️ IMAGINA UN EDIFICIO DE OFICINAS:                          ║
║                                                               ║
║  SIN GUARDIA (código sin Pydantic):                           ║
║  - Cualquiera entra con cualquier cosa                         ║
║  - Descubres problemas CUANDO YA PASARON                       ║
║  - "¿Por qué hay un test_size de 1.5?" → Error en producción   ║
║                                                               ║
║  CON GUARDIA (código con Pydantic):                           ║
║  - Verifica credenciales EN LA ENTRADA                         ║
║  - Problemas detectados ANTES de causar daño                   ║
║  - "test_size debe ser entre 0 y 1" → Error inmediato y claro  ║
║                                                               ║
║  PYDANTIC = El guardia de tu configuración                     ║
╚═══════════════════════════════════════════════════════════════╝
```

Código Real: BankChurn Config (Nivel Staff)

Este es el archivo `src/bankchurn/config.py` del portafolio:

```
"""Configuration management for BankChurn predictor.

Este módulo demuestra Pydantic a nivel profesional:
- Validación de rangos con Field
- Configuraciones anidadas
- Valores por defecto sensatos
- Carga desde YAML
"""

from __future__ import annotations

from pathlib import Path
from typing import Any, List

import yaml
from pydantic import BaseModel, Field


# ═══════════════════════════════════════════════════════════════════════
# CONFIGURACIONES ANIDADAS - Cada componente tiene su propia config
# ═══════════════════════════════════════════════════════════════════════

class LogisticRegressionConfig(BaseModel):
    """Hiperparámetros de Logistic Regression."""
    C: float = 0.1
    class_weight: str = "balanced"
    solver: str = "liblinear"
    max_iter: int = 1000


class RandomForestConfig(BaseModel):
    """Hiperparámetros de Random Forest."""
    n_estimators: int = 100
    max_depth: int = 10
    min_samples_split: int = 10
    min_samples_leaf: int = 5
    class_weight: str = "balanced_subsample"
    n_jobs: int = -1


class EnsembleConfig(BaseModel):
    """Configuración del ensemble."""
    voting: str = Field("soft", pattern="^(hard|soft)$")  # ← Solo permite "hard" o "soft"
    weights: List[float] = [0.4, 0.6]
```

```
# ═══════════════════════════════════════════════════════════════════════════
# CONFIGURACIÓN PRINCIPAL - Agrupa todo con validación
# ═══════════════════════════════════════════════════════════════════════════

class ModelConfig(BaseModel):
    """Configuración de entrenamiento del modelo."""
    type: str = "ensemble"
    test_size: float = Field(0.2, ge=0.0, le=1.0)   # ← VALIDACIÓN: entre 0 y 1
    random_state: int = 42
    cv_folds: int = Field(5, ge=2)                   # ← VALIDACIÓN: mínimo 2
    resampling_strategy: str = "none"

    # Configuraciones de modelos específicos (anidadas)
    ensemble: EnsembleConfig = EnsembleConfig()
    logistic_regression: LogisticRegressionConfig = LogisticRegressionConfig()
    random_forest: RandomForestConfig = RandomForestConfig()


class DataConfig(BaseModel):
    """Configuración de datos."""
    target_column: str = "Exited"
    categorical_features: List[str] = []
    numerical_features: List[str] = []
    drop_columns: List[str] = []


class MLflowConfig(BaseModel):
    """Configuración de MLflow tracking."""
    tracking_uri: str = "file:./mlruns"
    experiment_name: str = "bankchurn"
    enabled: bool = True


# ═══════════════════════════════════════════════════════════════════════════
# CONFIGURACIÓN RAÍZ - El punto de entrada
# ═══════════════════════════════════════════════════════════════════════════

class BankChurnConfig(BaseModel):
    """Configuración completa de BankChurn.

    Uso:
        config = BankChurnConfig.from_yaml("configs/config.yaml")
        print(config.model.test_size)  # 0.2
    """
    model: ModelConfig
    data: DataConfig
    mlflow: MLflowConfig

    @classmethod
    def from_yaml(cls, config_path: str | Path) -> BankChurnConfig:
        """Carga configuración desde archivo YAML.

        Parameters
        ----------
        config_path : str or Path
            Ruta al archivo YAML.

        Returns
        -------
        BankChurnConfig
            Configuración validada.

        Raises
        ------
        FileNotFoundError
            Si el archivo no existe.
        ValidationError
            Si la configuración es inválida.
        """
        config_path = Path(config_path)

        if not config_path.exists():
            raise FileNotFoundError(f"Config file not found: {config_path}")

        with open(config_path, "r") as f:
```

```
            config_dict = yaml.safe_load(f) or {}

        # Valores por defecto para secciones faltantes
        if "model" not in config_dict:
            config_dict["model"] = ModelConfig().dict()
        if "data" not in config_dict:
            config_dict["data"] = DataConfig().dict()
        if "mlflow" not in config_dict:
            config_dict["mlflow"] = MLflowConfig().dict()

        return cls(**config_dict)  # ← Pydantic valida automáticamente
```

El YAML Correspondiente

```
# configs/config.yaml
model:
  type: ensemble
  test_size: 0.2         # Si pones 1.5, Pydantic dará error
  random_state: 42
  cv_folds: 5            # Si pones 1, Pydantic dará error
  resampling_strategy: none

  ensemble:
    voting: soft         # Si pones "maybe", Pydantic dará error
    weights: [0.4, 0.6]

  random_forest:
    n_estimators: 200
    max_depth: 10

data:
  target_column: Exited
  categorical_features:
    - Geography
    - Gender
  numerical_features:
    - CreditScore
    - Age
    - Balance
  drop_columns:
    - RowNumber
    - CustomerId
    - Surname

mlflow:
  tracking_uri: "file:./mlruns"
  experiment_name: bankchurn
  enabled: true
```

Ejemplo de Error de Validación

```
# ❌ Esto FALLA inmediatamente con un error claro

config_dict = {
    "model": {
        "test_size": 1.5,  # ← Error: debe ser <= 1.0
        "cv_folds": 1,     # ← Error: debe ser >= 2
    },
    "data": {},
    "mlflow": {}
}

try:
    config = BankChurnConfig(**config_dict)
except ValidationError as e:
    print(e)
    # Output:
    # 2 validation errors for BankChurnConfig
    # model -> test_size
    #   ensure this value is less than or equal to 1.0 (type=value_error.number.not_le)
    # model -> cv_folds
    #   ensure this value is greater than or equal to 2 (type=value_error.number.not_ge)
```

1.3 src/ Layout: Estructura Profesional

La Analogía de la Casa

```
╔═══════════════════════════════════════════════════════════════╗
║  🏠 IMAGINA ORGANIZAR UNA CASA:                                ║
║                                                                ║
║  CASA DESORDENADA (código en raíz):                            ║
║  - Todo en el living: ropa, comida, herramientas               ║
║  - Imposible encontrar algo                                    ║
║  - Invitas a alguien: "perdón por el desorden"                 ║
║                                                                ║
║  CASA ORGANIZADA (src/ layout):                                ║
║  - Cocina para cocinar, baño para baño, closet para ropa       ║
║  - Cada cosa en su lugar                                       ║
║  - Invitas a alguien: "bienvenido, siéntate"                   ║
║                                                                ║
║  src/ layout = Organización profesional de código              ║
╚═══════════════════════════════════════════════════════════════╝
```

Estructura del Portafolio

```
BankChurn-Predictor/
├── src/                        # ← TODO el código fuente aquí
│   ├── __init__.py             # Hace src/ un paquete
│   └── bankchurn/              # ← El paquete principal
│       ├── __init__.py         # Exporta la API pública
│       ├── config.py           # Configuración Pydantic
│       ├── training.py         # Pipeline de entrenamiento
│       ├── evaluation.py       # Métricas y evaluación
│       ├── prediction.py       # Inferencia
│       ├── models.py           # Custom classifiers
│       └── cli.py              # Interfaz de línea de comandos
│
├── app/                        # ← Aplicaciones (no es un paquete)
│   └── fastapi_app.py          # API REST
│
├── tests/                      # ← Tests (espejo de src/)
│   ├── __init__.py
│   ├── conftest.py             # Fixtures compartidas
│   ├── test_config.py          # Tests para config.py
│   ├── test_training.py        # Tests para training.py
│   └── ...
│
├── configs/                    # ← Configuración externa
│   └── config.yaml
│
├── data/                       # ← Datos (gitignored)
│   └── raw/
│       └── Churn_Modelling.csv
│
├── artifacts/                  # ← Artefactos generados (gitignored)
│   ├── model.joblib
│   └── training_results.json
│
├── pyproject.toml              # ← Metadata del proyecto
├── Makefile                    # ← Comandos comunes
├── Dockerfile                  # ← Containerización
└── README.md                   # ← Documentación
```

¿Por qué src/ y no código en la raíz?

```
# ❌ PROBLEMA: Sin src/, Python puede importar código no instalado
# Esto causa el famoso "funciona en mi máquina pero no en CI"

# Estructura plana (problemática):
# myproject/
# ├── mymodule.py
# └── tests/
#     └── test_mymodule.py

# En test_mymodule.py:
import mymodule  # ← ¿De dónde viene? ¿Del directorio actual? ¿De pip?
```

```
# ✅ SOLUCIÓN: Con src/, el código DEBE estar instalado para importar
# myproject/
# ├── src/
# │   └── mymodule/
# │       └── __init__.py
# └── tests/
#     └── test_mymodule.py

# En test_mymodule.py:
from mymodule import something  # ← Solo funciona si `pip install -e .`
```

pyproject.toml: El Corazón del Proyecto

```
# pyproject.toml - Configuración completa del proyecto
[build-system]
requires = ["setuptools>=61.0"]
build-backend = "setuptools.build_meta"

[project]
name = "bankchurn"
version = "1.0.0"
description = "Bank Customer Churn Prediction System"
authors = [
    {name = "Daniel Duque", email = "duque@example.com"}
]
readme = "README.md"
requires-python = ">=3.10"
license = {text = "MIT"}

dependencies = [
    "pandas>=2.0.0",
    "scikit-learn>=1.3.0",
    "pydantic>=2.0.0",
    "pyyaml>=6.0",
    "mlflow>=2.9.0",
    "fastapi>=0.104.0",
    "uvicorn>=0.24.0",
]

[project.optional-dependencies]
dev = [
    "pytest>=7.4.0",
    "pytest-cov>=4.1.0",
    "black>=23.0.0",
    "mypy>=1.7.0",
    "ruff>=0.1.0",
]

[project.scripts]
bankchurn = "bankchurn.cli:main"  # ← Comando CLI

# ═══════════════════════════════════════════════════════════════════════
# HERRAMIENTAS
# ═══════════════════════════════════════════════════════════════════════

[tool.setuptools.packages.find]
where = ["src"]  # ← Busca paquetes en src/

[tool.pytest.ini_options]
testpaths = ["tests"]
python_files = ["test_*.py"]
addopts = "-v --cov=src/bankchurn --cov-report=term-missing"

[tool.coverage.run]
source = ["src"]
omit = ["tests/*"]

[tool.coverage.report]
fail_under = 79  # ← Coverage mínimo para pasar CI

[tool.black]
line-length = 100
target-version = ["py311"]
```

```
[tool.mypy]
python_version = "3.11"
warn_return_any = true
disallow_untyped_defs = true
ignore_missing_imports = true
```

Instalación en Modo Editable

```
# Instalar el paquete en modo editable (para desarrollo)
pip install -e .

# Ahora puedes importar desde cualquier lugar
python -c "from bankchurn.config import BankChurnConfig; print('✅ Funciona!')"

# Y los tests también funcionan
pytest tests/
```

1.4 Principios SOLID para ML

Single Responsibility: Un Módulo, Una Tarea

```
# ❌ ANTES: Un archivo hace TODO
# training.py (500 líneas)
def train_model(data_path, config_path, output_path):
    # Carga datos (líneas 1-50)
    # Limpia datos (líneas 51-100)
    # Feature engineering (líneas 101-200)
    # Entrena modelo (líneas 201-300)
    # Evalúa modelo (líneas 301-400)
    # Guarda artefactos (líneas 401-450)
    # Loguea a MLflow (líneas 451-500)
    pass

# ✅ DESPUÉS: Cada archivo tiene UNA responsabilidad
# src/bankchurn/
# ├── data.py         → Solo carga y valida datos
# ├── features.py     → Solo feature engineering
# ├── training.py     → Solo entrenamiento
# ├── evaluation.py   → Solo métricas
# └── prediction.py   → Solo inferencia
```

Código Real del Portafolio

```
# src/bankchurn/training.py - SOLO se encarga de entrenar
class ChurnTrainer:
    """Training pipeline - Single Responsibility."""

    def __init__(self, config: BankChurnConfig):
        self.config = config

    def load_data(self, path: Path) -> pd.DataFrame:
        """Delega a módulo de datos."""
        pass

    def prepare_features(self, df: pd.DataFrame) -> Tuple[pd.DataFrame, pd.Series]:
        """Prepara X e y."""
        pass

    def build_pipeline(self) -> Pipeline:
        """Construye el pipeline sklearn."""
        pass

    def fit(self, X: pd.DataFrame, y: pd.Series) -> None:
        """Entrena el modelo."""
        pass

    def cross_validate(self, X: pd.DataFrame, y: pd.Series) -> Dict[str, float]:
        """Valida con CV."""
        pass

# src/bankchurn/evaluation.py - SOLO se encarga de evaluar
def evaluate_model(
```

```
    model: Pipeline,
    X_test: pd.DataFrame,
    y_test: pd.Series
) -> Dict[str, float]:
    """Calcula métricas - Single Responsibility."""
    y_pred = model.predict(X_test)
    y_proba = model.predict_proba(X_test)[:, 1]

    return {
        "accuracy": accuracy_score(y_test, y_pred),
        "precision": precision_score(y_test, y_pred),
        "recall": recall_score(y_test, y_pred),
        "f1": f1_score(y_test, y_pred),
        "roc_auc": roc_auc_score(y_test, y_proba),
    }
```

1.5 Ejercicios Prácticos

Ejercicio 1: Añadir Type Hints

```
# ❌ Código sin tipos (típico de notebook)
# Tu tarea: Añade type hints completos

def process_training_data(df, config):
    target = config["target"]
    features = config["features"]

    X = df[features]
    y = df[target]

    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=config.get("test_size", 0.2)
    )

    return X_train, X_test, y_train, y_test


def calculate_metrics(y_true, y_pred):
    return {
        "accuracy": accuracy_score(y_true, y_pred),
        "f1": f1_score(y_true, y_pred)
    }
```

📝 Ver Solución

```
from typing import Dict, List, Tuple, Any
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, f1_score


def process_training_data(
    df: pd.DataFrame,
    config: Dict[str, Any]
) -> Tuple[pd.DataFrame, pd.DataFrame, pd.Series, pd.Series]:
    """Procesa datos para entrenamiento.

    Parameters
    ----------
    df : pd.DataFrame
        DataFrame con datos crudos.
    config : Dict[str, Any]
        Configuración con keys: "target", "features", "test_size" (opcional).

    Returns
    -------
    Tuple[pd.DataFrame, pd.DataFrame, pd.Series, pd.Series]
        X_train, X_test, y_train, y_test
    """
    target: str = config["target"]
    features: List[str] = config["features"]

    X: pd.DataFrame = df[features]
```

```
    y: pd.Series = df[target]

    test_size: float = config.get("test_size", 0.2)

    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=test_size, random_state=42
    )

    return X_train, X_test, y_train, y_test


def calculate_metrics(
    y_true: pd.Series,
    y_pred: pd.Series
) -> Dict[str, float]:
    """Calcula métricas de clasificación.

    Parameters
    ----------
    y_true : pd.Series
        Labels verdaderos.
    y_pred : pd.Series
        Predicciones del modelo.

    Returns
    -------
    Dict[str, float]
        Diccionario con accuracy y f1.
    """
    return {
        "accuracy": float(accuracy_score(y_true, y_pred)),
        "f1": float(f1_score(y_true, y_pred))
    }
```

Ejercicio 2: Crear Config con Pydantic

```
# Tu tarea: Crea una configuración Pydantic para TelecomAI
# Requisitos:
# - project_name: str
# - random_seed: int (entre 0 y 1000)
# - test_size: float (entre 0.1 y 0.5)
# - model_type: solo puede ser "logreg", "random_forest", o "gradient_boosting"
# - features: lista de strings
# - target: str

# Escribe tu código aquí:
from pydantic import BaseModel, Field
from typing import List, Literal

class TelecomConfig(BaseModel):
    # ... tu código
    pass
```

📝 Ver Solución

```
from pydantic import BaseModel, Field
from typing import List, Literal, Optional, Dict, Any
from pathlib import Path
import yaml


class TelecomConfig(BaseModel):
    """Configuración para TelecomAI Customer Intelligence."""

    project_name: str = Field(..., min_length=1)
    random_seed: int = Field(42, ge=0, le=1000)
    test_size: float = Field(0.2, ge=0.1, le=0.5)
    model_type: Literal["logreg", "random_forest", "gradient_boosting"] = "logreg"
    features: List[str] = Field(..., min_items=1)
    target: str

    # Opcionales
    threshold: float = Field(0.5, ge=0.0, le=1.0)
```

```
    mlflow_enabled: bool = True

    @classmethod
    def from_yaml(cls, path: str | Path) -> "TelecomConfig":
        with open(path) as f:
            data = yaml.safe_load(f)
        return cls(**data)

    class Config:
        extra = "forbid"  # No permite campos extra en el YAML


# Uso:
config = TelecomConfig(
    project_name="TelecomAI",
    features=["calls", "minutes", "messages", "mb_used"],
    target="is_ultra"
)

# Esto FALLA:
# config = TelecomConfig(
#     project_name="",  # Error: min_length=1
#     test_size=0.8,    # Error: le=0.5
#     model_type="xgboost",  # Error: not in Literal
#     features=[],      # Error: min_items=1
# )
```

Ejercicio 3: Convertir a src/ Layout

```
Tu tarea: Reorganiza esta estructura plana a src/ layout

ANTES:
myproject/
├── train.py
├── predict.py
├── utils.py
├── config.yaml
├── data.csv
└── test_train.py

DESPUÉS:
myproject/
├── src/
│   └── ???
├── tests/
│   └── ???
├── configs/
│   └── ???
├── data/
│   └── ???
└── pyproject.toml
```

📝 Ver Solución

```
myproject/
├── src/
│   ├── __init__.py
│   └── myproject/
│       ├── __init__.py
│       ├── training.py      # Antes: train.py
│       ├── prediction.py    # Antes: predict.py
│       └── utils.py
├── tests/
│   ├── __init__.py
│   ├── conftest.py          # Fixtures compartidas
│   └── test_training.py     # Antes: test_train.py
├── configs/
│   └── config.yaml
├── data/
│   └── raw/
│       └── data.csv
├── artifacts/               # Para modelos generados
│   └── .gitkeep
```

```
├── pyproject.toml
├── Makefile
└── README.md
```

🧨 Errores habituales y cómo depurarlos

En este módulo suelen aparecer siempre los mismos problemas. La idea no es solo evitarlos, sino **saber reconocerlos rápido** en tus propios proyectos.

1) Type hints + mypy: errores ruidosos en pandas/sklearn

Síntomas típicos

- `Function is missing a type annotation for parameter 'df'`
- `Incompatible return value type (got "DataFrame", expected "Series")`
- Cientos de warnings en librerías externas (`pandas.*`, `sklearn.*`).

Proceso para identificarlos

- Ejecuta siempre:
  `bash mypy src/ # o mypy src/bankchurn src/carvision src/telecomai`
- Localiza primero los errores **en tu código** (archivos dentro de `src/`), ignora de momento los de librerías.
- Si ves muchos errores en `site-packages` o módulos externos, revisa tu sección `[tool.mypy]` del `pyproject.toml` (ver ejemplo en este mismo módulo).

Cómo solucionarlos (patrón general)

- Añade tipos a **todas las firmas públicas** (funciones/clases usadas fuera de su archivo).
- Usa tipos específicos para ML:
- `pd.DataFrame`, `pd.Series`
- `NDArray[np.float64]`
- `BaseEstimator`, `Pipeline`
- Aísla tipos muy complejos usando `TypedDict` o `Alias`:
  `python class MetricsDict(TypedDict): accuracy: float f1: float roc_auc: float`
- Para **reducir ruido de mypy** con librerías ML:
- Configura `ignore_missing_imports = true` y los overrides mostrados en este módulo.
- Re-lanza `mypy` y verifica que solo quedan errores en tu código.

> 💡 **Regla práctica**: si mypy empieza a gritar en medio de un refactor, reduce el problema a una función pequeña, tipa bien esa función, y después propaga los tipos al resto.

2) Pydantic: `ValidationError` por config mal definida

Síntomas típicos

- Al cargar la configuración:
  `text pydantic.error_wrappers.ValidationError: 2 validation errors for ModelConfig model -> test_size ensure this value is less than or equal to 1.0 (type=value_error.number.not_le) model -> cv_folds ensure this value is greater than or equal to 2 (type=value_error.number.not_ge)`
- Tu servicio/API no arranca porque falla la lectura de `config.yaml`.

Proceso para identificarlos

- Localiza **qué modelo Pydantic** está fallando (`ModelConfig`, `BankChurnConfig`, `TelecomConfig`, etc.).
- Revisa el `traceback`: casi siempre indica **la ruta completa del campo** (`model -> test_size`, `data -> categorical_features`, etc.).
- Abre el YAML correspondiente (`configs/config.yaml`) y compara **valor real** vs **restricción en `Field(...)`**.

Cómo solucionarlos (patrón general)

- Ajusta el YAML para respetar los rangos:
- `test_size` entre `0.0` y `1.0`.
- `cv_folds` ≥ 2.
- Literales válidos (`voting: "hard" | "soft"`, `model_type: "logreg" | "random_forest" | ...`).
- Si el error te parece injustificado, revisa la declaración del modelo:
  `python test_size: float = Field(0.2, ge=0.0, le=1.0)`
  Quizá necesitas permitir un rango distinto en tu contexto.

- En desarrollo, **falla rápido**: no atrapes el `ValidationError` salvo para mostrar un mensaje más amigable; deja que la app se caiga antes que usar una config corrupta.

> 🔧 **Ejercicio mental**: rompe a propósito tu `configs/config.yaml` (pon `test_size: 1.5`) y observa el error. Luego arréglalo. Hazlo una vez y nunca más te asustará un `ValidationError` en producción.

3) src/ layout e imports: `ModuleNotFoundError` en CI pero no en tu máquina

Síntomas típicos

- En local "todo funciona", pero en GitHub Actions o en otra máquina obtienes:
  `text ModuleNotFoundError: No module named 'bankchurn'`
- Los tests solo pasan si ejecutas `pytest` desde la raíz exacta del proyecto.

Proceso para identificarlos

- Revisa la **estructura** de tu proyecto (debería parecerse al diagrama de este módulo):
- Código dentro de `src/<paquete>/`.
- Tests bajo `tests/` usando imports del paquete, no rutas relativas raras.
- Verifica tu `pyproject.toml`:
- `[project.name]` coincide con el paquete (`bankchurn`, `carvision`, `telecomai`).
- `[tool.setuptools.packages.find] where = ["src"]`.
- Comprueba si instalaste en modo editable:
  `bash pip install -e . python -c "import bankchurn; print(bankchurn.__file__)"`

Cómo solucionarlos (patrón general)

- Mueve el código de raíz a `src/` siguiendo el ejemplo de este módulo.
- Cambia imports tipo:
  `python # ❌ from .training import train_model (desde scripts sueltos) # ✅ from bankchurn.training import train_model`
- Asegúrate de que los comandos de CI usan instalación editable:
  ```yaml
- name: Install
  run: pip install -e ".[dev]"
  ```

> ⚠️ **Bandera roja**: si tus tests solo funcionan cuando haces `cd src` o ajustas manualmente `PYTHONPATH`, tu layout todavía no está bien resuelto.

4) Patrón general de debugging para este módulo

1. **Reproduce el error** con un comando simple y determinista:
2. `mypy src/`
3. `python -m src.proyecto.training`
4. `pytest -k nombre_test`.
5. **Lee literalmente** el mensaje de error (campo, valor, restricción).
6. **Conecta el error con el concepto del módulo**:
7. Type hints → firma de función o tipo de retorno.
8. Pydantic → `Field(...)` y YAML.
9. src/ layout → estructura de carpetas + `pyproject.toml` + instalación editable.
10. **Aplica el patrón de solución** que viste arriba.

Si automatizas este ciclo en tus tres proyectos del portafolio, tu tiempo de debugging se reduce drásticamente y es justo lo que se espera de un perfil Senior/Staff.

✅ Checkpoint: ¿Completaste el Módulo?

Antes de continuar, verifica:

- [ ] Tu código tiene type hints en todas las funciones
- [ ] Puedes ejecutar `mypy src/` sin errores críticos
- [ ] Tienes al menos una clase Pydantic para configuración
- [ ] Tu proyecto tiene estructura src/ layout
- [ ] Puedes instalar tu paquete con `pip install -e .`

🔗 ADR: ¿Por Qué Estas Decisiones?

ADR-001: Type Hints Obligatorios

Contexto: El código de ML suele ser difícil de mantener porque las funciones aceptan "cualquier cosa".

Decisión: Requerimos type hints en todas las funciones públicas.

Consecuencias:
- ✅ El IDE autocompleta correctamente
- ✅ Errores detectados antes de ejecutar
- ✅ Documentación implícita
- ❌ Más código que escribir inicialmente
- ❌ Curva de aprendizaje para tipos complejos

ADR-002: Pydantic para Configuración

Contexto: Configuraciones en diccionarios son propensas a errores.

Decisión: Toda configuración pasa por Pydantic.

Consecuencias:
- ✅ Validación automática
- ✅ Errores claros
- ✅ Documentación de la config
- ❌ Dependencia adicional
- ❌ Más verboso que un dict simple

ADR-003: src/ Layout

Contexto: Código en raíz causa problemas de importación.

Decisión: Todo código en `src/<paquete>/`.

Consecuencias:
- ✅ Importaciones consistentes
- ✅ Funciona igual en desarrollo y CI
- ✅ Estándar de la industria
- ❌ Requiere `pip install -e .`
- ❌ Path más largo para imports

📦 Cómo se Usó en el Portafolio

Este módulo se aplica **directamente** en los 3 proyectos del portafolio. Aquí están los archivos reales que implementan cada concepto:

Type Hints en el Portafolio

```
# BankChurn-Predictor/src/bankchurn/config.py (líneas 89-109)
@classmethod
def from_yaml(cls, config_path: str | Path) -> BankChurnConfig:
    """Load configuration from YAML file.

    Parameters
    ----------
    config_path : str or Path
        Path to YAML configuration file.

    Returns
    -------
    config : BankChurnConfig
        Validated configuration object.
    """
```

Pydantic en el Portafolio

Cada proyecto tiene su configuración Pydantic:

Proyecto	Archivo	Clases principales
BankChurn	`src/bankchurn/config.py`	`BankChurnConfig`, `ModelConfig`, `DataConfig`
CarVision	`src/carvision/config.py`	`CarVisionConfig`, `FiltersConfig`
TelecomAI	`src/telecomai/config.py`	`TelecomConfig`

```
# Ejemplo real: BankChurn-Predictor/src/bankchurn/config.py
class ModelConfig(BaseModel):
    """Model training configuration."""
    type: str = "ensemble"
    test_size: float = Field(0.2, ge=0.0, le=1.0)  # ← Validación automática
    random_state: int = 42
    cv_folds: int = Field(5, ge=2)  # ← Mínimo 2 folds
```

src/ Layout en el Portafolio

Los 3 proyectos siguen exactamente la estructura descrita:

```
BankChurn-Predictor/
├── src/bankchurn/       ← Paquete instalable
│   ├── __init__.py
│   ├── config.py        ← Pydantic configs
│   ├── pipeline.py      ← sklearn Pipeline
│   └── trainer.py       ← Clase de entrenamiento
├── pyproject.toml       ← Metadata y dependencias
└── setup.py             ← Fallback para pip install -e .
```

🔧 Ejercicio: Verifica en el Repo Real

```
# 1. Ve al proyecto BankChurn
cd BankChurn-Predictor

# 2. Instala en modo editable
pip install -e ".[dev]"

# 3. Verifica tipos con mypy
mypy src/bankchurn/config.py

# 4. Prueba que Pydantic valida correctamente
python -c "from bankchurn.config import BankChurnConfig; print(BankChurnConfig.from_yaml('configs/config.yaml'))"
```

💼 Consejos Profesionales

Recomendaciones para destacar en entrevistas y proyectos reales

Para Entrevistas

1. **Domina Type Hints**: Los entrevistadores valoran código tipado. Practica explicar por qué `def process(data: pd.DataFrame) -> Dict[str, float]` es mejor que `def process(data)`.
2. **Conoce Pydantic vs Dataclasses**: Pregunta común: "¿Cuándo usarías uno u otro?" Respuesta: Pydantic para validación de datos externos (APIs, configs), dataclasses para estructuras internas simples.
3. **Demuestra comprensión de `__init__.py`**: Explica cómo controla la API pública de un paquete y por qué `from package import *` es peligroso.

Para Proyectos Reales

Situación	Consejo
Código legacy sin tipos	Añade tipos gradualmente, empezando por funciones públicas
Validación de configs	Usa Pydantic con `model_validator` para validaciones cruzadas
Logs en producción	Usa `structlog` o `loguru` en lugar de `print()`
Errores en producción	Implementa excepciones personalizadas con contexto útil

Anti-patrones a Evitar

- ❌ `from typing import *` — importa solo lo que necesitas
- ❌ `except Exception:` sin logging — siempre registra el error
- ❌ Funciones de más de 50 líneas — refactoriza en funciones más pequeñas
- ❌ Nombres como `data`, `info`, `result` — usa nombres descriptivos

📺 Recursos Externos Recomendados

Ver RECURSOS_POR_MODULO.md para la lista completa.

🏷️	Recurso	Tipo
🔴	Type Hints - ArjanCodes	Video
🔴	Pydantic V2 Tutorial	Video
🟡	Python Type Checking - Real Python	Tutorial

Documentación oficial:
- PEP 484 – Type Hints
- Pydantic Documentation
- Python Packaging Guide

🔗 Referencias del Glosario

Ver 21_GLOSARIO.md para definiciones de:
- **Type Hints**: Anotaciones de tipos en Python
- **Pydantic**: Validación de datos con type hints
- **src/ Layout**: Estructura de proyecto profesional

✅ Ejercicios

Ver EJERCICIOS.md - Módulo 01:
- 1.1: Añadir type hints a funciones
- 1.2: Crear config con Pydantic
- 1.3: Estructurar proyecto con src/ layout

DISEÑO DE SISTEMAS

GUÍA MLOPS V5.0

PORTFOLIO EDITION

MÓDULO 02: DISEÑO DE SISTEMAS ML

Del Problema de Negocio a la Arquitectura Técnica

Guía MLOps v5.0: Senior Edition | DuqueOM | Noviembre 2025

📐 MÓDULO 02: Diseño de Sistemas ML

Del Problema de Negocio a la Arquitectura Técnica

"Un arquitecto Senior no dibuja casas bonitas; diseña sistemas que sobreviven a terremotos, a cambios de requisitos y a desarrolladores que se van."

Duración	Teoría	Práctica
5-6 horas	40%	60%

🎯 ADR de Inicio: ¿Por Qué Diseñar Antes de Codificar?

```
╔═══════════════════════════════════════════════════════════════════╗
│  ADR-002: Diseño Obligatorio Antes del Código                     │
╠═══════════════════════════════════════════════════════════════════╣
│                                                                   │
│  CONTEXTO:                                                        │
│  El 73% de proyectos ML que fallan lo hacen por problemas de DISEÑO,
│  no de algoritmos (Sculley et al., "Hidden Technical Debt in ML Systems").
│                                                                   │
│  DECISIÓN:                                                        │
│  Todo proyecto debe completar: ML Canvas + Diagrama de Arquitectura +
│  ADRs para decisiones técnicas clave ANTES de escribir código.    │
│                                                                   │
│  CONSECUENCIAS:                                                   │
│  (+) Alineamiento stakeholders-equipo desde el inicio             │
│  (+) Documentación de trade-offs para futuros desarrolladores     │
│  (+) Menor retrabajo por requisitos mal entendidos                │
│  (-) Añade 1-2 semanas al inicio del proyecto                     │
│                                                                   │
╚═══════════════════════════════════════════════════════════════════╝
```

Lo Que Lograrás en Este Módulo

1. **Traducir** problemas de negocio a problemas de ML con métricas claras
2. **Completar** un ML Canvas profesional
3. **Diseñar** arquitectura usando el modelo C 4
4. **Documentar** decisiones técnicas con ADRs
5. **Crear** un diagrama de flujo de datos

2.1 Traducción Negocio → ML (El Arte del Senior)

El Anti-Patrón: "Tengo Datos, Voy a Hacer ML"

```
╔═══════════════════════════════════════════════════════════════════╗
│                        ⚠️ EL ERROR DEL JUNIOR                     │
╠═══════════════════════════════════════════════════════════════════╣
│                                                                   │
│   JUNIOR:                                                         │
│   "Tengo datos de clientes → Voy a probar XGBoost → Algo saldrá"  │
│                                                                   │
│   PROBLEMA:                                                       │
│   • No sabe qué métrica optimizar (¿accuracy? ¿F1? ¿costo de negocio?)
│   • No sabe si el modelo genera valor                             │
│   • No puede priorizar features porque no entiende el negocio     │
│   • Cuando el proyecto "termina", nadie lo usa                    │
│                                                                   │
╠═══════════════════════════════════════════════════════════════════╣
│                        ✅ EL ENFOQUE DEL SENIOR                   │
╠═══════════════════════════════════════════════════════════════════╣
│                                                                   │
│   SENIOR:                                                         │
│   "El banco pierde $2M/año por churn → Predecir top 10% de riesgo →
│    Campaña de retención → ROI esperado $400K → Métricas:          │
```

```
│    Precision@10% > 50%, AUC > 0.85, Latencia < 100ms"                    │
│                                                                          │
│   VENTAJAS:                                                              │
│   • Métrica clara conectada a $$$                                        │
│   • Sabe cuándo el modelo es "suficientemente bueno"                     │
│   • Puede justificar inversión en infraestructura                        │
│   • El proyecto genera valor medible                                     │
│                                                                          │
└──────────────────────────────────────────────────────────────────────────┘
```

Framework de Traducción: Negocio → ML → Sistema

```
flowchart LR
    subgraph Negocio["🏢 NEGOCIO"]
        A[Problema de Negocio]
        B[KPIs de Éxito]
        C[Restricciones]
    end

    subgraph ML["🧠 ML"]
        D[Tipo de Problema]
        E[Métricas Técnicas]
        F[Features Candidatas]
    end

    subgraph Sistema["⚙️ SISTEMA"]
        G[Requisitos No-Funcionales]
        H[Arquitectura]
        I[Stack Tecnológico]
    end

    A --> D
    B --> E
    C --> G
    D --> H
    E --> H
    F --> H
    G --> I
```

Tabla de Traducción (Ejemplos)

Problema de Negocio	Tipo ML	Métrica Negocio	Métrica Técnica	Requisito Sistema
Reducir abandono de clientes	Clasificación Binaria	$ retenido/año	AUC-ROC, Precision@K	Batch diario o API < 100 ms
Estimar precio de vehículos	Regresión	% error en valuación	RMSE, MAPE	API síncrona < 200 ms
Detectar fraude en tarjetas	Anomaly Detection	$ fraude evitado	Precision, Recall	Streaming < 50 ms
Recomendar productos	Ranking/RecSys	Lift en ventas	NDCG@10, MAP	API < 100 ms, cold-start handling
Predecir demanda	Series Temporales	% reducción stockout	MAPE, Bias	Batch semanal

Ejercicio 2.1: Traduce Tu Problema

Para el proyecto que elegiste (BankChurn, CarVision, TelecomAI o propio):

1. **Problema de Negocio**: ¿Qué duele? ¿Cuánto cuesta?
2. **Tipo de ML**: ¿Clasificación, regresión, clustering, etc.?
3. **Métrica de Negocio**: ¿Cómo se mide el éxito en $$$?
4. **Métrica Técnica**: ¿Qué optimizamos? (AUC, RMSE, etc.)
5. **Requisito de Sistema**: ¿Batch o real-time? ¿Latencia?

2.2 ML Canvas: El Blueprint del Proyecto

¿Qué es el ML Canvas?

El **ML Canvas** es un framework de 1 página que captura todas las decisiones clave de un proyecto ML. Es como el Business Model Canvas pero para sistemas de ML.

```
╔═══════════════════════════════════════════════════════════════════════════╗
║                                ML CANVAS: BANKCHURN PREDICTOR                                ╒
╔═══════════════════════════════════════════════════════════════════════════╗
║                                                                                                ╒
║  ┌──────────────────────────────────────┐  ┌──────────────────────────────────────┐  ╒
║  │ 1. 🎯 PROBLEMA DE NEGOCIO            │  │ 2. 💰 PROPUESTA DE VALOR              │  ╒
║  │                                      │  │                                      │  ╒
║  │ • El banco pierde $2M/año por clientes │  │ • Reducir churn 20% = $400K ahorro/año │  ╒
║  │   que abandonan sin previo aviso     │  │ • Identificar top 10% clientes en riesgo │  ╒
║  │ • Equipo de retención actúa reactivamente │  │ • Tiempo de acción: de 0 días a 30 días previo │  ╒
║  │ • Costo de adquisición 5x vs retención │  │ • Campañas personalizadas por segmento riesgo │  ╒
║  │                                      │  │                                      │  ╒
║  └──────────────────────────────────────┘  └──────────────────────────────────────┘  ╒
║                                                                                                ╒
║  ┌──────────────────────────────────────┐  ┌──────────────────────────────────────┐  ╒
║  │ 3. 📊 DATOS DISPONIBLES               │  │ 4. 🔧 FEATURES CANDIDATAS             │  ╒
║  │                                      │  │                                      │  ╒
║  │ FUENTE: Sistema CRM (PostgreSQL)     │  │ DEMOGRÁFICAS:                        │  ╒
║  │ • 10,000 registros históricos (2 años) │  │ • Age, Gender, Geography             │  ╒
║  │ • Label: Exited (0=activo, 1=abandonó) │  │                                      │  ╒
║  │ • Frecuencia: Snapshot mensual       │  │ FINANCIERAS:                         │  ╒
║  │ • Latencia: T-1 día                  │  │ • CreditScore, Balance, EstimatedSalary │  ╒
║  │                                      │  │                                      │  ╒
║  │ CALIDAD:                             │  │ COMPORTAMIENTO:                      │  ╒
║  │ • Nulos: < 1%                        │  │ • Tenure, NumOfProducts, HasCrCard   │  ╒
║  │ • Desbalanceo: 20% churn (manejable) │  │ • IsActiveMember                     │  ╒
║  │ • Data drift: Estacional (navidad)   │  │                                      │  ╒
║  │                                      │  │ DERIVADAS (Feature Engineering):     │  ╒
║  │ RESTRICCIONES LEGALES:               │  │ • BalancePerProduct = Balance / NumOfProducts │  ╒
║  │ • GDPR: Pseudonimización requerida   │  │ • BalanceSalaryRatio = Balance / Salary │  ╒
║  │ • No usar: Raza, Religión, etc.      │  │ • TenureAgeRatio = Tenure / Age      │  ╒
║  │                                      │  │                                      │  ╒
║  └──────────────────────────────────────┘  └──────────────────────────────────────┘  ╒
║                                                                                                ╒
║  ┌──────────────────────────────────────┐  ┌──────────────────────────────────────┐  ╒
║  │ 5. 🤖 MODELO                          │  │ 6. 📏 MÉTRICAS DE ÉXITO               │  ╒
║  │                                      │  │                                      │  ╒
║  │ TIPO: Clasificación Binaria          │  │ NEGOCIO:                             │  ╒
║  │                                      │  │ • $ Retenido por Campaña > $400K/año │  ╒
║  │ BASELINE:                            │  │ • Lift vs random > 3x                │  ╒
║  │ • Logistic Regression (interpretable) │  │                                      │  ╒
║  │ • Umbral: 50% churn rate             │  │ MODELO:                              │  ╒
║  │                                      │  │ • AUC-ROC > 0.85 (target)            │  ╒
║  │ TARGET:                              │  │ • Precision@10% > 50%                │  ╒
║  │ • Random Forest / XGBoost            │  │ • Recall > 60% (no perder churners)  │  ╒
║  │ • Con class_weight='balanced'        │  │                                      │  ╒
║  │                                      │  │ SISTEMA:                             │  ╒
║  │ APPROACH:                            │  │ • Latencia P99 < 100ms               │  ╒
║  │ • Train/Test split temporal (no random) │  │ • Throughput > 100 req/s             │  ╒
║  │ • Cross-validation: TimeSeriesSplit  │  │ • Uptime > 99.5%                     │  ╒
║  │ • Hyperparameter tuning: Optuna      │  │ • Coverage > 80%                     │  ╒
║  │                                      │  │                                      │  ╒
║  └──────────────────────────────────────┘  └──────────────────────────────────────┘  ╒
║                                                                                                ╒
║  ┌──────────────────────────────────────┐  ┌──────────────────────────────────────┐  ╒
║  │ 7. ⚠️ RIESGOS Y MITIGACIONES          │  │ 8. 🚀 PLAN DE DESPLIEGUE              │  ╒
║  │                                      │  │                                      │  ╒
║  │ TÉCNICOS:                            │  │ MVP (Semana 10):                     │  ╒
║  │ • Desbalanceo → class_weight, SMOTE  │  │ • API REST (FastAPI)                 │  ╒
║  │ • Data leakage → Validación temporal │  │ • Docker container                   │  ╒
║  │ • Overfitting → Regularización, CV   │  │ • Consumidor: Dashboard BI (PowerBI) │  ╒
║  │                                      │  │ • Batch scoring diario               │  ╒
║  │ OPERACIONALES:                       │  │                                      │  ╒
║  │ • Model decay → Monitoreo + retrain  │  │ V2 (Mes 3):                          │  ╒
║  │ • Data drift → Evidently/NannyML     │  │ • Kubernetes deployment              │  ╒
║  │ • Latencia alta → Caching, async     │  │ • Integración CRM real-time          │  ╒
║  │                                      │  │ • A/B testing framework              │  ╒
║  │ ÉTICOS:                              │  │ • Reentrenamiento mensual automatizado │  ╒
```

```
│  │ • Sesgo geográfico → Fairness metrics      │   │                                          │  │
│  │ • Explicabilidad → SHAP values             │   │ CONSUMIDORES:                            │  │
│  │                                            │   │ • Equipo de Retención (principal)        │  │
│  │                                            │   │ • Dashboard Ejecutivo (secundario)       │  │
│  │                                            │   │ • CRM para campañas automatizadas        │  │
│  │                                            │   │                                          │  │
│  └────────────────────────────────────────────┘   └──────────────────────────────────────────┘  │
│                                                                                                  │
└──────────────────────────────────────────────────────────────────────────────────────────────────┘
```

Template Vacío para Tu Proyecto

```
# ML CANVAS: [NOMBRE DEL PROYECTO]

## 1. 🎯 Problema de Negocio
- ¿Qué duele?
- ¿Cuánto cuesta el problema actual?
- ¿Quién sufre?

## 2. 💰 Propuesta de Valor
- ¿Cómo ML alivia el dolor?
- ¿Cuál es el ROI esperado?
- ¿Qué decisiones habilita?

## 3. 📊 Datos Disponibles
- Fuente:
- Volumen:
- Frecuencia:
- Calidad (nulos, duplicados):
- Restricciones legales:

## 4. 🔧 Features Candidatas
- Demográficas:
- Transaccionales:
- Comportamiento:
- Derivadas (feature engineering):

## 5. 🤖 Modelo
- Tipo de problema:
- Baseline:
- Target:
- Approach de validación:

## 6. 📏 Métricas de Éxito
- Negocio:
- Modelo:
- Sistema:

## 7. ⚠️ Riesgos y Mitigaciones
- Técnicos:
- Operacionales:
- Éticos:

## 8. 🚀 Plan de Despliegue
- MVP:
- V2:
- Consumidores:
```

2.3 Arquitectura con el Modelo C4

¿Qué es C4?

El **modelo C** 4 (Context, Container, Component, Code) es un framework para documentar arquitectura de software en 4 niveles de zoom.

```
╔═══════════════════════════════════════════════════════════════════════════════╗
│                         MODELO C4: 4 NIVELES DE ZOOM                          │
╠═══════════════════════════════════════════════════════════════════════════════╗
│                                                                               │
│   NIVEL 1: CONTEXTO (System Context)                                          │
│   • Vista de pájaro: El sistema y sus usuarios/sistemas externos              │
│   • Audiencia: Todos (stakeholders, devs, ops)                                │
│   • Pregunta: "¿Qué es esto y quién lo usa?"                                  │
│                                                                               │
```

```
│   NIVEL 2: CONTENEDORES (Container)                                    │
│   • Zoom in: Aplicaciones, bases de datos, servicios                   │
│   • Audiencia: Arquitectos, tech leads                                 │
│   • Pregunta: "¿Qué partes tiene el sistema?"                          │
│                                                                        │
│   NIVEL 3: COMPONENTES (Component)                                     │
│   • Zoom in++: Módulos dentro de cada contenedor                       │
│   • Audiencia: Desarrolladores                                         │
│   • Pregunta: "¿Cómo está organizado internamente?"                    │
│                                                                        │
│   NIVEL 4: CÓDIGO (Code)                                               │
│   • Máximo zoom: Clases, funciones                                     │
│   • Audiencia: Desarrolladores (el que va a implementar)               │
│   • Nota: Usualmente se genera desde el código, no se dibuja           │
│                                                                        │
└────────────────────────────────────────────────────────────────────────┘
```

Nivel 1: Contexto del Sistema BankChurn

```
C4Context
    title Sistema BankChurn - Diagrama de Contexto

    Person(retention_team, "Equipo Retención", "Actúa sobre predicciones para retener clientes")
    Person(data_team, "Data Team", "Monitorea y mejora modelos")

    System(bankchurn, "BankChurn Predictor", "Predice probabilidad de churn y genera scores de riesgo")

    System_Ext(crm, "Sistema CRM", "Fuente de datos de clientes")
    System_Ext(bi, "Dashboard BI", "Visualiza scores y métricas")
    System_Ext(campaign, "Sistema Campañas", "Ejecuta acciones de retención")

    Rel(retention_team, bankchurn, "Consulta scores", "API/Dashboard")
    Rel(data_team, bankchurn, "Monitorea y retrains", "MLflow/Grafana")
    Rel(crm, bankchurn, "Envía datos clientes", "Batch/API")
    Rel(bankchurn, bi, "Envía predicciones", "API")
    Rel(bankchurn, campaign, "Envía lista de riesgo", "API/Webhook")
```

Nivel 2: Contenedores

```
C4Container
    title Sistema BankChurn - Diagrama de Contenedores

    Person(user, "Usuario", "Equipo Retención / Data Team")

    Container_Boundary(bankchurn, "BankChurn Predictor") {
        Container(api, "API REST", "FastAPI", "Expone endpoints de predicción y health")
        Container(model, "ML Pipeline", "Sklearn", "Pipeline entrenado: preprocessor + model")
        Container(mlflow, "MLflow", "Python", "Tracking de experimentos y model registry")
        ContainerDb(db, "Model Storage", "S3/Local", "Artefactos .pkl")
    }

    System_Ext(crm, "CRM", "PostgreSQL")
    System_Ext(prometheus, "Prometheus", "Métricas")
    System_Ext(grafana, "Grafana", "Dashboards")

    Rel(user, api, "HTTP/JSON", "REST")
    Rel(api, model, "Loads", "joblib")
    Rel(model, db, "Reads", "S3/File")
    Rel(mlflow, db, "Writes", "Artifacts")
    Rel(crm, api, "Batch data", "CSV/API")
    Rel(api, prometheus, "Exposes /metrics", "HTTP")
    Rel(prometheus, grafana, "Scrapes", "PromQL")
```

Nivel 3: Componentes (API Container)

```
C4Component
    title API REST - Diagrama de Componentes

    Container_Boundary(api, "API REST (FastAPI)") {
        Component(routes, "Routes", "fastapi.APIRouter", "Define endpoints: /predict, /health, /metrics")
        Component(schemas, "Schemas", "Pydantic", "Valida requests y responses")
        Component(inference, "Inference Service", "Python Class", "Carga modelo y ejecuta predicción")
        Component(middleware, "Middleware", "FastAPI", "Logging, timing, error handling")
```

```
        Component(config, "Config", "Pydantic Settings", "Lee variables de entorno")
    }

    ContainerDb(model_store, "Model Store", "S3")
    Container(prometheus, "Prometheus Client", "prometheus_client")

    Rel(routes, schemas, "Uses", "Type validation")
    Rel(routes, inference, "Calls", "predict()")
    Rel(inference, model_store, "Loads", "joblib.load()")
    Rel(middleware, prometheus, "Exposes", "Counters/Histograms")
    Rel(routes, config, "Reads", "Settings")
```

2.4 Diagrama de Flujo de Datos

Training Pipeline

```
flowchart LR
    subgraph Data["📦 DATA"]
        RAW[(Raw Data<br/>data/raw/)]
        PROC[(Processed<br/>data/processed/)]
    end

    subgraph Training["🔄 TRAINING"]
        LOAD[Load Data]
        FE[Feature<br/>Engineering]
        SPLIT[Train/Test<br/>Split]
        TRAIN[Train Model]
        EVAL[Evaluate]
    end

    subgraph Artifacts["📁 ARTIFACTS"]
        MODEL[(Pipeline.pkl<br/>models/)]
        METRICS[(Metrics<br/>mlruns/)]
    end

    subgraph Tracking["📊 TRACKING"]
        MLFLOW[MLflow<br/>Server]
    end

    RAW --> LOAD
    LOAD --> FE
    FE --> PROC
    FE --> SPLIT
    SPLIT --> TRAIN
    TRAIN --> EVAL
    EVAL --> MODEL
    EVAL --> METRICS
    METRICS --> MLFLOW
    MODEL --> MLFLOW

    style Data fill:#e1f5fe
    style Training fill:#fff3e0
    style Artifacts fill:#e8f5e9
    style Tracking fill:#f3e5f5
```

Inference Pipeline

```
flowchart LR
    subgraph Client["👤 CLIENT"]
        REQ[JSON Request]
        RESP[JSON Response]
    end

    subgraph API["🌐 API"]
        VALID[Validate<br/>Pydantic]
        INFER[Inference<br/>Service]
        FORMAT[Format<br/>Response]
    end

    subgraph Model["🤖 MODEL"]
        LOAD[Load<br/>Pipeline]
        PREDICT[Predict<br/>Proba]
    end
```

```
    subgraph Monitor["📊 MONITOR"]
        LOGS[Structured<br/>Logs]
        METRICS[Prometheus<br/>Metrics]
    end

    REQ --> VALID
    VALID --> INFER
    INFER --> LOAD
    LOAD --> PREDICT
    PREDICT --> FORMAT
    FORMAT --> RESP

    INFER --> LOGS
    INFER --> METRICS

    style Client fill:#e3f2fd
    style API fill:#fff8e1
    style Model fill:#e8f5e9
    style Monitor fill:#fce4ec
```

2.5 Architecture Decision Records (ADRs)

¿Qué es un ADR?

Un **ADR** documenta una decisión arquitectónica importante: el contexto, la decisión tomada, y las consecuencias.

Template ADR

```
# ADR-XXX: [Título de la Decisión]

## Estado
[Propuesto | Aceptado | Deprecado | Superseded por ADR-YYY]

## Contexto
¿Cuál es el problema que estamos tratando de resolver?
¿Qué restricciones tenemos?

## Decisión
¿Qué decidimos hacer?

## Consecuencias

### Positivas
-

### Negativas
-

### Neutras
-

## Alternativas Consideradas
| Alternativa | Pros | Contras | Razón de Rechazo |
| ----------- | ---- | ------- | ---------------- |
|             |      |         |                  |

## Referencias
- Links a documentación, papers, etc.
```

ADRs Ejemplo para BankChurn

ADR-003: FastAPI sobre Flask

```
# ADR-003: Usar FastAPI para la API REST

## Estado
Aceptado

## Contexto
Necesitamos exponer el modelo como API REST. Las opciones principales son:
- Flask (maduro, amplia adopción)
- FastAPI (moderno, async, tipado)
- Django REST (batteries-included, pero pesado)

## Decisión
```

```
Usaremos **FastAPI** para la API REST.

## Consecuencias

### Positivas
- Validación automática con Pydantic (ya usamos para config)
- Documentación OpenAPI generada automáticamente
- Soporte nativo async (mejor performance bajo carga)
- Type hints forzados (consistente con nuestro código)
- Mejor performance que Flask (Starlette + Uvicorn)

### Negativas
- Menos tutoriales/recursos que Flask (aunque creciendo rápidamente)
- Requiere entender async/await para features avanzados
- Algunos desarrolladores pueden no estar familiarizados

### Neutras
- Similar curva de aprendizaje inicial que Flask

## Alternativas Consideradas
| Alternativa | Pros | Contras | Razón de Rechazo |
| ----------- | ---- | ------- | ---------------- |
| Flask | Maduro, muchos recursos | Sin async, sin tipos, docs manual | Performance y DX inferior |
| Django REST | Batteries-included | Muy pesado para API simple, ORM no necesario | Overkill para nuestro caso |

## Referencias
- [FastAPI vs Flask Benchmark](https://fastapi.tiangolo.com/benchmarks/)
- [Why FastAPI](https://fastapi.tiangolo.com/features/)
```

ADR-004: DVC sobre Git LFS

```
# ADR-004: Usar DVC para Versionado de Datos

## Estado
Aceptado

## Contexto
Necesitamos versionar:
- Dataset de entrenamiento (CSV ~50MB)
- Modelos entrenados (PKL ~10MB)
- Posible crecimiento a GB en el futuro

## Decisión
Usaremos **DVC** (Data Version Control) con remote storage en S3/GCS.

## Consecuencias

### Positivas
- Integración nativa con Git (cada versión de datos linked a commit)
- Pipelines declarativos (dvc.yaml)
- Múltiples backends de storage (local, S3, GCS, Azure)
- Reproducibilidad con `dvc repro`
- Comunidad activa, bien documentado

### Negativas
- Curva de aprendizaje adicional
- Requiere setup de remote storage para colaboración
- Puede ser overkill para datasets muy pequeños

### Neutras
- CLI similar a Git (familiar)

## Alternativas Consideradas
| Alternativa | Pros | Contras | Razón de Rechazo |
| ----------- | ---- | ------- | ---------------- |
| Git LFS | Simple, integrado en Git | No soporta pipelines, costoso para archivos grandes | Sin reproducibilidad de
pipelines |
| Delta Lake | Excelente para data lakes | Requiere Spark, overkill para nuestro caso | Complejidad innecesaria |
| DagsHub | DVC + MLflow hosted | Vendor lock-in, costo | Preferimos self-hosted |

## Referencias
- [DVC vs Git LFS](https://dvc.org/doc/user-guide/large-dataset-optimization)
```

2.6 Ejercicio Integrador: Diseña Tu Sistema

Entregables

Para tu proyecto elegido, crea los siguientes archivos en `docs/`:

1. **`ML_CANVAS.md`**: ML Canvas completo (usa el template)
2. **`ARCHITECTURE.md`**: Diagramas C4 (al menos Contexto y Contenedores)
3. **`decisions/ADR-001.md`**: Al menos 2 ADRs para decisiones clave

Criterios de Evaluación

Criterio	Básico (60-69)	Competente (70-84)	Destacado (85-100)
ML Canvas	Secciones incompletas	Todas las secciones, algunos detalles vagos	Completo con métricas específicas y cuantificadas
Diagramas	Solo texto descriptivo	Diagramas ASCII o básicos	Mermaid/PlantUML correctos y claros
ADRs	Sin ADRs	1 ADR básico	2+ ADRs con alternativas y trade-offs

🧨 Errores habituales y cómo depurarlos en el diseño de sistemas ML

Este módulo es de **arquitectura y diseño**, así que muchos errores no se ven como stack traces, sino como **malas decisiones** que explotan meses después. La idea es aprender a detectarlos temprano.

1) ML Canvas bonito pero inútil (problema de negocio vago)

Síntomas típicos

- El ML Canvas está lleno de buzzwords: "mejorar la experiencia del usuario", "optimizar procesos".
- No hay números: ni costo actual, ni ROI esperado, ni objetivo cuantitativo.
- Nadie del negocio puede decir si el modelo "valió la pena" o no.

Cómo identificarlo

- Relee tus secciones 1. **Problema de Negocio** y 2. **Propuesta de Valor**:
- ¿Hay cantidades concretas? (`$`, %, horas, tickets, churn...)
- ¿Existe una hipótesis de mejora medible?
- Pregúntate: "si mañana entrego el modelo, ¿cómo sabría si impactó algo?".

Cómo corregirlo

- Fuerza al menos:
- 1 métrica de negocio actual (ej: churn 20%, tiempo de resolución 48h).
- 1 objetivo de mejora (reducir churn a 16%, bajar a 24h).
- 1 métrica técnica alineada (AUC, RMSE, etc.).
- Usa como referencia los ejemplos de **BankChurn**, **CarVision** y **TelecomAI**:
- Revisa sus READMEs y métricas en MLflow para ver cómo se conectan a objetivos claros.

2) Diagramas C4 que no reflejan el código real

Síntomas típicos

- El diagrama de contenedores muestra 10 microservicios, pero en el repo solo hay 1 API monolítica.
- Aparecen bases de datos o colas que **no existen** en `docker-compose.demo.yml` ni en `k8s/`.
- Personas nuevas en el equipo se confunden porque "la arquitectura del doc no coincide con la realidad".

Cómo identificarlo

- Abre simultáneamente:
- `docs/architecture/*.md` en la raíz del portafolio.
- `BankChurn-Predictor/docs/ARCHITECTURE.md`, `CarVision-Market-Intelligence/docs/ARCHITECTURE.md`, `TelecomAI-Customer-Intelligence/docs/ARCHITECTURE.md`.
- `docker-compose.demo.yml` y los manifests de `k8s/`.
- Recorre tu diagrama C4 y marca:
- ¿Existe un **mapeo 1:1** entre contenedores y artefactos reales (servicio Docker, deployment de K8s, app FastAPI/Streamlit)?

Cómo corregirlo

- Primero, **documenta la arquitectura que realmente existe hoy** (MVP), no la ideal de dentro de 1 año.
- Asegúrate de que cada contenedor en el diagrama:
- Tiene un `Dockerfile` o entrada en `docker-compose.demo.yml`.
- O es un servicio externo claramente etiquetado (MLflow, Prometheus, Grafana, CRM, etc.).
- Para la arquitectura futura, sepárala explícitamente como **"V 2 / visión"** para no confundir.

3) Ignorar requisitos no funcionales (latencia, throughput, observabilidad)

Síntomas típicos

- El modelo en notebook va bien, pero la API en producción tiene:
- Latencias > 2 – 3 s.
- Timeouts bajo carga.
- Métricas inexistentes o imposibles de interpretar.
- No hay líneas claras en el ML Canvas ni en C 4 sobre **SLAs/SLIs**.

Cómo identificarlo

- Revisa las secciones 6 **. Métricas de Éxito** y tus diagramas:
- ¿Hablaste de latencia, throughput, uptime, métricas de observabilidad?
- ¿Tu diagrama de inferencia incluye Prometheus/Grafana/Logging como en `GUIA_AUDIOVISUAL.md` y los manifests de `k8s/`?

Cómo corregirlo

- Añade explícitamente en el ML Canvas:
- **Métricas de sistema** (latencia P 9 5 /P 9 9 , QPS, uptime, tiempo de warmup).
- **Métricas de monitoreo** (errores 5 xx, requests por endpoint, uso de CPU/memoria).
- Refleja esos componentes en C 4 :
- Contenedores Prometheus/Grafana.
- Endpoints `/metrics` en las APIs FastAPI.
- Conecta esto al futuro módulo de **Observabilidad (1 6 _OBSERVABILIDAD)** para que el diseño no sea "ciego".

4) ADRs inexistentes o que nadie lee

Síntomas típicos

- Decisiones importantes viven solo en la cabeza de alguien: "eso lo decidió X hace meses".
- Existen ADRs, pero:
- Están vacíos, sin alternativas ni consecuencias.
- Nadie los actualiza cuando se revierte una decisión.

Cómo identificarlo

- Revisa `DECISIONES_TECH.md` y cualquier carpeta `docs/decisions/`.
- Pregúntate para cada sección del sistema:
- ¿Por qué FastAPI y no Flask?
- ¿Por qué DVC y no Git LFS?
- ¿Por qué MLflow y no W&B?
- Si la respuesta no está escrita en un ADR, tienes una **decisión tácita** peligrosa.

Cómo corregirlo

- Para cada decisión grande (API, tracking, versionado de datos, infraestructura):
- Crea un ADR corto siguiendo el template de este módulo.
- Añade al menos 1 **alternativa rechazada** y la razón.
- Marca el estado (`Aceptado`, `Deprecado`, `Superseded`) cuando cambies de opinión.
- Enlaza los ADRs desde `DECISIONES_TECH.md` y desde la documentación de cada proyecto.

5) Patrón de debugging de diseño

1 . **Empieza en el negocio**: revisa si el problema y el ROI están cuantificados.
2 . **Sigue con el Canvas**: ¿están completas las 8 secciones? ¿faltan riesgos o restricciones legales?
3 . **Baja a C 4** : verifica que contextos y contenedores existen realmente en código/infra.
4 . **Cierra con ADRs**: asegúrate de que las decisiones clave no viven solo en la memoria del equipo.

Si recorres este pipeline de pensamiento cada vez que diseñas (o revisas) un sistema, pensarás como un arquitecto Senior incluso en proyectos pequeños.

2.7 Autoevaluación

Checklist

```
TRADUCCIÓN NEGOCIO → ML:
[ ] Puedo identificar el problema de negocio detrás de un proyecto ML
[ ] Sé calcular ROI esperado de una solución ML
[ ] Puedo elegir la métrica técnica correcta según el problema

ML CANVAS:
[ ] Puedo completar las 8 secciones del ML Canvas
[ ] Sé identificar riesgos técnicos, operacionales y éticos
[ ] Puedo definir métricas de negocio, modelo y sistema

ARQUITECTURA C4:
[ ] Entiendo los 4 niveles del modelo C4
[ ] Puedo dibujar diagramas de Contexto y Contenedores
[ ] Sé usar Mermaid para diagramas

ADRs:
[ ] Entiendo el propósito de los ADRs
[ ] Puedo documentar decisiones con alternativas y trade-offs
[ ] Sé cuándo crear un nuevo ADR vs actualizar uno existente
```

Preguntas de Reflexión

1. ¿Por qué es importante cuantificar el problema de negocio antes de empezar?
2. ¿Qué pasa si no documentas las decisiones arquitectónicas?
3. ¿Cuándo es apropiado NO usar ML para resolver un problema?

📦 Cómo se Usó en el Portafolio

El diseño de sistemas no es solo teoría. Aquí está cómo se aplicó en el portafolio real:

ML Canvas del Portafolio

Cada proyecto tiene su Canvas implícito en la documentación:

Proyecto	Problema de Negocio	Métrica de Negocio	Métrica ML
BankChurn	Reducir pérdida de clientes	Retención + 5 %	AUC-ROC, Recall
CarVision	Pricing automatizado de autos	Error de precio < 10 %	MAE, R^2
TelecomAI	Segmentación de clientes	Campañas personalizadas	Accuracy, F1

Arquitectura C4 del Portafolio

```
┌─────────────────────────────────────────────────────────────┐
│                  ML-MLOps-Portfolio (CONTEXTO)              │
├─────────────────────────────────────────────────────────────┤
│                                                             │
│  ┌──────────────┐  ┌──────────────┐  ┌──────────────┐      │
│  │  BankChurn   │  │  CarVision   │  │  TelecomAI   │      │
│  │  Predictor   │  │  Market Intel│  │  Customer Int│      │
│  └──────┬───────┘  └──────┬───────┘  └──────┬───────┘      │
│         │                 │                 │              │
│         └─────────────────┴─────────────────┘              │
│                           │                                │
│                   ┌───────▼──────┐                         │
│                   │  common_utils/ │                       │
│                   │  (logger, seed) │                      │
│                   └──────┬───────┘                         │
│                          │                                 │
│          ┌───────────────┴──────────────┐                  │
│          │        GitHub Actions CI     │                  │
│          │  (ci-mlops.yml, matrix testing)                 │
│          └──────────────────────────────┘                  │
│                                                             │
└─────────────────────────────────────────────────────────────┘
```

ADRs del Portafolio

Las decisiones arquitectónicas están documentadas en:
- `docs/guia_mlops/DECISIONES_TECH.md` → ADRs globales
- `*/docs/ARCHITECTURE.md` → ADRs por proyecto

Ejemplo ADR real del portafolio:

```
# ADR-001: Pipeline Unificado vs Artefactos Separados

## Contexto
Inicialmente BankChurn guardaba preprocessor.pkl y model.pkl por separado.

## Decisión
Unificar todo en un solo pipeline.pkl

## Consecuencias
✅ Elimina training-serving skew
✅ Un solo artefacto para deploy
❌ Archivo más grande
```

🔧 Ejercicio: Revisa la Arquitectura Real

```
# Ver la arquitectura de BankChurn
cat BankChurn-Predictor/docs/architecture.md

# Ver las decisiones técnicas globales
cat docs/guia_mlops/DECISIONES_TECH.md
```

📊 Diagramas Mermaid de Arquitectura

Flujo de Datos MLOps

```
flowchart LR
    subgraph Data["📊 Data Layer"]
        RAW[(Raw Data)]
        DVC[DVC Storage]
    end

    subgraph Training["🎯 Training"]
        PIPE[sklearn Pipeline]
        MLFLOW[MLflow Tracking]
        ART[Artifacts]
    end

    subgraph Serving["🚀 Serving"]
        API[FastAPI]
        DASH[Streamlit]
    end

    subgraph Ops["⚙️ Operations"]
        CI[GitHub Actions]
        DOCKER[Docker]
        K8S[Kubernetes]
    end

    RAW --> DVC
    DVC --> PIPE
    PIPE --> MLFLOW
    PIPE --> ART
    ART --> API
    ART --> DASH
    CI --> DOCKER
    DOCKER --> K8S
    K8S --> API
```

Pipeline de ML (C4 - Container)

```
flowchart TB
    subgraph Pipeline["sklearn Pipeline"]
        direction TB
        FE[FeatureEngineer]
        CT[ColumnTransformer]
```

```
        MODEL[RandomForest]

        FE --> CT --> MODEL
    end

    subgraph Preprocessor["ColumnTransformer"]
        NUM[Numeric: Imputer + Scaler]
        CAT[Categorical: OneHotEncoder]
        BIN[Binary: Passthrough]
    end

    CT -.-> Preprocessor
```

CI/CD Pipeline

```
flowchart LR
    PUSH[Git Push] --> LINT[Lint & Format]
    LINT --> TEST[pytest + Coverage]
    TEST --> SEC[Security Scan]
    SEC --> BUILD[Docker Build]
    BUILD --> PUSH_REG[Push to Registry]
    PUSH_REG --> DEPLOY[Deploy to K8s]

    style PUSH fill:#e1f5fe
    style DEPLOY fill:#c8e6c9
```

💼 Consejos Profesionales

Recomendaciones para destacar en entrevistas y proyectos reales

Para Entrevistas de System Design

1. **Siempre empieza con requisitos**: Antes de dibujar, pregunta sobre escala, latencia esperada, y casos de uso principales.
2. **Conoce los trade-offs**: "¿Por qué elegiste esta arquitectura?" es la pregunta que siempre viene. Ten lista tu justificación.
3. **Menciona observabilidad**: Pocos candidatos hablan de logs, métricas y alertas. Hacerlo te diferencia.

Para Proyectos Reales

Situación	Consejo
Sistema nuevo	Empieza simple (monolito), escala después
Alta disponibilidad	Diseña para fallos: circuit breakers, retries, fallbacks
Decisiones de arquitectura	Documenta en ADRs (Architecture Decision Records)
Integración con ML	Separa serving de training, usa feature stores

Patrones que Debes Conocer

- **Batch vs Streaming**: Cuándo usar cada uno para ML pipelines
- **Event-Driven**: Para sistemas desacoplados y escalables
- **CQRS**: Cuando lectura y escritura tienen requisitos muy diferentes
- **Saga Pattern**: Para transacciones distribuidas

📺 Recursos Externos Recomendados

Ver RECURSOS_POR_MODULO.md para la lista completa.

🏷️	Recurso	Tipo
🔴	System Design for ML - Chip Huyen	Video
🟡	C4 Model - Simon Brown	Video
🟢	ML Canvas	Herramienta

🔗 Referencias del Glosario

Ver 21_GLOSARIO.md para definiciones de:
- **ADR**: Architecture Decision Record
- **C4 Model**: Modelo de documentación de arquitectura
- **ML Canvas**: Plantilla para diseño de proyectos ML

✅ Ejercicios

Ver EJERCICIOS.md - Módulo 02:
- 2.1: Completar ML Canvas para un proyecto
- 2.2: Escribir un ADR para una decisión técnica

➡️ Siguiente Paso

Con el diseño completo, es hora de configurar un **entorno de desarrollo profesional**.

Ir a Módulo 03: Estructura de Proyecto →

ESTRUCTURA DE PROYECTO

GUÍA MLOPS V5.0

PORTFOLIO EDITION

03. Estructura de Proyecto ML Profesional

🎯 Objetivo del Módulo

Crear la estructura de proyecto que usarás en los 3 proyectos del portafolio.

```
╔═══════════════════════════════════════════════════════════════════╗
│                                                                   │
│  Una buena estructura de proyecto es como los cimientos de una casa:
│  invisible cuando está bien hecha, DESASTROSA cuando está mal.    │
│                                                                   │
╚═══════════════════════════════════════════════════════════════════╝
```

📋 La Estructura del Portafolio

```
MiProyecto-ML/
│
├── src/                        # 📦 CÓDIGO FUENTE (instalable)
│   ├── __init__.py
│   └── miproyecto/
│       ├── __init__.py
│       ├── config.py           # Configuración Pydantic
│       ├── data.py             # Carga y validación de datos
│       ├── features.py         # Feature engineering
│       ├── training.py         # Pipeline de entrenamiento
│       ├── evaluation.py       # Métricas y evaluación
│       ├── prediction.py       # Inferencia
│       └── models.py           # Custom models/transformers
│
├── app/                        # 🌐 APLICACIONES
│   ├── fastapi_app.py          # API REST
│   └── streamlit_app.py        # Dashboard (opcional)
│
├── tests/                      # 🧪 TESTS (espejo de src/)
│   ├── __init__.py
│   ├── conftest.py             # Fixtures compartidas
│   ├── test_config.py
│   ├── test_data.py
│   ├── test_features.py
│   ├── test_training.py
│   └── test_api.py
│
├── configs/                    # ⚙️ CONFIGURACIÓN
│   └── config.yaml             # Hiperparámetros, paths, etc.
│
├── data/                       # 📊 DATOS (gitignored)
│   ├── raw/                    # Datos originales
│   └── processed/              # Datos procesados (opcional)
│
├── artifacts/                  # 📁 ARTEFACTOS (gitignored)
│   ├── model.joblib            # Modelo entrenado
│   └── metrics.json            # Métricas de entrenamiento
│
├── scripts/                    # 🔧 SCRIPTS AUXILIARES
│   └── run_mlflow.py           # Script de MLflow
│
├── docs/                       # 📖 DOCUMENTACIÓN
│   ├── model_card.md
│   └── data_card.md
│
├── infra/                      # 🏗️ INFRAESTRUCTURA (opcional)
│   └── terraform/
│
├── pyproject.toml              # 📋 METADATA DEL PROYECTO
├── requirements.txt            # 📋 DEPENDENCIAS (para CI)
├── Makefile                    # 🔨 COMANDOS COMUNES
├── Dockerfile                  # 🐳 CONTAINERIZACIÓN
├── .github/workflows/          # 🔄 CI/CD
│   └── ci.yml
├── .gitignore                  # 🚫 ARCHIVOS IGNORADOS
├── .pre-commit-config.yaml     # 🔍 HOOKS PRE-COMMIT
└── README.md                   # 📖 DOCUMENTACIÓN PRINCIPAL
```

🧩 Cómo se aplica en este portafolio

Esta estructura no es teórica: los 3 **proyectos** del portafolio la siguen con ligeras variaciones. Esto conecta directamente con los macro-módulos 00 y 01 de la **Ruta** 0 **→ Senior/Staff** descrita en el SYLLABUS.

Proyecto	Carpeta raíz	Paquete principal	Archivos clave
BankChurn Predictor	`BankChurn-Predictor/`	`src/bankchurn/`	`pyproject.toml`, `main.py`, `Makefile`, `tests/`
CarVision Market Intelligence	`CarVision-Market-Intelligence/`	`src/carvision/`	`pyproject.toml`, `main.py`, `Makefile`, `tests/`
TelecomAI Customer Intelligence	`TelecomAI-Customer-Intelligence/`	`src/telecom/`	`pyproject.toml`, `main.py`, `Makefile`, `tests/`

Para aprovechar este módulo al máximo en el repositorio real:

- **Compara** el árbol genérico de `MiProyecto-ML/` con, por ejemplo, `BankChurn-Predictor/` (fíjate especialmente en `src/`, `configs/`, `tests/`, `Makefile` y `pyproject.toml`).
- **Verifica** que los comandos que defines aquí (`make install`, `make test`, `make train`, `make serve`) tienen su equivalente funcional en los Makefiles de cada proyecto.
- **Usa** esta plantilla como referencia si creas un **cuarto proyecto** durante el 20_PROYECTO_INTEGRADOR.

📄 pyproject.toml Completo

```
# pyproject.toml - El corazón del proyecto

[build-system]
requires = ["setuptools>=61.0"]
build-backend = "setuptools.build_meta"

[project]
name = "bankchurn"
version = "1.0.0"
description = "Bank Customer Churn Prediction System"
readme = "README.md"
requires-python = ">=3.10"
license = {text = "MIT"}
authors = [
    {name = "Tu Nombre", email = "tu@email.com"}
]
keywords = ["machine-learning", "churn", "prediction"]

dependencies = [
    "pandas>=2.0.0",
    "numpy>=1.24.0",
    "scikit-learn>=1.3.0",
    "pydantic>=2.0.0",
    "pyyaml>=6.0",
    "joblib>=1.3.0",
]

[project.optional-dependencies]
api = [
    "fastapi>=0.104.0",
    "uvicorn>=0.24.0",
]
mlflow = [
    "mlflow>=2.9.0",
]
dev = [
    "pytest>=7.4.0",
    "pytest-cov>=4.1.0",
    "black>=23.0.0",
```

```
    "ruff>=0.1.0",
    "mypy>=1.7.0",
    "pre-commit>=3.5.0",
]
all = [
    "bankchurn[api,mlflow,dev]",
]

[project.scripts]
bankchurn = "bankchurn.cli:main"

[tool.setuptools.packages.find]
where = ["src"]

# ═══════════════════════════════════════════════════════════════════════
# HERRAMIENTAS
# ═══════════════════════════════════════════════════════════════════════

[tool.pytest.ini_options]
testpaths = ["tests"]
python_files = ["test_*.py"]
addopts = "-v --cov=src/bankchurn --cov-report=term-missing"

[tool.coverage.run]
source = ["src"]
omit = ["tests/*"]

[tool.coverage.report]
fail_under = 79

[tool.black]
line-length = 100
target-version = ["py311"]

[tool.ruff]
line-length = 100
select = ["E", "F", "I", "W"]
ignore = ["E501"]

[tool.mypy]
python_version = "3.11"
ignore_missing_imports = true
```

🔨 Makefile

```
# Makefile - Comandos comunes

.PHONY: install test lint format train serve clean

# Instalación
install:
pip install -e ".[all]"

install-prod:
pip install -e ".[api]"

# Testing
test:
pytest --cov=src/ --cov-fail-under=80

test-fast:
pytest -m "not slow" -x

# Linting y formato
lint:
ruff check src/ tests/
mypy src/

format:
black src/ tests/ app/
ruff check --fix src/ tests/

# Entrenamiento
train:
```

```
python main.py --seed 42 train --config configs/config.yaml --input data/raw/Churn.csv
serve:
uvicorn app.fastapi_app:app --host 0.0.0.0 --port 8000 --reload

serve-prod:
uvicorn app.fastapi_app:app --host 0.0.0.0 --port 8000

# Docker
docker-build:
docker build -t bankchurn:latest .

docker-run:
docker run -p 8000:8000 bankchurn:latest

# MLflow
mlflow-ui:
mlflow ui --host 0.0.0.0 --port 5000

# Limpieza
clean:
rm -rf __pycache__ .pytest_cache .mypy_cache .ruff_cache
rm -rf *.egg-info build dist
rm -rf htmlcov .coverage
```

🚫 .gitignore

```
# Python
__pycache__/
*.py[cod]
*.pyo
.pytest_cache/
.mypy_cache/
*.egg-info/
dist/
build/

# Entornos
.venv/
venv/
env/

# Datos y artefactos (muy grandes para Git)
data/
artifacts/
models/
*.joblib
*.pkl
*.h5

# MLflow
mlruns/

# IDE
.vscode/
.idea/
*.swp

# OS
.DS_Store
Thumbs.db

# Logs
*.log
logs/

# Coverage
.coverage
htmlcov/

# Env vars
.env
.env.local
```

🧨 Errores habituales y cómo depurarlos en la estructura de proyecto

Aquí los problemas ya no son algoritmos, sino **cómo está organizado el repo**. Son los típicos errores que hacen que algo "funcione en mi máquina pero no en CI" o que el repo se vuelva inmanejable.

1) `ModuleNotFoundError` y tests que solo funcionan desde ciertos directorios

Síntomas típicos

- En local, ejecutar `pytest` desde la raíz funciona, pero en CI falla con:
  `text ModuleNotFoundError: No module named 'miproyecto'`
- Tienes que hacer trucos como `cd src` o modificar `PYTHONPATH` para que los imports funcionen.

Cómo identificarlo

- Revisa tu estructura real:
- ¿El código está en `src/miproyecto/` o repartido por la raíz?
- ¿Los tests importan el paquete (`from miproyecto import ...`) o archivos sueltos (`import training`)?
- Mira tu `pyproject.toml`:
- `[project.name]` → ¿coincide con el nombre del paquete (`miproyecto`, `bankchurn`, etc.)?
- `[tool.setuptools.packages.find] where = ["src"]` → ¿está configurado?

Cómo corregirlo

- Mueve el código a `src/<nombre_paquete>/` siguiendo el árbol de este módulo.
- Asegúrate de que los tests importan siempre el paquete, no rutas relativas.
- Instala en modo editable durante desarrollo/CI:
  `bash pip install -e ".[dev]"`

2) Datos y modelos dentro de Git (repos gigantes e impracticables)

Síntomas típicos

- El repo pesa cientos de MB porque hay CSVs y modelos `.pkl`/`.joblib` versionados.
- `git pull` y `git clone` son lentos, y los PRs están llenos de cambios binarios.

Cómo identificarlo

- Ejecuta `git status` y revisa si aparecen archivos en `data/`, `artifacts/`, `models/`.
- Abre tu `.gitignore` y comprueba si tienes entradas como:
- `data/`, `artifacts/`, `models/`, `*.joblib`, `*.pkl`, `mlruns/`.

Cómo corregirlo

- Añade las rutas correctas a `.gitignore` (usa el snippet de este módulo como base).
- Mantén en Git **solo**:
- Código (`src/`, `app/`, `tests/`).
- Config (`configs/`).
- Infra y docs.
- Para datos/modelos usa DVC o un storage externo (se profundiza en `06_VERSIONADO_DATOS.md`).

3) Tests que no reflejan el árbol de `src/`

Síntomas típicos

- Cambias algo en `src/miproyecto/features.py` y ningún test falla, aunque has roto lógica.
- Hay tests sueltos sin relación clara con los módulos de producción.

Cómo identificarlo

- Compara árboles:
- En `src/miproyecto/`: `config.py`, `data.py`, `features.py`, `training.py`, `evaluation.py`, `prediction.py`.
- En `tests/`: ¿existen `test_config.py`, `test_data.py`, `test_features.py`, etc.?
- Revisa el `pyproject.toml` o `pytest.ini` para ver qué carpeta se usa como `testpaths`.

Cómo corregirlo

- Crea un **espejo sencillo**: por cada módulo importante en `src/`, un test correspondiente en `tests/`.
- Usa `conftest.py` para compartir fixtures (datasets pequeños, config de prueba, etc.).
- Integra `pytest --cov=src/` en tu CI para detectar huecos de cobertura.

4) Makefile y comandos que no se pueden ejecutar

Síntomas típicos

- El README dice `make train`, pero:
- El target `train` no existe.
- O llama a rutas que no existen (`data/raw/archivo_que_no_existe.csv`).

Cómo identificarlo

- Desde la raíz del proyecto, ejecuta:
  `bash make help # si tienes target de ayuda make train`
- Observa los comandos reales que se ejecutan y compáralos con:
- La estructura de carpetas (`data/raw`, `configs/config.yaml`).
- El CLI real (como `src/bankchurn/cli.py` en BankChurn).

Cómo corregirlo

- Ajusta el `Makefile` para que:
- Use rutas reales (`data/raw/Churn.csv`, etc.).
- Delegue en el CLI real (`python main.py ...` o `python -m miproyecto.cli ...`).
- Mantén el `Makefile` como **fachada del developer experience**: pocos comandos (`install`, `test`, `train`, `serve`) pero sólidos.

5) Patrón general de debugging de estructura

1. **Revisa el árbol de directorios** contra la plantilla de este módulo.
2. **Comprueba imports** corriendo un `python -c` que importe tu paquete.
3. **Ejecuta los comandos principales** (`make install`, `make test`, `make train`, `make serve`).
4. **Asegura que datos/artefactos no están en Git** y que `.gitignore` los protege.

Este checklist de estructura es lo primero que un revisor Senior mira cuando abre un repo ML: si esto está bien, todo lo demás es mucho más fácil de mantener.

💼 Consejos Profesionales

Recomendaciones para destacar en entrevistas y proyectos reales

Para Entrevistas

1. **Explica tu estructura**: Los entrevistadores valoran que puedas justificar cada carpeta y archivo de tu proyecto.
2. **Cookiecutter es tu amigo**: Menciona que usas plantillas estandarizadas para consistencia entre proyectos.
3. **Conoce la diferencia `src/` vs flat**: Explica por qué `src/` layout previene imports accidentales del código local.

Para Proyectos Reales

Situación	Consejo
Proyecto nuevo	Usa cookiecutter-data-science o similar como base
Equipo grande	Documenta convenciones en CONTRIBUTING.md
Monorepo vs Multirepo	Monorepo para proyectos relacionados, multirepo para independientes
Configs	Nunca hardcodees: usa archivos YAML + variables de entorno

Checklist de Proyecto Profesional

- [ ] README.md con badges, instalación, y uso rápido
- [ ] pyproject.toml con metadata completa
- [ ] Makefile con comandos estándar (install, test, lint)
- [ ] .pre-commit-config.yaml para calidad automática
- [ ] tests/ con estructura que refleja src/

📺 Recursos Externos Recomendados

Ver RECURSOS_POR_MODULO.md para la lista completa.

🏷️	Recurso	Tipo
🔴	Python Project Structure - ArjanCodes	Video
🟡	src Layout - Packaging Guide	Docs

🔗 Referencias del Glosario

Ver 21_GLOSARIO.md para definiciones de:
- **src/ Layout**: Estructura de proyecto profesional
- **pyproject.toml**: Archivo de configuración moderno
- **Makefile**: Automatización de comandos

📋 Plantillas Relacionadas

Ver templates/ para plantillas listas:
- pyproject_template.toml — Configuración de paquete Python
- README_TEMPLATE.md — README profesional
- Makefile — Automatización de tareas

✅ Ejercicios

Ver EJERCICIOS.md - Módulo 03:
- 3.1: Crear estructura de proyecto
- 3.2: Configurar pyproject.toml

Ejercicio rápido:

```
mkdir -p mi-proyecto/{src/miproyecto,app,tests,configs,data/raw,artifacts,scripts,docs}
touch mi-proyecto/src/__init__.py mi-proyecto/src/miproyecto/__init__.py
touch mi-proyecto/tests/__init__.py mi-proyecto/README.md
```

ENTORNOS

GUÍA MLOPS V5.0

PORTFOLIO EDITION

MÓDULO 03: ENTORNOS PROFESIONALES

Virtualenv vs Conda vs Poetry vs Docker: Análisis Comparativo

Guía MLOps v5.0: Senior Edition | DuqueOM | Noviembre 2025

🔧 MÓDULO 03: Entornos Profesionales

El Arte de la Reproducibilidad a Nivel de Dependencias

"'Funciona en mi máquina' es la excusa más cara de la industria."

Duración	Teoría	Práctica
4 - 5 horas	30%	70%

🎯 ADR de Inicio: ¿Por Qué Importan los Entornos?

```
╔═══════════════════════════════════════════════════════════════════╗
│  ADR-005: Gestión de Entornos como Práctica Obligatoria           │
╠═══════════════════════════════════════════════════════════════════╗
│                                                                   │
│  CONTEXTO:                                                        │
│  El 30% de bugs en producción ML se deben a diferencias de versiones
│  entre desarrollo y producción (Google ML Engineering Best Practices).
│                                                                   │
│  DECISIÓN:                                                        │
│  Todo proyecto DEBE tener un sistema de gestión de dependencias con
│  versiones pinneadas y un método documentado de reproducir el entorno.
│                                                                   │
│  CONSECUENCIAS:                                                   │
│  (+) Reproducibilidad garantizada entre máquinas                  │
│  (+) Onboarding de nuevos desarrolladores en minutos, no días     │
│  (+) CI/CD confiable (mismas versiones siempre)                   │
│  (-) Setup inicial requiere más tiempo                            │
│                                                                   │
╚═══════════════════════════════════════════════════════════════════╝
```

Lo Que Lograrás en Este Módulo

1. **Entender** las diferencias entre venv, Conda, Poetry y Docker
2. **Elegir** la herramienta correcta según tu proyecto
3. **Configurar** un entorno profesional con lockfiles
4. **Integrar** el entorno con CI/CD

🧩 Cómo se aplica en este portafolio

- En **BankChurn-Predictor**, **CarVision-Market-Intelligence** y **TelecomAI-Customer-Intelligence** ya encontrarás:
- Ficheros `requirements-core.txt`, `requirements.in` y `requirements.txt` para gestionar dependencias de forma reproducible.
- Un `Makefile` con targets como `install`, `test` y `serve` que asumen un entorno activo.
- Archivos `docker-compose.demo.yml` y `docker-compose.yml` que levantan el stack completo (APIs, MLflow, dashboards).
- Usa este módulo para entender **por qué** esas piezas existen y cómo recrear el mismo entorno desde cero en tu máquina o en CI/CD.

3.1 El Problema: "Funciona en Mi Máquina"

```
╔═══════════════════════════════════════════════════════════════════╗
│                    😱 EL HORROR DE LAS DEPENDENCIAS               │
╠═══════════════════════════════════════════════════════════════════╗
│                                                                   │
│   ESCENARIO TÍPICO:                                               │
│                                                                   │
│   Developer A (laptop):                                           │
│   • Python 3.11.4                                                 │
│   • scikit-learn 1.3.0                                            │
│   • pandas 2.0.3                                                  │
```

```
│   • numpy 1.24.3                                                          │
│   → "Todo funciona perfecto" ✅                                            │
│                                                                            │
│   Developer B (otra laptop):                                               │
│   • Python 3.9.7                                                           │
│   • scikit-learn 1.0.2                                                     │
│   • pandas 1.4.0                                                           │
│   • numpy 1.21.0                                                           │
│   → "AttributeError: module 'sklearn' has no attribute 'X'" ❌             │
│                                                                            │
│   Servidor de producción:                                                  │
│   • Python 3.8.10                                                          │
│   • Versiones "whatever pip installed"                                     │
│   → CRASH EN PRODUCCIÓN 💥                                                 │
│                                                                            │
└────────────────────────────────────────────────────────────────────────────┘
```

Las 4 Capas de Reproducibilidad

```
flowchart TB
    subgraph L4["🐳 NIVEL 4: Contenedor"]
        D[Docker/Podman]
    end

    subgraph L3["📦 NIVEL 3: Gestor de Paquetes"]
        C[Poetry / pip-tools / Conda]
    end

    subgraph L2["🔒 NIVEL 2: Entorno Virtual"]
        B[venv / virtualenv / conda env]
    end

    subgraph L1["🐍 NIVEL 1: Versión Python"]
        A[pyenv / conda / asdf]
    end

    L1 --> L2 --> L3 --> L4

    style L1 fill:#ffecb3
    style L2 fill:#c8e6c9
    style L3 fill:#bbdefb
    style L4 fill:#e1bee7
```

3.2 Comparativa de Herramientas

Matriz de Decisión

Criterio	venv + pip	Conda	Poetry	Docker Dev
Simplicidad	⭐⭐⭐⭐⭐	⭐⭐⭐	⭐⭐⭐⭐	⭐⭐
Reproducibilidad	⭐⭐	⭐⭐⭐	⭐⭐⭐⭐⭐	⭐⭐⭐⭐⭐
Deps no-Python	❌	✅	❌	✅
Lockfile nativo	❌ (req pip-tools)	❌	✅	N/A
Speed	⭐⭐⭐⭐⭐	⭐⭐	⭐⭐⭐⭐	⭐⭐⭐
CI/CD friendly	✅	⚠️	✅	✅
Espacio disco	Bajo	Alto	Bajo	Medio-Alto
Curva aprendizaje	Baja	Media	Baja	Media

¿Cuándo Usar Cada Uno?

```
╔═══════════════════════════════════════════════════════════════════════════╗
│                    GUÍA DE SELECCIÓN DE HERRAMIENTA                       │
╠═══════════════════════════════════════════════════════════════════════════╗
│                                                                           │
```

```
│  USA venv + pip-tools SI:                                                  │
│  • Proyecto simple, solo dependencias Python                               │
│  • Equipo pequeño, ya conoce pip                                           │
│  • CI/CD en GitHub Actions (pip es más rápido)                             │
│  • No necesitas lockfile sofisticado                                       │
│                                                                            │
│  USA Conda SI:                                                             │
│  • Necesitas librerías con dependencias C/C++ (CUDA, MKL, OpenCV)          │
│  • Trabajas en Data Science pesado (numpy, scipy optimizados)              │
│  • Tu equipo ya usa Conda                                                  │
│  • Necesitas múltiples versiones de Python fácilmente                      │
│                                                                            │
│  USA Poetry SI:                                                            │
│  • Proyecto serio que necesita reproducibilidad exacta                     │
│  • Quieres publicar en PyPI                                                │
│  • Valoras lockfiles y dependency resolution robusta                       │
│  • Equipo moderno que aprecia herramientas bien diseñadas                  │
│                                                                            │
│  USA Docker Dev Containers SI:                                             │
│  • Reproducibilidad TOTAL es crítica                                       │
│  • Múltiples servicios (DB, Redis, etc.) en desarrollo                     │
│  • Onboarding debe ser "clone & run"                                       │
│  • Equipo usa VS Code con extensión Dev Containers                         │
│                                                                            │
└────────────────────────────────────────────────────────────────────────────┘
```

3.3 Opción 1: venv + pip-tools (Simple y Efectivo)

Setup Básico

```
# Crear entorno virtual
python3.11 -m venv .venv

# Activar
source .venv/bin/activate  # Linux/Mac
# .venv\Scripts\activate   # Windows

# Instalar pip-tools para lockfiles
pip install pip-tools
```

Estructura de Archivos

```
project/
├── requirements.in        # Dependencias directas (lo que escribes)
├── requirements.txt       # Lockfile generado (no editar manualmente)
├── requirements-dev.in    # Dependencias de desarrollo
├── requirements-dev.txt   # Lockfile de desarrollo
└── .python-version        # Versión de Python (para pyenv)
```

requirements.in (Lo que escribes)

```
# requirements.in - Dependencias directas
# Solo especifica las que usas directamente, pip-tools resuelve el resto

pandas>=2.0.0,<3.0.0
scikit-learn>=1.3.0
pydantic>=2.0.0
fastapi>=0.100.0
mlflow>=2.8.0
pyyaml>=6.0
```

Generar Lockfile

```
# Genera requirements.txt con TODAS las versiones exactas
pip-compile requirements.in --output-file=requirements.txt

# Para desarrollo
pip-compile requirements-dev.in --output-file=requirements-dev.txt

# Instalar desde lockfile
pip-sync requirements.txt requirements-dev.txt
```

requirements.txt Generado (NO EDITAR)

```
# This file is autogenerated by pip-compile with Python 3.11
# Do not edit manually.

annotated-types==0.6.0
    # via pydantic
anyio==4.0.0
    # via
    #   httpx
    #   starlette
certifi==2023.11.17
    # via httpx
fastapi==0.104.1
    # via -r requirements.in
numpy==1.26.2
    # via
    #   pandas
    #   scikit-learn
pandas==2.1.3
    # via -r requirements.in
pydantic==2.5.2
    # via
    #   -r requirements.in
    #   fastapi
# ... etc (versiones EXACTAS)
```

Makefile para Automatización

```
# Makefile
.PHONY: venv install lock sync clean

PYTHON := python3.11
VENV := .venv
BIN := $(VENV)/bin

venv:
    $(PYTHON) -m venv $(VENV)
    $(BIN)/pip install --upgrade pip pip-tools

lock: venv
    $(BIN)/pip-compile requirements.in -o requirements.txt
    $(BIN)/pip-compile requirements-dev.in -o requirements-dev.txt

sync: venv
    $(BIN)/pip-sync requirements.txt requirements-dev.txt

install: venv lock sync

clean:
    rm -rf $(VENV)
    rm -f requirements.txt requirements-dev.txt
```

3.4 Opción 2: Poetry (Moderno y Robusto)

Instalación

```
# Instalar Poetry (método oficial)
curl -sSL https://install.python-poetry.org | python3 -

# Verificar
poetry --version
```

Inicializar Proyecto

```
# En proyecto existente
poetry init

# O crear nuevo proyecto
poetry new bankchurn-predictor
```

pyproject.toml Completo

```
[tool.poetry]
name = "bankchurn"
version = "0.1.0"
description = "Predictor de churn bancario con MLOps"
authors = ["Tu Nombre <tu@email.com>"]
readme = "README.md"
packages = [{include = "bankchurn", from = "src"}]

[tool.poetry.dependencies]
python = "^3.10"
pandas = "^2.0.0"
scikit-learn = "^1.3.0"
pydantic = "^2.0.0"
fastapi = "^0.104.0"
uvicorn = "^0.24.0"
mlflow = "^2.8.0"
pyyaml = "^6.0"
joblib = "^1.3.0"

[tool.poetry.group.dev.dependencies]
pytest = "^7.4.0"
pytest-cov = "^4.1.0"
mypy = "^1.6.0"
ruff = "^0.1.0"
pre-commit = "^3.5.0"
ipython = "^8.0.0"

[tool.poetry.group.docs.dependencies]
mkdocs = "^1.5.0"
mkdocs-material = "^9.4.0"

[tool.poetry.scripts]
bankchurn-train = "bankchurn.cli:train"
bankchurn-predict = "bankchurn.cli:predict"

[build-system]
requires = ["poetry-core"]
build-backend = "poetry.core.masonry.api"

# ═══════════════════════════════════════════════════════════════════
# CONFIGURACIÓN DE HERRAMIENTAS
# ═══════════════════════════════════════════════════════════════════

[tool.ruff]
line-length = 100
select = ["E", "F", "I", "W", "B", "C4", "UP"]
ignore = ["E501"]
src = ["src"]

[tool.mypy]
python_version = "3.11"
warn_return_any = true
disallow_untyped_defs = true
ignore_missing_imports = true

[tool.pytest.ini_options]
testpaths = ["tests"]
addopts = "-v --cov=src/bankchurn --cov-report=term-missing"

[tool.coverage.run]
source = ["src"]
omit = ["tests/*"]
```

Comandos Esenciales

```
# Instalar dependencias (crea poetry.lock automáticamente)
poetry install

# Añadir dependencia
poetry add pandas
poetry add --group dev pytest
```

```
# Actualizar dependencias
poetry update

# Ejecutar comando en el entorno
poetry run python src/bankchurn/main.py
poetry run pytest

# Activar shell en el entorno
poetry shell

# Exportar a requirements.txt (para Docker)
poetry export -f requirements.txt --output requirements.txt --without-hashes

# Build del paquete
poetry build
```

poetry.lock (Generado Automáticamente)

El archivo `poetry.lock` contiene TODAS las versiones exactas de TODAS las dependencias (directas y transitivas). **SIEMPRE** commitear este archivo.

3.5 Opción 3: Conda (Para Data Science Pesado)

Cuándo Conda es la Mejor Opción

```
╔═══════════════════════════════════════════════════════════════════╗
║                        ✅ USA CONDA SI NECESITAS:                 ║
╟═══════════════════════════════════════════════════════════════════╢
║                                                                   ║
║   • CUDA / cuDNN para GPU computing                               ║
║   • NumPy/SciPy compilados con MKL (Intel) o OpenBLAS optimizado  ║
║   • OpenCV con dependencias de sistema                            ║
║   • R + Python en el mismo entorno                                 ║
║   • Librerías geoespaciales (GDAL, GEOS, PROJ)                     ║
║   • Dependencias de sistema que pip no puede instalar             ║
║                                                                   ║
╚═══════════════════════════════════════════════════════════════════╝
```

environment.yml

```
# environment.yml
name: bankchurn
channels:
  - conda-forge  # Preferir conda-forge sobre defaults
  - defaults

dependencies:
  # Python version
  - python=3.11

  # Core data science (optimizados con MKL)
  - numpy=1.26.*
  - pandas=2.1.*
  - scikit-learn=1.3.*

  # Si necesitas GPU
  # - pytorch
  # - cudatoolkit=11.8

  # Dependencias que tienen componentes C
  - pyyaml
  - joblib

  # pip dependencies (las que no están en conda o prefieres de PyPI)
  - pip
  - pip:
    - pydantic>=2.0.0
    - fastapi>=0.104.0
    - uvicorn>=0.24.0
    - mlflow>=2.8.0
    - pytest>=7.4.0
    - mypy>=1.6.0
    - ruff>=0.1.0
```

Comandos Conda

```
# Crear entorno desde archivo
conda env create -f environment.yml

# Activar
conda activate bankchurn

# Exportar entorno exacto (para reproducibilidad)
conda env export > environment-lock.yml

# Exportar solo dependencias explícitas
conda env export --from-history > environment.yml

# Actualizar entorno
conda env update -f environment.yml --prune

# Listar entornos
conda env list

# Eliminar entorno
conda env remove -n bankchurn
```

Mamba: Conda Acelerado

```
# Instalar mamba (resolver mucho más rápido)
conda install -c conda-forge mamba

# Usar mamba en lugar de conda
mamba env create -f environment.yml
mamba install numpy
```

3.6 Opción 4: Docker Dev Containers

¿Por Qué Docker para Desarrollo?

```
╔═══════════════════════════════════════════════════════════════════╗
║                      DOCKER DEV CONTAINERS: PROS/CONS             ║
╠═══════════════════════════════════════════════════════════════════╣
║                                                                   ║
║   ✅ PROS:                                                        ║
║   • Reproducibilidad TOTAL (mismo OS, mismas versiones de todo)   ║
║   • Onboarding = "git clone && code ." (con VS Code Dev Containers) ║
║   • Mismo entorno en dev, CI y producción                         ║
║   • Puedes incluir servicios (PostgreSQL, Redis, MLflow server)   ║
║                                                                   ║
║   ❌ CONS:                                                        ║
║   • Overhead de Docker (memoria, CPU)                             ║
║   • Más complejo de configurar inicialmente                       ║
║   • Debugging puede ser más difícil                               ║
║   • Performance de I/O en volúmenes (especialmente macOS)         ║
║                                                                   ║
╚═══════════════════════════════════════════════════════════════════╝
```

.devcontainer/devcontainer.json

```
{
    "name": "BankChurn Dev",
    "dockerComposeFile": "docker-compose.yml",
    "service": "app",
    "workspaceFolder": "/workspace",

    "customizations": {
        "vscode": {
            "extensions": [
                "ms-python.python",
                "ms-python.vscode-pylance",
                "charliermarsh.ruff",
                "ms-toolsai.jupyter",
                "redhat.vscode-yaml",
                "GitHub.copilot"
            ],
```

```
            "settings": {
                "python.defaultInterpreterPath": "/workspace/.venv/bin/python",
                "python.formatting.provider": "none",
                "editor.formatOnSave": true,
                "[python]": {
                    "editor.defaultFormatter": "charliermarsh.ruff"
                }
            }
        }
    },

    "postCreateCommand": "make install",

    "forwardPorts": [8000, 5000, 3000],

    "remoteUser": "vscode"
}
```

.devcontainer/docker-compose.yml

```
version: '3.8'

services:
  app:
    build:
      context: ..
      dockerfile: .devcontainer/Dockerfile
    volumes:
      - ..:/workspace:cached
      - venv:/workspace/.venv
    environment:
      - PYTHONDONTWRITEBYTECODE=1
      - PYTHONUNBUFFERED=1
    command: sleep infinity

  mlflow:
    image: ghcr.io/mlflow/mlflow:v2.8.0
    ports:
      - "5000:5000"
    volumes:
      - mlflow-data:/mlflow
    command: mlflow server --host 0.0.0.0 --backend-store-uri sqlite:///mlflow/mlflow.db --default-artifact-root /
mlflow/artifacts

  postgres:
    image: postgres:15
    environment:
      POSTGRES_USER: bankchurn
      POSTGRES_PASSWORD: bankchurn
      POSTGRES_DB: bankchurn
    volumes:
      - postgres-data:/var/lib/postgresql/data
    ports:
      - "5432:5432"

volumes:
  venv:
  mlflow-data:
  postgres-data:
```

.devcontainer/Dockerfile

```
FROM python:3.11-slim

# Dependencias del sistema
RUN apt-get update && apt-get install -y --no-install-recommends \
    git \
    curl \
    make \
    && rm -rf /var/lib/apt/lists/*

# Usuario no-root
ARG USERNAME=vscode
```

```
ARG USER_UID=1000
ARG USER_GID=$USER_UID

RUN groupadd --gid $USER_GID $USERNAME \
    && useradd --uid $USER_UID --gid $USER_GID -m $USERNAME

# Workspace
WORKDIR /workspace

# Cambiar a usuario no-root
USER $USERNAME

# Pre-instalar pip-tools
RUN pip install --user pip-tools

ENV PATH="/home/${USERNAME}/.local/bin:${PATH}"
```

3.7 Integración con CI/CD

GitHub Actions con pip

```
# .github/workflows/ci.yml
name: CI

on: [push, pull_request]

jobs:
  test:
    runs-on: ubuntu-latest

    steps:
      - uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: '3.11'
          cache: 'pip'  # Cachea dependencias

      - name: Install dependencies
        run: |
          pip install -r requirements.txt
          pip install -r requirements-dev.txt

      - name: Run tests
        run: pytest tests/ -v --cov
```

GitHub Actions con Poetry

```
name: CI

on: [push, pull_request]

jobs:
  test:
    runs-on: ubuntu-latest

    steps:
      - uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: '3.11'

      - name: Install Poetry
        uses: snok/install-poetry@v1
        with:
          version: 1.7.0
          virtualenvs-create: true
          virtualenvs-in-project: true

      - name: Load cached venv
        uses: actions/cache@v3
```

```
        with:
          path: .venv
          key: venv-${{ runner.os }}-${{ hashFiles('poetry.lock') }}

      - name: Install dependencies
        run: poetry install --no-interaction

      - name: Run tests
        run: poetry run pytest tests/ -v --cov
```

🧨 Errores habituales y cómo depurarlos en entornos

Los problemas de este módulo se manifiestan como **inconsistencias entre máquinas**: algo funciona en tu laptop pero no en el servidor, o en CI. Aquí van los patrones más frecuentes y cómo atacarlos.

1) Entorno virtual mal activado (`pip` instala en el sitio equivocado)

Síntomas típicos

- Ejecutas `pip install` y luego `python -c "import pandas"` y obtienes `ModuleNotFoundError`.
- Tienes varias versiones de Python (`python`, `python3`, `pyenv`, Conda) y no sabes cuál está usando tu proyecto.
- En CI funciona con una versión de paquete y en local con otra.

Cómo identificarlo

- Ejecuta:
  `bash which python python -m pip --version`
  y verifica que ambos apuntan al **mismo entorno** (`.venv/bin/python`, por ejemplo).
- En Windows, revisa la ruta de `Scripts` y que estés en el entorno correcto (`(.venv)` en el prompt).

Cómo corregirlo

- Usa siempre `python -m pip` en lugar de `pip` a secas:
  `bash python -m pip install -r requirements.txt`
- Documenta en el README/Makefile **cómo activar el entorno** (`source .venv/bin/activate`, `poetry shell`, `conda activate ...`).
- Si usas `.python-version` con `pyenv`, asegúrate de que coincide con la versión definida en `pyproject.toml` o `environment.yml`.

2) Lockfiles ignorados (`requirements.txt` / `poetry.lock` / `environment-lock.yml`)

Síntomas típicos

- Dos personas hacen `pip install -r requirements.txt` y obtienen versiones distintas de las mismas librerías.
- En tu máquina funciona con `pandas==2.0.3` pero en producción falla con `pandas==2.2.0`.
- `poetry.lock` o `requirements-dev.txt` no están commiteados.

Cómo identificarlo

- Revisa el repositorio:
- ¿Existe `requirements.txt` generado por pip-tools y está en Git?
- ¿Existe `poetry.lock` y está versionado?
- ¿Hay algún `environment-lock.yml` de Conda?
- Compara lo que dice el lockfile con lo que tienes instalado:
  `bash pip freeze | grep pandas`

Cómo corregirlo

- **Siempre** commitea el lockfile (requirements.txt, poetry.lock, environment-lock.yml).
- Define una única fuente de verdad: si usas pip-tools, no edites `requirements.txt` a mano, solo `requirements.in`.
- En CI, instala **desde el lockfile**, no desde las dependencias sueltas.

3) Mezclar gestores (pip + Conda + Poetry + Docker) sin una estrategia clara

Síntomas típicos

- Instalas algo con `conda install` y luego con `pip install` y el entorno se rompe.
- Tienes `environment.yml`, `requirements.txt` y `pyproject.toml` en el mismo proyecto sin que ninguno esté claro.
- El contenedor Docker instala versiones diferentes a las de tu entorno local.

Cómo identificarlo

- Lista tus archivos de configuración: ¿hay más de un gestor activo a la vez?

- Revisa el `Dockerfile`: ¿instala desde `requirements.txt`, desde `pyproject.toml` exportado o desde `environment.yml`?

Cómo corregirlo

- Elige un **flujo principal** por proyecto:
- pip-tools → `requirements.in` → `requirements.txt` → Docker/CI.
- Poetry → `pyproject.toml` + `poetry.lock` → export a `requirements.txt` solo para Docker.
- Conda → `environment.yml`/`environment-lock.yml` → `conda env create` en todas partes.
- Documenta en este módulo (y en el README del proyecto) **qué gestor es el canónico** y qué archivos deben editarse.

4) CI instala un entorno distinto al local

Síntomas típicos

- En local todo pasa, pero en GitHub Actions los tests fallan por versiones de librerías.
- Ves que en CI se instala directamente con `pip install -r requirements.txt` pero en local usas Poetry o Conda.

Cómo identificarlo

- Abre el workflow (`.github/workflows/*.yml`) y verifica:
- ¿Está usando la misma versión de Python que tú?
- ¿Instala dependencias desde los mismos archivos (`requirements.txt`, `poetry.lock`, `environment.yml`)?

Cómo corregirlo

- Alinea CI con tu flujo local:
- pip-tools: usa el snippet de "GitHub Actions con pip" de este módulo.
- Poetry: usa el bloque de "GitHub Actions con Poetry" y cachea `.venv`.
- Conda: usa `conda env create -f environment.yml` o `mamba`.
- Haz al menos una vez la prueba de **clonar en limpio** y seguir los pasos de CI en tu máquina; esto detecta diferencias.

5) Docker que no refleja el entorno real

Síntomas típicos

- La aplicación en Docker falla con `ImportError` o con versiones diferentes de librerías.
- Tu `Dockerfile` instala con `pip install pandas scikit-learn ...` en lugar de usar el lockfile.

Cómo identificarlo

- Revisa el `Dockerfile` (y `.devcontainer/Dockerfile` si aplica):
- ¿Copia `requirements.txt` o usa `poetry export` antes de instalar?
- ¿Especifica la misma versión de Python que usas localmente?

Cómo corregirlo

- Haz que Docker **derive** de tu configuración de entorno:
- Con pip-tools: `COPY requirements.txt` → `pip install -r requirements.txt`.
- Con Poetry: `poetry export -f requirements.txt --output requirements.txt` y usa eso en la imagen.
- Mantén la versión de Python del contenedor alineada con tu `.python-version` / `pyproject.toml` / `environment.yml`.

Patrón general de debugging de entornos

1. **Congela la versión de Python**: pyenv, Conda o imagen base de Docker clara.
2. **Define un único gestor principal** (pip-tools, Poetry o Conda) y sigue su flujo.
3. **Asegúrate de que CI y Docker consumen el mismo lockfile**.
4. **Verifica el entorno activado** antes de instalar o ejecutar (`which python`, `python -m pip`).

Con este patrón, "funciona en mi máquina" se convierte en "funciona en cualquier máquina que siga estos pasos".

3.8 Ejercicio Práctico: Configura Tu Entorno

Opción A: pip-tools (Recomendado para empezar)

```
# 1. Crear estructura
mkdir -p bankchurn-predictor && cd bankchurn-predictor

# 2. Crear archivos
cat > requirements.in << 'EOF'
pandas>=2.0.0
```

```
scikit-learn>=1.3.0
pydantic>=2.0.0
fastapi>=0.104.0
uvicorn>=0.24.0
mlflow>=2.8.0
pyyaml>=6.0
joblib>=1.3.0
EOF

cat > requirements-dev.in << 'EOF'
-r requirements.in
pytest>=7.4.0
pytest-cov>=4.1.0
mypy>=1.6.0
ruff>=0.1.0
pre-commit>=3.5.0
EOF

# 3. Crear entorno y lockfiles
python3.11 -m venv .venv
source .venv/bin/activate
pip install pip-tools
pip-compile requirements.in
pip-compile requirements-dev.in
pip-sync requirements.txt requirements-dev.txt

# 4. Verificar
python -c "import pandas; print(pandas.__version__)"
```

Opción B: Poetry

```
# 1. Crear proyecto
poetry new bankchurn-predictor --src
cd bankchurn-predictor

# 2. Añadir dependencias
poetry add pandas scikit-learn pydantic fastapi uvicorn mlflow pyyaml joblib
poetry add --group dev pytest pytest-cov mypy ruff pre-commit

# 3. Instalar
poetry install

# 4. Verificar
poetry run python -c "import pandas; print(pandas.__version__)"
```

Checklist de Verificación

```
[ ] Entorno virtual creado y activable
[ ] Lockfile generado con versiones exactas
[ ] Lockfile commiteado en Git
[ ] Puedo recrear el entorno desde cero
[ ] CI puede instalar las mismas versiones
```

3.9 Autoevaluación

Checklist de Competencias

```
CONCEPTOS:
[ ] Entiendo la diferencia entre dependencias directas y transitivas
[ ] Sé qué es un lockfile y por qué es importante
[ ] Puedo explicar cuándo usar Conda vs pip vs Poetry

pip-tools:
[ ] Puedo crear requirements.in con restricciones de versión
[ ] Sé usar pip-compile y pip-sync
[ ] Entiendo el formato del lockfile generado

Poetry:
[ ] Puedo crear un pyproject.toml funcional
[ ] Sé añadir dependencias y grupos de dependencias
[ ] Puedo exportar a requirements.txt para Docker

CI/CD:
```

```
[ ] Puedo configurar caching de dependencias en GitHub Actions
[ ] Sé cómo asegurar reproducibilidad en CI
```

Preguntas de Reflexión

1. ¿Por qué no basta con `pip install pandas` sin especificar versión?
2. ¿Cuál es la diferencia entre `requirements.in` y `requirements.txt`?
3. ¿Cuándo preferirías Conda sobre Poetry?
4. ¿Por qué es importante cachear dependencias en CI?

📦 Cómo se Usó en el Portafolio

Cada proyecto del portafolio implementa la gestión de entornos descrita:

pyproject.toml Real

```
# BankChurn-Predictor/pyproject.toml (extracto)
[project]
name = "bankchurn"
version = "0.1.0"
requires-python = ">=3.10"
dependencies = [
    "pandas>=2.0.0",
    "scikit-learn>=1.3.0",
    "pydantic>=2.5.0",
    "mlflow>=2.9.0",
]

[project.optional-dependencies]
dev = [
    "pytest>=7.4.0",
    "pytest-cov>=4.1.0",
    "ruff>=0.1.9",
]
```

Comandos Make del Portafolio

Todos los proyectos tienen Makefile con comandos consistentes:

```
# Comandos disponibles en los 3 proyectos
make install     # pip install -e ".[dev]"
make test        # pytest con coverage
make lint        # ruff check
make train       # Entrena el modelo
make serve       # Inicia API FastAPI
```

Estructura de Dependencias

Proyecto	Archivo	Dependencias Core
BankChurn	`pyproject.toml`	pandas, sklearn, pydantic, mlflow
CarVision	`pyproject.toml`	pandas, sklearn, pydantic, pyyaml
TelecomAI	`pyproject.toml`	pandas, sklearn, pydantic

🔧 Ejercicio: Instala un Proyecto Real

```
# 1. Ve a BankChurn
cd BankChurn-Predictor

# 2. Crea entorno virtual
python -m venv .venv
source .venv/bin/activate  # Linux/Mac
# .venv\Scripts\activate   # Windows

# 3. Instala con dependencias de desarrollo
pip install -e ".[dev]"

# 4. Verifica que funciona
```

```
python -c "from bankchurn.config import BankChurnConfig; print('OK')"
make test
```

💼 Consejos Profesionales

Recomendaciones para destacar en entrevistas y proyectos reales

Para Entrevistas

1. **"¿Cómo manejas dependencias?"**: Explica pip-tools, Poetry, o uv. Menciona lock files y reproducibilidad.
2. **Containers vs Virtualenvs**: Conoce cuándo usar cada uno (dev local vs producción).
3. **DevContainers**: Menciona que usas VS Code DevContainers para entornos reproducibles.

Para Proyectos Reales

Situación	Consejo
Conflictos de dependencias	Usa pip-compile para resolver y fijar versiones
CI/CD	Usa la misma imagen Docker en local y CI
Múltiples versiones de Python	pyenv + tox para testing multi-versión
Dependencias de sistema	Documenta en Dockerfile o README

Herramientas Modernas

- **uv**: Reemplazo rápido de pip (10-100x más rápido)
- **pip-tools**: pip-compile + pip-sync para reproducibilidad
- **Poetry**: Gestión completa de dependencias y publicación
- **Conda**: Para dependencias científicas complejas (CUDA, etc.)

📺 Recursos Externos Recomendados

Ver RECURSOS_POR_MODULO.md para la lista completa.

🏷️	Recurso	Tipo
🔴	Python Virtual Environments - Corey Schafer	Video
🟡	pip-tools Tutorial	Video

🔗 Referencias del Glosario

Ver 21_GLOSARIO.md para definiciones de:
- **venv**: Entornos virtuales de Python
- **pip-tools**: Gestión de dependencias
- **pyproject.toml**: Configuración de proyecto moderno

✅ Ejercicios

Ver EJERCICIOS.md - Módulo 04:
- 4.1: Crear entorno virtual
- 4.2: Configurar pip-tools

➡️ Siguiente Paso

Con el entorno configurado, es hora de dominar **Git profesionalmente**.

Ir a Módulo 05: Git Profesional →

GIT PROFESIONAL

GUÍA MLOPS V5.0

PORTFOLIO EDITION

MÓDULO 04: GIT PROFESIONAL

Más Allá del Commit: Conventional Commits, Hooks y Branching

Guía MLOps v5.0: Senior Edition | DuqueOM | Noviembre 2025

🌿 MÓDULO 04: Git Profesional

Control de Versiones que Impresiona en Code Review

"Un historial de Git limpio es la documentación que nunca miente."

Duración	Teoría	Práctica
4 - 5 horas	25%	75%

🎯 Lo Que Lograrás en Este Módulo

1. **Escribir** commits que cuentan una historia clara
2. **Configurar** pre-commit hooks que previenen errores
3. **Aplicar** estrategias de branching profesionales
4. **Dominar** comandos avanzados (rebase, cherry-pick, bisect)

🧩 Cómo se aplica en este portafolio

- El repositorio `ML-MLOps-Portfolio` y los tres proyectos (`BankChurn-Predictor`, `CarVision-Market-Intelligence`, `TelecomAI-Customer-Intelligence`) ya usan:
- Historial basado en **Conventional Commits**.
- Hooks de **pre-commit** configurados en `.pre-commit-config.yaml`.
- Escaneo de seguridad con **Gitleaks** vía `.gitleaks.toml` y workflows de CI.
- Usa este módulo como guía para entender y ajustar esos flujos en tu propio fork del portafolio y para mantener un historial que soporte entrevistas técnicas Senior/Staff.

4.1 Conventional Commits: El Estándar de Industria

¿Por Qué Importa el Formato del Commit?

```
╔═══════════════════════════════════════════════════════════════════╗
║                   ❌ HISTORIAL TÍPICO (CAÓTICO)                   ║
╠═══════════════════════════════════════════════════════════════════╣
║                                                                   ║
║   * fix                                                           ║
║   * wip                                                           ║
║   * más cambios                                                   ║
║   * asdfgh                                                        ║
║   * funcionaaaa                                                   ║
║   * ahora sí                                                      ║
║   * merge conflict resuelto                                       ║
║   * updates                                                       ║
║                                                                   ║
║   PROBLEMAS:                                                      ║
║   • Imposible saber qué cambió sin leer el código                 ║
║   • No puedes generar changelog automático                        ║
║   • git bisect es inútil                                          ║
║   • Code review es un infierno                                    ║
║                                                                   ║
╠═══════════════════════════════════════════════════════════════════╣
║                   ✅ HISTORIAL PROFESIONAL (CONVENTIONAL)         ║
╠═══════════════════════════════════════════════════════════════════╣
║                                                                   ║
║   * feat(api): add /predict endpoint with batch support           ║
║   * fix(training): handle NaN values in CreditScore column        ║
║   * test(pipeline): add integration tests for full pipeline       ║
║   * docs(readme): update installation instructions                ║
║   * refactor(config): migrate from dict to Pydantic models        ║
║   * ci(actions): add caching for pip dependencies                 ║
║   * perf(inference): reduce latency from 150ms to 45ms            ║
║                                                                   ║
║   BENEFICIOS:                                                     ║
```

```
│   • Changelog generado automáticamente                                    │
│   • Semantic versioning automático                                         │
│   • git bisect encuentra bugs rápidamente                                  │
│   • Code review enfocado                                                   │
│                                                                            │
└────────────────────────────────────────────────────────────────────────────┘
```

Anatomía de un Conventional Commit

```
<type>(<scope>): <description>

[optional body]

[optional footer(s)]
```

Tipos Permitidos

Tipo	Cuándo Usar	Ejemplo
`feat`	Nueva funcionalidad	`feat(api): add batch prediction endpoint`
`fix`	Corrección de bug	`fix(training): handle missing values in Age`
`docs`	Solo documentación	`docs(readme): add API usage examples`
`style`	Formato (no afecta lógica)	`style: apply ruff formatting`
`refactor`	Refactor sin cambio funcional	`refactor(config): use Pydantic BaseSettings`
`test`	Añadir o corregir tests	`test(inference): add unit tests for predictor`
`perf`	Mejora de performance	`perf(pipeline): cache preprocessor transformations`
`ci`	Cambios en CI/CD	`ci(actions): add Python 3.12 to test matrix`
`build`	Cambios en build/deps	`build(deps): upgrade scikit-learn to 1.4.0`
`chore`	Mantenimiento general	`chore: update .gitignore`

Scopes Comunes en MLOps

```
# Por componente
feat(training): ...
feat(inference): ...
feat(api): ...
feat(config): ...
feat(data): ...

# Por capa
feat(model): ...
feat(features): ...
feat(pipeline): ...

# Por herramienta
ci(actions): ...
ci(docker): ...
ci(dvc): ...
```

Ejemplos Completos

```
# Simple
git commit -m "feat(api): add health check endpoint"

# Con body explicativo
git commit -m "fix(training): handle class imbalance in target variable

The training was failing silently when class ratio exceeded 1:10.
Added class_weight='balanced' to RandomForestClassifier.

Fixes #123"
```

```
# Breaking change (incrementa MAJOR version)
git commit -m "feat(api)!: change response format to include confidence scores

BREAKING CHANGE: The /predict response now returns an object instead of
a single float. Clients must update to handle the new format:
{\"probability\": 0.85, \"confidence\": 0.92, \"prediction\": \"churn\"}"
```

Configurar Commitlint (Validación Automática)

```
# Instalar commitlint
npm install -g @commitlint/cli @commitlint/config-conventional

# Crear config
cat > commitlint.config.js << 'EOF'
module.exports = {
  extends: ['@commitlint/config-conventional'],
  rules: {
    'scope-enum': [2, 'always', [
      'api', 'training', 'inference', 'config', 'data',
      'pipeline', 'model', 'features', 'tests', 'docs',
      'ci', 'docker', 'dvc', 'deps'
    ]],
    'subject-case': [2, 'always', 'lower-case'],
  }
};
EOF
```

4.2 Pre-commit Hooks: Prevenir Errores Antes del Commit

¿Qué Son los Pre-commit Hooks?

```
╔═══════════════════════════════════════════════════════════════════╗
║                      PRE-COMMIT: EL GUARDIÁN                      ║
╠═══════════════════════════════════════════════════════════════════╣
║                                                                   ║
║                       git commit -m "feat: ..."                   ║
║                                   │                               ║
║                                   ▼                               ║
║                     ┌─────────────────────────────┐               ║
║                     │      PRE-COMMIT HOOKS       │               ║
║                     │                             │               ║
║                     │  ✓ Formatear código (ruff)  │               ║
║                     │  ✓ Lint (ruff check)        │               ║
║                     │  ✓ Type check (mypy)        │               ║
║                     │  ✓ Tests rápidos            │               ║
║                     │  ✓ Validar YAML/JSON        │               ║
║                     │  ✓ Detectar secretos        │               ║
║                     │  ✓ Validar commit message   │               ║
║                     │                             │               ║
║                     └─────────────────────────────┘               ║
║                                   │                               ║
║                          ┌────────┴────────┐                      ║
║                          ▼                 ▼                      ║
║                     ALL PASS ✅          ANY FAIL ❌               ║
║                     Commit OK            Commit BLOCKED            ║
║                                                                   ║
╚═══════════════════════════════════════════════════════════════════╝
```

Instalación y Setup

```
# Instalar pre-commit
pip install pre-commit

# Instalar hooks en el repo
pre-commit install
pre-commit install --hook-type commit-msg  # Para commitlint

# Ejecutar en todos los archivos (primera vez)
pre-commit run --all-files
```

.pre-commit-config.yaml Completo

```yaml
# .pre-commit-config.yaml
repos:
  # ═══════════════════════════════════════════════════════════════════════
  # FORMATEO Y LINTING
  # ═══════════════════════════════════════════════════════════════════════
  - repo: https://github.com/astral-sh/ruff-pre-commit
    rev: v0.1.6
    hooks:
      - id: ruff
        args: [--fix, --exit-non-zero-on-fix]
      - id: ruff-format

  # ═══════════════════════════════════════════════════════════════════════
  # TYPE CHECKING
  # ═══════════════════════════════════════════════════════════════════════
  - repo: https://github.com/pre-commit/mirrors-mypy
    rev: v1.7.0
    hooks:
      - id: mypy
        args: [--ignore-missing-imports]
        additional_dependencies:
          - pydantic>=2.0.0
          - types-PyYAML

  # ═══════════════════════════════════════════════════════════════════════
  # GENERAL
  # ═══════════════════════════════════════════════════════════════════════
  - repo: https://github.com/pre-commit/pre-commit-hooks
    rev: v4.5.0
    hooks:
      - id: trailing-whitespace
      - id: end-of-file-fixer
      - id: check-yaml
        args: [--unsafe]  # Para YAML con tags como !ref
      - id: check-json
      - id: check-toml
      - id: check-added-large-files
        args: [--maxkb=1000]
      - id: check-merge-conflict
      - id: detect-private-key
      - id: no-commit-to-branch
        args: [--branch, main, --branch, master]

  # ═══════════════════════════════════════════════════════════════════════
  # SEGURIDAD
  # ═══════════════════════════════════════════════════════════════════════
  - repo: https://github.com/Yelp/detect-secrets
    rev: v1.4.0
    hooks:
      - id: detect-secrets
        args: [--baseline, .secrets.baseline]

  - repo: https://github.com/PyCQA/bandit
    rev: 1.7.5
    hooks:
      - id: bandit
        args: [-c, pyproject.toml]
        additional_dependencies: ["bandit[toml]"]

  # ═══════════════════════════════════════════════════════════════════════
  # CONVENTIONAL COMMITS
  # ═══════════════════════════════════════════════════════════════════════
  - repo: https://github.com/compilerla/conventional-pre-commit
    rev: v3.0.0
    hooks:
      - id: conventional-pre-commit
        stages: [commit-msg]
        args: [feat, fix, docs, style, refactor, test, perf, ci, build, chore]

  # ═══════════════════════════════════════════════════════════════════════
  # JUPYTER NOTEBOOKS
  # ═══════════════════════════════════════════════════════════════════════
  - repo: https://github.com/kynan/nbstripout
```

```
    rev: 0.6.1
    hooks:
      - id: nbstripout  # Limpia outputs de notebooks

  # ═══════════════════════════════════════════════════════════════════
  # DOCKER
  # ═══════════════════════════════════════════════════════════════════
  - repo: https://github.com/hadolint/hadolint
    rev: v2.12.0
    hooks:
      - id: hadolint-docker
        args: [--ignore, DL3008, --ignore, DL3013]

# Configuración global
default_language_version:
  python: python3.11

ci:
  autofix_commit_msg: "style: auto-fix by pre-commit hooks"
  autoupdate_commit_msg: "chore: update pre-commit hooks"
```

pyproject.toml Sección Bandit

```
# pyproject.toml
[tool.bandit]
exclude_dirs = ["tests", "scripts"]
skips = ["B101"]  # Skip assert warnings in tests
```

Comandos Pre-commit Útiles

```
# Ejecutar en archivos staged
pre-commit run

# Ejecutar en todos los archivos
pre-commit run --all-files

# Ejecutar hook específico
pre-commit run ruff --all-files
pre-commit run mypy --all-files

# Actualizar hooks a últimas versiones
pre-commit autoupdate

# Skip hooks temporalmente (emergencia)
git commit --no-verify -m "hotfix: emergency fix"
# ⚠️ USAR SOLO EN EMERGENCIAS
```

4.3 Estrategias de Branching

Git Flow vs GitHub Flow vs Trunk-Based

```
╔═══════════════════════════════════════════════════════════════════════╗
║                        COMPARATIVA DE ESTRATEGIAS                     ║
╠═══════════════════════════════════════════════════════════════════════╣
║                                                                       ║
║  GIT FLOW (Complejo, releases programados)                            ║
║  ═══════════════════════════════════════                              ║
║  main ──●──────────────────●──────────────●──                        ║
║          \                 / \            /                         ║
║  develop  ●─●─●─●─●─●─●   ●─●─●─●─●                                  ║
║              \     /               /                                 ║
║  feature      ●─●                 /                                  ║
║                   \             /                                    ║
║  release            ●─●─●──────●                                     ║
║                                                                       ║
║  ✅ Para: Apps con releases programados, equipos grandes              ║
║  ❌ No para: MLOps (demasiado overhead), startups                     ║
║                                                                       ║
╠═══════════════════════════════════════════════════════════════════════╣
║                                                                       ║
║  GITHUB FLOW (Simple, CD continuo) ← RECOMENDADO PARA MLOPS           ║
║  ═══════════════════════════════════════════════════                  ║
║  main ──●────●────●────●────●────●────●──                             ║
```

```
│          \  /      \  /      \  /                                    │
│  feature  ●         ●         ●                                     │
│           PR       PR        PR                                     │
│                                                                     │
│  ✅ Para: MLOps, CI/CD frecuente, equipos pequeños-medianos         │
│  ✅ Simple: Solo main + feature branches                            │
│                                                                     │
├─────────────────────────────────────────────────────────────────────┤
│                                                                     │
│  TRUNK-BASED (Avanzado, feature flags)                              │
│  ──────────────────────────────────────                             │
│  main ──●──●──●──●──●──●──●──●──●──●──                              │
│         │  │  │  │  │  │  │  │  │  │                               │
│         └──┴──┴──┴──┴──┴──┴──┴──┴──┘                               │
│         Commits directos a main (con feature flags)                 │
│                                                                     │
│  ✅ Para: Equipos muy maduros, deploys múltiples/día                │
│  ❌ No para: Equipos nuevos, sin feature flags robustos             │
│                                                                     │
└─────────────────────────────────────────────────────────────────────┘
```

GitHub Flow para MLOps (Recomendado)

```
gitGraph
    commit id: "initial"
    branch feature/add-mlflow
    commit id: "feat(tracking): add MLflow integration"
    commit id: "test(tracking): add tests for experiment tracking"
    checkout main
    merge feature/add-mlflow id: "PR #12"
    branch feature/api-batch
    commit id: "feat(api): add batch prediction endpoint"
    checkout main
    merge feature/api-batch id: "PR #13"
    branch fix/nan-handling
    commit id: "fix(training): handle NaN in features"
    checkout main
    merge fix/nan-handling id: "PR #14"
```

Convenciones de Naming para Branches

```
# Features
feature/add-mlflow-tracking
feature/api-batch-prediction
feature/JIRA-123-user-auth

# Fixes
fix/nan-handling
fix/memory-leak-inference
fix/JIRA-456-login-error

# Refactors
refactor/config-pydantic
refactor/training-pipeline

# Experiments (para ML)
experiment/xgboost-vs-rf
experiment/feature-selection

# Releases (si usas Git Flow)
release/1.2.0
hotfix/1.2.1
```

4.4 Comandos Avanzados que Todo Senior Debe Conocer

Rebase Interactivo: Limpiar Historial

```
# Últimos 3 commits
git rebase -i HEAD~3

# Opciones en el editor:
# pick   = usar commit as-is
# reword = cambiar mensaje
```

```
# edit   = pausar para editar
# squash = combinar con anterior
# fixup  = combinar sin mensaje
# drop   = eliminar commit

# Ejemplo: Combinar 3 commits WIP en uno
# pick abc123 feat(api): add endpoint
# squash def456 wip
# squash ghi789 fix typo
# → Se convierten en un solo commit limpio
```

Cherry-pick: Traer Commits Específicos

```
# Traer un commit de otra rama
git cherry-pick abc123

# Traer varios commits
git cherry-pick abc123 def456

# Traer sin commitear (para combinar)
git cherry-pick --no-commit abc123
```

Bisect: Encontrar el Commit que Rompió Algo

```
# Iniciar bisect
git bisect start

# Marcar estado actual como malo
git bisect bad

# Marcar un commit conocido como bueno
git bisect good v1.0.0

# Git te lleva a un commit intermedio
# Testear y marcar:
git bisect good  # Si funciona
git bisect bad   # Si está roto

# Repetir hasta encontrar el commit culpable
# Al final:
git bisect reset
```

Stash: Guardar Cambios Temporalmente

```
# Guardar cambios actuales
git stash

# Con mensaje descriptivo
git stash push -m "WIP: refactoring config"

# Listar stashes
git stash list

# Aplicar último stash
git stash pop

# Aplicar stash específico
git stash apply stash@{2}

# Crear branch desde stash
git stash branch feature/from-stash
```

Reflog: Recuperar lo "Perdido"

```
# Ver historial de operaciones
git reflog

# Recuperar commit "perdido" después de reset
git reflog
# abc123 HEAD@{3}: commit: feat: important change
git checkout abc123
```

```
# o
git reset --hard abc123
```

4.5 .gitignore Profesional para MLOps

```
# .gitignore para proyectos MLOps

# ═══════════════════════════════════════════════════════════════════
# PYTHON
# ═══════════════════════════════════════════════════════════════════
__pycache__/
*.py[cod]
*$py.class
*.so
.Python
build/
develop-eggs/
dist/
downloads/
eggs/
.eggs/
lib/
lib64/
parts/
sdist/
var/
wheels/
*.egg-info/
.installed.cfg
*.egg

# ═══════════════════════════════════════════════════════════════════
# ENTORNOS VIRTUALES
# ═══════════════════════════════════════════════════════════════════
.venv/
venv/
ENV/
env/
.conda/

# ═══════════════════════════════════════════════════════════════════
# IDEs
# ═══════════════════════════════════════════════════════════════════
.idea/
.vscode/
*.swp
*.swo
*~
.spyderproject
.spyproject

# ═══════════════════════════════════════════════════════════════════
# JUPYTER NOTEBOOKS
# ═══════════════════════════════════════════════════════════════════
.ipynb_checkpoints/
*.ipynb_checkpoints/

# ═══════════════════════════════════════════════════════════════════
# DATOS Y MODELOS (gestionados por DVC)
# ═══════════════════════════════════════════════════════════════════
data/raw/*
data/processed/*
models/*.pkl
models/*.joblib
!data/raw/.gitkeep
!data/processed/.gitkeep
!models/.gitkeep

# DVC
/data/*.csv
/data/*.parquet

# ═══════════════════════════════════════════════════════════════════
# MLFLOW
```

```
# ═══════════════════════════════════════════════════════════════════
mlruns/
mlartifacts/

# ═══════════════════════════════════════════════════════════════════
# SECRETOS Y CONFIGURACIÓN LOCAL
# ═══════════════════════════════════════════════════════════════════
.env
.env.*
!.env.example
*.pem
*.key
secrets/
credentials/

# ═══════════════════════════════════════════════════════════════════
# TESTING Y COVERAGE
# ═══════════════════════════════════════════════════════════════════
.coverage
.pytest_cache/
htmlcov/
.tox/
.nox/
coverage.xml
*.cover
.hypothesis/

# ═══════════════════════════════════════════════════════════════════
# BUILDS Y DOCS
# ═══════════════════════════════════════════════════════════════════
site/
docs/_build/
*.log

# ═══════════════════════════════════════════════════════════════════
# OS
# ═══════════════════════════════════════════════════════════════════
.DS_Store
Thumbs.db
```

4.6 Ejercicio Integrador: Setup Completo de Git

Paso 1: Configurar Git Global

```
# Identidad
git config --global user.name "Tu Nombre"
git config --global user.email "tu@email.com"

# Editor (VS Code)
git config --global core.editor "code --wait"

# Alias útiles
git config --global alias.st "status -sb"
git config --global alias.co "checkout"
git config --global alias.br "branch"
git config --global alias.cm "commit -m"
git config --global alias.lg "log --oneline --graph --all"
git config --global alias.last "log -1 HEAD --stat"
git config --global alias.unstage "reset HEAD --"

# Auto-setup remote tracking
git config --global push.autoSetupRemote true

# Default branch
git config --global init.defaultBranch main
```

Paso 2: Inicializar Proyecto

```
# Crear repo
mkdir bankchurn-predictor && cd bankchurn-predictor
git init

# Crear estructura
mkdir -p src/bankchurn/{data,models,utils} tests/{unit,integration} configs docs
```

```
# Archivos base
touch src/bankchurn/__init__.py
touch .gitignore .pre-commit-config.yaml pyproject.toml README.md

# Primer commit
git add .
git commit -m "chore: initial project structure"
```

Paso 3: Configurar Pre-commit

```
# Instalar
pip install pre-commit

# Copiar el .pre-commit-config.yaml de la sección 4.2

# Instalar hooks
pre-commit install
pre-commit install --hook-type commit-msg

# Ejecutar en todos los archivos
pre-commit run --all-files
```

Paso 4: Crear Feature Branch y PR

```
# Crear branch
git checkout -b feature/add-config

# Hacer cambios...
# Commit con conventional commits
git commit -m "feat(config): add Pydantic configuration models"

# Push
git push -u origin feature/add-config

# Crear PR en GitHub
# (usar template de PR si existe)
```

Checklist de Verificación

```
CONFIGURACIÓN:
[ ] Git configurado con nombre y email
[ ] Alias útiles configurados
[ ] Default branch es main

PRE-COMMIT:
[ ] pre-commit instalado
[ ] Hooks activos (commit + commit-msg)
[ ] Todos los hooks pasan en --all-files

 FLUJO:
 [ ] Puedo crear feature branches correctamente
 [ ] Commits siguen Conventional Commits
 [ ] .gitignore excluye archivos correctos
 ```


🧨 Errores habituales y cómo depurarlos en Git

Git aquí no es solo “guardar versiones”, sino soportar **flujos de trabajo profesionales** con branches, hooks y CI.
Estos son los errores más frecuentes en el portafolio y cómo atacarlos.

1) Commits que rompen el formato (Conventional Commits / commitlint / pre-commit)

Síntomas típicos

- `git commit` falla con mensajes como:
  ```text
  ⧗   input: fix: arreglos varios
  ✖   subject may not be empty [subject-empty]
  ✖   type must be one of [feat, fix, docs, style, ...]
  ```
- Hooks de `conventional-pre-commit` o `commitlint` bloquean el commit.
```
```

Cómo identificarlo

- Mira el mensaje de error completo del hook (no solo la última línea).
- Abre `commitlint.config.js` o `.pre-commit-config.yaml` y revisa:
  - Tipos permitidos (`feat`, `fix`, `docs`, etc.).
  - Scopes permitidos, si hay regla `scope-enum`.

Cómo corregirlo

- Ajusta tu mensaje al formato:
  ```bash
  git commit -m "feat(api): add /predict endpoint"
  git commit -m "fix(training): handle NaN in CreditScore"
  ```
- Si necesitas un scope nuevo (ej. `monitoring`), añádelo explícitamente a la regla de `scope-enum` y commitea ese cambio primero.

2) Hooks de pre-commit que “rompen todo” o tardan demasiado

Síntomas típicos

- Hacer `git commit` tarda mucho porque corre todos los tests y linters siempre.
- No entiendes qué hook falla; solo ves “pre-commit failed”.

Cómo identificarlo

- Ejecuta manualmente:
  ```bash
  pre-commit run --all-files
  ```
  y revisa qué hook está fallando (ruff, mypy, bandit, etc.).
- Abre `.pre-commit-config.yaml` y verifica qué rutas cubre cada hook.

Cómo corregirlo

- Para hooks muy pesados (tests completos, mypy en todo el repo):
  - Limita los paths relevantes (`files:` o `types:`) o muévelos a CI.
- Usa `pre-commit autoupdate` si un hook da errores por versiones muy viejas.
- Solo en emergencias, puedes hacer `git commit --no-verify`, pero documenta el motivo y arregla los hooks después.

3) Ramas desincronizadas y merges sucios

Síntomas típicos

- `git push` falla con `non-fast-forward`.
- Merge commits llenos de conflictos y mensajes genéricos.

Cómo identificarlo

- Revisa el historial con:
  ```bash
  git log --oneline --graph --all
  ```
  y mira si tu rama feature está muy alejada de `main`.

Cómo corregirlo

- Mantén tu feature branch fresca:
  ```bash
  git checkout feature/mi-feature
  git fetch origin
  git rebase origin/main
  ```
- Si el equipo prefiere `merge` sobre `rebase`, al menos haz `git pull --rebase` para evitar merges de “merge commits vacíos”.
- Usa PRs pequeños y frecuentes en lugar de ramas gigantes de semanas.

4) Archivos enormes, datos o secretos en el historial

Síntomas típicos

```
- El repo pesa cientos de MB; `git clone` es lento.
- `detect-secrets` o `gitleaks` encuentran claves/API keys en el historial.

**Cómo identificarlo**

- Ejecuta:
  ```bash
  git lfs track
  git rev-list --objects --all | sort -k 2 | tail -n 20
  ```
  para ver los blobs más grandes.
- Corre los hooks de seguridad (`detect-secrets`, `gitleaks`) y revisa sus reportes.

Cómo corregirlo

- Añade en `.gitignore` lo que no deba ir a Git (`data/`, `artifacts/`, `mlruns/`, etc.).
- Para secretos ya commiteados:
  - Rota la credencial en el proveedor (AWS, GCP, etc.).
  - Usa herramientas como `git filter-repo` para limpiar el historial (mencionado solo como referencia; normalmente se
hace una vez y con cuidado).

5) Patrón de debugging de Git en el portafolio

1. **Inspecciona el historial** con `git log --oneline --graph --all`.
2. **Verifica configuración local** (`git config --list --show-origin`).
3. **Reproduce el problema** (hook que falla, merge conflict, etc.) y lee el mensaje completo.
4. **Conecta el problema** con el concepto del módulo:
   - Formato de commits → Conventional Commits + commitlint.
   - Calidad del código → hooks de pre-commit.
   - Flujo de ramas → GitHub Flow (main + feature branches).
5. **Aplica el patrón de solución** que ya tienes documentado en este módulo.

Con este enfoque, Git deja de ser “magia negra” y se convierte en una herramienta predecible y aliada de tu flujo MLOps.

4.7 Autoevaluación

Preguntas de Reflexión

1. ¿Por qué Conventional Commits permite generar changelogs automáticamente?
2. ¿Cuál es la diferencia entre `git rebase` y `git merge`?
3. ¿Cuándo usarías `git stash` vs crear un branch?
4. ¿Por qué `no-commit-to-branch` es un hook útil?

Comandos que Debes Dominar

```bash
# Básicos
git status, add, commit, push, pull

# Branching
git branch, checkout -b, merge

# Historial
git log --oneline --graph, diff, show

# Avanzados
git rebase -i, cherry-pick, bisect, stash, reflog

# Pre-commit
pre-commit run, --all-files, autoupdate
```

📦 Cómo se Usó en el Portafolio

El portafolio implementa todas las prácticas de Git profesional descritas:

.pre-commit-config.yaml Real

```
# ML-MLOps-Portfolio/.pre-commit-config.yaml (extracto)
repos:
  - repo: https://github.com/pre-commit/pre-commit-hooks
    rev: v4.5.0
    hooks:
      - id: trailing-whitespace
```
```

```
      - id: end-of-file-fixer
      - id: check-yaml
      - id: check-added-large-files
        args: ['--maxkb=5000']

  - repo: https://github.com/astral-sh/ruff-pre-commit
    rev: v0.1.9
    hooks:
      - id: ruff
        args: [--fix]
      - id: ruff-format

  - repo: https://github.com/gitleaks/gitleaks
    rev: v8.18.1
    hooks:
      - id: gitleaks
```

Conventional Commits del Portafolio

Ejemplos de commits reales en el historial:

```
# Ejemplos del historial real del portafolio
feat(bankchurn): add unified sklearn pipeline
fix(carvision): prevent data leakage in FeatureEngineer
docs(guia): add module 11 Testing ML
test(telecomai): increase coverage to 97%
ci(actions): add matrix testing for Python 3.10/3.11
refactor(config): migrate to Pydantic v2
```

Branching Strategy

El portafolio usa **GitHub Flow** simplificado:

```
main ──────────────────────────────────────────────►
       │                    │
       └── feature/xyz ─────┘ (PR + CI verde + merge)
```

🔧 Ejercicio: Configura Pre-commit

```
# 1. Ve a la raíz del portafolio
cd ML-MLOps-Portfolio

# 2. Instala pre-commit
pip install pre-commit

# 3. Instala los hooks
pre-commit install

# 4. Ejecuta en todos los archivos
pre-commit run --all-files

# 5. Haz un commit y verifica que los hooks se ejecutan
echo "# test" >> test.md
git add test.md
git commit -m "test: verify pre-commit hooks"  # Los hooks se ejecutan aquí
git reset --soft HEAD~1  # Deshaz el commit de prueba
rm test.md
```

💼 Consejos Profesionales

Recomendaciones para destacar en entrevistas y proyectos reales

Para Entrevistas

1. **Conventional Commits**: Explica por qué `feat:`, `fix:`, `docs:` facilitan changelogs automáticos.
2. **Git Flow vs Trunk-Based**: Conoce ambos y cuándo usar cada uno.
3. **Rebase vs Merge**: Pregunta clásica. Respuesta: rebase para historia limpia, merge para preservar contexto.

Para Proyectos Reales

Situación	Consejo
Commits grandes	Divide en commits atómicos con `git add -p`
Historia sucia	Usa `git rebase -i` antes de PR
Secretos en repo	Usa git-secrets o gitleaks en pre-commit
Colaboración	PRs pequeños (< 400 líneas) se revisan mejor

Comandos que Debes Dominar

```
git stash push -m "descripción"  # Guardar trabajo temporal
git bisect start                 # Encontrar commit que introdujo bug
git reflog                       # Recuperar commits "perdidos"
git cherry-pick <commit>         # Aplicar commit específico
```

📺 Recursos Externos Recomendados

Ver RECURSOS_POR_MODULO.md para la lista completa.

🏷️	Recurso	Tipo
🔴	Git for Professionals - freeCodeCamp	Video
🟡	Conventional Commits	Docs

🔗 Referencias del Glosario

Ver 21_GLOSARIO.md para definiciones de:
- **pre-commit**: Hooks de validación antes de commit
- **Conventional Commits**: Formato estándar de mensajes
- **GitHub Flow**: Workflow de branching

✅ Ejercicios

Ver EJERCICIOS.md - Módulo 05:
- 5.1: Configurar .gitignore profesional
- 5.2: Instalar pre-commit hooks

➡️ Siguiente Paso

Con Git dominado, es hora de versionar **datos** profesionalmente.

Ir a Módulo 06: Versionado de Datos →

VERSIONADO DE DATOS

GUÍA MLOPS V5.0

PORTFOLIO EDITION

MÓDULO 05: INGENIERÍA DE DATOS Y DVC

Versionado de Datos, DAGs y Reproducibilidad

Guía MLOps v5.0: Senior Edition | DuqueOM | Noviembre 2025

📊 MÓDULO 05: Ingeniería de Datos y DVC

El Arte de Versionar lo que Git No Puede

"Si no puedo recrear tus datos, no puedo reproducir tu modelo."

Duración	Teoría	Práctica
5 - 6 horas	30%	70%

🎯 ADR de Inicio: ¿Cuándo (NO) Usar DVC?

```
╔═══════════════════════════════════════════════════════════════════╗
║  ADR-006: Criterios para Usar DVC                                 ║
╠═══════════════════════════════════════════════════════════════════╣
║                                                                   ║
║  ✅ USA DVC SI:                                                   ║
║  • Datos > 100MB que no caben cómodamente en Git                 ║
║  • Necesitas reproducibilidad exacta de datasets                 ║
║  • Equipo colabora en el mismo pipeline de datos                 ║
║  • Quieres DAGs declarativos para pipelines                      ║
║  • Datos son batch (no streaming)                                 ║
║                                                                   ║
║  ❌ NO USES DVC SI:                                               ║
║  • Datos < 50MB y no cambian frecuentemente → Git LFS o Git directo ║
║  • Datos son streaming (Kafka, Kinesis) → No aplica versionado batch ║
║  • Ya tienes Data Lake con Delta Lake/Iceberg → Usar versionado nativo ║
║  • Solo 1 persona trabaja en el proyecto → Puede ser overkill     ║
║  • Pipeline ya está en Airflow/Prefect → Evitar duplicación       ║
║                                                                   ║
║  DECISIÓN PARA BANKCHURN:                                         ║
║  Usar DVC porque: datos ~50MB con potencial de crecer, equipo colabora, ║
║  queremos reproducibilidad completa, y el pipeline es batch.      ║
║                                                                   ║
╚═══════════════════════════════════════════════════════════════════╝
```

Lo Que Lograrás en Este Módulo

1. **Entender** el problema del versionado de datos en ML
2. **Configurar** DVC con remote storage
3. **Crear** pipelines reproducibles con `dvc.yaml`
4. **Diseñar** DAGs para proyectos complejos

🧩 Cómo se aplica en este portafolio

- En `BankChurn-Predictor/` ya tienes configurado DVC con:
- `dvc.yaml` y `params.yaml` en la raíz del proyecto.
- Carpeta `data/` con datasets y `.dvc/` con metadatos de versionado.
- Desde esa carpeta puedes practicar el flujo completo de este módulo ejecutando: `bash cd BankChurn-Predictor dvc status dvc repro dvc pull`
- Aplica los mismos principios a futuros proyectos del portafolio para mantener datos y pipelines de forma reproducible, especialmente cuando crees el proyecto integrador.

5.1 El Problema: Git No Escala para Datos

```
╔═══════════════════════════════════════════════════════════════════╗
║                    😱 EL INFIERNO DEL VERSIONADO DE DATOS         ║
╠═══════════════════════════════════════════════════════════════════╣
║                                                                   ║
║   SIN VERSIONADO:                                                 ║
║                                                                   ║
║   data/                                                           ║
║   ├── churn.csv                  # ¿Original o procesado?         ║
║   ├── churn_v2.csv               # ¿Qué cambió?                   ║
```

```
│   ├── churn_final.csv             # ¿Es realmente el final?                  │
│   ├── churn_final_v2.csv          # 😱                                       │
│   ├── churn_final_FINAL.csv       # 💀                                       │
│   └── churn_20231115_backup.csv   # ???                                      │
│                                                                               │
│   PROBLEMAS:                                                                  │
│   • No sé qué datos usó el modelo v1.2.3                                      │
│   • No puedo reproducir resultados de hace 2 meses                            │
│   • Git se rompe con archivos grandes                                         │
│   • Colaboración es imposible ("¿tienes el CSV actualizado?")                 │
│                                                                               │
╞═══════════════════════════════════════════════════════════════════════════════╡
│                                                                               │
│   CON DVC:                                                                    │
│                                                                               │
│   data/                                                                       │
│   └── raw/                                                                    │
│       └── churn.csv.dvc     # Metadatos en Git, datos en storage              │
│                                                                               │
│   git checkout v1.2.3 && dvc checkout                                         │
│   → Tengo EXACTAMENTE los datos de esa versión                                │
│                                                                               │
└───────────────────────────────────────────────────────────────────────────────┘
```

Comparativa de Soluciones

Solución	Tamaño Máx	Versionado	Pipelines	Costo	Complejidad
Git directo	~ 1 0 MB	✅	❌	Gratis	Baja
Git LFS	~ 2 GB	✅	❌	$$$	Baja
DVC	Ilimitado	✅	✅	Storage	Media
Delta Lake	Ilimitado	✅	❌	Spark	Alta
LakeFS	Ilimitado	✅	❌	Server	Alta

5.2 Configuración Inicial de DVC

Instalación

```
# Con pip
pip install dvc

# Con extras para storage
pip install "dvc[s3]"      # Amazon S3
pip install "dvc[gs]"      # Google Cloud Storage
pip install "dvc[azure]"   # Azure Blob Storage
pip install "dvc[gdrive]"  # Google Drive (para proyectos personales)
```

Inicialización

```
# En un repo Git existente
cd bankchurn-predictor
dvc init

# Esto crea:
# .dvc/           - Directorio de configuración
# .dvc/.gitignore
# .dvc/config
# .dvcignore      - Qué ignorar (como .gitignore)
```

Configurar Remote Storage

```
# ═══════════════════════════════════════════════════════════════
# OPCIÓN 1: Local (para desarrollo)
# ═══════════════════════════════════════════════════════════════
dvc remote add -d localremote /path/to/dvc-storage
# -d = default remote
```

```
# ═══════════════════════════════════════════════════════════════════
# OPCIÓN 2: Amazon S3
# ═══════════════════════════════════════════════════════════════════
dvc remote add -d s3remote s3://my-bucket/dvc-storage
dvc remote modify s3remote region us-east-1
# Credenciales: AWS_ACCESS_KEY_ID, AWS_SECRET_ACCESS_KEY en env

# ═══════════════════════════════════════════════════════════════════
# OPCIÓN 3: Google Cloud Storage
# ═══════════════════════════════════════════════════════════════════
dvc remote add -d gcsremote gs://my-bucket/dvc-storage
# Credenciales: GOOGLE_APPLICATION_CREDENTIALS en env

# ═══════════════════════════════════════════════════════════════════
# OPCIÓN 4: Google Drive (Gratis, bueno para proyectos personales)
# ═══════════════════════════════════════════════════════════════════
dvc remote add -d gdriveremote gdrive://folder-id
# La primera vez pedirá autenticación OAuth

# ═══════════════════════════════════════════════════════════════════
# Ver configuración
# ═══════════════════════════════════════════════════════════════════
cat .dvc/config
```

Estructura de Directorios Recomendada

```
bankchurn-predictor/
├── data/
│   ├── raw/                    # Datos originales (DVC tracked)
│   │   ├── .gitkeep
│   │   └── churn.csv          # → churn.csv.dvc en Git
│   ├── processed/             # Datos procesados (output de pipeline)
│   │   └── .gitkeep
│   └── external/              # Datos de terceros
│       └── .gitkeep
├── models/                    # Modelos entrenados (DVC tracked)
│   └── .gitkeep
├── .dvc/
│   └── config
├── .dvcignore
└── dvc.yaml                   # Pipeline definition
```

5.3 Versionado Básico de Archivos

Añadir Datos a DVC

```
# Añadir archivo
dvc add data/raw/churn.csv

# Esto crea:
# data/raw/churn.csv.dvc   - Metadatos (hash, size)
# data/raw/.gitignore      - Ignora el CSV en Git

# Ver contenido del .dvc
cat data/raw/churn.csv.dvc
```

```
# data/raw/churn.csv.dvc
outs:
- md5: abc123def456...
  size: 52428800
  hash: md5
  path: churn.csv
```

Flujo de Trabajo

```
# 1. Modificar datos
# ... (actualizar churn.csv con nuevos registros)

# 2. Actualizar tracking
dvc add data/raw/churn.csv

# 3. Commit ambos cambios
```

```
git add data/raw/churn.csv.dvc data/raw/.gitignore
git commit -m "data(raw): update churn dataset with Q4 2024 data"

# 4. Push datos a remote
dvc push

# 5. Push código a Git
git push
```

Recuperar Datos de Versión Anterior

```
# Ver versiones del archivo
git log data/raw/churn.csv.dvc

# Checkout versión específica
git checkout v1.0.0 -- data/raw/churn.csv.dvc
dvc checkout data/raw/churn.csv

# O más simple: checkout todo
git checkout v1.0.0
dvc checkout
# → Ahora tienes código Y datos de v1.0.0
```

5.4 Pipelines con dvc.yaml (El Poder Real)

¿Por Qué Pipelines?

```
╔═══════════════════════════════════════════════════════════════════════╗
║                        PIPELINES DVC: REPRODUCIBILIDAD                ║
╟═══════════════════════════════════════════════════════════════════════╢
║                                                                       ║
║   SIN PIPELINE:                                                      ║
║   "Para reproducir, ejecuta preprocess.py, luego train.py, luego..."  ║
║   "Ah, pero primero asegúrate de tener los datos correctos..."        ║
║   "Y usa los mismos hiperparámetros que están en... algún lugar..."   ║
║                                                                       ║
║   CON PIPELINE DVC:                                                   ║
║   $ dvc repro                                                         ║
║   → Ejecuta TODO automáticamente, en orden correcto,                  ║
║     saltando stages que no cambiaron.                                 ║
║                                                                       ║
╚═══════════════════════════════════════════════════════════════════════╝
```

dvc.yaml Completo para BankChurn

```
# dvc.yaml
stages:
  # ═══════════════════════════════════════════════════════════════════
  # STAGE 1: Preparación de Datos
  # ═══════════════════════════════════════════════════════════════════
  prepare:
    cmd: python src/bankchurn/data/prepare.py
    deps:
      - src/bankchurn/data/prepare.py
      - data/raw/churn.csv
      - configs/config.yaml
    params:
      - prepare.test_size
      - prepare.random_state
    outs:
      - data/processed/train.csv
      - data/processed/test.csv

  # ═══════════════════════════════════════════════════════════════════
  # STAGE 2: Feature Engineering
  # ═══════════════════════════════════════════════════════════════════
  featurize:
    cmd: python src/bankchurn/features/build.py
    deps:
      - src/bankchurn/features/build.py
      - data/processed/train.csv
      - data/processed/test.csv
      - configs/config.yaml
```

```
    params:
      - features.numerical
      - features.categorical
    outs:
      - data/processed/train_features.pkl
      - data/processed/test_features.pkl

  # ═══════════════════════════════════════════════════════════════════
  # STAGE 3: Entrenamiento
  # ═══════════════════════════════════════════════════════════════════
  train:
    cmd: python src/bankchurn/training.py
    deps:
      - src/bankchurn/training.py
      - data/processed/train_features.pkl
      - configs/config.yaml
    params:
      - train.n_estimators
      - train.max_depth
      - train.random_state
    outs:
      - models/pipeline.pkl
    metrics:
      - metrics/train_metrics.json:
          cache: false

  # ═══════════════════════════════════════════════════════════════════
  # STAGE 4: Evaluación
  # ═══════════════════════════════════════════════════════════════════
  evaluate:
    cmd: python src/bankchurn/evaluate.py
    deps:
      - src/bankchurn/evaluate.py
      - models/pipeline.pkl
      - data/processed/test_features.pkl
    metrics:
      - metrics/eval_metrics.json:
          cache: false
    plots:
      - metrics/roc_curve.json:
          x: fpr
          y: tpr
      - metrics/confusion_matrix.json:
          template: confusion
          x: predicted
          y: actual
```

params.yaml (Configuración del Pipeline)

```
# params.yaml
prepare:
  test_size: 0.2
  random_state: 42

features:
  numerical:
    - CreditScore
    - Age
    - Tenure
    - Balance
    - NumOfProducts
    - EstimatedSalary
  categorical:
    - Geography
    - Gender

train:
  n_estimators: 100
  max_depth: 10
  random_state: 42
```

Comandos de Pipeline

```
# ═══════════════════════════════════════════════════════════════════
# REPRODUCIR PIPELINE
# ═══════════════════════════════════════════════════════════════════

# Ejecutar todo el pipeline
dvc repro

# Ejecutar stage específico (y sus dependencias)
dvc repro train

# Forzar re-ejecución (aunque no haya cambios)
dvc repro --force

# Ver qué se ejecutaría sin ejecutar
dvc repro --dry

# ═══════════════════════════════════════════════════════════════════
# VISUALIZAR PIPELINE
# ═══════════════════════════════════════════════════════════════════

# Ver DAG en terminal
dvc dag

# Generar imagen del DAG
dvc dag --dot | dot -Tpng -o pipeline.png

# Ver dependencias de un stage
dvc dag --outs train
```

Visualización del DAG

```
╔═══════════════════════════════════════════════════════════════════╗
║                          DVC DAG: BANKCHURN                       ║
╟═══════════════════════════════════════════════════════════════════╢
│                                                                   │
│                        ┌─────────────┐                            │
│                        │ data/raw/*.csv │                         │
│                        │ configs/*.yaml │                         │
│                        └─────────────┘                            │
│                               │                                   │
│                               ▼                                   │
│                        ┌─────────────┐                            │
│                        │   prepare   │                            │
│                        └─────────────┘                            │
│                               │                                   │
│                               ▼                                   │
│                        ┌─────────────┐                            │
│                        │ featurize   │                            │
│                        └─────────────┘                            │
│                               │                                   │
│                     ┌─────────┴─────────┐                         │
│                     ▼                   ▼                         │
│             ┌─────────────┐     ┌─────────────┐                   │
│             │   train     │     │ (test data) │                   │
│             └─────────────┘     └─────────────┘                   │
│                     │                   │                         │
│                     └─────────┬─────────┘                         │
│                               ▼                                   │
│                        ┌─────────────┐                            │
│                        │ evaluate    │                            │
│                        └─────────────┘                            │
│                               │                                   │
│                               ▼                                   │
│                        ┌─────────────┐                            │
│                        │ metrics/    │                            │
│                        └─────────────┘                            │
│                                                                   │
╚═══════════════════════════════════════════════════════════════════╝
```

5.5 Métricas y Experimentos

Tracking de Métricas

```
# Ver métricas actuales
dvc metrics show

# Comparar con otra rama/commit
dvc metrics diff HEAD~1

# Output ejemplo:
# Path                     Metric    HEAD     HEAD~1   Change
# metrics/eval_metrics.json  auc_roc   0.8721   0.8534   0.0187
# metrics/eval_metrics.json  f1        0.7234   0.7012   0.0222
```

Experimentos con DVC

```
# ═══════════════════════════════════════════════════════════════
# EJECUTAR EXPERIMENTOS
# ═══════════════════════════════════════════════════════════════

# Experimento con cambio de parámetro
dvc exp run --set-param train.n_estimators=200

# Múltiples experimentos en paralelo
dvc exp run --queue --set-param train.n_estimators=100
dvc exp run --queue --set-param train.n_estimators=200
dvc exp run --queue --set-param train.n_estimators=300
dvc exp run --run-all --parallel 3

# ═══════════════════════════════════════════════════════════════
# COMPARAR EXPERIMENTOS
# ═══════════════════════════════════════════════════════════════

# Ver todos los experimentos
dvc exp show

# Output:
# ┌──────────────────┬──────────────┬──────────────┬──────────────┐
# │ Experiment       │ auc_roc      │ f1           │ n_estimators │
# ├──────────────────┼──────────────┼──────────────┼──────────────┤
# │ main             │ 0.8721       │ 0.7234       │ 100          │
# │ exp-abc123       │ 0.8856       │ 0.7421       │ 200          │
# │ exp-def456       │ 0.8812       │ 0.7356       │ 300          │
# └──────────────────┴──────────────┴──────────────┴──────────────┘

# ═══════════════════════════════════════════════════════════════
# APLICAR MEJOR EXPERIMENTO
# ═══════════════════════════════════════════════════════════════

# Aplicar a workspace
dvc exp apply exp-abc123

# O crear branch
dvc exp branch exp-abc123 feature/best-model
```

5.6 Patrones Avanzados

Multi-Output Stages

```
# dvc.yaml
stages:
  split:
    cmd: python src/split.py
    deps:
      - data/raw/full_dataset.csv
    outs:
      - data/processed/train.csv
      - data/processed/val.csv
      - data/processed/test.csv
```

Stages Condicionales (foreach)

```
# dvc.yaml - Entrenar múltiples modelos
stages:
  train:
    foreach:
      - random_forest
      - xgboost
      - lightgbm
    do:
      cmd: python src/train.py --model ${item}
      deps:
        - src/train.py
        - data/processed/train.csv
      params:
        - train.${item}
      outs:
        - models/${item}.pkl
      metrics:
        - metrics/${item}_metrics.json:
            cache: false
```

Integración con MLflow

```
# src/bankchurn/training.py
import mlflow
import dvc.api
import yaml

def train():
    # Obtener parámetros de DVC
    params = dvc.api.params_show()

    with mlflow.start_run():
        # Log parámetros
        mlflow.log_params(params["train"])

        # Entrenar...
        model = train_model(params["train"])

        # Log métricas
        metrics = evaluate(model)
        mlflow.log_metrics(metrics)

        # Guardar métricas para DVC también
        with open("metrics/train_metrics.json", "w") as f:
            json.dump(metrics, f)

        # Log modelo
        mlflow.sklearn.log_model(model, "model")
---

## 🧨 Errores habituales y cómo depurarlos en DVC

Aunque DVC parece “solo añadir un comando más”, en la práctica los errores suelen venir de **desalineación entre Git,
datos y configuración**.

### 1) Datos no aparecen al clonar el repo (`dvc pull`/`dvc checkout` olvidados)

**Síntomas típicos**

- Clonas el repositorio, ejecutas el código y obtienes errores como:
  ```text
  FileNotFoundError: data/raw/churn.csv not found
  ```
- La carpeta `data/` está vacía o solo tiene `.gitkeep`.

Cómo identificarlo

- Ejecuta:
  ```bash
  dvc list .
  dvc status
  ```
```

```
  para ver qué outs están trackeados.
- Mira si existen archivos `.dvc` (`data/raw/churn.csv.dvc`) pero no los datos reales.

**Cómo corregirlo**

- Después de clonar o cambiar de rama/tag, **siempre** ejecuta:
  ```bash
  dvc pull      # trae los datos desde el remote
  dvc checkout  # sincroniza versiones de datos con los .dvc actuales
  ```
- Documenta esto en el README del proyecto y en este módulo como parte del flujo estándar.

---

2) `.dvc` committeados pero remote sin configurar (`dvc push` fallando)

Síntomas típicos

- Haces `dvc push` y ves errores tipo:
  ```text
  ERROR: failed to push data to the cloud - config file error
  ```
  o credenciales faltantes.
- Compañeros de equipo tienen los `.dvc`, pero `dvc pull` no trae nada.

Cómo identificarlo

- Revisa `.dvc/config` para ver qué remote está configurado (`localremote`, `s3remote`, etc.).
- Ejecuta `dvc remote list` y valida que el remote por defecto (`-d`) exista y sea accesible.

Cómo corregirlo

- Asegúrate de que todos usen **el mismo nombre de remote** y que esté configurado en el repo (no solo en local).
- Para remotes cloud (S3, GCS): documenta las variables de entorno necesarias (`AWS_ACCESS_KEY_ID`, etc.).
- Haz un `dvc push` de prueba y luego un `dvc pull` desde otra máquina para validar.

---

3) `dvc repro` no ejecuta stages que esperas (cambios no detectados)

Síntomas típicos

- Modificas código o parámetros, ejecutas `dvc repro` y ves:
  ```text
  Stage 'train' didn't change, skipping
  ```
  aunque esperabas que volviera a entrenar.

Cómo identificarlo

- Mira el `dvc.yaml` y verifica que:
  - El script que cambiaste esté en `deps:` del stage.
  - Los parámetros que tocaste estén en `params:`.

Cómo corregirlo

- Asegúrate de listar **todas las dependencias reales** en `deps:` (scripts, configs, datos intermedios).
- Si cambiaste parámetros en `params.yaml`, agrégalos a la lista `params:` del stage correspondiente.
- Si quieres forzar una re-ejecución puntual, usa `dvc repro --force train`.

---

4) Conflictos entre `.gitignore` y `.dvc` (datos en Git por accidente)

Síntomas típicos

- Ves archivos grandes (`data/raw/*.csv`, `models/*.pkl`) en `git status`.
- Existen `.dvc` pero los datos también se han añadido a Git.

Cómo identificarlo

- Revisa `data/raw/.gitignore` generado por `dvc add` y el `.gitignore` del proyecto principal; puede que se estén
pisando.

Cómo corregirlo
```

- Respeta el patrón DVC:
  - Los datos **no** se añaden a Git, solo los `.dvc`.
  - Asegúrate de que `.gitignore` incluya las carpetas de datos/artefactos y que no contradiga los `.gitignore` generados por DVC.
- Si ya has commiteado datos grandes, elimínalos del historial (o al menos del último commit) y deja solo los `.dvc`.

---

5) DVC + CI/CD: pipelines que fallan en GitHub Actions

Síntomas típicos

- En CI, `dvc repro` falla porque no encuentra datos o no tiene acceso al remote.

Cómo identificarlo

- Revisa el workflow de CI y verifica si:
  - Has instalado DVC con los extras correctos (`dvc[s3]`, etc.).
  - Has configurado variables de entorno con credenciales.
  - Estás ejecutando `dvc pull` **antes** de correr el pipeline.

Cómo corregirlo

- Añade pasos en tu workflow:
  ```yaml
  - name: Install DVC
    run: pip install "dvc[s3]"

  - name: Pull data with DVC
    run: dvc pull

  - name: Run pipeline
    run: dvc repro
  ```
- Usa `dvc repro --dry` localmente para ver qué debería ejecutarse antes de llevarlo a CI.

Patrón general de debugging en DVC

1. **Inspecciona el estado** con `dvc status` y `dvc dag`.
2. **Verifica remotes y credenciales** (`dvc remote list`, `.dvc/config`).
3. **Comprueba deps/outs/params** en `dvc.yaml` para el stage problemático.
4. **Sincroniza Git + DVC**: `git checkout <tag/branch>` seguido de `dvc checkout` y `dvc pull` si hace falta.

Con este checklist, DVC pasa de ser “caja negra que falla” a una herramienta controlable para reproducir datos y pipelines.

5.7 Ejercicio Integrador

Setup Completo de DVC

```bash

# 1. Inicializar DVC
cd bankchurn-predictor
dvc init

# 2. Configurar remote (local para empezar)
mkdir -p ~/dvc-storage
dvc remote add -d localremote ~/dvc-storage

# 3. Crear estructura de datos
mkdir -p data/{raw,processed} models metrics

# 4. Añadir datos raw

# (asumiendo que tienes churn.csv)
cp /path/to/churn.csv data/raw/
dvc add data/raw/churn.csv

# 5. Crear dvc.yaml (copiar del ejemplo anterior)
```

```
# 6. Crear params.yaml

# 7. Commit todo
git add .
git commit -m "data(dvc): setup DVC pipeline"

# 8. Ejecutar pipeline
dvc repro

# 9. Push a remote
dvc push
git push
```

Checklist de Verificación

```
CONFIGURACIÓN:
[ ] DVC inicializado
[ ] Remote configurado y funcionando
[ ] Datos raw tracked con DVC

PIPELINE:
[ ] dvc.yaml con stages definidos
[ ] params.yaml con parámetros
[ ] dvc repro ejecuta sin errores

VERSIONADO:
[ ] Puedo hacer git checkout + dvc checkout a versiones anteriores
[ ] dvc push/pull funcionan correctamente
[ ] Métricas se trackean con dvc metrics show
```

5.8 Autoevaluación

Preguntas de Reflexión

1. ¿Por qué DVC usa hashes MD5 en lugar de guardar los archivos?
2. ¿Qué pasa si cambio `params.yaml` pero no el código?
3. ¿Cuándo DVC salta un stage sin ejecutarlo?
4. ¿Cómo integrarías DVC con GitHub Actions para CI?

📦 Cómo se Usó en el Portafolio

El portafolio tiene DVC configurado a nivel global:

Estructura DVC del Portafolio

```
ML-MLOps-Portfolio/
├── .dvc/                  # Configuración DVC
│   └── config             # Remote storage config
├── .dvc-storage/          # Remote local (para demo)
├── .dvcignore            # Archivos a ignorar
└── */data/raw/*.dvc       # Archivos .dvc en cada proyecto
```

Archivos .dvc Reales

```
# BankChurn-Predictor/data/raw/bank_churn.csv.dvc
md5: abc123def456...
size: 1234567
path: bank_churn.csv

# CarVision-Market-Intelligence/data/raw/car_prices.csv.dvc
md5: xyz789ghi012...
size: 2345678
path: car_prices.csv
```

Flujo de Datos en el Portafolio

```
┌──────────────────────────────────────────────────────────────┐
│                      FLUJO DE DATOS DVC                      │
├──────────────────────────────────────────────────────────────┤
```

```
│                                                                  │
│  data/raw/*.csv    →    .dvc files    →    .dvc-storage/        │
│  (gitignored)           (tracked)          (remote local)       │
│                                                                  │
│  Para CI/CD:                                                     │
│  git clone → dvc pull → datos disponibles                        │
│                                                                  │
└──────────────────────────────────────────────────────────────────┘
```

Comandos DVC del Portafolio

```
# Ver qué datos están trackeados
dvc status

# Obtener datos después de clonar
dvc pull

# Agregar nuevos datos
dvc add data/raw/nuevos_datos.csv
git add data/raw/nuevos_datos.csv.dvc data/raw/.gitignore
git commit -m "data(dvc): add nuevos_datos"
dvc push
```

🔧 Ejercicio: Trabaja con DVC Real

```
# 1. Ve a la raíz del portafolio
cd ML-MLOps-Portfolio

# 2. Verifica estado de DVC
dvc status

# 3. Obtén los datos (si no los tienes)
dvc pull

# 4. Verifica que los datos existen
ls -la BankChurn-Predictor/data/raw/
ls -la CarVision-Market-Intelligence/data/raw/

# 5. Experimenta: modifica params y reproduce
cd BankChurn-Predictor
dvc repro  # Si tienes dvc.yaml configurado
```

💼 Consejos Profesionales

Recomendaciones para destacar en entrevistas y proyectos reales

Para Entrevistas

1. **DVC vs Git LFS**: Explica que DVC es específico para ML (pipelines, métricas), LFS es genérico para archivos grandes.
2. **Reproducibilidad**: Menciona que puedes recrear cualquier experimento con `dvc checkout` + `git checkout`.
3. **Data Lineage**: Explica cómo DVC trackea la procedencia de datos transformados.

Para Proyectos Reales

Situación	Consejo
Datos sensibles	Usa DVC con storage encriptado (S 3 + KMS)
Datasets grandes	Usa `dvc push/pull` selectivo por carpeta
CI/CD	Cachea datos en CI para evitar descargas repetidas
Colaboración	Documenta dónde está el remote storage

Flujo Profesional de Datos

1. Raw data → nunca modificar, solo agregar

2. Processed data → versionado con DVC
3. Features → cacheados para reutilización
4. Modelos → versionados con métricas

📺 Recursos Externos Recomendados

Ver RECURSOS_POR_MODULO.md para la lista completa.

🏷️	Recurso	Tipo
🔴	DVC Tutorial - DataTalks	Video
🟡	DVC Documentation	Docs

🔗 Referencias del Glosario

Ver 21_GLOSARIO.md para definiciones de:
- **DVC**: Data Version Control
- **Remote Storage**: Almacenamiento externo para datos
- **dvc.yaml**: Definición de pipelines reproducibles

✅ Ejercicios

Ver EJERCICIOS.md - Módulo 06:
- 6.1: Configurar DVC en proyecto
- 6.2: Push/pull de datos

🎤 Checkpoint: Simulacro Junior

🎯 **¡Has completado los fundamentos!** (Módulos 01-06)

Si buscas posiciones **Junior ML Engineer**, ahora es buen momento para practicar:

→ SIMULACRO_ENTREVISTA_JUNIOR.md
- 50 preguntas de Python, ML básico, Git y estructura
- Enfoque en fundamentos y capacidad de aprendizaje

➡️ Siguiente Paso

Con datos versionados, es hora de construir **pipelines de sklearn avanzados**.

Ir a Módulo 07: sklearn Pipelines →

[← Git Profesional](#mod_05_GIT_PROFESIONAL) | [Siguiente: sklearn Pipelines →](#mod_07_SKLEARN_PIPELINES)

SIMULACRO ENTREVISTA JUNIOR

GUÍA MLOPS V5.0

PORTFOLIO EDITION

🎯 Simulacro de Entrevista Junior ML Engineer

Portafolio MLOps — 50 Preguntas Fundamentales

Autor del Portafolio: Daniel Duque (DuqueOM)
Versión: 1.0
Fecha: Diciembre 2025
Nivel: Junior (0-2 años de experiencia)

📋 Índice

1. Python Básico
2. Machine Learning Fundamentos
3. Datos y Preprocesamiento
4. Git y Herramientas
5. Práctica con el Portafolio

🎯 Antes de Empezar

¿Qué se espera de un Junior?

Lo que SÍ se espera	Lo que NO se espera
Fundamentos sólidos de Python	Diseño de arquitecturas complejas
Entender train/test split	Optimización de hiperparámetros avanzada
Saber qué es overfitting	Implementar MLOps completo
Usar Git básico	CI/CD avanzado
Leer y modificar código existente	Escribir código de producción desde cero
Hacer preguntas inteligentes	Tener todas las respuestas

Consejos para la Entrevista

1. **Sé honesto**: "No lo sé, pero lo investigaría así..." es mejor que inventar
2. **Muestra curiosidad**: Haz preguntas sobre el código que ves
3. **Relaciona con el portafolio**: "En BankChurn aprendí que..."
4. **Piensa en voz alta**: El proceso importa más que la respuesta perfecta

1. Python Básico (Preguntas 1-10)

Pregunta 1: Tipos de Datos

¿Cuál es la diferencia entre lista, tupla y diccionario?

Respuesta:

```
# Lista: mutable, ordenada
features = ["age", "salary", "tenure"]
features.append("score")  # OK

# Tupla: inmutable, ordenada
coordinates = (40.7, -74.0)
# coordinates[0] = 41.0  # ERROR

# Diccionario: mutable, key-value
customer = {"id": 123, "name": "John", "churn": False}
customer["score"] = 0.85  # OK
```

Cuándo usar cada uno:
- **Lista**: Colección que cambiará (features a seleccionar)
- **Tupla**: Datos que no deben cambiar (coordenadas, constantes)
- **Diccionario**: Acceso por clave (configuración, datos de cliente)

Pregunta 2: List Comprehension

Reescribe este código con list comprehension:

```
result = []
for x in range(10):
    if x % 2 == 0:
        result.append(x**2)
```

Respuesta:

```
result = [x**2 for x in range(10) if x % 2 == 0]
# [0, 4, 16, 36, 64]
```

Ventajas:
- Más conciso
- Más rápido (optimizado internamente)
- Más "pythónico"

Pregunta 3: Funciones y Argumentos

¿Qué hace `*args` y `**kwargs`?

Respuesta:

```
def log_experiment(*args, **kwargs):
    # args: tupla de argumentos posicionales
    # kwargs: diccionario de argumentos con nombre
    print(f"Metrics: {args}")
    print(f"Config: {kwargs}")

log_experiment(0.85, 0.82, model="rf", n_estimators=100)
# Metrics: (0.85, 0.82)
# Config: {'model': 'rf', 'n_estimators': 100}
```

En el portafolio (`BankChurn/trainer.py`):

```
def __init__(self, config: BankChurnConfig, **kwargs):
    self.config = config
    self.extra_params = kwargs  # Flexibilidad para params adicionales
```

Pregunta 4: Manejo de Errores

¿Por qué usamos try/except?

Respuesta:

```
def load_data(path: str) -> pd.DataFrame:
    try:
        df = pd.read_csv(path)
        return df
    except FileNotFoundError:
        print(f"Error: {path} no existe")
        raise
    except pd.errors.EmptyDataError:
        print("Error: archivo vacío")
        raise
```

Buenas prácticas:
- Capturar excepciones específicas, no genéricas
- Hacer logging del error
- Re-lanzar si no puedes manejarlo

Pregunta 5: Import y Módulos

¿Cuál es la diferencia entre estas formas de import?

Respuesta:

```
# Importar módulo completo
import pandas as pd
df = pd.read_csv("data.csv")

# Importar función específica
from sklearn.model_selection import train_test_split
```

```
X_train, X_test = train_test_split(X)

# Importar todo (⚠️ evitar en producción)
from math import *  # Contamina el namespace
```

Best practice: Importar lo que necesitas, usar alias estándar (`pd`, `np`, `plt`).

Pregunta 6: Type Hints

¿Qué significan los type hints y por qué usarlos?

Respuesta:

```
def predict_churn(
    credit_score: int,
    age: int,
    is_active: bool
) -> float:
    """Retorna probabilidad de churn."""
    ...
```

Beneficios:
1. **Documentación**: Claro qué espera y retorna
2. **IDE support**: Autocompletado, detección de errores
3. **Tooling**: `mypy` puede verificar tipos

En el portafolio: Todos los archivos usan type hints (`config.py`, `training.py`).

Pregunta 7: Clases Básicas

¿Qué es `__init__` y `self`?

Respuesta:

```
class BankChurnTrainer:
    def __init__(self, config):
        # Constructor: se ejecuta al crear instancia
        self.config = config  # self = esta instancia
        self.model_ = None

    def train(self, X, y):
        # self permite acceder a atributos de la instancia
        if self.config.model_type == "rf":
            self.model_ = RandomForestClassifier()
        self.model_.fit(X, y)

# Uso
trainer = BankChurnTrainer(config)  # __init__ se llama aquí
trainer.train(X, y)
```

Pregunta 8: Lectura de Archivos

¿Cómo lees un archivo CSV con pandas?

Respuesta:

```
import pandas as pd

# Básico
df = pd.read_csv("data/raw/Churn.csv")

# Con opciones
df = pd.read_csv(
    "data/raw/Churn.csv",
    sep=",",
    encoding="utf-8",
    na_values=["", "NA", "null"],
    dtype={"customer_id": str}
)

# Verificar
print(df.shape)       # (10000, 14)
print(df.info())      # Tipos y nulls
print(df.head())      # Primeras filas
```

Pregunta 9: Entornos Virtuales

¿Por qué usamos entornos virtuales?

Respuesta:

```
# Crear entorno
python -m venv .venv

# Activar
source .venv/bin/activate  # Linux/Mac
.venv\Scripts\activate     # Windows

# Instalar dependencias
pip install -r requirements.txt
```

Razones:
1. **Aislamiento**: Cada proyecto tiene sus propias versiones
2. **Reproducibilidad**: Mismo entorno en cualquier máquina
3. **Evita conflictos**: sklearn 1.3 en proyecto A, sklearn 1.2 en proyecto B

Pregunta 10: Debugging Básico

¿Cómo depuras código en Python?

Respuesta:

```
# 1. Print statements (básico pero útil)
print(f"X shape: {X.shape}, y shape: {y.shape}")

# 2. Usar assert
assert X.shape[0] == y.shape[0], "Mismatch en filas"

# 3. Breakpoints en IDE (recomendado)
# Poner breakpoint y usar F5 para debugear

# 4. pdb (en terminal)
import pdb; pdb.set_trace()

# 5. Logging (producción)
import logging
logging.debug(f"Loaded {len(df)} rows")
```

2. Machine Learning Fundamentos (Preguntas 11-20)

Pregunta 11: Train/Test Split

¿Por qué separamos datos en train y test?

Respuesta:

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,      # 80/20 split
    random_state=42,    # Reproducibilidad
    stratify=y          # Mantener proporción de clases
)
```

Razón: Evaluar cómo el modelo generaliza a datos **nunca vistos**.
- **Train**: Aprende patrones
- **Test**: Simula producción, mide rendimiento real

Error común: Usar test para ajustar modelo → overfitting al test.

Pregunta 12: Overfitting vs Underfitting

Explica overfitting y underfitting.

Respuesta:

Concepto	Síntomas	Causa	Solución
Overfitting	Train acc: 99%, Test acc: 70%	Modelo muy complejo	Regularización, más datos, simplificar
Underfitting	Train acc: 60%, Test acc: 58%	Modelo muy simple	Más features, modelo más complejo

```
# Detectar en el portafolio
print(f"Train accuracy: {model.score(X_train, y_train):.2%}")
print(f"Test accuracy: {model.score(X_test, y_test):.2%}")

# Si diferencia > 10%, posible overfitting
```

Pregunta 13: Clasificación vs Regresión

¿Cuándo usar clasificación y cuándo regresión?

Respuesta:

Problema	Tipo	Target	Métrica
¿Cliente hará churn?	Clasificación	Sí/No (0 / 1)	Accuracy, F 1 , AUC
¿Cuánto cuesta el auto?	Regresión	Precio ($)	RMSE, MAE, R^2
¿Qué plan elegirá?	Clasificación multiclase	A/B/C	Accuracy, F 1 macro

En el portafolio:
- **BankChurn**: Clasificación binaria (churn: 0 / 1)
- **CarVision**: Regresión (precio continuo)
- **TelecomAI**: Clasificación multiclase (tipo de plan)

Pregunta 14: Cross-Validation

¿Qué es cross-validation y por qué usarla?

Respuesta:

```
from sklearn.model_selection import cross_val_score

scores = cross_val_score(model, X, y, cv=5)
print(f"Accuracy: {scores.mean():.3f} (+/- {scores.std()*2:.3f})")
```

Proceso K-Fold (K= 5):
1. Divide datos en 5 partes iguales
2. Entrena en 4 , valida en 1
3. Repite 5 veces (cada parte es validación una vez)
4. Promedia resultados

Ventajas:
- Usa todos los datos para entrenar y validar
- Estimación más robusta del rendimiento
- Detecta variabilidad del modelo

Pregunta 15: Feature Scaling

¿Por qué normalizamos features?

Respuesta:

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_train)

# Antes: age=[18-92], salary=[20000-200000]
# Después: ambas con media=0, std=1
```

Razones:
1. **Algoritmos sensibles a escala**: SVM, KNN, redes neuronales
2. **Gradiente descent**: Converge más rápido
3. **Interpretación**: Coeficientes comparables

Algoritmos que NO necesitan scaling: Random Forest, Decision Tree, XGBoost.

Pregunta 16: One-Hot Encoding

¿Cómo manejas variables categóricas?

Respuesta:

```
from sklearn.preprocessing import OneHotEncoder

encoder = OneHotEncoder(sparse_output=False, handle_unknown='ignore')
X_encoded = encoder.fit_transform(df[['Geography', 'Gender']])

# Geography: France, Germany, Spain
# → Geography_France, Geography_Germany, Geography_Spain
```

Alternativas:
- **Label Encoding**: Para ordinales (Bajo < Medio < Alto)
- **Target Encoding**: Codifica con la media del target (⚠️ riesgo de leakage)

Pregunta 17: Missing Values

¿Cómo manejas valores faltantes?

Respuesta:

```
from sklearn.impute import SimpleImputer

# Numéricos: media o mediana
imputer_num = SimpleImputer(strategy='median')

# Categóricos: moda o valor constante
imputer_cat = SimpleImputer(strategy='constant', fill_value='Unknown')
```

Estrategias:

Caso	Estrategia
Pocos missing (< 5 %)	Imputar con media/moda
Muchos missing	Considerar eliminar columna
Missing tiene significado	Crear feature `is_missing`

Pregunta 18: Random Forest

Explica cómo funciona Random Forest.

Respuesta:

```
from sklearn.ensemble import RandomForestClassifier

rf = RandomForestClassifier(
    n_estimators=100,  # 100 árboles
    max_depth=10,      # Profundidad máxima
    random_state=42
)
```

Concepto simple:
1. Crea N árboles de decisión
2. Cada árbol usa subset aleatorio de datos y features
3. Predicción final = voto mayoritario (clasificación) o promedio (regresión)

Ventajas: Robusto, pocas configuraciones, maneja bien missing values.

Pregunta 19: Métricas de Clasificación

¿Qué es accuracy, precision, recall y F 1 ?

Respuesta:

```
from sklearn.metrics import classification_report

print(classification_report(y_test, y_pred))
```

Métrica	Fórmula	Cuándo priorizar
Accuracy	Correctos / Total	Clases balanceadas
Precision	TP / (TP + FP)	Costo alto de falsos positivos
Recall	TP / (TP + FN)	Costo alto de falsos negativos
F 1	2 × (P × R) / (P + R)	Balance entre P y R

En BankChurn: Priorizo **Recall** (no queremos perder clientes que harán churn).

Pregunta 20: Curva ROC y AUC

¿Qué es AUC-ROC?

Respuesta:

```
from sklearn.metrics import roc_auc_score, roc_curve

# AUC: Área bajo la curva ROC
auc = roc_auc_score(y_test, y_pred_proba[:, 1])
print(f"AUC: {auc:.3f}")
```

Interpretación:
- **AUC =** 1.0 : Clasificador perfecto
- **AUC =** 0.5 : Clasificador aleatorio
- **AUC >** 0.8 : Generalmente bueno

Ventaja: Funciona bien con clases desbalanceadas.

3. Datos y Preprocesamiento (Preguntas 21-30)

Pregunta 21: Exploración de Datos

¿Qué haces primero cuando recibes un dataset?

Respuesta:

```
import pandas as pd

df = pd.read_csv("data.csv")

# 1. Dimensiones
print(f"Shape: {df.shape}")  # (filas, columnas)

# 2. Tipos de datos
print(df.dtypes)

# 3. Missing values
print(df.isnull().sum())

# 4. Estadísticas básicas
print(df.describe())

# 5. Primeras filas
print(df.head())

# 6. Target distribution
print(df['target'].value_counts(normalize=True))
```

Pregunta 22: Detección de Outliers

¿Cómo detectas outliers?

Respuesta:

```
import numpy as np

# Método IQR
Q1 = df['Balance'].quantile(0.25)
Q3 = df['Balance'].quantile(0.75)
IQR = Q3 - Q1

lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR

outliers = df[(df['Balance'] < lower) | (df['Balance'] > upper)]
print(f"Outliers: {len(outliers)}")
```

Qué hacer con outliers:
1. Verificar si son errores de datos → corregir
2. Si son legítimos → considerar winsorization o mantener
3. Para modelos sensibles → eliminar o transformar

Pregunta 23: Correlación

¿Cómo identificas features correlacionadas?

Respuesta:

```
import seaborn as sns
import matplotlib.pyplot as plt

# Matriz de correlación
corr = df.corr()

# Heatmap
plt.figure(figsize=(10, 8))
sns.heatmap(corr, annot=True, cmap='coolwarm')
plt.show()

# Features altamente correlacionadas (>0.9)
high_corr = (corr.abs() > 0.9) & (corr != 1.0)
```

¿Por qué importa? Features muy correlacionadas son redundantes → considerar eliminar una.

Pregunta 24: Desbalance de Clases

¿Qué haces cuando tienes 95% clase A y 5% clase B?

Respuesta:

```
# 1. Cambiar métrica (no usar accuracy)
from sklearn.metrics import f1_score, recall_score

# 2. Class weights
from sklearn.linear_model import LogisticRegression
model = LogisticRegression(class_weight='balanced')

# 3. Oversampling (SMOTE)
from imblearn.over_sampling import SMOTE
X_res, y_res = SMOTE().fit_resample(X, y)

# 4. Undersampling
from imblearn.under_sampling import RandomUnderSampler
```

En BankChurn: 80/20 balance → usamos `class_weight='balanced'` y F1.

Pregunta 25: Feature Selection

¿Cómo seleccionas features importantes?

Respuesta:

```
from sklearn.ensemble import RandomForestClassifier

# 1. Feature importance de RF
rf = RandomForestClassifier().fit(X, y)
importances = pd.DataFrame({
    'feature': X.columns,
    'importance': rf.feature_importances_
}).sort_values('importance', ascending=False)

# 2. Correlación con target
correlations = df.corr()['target'].abs().sort_values(ascending=False)

# 3. SelectKBest
from sklearn.feature_selection import SelectKBest, f_classif
selector = SelectKBest(f_classif, k=10)
X_selected = selector.fit_transform(X, y)
```

Pregunta 26: Data Leakage

¿Qué es data leakage y cómo evitarlo?

Respuesta:

Data leakage = cuando información del futuro o del target filtra al entrenamiento.

```
# ❌ MAL: fit scaler en TODO antes de split
scaler.fit(X)  # Ve datos de test
X_train, X_test = train_test_split(X)

# ✅ BIEN: fit solo en train
X_train, X_test = train_test_split(X)
scaler.fit(X_train)  # Solo ve train
X_train = scaler.transform(X_train)
X_test = scaler.transform(X_test)
```

En el portafolio: Usamos Pipeline de sklearn que maneja esto automáticamente.

Pregunta 27: Pipelines de sklearn

¿Por qué usar Pipeline?

Respuesta:

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier

pipe = Pipeline([
    ('scaler', StandardScaler()),
    ('model', RandomForestClassifier())
])

# Un solo fit/predict
pipe.fit(X_train, y_train)
y_pred = pipe.predict(X_test)
```

Beneficios:

1. **Evita leakage**: fit solo en train automáticamente
2. **Código limpio**: Todo en un objeto
3. **Fácil deploy**: `joblib.dump(pipe, 'model.joblib')`
4. **Reproducibilidad**: Mismo proceso siempre

Pregunta 28: Guardado de Modelos

¿Cómo guardas y cargas un modelo entrenado?

Respuesta:

```
import joblib

# Guardar
joblib.dump(model, 'artifacts/model.joblib')
```

```
# Cargar
model = joblib.load('artifacts/model.joblib')

# Usar
prediction = model.predict(new_data)
```

En producción (FastAPI):

```
@lru_cache()
def load_model():
    return joblib.load("artifacts/pipeline.joblib")
```

Pregunta 29: Validación de Datos

¿Cómo validas que los datos de entrada son correctos?

Respuesta:

```
from pydantic import BaseModel, Field, validator

class CustomerInput(BaseModel):
    credit_score: int = Field(ge=300, le=850)
    age: int = Field(ge=18, le=100)
    geography: str

    @validator('geography')
    def geography_valid(cls, v):
        valid = ['France', 'Germany', 'Spain']
        if v not in valid:
            raise ValueError(f'Must be one of {valid}')
        return v
```

Beneficios: Errores claros antes de llegar al modelo.

Pregunta 30: Reproducibilidad

¿Cómo garantizas que tu experimento sea reproducible?

Respuesta:

```
import random
import numpy as np

# 1. Fijar seeds
SEED = 42
random.seed(SEED)
np.random.seed(SEED)

# 2. En modelos
model = RandomForestClassifier(random_state=SEED)

# 3. En split
train_test_split(X, y, random_state=SEED)

# 4. Documentar versiones
# requirements.txt o pyproject.toml con versiones fijas
```

4. Git y Herramientas (Preguntas 31-40)

Pregunta 31: Git Básico

¿Cuál es el flujo básico de Git?

Respuesta:

```
# 1. Ver estado
git status

# 2. Añadir cambios
git add .                     # Todo
git add archivo.py            # Específico
```

```
# 3. Commit
git commit -m "feat: add preprocessing step"

# 4. Push
git push origin main

# 5. Pull (obtener cambios)
git pull origin main
```

Pregunta 32: Branches

¿Por qué usar branches?

Respuesta:

```
# Crear branch
git checkout -b feature/add-validation

# Trabajar...
git add .
git commit -m "feat: add pydantic validation"

# Push branch
git push origin feature/add-validation

# Crear Pull Request en GitHub
# Después de aprobar, merge a main
```

Razones:
- Aislar cambios
- Revisar código antes de merge
- Mantener main siempre funcional

Pregunta 33: .gitignore

¿Qué debe ir en .gitignore?

Respuesta:

```
# Datos (grandes, sensibles)
data/
*.csv
*.parquet

# Artefactos
artifacts/
*.joblib
*.pkl

# Entornos
.venv/
__pycache__/

# IDEs
.vscode/
.idea/

# Logs
*.log
mlruns/
```

Regla: No subir datos grandes, artefactos binarios, ni secretos.

Pregunta 34: Requirements

¿Cómo manejas dependencias?

Respuesta:

```
# Crear requirements.txt
pip freeze > requirements.txt

# Mejor: usar pip-tools
pip-compile requirements.in > requirements.txt
```

```
# Instalar
pip install -r requirements.txt

# Moderno: pyproject.toml
pip install -e ".[dev]"
```

Pregunta 35: Makefile

¿Para qué sirve un Makefile?

Respuesta:

```
.PHONY: install test train

install:
    pip install -e ".[dev]"

test:
    pytest tests/ -v --cov=src

train:
    python main.py --config configs/config.yaml

lint:
    ruff check src/
```

Uso:

```
make install
make test
make train
```

Beneficio: Comandos estándar, documentados, fáciles de recordar.

Pregunta 36: pytest Básico

¿Cómo escribes un test básico?

Respuesta:

```
# tests/test_data.py
import pytest
import pandas as pd

def test_load_data():
    df = pd.read_csv("data/raw/sample.csv")
    assert len(df) > 0
    assert "target" in df.columns

def test_no_nulls_in_target():
    df = pd.read_csv("data/raw/sample.csv")
    assert df["target"].isnull().sum() == 0

# Ejecutar
# pytest tests/test_data.py -v
```

Pregunta 37: Estructura de Proyecto

¿Cómo organizas un proyecto ML?

Respuesta:

```
mi-proyecto/
├── src/miproyecto/     # Código fuente
│   ├── __init__.py
│   ├── config.py       # Configuración
│   ├── data.py         # Carga de datos
│   ├── features.py     # Feature engineering
│   └── training.py     # Entrenamiento
├── app/                # APIs
├── tests/              # Tests
├── configs/            # YAML configs
├── data/raw/           # Datos
```

```
├── artifacts/          # Modelos guardados
├── pyproject.toml      # Dependencias
├── Makefile
└── README.md
```

Pregunta 38: README

¿Qué debe tener un buen README?

Respuesta:

```
# Nombre del Proyecto

## Descripción
Qué hace el proyecto, problema que resuelve.

## Instalación
```bash
pip install -e .
```

Uso Rápido

```
from miproyecto import predict
result = predict(data)
```

Estructura

Árbol de directorios.

Tests

```
make test
```

Autor

Nombre, contacto.

```
## Pregunta 39: Docker Básico
**¿Qué es Docker y por qué usarlo?**

### Respuesta:
Docker empaqueta tu aplicación con todas sus dependencias.

```dockerfile
FROM python:3.11-slim

WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt

COPY . .
CMD ["python", "main.py"]
```

```
# Construir
docker build -t mi-app .

# Ejecutar
docker run mi-app
```

Beneficio: "Funciona en mi máquina" → Funciona en cualquier máquina.

Pregunta 40: APIs Básicas

¿Qué es una API REST?

Respuesta:

API = Interfaz para que otros programas usen tu código.

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/health")
def health():
    return {"status": "ok"}

@app.post("/predict")
def predict(data: dict):
    # Usar modelo
    return {"prediction": result}
```

```
# Ejecutar
uvicorn app:app --reload

# Probar
curl http://localhost:8000/health
```

5. Práctica con el Portafolio (Preguntas 41-50)

Pregunta 41: Describir el Portafolio

Cuéntame sobre el portafolio.

Respuesta:

"Es un portafolio de MLOps con 3 proyectos production-ready:

1. **BankChurn-Predictor**: Clasificación binaria para predecir churn de clientes bancarios. Pipeline sklearn unificado, FastAPI, 79% coverage.
2. **CarVision-Market-Intelligence**: Regresión para predecir precios de autos usados. FeatureEngineer centralizado, Streamlit dashboard.
3. **TelecomAI**: Clasificación multiclase para segmentación de clientes de telecom.

Todos siguen las mismas prácticas: estructura src/, Pydantic para configs, pytest, GitHub Actions CI."

Pregunta 42: Ejecutar el Proyecto

¿Cómo ejecuto BankChurn?

Respuesta:

```
# 1. Clonar
git clone https://github.com/duqueom/ML-MLOps-Portfolio.git
cd ML-MLOps-Portfolio/BankChurn-Predictor

# 2. Crear entorno
python -m venv .venv
source .venv/bin/activate

# 3. Instalar
pip install -e ".[dev]"

# 4. Entrenar
python main.py --config configs/config.yaml

# 5. API
uvicorn app.fastapi_app:app --reload

# 6. Tests
pytest tests/ -v
```

Pregunta 43: Entender el Pipeline

¿Cómo funciona el pipeline de BankChurn?

Respuesta:

```
# 1. Cargar config
config = BankChurnConfig.from_yaml("configs/config.yaml")
```

```
# 2. Cargar datos
df = pd.read_csv(config.data.raw_path)

# 3. Crear trainer
trainer = Trainer(config)

# 4. Entrenar (dentro crea Pipeline sklearn)
trainer.fit(X, y)
# Pipeline = [preprocessor, model]
# preprocessor = ColumnTransformer(numeric_pipe, categorical_pipe)

# 5. Evaluar
metrics = trainer.evaluate(X_test, y_test)

# 6. Guardar
trainer.save("artifacts/")
```

Pregunta 44: Modificar el Código

¿Cómo añadirías una nueva feature?

Respuesta:

```
# 1. En config.yaml, añadir columna
features:
  numerical:
    - CreditScore
    - Age
    - NewFeature  # Nueva

# 2. Si requiere transformación, editar FeatureEngineer
class FeatureEngineer:
    def transform(self, X):
        X['NewFeature'] = X['Col1'] / X['Col2']
        return X

# 3. Agregar test
def test_new_feature():
    fe = FeatureEngineer()
    result = fe.transform(sample_df)
    assert 'NewFeature' in result.columns

# 4. Ejecutar tests
pytest tests/test_features.py -v
```

Pregunta 45: Leer un Error

Este código falla. ¿Por qué?

```
X_train = scaler.fit_transform(X_train)
X_test = scaler.fit_transform(X_test)
```

Respuesta:

Problema: `fit_transform` en test causa data leakage.

```
# ✅ Correcto
X_train = scaler.fit_transform(X_train)  # fit + transform
X_test = scaler.transform(X_test)        # solo transform
```

El scaler debe aprender (fit) solo de training data.

Pregunta 46: Interpretar Métricas

El modelo tiene accuracy 95% pero el negocio no está contento. ¿Por qué?

Respuesta:

Posibles razones:

1. **Clases desbalanceadas**: Si 95% son clase 0, predecir siempre 0 da 95% accuracy pero es inútil.
2. **Métrica incorrecta**: El negocio necesita recall (no perder churners) pero optimizaste accuracy.
3. **Falsos negativos costosos**: Cada cliente que hace churn y no detectamos cuesta $X.

Solución: Usar F 1 , recall, o una métrica de negocio (costo).

Pregunta 47: Configuración YAML

¿Por qué usar archivos YAML para configuración?

Respuesta:

```
# configs/config.yaml
model:
  type: "random_forest"
  n_estimators: 100
  max_depth: 10

data:
  raw_path: "data/raw/Churn.csv"
  test_size: 0.2

training:
  random_state: 42
```

Ventajas:
1. **Separación**: Cambiar parámetros sin tocar código
2. **Versionable**: Git puede trackear cambios
3. **Legible**: Fácil de entender
4. **Reproducibilidad**: Guardar config de cada experimento

Pregunta 48: CI/CD Básico

¿Qué hace el workflow de GitHub Actions?

Respuesta:

```
# .github/workflows/ci.yml
name: CI
on: [push, pull_request]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
      - run: pip install -e ".[dev]"
      - run: pytest tests/ -v
```

Flujo:
1. Push código → GitHub Actions se activa
2. Crea máquina virtual limpia
3. Instala dependencias
4. Ejecuta tests
5. Reporta pass/fail

Pregunta 49: Debugging en Producción

El API retorna error 5 0 0. ¿Cómo lo depuras?

Respuesta:

```
# 1. Ver logs
uvicorn app:app --log-level debug

# 2. Añadir logging
import logging
logging.basicConfig(level=logging.DEBUG)

@app.post("/predict")
def predict(data: Input):
    logging.debug(f"Input: {data}")
    try:
        result = model.predict(...)
        logging.debug(f"Result: {result}")
        return result
```

```
    except Exception as e:
        logging.error(f"Error: {e}")
        raise

# 3. Probar localmente
curl -X POST http://localhost:8000/predict \
  -H "Content-Type: application/json" \
  -d '{"credit_score": 650, ...}'
```

Pregunta 50: Próximos Pasos

¿Qué aprenderías después de este portafolio?

Respuesta:

"Con las bases del portafolio, me gustaría profundizar en:

1. **MLflow/Experiment Tracking**: Ya está configurado, pero quiero usarlo más para comparar experimentos sistemáticamente.
2. **Docker avanzado**: Optimizar imágenes, multi-stage builds.
3. **Testing más robusto**: Añadir tests de integración, property-based testing.
4. **Kubernetes básico**: Entender cómo escalar los servicios.
5. **Monitoreo en producción**: Detectar drift, alertas.

El portafolio me dio la base; ahora quiero profundizar en cada área."

📚 Recursos para Preparación

Módulos de la Guía Relacionados

Pregunta	Módulo
Python básico	01_PYTHON_MODERNO.md
ML fundamentos	07_SKLEARN_PIPELINES.md, 08_INGENIERIA_FEATURES.md
Git	05_GIT_PROFESIONAL.md
Testing	11_TESTING_ML.md
APIs	14_FASTAPI.md

Checklist Pre-Entrevista

- [ ] Puedo ejecutar `make install && make test` en BankChurn
- [ ] Entiendo qué hace cada archivo en `src/bankchurn/`
- [ ] Sé explicar train/test split y por qué importa
- [ ] Puedo leer y modificar el `config.yaml`
- [ ] Entiendo el flujo Git básico

¡Éxito en tu entrevista! 🚀

Recuerda: ser Junior significa estar aprendiendo. Muestra curiosidad y ganas de aprender.

SKLEARN PIPELINES

GUÍA MLOPS V5.0

PORTFOLIO EDITION

07. sklearn Pipelines: El Corazón de MLOps

🎯 Objetivo del Módulo

Dominar el patrón más importante de ML profesional: **pipelines unificados** que garantizan reproducibilidad desde entrenamiento hasta producción.

```
╔═══════════════════════════════════════════════════════════════════╗
│                                                                   │
│  🚨 EL ERROR #1 EN PRODUCCIÓN ML:                                 │
│                                                                   │
│  Entrenar con una transformación, servir con otra.                │
│                                                                   │
│  Ejemplo real:                                                    │
│  • Training: StandardScaler fitted en train set (mean=45000, std=20000)  │
│  • Production: StandardScaler fitted en cada request (mean=???, std=???) │
│  • Resultado: Predicciones COMPLETAMENTE diferentes               │
│                                                                   │
│  🛡️ LA SOLUCIÓN: Pipeline unificado que guarda TODO junto         │
│                                                                   │
╚═══════════════════════════════════════════════════════════════════╝
```

📋 Contenido

1. ¿Por Qué Pipelines?
2. ColumnTransformer: Transformaciones Paralelas
3. Custom Transformers
4. Pipeline Completo: Código Real
5. Ejercicios Prácticos

7.1 ¿Por Qué Pipelines?

La Analogía de la Línea de Ensamblaje

```
╔═══════════════════════════════════════════════════════════════════╗
│  🏭 IMAGINA UNA FÁBRICA DE AUTOS:                                 │
│                                                                   │
│  SIN LÍNEA DE ENSAMBLAJE (código suelto):                         │
│  • Trabajador 1 pone ruedas, pero a veces se le olvida            │
│  • Trabajador 2 pinta, pero usa colores diferentes cada día       │
│  • Trabajador 3 instala motor, pero a veces del modelo equivocado │
│  • Resultado: Cada auto es diferente, imposible de mantener       │
│                                                                   │
│  CON LÍNEA DE ENSAMBLAJE (Pipeline):                              │
│  • Paso 1: Chasis → Paso 2: Motor → Paso 3: Pintura → Paso 4: Ruedas │
│  • Cada paso está definido y es SIEMPRE igual                     │
│  • El proceso completo es una sola unidad                         │
│  • Resultado: Todos los autos son consistentes                    │
│                                                                   │
│  sklearn Pipeline = Línea de ensamblaje para ML                   │
╚═══════════════════════════════════════════════════════════════════╝
```

El Problema Real: Training-Serving Skew

```
# ❌ CÓDIGO PROBLEMÁTICO (muy común en notebooks convertidos a producción)

# === ENTRENAMIENTO ===
from sklearn.preprocessing import StandardScaler, OneHotEncoder

# Ajustar scaler en datos de entrenamiento
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train[num_cols])  # ← fit aquí

encoder = OneHotEncoder()
X_train_encoded = encoder.fit_transform(X_train[cat_cols])  # ← fit aquí

# Entrenar modelo
model = RandomForestClassifier()
model.fit(X_train_processed, y_train)

# Guardar modelo... pero ¿y el scaler? ¿y el encoder?
```

```
joblib.dump(model, "model.pkl")  # ← Solo guarda el modelo!

# === PRODUCCIÓN (meses después, otro desarrollador) ===
model = joblib.load("model.pkl")

# ¿Cómo transformo los datos nuevos?
# 🤷 No tengo el scaler ni el encoder fitted
# 🤷 Incluso si los tuviera, ¿cómo sé qué columnas usar?
# 🤷 ¿Era StandardScaler o MinMaxScaler?

# "Solución" del desarrollador desesperado:
scaler = StandardScaler()
X_new_scaled = scaler.fit_transform(X_new[num_cols])  # ← fit en datos NUEVOS!
# ⚠️ Ahora mean y std son DIFERENTES a los de entrenamiento
# ⚠️ Las predicciones son BASURA

# ============================================================================
# ✅ SOLUCIÓN: Pipeline Unificado
# ============================================================================

# === ENTRENAMIENTO ===
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer

# Definir pipeline completo
pipeline = Pipeline([
    ('preprocessor', ColumnTransformer([
        ('num', StandardScaler(), num_cols),
        ('cat', OneHotEncoder(handle_unknown='ignore'), cat_cols)
    ])),
    ('model', RandomForestClassifier())
])

# Un solo fit entrena TODO
pipeline.fit(X_train, y_train)

# Guardar TODO junto
joblib.dump(pipeline, "pipeline.pkl")  # ← Scaler + Encoder + Model

# === PRODUCCIÓN ===
pipeline = joblib.load("pipeline.pkl")

# Una sola llamada hace TODO (con los parámetros de entrenamiento)
predictions = pipeline.predict(X_new)  # ← Transforma Y predice

# ✅ El scaler usa mean/std del entrenamiento
# ✅ El encoder conoce las categorías del entrenamiento
# ✅ Las predicciones son consistentes
```

7.2 ColumnTransformer: Transformaciones Paralelas

El Problema: Diferentes Columnas, Diferentes Tratamientos

```
Datos de un banco:
┌──────────────┬────────────┬──────────┬──────────┬─────────┐
│ CreditScore │ Geography │ Gender  │   Age   │ Balance│
├──────────────┼────────────┼──────────┼──────────┼─────────┤
│     619     │  France   │  Female │    42   │  10000 │
│     608     │   Spain   │  Female │    41   │  83808 │
│     502     │  France   │  Female │    42   │      0 │
└──────────────┴────────────┴──────────┴──────────┴─────────┘

Columnas numéricas (CreditScore, Age, Balance):
→ StandardScaler: normalizar a mean=0, std=1

Columnas categóricas (Geography, Gender):
→ OneHotEncoder: convertir a columnas binarias
```

ColumnTransformer: La Solución Elegante

```
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline
```

```
# Definir qué columnas son de cada tipo
num_cols = ["CreditScore", "Age", "Tenure", "Balance", "NumOfProducts", "EstimatedSalary"]
cat_cols = ["Geography", "Gender"]

# Pipeline para numéricas: Imputar NaN → Escalar
num_pipeline = Pipeline([
    ('imputer', SimpleImputer(strategy='median')),  # NaN → mediana
    ('scaler', StandardScaler())                     # Normalizar
])

# Pipeline para categóricas: Imputar NaN → One-Hot
cat_pipeline = Pipeline([
    ('imputer', SimpleImputer(strategy='constant', fill_value='Unknown')),
    ('encoder', OneHotEncoder(handle_unknown='ignore'))  # Categorías nuevas → ignorar
])

# ColumnTransformer: Aplica cada pipeline a sus columnas
preprocessor = ColumnTransformer(
    transformers=[
        ('num', num_pipeline, num_cols),  # (nombre, transformer, columnas)
        ('cat', cat_pipeline, cat_cols)
    ],
    remainder='drop'  # Columnas no listadas se eliminan
)

# Resultado: Un solo objeto que sabe transformar todo
X_processed = preprocessor.fit_transform(X_train)
```

Visualización del Flujo

```
                        ColumnTransformer
                              |
              ┌───────────────┼───────────────┐
              |               |               |
              ▼               ▼               ▼
        num_pipeline    cat_pipeline    remainder
              |               |               |
              |               |               |
    ┌─────────┴─────────┐     |               |
    |                   |     |               |
    ▼                   ▼     ▼               ▼
┌─────────┐         ┌─────────┐ ┌─────────┐   drop
|Imputer  |         | Scaler  | | Imputer |
|(median) |         |         | |(Unknown)|
└─────────┘         └─────────┘ └─────────┘
                                     |
                                     |
                                     ▼
                                 ┌─────────┐
                                 | OneHot  |
                                 | Encoder |
                                 └─────────┘
                                     |
              ┌──────────────────────┼──────────────────────┐
              |                      |                      |
              ▼                      ▼                      ▼
    [6 columnas numéricas]  [3 Geography cols]  [2 Gender cols]
         escaladas            (France, Spain,     (Female, Male)
                               Germany)

    Output: 11 columnas totales (6 + 3 + 2)
```

7.3 Custom Transformers: Tu Superpoder

¿Cuándo Crear un Custom Transformer?

```
╔═══════════════════════════════════════════════════════════════════╗
║  Crea un Custom Transformer cuando:                               ║
║                                                                   ║
║  ✅ Necesitas feature engineering específico del dominio           ║
║  ✅ La transformación debe aplicarse igual en train y producción   ║
║  ✅ sklearn no tiene un transformer que haga lo que necesitas      ║
║                                                                   ║
║  Ejemplos del portafolio:                                         ║
```

```
╟  • CarVision: Calcular vehicle_age desde model_year                    ╢
╟  • CarVision: Extraer brand desde model                                ╢
╟  • BankChurn: Resampling para clases desbalanceadas                    ╢
╚═══════════════════════════════════════════════════════════════════════╝
```

Ejemplo 1: FeatureEngineer (CarVision)

```
# src/carvision/features.py - Código REAL del portafolio

from __future__ import annotations

from typing import Optional

import pandas as pd
from sklearn.base import BaseEstimator, TransformerMixin


class FeatureEngineer(BaseEstimator, TransformerMixin):
    """
    Centralized feature engineering to ensure consistency across
    Training, Inference, and Analysis.

    Este transformer garantiza que las mismas transformaciones
    se apliquen en:
    1. Entrenamiento (training.py)
    2. Inferencia API (fastapi_app.py)
    3. Dashboard (streamlit_app.py)

    Attributes
    ----------
    current_year : int, optional
        Año para calcular vehicle_age. Si None, usa año actual.

    Examples
    --------
    >>> fe = FeatureEngineer(current_year=2024)
    >>> df_transformed = fe.fit_transform(df)
    >>> print(df_transformed.columns)
    # Incluye: vehicle_age, brand (derivadas de model_year y model)
    """

    def __init__(self, current_year: Optional[int] = None):
        self.current_year = current_year

    def fit(self, X: pd.DataFrame, y: pd.DataFrame = None) -> "FeatureEngineer":
        """Fit no hace nada (stateless transformer)."""
        # Este transformer es stateless: no aprende nada de los datos
        # Solo necesita fit() para ser compatible con Pipeline
        return self

    def transform(self, X: pd.DataFrame) -> pd.DataFrame:
        """Aplica feature engineering.

        Features creadas:
        - vehicle_age: current_year - model_year
        - brand: primera palabra de model
        - price_per_mile: price / odometer (solo si price existe)
        """
        X = X.copy()  # ← Nunca modificar el input original

        # Usar año configurado o año actual
        year = self.current_year or pd.Timestamp.now().year

        # Feature: Edad del vehículo
        if "model_year" in X.columns:
            X["vehicle_age"] = year - X["model_year"]

        # Feature: Marca (primera palabra del modelo)
        if "model" in X.columns:
            X["brand"] = X["model"].astype(str).str.split().str[0]

        # Features derivadas (solo en training, no en inferencia)
        # Porque price no está disponible en inferencia
        if "odometer" in X.columns and "price" in X.columns:
```

```
            X["price_per_mile"] = X["price"] / (X["odometer"] + 1)

        return X

    # Métodos opcionales para mejor introspección
    def get_feature_names_out(self, input_features=None):
        """Retorna nombres de features de salida."""
        base = list(input_features) if input_features else []
        return base + ["vehicle_age", "brand"]
```

Ejemplo 2: ResampleClassifier (BankChurn)

```
# src/bankchurn/models.py - Código REAL del portafolio

from __future__ import annotations

import numpy as np
import pandas as pd
from sklearn.base import BaseEstimator, ClassifierMixin
from sklearn.utils.validation import check_is_fitted


class ResampleClassifier(BaseEstimator, ClassifierMixin):
    """Custom classifier with resampling for imbalanced datasets.

    Implementa oversampling (SMOTE), undersampling, y class weighting
    para mejorar performance en clasificación desbalanceada.

    Este wrapper permite:
    1. Probar diferentes estrategias de resampling fácilmente
    2. Mantener la interfaz sklearn estándar (fit/predict)
    3. Ser parte de un Pipeline (incluyendo GridSearchCV)

    Parameters
    ----------
    estimator : estimator object, optional
        Clasificador base. Si None, usa LogisticRegression.
    strategy : {"none", "oversample", "undersample", "class_weight"}
        Estrategia de resampling:
        - "none": Sin resampling
        - "oversample": SMOTE oversampling de clase minoritaria
        - "undersample": Undersampling de clase mayoritaria
        - "class_weight": Balanceo automático de pesos
    random_state : int, default=42
        Semilla para reproducibilidad.

    Examples
    --------
    >>> clf = ResampleClassifier(
    ...     estimator=RandomForestClassifier(),
    ...     strategy="oversample",
    ...     random_state=42
    ... )
    >>> clf.fit(X_train, y_train)
    >>> predictions = clf.predict(X_test)
    """

    def __init__(
        self,
        estimator: BaseEstimator | None = None,
        strategy: str = "none",
        random_state: int = 42,
    ) -> None:
        self.estimator = estimator
        self.strategy = strategy
        self.random_state = random_state

    def fit(self, X: np.ndarray, y: np.ndarray) -> "ResampleClassifier":
        """Entrena el clasificador con resampling opcional."""
        from sklearn.linear_model import LogisticRegression

        # Inicializar estimador si no se proporcionó
        if self.estimator is None:
            self.estimator_ = LogisticRegression(random_state=self.random_state)
```

```
        else:
            # Clonar para no modificar el original
            from sklearn.base import clone
            self.estimator_ = clone(self.estimator)

        # Guardar clases (requerido por sklearn)
        self.classes_ = np.unique(y)

        # Aplicar estrategia de resampling
        X_resampled, y_resampled = self._apply_resampling(X, y)

        # Entrenar estimador base
        self.estimator_.fit(X_resampled, y_resampled)

        return self

    def _apply_resampling(
        self, X: np.ndarray, y: np.ndarray
    ) -> tuple[np.ndarray, np.ndarray]:
        """Aplica la estrategia de resampling."""
        if self.strategy == "none":
            return X, y

        elif self.strategy == "oversample":
            try:
                from imblearn.over_sampling import SMOTE
                smote = SMOTE(random_state=self.random_state)
                return smote.fit_resample(X, y)
            except ImportError:
                # Si imblearn no está instalado, ignorar
                return X, y

        elif self.strategy == "undersample":
            try:
                from imblearn.under_sampling import RandomUnderSampler
                rus = RandomUnderSampler(random_state=self.random_state)
                return rus.fit_resample(X, y)
            except ImportError:
                return X, y

        elif self.strategy == "class_weight":
            # No modifica datos, el estimador maneja los pesos
            if hasattr(self.estimator_, 'class_weight'):
                self.estimator_.set_params(class_weight='balanced')
            return X, y

        else:
            raise ValueError(f"Unknown strategy: {self.strategy}")

    def predict(self, X: np.ndarray) -> np.ndarray:
        """Predice clases."""
        check_is_fitted(self, ['estimator_', 'classes_'])
        return self.estimator_.predict(X)

    def predict_proba(self, X: np.ndarray) -> np.ndarray:
        """Predice probabilidades."""
        check_is_fitted(self, ['estimator_', 'classes_'])
        return self.estimator_.predict_proba(X)
```

La Plantilla: Crea Tu Propio Transformer

```
from sklearn.base import BaseEstimator, TransformerMixin

class MiTransformer(BaseEstimator, TransformerMixin):
    """
    Plantilla para crear transformers custom.

    REGLAS IMPORTANTES:
    1. __init__ solo guarda parámetros (no computa nada)
    2. fit() aprende de los datos (puede ser no-op)
    3. transform() aplica la transformación
    4. Nunca modificar input, siempre X.copy()
    """
```

```
    def __init__(self, param1: str = "default", param2: int = 10):
        # Solo guardar parámetros, NO computar nada
        self.param1 = param1
        self.param2 = param2

    def fit(self, X, y=None):
        """Aprende de los datos (opcional).

        Ejemplos de qué aprender:
        - Media/std para normalización
        - Vocabulario para encoding
        - Umbrales para binning
        """
        # Si el transformer es stateless, solo retorna self
        # Si aprende algo:
        # self.learned_param_ = compute_something(X)
        return self

    def transform(self, X):
        """Aplica la transformación."""
        X = X.copy()  # ← Siempre copiar
        # ... tu lógica de transformación ...
        return X
```

7.4 Pipeline Completo: Código Real

CarVision: El Pipeline de 3 Etapas

```
# src/carvision/training.py - Pipeline REAL del portafolio

from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.impute import SimpleImputer
from sklearn.ensemble import RandomForestRegressor

from src.carvision.features import FeatureEngineer

def build_pipeline(cfg: dict) -> Pipeline:
    """Construye el pipeline completo de CarVision.

    Estructura: Features → Preprocessing → Model

    Esta arquitectura de 3 etapas garantiza:
    1. Feature engineering consistente (FeatureEngineer)
    2. Preprocesamiento apropiado por tipo de columna (ColumnTransformer)
    3. Modelo entrenado con datos correctamente transformados
    """
    # Parámetros de configuración
    num_cols = cfg["preprocessing"]["numeric_features"]
    cat_cols = cfg["preprocessing"]["categorical_features"]
    dataset_year = cfg.get("dataset_year", 2024)
    rf_params = cfg["training"].get("random_forest_params", {})

    # Etapa 1: Feature Engineering
    feature_engineer = FeatureEngineer(current_year=dataset_year)

    # Etapa 2: Preprocessing (después de feature engineering)
    # Nota: Las columnas aquí son las que EXISTEN después del FeatureEngineer
    preprocessor = ColumnTransformer(
        transformers=[
            ('num', Pipeline([
                ('imputer', SimpleImputer(strategy='median')),
                ('scaler', StandardScaler())
            ]), num_cols),
            ('cat', Pipeline([
                ('imputer', SimpleImputer(strategy='most_frequent')),
                ('encoder', OneHotEncoder(handle_unknown='ignore', sparse_output=False))
            ]), cat_cols)
        ],
        remainder='drop'
    )

    # Etapa 3: Modelo
```

```
    model = RandomForestRegressor(**rf_params)

    # Pipeline completo: Una sola unidad entrenable/guardable
    pipeline = Pipeline([
        ('features', feature_engineer),    # Crea vehicle_age, brand
        ('pre', preprocessor),              # Escala y encoda
        ('model', model)                    # Predice
    ])

    return pipeline


# === USO ===
# Entrenamiento
pipeline = build_pipeline(config)
pipeline.fit(X_train, y_train)

# Guardar TODO junto
joblib.dump(pipeline, "artifacts/model.joblib")

# Producción
pipeline = joblib.load("artifacts/model.joblib")
price = pipeline.predict(X_new)  # Una llamada hace TODO
```

BankChurn: Pipeline con Ensemble

```
# src/bankchurn/training.py - Pipeline REAL del portafolio

def build_pipeline(self) -> Pipeline:
    """Construye el pipeline de BankChurn.

    Estructura:
    - Preprocessing: ColumnTransformer (num + cat)
    - Model: VotingClassifier o ResampleClassifier
    """
    # Columnas desde config
    num_cols = self.config.data.numerical_features
    cat_cols = self.config.data.categorical_features

    # Preprocessing
    preprocessor = ColumnTransformer(
        transformers=[
            ('num', Pipeline([
                ('imputer', SimpleImputer(strategy='median')),
                ('scaler', StandardScaler())
            ]), num_cols),
            ('cat', Pipeline([
                ('imputer', SimpleImputer(strategy='constant', fill_value='Unknown')),
                ('encoder', OneHotEncoder(handle_unknown='ignore'))
            ]), cat_cols)
        ]
    )

    # Modelo: Ensemble o single model
    if self.config.model.type == "ensemble":
        model = VotingClassifier(
            estimators=[
                ('lr', LogisticRegression(
                    **self.config.model.logistic_regression.dict()
                )),
                ('rf', RandomForestClassifier(
                    **self.config.model.random_forest.dict()
                ))
            ],
            voting=self.config.model.ensemble.voting,
            weights=self.config.model.ensemble.weights
        )
    else:
        # Con wrapper de resampling
        model = ResampleClassifier(
            estimator=RandomForestClassifier(
                **self.config.model.random_forest.dict()
            ),
            strategy=self.config.model.resampling_strategy,
```

```
            random_state=self.random_state
        )

    # Pipeline final
    return Pipeline([
        ('preprocessor', preprocessor),
        ('model', model)
    ])
```

7.5 Ejercicios Prácticos

Ejercicio 1: Construir un ColumnTransformer

```
# Datos de telecom:
# - calls: float (numérico)
# - minutes: float (numérico)
# - messages: int (numérico)
# - mb_used: float (numérico)
# - plan_type: str (categórico) - "basic", "premium"
# - region: str (categórico) - "north", "south", "east", "west"

# Tu tarea: Crea un ColumnTransformer que:
# 1. Escale las columnas numéricas con StandardScaler
# 2. Encode las columnas categóricas con OneHotEncoder
# 3. Maneje valores faltantes apropiadamente

num_cols = ["calls", "minutes", "messages", "mb_used"]
cat_cols = ["plan_type", "region"]

# Escribe tu código aquí:
preprocessor = ColumnTransformer(
    # ...
)
```

📝 Ver Solución

```
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.impute import SimpleImputer

num_cols = ["calls", "minutes", "messages", "mb_used"]
cat_cols = ["plan_type", "region"]

preprocessor = ColumnTransformer(
    transformers=[
        ('num', Pipeline([
            ('imputer', SimpleImputer(strategy='median')),
            ('scaler', StandardScaler())
        ]), num_cols),
        ('cat', Pipeline([
            ('imputer', SimpleImputer(strategy='most_frequent')),
            ('encoder', OneHotEncoder(handle_unknown='ignore', sparse_output=False))
        ]), cat_cols)
    ],
    remainder='drop'
)

# Verificar
print(f"Transformers: {[t[0] for t in preprocessor.transformers]}")
# Output: ['num', 'cat']
```

Ejercicio 2: Crear un Custom Transformer

```
# Tu tarea: Crea un transformer que calcule ratios de uso de telecom
#
# Features a crear:
# - minutes_per_call = minutes / (calls + 1)
# - mb_per_message = mb_used / (messages + 1)
# - total_usage = calls + messages + (mb_used / 1000)
#
# Requisitos:
# - Debe heredar de BaseEstimator y TransformerMixin
```

```
# - fit() debe retornar self
# - transform() debe retornar DataFrame con nuevas columnas

from sklearn.base import BaseEstimator, TransformerMixin

class TelecomFeatureEngineer(BaseEstimator, TransformerMixin):
    # Tu código aquí
    pass
```

📝 Ver Solución

```
from sklearn.base import BaseEstimator, TransformerMixin
import pandas as pd


class TelecomFeatureEngineer(BaseEstimator, TransformerMixin):
    """Feature engineering para datos de telecom."""

    def __init__(self):
        pass

    def fit(self, X: pd.DataFrame, y=None):
        """No aprende nada (stateless)."""
        return self

    def transform(self, X: pd.DataFrame) -> pd.DataFrame:
        """Crea features derivadas."""
        X = X.copy()

        # Minutos por llamada
        if "minutes" in X.columns and "calls" in X.columns:
            X["minutes_per_call"] = X["minutes"] / (X["calls"] + 1)

        # MB por mensaje
        if "mb_used" in X.columns and "messages" in X.columns:
            X["mb_per_message"] = X["mb_used"] / (X["messages"] + 1)

        # Uso total normalizado
        if all(col in X.columns for col in ["calls", "messages", "mb_used"]):
            X["total_usage"] = X["calls"] + X["messages"] + (X["mb_used"] / 1000)

        return X

    def get_feature_names_out(self, input_features=None):
        """Retorna nombres de features creadas."""
        return ["minutes_per_call", "mb_per_message", "total_usage"]


# Verificar
import pandas as pd

df = pd.DataFrame({
    "calls": [50, 100],
    "minutes": [200, 500],
    "messages": [100, 50],
    "mb_used": [5000, 10000]
})

fe = TelecomFeatureEngineer()
df_transformed = fe.fit_transform(df)
print(df_transformed.columns.tolist())
# Output incluye: minutes_per_call, mb_per_message, total_usage
```

Ejercicio 3: Pipeline Completo para TelecomAI

```
# Tu tarea: Construye un pipeline completo para TelecomAI
#
# Estructura:
# 1. TelecomFeatureEngineer (del ejercicio anterior)
# 2. ColumnTransformer para preprocessing
# 3. LogisticRegression como modelo
#
# El pipeline debe ser guardable con joblib
```

```
# Tu código aquí:
def build_telecom_pipeline():
    pass
```

📝 Ver Solución

```
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
import joblib


def build_telecom_pipeline(config: dict = None) -> Pipeline:
    """Construye pipeline completo para TelecomAI."""

    # Configuración por defecto
    if config is None:
        config = {
            "num_cols": ["calls", "minutes", "messages", "mb_used",
                         "minutes_per_call", "mb_per_message", "total_usage"],
            "cat_cols": [],
            "random_state": 42
        }

    num_cols = config["num_cols"]
    cat_cols = config.get("cat_cols", [])

    # Etapa 1: Feature Engineering
    feature_engineer = TelecomFeatureEngineer()

    # Etapa 2: Preprocessing
    transformers = [
        ('num', Pipeline([
            ('imputer', SimpleImputer(strategy='median')),
            ('scaler', StandardScaler())
        ]), num_cols)
    ]

    if cat_cols:
        transformers.append(
            ('cat', Pipeline([
                ('imputer', SimpleImputer(strategy='most_frequent')),
                ('encoder', OneHotEncoder(handle_unknown='ignore'))
            ]), cat_cols)
        )

    preprocessor = ColumnTransformer(
        transformers=transformers,
        remainder='drop'
    )

    # Etapa 3: Modelo
    model = LogisticRegression(
        random_state=config.get("random_state", 42),
        max_iter=1000
    )

    # Pipeline completo
    pipeline = Pipeline([
        ('features', feature_engineer),
        ('preprocessor', preprocessor),
        ('model', model)
    ])

    return pipeline


# Uso
pipeline = build_telecom_pipeline()
```

```
# pipeline.fit(X_train, y_train)
# joblib.dump(pipeline, "artifacts/model.joblib")
```

🧨 Errores habituales y cómo depurarlos en sklearn Pipelines

Los errores en este módulo rara vez son "fallos exóticos" del algoritmo; casi siempre son **desalineaciones** entre datos, columnas, transformers y cómo guardas/cargas el pipeline.

1) `ValueError: number of features does not match` (mismatch entre train e inference)

Síntomas típicos

- En entrenamiento todo bien, pero al predecir obtienes:
  `text ValueError: X has 15 features, but StandardScaler is expecting 12 features as input.`
- O bien errores de índice similares en `OneHotEncoder`.

Cómo identificarlo

- Verifica que usas **el mismo pipeline serializado** en training e inference:
- ¿Guardas y cargas `pipeline.pkl`/`model.joblib`, o solo el modelo suelto?
- Comprueba que las columnas de entrada en producción tienen el mismo orden y nombres que en entrenamiento.

Cómo corregirlo

- En el portafolio, **siempre** serializa el pipeline completo:
  `python joblib.dump(pipeline, "artifacts/model.joblib") pipeline = joblib.load("artifacts/model.joblib")`
- Asegúrate de que el orden y nombres de columnas que construyes en la API/Streamlit coincidan con las listas `num_cols` y `cat_cols` del pipeline.

2) Data leakage por features que usan el target (especialmente en CarVision)

Síntomas típicos

- Métricas en training/validation son **sospechosamente altas**, pero en producción caen.
- Features como `price_per_mile` o `price_category` dependen de la variable objetivo (`price`).

Cómo identificarlo

- Examina tu `FeatureEngineer` y lista de columnas que entran al modelo:
- ¿Estás incluyendo columnas derivadas del target en el `ColumnTransformer`?
- Revisa tu config (`cfg["preprocessing"]["numeric_features"]`, etc.) y confirma que solo incluyes features válidos.

Cómo corregirlo

- Asegúrate de que features que dependen del target **no** se usen como input del modelo.
- En CarVision, por ejemplo, `price_per_mile` y `price_category` se calculan solo para análisis, pero se excluyen de `num_cols` para el pipeline.

3) Custom transformers que modifican el input in-place o no respetan la API sklearn

Síntomas típicos

- Errores del tipo:
  `text TypeError: __init__() takes 1 positional argument but 2 were given`
  o
  `text AttributeError: 'MiTransformer' object has no attribute 'fit'`
- Comportamientos raros donde un transformer "ensucia" los datos para otros steps.

Cómo identificarlo

- Revisa que tu transformer:
- Herede de `BaseEstimator` y `TransformerMixin`.
- Tenga `__init__`, `fit`, `transform` con las firmas estándar.
- Use `X = X.copy()` dentro de `transform`.

Cómo corregirlo

- Usa la plantilla de este módulo (`MiTransformer`) como referencia.
- Evita lógica pesada en `__init__`; ahí solo se guardan parámetros.
- Añade tests unitarios simples (`fit_transform` sobre un `DataFrame` pequeño) para validar que mantiene columnas esperadas.

4) Pipelines diferentes en training y en la API

Síntomas típicos

- El pipeline usado en `training.py` no coincide con el que se monta en `fastapi_app.py` o `streamlit_app.py`.
- Bugs donde la API aplica transformaciones manuales **además** del pipeline.

Cómo identificarlo

- Busca en el proyecto si estás construyendo pipelines duplicados:
- En CarVision, la única fuente de verdad debe ser `build_pipeline` en `src/carvision/training.py`.
- La API y Streamlit solo deberían **cargar** el pipeline serializado, no recrearlo a mano.

Cómo corregirlo

- Centraliza la construcción del pipeline en una función (`build_pipeline` / `build_telecom_pipeline`).
- En la API/Streamlit, no replicar lógicas de preprocesado; limitarse a cargar y usar el pipeline.

5) Patrón general de debugging para pipelines

1. **Reproduce el error** con un input mínimo (1 – 2 filas de `DataFrame`).
2. **Inspecciona shapes y columnas** tras cada etapa:
3. Usa `pipeline.named_steps["pre"].transform(X_sample)` o similares.
4. **Verifica la serialización**: guarda, vuelve a cargar, y compara predicciones en un mismo batch.
5. **Conecta el problema** con el concepto del módulo:
6. Training-serving skew → pipeline parcial o mal serializado.
7. Mismatch de columnas → listas `num_cols`/`cat_cols` desincronizadas.
8. Transformers rotos → no respetan `fit`/`transform`.

Con este enfoque, los pipelines dejan de ser una “caja negra mágica” y se convierten en una línea de ensamblaje transparente y depurable.

✅ Checkpoint: ¿Completaste el Módulo?

Antes de continuar, verifica:

- [ ] Entiendes por qué los pipelines previenen training-serving skew
- [ ] Sabes usar ColumnTransformer para diferentes tipos de columnas
- [ ] Puedes crear un Custom Transformer con fit/transform
- [ ] Has construido un pipeline de 3 etapas (features → preprocessing → model)
- [ ] Puedes guardar y cargar un pipeline completo con joblib

🔗 ADR: Decisiones de Arquitectura

ADR-007: Pipeline Unificado Obligatorio

Contexto: Transformaciones separadas causan inconsistencias en producción.

Decisión: Todo el flujo (features → preprocessing → model) debe estar en un solo Pipeline.

Consecuencias:
- ✅ Una sola serialización guarda todo
- ✅ Imposible olvidar una transformación
- ✅ Reproducibilidad garantizada
- ❌ Más complejo de debuggear (caja negra)
- ❌ Requiere entender sklearn profundamente

ADR-008: Custom Transformers para Feature Engineering

Contexto: sklearn no tiene transformers para lógica de negocio específica.

Decisión: Crear FeatureEngineer como TransformerMixin.

Consecuencias:
- ✅ Reutilizable en train, API, y dashboard
- ✅ Testeable unitariamente
- ✅ Documentación clara de features derivadas
- ❌ Más código que escribir
- ❌ Requiere entender BaseEstimator/TransformerMixin

📦 Cómo se Usó en el Portafolio

Los pipelines sklearn son el corazón de los 3 proyectos del portafolio:

Pipeline Unificado de BankChurn

```
# BankChurn-Predictor/src/bankchurn/pipeline.py (estructura real)
def build_pipeline(config: BankChurnConfig) -> Pipeline:
    """Pipeline completo de 3 etapas."""
    return Pipeline([
        ('preprocessor', ColumnTransformer([
            ('num', Pipeline([
                ('imputer', SimpleImputer(strategy='median')),
                ('scaler', StandardScaler())
            ]), config.data.numerical_features),
            ('cat', Pipeline([
                ('imputer', SimpleImputer(strategy='constant', fill_value='missing')),
                ('encoder', OneHotEncoder(handle_unknown='ignore'))
            ]), config.data.categorical_features)
        ])),
        ('model', get_model(config))
    ])
```

FeatureEngineer de CarVision

```
# CarVision-Market-Intelligence/src/carvision/features.py
class FeatureEngineer(BaseEstimator, TransformerMixin):
    """Custom transformer para features de autos."""

    def __init__(self, current_year: int = None):
        self.current_year = current_year

    def fit(self, X, y=None):
        return self

    def transform(self, X):
        X = X.copy()
        # vehicle_age, brand, mileage_category, etc.
        return X
```

Archivos Clave por Proyecto

Proyecto	Pipeline	Features	Artefacto
BankChurn	`src/bankchurn/pipeline.py`	En preprocessor	`artifacts/pipeline.joblib`
CarVision	`src/carvision/pipeline.py`	`src/carvision/features.py`	`artifacts/pipeline.joblib`
TelecomAI	`src/telecomai/training.py`	En pipeline	`artifacts/model.joblib`

🔧 Ejercicio: Explora los Pipelines Reales

```
# 1. Ve a BankChurn y carga el pipeline
cd BankChurn-Predictor
python -c "
import joblib
pipe = joblib.load('artifacts/pipeline.joblib')
print('Steps:', [name for name, _ in pipe.steps])
print('Preprocessor:', pipe.named_steps['preprocessor'])
"

# 2. Inspecciona el FeatureEngineer de CarVision
cat CarVision-Market-Intelligence/src/carvision/features.py
```

💼 Consejos Profesionales

Recomendaciones para destacar en entrevistas y proyectos reales

Para Entrevistas

1. **¿Por qué Pipelines?**: Evitan data leakage, garantizan reproducibilidad, simplifican deployment.
2. **Custom Transformers**: Demuestra que puedes crear transformadores con `fit()` y `transform()`.
3. **ColumnTransformer**: Explica cómo aplicar diferentes transformaciones a diferentes columnas.

Para Proyectos Reales

Situación	Consejo
Features nuevas	Añade transformadores al pipeline, no código suelto
Debugging	Usa `pipeline.named_steps` para inspeccionar etapas
Producción	Serializa el pipeline completo, no solo el modelo
Testing	Testea cada transformador individualmente

Patrones Avanzados

- **FeatureUnion**: Combinar features de diferentes fuentes
- **Pipeline dentro de Pipeline**: Para transformaciones complejas
- **make_pipeline**: Sintaxis simplificada sin nombres
- **clone**: Para cross-validation sin modificar original

📺 Recursos Externos Recomendados

Ver RECURSOS_POR_MODULO.md para la lista completa.

🏷️	Recurso	Tipo
🔴	sklearn Pipelines - Data School	Video
🟡	Custom Transformers - ArjanCodes	Video

Documentación oficial:
- sklearn Pipeline
- ColumnTransformer
- Custom Transformers

🔗 Referencias del Glosario

Ver 21_GLOSARIO.md para definiciones de:
- **Pipeline**: Cadena de transformaciones + modelo
- **ColumnTransformer**: Procesamiento paralelo de columnas
- **Data Leakage**: Filtración de información del target

✅ Ejercicios

Ver EJERCICIOS.md - Módulo 07:
- 7.1: Pipeline básico con scaler + modelo
- 7.2: ColumnTransformer para features mixtas

INGENIERÍA DE FEATURES

GUÍA MLOPS V5.0

PORTFOLIO EDITION

08. Ingeniería de Features para ML

🎯 Objetivo del Módulo

Dominar la creación de features sin introducir **data leakage**, el error más peligroso y difícil de detectar en ML.

```
╔═══════════════════════════════════════════════════════════════╗
│  🚨 DATA LEAKAGE: El Asesino Silencioso de Modelos            │
│                                                               │
│  Tu modelo tiene 99% accuracy en validación...                │
│  ...pero 50% en producción.                                   │
│                                                               │
│  ¿Por qué? Porque durante el entrenamiento, el modelo "vio" información
│  que NO tendrá disponible cuando haga predicciones reales.    │
│                                                               │
│  Es como estudiar para un examen con las respuestas en la mano.
│  Sacas 100 en el examen de práctica, pero 0 en el real.       │
╚═══════════════════════════════════════════════════════════════╝
```

📋 Contenido

1. ¿Qué es Data Leakage?
2. Tipos de Leakage en ML
3. Caso Real: CarVision
4. Prevención con Pipelines
5. Feature Engineering Seguro

8.1 ¿Qué es Data Leakage?

La Analogía del Detective

```
╔═══════════════════════════════════════════════════════════╗
│  🔍 IMAGINA UN DETECTIVE RESOLVIENDO UN CASO:             │
│                                                           │
│  SIN LEAKAGE (correcto):                                  │
│  • El detective solo tiene las pistas disponibles AL MOMENTO del crimen
│  • Debe deducir quién es el culpable con información limitada
│  • Es difícil, pero es la realidad                        │
│                                                           │
│  CON LEAKAGE (trampa):                                    │
│  • El detective tiene acceso al informe FINAL del caso    │
│  • Ya sabe quién es el culpable antes de investigar       │
│  • "Resuelve" el caso fácilmente, pero no aprendió nada   │
│                                                           │
│  EN ML:                                                   │
│  • El modelo debe predecir usando SOLO información disponible
│    en el momento de la predicción                         │
│  • Si usas información del futuro o del target, es TRAMPA │
╚═══════════════════════════════════════════════════════════╝
```

Ejemplo Clásico: Predecir Precio con precio_per_mile

```
# ❌ LEAKAGE: Usando feature derivada del target

# Datos originales
df = pd.DataFrame({
    'price': [15000, 25000, 35000],      # Target a predecir
    'odometer': [80000, 50000, 20000],
})

# Feature engineering INCORRECTO
df['price_per_mile'] = df['price'] / df['odometer']  # ← LEAKAGE!

# ¿Por qué es leakage?
# price_per_mile = price / odometer
# Por lo tanto: price = price_per_mile * odometer
# El modelo "aprende" a multiplicar, no a predecir precios reales

# En producción:
```

```
# - No tienes el price (es lo que quieres predecir)
# - No puedes calcular price_per_mile
# - El modelo no sabe qué hacer
```

8.2 Tipos de Leakage

1. Target Leakage (Feature contiene información del target)

```
# ❌ MALO: Feature calculada con el target
df['price_category'] = pd.cut(df['price'], bins=[0, 10000, 50000, inf])

# El modelo aprende: "si price_category es 'alto', predice price alto"
# Pero en producción NO tienes price_category porque no tienes price
```

2. Train-Test Contamination (Datos de test "filtrados" a train)

```
# ❌ MALO: Normalizar ANTES de split
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)  # ← Usa estadísticas de TODO X
X_train, X_test = train_test_split(X_scaled)
# El scaler "vio" datos de test durante fit

# ✅ CORRECTO: Normalizar DESPUÉS de split
X_train, X_test = train_test_split(X)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)  # Solo train
X_test_scaled = scaler.transform(X_test)        # Usa params de train
```

3. Temporal Leakage (Usar información del futuro)

```
# ❌ MALO: Predecir churn de enero usando datos de febrero
df['avg_purchases_next_month'] = ...  # Información del futuro

# ✅ CORRECTO: Solo usar información disponible al momento de predicción
df['avg_purchases_last_3_months'] = ...  # Información del pasado
```

8.3 Caso Real: CarVision

El Problema Original

En CarVision, teníamos features que causaban leakage:

```
# src/carvision/features.py - ANTES (con leakage potencial)

class FeatureEngineer:
    def transform(self, X):
        X = X.copy()

        # ✅ OK: vehicle_age no depende del target
        X['vehicle_age'] = 2024 - X['model_year']

        # ✅ OK: brand no depende del target
        X['brand'] = X['model'].str.split().str[0]

        # ⚠️ PELIGRO: price_per_mile DEPENDE de price (target)
        X['price_per_mile'] = X['price'] / (X['odometer'] + 1)

        # ⚠️ PELIGRO: price_category DEPENDE de price (target)
        X['price_category'] = pd.cut(X['price'], ...)

        return X
```

La Solución: drop_columns en Config

```
# configs/config.yaml

preprocessing:
  numeric_features:
    - odometer
    - vehicle_age
  categorical_features:
    - fuel
```

```
    - transmission
    - brand
  drop_columns:           # ← Features que causan leakage
    - price_per_mile      # Depende de price
    - price_category      # Depende de price
```

```
# src/carvision/data.py

def infer_feature_types(df, target, drop_columns=None, ...):
    """Infiere tipos de features, excluyendo las que causan leakage."""

    # Columnas a excluir
    exclude = {target}  # Siempre excluir el target
    if drop_columns:
        exclude.update(drop_columns)  # Excluir features con leakage

    # Inferir tipos solo de columnas seguras
    for col in df.columns:
        if col in exclude:
            continue  # Saltar columnas peligrosas
        # ... resto de la lógica
```

¿Por qué NO eliminamos price_per_mile del FeatureEngineer?

```
# La feature EXISTE en el transformer, pero se ELIMINA antes del modelo

# Motivo: price_per_mile es útil para ANÁLISIS (no para predicción)
# En el dashboard de Streamlit, usamos price_per_mile para visualizaciones
# Pero en el modelo de predicción, la eliminamos

# Flujo:
# 1. FeatureEngineer crea price_per_mile (para análisis)
# 2. Config especifica drop_columns = [price_per_mile]
# 3. ColumnTransformer NO incluye price_per_mile en sus transformers
# 4. Modelo entrena sin price_per_mile
```

8.4 Prevención con Pipelines

El Pipeline como Barrera Anti-Leakage

```
# ✅ CORRECTO: Pipeline garantiza orden correcto

from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer

# Definir QUÉ columnas usar (excluyendo las peligrosas)
num_cols = ['odometer', 'vehicle_age']  # SIN price_per_mile
cat_cols = ['fuel', 'transmission', 'brand']

# Pipeline aplica transformaciones EN ORDEN
pipeline = Pipeline([
    ('features', FeatureEngineer()),      # Crea features
    ('pre', ColumnTransformer([           # Solo usa features SEGURAS
        ('num', StandardScaler(), num_cols),
        ('cat', OneHotEncoder(), cat_cols)
    ])),
    ('model', RandomForestRegressor())
])

# fit() entrena todo con datos de TRAIN solamente
pipeline.fit(X_train, y_train)

# predict() aplica las MISMAS transformaciones
# usando parámetros aprendidos de TRAIN
predictions = pipeline.predict(X_test)
```

Diagrama del Flujo Seguro

```
┌──────────────────────────────────────────────────────────────────────────┐
│                      FLUJO ANTI-LEAKAGE CON PIPELINE                     │
├──────────────────────────────────────────────────────────────────────────┤
│                                                                          │
```

```
│  ENTRENAMIENTO:                                                          │
│  ┌──────────┐    ┌──────────────┐    ┌──────────┐    ┌──────────┐       │
│  │ X_train  │──►│FeatureEng    │──►│DropDanger │──►│ Scaler   │       │
│  │          │    │ (crea features)│    │ (elimina  │    │ fit()    │       │
│  └──────────┘    └──────────────┘    │  leakage) │    └──────────┘       │
│                                       └──────────┘         │              │
│                                                            ▼              │
│                                                       ┌──────────┐        │
│                                                       │  Model   │        │
│                                                       │  fit()   │        │
│                                                       └──────────┘        │
│                                                                           │
│  PREDICCIÓN:                                                              │
│  ┌──────────┐    ┌──────────────┐    ┌──────────┐    ┌──────────┐       │
│  │ X_new    │──►│FeatureEng    │──►│DropDanger │──►│ Scaler   │       │
│  │          │    │ (mismas feat)│    │ (mismas   │    │transform │       │
│  └──────────┘    └──────────────┘    │  columnas) │    │ (NO fit) │       │
│                                       └──────────┘    └──────────┘       │
│                                                            │              │
│                                                            ▼              │
│                                                       ┌──────────┐        │
│                                                       │  Model   │        │
│                                                       │ predict()│        │
│                                                       └──────────┘        │
│                                                                           │
└───────────────────────────────────────────────────────────────────────────┘
```

8.5 Feature Engineering Seguro

Checklist Anti-Leakage

```
╔═══════════════════════════════════════════════════════════════════════╗
╔═══════════════════════════════════════════════════════════════════════╗
║  ✅ CHECKLIST ANTES DE CREAR UNA FEATURE                              ║
╔═══════════════════════════════════════════════════════════════════════╗
║                                                                       ║
║  1. ¿Esta feature estará disponible en producción?                   ║
║     □ SÍ → OK                                                        ║
║     □ NO → ❌ NO USAR para predicción                                ║
║                                                                       ║
║  2. ¿Esta feature usa información del target (directa o indirectamente)? ║
║     □ NO → OK                                                        ║
║     □ SÍ → ❌ LEAKAGE - eliminar o recalcular sin target             ║
║                                                                       ║
║  3. ¿Esta feature usa información del futuro?                         ║
║     □ NO → OK                                                        ║
║     □ SÍ → ❌ TEMPORAL LEAKAGE - usar solo datos pasados             ║
║                                                                       ║
║  4. ¿Las estadísticas de esta feature se calcularon con datos de test? ║
║     □ NO → OK                                                        ║
║     □ SÍ → ❌ TRAIN-TEST CONTAMINATION - recalcular solo con train   ║
║                                                                       ║
╔═══════════════════════════════════════════════════════════════════════╗
```

Features Seguras vs Peligrosas

Feature	Segura	Motivo
`vehicle_age = 2024 - model_year`	✅	No depende del target
`brand = model.split()[0]`	✅	No depende del target
`is_luxury = brand in ['bmw', 'mercedes']`	✅	No depende del target
`price_per_mile = price / odometer`	❌	Usa el target (price)
`price_category = cut(price)`	❌	Usa el target (price)
`avg_price_by_brand` (calculado con todo el dataset)	❌	Contamina train/test

Código: Feature Engineering Seguro

```
# src/carvision/features.py - Versión SEGURA

class FeatureEngineer(BaseEstimator, TransformerMixin):
    """Feature engineering sin leakage."""

    def __init__(self, current_year: int = None):
        self.current_year = current_year

    def fit(self, X, y=None):
        # Stateless: no aprende nada que pueda causar leakage
        return self

    def transform(self, X):
        X = X.copy()
        year = self.current_year or pd.Timestamp.now().year

        # ✅ SEGURO: Solo usa columnas de entrada (no target)
        if 'model_year' in X.columns:
            X['vehicle_age'] = year - X['model_year']

        if 'model' in X.columns:
            X['brand'] = X['model'].astype(str).str.split().str[0]

        # ⚠️ CONDICIONAL: Solo crear si price existe (para análisis)
        # El modelo NO usará estas features (drop_columns en config)
        if 'price' in X.columns and 'odometer' in X.columns:
            X['price_per_mile'] = X['price'] / (X['odometer'] + 1)

        return X
```

✅ Ejercicio: Detectar Leakage

```
# Analiza este código y encuentra todos los casos de leakage

def prepare_data(df):
    # 1. Normalizar todas las features
    scaler = StandardScaler()
    df[['age', 'income']] = scaler.fit_transform(df[['age', 'income']])

    # 2. Crear features
    df['income_category'] = pd.cut(df['target_income'], bins=3)
    df['age_bucket'] = pd.cut(df['age'], bins=[0, 30, 50, 100])

    # 3. Split
    X_train, X_test = train_test_split(df.drop('target_income', axis=1))
    y_train, y_test = train_test_split(df['target_income'])

    return X_train, X_test, y_train, y_test
```

📝 Ver Solución

```
# PROBLEMAS DETECTADOS:

# 1. ❌ TRAIN-TEST CONTAMINATION (línea 3-4)
# scaler.fit_transform se aplica a TODO el dataset antes del split
# El scaler "ve" estadísticas de test durante entrenamiento

# 2. ❌ TARGET LEAKAGE (línea 7)
# income_category se calcula usando target_income
# El modelo aprenderá a "leer" el target desde esta feature

# 3. ❌ SPLIT INCONSISTENTE (líneas 11-12)
# train_test_split se llama dos veces con diferentes random states
# X_train no corresponde con y_train

# VERSIÓN CORREGIDA:
def prepare_data_correct(df):
    # 1. Split PRIMERO
    train_df, test_df = train_test_split(df, test_size=0.2, random_state=42)

    # 2. Features SIN leakage
```

```
    for data in [train_df, test_df]:
        data['age_bucket'] = pd.cut(data['age'], bins=[0, 30, 50, 100])
        # NO crear income_category - usa el target

    # 3. Separar X e y
    X_train = train_df.drop('target_income', axis=1)
    y_train = train_df['target_income']
    X_test = test_df.drop('target_income', axis=1)
    y_test = test_df['target_income']

    # 4. Escalar SOLO con datos de train
    scaler = StandardScaler()
    X_train[['age', 'income']] = scaler.fit_transform(X_train[['age', 'income']])
    X_test[['age', 'income']] = scaler.transform(X_test[['age', 'income']])

    return X_train, X_test, y_train, y_test
```

✅ Checkpoint

- [ ] Entiendes qué es data leakage y por qué es peligroso
- [ ] Puedes identificar los `3` tipos de leakage
- [ ] Sabes cómo usar `drop_columns` para eliminar features peligrosas
- [ ] Entiendes por qué el Pipeline previene leakage
- [ ] Puedes aplicar el checklist anti-leakage a nuevas features

📦 Cómo se Usó en el Portafolio

El proyecto **CarVision** es el ejemplo principal de feature engineering seguro:

FeatureEngineer Centralizado

```
# CarVision-Market-Intelligence/src/carvision/features.py
class FeatureEngineer(BaseEstimator, TransformerMixin):
    """Centraliza TODO el feature engineering.

    Usado en: training, FastAPI, Streamlit - siempre igual.
    """

    def __init__(self, current_year: int = None):
        self.current_year = current_year

    def transform(self, X):
        X = X.copy()
        year = self.current_year or pd.Timestamp.now().year

        # ✅ Features SEGURAS (no usan target)
        X['vehicle_age'] = year - X['model_year']
        X['brand'] = X['model'].str.split().str[0]
        X['mileage_category'] = pd.cut(X['odometer'], bins=[0, 50000, 100000, float('inf')])

        return X
```

Prevención de Leakage en Config

```
# CarVision-Market-Intelligence/configs/config.yaml
data:
  target_column: price
  drop_columns:
    - price_per_mile    # ❌ Usa target
    - price_category    # ❌ Usa target
    - id                # No predictivo
```

Caso Real: Bug Corregido

El portafolio tuvo un bug de leakage que fue corregido:

```
# ❌ ANTES (con leakage)
X['price_per_mile'] = X['price'] / X['odometer']  # Usaba el target!

# ✅ DESPUÉS (sin leakage)
# price_per_mile se elimina en drop_columns
# Solo se calcula para análisis exploratorio, NO para el modelo
```

Archivos Clave

Proyecto	Feature Engineering	Anti-Leakage
CarVision	`src/carvision/features.py`	`drop_columns` en config
BankChurn	En `ColumnTransformer`	Sin features derivadas del target
TelecomAI	En pipeline	Sin features peligrosas

🔧 Ejercicio: Audita CarVision

```
# 1. Revisa el FeatureEngineer
cat CarVision-Market-Intelligence/src/carvision/features.py

# 2. Verifica drop_columns en config
cat CarVision-Market-Intelligence/configs/config.yaml | grep -A5 "drop_columns"

# 3. Ejecuta tests para verificar que no hay leakage
cd CarVision-Market-Intelligence
pytest tests/test_features.py -v
```

💼 Consejos Profesionales

Recomendaciones para destacar en entrevistas y proyectos reales

Para Entrevistas

1. **Feature Store**: Explica por qué centralizar features mejora consistencia training/serving.
2. **Data Leakage**: Da ejemplos concretos (usar target en features, información del futuro).
3. **Feature Selection**: Conoce métodos (mutual information, RFE, importancia de modelo).

Para Proyectos Reales

Situación	Consejo
Features temporales	Cuidado con leakage: no uses info futura
Categorías nuevas	Usa `handle_unknown='ignore'` en encoders
Features de texto	TF-IDF para baseline, embeddings para avanzado
Interacciones	PolynomialFeatures con grado 2 máximo

Checklist de Feature Engineering

- [ ] Sin data leakage verificado
- [ ] Transformaciones aplicadas consistentemente train/serve
- [ ] Features documentadas (significado, fuente, transformación)
- [ ] Outliers manejados (clip, winsorize, o flag)
- [ ] Missing values con estrategia clara

📺 Recursos Externos Recomendados

Ver RECURSOS_POR_MODULO.md para la lista completa.

🏷️	Recurso	Tipo
🔴	Feature Engineering for ML - Krish Naik	Video
🟡	Avoiding Data Leakage	Video

🔗 Referencias del Glosario

Ver 2 1 _GLOSARIO.md para definiciones de:
- **Data Leakage**: Filtración de información del target

- **Feature Engineering**: Creación de variables predictivas
- **ColumnTransformer**: Procesamiento paralelo de columnas

✅ Ejercicios

Ver EJERCICIOS.md - Módulo 08:
- 8.1: Detectar data leakage
- 8.2: Pipeline sin leakage

TRAINING PROFESIONAL

GUÍA MLOPS V5.0

PORTFOLIO EDITION

09. Training Profesional

🎯 Objetivo del Módulo

Implementar un pipeline de entrenamiento robusto, reproducible y loggeable como `ChurnTrainer` de BankChurn.

```
╔═══════════════════════════════════════════════════════════════════╗
║                                                                   ║
║  NOTEBOOK TÍPICO:                    CÓDIGO PROFESIONAL:          ║
║  ────────────────                    ───────────────────          ║
║  • 500 líneas en un archivo          • Clases modulares           ║
║  • Variables globales                • Configuración externa      ║
║  • Sin logging                       • Logging estructurado       ║
║  • "Funcionó... creo"                • Métricas rastreadas        ║
║  • Imposible reproducir              • 100% reproducible          ║
║                                                                   ║
╚═══════════════════════════════════════════════════════════════════╝
```

🧩 Cómo se aplica en este portafolio

- Este módulo se apoya directamente en el código real de **BankChurn-Predictor**:
- `src/bankchurn/training.py` (`ChurnTrainer`).
- `configs/config.yaml` (config Pydantic usada en el trainer).
- Carpeta `artifacts/` donde se guardan `model.joblib` y `training_results.json`.
- También establece el patrón que luego deberás replicar en **CarVision** y **TelecomAI** (por ejemplo, implementando tu propio `PriceTrainer` o `TelecomTrainer`) y que se conecta con los módulos de **Experiment Tracking** (MLflow) y **CI/CD**.

📋 Contenido

1. Arquitectura de una Clase Trainer
2. Carga y Validación de Datos
3. Cross-Validation Profesional
4. Gestión de Artefactos
5. Logging y Métricas

9.1 Arquitectura de una Clase Trainer

Código Real: ChurnTrainer (BankChurn)

```
# src/bankchurn/training.py - Estructura REAL del portafolio

from __future__ import annotations

import logging
from pathlib import Path
from typing import Dict, Tuple

import joblib
import mlflow
import numpy as np
import pandas as pd
from sklearn.model_selection import StratifiedKFold, train_test_split
from sklearn.pipeline import Pipeline

from .config import BankChurnConfig

logger = logging.getLogger(__name__)


class ChurnTrainer:
    """Pipeline de entrenamiento para predicción de churn.

    Esta clase encapsula TODO el flujo de entrenamiento:
    1. Carga y validación de datos
    2. Preparación de features
    3. Construcción del pipeline
    4. Entrenamiento con cross-validation
    5. Evaluación final
    6. Guardado de artefactos
```

```
    Parameters
    ----------
    config : BankChurnConfig
        Configuración validada con Pydantic.
    random_state : int, optional
        Semilla para reproducibilidad.

    Attributes
    ----------
    model_ : Pipeline
        Pipeline entrenado (disponible después de fit).
    cv_results_ : dict
        Resultados de cross-validation.
    test_results_ : dict
        Resultados en test set.

    Examples
    --------
    >>> config = BankChurnConfig.from_yaml("configs/config.yaml")
    >>> trainer = ChurnTrainer(config)
    >>> trainer.run("data/raw/churn.csv", "artifacts/")
    """

    def __init__(self, config: BankChurnConfig, random_state: int = None):
        self.config = config
        self.random_state = random_state or config.model.random_state

        # Atributos que se llenan durante entrenamiento
        self.model_: Pipeline | None = None
        self.cv_results_: Dict[str, float] | None = None
        self.test_results_: Dict[str, float] | None = None

        # Configurar MLflow si está habilitado
        if self.config.mlflow.enabled:
            self._setup_mlflow()

    def _setup_mlflow(self) -> None:
        """Configura MLflow tracking."""
        try:
            mlflow.set_tracking_uri(self.config.mlflow.tracking_uri)
            mlflow.set_experiment(self.config.mlflow.experiment_name)
            logger.info(f"MLflow tracking: {self.config.mlflow.tracking_uri}")
        except Exception as e:
            logger.warning(f"MLflow setup failed: {e}")

    def run(self, input_path: str | Path, output_dir: str | Path) -> Dict:
        """Ejecuta el pipeline completo de entrenamiento.

        Parameters
        ----------
        input_path : str or Path
            Ruta al CSV de entrada.
        output_dir : str or Path
            Directorio para guardar artefactos.

        Returns
        -------
        dict
            Resultados de entrenamiento y evaluación.
        """
        output_dir = Path(output_dir)
        output_dir.mkdir(parents=True, exist_ok=True)

        logger.info("=" * 60)
        logger.info("INICIANDO ENTRENAMIENTO")
        logger.info("=" * 60)

        # 1. Cargar datos
        data = self.load_data(input_path)

        # 2. Preparar features
        X, y = self.prepare_features(data)

        # 3. Split train/test
        X_train, X_test, y_train, y_test = train_test_split(
```

```
            X, y,
            test_size=self.config.model.test_size,
            random_state=self.random_state,
            stratify=y  # Mantener proporción de clases
        )
        logger.info(f"Train: {len(X_train)}, Test: {len(X_test)}")

        # 4. Construir pipeline
        self.model_ = self.build_pipeline()

        # 5. Cross-validation
        self.cv_results_ = self.cross_validate(X_train, y_train)

        # 6. Entrenar modelo final
        self.model_.fit(X_train, y_train)

        # 7. Evaluar en test
        self.test_results_ = self.evaluate(X_test, y_test)

        # 8. Guardar artefactos
        self.save_artifacts(output_dir)

        # 9. Log a MLflow
        if self.config.mlflow.enabled:
            self._log_to_mlflow()

        logger.info("=" * 60)
        logger.info("ENTRENAMIENTO COMPLETADO")
        logger.info("=" * 60)

        return {
            "cv_results": self.cv_results_,
            "test_results": self.test_results_,
        }
```

Diagrama del Flujo

```
┌──────────────────────────────────────────────────────────────────────┐
│                        ChurnTrainer.run() Flow                       │
├──────────────────────────────────────────────────────────────────────┤
│                                                                      │
│   ┌──────────┐    ┌──────────┐    ┌──────────┐    ┌──────────┐      │
│   │ Config   │───►│  Load    │───►│ Prepare  │───►│  Split   │      │
│   │  YAML    │    │  Data    │    │Features  │    │Train/Test│      │
│   └──────────┘    └──────────┘    └──────────┘    └──────┬───────┘      │
│                                                          │           │
│                                                          ▼           │
│   ┌──────────┐    ┌──────────┐    ┌──────────┐    ┌──────────┐      │
│   │ MLflow   │◄───│  Save    │◄───│Evaluate  │◄───│  Train   │      │
│   │  Log     │    │Artifacts │    │ (Test)   │    │ + CV     │      │
│   └──────────┘    └──────────┘    └──────────┘    └──────────┘      │
│                                                                      │
│   OUTPUT:                                                            │
│   • model.joblib (Pipeline completo)                                 │
│   • training_results.json (métricas)                                 │
│   • MLflow run (experimento rastreado)                               │
│                                                                      │
└──────────────────────────────────────────────────────────────────────┘
```

9.2 Carga y Validación de Datos

```
# Continuación de ChurnTrainer

def load_data(self, input_path: str | Path) -> pd.DataFrame:
    """Carga y valida datos de entrada.

    Parameters
    ----------
    input_path : str or Path
        Ruta al archivo CSV.

    Returns
    -------
```

```
    pd.DataFrame
        Datos cargados y validados.

    Raises
    ------
    FileNotFoundError
        Si el archivo no existe.
    ValueError
        Si faltan columnas requeridas.
    """
    input_path = Path(input_path)

    if not input_path.exists():
        raise FileNotFoundError(f"Archivo no encontrado: {input_path}")

    # Cargar CSV
    data = pd.read_csv(input_path)
    logger.info(f"Datos cargados: {data.shape[0]} filas, {data.shape[1]} columnas")

    # Validar columnas requeridas
    required = {self.config.data.target_column}
    required.update(self.config.data.numerical_features)
    required.update(self.config.data.categorical_features)

    missing = required - set(data.columns)
    if missing:
        raise ValueError(f"Columnas faltantes: {missing}")

    # Log de estadísticas básicas
    target = self.config.data.target_column
    class_dist = data[target].value_counts(normalize=True)
    logger.info(f"Distribución de clases:\n{class_dist}")

    # Alertar si hay desbalance severo
    minority_pct = class_dist.min()
    if minority_pct < 0.1:
        logger.warning(f"⚠️ Desbalance severo: clase minoritaria = {minority_pct:.1%}")

    return data


def prepare_features(self, data: pd.DataFrame) -> Tuple[pd.DataFrame, pd.Series]:
    """Prepara features y target.

    Aplica:
    1. Eliminación de columnas innecesarias (drop_columns)
    2. Separación de X e y
    """
    # Columnas a eliminar
    drop_cols = self.config.data.drop_columns + [self.config.data.target_column]
    drop_cols = [c for c in drop_cols if c in data.columns]

    X = data.drop(columns=drop_cols)
    y = data[self.config.data.target_column]

    logger.info(f"Features: {X.shape[1]}, Target: {y.name}")

    return X, y
```

9.3 Cross-Validation Profesional

```
def cross_validate(self, X: pd.DataFrame, y: pd.Series) -> Dict[str, float]:
    """Ejecuta cross-validation estratificada.

    Stratified K-Fold mantiene la proporción de clases en cada fold,
    crucial para datasets desbalanceados.
    """
    from sklearn.metrics import f1_score, roc_auc_score

    cv = StratifiedKFold(
        n_splits=self.config.model.cv_folds,
        shuffle=True,
        random_state=self.random_state
    )
```

```
    f1_scores = []
    auc_scores = []

    logger.info(f"Cross-validation con {self.config.model.cv_folds} folds...")

    for fold, (train_idx, val_idx) in enumerate(cv.split(X, y), 1):
        X_train_cv = X.iloc[train_idx]
        X_val_cv = X.iloc[val_idx]
        y_train_cv = y.iloc[train_idx]
        y_val_cv = y.iloc[val_idx]

        # Clonar pipeline para este fold
        from sklearn.base import clone
        fold_pipeline = clone(self.model_)

        # Entrenar en este fold
        fold_pipeline.fit(X_train_cv, y_train_cv)

        # Evaluar
        y_pred = fold_pipeline.predict(X_val_cv)
        y_proba = fold_pipeline.predict_proba(X_val_cv)[:, 1]

        f1 = f1_score(y_val_cv, y_pred)
        auc = roc_auc_score(y_val_cv, y_proba)

        f1_scores.append(f1)
        auc_scores.append(auc)

        logger.info(f"  Fold {fold}: F1={f1:.4f}, AUC={auc:.4f}")

    results = {
        "f1_mean": np.mean(f1_scores),
        "f1_std": np.std(f1_scores),
        "auc_mean": np.mean(auc_scores),
        "auc_std": np.std(auc_scores),
    }

    logger.info(f"CV Results: F1={results['f1_mean']:.4f} ± {results['f1_std']:.4f}")
    logger.info(f"CV Results: AUC={results['auc_mean']:.4f} ± {results['auc_std']:.4f}")

    return results
```

9.4 Gestión de Artefactos

```
def save_artifacts(self, output_dir: Path) -> None:
    """Guarda modelo y resultados."""
    import json

    # 1. Guardar modelo (pipeline completo)
    model_path = output_dir / "model.joblib"
    joblib.dump(self.model_, model_path)
    logger.info(f"Modelo guardado: {model_path}")

    # 2. Guardar resultados como JSON
    results = {
        "cv_results": self.cv_results_,
        "test_results": self.test_results_,
        "config": {
            "model_type": self.config.model.type,
            "test_size": self.config.model.test_size,
            "cv_folds": self.config.model.cv_folds,
            "random_state": self.random_state,
        }
    }

    results_path = output_dir / "training_results.json"
    with open(results_path, "w") as f:
        json.dump(results, f, indent=2, default=str)
    logger.info(f"Resultados guardados: {results_path}")


def evaluate(self, X_test: pd.DataFrame, y_test: pd.Series) -> Dict:
    """Evalúa en test set."""
```

```
    from sklearn.metrics import (
        accuracy_score, precision_score, recall_score,
        f1_score, roc_auc_score, confusion_matrix
    )

    y_pred = self.model_.predict(X_test)
    y_proba = self.model_.predict_proba(X_test)[:, 1]

    results = {
        "metrics": {
            "accuracy": accuracy_score(y_test, y_pred),
            "precision": precision_score(y_test, y_pred),
            "recall": recall_score(y_test, y_pred),
            "f1_score": f1_score(y_test, y_pred),
            "roc_auc": roc_auc_score(y_test, y_proba),
        },
        "confusion_matrix": confusion_matrix(y_test, y_pred).tolist(),
    }

    logger.info("Test Results:")
    for metric, value in results["metrics"].items():
        logger.info(f"  {metric}: {value:.4f}")

    return results
```

9.5 Logging y Métricas

Configurar Logging Profesional

```
# src/bankchurn/__init__.py o en el módulo principal

import logging
import sys

def setup_logging(level: str = "INFO") -> None:
    """Configura logging estructurado."""

    # Formato profesional
    fmt = "%(asctime)s | %(levelname)-8s | %(name)s | %(message)s"
    datefmt = "%Y-%m-%d %H:%M:%S"

    logging.basicConfig(
        level=getattr(logging, level.upper()),
        format=fmt,
        datefmt=datefmt,
        handlers=[
            logging.StreamHandler(sys.stdout),
            logging.FileHandler("training.log", mode="a"),
        ]
    )

    # Reducir verbosidad de librerías externas
    logging.getLogger("urllib3").setLevel(logging.WARNING)
    logging.getLogger("mlflow").setLevel(logging.WARNING)


# Uso
setup_logging("INFO")
```

Integración con MLflow

```
def _log_to_mlflow(self) -> None:
    """Loguea métricas y artefactos a MLflow."""
    with mlflow.start_run(run_name="training"):
        # Parámetros
        mlflow.log_params({
            "model_type": self.config.model.type,
            "test_size": self.config.model.test_size,
            "cv_folds": self.config.model.cv_folds,
            "random_state": self.random_state,
        })

        # Métricas de CV
        if self.cv_results_:
```

```
            mlflow.log_metrics({
                f"cv_{k}": v for k, v in self.cv_results_.items()
            })

        # Métricas de test
        if self.test_results_:
            mlflow.log_metrics({
                f"test_{k}": v
                for k, v in self.test_results_["metrics"].items()
            })

        # Artefactos
        mlflow.log_artifact("artifacts/training_results.json")

        logger.info(f"MLflow run logged: {mlflow.active_run().info.run_id}")
```

🧨 Errores habituales y cómo depurarlos en training

En este módulo, casi todos los problemas vienen de **datos mal preparados**, **splits inconsistentes** o **logging incompleto**. Aquí están los patrones más frecuentes.

1) `KeyError` o `ValueError` al cargar datos (columnas/config mal alineadas)

Síntomas típicos

- `ValueError: Columnas faltantes: {'CreditScore', 'Age', ...}`
- `KeyError: 'Exited'` al intentar acceder al target.

Cómo identificarlo

- Compara `self.config.data.*` con las columnas reales del CSV.
- Usa `load_data` como punto único de verdad y revisa sus logs (número de filas/columnas, distribución de clases).

Cómo corregirlo

- Ajusta `configs/config.yaml` para que `target_column`, `numerical_features`, `categorical_features` reflejen **exactamente** el dataset.
- Mantén `prepare_features` simple: usar solo `drop_columns` + separación de `X` e `y`.

2) Resultados no reproducibles (semilla mal gestionada)

Síntomas típicos

- Cada ejecución de `ChurnTrainer.run` produce métricas distintas sin razón aparente.

Cómo identificarlo

- Revisa la inicialización de `ChurnTrainer` y el uso de `random_state` en:
- `train_test_split`.
- `StratifiedKFold`.
- Modelos base (`RandomForest`, etc.).

Cómo corregirlo

- Asegúrate de que la semilla venga de un solo lugar (config) y se pase a todos los componentes relevantes.
- Si usas utilidades como `common_utils.seed.set_seed`, llama a esa función al inicio de `run` o en el CLI.

3) Cross-validation engañosa (leakage entre folds o CV desalineado con test)

Síntomas típicos

- Métricas de CV muy buenas, pero test set mucho peor.
- Folds con distribución de clases muy desigual.

Cómo identificarlo

- Verifica que usas `StratifiedKFold` para clasificación desbalanceada.
- Asegúrate de clonar el pipeline (`clone(self.model_)`) en cada fold, no reutilizar el mismo objeto.

Cómo corregirlo

- Mantén el orden: definir pipeline completo **antes** de CV y clonar dentro del loop.
- No mezcles datos de test en CV; usa `train_test_split` una sola vez, luego CV solo en `X_train, y_train`.

4) Artefactos inconsistentes (modelo y métricas que no corresponden)

Síntomas típicos

- `training_results.json` y `model.joblib` provienen de ejecuciones distintas.
- MLflow muestra métricas que no coinciden con los artefactos locales.

Cómo identificarlo

- Revisa que `save_artifacts` se llama **después** de entrenar el modelo final y evaluar en test.
- Comprueba timestamps y contenido de `artifacts/model.joblib` y `artifacts/training_results.json`.

Cómo corregirlo

- Asegura el orden en `run`: CV → `fit` final → `evaluate` → `save_artifacts` → `_log_to_mlflow`.
- No reutilices artefactos viejos; limpia la carpeta `artifacts/` antes de grandes cambios.

5) MLflow no registra nada o registra en el lugar equivocado

Síntomas típicos

- Corres `ChurnTrainer.run` pero no ves runs nuevos en la UI de MLflow.
- Métricas aparecen en otro experimento o en otro tracking URI.

Cómo identificarlo

- Imprime/inspecciona `self.config.mlflow.tracking_uri` y `experiment_name`.
- Verifica variables de entorno (`MLFLOW_TRACKING_URI`) si las usas en scripts aparte (`run_mlflow.py`).

Cómo corregirlo

- Centraliza la configuración en `BankChurnConfig` y `_setup_mlflow`, evitando `mlflow.set_tracking_uri` dispersos en el código.
- En `run_mlflow.py`, asegúrate de usar el mismo `tracking_uri` y `experiment` que usaste durante el entrenamiento.

Patrón general de debugging para training

1. **Empieza por los datos**: confirma que `load_data` y `prepare_features` producen `X, y` con las formas y columnas esperadas.
2. **Verifica el split**: revisa distribución de clases en `train/test` y en cada fold.
3. **Comprueba los artefactos**: que `model.joblib` y `training_results.json` se regeneren juntos.
4. **Sincroniza con MLflow**: compara métricas locales con lo que ves en la UI.

Con este enfoque, el entrenamiento deja de ser “caja negra” y se convierte en un pipeline controlado y auditable, como se espera en un rol Senior/Staff.

✅ Ejercicio: Implementar tu Trainer

Crea una clase `PriceTrainer` para CarVision siguiendo el patrón:

```
class PriceTrainer:
    """Tu tarea: implementar siguiendo el patrón de ChurnTrainer."""

    def __init__(self, config: dict):
        # TODO: Inicializar atributos
        pass

    def run(self, input_path: Path, output_dir: Path) -> dict:
        # TODO: Implementar flujo completo
        pass

    def load_data(self, path: Path) -> pd.DataFrame:
        # TODO: Cargar y validar
        pass

    def build_pipeline(self) -> Pipeline:
        # TODO: Construir pipeline [features -> pre -> model]
        pass
```

📦 Cómo se Usó en el Portafolio

El proyecto **BankChurn** implementa el patrón de training profesional completo:

Clase ChurnTrainer Real

```
# BankChurn-Predictor/src/bankchurn/trainer.py (estructura)
class ChurnTrainer:
    """Entrenador profesional con CV, MLflow y artefactos."""

    def __init__(self, config: BankChurnConfig):
        self.config = config
        self.model_ = None
        self.metrics_ = {}

    def run(self, input_path: Path, output_dir: Path) -> dict:
        """Flujo completo: load → split → CV → train → evaluate → save."""
        df = self.load_data(input_path)
        X, y = self.prepare_features(df)
        X_train, X_test, y_train, y_test = self.split_data(X, y)

        # Cross-validation
        cv_scores = self.cross_validate(X_train, y_train)

        # Entrenamiento final
        self.model_ = self.build_pipeline()
        self.model_.fit(X_train, y_train)

        # Evaluación
        self.metrics_ = self.evaluate(X_test, y_test)

        # Guardar artefactos
        self.save_artifacts(output_dir)

        return self.metrics_
```

Flujo de Entrenamiento

```
┌──────────────────────────────────────────────────────────────┐
│                    FLUJO DE TRAINING                         │
├──────────────────────────────────────────────────────────────┤
│                                                              │
│  load_data → prepare_features → split_data                   │
│      │              │               │                        │
│      ▼              ▼               ▼                        │
│  DataFrame     X, y arrays    train/test split               │
│                                      │                       │
│                    ┌─────────────────┴─────────────────┐     │
│                    │                                   │     │
│             cross_validate                    build_pipeline │
│                    │                                   │     │
│             cv_scores                          Pipeline      │
│                    │                                   │     │
│                    └─────────────────┬─────────────────┘     │
│                                      │                       │
│                              model_.fit()                    │
│                                      │                       │
│                              evaluate()                      │
│                                      │                       │
│                          save_artifacts()                    │
│                                      │                       │
│                       pipeline.joblib + metrics.json         │
│                                                              │
└──────────────────────────────────────────────────────────────┘
```

Archivos de Training por Proyecto

Proyecto	Trainer	Config	Artefactos
BankChurn	`src/bankchurn/trainer.py`	`configs/config.yaml`	`artifacts/`
CarVision	`main.py`	`configs/config.yaml`	`artifacts/`
TelecomAI	`src/telecomai/training.py`	`configs/config.yaml`	`artifacts/`

🔧 Ejercicio: Ejecuta Training Real

```
# 1. Ve a BankChurn
cd BankChurn-Predictor

# 2. Entrena el modelo
python main.py --config configs/config.yaml

# 3. Verifica artefactos generados
ls -la artifacts/
cat artifacts/training_results.json

# 4. Verifica MLflow
mlflow ui  # Abre http://localhost:5000
```

💼 Consejos Profesionales

Recomendaciones para destacar en entrevistas y proyectos reales

Para Entrevistas

1. **Cross-Validation**: Explica stratified k-fold, time series split, y cuándo usar cada uno.
2. **Hyperparameter Tuning**: RandomSearch vs GridSearch vs Bayesian (Optuna).
3. **Métricas de negocio**: Traduce métricas técnicas a impacto de negocio.

Para Proyectos Reales

Situación	Consejo
Clases desbalanceadas	SMOTE, class_weight, o threshold tuning
Overfitting	Early stopping, regularización, más datos
Modelo en producción	Entrena con todos los datos al final
Reproducibilidad	Fija seeds en todos los componentes

Pipeline de Training Profesional

```
1. Split estratificado (train/val/test)
2. Feature engineering solo en train
3. Hyperparameter tuning con val
4. Evaluación final en test (una sola vez)
5. Re-entrenamiento con todos los datos
6. Versionado de modelo + métricas
```

📺 Recursos Externos Recomendados

Ver RECURSOS_POR_MODULO.md para la lista completa.

🏷️	Recurso	Tipo
🔴	Cross-Validation - StatQuest	Video
🟡	ML Training Best Practices	Video

🔗 Referencias del Glosario

Ver 21_GLOSARIO.md para definiciones de:
- **Cross-Validation**: Validación cruzada para evaluar modelos
- **class_weight**: Manejo de clases desbalanceadas
- **Reproducibility**: Resultados repetibles con random_state

✅ Ejercicios

Ver EJERCICIOS.md - Módulo 09:
- 9.1: Implementar Trainer class
- 9.2: Garantizar reproducibilidad

EXPERIMENT TRACKING

GUÍA MLOPS V5.0

PORTFOLIO EDITION

10. Experiment Tracking con MLflow

🎯 Objetivo del Módulo

Implementar tracking de experimentos como lo hace el portafolio con `run_mlflow.py`.

```
╔═══════════════════════════════════════════════════════════════════╗
│                                                                   │
│  SIN MLFLOW:                          CON MLFLOW:                 │
│  ───────────                          ───────────                 │
│  "¿Qué hiperparámetros usé hace       "MLflow run abc123: RF con  │
│   2 semanas cuando obtuve F1=0.85?"   n_estimators=200, F1=0.85"  │
│                                                                   │
│  "¿Dónde guardé ese modelo bueno?"    "Artifacts en run abc123/model.pkl"  │
│                                                                   │
│  "¿Por qué este modelo es peor?"      "Comparar runs en UI: diff params"   │
│                                                                   │
╚═══════════════════════════════════════════════════════════════════╝
```

🧩 Cómo se aplica en este portafolio

- En **BankChurn-Predictor** ya tienes:
- `scripts/run_mlflow.py` como script de logging posterior al entrenamiento.
- Configuración de MLflow en `configs/config.yaml` y `src/bankchurn/config.py`.
- El archivo `docker-compose.mlflow.yml` en la raíz del repo levanta un servidor MLflow real que puedes usar para practicar este módulo.
- El mismo patrón de logging puedes aplicarlo a **CarVision** y **TelecomAI**, usando sus `artifacts/` y modelos entrenados como fuente de métricas y artifacts.

📋 Contenido

1. Conceptos de MLflow
2. Setup y Configuración
3. Logging de Experimentos
4. Model Registry
5. Código Real del Portafolio

10.1 Conceptos de MLflow

Los 4 Componentes

```
┌─────────────────────────────────────────────────────────────────┐
│                         MLFLOW COMPONENTS                       │
├─────────────────────────────────────────────────────────────────┤
│                                                                 │
│  1. TRACKING                    2. PROJECTS                     │
│  ────────────                   ───────────                     │
│  • Log params, metrics          • Empaquetar código             │
│  • Guardar artifacts            • MLproject file                │
│  • Comparar runs                • Reproducibilidad              │
│                                                                 │
│  3. MODELS                      4. REGISTRY                     │
│  ──────────                     ────────────                    │
│  • Formato estándar             • Versionado de modelos         │
│  • Flavors (sklearn, pytorch)   • Staging → Production          │
│  • Serving                      • Aprobaciones                  │
│                                                                 │
└─────────────────────────────────────────────────────────────────┘

EN ESTE PORTAFOLIO USAMOS: Tracking + Registry
```

Jerarquía de MLflow

```
MLflow Server
└── Experiment: "BankChurn"
    ├── Run: abc123 (2024-01-15)
    │   ├── Parameters: {n_estimators: 100, max_depth: 10}
    │   ├── Metrics: {f1: 0.60, auc: 0.85}
    │   └── Artifacts: [model.pkl, config.yaml]
    │
    ├── Run: def456 (2024-01-16)
```

```
    │   ├── Parameters: {n_estimators: 200, max_depth: 15}
    │   ├── Metrics: {f1: 0.62, auc: 0.86}
    │   └── Artifacts: [model.pkl, config.yaml]
    │
    └── Run: ghi789 (2024-01-17) ← MEJOR
        ├── Parameters: {n_estimators: 200, max_depth: 10}
        ├── Metrics: {f1: 0.65, auc: 0.88}
        └── Artifacts: [model.pkl, config.yaml]
```

10.2 Setup y Configuración

Opción 1: Local (File Store)

```
# Más simple, para desarrollo local
import mlflow

mlflow.set_tracking_uri("file:./mlruns")  # Guarda en carpeta local
mlflow.set_experiment("my-experiment")
```

Opción 2: Servidor MLflow (Producción)

```
# docker-compose.mlflow.yml del portafolio
services:
  mlflow:
    image: ghcr.io/mlflow/mlflow:v2.9.2
    ports:
      - "5000:5000"
    volumes:
      - mlflow-artifacts:/mlflow
    command: >
      mlflow server
      --backend-store-uri sqlite:///mlflow/mlflow.db
      --default-artifact-root /mlflow/artifacts
      --host 0.0.0.0
      --port 5000
```

```
# Conectar al servidor
import mlflow

mlflow.set_tracking_uri("http://localhost:5000")
mlflow.set_experiment("BankChurn")
```

Configuración en el Portafolio

```
# configs/config.yaml (BankChurn)
mlflow:
  tracking_uri: "file:./mlruns"      # Local para desarrollo
  experiment_name: "bankchurn"
  enabled: true
```

```
# src/bankchurn/config.py
class MLflowConfig(BaseModel):
    tracking_uri: str = "file:./mlruns"
    experiment_name: str = "bankchurn"
    enabled: bool = True
```

10.3 Logging de Experimentos

API Básica

```
import mlflow

# Iniciar un run
with mlflow.start_run(run_name="experiment-1"):

    # 1. LOG PARAMETERS (hiperparámetros, config)
    mlflow.log_param("n_estimators", 200)
    mlflow.log_param("max_depth", 10)
    mlflow.log_params({  # Múltiples a la vez
        "learning_rate": 0.1,
```

```
        "model_type": "random_forest"
    })

    # 2. LOG METRICS (resultados)
    mlflow.log_metric("f1_score", 0.65)
    mlflow.log_metric("auc_roc", 0.88)
    mlflow.log_metrics({  # Múltiples a la vez
        "precision": 0.70,
        "recall": 0.61
    })

    # 3. LOG ARTIFACTS (archivos)
    mlflow.log_artifact("configs/config.yaml")
    mlflow.log_artifact("artifacts/training_results.json")

    # 4. LOG MODEL (modelo serializado con metadata)
    mlflow.sklearn.log_model(
        pipeline,
        artifact_path="model",
        registered_model_name="BankChurnClassifier"
    )
```

Métricas por Época/Paso

```
# Para modelos que entrenan por épocas
for epoch in range(100):
    train_loss = train_one_epoch()
    val_loss = validate()

    mlflow.log_metrics({
        "train_loss": train_loss,
        "val_loss": val_loss
    }, step=epoch)  # ← step permite graficar evolución
```

10.4 Model Registry

Registrar un Modelo

```
# Durante el run
mlflow.sklearn.log_model(
    pipeline,
    artifact_path="model",
    registered_model_name="BankChurnClassifier"  # ← Registra automáticamente
)

# O después del run
mlflow.register_model(
    model_uri=f"runs:/{run_id}/model",
    name="BankChurnClassifier"
)
```

Transiciones de Estado

```
from mlflow.tracking import MlflowClient

client = MlflowClient()

# Promover a Staging
client.transition_model_version_stage(
    name="BankChurnClassifier",
    version=1,
    stage="Staging"
)

# Promover a Production (después de validación)
client.transition_model_version_stage(
    name="BankChurnClassifier",
    version=1,
    stage="Production"
)
```

Cargar Modelo desde Registry

```
# Cargar versión específica
model = mlflow.sklearn.load_model("models:/BankChurnClassifier/1")

# Cargar stage específico
model = mlflow.sklearn.load_model("models:/BankChurnClassifier/Production")

# Cargar último modelo (latest)
model = mlflow.sklearn.load_model("models:/BankChurnClassifier/latest")
```

10.5 Código Real del Portafolio

scripts/run_mlflow.py (BankChurn)

```
#!/usr/bin/env python3
"""Log training results to MLflow.

Este script se ejecuta DESPUÉS del entrenamiento para:
1. Leer resultados de artifacts/training_results.json
2. Calcular métricas de negocio (revenue saved, etc.)
3. Loguear todo a MLflow
4. Opcionalmente registrar el modelo

Uso:
    python scripts/run_mlflow.py

Environment Variables:
    MLFLOW_TRACKING_URI: URI del servidor MLflow
    MLFLOW_EXPERIMENT_NAME: Nombre del experimento
"""

from __future__ import annotations

import json
import os
from pathlib import Path

import joblib

try:
    import mlflow
    import mlflow.sklearn
    from mlflow.tracking import MlflowClient
except ImportError:
    mlflow = None

from sklearn.pipeline import Pipeline


def main() -> None:
    # Configuración desde environment
    tracking_uri = os.getenv("MLFLOW_TRACKING_URI", "file:./mlruns")
    experiment = os.getenv("MLFLOW_EXPERIMENT_NAME", "BankChurn")

    # Cargar resultados del entrenamiento
    results_path = Path("artifacts/training_results.json")
    if not results_path.exists():
        print(f"No se encontró {results_path}. Ejecuta training primero.")
        return

    data = json.loads(results_path.read_text())

    # Extraer métricas
    cv = data.get("cv_results", {})
    test = data.get("test_results", {}).get("metrics", {})

    metrics = {}
    for k, v in cv.items():
        if isinstance(v, (int, float)):
            metrics[f"cv_{k}"] = float(v)
    for k, v in test.items():
        if isinstance(v, (int, float)):
            metrics[f"test_{k}"] = float(v)
```

```
    # Calcular métricas de negocio
    cm = data.get("test_results", {}).get("confusion_matrix")
    if cm and len(cm) == 2:
        tn, fp = cm[0]
        fn, tp = cm[1]

        # Parámetros de negocio (configurables)
        clv = float(os.getenv("BC_CLV_USD", "2300"))  # Customer Lifetime Value
        retention_rate = float(os.getenv("BC_RETENTION_RATE", "0.3"))

        saved_customers = tp * retention_rate
        saved_revenue = saved_customers * clv

        metrics.update({
            "biz_detected_churners": float(tp),
            "biz_saved_customers": saved_customers,
            "biz_saved_revenue_usd": saved_revenue,
            "biz_false_positives": float(fp),
            "biz_missed_churners": float(fn),
        })

    if mlflow is None:
        print("MLflow no instalado. Métricas:", metrics)
        return

    # Configurar MLflow
    mlflow.set_tracking_uri(tracking_uri)
    mlflow.set_experiment(experiment)

    # Crear run
    with mlflow.start_run(run_name="demo-logging"):
        # Log parámetros
        mlflow.log_params({
            "run_type": "demo",
            "source": "run_mlflow.py"
        })

        # Log métricas
        mlflow.log_metrics(metrics)

        # Log artifacts
        for artifact in [
            Path("artifacts/training_results.json"),
            Path("configs/config.yaml"),
        ]:
            if artifact.exists():
                try:
                    mlflow.log_artifact(str(artifact))
                except PermissionError:
                    print(f"Skipping {artifact}: permission denied")

        # Log modelo si existe
        model_path = Path("models/model_v1.0.0.pkl")
        if model_path.exists():
            try:
                obj = joblib.load(model_path)
                if isinstance(obj, dict) and "pipeline" in obj:
                    pipe = obj["pipeline"]
                elif isinstance(obj, Pipeline):
                    pipe = obj
                else:
                    pipe = None

                if pipe:
                    mlflow.sklearn.log_model(
                        pipe,
                        artifact_path="model",
                        registered_model_name="BankChurnClassifier"
                    )
            except Exception as e:
                print(f"Model logging skipped: {e}")

        print(f"✅ MLflow run logged to {tracking_uri}")
        print(f"   Experiment: {experiment}")
```

```
        print(f"   Metrics: {len(metrics)} logged")


if __name__ == "__main__":
    main()
```

Makefile Integration

```
# Makefile
.PHONY: mlflow-demo mlflow-ui

mlflow-demo:
@echo "Logging to MLflow..."
MLFLOW_TRACKING_URI=file:./mlruns python scripts/run_mlflow.py

mlflow-ui:
@echo "Starting MLflow UI at http://localhost:5000"
mlflow ui --host 0.0.0.0 --port 5000
```

🧨 Errores habituales y cómo depurarlos en MLflow

MLflow añade una capa extra (servidor, rutas, artefactos), así que muchos errores son de **configuración** más que de código puro.

1) Runs que no aparecen en la UI (tracking_uri/experimento incorrectos)

Síntomas típicos

- Ejecutas training o `run_mlflow.py` y no ves nada nuevo en `http://localhost:5000`.

Cómo identificarlo

- Imprime `mlflow.get_tracking_uri()` y el experimento actual (`mlflow.get_experiment_by_name(...)`).
- Verifica si estás usando `file:./mlruns` mientras tienes un servidor en Docker (`http://localhost:5000`).

Cómo corregirlo

- Define claramente en config:
- Desarrollo local → `tracking_uri: "file:./mlruns"`.
- Demo/stack Docker → `tracking_uri: "http://mlflow:5000"` o `http://localhost:5000`.
- Asegúrate de que tanto `ChurnTrainer` como `scripts/run_mlflow.py` lean del mismo origen (YAML/env vars).

2) Errores al registrar modelos (`MlflowException`, permisos, backend)

Síntomas típicos

- Al llamar `mlflow.sklearn.log_model(..., registered_model_name=...)` obtienes errores sobre base de datos o registry no configurado.

Cómo identificarlo

- Si usas solo `file:./mlruns` sin servidor, el **registry completo** no está disponible.

Cómo corregirlo

- Para desarrollo ligero, limita el uso de registry (puedes usar solo tracking + artifacts).
- Para un registry completo, usa el `docker-compose.mlflow.yml` del portafolio con backend SQLite/postgres y apunta `MLFLOW_TRACKING_URI` al servidor.

3) Artifacts que no se encuentran o no se suben

Síntomas típicos

- Errores tipo `FileNotFoundError` al hacer `mlflow.log_artifact`.
- No ves `training_results.json` ni `config.yaml` en la pestaña de artifacts.

Cómo identificarlo

- Revisa rutas relativas en `run_mlflow.py` y asegúrate de que ejecutas el script desde la raíz del proyecto.

Cómo corregirlo

- Usa rutas consistentes (por ejemplo `artifacts/training_results.json`) y verifica que el archivo exista antes de loguearlo.
- Si corres dentro de Docker, revisa que el volumen monte correctamente `artifacts/` y `configs/`.

4) Problemas con MLflow en Docker (puertos, hostnames, permisos)

Síntomas típicos

- `ConnectionError` al intentar conectar a `http://localhost:5000` desde un contenedor.
- Logs que muestran errores de permisos en `/mlflow`.

Cómo identificarlo

- Examina `docker-compose.mlflow.yml` y las variables de entorno de tus servicios.

Cómo corregirlo

- Dentro de un contenedor, usa el hostname del servicio (`http://mlflow:5000`) en lugar de `localhost`.
- Asegúrate de que el volumen `mlflow-artifacts` tenga permisos de escritura correctos (usuario del contenedor).

Patrón general de debugging en MLflow

1. **Comprueba tracking_uri y experimento** antes de iniciar el run.
2. **Valida artifacts y modelos**: que los paths existen y se cargan correctamente.
3. **Reproduce localmente con file store** (`file:./mlruns`) antes de ir a servidor Docker.
4. **Verifica desde la UI** que params, metrics y artifacts coincidan con lo que esperas de tu código.

Con este patrón, MLflow pasa de ser “caja negra” a una herramienta confiable para explicar, comparar y promover modelos.

✅ Ejercicio: Integrar MLflow en TelecomAI

1. Crea `scripts/run_mlflow.py` para TelecomAI
2. Log las métricas: accuracy, f 1 , precision, recall, roc_auc
3. Calcula métricas de negocio (customers retained, revenue saved)
4. Registra el modelo como "TelecomPlanClassifier"

📦 Cómo se Usó en el Portafolio

MLflow está integrado en los 3 proyectos del portafolio:

Configuración MLflow en BankChurn

```
# BankChurn-Predictor/src/bankchurn/config.py
class MLflowConfig(BaseModel):
    """MLflow tracking configuration."""
    tracking_uri: str = "file:./mlruns"  # Local por defecto
    experiment_name: str = "bankchurn"
    enabled: bool = True
```

Integración en Trainer

```
# BankChurn-Predictor/src/bankchurn/trainer.py (extracto)
def _log_to_mlflow(self):
    """Log experimento a MLflow."""
    if not self.config.mlflow.enabled:
        return

    mlflow.set_tracking_uri(self.config.mlflow.tracking_uri)
    mlflow.set_experiment(self.config.mlflow.experiment_name)

    with mlflow.start_run():
        # Parámetros
        mlflow.log_params({
            "model_type": self.config.model.type,
            "test_size": self.config.model.test_size,
            "cv_folds": self.config.model.cv_folds,
        })

        # Métricas
        mlflow.log_metrics(self.metrics_)

        # Modelo
        mlflow.sklearn.log_model(self.model_, "model")
```

Estructura de mlruns/

```
BankChurn-Predictor/
└── mlruns/
    ├── 0/                    # Default experiment
    └── 123456789/            # bankchurn experiment
        └── abc123def456/     # Run ID
            ├── artifacts/
            │   └── model/
            ├── metrics/
            │   ├── accuracy
            │   ├── f1_score
            │   └── roc_auc
            ├── params/
            │   ├── model_type
            │   └── cv_folds
            └── meta.yaml
```

MLflow por Proyecto

Proyecto	Tracking URI	Experiment	Métricas Principales
BankChurn	`file:./mlruns`	`bankchurn`	accuracy, f_1, roc_auc
CarVision	`file:./mlruns`	`carvision`	mae, rmse, r^2
TelecomAI	`file:./mlruns`	`telecomai`	accuracy, f_1_weighted

🔧 Ejercicio: Explora MLflow Real

```
# 1. Ve a BankChurn
cd BankChurn-Predictor

# 2. Entrena con MLflow habilitado
python main.py --config configs/config.yaml

# 3. Inicia la UI de MLflow
mlflow ui --backend-store-uri file:./mlruns

# 4. Abre en navegador
# http://localhost:5000

# 5. Explora:
# - Compara runs
# - Ve artifacts
# - Registra modelo en Model Registry
```

💼 Consejos Profesionales

Recomendaciones para destacar en entrevistas y proyectos reales

Para Entrevistas

1. **MLflow vs W&B vs Neptune**: Conoce trade-offs (MLflow open-source, W&B mejor UI, Neptune escalabilidad).
2. **Model Registry**: Explica stages (Staging → Production → Archived).
3. **Reproducibilidad**: Cómo reconstruir cualquier experimento desde el tracking.

Para Proyectos Reales

Situación	Consejo
Equipo distribuido	Usa servidor MLflow centralizado
Muchos experimentos	Organiza con tags y naming conventions
Modelos grandes	Usa artifact storage externo (S_3, GCS)
Comparación	Siempre registra baseline para comparar

Qué Trackear Siempre

- **Params**: Hiperparámetros, versiones de datos
- **Metrics**: Train/val/test, métricas de negocio
- **Artifacts**: Modelo, configs, plots, requirements.txt
- **Tags**: Git commit, autor, dataset version

📺 Recursos Externos Recomendados

Ver RECURSOS_POR_MODULO.md para la lista completa.

🏷️	Recurso	Tipo
🔴	MLflow Tutorial - Krish Naik	Video
🟡	MLflow Complete Course	Video

Documentación oficial:
- MLflow Tracking
- MLflow Model Registry

🔗 Referencias del Glosario

Ver 21_GLOSARIO.md para definiciones de:
- **MLflow**: Plataforma de experiment tracking
- **Model Registry**: Registro de versiones de modelos
- **Artifact**: Archivo asociado a un experimento

✅ Ejercicios

Ver EJERCICIOS.md - Módulo 10:
- 10.1: MLflow básico (params, metrics, model)
- 10.2: Comparar múltiples experimentos

TESTING ML

GUÍA MLOPS V5.0

PORTFOLIO EDITION

11. Testing para Machine Learning

🎯 Objetivo del Módulo

Dominar el testing en proyectos ML para alcanzar `80` **%+ de coverage** sin tests frágiles ni falsos positivos.

```
╔═══════════════════════════════════════════════════════════════════╗
║                                                                   ╕
│  🚨 LA REALIDAD DEL ML SIN TESTS:                                 │
│                                                                   │
│  "El modelo funcionaba ayer, hoy da predicciones random"          │
│  "Cambié una línea y rompí todo el pipeline"                      │
│  "No sé si el bug está en los datos, el preprocesamiento, o el modelo"
│                                                                   │
│  🛡️ LA REALIDAD CON TESTS:                                        │
│                                                                   │
│  "CI me avisó que rompí algo antes de hacer merge"                │
│  "Sé exactamente qué componente falló"                            │
│  "Puedo refactorizar con confianza"                               │
╘                                                                   ╛
╚═══════════════════════════════════════════════════════════════════╝
```

📋 Contenido

1. La Pirámide de Testing en ML
2. Fixtures y conftest.py
3. Unit Tests: Funciones Individuales
4. Data Tests: Validación de Datos
5. Model Tests: Comportamiento del Modelo
6. Integration Tests: Pipeline Completo
7. Alcanzar `80`% Coverage

🧩 Cómo se aplica en este portafolio

- Cada uno de los tres proyectos tiene una carpeta `tests/` rica en ejemplos reales:
- **BankChurn-Predictor**: tests de pipeline de entrenamiento y métricas.
- **CarVision-Market-Intelligence**: tests de features, datos y modelo (incluidos en este módulo).
- **TelecomAI-Customer-Intelligence**: tests centrados en clasificación y contratos de datos.
- El workflow `ci-mlops.yml` ejecuta estos tests en matrix (`3` proyectos × `2` versiones de Python) y aplica thresholds de coverage (`79`–`80`%). Este módulo te da el modelo mental para entender y extender esos tests sin romper la pirámide de testing.

11.1 La Pirámide de Testing en ML

Analogía: Inspección de un Avión

```
╔═══════════════════════════════════════════════════════════════╗
║                                                               ╕
│  ✈️ ANTES DE CADA VUELO, SE INSPECCIONA:                      │
│                                                               │
│  NIVEL 1 - Componentes individuales (Unit Tests):             │
│  • Cada tornillo está apretado                                │
│  • Cada cable está conectado                                  │
│  • Cada sensor funciona                                       │
│                                                               │
│  NIVEL 2 - Sistemas (Integration Tests):                      │
│  • El motor arranca correctamente                             │
│  • Los flaps responden a los controles                        │
│  • El sistema hidráulico mantiene presión                     │
│                                                               │
│  NIVEL 3 - Vuelo de prueba (E2E Tests):                       │
│  • El avión despega, vuela, y aterriza                        │
│  • Todo funciona junto bajo condiciones reales                │
│                                                               │
│  EN ML ES IGUAL: Testeas componentes → sistemas → pipeline completo
╘                                                               ╛
╚═══════════════════════════════════════════════════════════════╝
```

La Pirámide Específica para ML

```
                    ┌─────────────┐
                    │   E2E Tests │ ← 5-10% de tests
```

```
                        │   (API real)    │   Lentos, costosos
                        └─────────────────┘   test_api_e2e.py
                                │
                        ┌───────────────────┐
                        │  Integration Tests  │ ← 15-20% de tests
                        │ (pipeline.fit())    │   Verifican interacción
                        └───────────────────┘   test_training.py
                                │
                ┌─────────────────────────────┐
                │          Model Tests        │ ← 20-25% de tests
                │ (predicciones, métricas, shapes) │   Específicos de ML
                └─────────────────────────────┘   test_model_logic.py
                                │
          ┌──────────────────────────────────┐
          │              Data Tests          │ ← 20-25% de tests
          │   (schema, rangos, distribuciones, NaN)  │   Validan datos
          └──────────────────────────────────┘   test_data.py
                                │
                                │
┌──────────────────────────────────────────────────────────────┐
│                          Unit Tests                          │ ← 30-40% de tests
│              (funciones individuales, transformers)          │   Rápidos, muchos
└──────────────────────────────────────────────────────────────┘   test_features.py
```

Coverage del Portafolio Real

Proyecto	Coverage	Tests	Tipo Principal
BankChurn	79.5%	45+	Unit + Integration
CarVision	97%	50+	Unit + Data + Model
TelecomAI	97%	35+	Unit + Integration

11.2 Fixtures y conftest.py

¿Qué es una Fixture?

```
╔═══════════════════════════════════════════════════════════════╗
║  🧪 FIXTURE = Datos o recursos preparados para tests          ║
║                                                               ║
║  Analogía del laboratorio:                                    ║
║  • Antes de cada experimento, preparas tus instrumentos       ║
║  • Los instrumentos son los mismos para varios experimentos   ║
║  • No los preparas desde cero cada vez                        ║
║                                                               ║
║  En pytest:                                                   ║
║  • Fixture prepara datos/modelos/configs                      ║
║  • Se reutiliza en múltiples tests                            ║
║  • Se limpia automáticamente después                          ║
╚═══════════════════════════════════════════════════════════════╝
```

conftest.py del Portafolio (CarVision)

```python
# tests/conftest.py - Código REAL del portafolio

import pytest
import pandas as pd
import numpy as np
from pathlib import Path
import tempfile
import yaml

# ═══════════════════════════════════════════════════════════════════
# FIXTURES DE DATOS
# ═══════════════════════════════════════════════════════════════════

@pytest.fixture
def sample_data() -> pd.DataFrame:
    """DataFrame pequeño para tests rápidos.

    Este fixture se usa en MUCHOS tests:
    - test_data.py: Verificar carga y limpieza
```

```
    - test_features.py: Verificar feature engineering
    - test_model.py: Verificar predicciones
    """
    return pd.DataFrame({
        "price": [15000, 25000, 35000, 45000, 55000],
        "model_year": [2015, 2018, 2020, 2019, 2021],
        "odometer": [80000, 45000, 20000, 30000, 10000],
        "model": ["ford f-150", "toyota camry", "honda civic",
                  "chevrolet silverado", "ford mustang"],
        "fuel": ["gas", "gas", "gas", "diesel", "gas"],
        "transmission": ["automatic", "automatic", "manual",
                         "automatic", "manual"],
        "condition": ["good", "excellent", "like new",
                      "good", "excellent"],
    })


@pytest.fixture
def sample_data_with_nulls() -> pd.DataFrame:
    """DataFrame con valores faltantes para probar imputación."""
    return pd.DataFrame({
        "price": [15000, None, 35000, 45000, None],
        "model_year": [2015, 2018, None, 2019, 2021],
        "odometer": [80000, None, 20000, 30000, 10000],
        "model": ["ford f-150", None, "honda civic", None, "ford mustang"],
        "fuel": ["gas", "gas", None, "diesel", "gas"],
        "transmission": [None, "automatic", "manual", "automatic", None],
    })


@pytest.fixture
def large_sample_data() -> pd.DataFrame:
    """DataFrame más grande para tests de performance."""
    np.random.seed(42)
    n = 1000
    return pd.DataFrame({
        "price": np.random.uniform(5000, 80000, n),
        "model_year": np.random.randint(2010, 2024, n),
        "odometer": np.random.uniform(0, 200000, n),
        "model": np.random.choice(["ford f-150", "toyota camry", "honda civic"], n),
        "fuel": np.random.choice(["gas", "diesel", "electric"], n),
        "transmission": np.random.choice(["automatic", "manual"], n),
    })


# ═══════════════════════════════════════════════════════════════════════
# FIXTURES DE CONFIGURACIÓN
# ═══════════════════════════════════════════════════════════════════════

@pytest.fixture
def sample_config() -> dict:
    """Configuración mínima para tests."""
    return {
        "seed": 42,
        "dataset_year": 2024,
        "paths": {
            "data_path": "data/raw/vehicles_us.csv",
            "artifacts_dir": "artifacts",
            "model_path": "artifacts/model.joblib",
        },
        "preprocessing": {
            "numeric_features": ["odometer", "vehicle_age"],
            "categorical_features": ["fuel", "transmission", "brand"],
            "drop_columns": ["price_per_mile", "price_category"],
            "filters": {
                "price_min": 1000,
                "price_max": 100000,
            }
        },
        "training": {
            "target": "price",
            "test_size": 0.2,
            "val_size": 0.1,
            "shuffle": True,
            "model": "random_forest",
```

```
            "random_forest_params": {
                "n_estimators": 10,  # Pocos para tests rápidos
                "max_depth": 5,
                "random_state": 42,
            }
        }
    }


@pytest.fixture
def temp_config_file(sample_config, tmp_path) -> Path:
    """Crea archivo config temporal para tests de carga."""
    config_path = tmp_path / "config.yaml"
    with open(config_path, "w") as f:
        yaml.dump(sample_config, f)
    return config_path


# ═══════════════════════════════════════════════════════════════════════════
# FIXTURES DE MODELO
# ═══════════════════════════════════════════════════════════════════════════

@pytest.fixture
def fitted_pipeline(sample_data, sample_config):
    """Pipeline entrenado para tests de predicción."""
    from src.carvision.training import build_pipeline
    from src.carvision.features import FeatureEngineer

    # Preparar datos
    fe = FeatureEngineer(current_year=2024)
    df = fe.transform(sample_data)

    X = df.drop(columns=["price"])
    y = df["price"]

    # Construir y entrenar pipeline
    # Nota: Usamos config simplificada para velocidad
    pipeline = build_pipeline(sample_config)
    pipeline.fit(X, y)

    return pipeline


# ═══════════════════════════════════════════════════════════════════════════
# FIXTURES ESPECIALES
# ═══════════════════════════════════════════════════════════════════════════

@pytest.fixture
def temp_artifacts_dir(tmp_path) -> Path:
    """Directorio temporal para artefactos."""
    artifacts = tmp_path / "artifacts"
    artifacts.mkdir()
    return artifacts


@pytest.fixture(scope="module")
def slow_fixture():
    """Fixture que tarda en crearse - se reutiliza en todo el módulo.

    scope="module" significa que se crea UNA vez por archivo de test,
    no una vez por cada test.
    """
    import time
    time.sleep(0.1)  # Simula operación lenta
    return {"expensive_resource": True}
```

Uso de Fixtures en Tests

```
# tests/test_features.py

def test_feature_engineer_creates_vehicle_age(sample_data):
    """Test que usa la fixture sample_data."""
    from src.carvision.features import FeatureEngineer
```

```
    fe = FeatureEngineer(current_year=2024)
    result = fe.transform(sample_data)

    assert "vehicle_age" in result.columns
    assert result["vehicle_age"].iloc[0] == 2024 - 2015  # 9 años


def test_pipeline_predicts_positive_prices(fitted_pipeline, sample_data):
    """Test que usa DOS fixtures."""
    X = sample_data.drop(columns=["price"])
    predictions = fitted_pipeline.predict(X)

    assert all(predictions > 0), "Precios deben ser positivos"
```

11.3 Unit Tests: Funciones Individuales

Qué Testear en Unit Tests

```
# tests/test_features.py - Código REAL del portafolio

import pytest
import pandas as pd
from src.carvision.features import FeatureEngineer


class TestFeatureEngineer:
    """Tests unitarios para FeatureEngineer."""

    # ═══════════════════════════════════════════════════════════════════════
    # TEST: Creación de features
    # ═══════════════════════════════════════════════════════════════════════

    def test_creates_vehicle_age(self, sample_data):
        """Verifica que vehicle_age se calcula correctamente."""
        fe = FeatureEngineer(current_year=2024)
        result = fe.transform(sample_data)

        assert "vehicle_age" in result.columns
        # 2024 - 2015 = 9 años para el primer registro
        assert result.loc[0, "vehicle_age"] == 9

    def test_creates_brand_from_model(self, sample_data):
        """Verifica que brand extrae la primera palabra de model."""
        fe = FeatureEngineer(current_year=2024)
        result = fe.transform(sample_data)

        assert "brand" in result.columns
        assert result.loc[0, "brand"] == "ford"  # "ford f-150" → "ford"
        assert result.loc[1, "brand"] == "toyota"  # "toyota camry" → "toyota"

    # ═══════════════════════════════════════════════════════════════════════
    # TEST: Manejo de edge cases
    # ═══════════════════════════════════════════════════════════════════════

    def test_handles_missing_model_year(self):
        """Verifica comportamiento con model_year faltante."""
        df = pd.DataFrame({
            "price": [15000],
            "model": ["ford f-150"],
            # Sin model_year
        })

        fe = FeatureEngineer(current_year=2024)
        result = fe.transform(df)

        # No debe crear vehicle_age si no hay model_year
        assert "vehicle_age" not in result.columns

    def test_handles_missing_model_column(self):
        """Verifica comportamiento sin columna model."""
        df = pd.DataFrame({
            "price": [15000],
            "model_year": [2015],
            # Sin model
```

```
        })

        fe = FeatureEngineer(current_year=2024)
        result = fe.transform(df)

        # No debe crear brand si no hay model
        assert "brand" not in result.columns

    # ═══════════════════════════════════════════════════════════════════════
    # TEST: Inmutabilidad
    # ═══════════════════════════════════════════════════════════════════════

    def test_does_not_modify_input(self, sample_data):
        """Verifica que el DataFrame original no se modifica."""
        original_columns = sample_data.columns.tolist()
        original_values = sample_data.copy()

        fe = FeatureEngineer(current_year=2024)
        _ = fe.transform(sample_data)

        # Columnas originales sin cambio
        assert sample_data.columns.tolist() == original_columns
        # Valores originales sin cambio
        pd.testing.assert_frame_equal(sample_data, original_values)

    # ═══════════════════════════════════════════════════════════════════════
    # TEST: Compatibilidad con sklearn
    # ═══════════════════════════════════════════════════════════════════════

    def test_fit_returns_self(self, sample_data):
        """Verifica que fit() retorna self (requerido por sklearn)."""
        fe = FeatureEngineer(current_year=2024)
        result = fe.fit(sample_data)

        assert result is fe

    def test_works_in_pipeline(self, sample_data):
        """Verifica que funciona dentro de un Pipeline sklearn."""
        from sklearn.pipeline import Pipeline
        from sklearn.preprocessing import StandardScaler

        pipe = Pipeline([
            ("features", FeatureEngineer(current_year=2024)),
            ("scaler", StandardScaler())
        ])

        # No debe lanzar excepción
        # (Solo probamos que no falla, no el resultado)
        try:
            # Seleccionar solo columnas numéricas para StandardScaler
            numeric_cols = ["price", "model_year", "odometer"]
            result = pipe.fit_transform(sample_data[numeric_cols])
            assert result is not None
        except Exception as e:
            pytest.fail(f"Pipeline falló: {e}")
```

11.4 Data Tests: Validación de Datos

Tests de Schema y Calidad de Datos

```
# tests/test_data.py - Código REAL del portafolio

import pytest
import pandas as pd
import numpy as np
from src.carvision.data import load_data, clean_data


class TestDataLoading:
    """Tests de carga y validación de datos."""

    # ═══════════════════════════════════════════════════════════════════════
    # TEST: Schema validation
    # ═══════════════════════════════════════════════════════════════════════
```

```
    def test_required_columns_exist(self, sample_data):
        """Verifica que existen las columnas requeridas."""
        required = ["price", "model_year", "odometer", "model", "fuel"]

        for col in required:
            assert col in sample_data.columns, f"Falta columna: {col}"

    def test_column_types(self, sample_data):
        """Verifica tipos de datos correctos."""
        # Numéricas
        assert pd.api.types.is_numeric_dtype(sample_data["price"])
        assert pd.api.types.is_numeric_dtype(sample_data["model_year"])
        assert pd.api.types.is_numeric_dtype(sample_data["odometer"])

        # Categóricas/String
        assert pd.api.types.is_object_dtype(sample_data["model"])
        assert pd.api.types.is_object_dtype(sample_data["fuel"])

    # ═══════════════════════════════════════════════════════════════════
    # TEST: Value ranges
    # ═══════════════════════════════════════════════════════════════════

    def test_price_is_positive(self, sample_data):
        """Verifica que precios son positivos."""
        assert (sample_data["price"] > 0).all(), "Hay precios <= 0"

    def test_price_in_reasonable_range(self, sample_data):
        """Verifica que precios están en rango razonable."""
        assert sample_data["price"].min() >= 100, "Precio muy bajo (posible error)"
        assert sample_data["price"].max() <= 500000, "Precio muy alto (posible error)"

    def test_model_year_in_range(self, sample_data):
        """Verifica que años son razonables."""
        current_year = pd.Timestamp.now().year

        assert sample_data["model_year"].min() >= 1900, "Año muy antiguo"
        assert sample_data["model_year"].max() <= current_year + 1, "Año futuro"

    def test_odometer_is_non_negative(self, sample_data):
        """Verifica que odometer no es negativo."""
        assert (sample_data["odometer"] >= 0).all(), "Hay odometer negativo"

    # ═══════════════════════════════════════════════════════════════════
    # TEST: Categorical values
    # ═══════════════════════════════════════════════════════════════════

    def test_fuel_valid_values(self, sample_data):
        """Verifica que fuel tiene valores válidos."""
        valid_fuels = {"gas", "diesel", "electric", "hybrid", "other"}
        actual_fuels = set(sample_data["fuel"].dropna().unique())

        invalid = actual_fuels - valid_fuels
        assert len(invalid) == 0, f"Valores de fuel inválidos: {invalid}"

    def test_transmission_valid_values(self, sample_data):
        """Verifica que transmission tiene valores válidos."""
        valid = {"automatic", "manual", "other"}
        actual = set(sample_data["transmission"].dropna().unique())

        invalid = actual - valid
        assert len(invalid) == 0, f"Valores de transmission inválidos: {invalid}"


class TestDataCleaning:
    """Tests de limpieza de datos."""

    def test_clean_data_removes_invalid_prices(self, sample_data):
        """Verifica que clean_data filtra precios fuera de rango."""
        # Añadir registro con precio inválido
        bad_data = pd.concat([
            sample_data,
            pd.DataFrame({"price": [100], "model_year": [2020],
                          "odometer": [1000], "model": ["test"],
                          "fuel": ["gas"], "transmission": ["automatic"],
```

```
                        "condition": ["good"]})
        ], ignore_index=True)

        filters = {"price_min": 1000, "price_max": 100000}
        cleaned = clean_data(bad_data, filters=filters)

        # El registro con price=100 debe ser eliminado
        assert len(cleaned) == len(sample_data)
        assert (cleaned["price"] >= 1000).all()

    def test_clean_data_handles_nulls(self, sample_data_with_nulls):
        """Verifica que clean_data no falla con NaN."""
        # No debe lanzar excepción
        filters = {"price_min": 1000}
        cleaned = clean_data(sample_data_with_nulls, filters=filters)

        assert cleaned is not None
        # Registros con price=None deben ser eliminados o manejados
        assert len(cleaned) <= len(sample_data_with_nulls)
```

11.5 Model Tests: Comportamiento del Modelo

Tests Específicos de ML

```
# tests/test_model_logic.py - Código REAL del portafolio

import pytest
import numpy as np
import pandas as pd


class TestModelPredictions:
    """Tests de comportamiento del modelo."""

    # ═══════════════════════════════════════════════════════════════════
    # TEST: Output shape y tipo
    # ═══════════════════════════════════════════════════════════════════

    def test_predict_returns_correct_shape(self, fitted_pipeline, sample_data):
        """Verifica que predict retorna un array del tamaño correcto."""
        X = sample_data.drop(columns=["price"])
        predictions = fitted_pipeline.predict(X)

        assert len(predictions) == len(X), "Número de predicciones incorrecto"

    def test_predict_returns_numeric(self, fitted_pipeline, sample_data):
        """Verifica que predicciones son numéricas."""
        X = sample_data.drop(columns=["price"])
        predictions = fitted_pipeline.predict(X)

        assert np.issubdtype(predictions.dtype, np.number), "Predicciones no son numéricas"

    def test_predict_no_nan(self, fitted_pipeline, sample_data):
        """Verifica que no hay NaN en predicciones."""
        X = sample_data.drop(columns=["price"])
        predictions = fitted_pipeline.predict(X)

        assert not np.isnan(predictions).any(), "Hay NaN en predicciones"

    # ═══════════════════════════════════════════════════════════════════
    # TEST: Rangos razonables
    # ═══════════════════════════════════════════════════════════════════

    def test_predictions_are_positive(self, fitted_pipeline, sample_data):
        """Verifica que precios predichos son positivos."""
        X = sample_data.drop(columns=["price"])
        predictions = fitted_pipeline.predict(X)

        assert (predictions > 0).all(), "Hay predicciones <= 0"

    def test_predictions_in_training_range(self, fitted_pipeline, sample_data):
        """Verifica que predicciones están en rango similar al training."""
        X = sample_data.drop(columns=["price"])
        y = sample_data["price"]
```

```
        predictions = fitted_pipeline.predict(X)

        # Predicciones deben estar dentro de un margen razonable
        min_price = y.min() * 0.1  # 10% del mínimo
        max_price = y.max() * 3.0  # 300% del máximo

        assert predictions.min() >= min_price, "Predicción muy baja"
        assert predictions.max() <= max_price, "Predicción muy alta"

    # ═══════════════════════════════════════════════════════════════════
    # TEST: Consistencia (determinismo)
    # ═══════════════════════════════════════════════════════════════════

    def test_predictions_are_deterministic(self, fitted_pipeline, sample_data):
        """Verifica que mismos inputs dan mismos outputs."""
        X = sample_data.drop(columns=["price"])

        pred1 = fitted_pipeline.predict(X)
        pred2 = fitted_pipeline.predict(X)

        np.testing.assert_array_equal(pred1, pred2,
            "Predicciones no son determinísticas")

    # ═══════════════════════════════════════════════════════════════════
    # TEST: Sensibilidad a features
    # ═══════════════════════════════════════════════════════════════════

    def test_higher_odometer_lower_price(self, fitted_pipeline, sample_data):
        """Verifica que mayor odometer tiende a menor precio.

        Este es un test de "sanity check": verifica que el modelo
        aprendió relaciones básicas del dominio.
        """
        X = sample_data.drop(columns=["price"]).copy()

        # Predicción con odometer original
        pred_low_odo = fitted_pipeline.predict(X)

        # Aumentar odometer significativamente
        X_high_odo = X.copy()
        X_high_odo["odometer"] = X["odometer"] * 3
        pred_high_odo = fitted_pipeline.predict(X_high_odo)

        # En promedio, mayor odometer → menor precio
        # (No requiere que TODOS sean menores, solo el promedio)
        assert pred_high_odo.mean() < pred_low_odo.mean(), \
            "Modelo no aprendió que mayor odometer = menor precio"

    def test_newer_car_higher_price(self, fitted_pipeline, sample_data):
        """Verifica que autos más nuevos tienden a mayor precio."""
        X = sample_data.drop(columns=["price"]).copy()

        # Predicción con model_year original
        pred_original = fitted_pipeline.predict(X)

        # Hacer autos 5 años más nuevos
        X_newer = X.copy()
        X_newer["model_year"] = X["model_year"] + 5
        pred_newer = fitted_pipeline.predict(X_newer)

        # En promedio, más nuevo → mayor precio
        assert pred_newer.mean() > pred_original.mean(), \
            "Modelo no aprendió que autos nuevos cuestan más"


class TestModelMetrics:
    """Tests de métricas del modelo."""

    def test_rmse_below_threshold(self, fitted_pipeline, sample_data):
        """Verifica que RMSE está por debajo de umbral aceptable."""
        from sklearn.metrics import mean_squared_error

        X = sample_data.drop(columns=["price"])
        y = sample_data["price"]
        predictions = fitted_pipeline.predict(X)
```

```
        rmse = np.sqrt(mean_squared_error(y, predictions))

        # RMSE debe ser menor que el 50% del precio promedio
        # (umbral arbitrario para sample data pequeño)
        threshold = y.mean() * 0.5
        assert rmse < threshold, f"RMSE={rmse:.2f} > threshold={threshold:.2f}"

    def test_r2_above_threshold(self, fitted_pipeline, sample_data):
        """Verifica que R² está por encima de umbral mínimo."""
        from sklearn.metrics import r2_score

        X = sample_data.drop(columns=["price"])
        y = sample_data["price"]
        predictions = fitted_pipeline.predict(X)

        r2 = r2_score(y, predictions)

        # R² debe ser positivo (mejor que predecir la media)
        # Nota: Con sample data pequeño, R² puede ser bajo
        assert r2 > 0.0, f"R²={r2:.3f} <= 0 (peor que baseline)"
```

11.6 Integration Tests: Pipeline Completo

Tests End-to-End

```
# tests/test_main_workflow.py - Código REAL del portafolio

import pytest
import pandas as pd
import numpy as np
from pathlib import Path
import tempfile
import joblib


class TestTrainingWorkflow:
    """Tests de integración del flujo completo de entrenamiento."""

    def test_full_training_pipeline(self, sample_data, sample_config, tmp_path):
        """Test end-to-end: datos → entrenamiento → modelo guardado."""
        from src.carvision.training import train_model

        # Configurar paths temporales
        sample_config["paths"]["artifacts_dir"] = str(tmp_path)
        sample_config["paths"]["model_path"] = str(tmp_path / "model.joblib")

        # Guardar datos temporales
        data_path = tmp_path / "data.csv"
        sample_data.to_csv(data_path, index=False)
        sample_config["paths"]["data_path"] = str(data_path)

        # Ejecutar entrenamiento
        result = train_model(sample_config)

        # Verificaciones
        assert "rmse" in result, "Falta métrica RMSE"
        assert result["rmse"] > 0, "RMSE debe ser positivo"

        # Verificar que modelo se guardó
        model_path = Path(sample_config["paths"]["model_path"])
        assert model_path.exists(), "Modelo no se guardó"

        # Verificar que modelo se puede cargar
        model = joblib.load(model_path)
        assert model is not None, "Modelo no se puede cargar"

    def test_training_creates_all_artifacts(self, sample_data, sample_config, tmp_path):
        """Verifica que entrenamiento crea todos los artefactos esperados."""
        from src.carvision.training import train_model

        # Setup
        sample_config["paths"]["artifacts_dir"] = str(tmp_path)
        sample_config["paths"]["model_path"] = str(tmp_path / "model.joblib")
```

```
        sample_config["paths"]["metrics_path"] = str(tmp_path / "metrics.json")

        data_path = tmp_path / "data.csv"
        sample_data.to_csv(data_path, index=False)
        sample_config["paths"]["data_path"] = str(data_path)

        # Train
        train_model(sample_config)

        # Verificar artefactos
        assert (tmp_path / "model.joblib").exists(), "Falta model.joblib"
        # metrics.json es opcional en algunos configs

    def test_loaded_model_predicts_correctly(self, sample_data, sample_config, tmp_path):
        """Verifica que modelo guardado predice igual que antes de guardar."""
        from src.carvision.training import train_model, build_pipeline
        from src.carvision.features import FeatureEngineer

        # Setup y entrenamiento
        sample_config["paths"]["artifacts_dir"] = str(tmp_path)
        model_path = tmp_path / "model.joblib"
        sample_config["paths"]["model_path"] = str(model_path)

        data_path = tmp_path / "data.csv"
        sample_data.to_csv(data_path, index=False)
        sample_config["paths"]["data_path"] = str(data_path)

        train_model(sample_config)

        # Cargar modelo
        loaded_model = joblib.load(model_path)

        # Predecir con nuevo dato
        new_data = pd.DataFrame({
            "model_year": [2020],
            "odometer": [30000],
            "model": ["ford f-150"],
            "fuel": ["gas"],
            "transmission": ["automatic"],
        })

        prediction = loaded_model.predict(new_data)

        # Verificaciones básicas
        assert len(prediction) == 1
        assert prediction[0] > 0
        assert not np.isnan(prediction[0])


class TestAPIWorkflow:
    """Tests de integración del API."""

    @pytest.mark.slow
    def test_api_prediction_endpoint(self, fitted_pipeline, tmp_path):
        """Test E2E del endpoint de predicción."""
        from fastapi.testclient import TestClient
        import sys

        # Guardar modelo para el API
        model_path = tmp_path / "model.joblib"
        joblib.dump(fitted_pipeline, model_path)

        # Importar app (puede requerir configuración de paths)
        # Este test asume que ARTIFACTS_DIR está configurado
        import os
        os.environ["ARTIFACTS_DIR"] = str(tmp_path)

        try:
            from app.fastapi_app import app
            client = TestClient(app)

            # Request de predicción
            response = client.post("/predict", json={
                "model_year": 2020,
                "odometer": 30000,
```

```
            "model": "ford f-150",
            "fuel": "gas",
            "transmission": "automatic"
        })

        assert response.status_code == 200
        data = response.json()
        assert "prediction" in data
        assert data["prediction"] > 0
    except ImportError:
        pytest.skip("FastAPI app not available")
```

🧨 Errores habituales y cómo depurarlos en testing ML

Aunque pytest es muy potente, en ML es fácil caer en tests frágiles o engañosos. Estos son los patrones más comunes y cómo atacarlos.

1) Tests que dependen de datos reales o externos

Síntomas típicos

- Tests que leen de `data/raw/...` o llaman APIs externas.
- Fallan solo en CI o solo en ciertas máquinas.

Cómo identificarlo

- Busca en tests accesos directos a rutas del proyecto o a recursos externos.
- Revisa que tus fixtures (`sample_data`, `sample_config`, etc.) no lean de archivos reales salvo cuando se prueban funciones de I/O.

Cómo corregirlo

- Usa **fixtures sintéticas** en memoria para la mayoría de tests.
- Deja el acceso a disco/red solo en tests de integración marcados (`@pytest.mark.integration` o `@pytest.mark.slow`).

2) Fixtures mal definidas (estado compartido, side-effects)

Síntomas típicos

- Tests que pasan individualmente pero fallan cuando se ejecutan todos juntos.
- Mutación de `DataFrame` compartido entre tests.

Cómo identificarlo

- Revisa que tus fixtures devuelvan **nuevas instancias** o copias (`df.copy()`) cuando sea necesario.
- Usa `scope="function"` por defecto; solo usa `module`/`session` para recursos pesados cuidadosamente diseñados.

Cómo corregirlo

- Asegura inmutabilidad en el código (ej. `X = X.copy()` en transformers) y en los tests (no reutilizar el mismo objeto mutable entre múltiples asserts sin reset).

3) Coverage alto pero sin cubrir lo importante

Síntomas típicos

- Reporte de coverage muestra 80–90%, pero:
- No hay tests de datos.
- No hay tests de comportamiento del modelo.
- Solo se testea “feliz path”.

Cómo identificarlo

- Usa `--cov-report=term-missing` para ver **qué líneas** no se cubren.
- Verifica que las capas clave (data, features, training, prediction) tienen tests dedicados.

Cómo corregirlo

- Añade tests para:
- Validación de datos (rangos, NaN, tipos).
- Comportamiento básico del modelo (shapes, ranges, determinismo).
- Al menos un flujo de integración (`train_model`, API `/predict`).

4) Tests lentos que bloquean el flujo de trabajo

Síntomas típicos

- `pytest` tarda minutos porque ejecuta entrenamiento completo en cada test.

Cómo identificarlo

- Localiza tests que entrenan modelos con muchos datos o hiperparámetros pesados.

Cómo corregirlo

- Para tests, usa:
- **datasets pequeños**.
- Modelos simplificados (pocos árboles, menor profundidad).
- Marcadores `@pytest.mark.slow` para separar tests pesados.
- Ajusta tu CI para ejecutar rápida y frecuentemente los tests rápidos, y los lentos solo en ciertas ramas.

5) Patrón general de debugging en tests ML

1. **Reproduce el fallo en local** con el mismo comando que CI (`pytest` con las mismas flags).
2. **Aísla el test problemático** usando `-k` o el nombre del test.
3. **Inspecciona fixtures y datos**: asegúrate de que no hay estado compartido inesperado.
4. **Conecta el problema** con la capa correspondiente (datos, features, modelo, API) y ajusta los tests para cubrir el caso real que quieres proteger.

Con esta mentalidad, los tests dejan de ser una carga y se convierten en la red de seguridad que te permite refactorizar con confianza.

11.7 Alcanzar 80% Coverage

Configuración de pytest-cov

```
# pyproject.toml

[tool.pytest.ini_options]
testpaths = ["tests"]
python_files = ["test_*.py"]
python_classes = ["Test*"]
python_functions = ["test_*"]
addopts = [
    "-v",
    "--cov=src/carvision",
    "--cov-report=term-missing",
    "--cov-report=html",
    "--cov-fail-under=80",
]
markers = [
    "slow: marks tests as slow (deselect with '-m \"not slow\"')",
    "integration: marks tests as integration tests",
]

[tool.coverage.run]
source = ["src"]
omit = [
    "tests/*",
    "*/__init__.py",
    "*/visualization.py",  # Excluir código de UI
]

[tool.coverage.report]
fail_under = 80
exclude_lines = [
    "pragma: no cover",
    "if __name__ == .__main__.:",
    "raise NotImplementedError",
]
```

Ejecutar Tests con Coverage

```
# Tests rápidos (sin slow)
pytest -m "not slow"

# Todos los tests con coverage
```

```
pytest --cov=src/carvision --cov-report=term-missing

# Solo ver coverage sin ejecutar tests
pytest --cov=src/carvision --cov-report=html
# Abre htmlcov/index.html en el navegador

# Verificar que pasa el threshold
pytest --cov-fail-under=80
```

Estrategias para Aumentar Coverage

```
╔═══════════════════════════════════════════════════════════════╗
║  📈 CÓMO PASAR DE 60% A 80% COVERAGE                          ╒
╟═══════════════════════════════════════════════════════════════╢
│                                                               │
│  1. IDENTIFICAR GAPS                                          │
│     pytest --cov-report=term-missing                          │
│     → Muestra líneas NO cubiertas                             │
│                                                               │
│  2. PRIORIZAR                                                 │
│     • Lógica de negocio crítica (training, prediction)        │
│     • Código que maneja errores                               │
│     • Branches condicionales (if/else)                        │
│                                                               │
│  3. EXCLUIR LO QUE NO VALE LA PENA                            │
│     • Código de visualización (Streamlit, plots)              │
│     • Scripts de utilidad one-off                             │
│     • Código de terceros                                      │
│                                                               │
│  4. TESTEAR EDGE CASES                                        │
│     • ¿Qué pasa con NaN?                                      │
│     • ¿Qué pasa con lista vacía?                              │
│     • ¿Qué pasa con tipos incorrectos?                        │
╘═══════════════════════════════════════════════════════════════╛
```

✅ Checkpoint: ¿Completaste el Módulo?

Antes de continuar, verifica:

- [ ] Tienes `tests/conftest.py` con fixtures reutilizables
- [ ] Tienes tests unitarios para tus transformers
- [ ] Tienes tests de validación de datos
- [ ] Tienes tests de comportamiento del modelo
- [ ] Tienes al menos un test de integración
- [ ] Tu coverage es >= 80%

📦 Cómo se Usó en el Portafolio

Los 3 proyectos del portafolio implementan testing profesional con 80%+ coverage:

Coverage por Proyecto

Proyecto	Coverage	Tests	Archivos Clave
BankChurn	79%+	45+	`tests/conftest.py`, `test_pipeline.py`
CarVision	97%	50+	`tests/test_features.py`, `test_data.py`
TelecomAI	97%	35+	`tests/test_training.py`

conftest.py Real (CarVision)

```
# CarVision-Market-Intelligence/tests/conftest.py
import pytest
import pandas as pd

@pytest.fixture
def sample_df():
    """DataFrame de prueba con datos realistas."""
    return pd.DataFrame({
```

```
        'model': ['Ford F-150', 'Toyota Camry'],
        'model_year': [2020, 2019],
        'odometer': [50000, 30000],
        'price': [35000, 25000]
    })

@pytest.fixture
def config():
    """Configuración de prueba."""
    return {
        'data': {'target_column': 'price'},
        'model': {'random_state': 42}
    }
```

Estructura de Tests

```
tests/
├── conftest.py           # Fixtures compartidas
├── test_config.py        # Tests de configuración
├── test_data.py          # Tests de carga/validación
├── test_features.py      # Tests de FeatureEngineer
├── test_pipeline.py      # Tests de pipeline
├── test_training.py      # Tests de entrenamiento
└── test_api.py           # Tests de FastAPI
```

🔧 Ejercicio: Ejecuta Tests Reales

```
# 1. Ejecuta tests de CarVision
cd CarVision-Market-Intelligence
pytest tests/ -v --cov=src/carvision --cov-report=term-missing

# 2. Ve qué líneas NO están cubiertas
pytest --cov-report=html  # Genera htmlcov/index.html

# 3. Compara con BankChurn
cd ../BankChurn-Predictor
pytest tests/ -v --cov=src/bankchurn
```

💼 Consejos Profesionales

Recomendaciones para destacar en entrevistas y proyectos reales

Para Entrevistas

1. **Testing ML es diferente**: Explica tests de datos, modelo, y serving (no solo código).
2. **Great Expectations**: Menciona que usas validación de datos como parte del pipeline.
3. **Coverage no es todo**: Un modelo con 100% coverage puede fallar en producción.

Para Proyectos Reales

Tipo de Test	Qué Verificar
Data Tests	Schema, rangos, distribuciones, nulls
Model Tests	Métricas mínimas, overfitting, invariancia
Integration	Pipeline end-to-end, API responses
Performance	Latencia, throughput, memory

Estrategia de Testing ML

```
Unit Tests:     Transformadores individuales
Integration:    Pipeline completo con datos sintéticos
Validation:     Métricas en holdout real
Monitoring:     Drift detection en producción
```

📺 Recursos Externos Recomendados

Ver RECURSOS_POR_MODULO.md para la lista completa.

🏷️	Recurso	Tipo
🔴	pytest Tutorial - ArjanCodes	Video
🟡	Testing for Data Science - Eric Ma	Video

Documentación oficial:
- pytest Documentation
- pytest-cov
- Great Expectations - Data validation

🔗 Referencias del Glosario

Ver 21_GLOSARIO.md para definiciones de:
- **conftest.py**: Fixtures compartidas de pytest
- **Coverage**: Porcentaje de código ejecutado por tests
- **Fixture**: Setup reutilizable para tests

✅ Ejercicios

Ver EJERCICIOS.md - Módulo 11:
- 11.1: Test de validación de datos
- 11.2: Test de pipeline ML

CI/CD

GUÍA MLOPS V5.0

PORTFOLIO EDITION

13. CI/CD con GitHub Actions

🎯 Objetivo del Módulo

Implementar un pipeline CI/CD profesional que valide automáticamente tu código en cada push, como el workflow `ci-mlops.yml` del portafolio.

```
╔═══════════════════════════════════════════════════════════════════╗
╗                                                                   ╗
╗  🔄 CI/CD = Tu Guardián Automático                                ╗
╗                                                                   ╗
╗  ANTES (sin CI/CD):                                               ╗
╗  • "Olvidé correr los tests antes de mergear"                     ╗
╗  • "Rompí producción con un cambio pequeño"                       ╗
╗  • "No sabía que mi código no pasaba linting"                     ╗
╗                                                                   ╗
╗  DESPUÉS (con CI/CD):                                             ╗
╗  • Cada push ejecuta tests automáticamente                        ╗
╗  • No puedes mergear si los tests fallan                          ╗
╗  • Coverage, linting, y seguridad verificados siempre             ╗
╗                                                                   ╗
╚═══════════════════════════════════════════════════════════════════╝
```

📋 Contenido

1. Anatomía de un Workflow
2. Matrix Testing: Múltiples Versiones
3. Coverage Enforcement
4. Security Scanning
5. Docker Build y Push
6. El Workflow Completo del Portafolio

13.1 Anatomía de un Workflow

Estructura Básica

```
# .github/workflows/ci.yml

name: CI Pipeline                    # Nombre visible en GitHub

on:                                   # ¿Cuándo ejecutar?
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]

jobs:                                 # ¿Qué ejecutar?
  test:
    runs-on: ubuntu-latest           # Sistema operativo
    steps:                           # Pasos secuenciales
      - uses: actions/checkout@v4    # Paso 1: Descargar código
      - uses: actions/setup-python@v5 # Paso 2: Configurar Python
        with:
          python-version: '3.11'
      - run: pip install -r requirements.txt  # Paso 3: Instalar deps
      - run: pytest                           # Paso 4: Correr tests
```

Analogía: La Línea de Inspección de Calidad

```
╔═══════════════════════════════════════════════════════════════════╗
╗  🏭 IMAGINA UNA FÁBRICA DE AUTOS:                                 ╗
╗                                                                   ╗
╗  Workflow = Línea de inspección de calidad                         ╗
╗                                                                   ╗
╗  on (trigger):                                                    ╗
╗  → "Cada vez que un auto nuevo llega a la línea"                   ╗
╗                                                                   ╗
╗  jobs:                                                            ╗
╗  → Diferentes estaciones de inspección                             ╗
╗                                                                   ╗
╗  steps:                                                           ╗
```

```
│  → Tareas específicas en cada estación                                  │
│                                                                         │
│  ┌──────────┐   ┌──────────┐   ┌──────────┐   ┌──────────┐             │
│  │ Checkout │──►│ Install  │──►│  Test    │──►│  Build   │             │
│  │  (get    │   │  (prep   │   │  (run    │   │ (create  │             │
│  │  code)   │   │  tools)  │   │  tests)  │   │ Docker)  │             │
│  └──────────┘   └──────────┘   └──────────┘   └──────────┘             │
│                                                                         │
└─────────────────────────────────────────────────────────────────────────┘
```

13.2 Matrix Testing: Múltiples Versiones

El Problema: "Funciona en mi versión de Python"

```yaml
# ❌ ANTES: Solo pruebas con una versión
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/setup-python@v5
        with:
          python-version: '3.11'  # ¿Y si alguien usa 3.12?
```

La Solución: Matrix Strategy

```yaml
# ✅ DESPUÉS: Pruebas con múltiples versiones
# Código REAL de ci-mlops.yml del portafolio

jobs:
  tests:
    name: Tests & Coverage
    runs-on: ubuntu-latest
    strategy:
      fail-fast: false                  # No cancelar otros si uno falla
      matrix:
        python-version: ['3.11', '3.12']  # 2 versiones de Python
        project:                           # 3 proyectos
          - BankChurn-Predictor
          - CarVision-Market-Intelligence
          - TelecomAI-Customer-Intelligence

    # Esto crea 2 x 3 = 6 jobs paralelos:
    # - BankChurn con Python 3.11
    # - BankChurn con Python 3.12
    # - CarVision con Python 3.11
    # - CarVision con Python 3.12
    # - TelecomAI con Python 3.11
    # - TelecomAI con Python 3.12

    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Set up Python ${{ matrix.python-version }}
        uses: actions/setup-python@v5
        with:
          python-version: ${{ matrix.python-version }}
          cache: 'pip'  # Cache de dependencias para velocidad

      - name: Install dependencies
        working-directory: ${{ matrix.project }}
        run: |
          python -m pip install --upgrade pip
          pip install -r requirements.txt
          pip install pytest pytest-cov

      - name: Run tests
        working-directory: ${{ matrix.project }}
        run: pytest --cov=src/ --cov-fail-under=80
```

Visualización del Matrix

```
                    Python 3.11          Python 3.12
                  ┌──────────────┐     ┌──────────────┐
BankChurn         │   Job 1      │     │   Job 2      │
                  │   ✅ Pass    │     │   ✅ Pass    │
                  └──────────────┘     └──────────────┘

                  ┌──────────────┐     ┌──────────────┐
CarVision         │   Job 3      │     │   Job 4      │
                  │   ✅ Pass    │     │   ✅ Pass    │
                  └──────────────┘     └──────────────┘

                  ┌──────────────┐     ┌──────────────┐
TelecomAI         │   Job 5      │     │   Job 6      │
                  │   ✅ Pass    │     │   ✅ Pass    │
                  └──────────────┘     └──────────────┘

Total: 6 jobs ejecutándose EN PARALELO
```

13.3 Coverage Enforcement

Thresholds por Proyecto

```
# Código REAL de ci-mlops.yml

- name: Run tests with coverage
  working-directory: ${{ matrix.project }}
  run: |
    # Cada proyecto puede tener diferente threshold
    if [ "${{ matrix.project }}" = "BankChurn-Predictor" ]; then
      COV_TARGET="src"
      THRESHOLD=79
    elif [ "${{ matrix.project }}" = "CarVision-Market-Intelligence" ]; then
      COV_TARGET="src/carvision"
      THRESHOLD=80
    else
      COV_TARGET="src/telecom"
      THRESHOLD=80
    fi

    pytest --maxfail=1 --disable-warnings -q \
      -m "not slow" \
      --cov=$COV_TARGET \
      --cov-report=xml \
      --cov-report=term-missing \
      --cov-fail-under=$THRESHOLD  # ← FALLA si está por debajo
```

Upload de Coverage a Codecov

```
- name: Upload coverage to Codecov
  uses: codecov/codecov-action@v5
  with:
    files: ${{ matrix.project }}/coverage.xml
    flags: ${{ matrix.project }}
    name: ${{ matrix.project }}-coverage-${{ matrix.python-version }}
    fail_ci_if_error: false  # No fallar si Codecov tiene problemas

- name: Upload coverage artifact
  uses: actions/upload-artifact@v5
  with:
    name: coverage-${{ matrix.project }}-py${{ matrix.python-version }}
    path: ${{ matrix.project }}/coverage.xml
    retention-days: 30
```

13.4 Security Scanning

Múltiples Capas de Seguridad

```
# Job de seguridad - Código REAL del portafolio

security-scan:
```

```
  name: Security Scan
  runs-on: ubuntu-latest
  needs: [tests]  # Solo corre si tests pasan

  steps:
    - uses: actions/checkout@v4
      with:
        fetch-depth: 0  # Necesario para gitleaks (analiza historial)

    # 1. GITLEAKS: Detecta secretos en el código
    - name: Gitleaks (Secret Detection)
      uses: gitleaks/gitleaks-action@v2
      env:
        GITHUB_TOKEN: ${{ secrets.GITHUB_TOKEN }}

    # 2. BANDIT: Análisis de seguridad de Python
    - name: Set up Python
      uses: actions/setup-python@v5
      with:
        python-version: '3.12'

    - name: Run Bandit
      run: |
        pip install bandit
        for project in BankChurn-Predictor CarVision-Market-Intelligence TelecomAI-Customer-Intelligence; do
          echo "Scanning $project..."
          bandit -r "$project/src" -f json -o "bandit-$project.json" || true
        done

    # 3. PIP-AUDIT: Vulnerabilidades en dependencias
    - name: Run pip-audit
      run: |
        pip install pip-audit
        for project in BankChurn-Predictor CarVision-Market-Intelligence TelecomAI-Customer-Intelligence; do
          echo "Auditing $project..."
          pip-audit -r "$project/requirements.txt" --format json || true
        done
```

TRIVY: Escaneo de Imágenes Docker

```
docker-security:
  name: Docker Security Scan
  runs-on: ubuntu-latest
  needs: [docker-build]

  steps:
    - name: Run Trivy vulnerability scanner
      uses: aquasecurity/trivy-action@master
      with:
        image-ref: 'ml-portfolio-bankchurn:latest'
        format: 'sarif'
        output: 'trivy-results.sarif'
        severity: 'CRITICAL,HIGH'

    - name: Upload Trivy scan results
      uses: github/codeql-action/upload-sarif@v3
      with:
        sarif_file: 'trivy-results.sarif'
```

13.5 Docker Build y Push

Build Multi-Proyecto

```
docker-build:
  name: Docker Build
  runs-on: ubuntu-latest
  needs: [tests, quality-gates]
  if: github.event_name == 'push' && github.ref == 'refs/heads/main'

  strategy:
    matrix:
      include:
        - project: BankChurn-Predictor
          image: ml-portfolio-bankchurn
```

```
        - project: CarVision-Market-Intelligence
          image: ml-portfolio-carvision
        - project: TelecomAI-Customer-Intelligence
          image: ml-portfolio-telecom

  steps:
    - uses: actions/checkout@v4

    - name: Set up Docker Buildx
      uses: docker/setup-buildx-action@v3

    - name: Login to GitHub Container Registry
      uses: docker/login-action@v3
      with:
        registry: ghcr.io
        username: ${{ github.actor }}
        password: ${{ secrets.GITHUB_TOKEN }}

    - name: Build and push
      uses: docker/build-push-action@v5
      with:
        context: ./${{ matrix.project }}
        push: true
        tags: |
          ghcr.io/${{ github.repository_owner }}/${{ matrix.image }}:latest
          ghcr.io/${{ github.repository_owner }}/${{ matrix.image }}:${{ github.sha }}
        cache-from: type=gha
        cache-to: type=gha,mode=max
```

13.6 El Workflow Completo del Portafolio

Diagrama del Pipeline

```
┌──────────────────────────────────────────────────────────────────────────┐
│                        CI/CD Pipeline: ci-mlops.yml                      │
├──────────────────────────────────────────────────────────────────────────┤
│                                                                          │
│  TRIGGER: push to main/develop OR pull_request to main                   │
│                                                                          │
│  ┌──────────────────────────────────────────────────────────────────┐    │
│  │                         JOB 1: tests                             │    │
│  │  Matrix: Python 3.11/3.12 × 3 proyectos = 6 jobs paralelos       │    │
│  │                                                                  │    │
│  │  Steps:                                                          │    │
│  │  1. Checkout code                                                │    │
│  │  2. Setup Python (con cache)                                     │    │
│  │  3. Install dependencies                                         │    │
│  │  4. Run linting (flake8, black, isort)                           │    │
│  │  5. Run tests with coverage                                      │    │
│  │  6. Upload coverage to Codecov                                   │    │
│  └──────────────────────────────────────────────────────────────────┘    │
│                              |                                           │
│                              ▼                                           │
│  ┌──────────────────────────────────────────────────────────────────┐    │
│  │                      JOB 2: quality-gates                        │    │
│  │  needs: [tests]                                                  │    │
│  │                                                                  │    │
│  │  Steps:                                                          │    │
│  │  1. Check Black formatting                                       │    │
│  │  2. Check import sorting (isort)                                 │    │
│  │  3. Run flake8 strict                                            │    │
│  └──────────────────────────────────────────────────────────────────┘    │
│                              |                                           │
│                              ▼                                           │
│  ┌──────────────────────────────────────────────────────────────────┐    │
│  │                      JOB 3: security-scan                        │    │
│  │  needs: [tests]                                                  │    │
│  │                                                                  │    │
│  │  Steps:                                                          │    │
│  │  1. Gitleaks (secretos)                                          │    │
│  │  2. Bandit (código Python)                                       │    │
│  │  3. pip-audit (dependencias)                                     │    │
│  └──────────────────────────────────────────────────────────────────┘    │
│                              |                                           │
```

```
│                              ▼                                          │
│  ┌──────────────────────────────────────────────────────────────────┐  │
│  │                      JOB 4: docker-build                         │  │
│  │  needs: [tests, quality-gates]                                   │  │
│  │  if: push to main                                                │  │
│  │                                                                  │  │
│  │  Steps:                                                          │  │
│  │  1. Setup Docker Buildx                                          │  │
│  │  2. Login to GHCR                                                │  │
│  │  3. Build multi-stage images                                     │  │
│  │  4. Push to registry                                             │  │
│  └──────────────────────────────────────────────────────────────────┘  │
│                              │                                          │
│                              ▼                                          │
│  ┌──────────────────────────────────────────────────────────────────┐  │
│  │                        JOB 5: e2e-test                           │  │
│  │  needs: [docker-build]                                           │  │
│  │                                                                  │  │
│  │  Steps:                                                          │  │
│  │  1. Start Docker Compose stack                                   │  │
│  │  2. Wait for services                                            │  │
│  │  3. Run API health checks                                        │  │
│  │  4. Run integration tests                                        │  │
│  │  5. Cleanup                                                      │  │
│  └──────────────────────────────────────────────────────────────────┘  │
│                                                                         │
└─────────────────────────────────────────────────────────────────────────┘
```

El Archivo Completo

```yaml
# .github/workflows/ci-mlops.yml - Versión simplificada del portafolio

name: CI/CD MLOps Portfolio

on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]

permissions:
  actions: read
  contents: read
  security-events: write
  packages: write

env:
  PYTHON_VERSION: '3.12'

jobs:
  # ═══════════════════════════════════════════════════════════════════════
  # JOB 1: Tests con Coverage
  # ═══════════════════════════════════════════════════════════════════════
  tests:
    name: Tests & Coverage
    runs-on: ubuntu-latest
    strategy:
      fail-fast: false
      matrix:
        python-version: ['3.11', '3.12']
        project:
          - BankChurn-Predictor
          - CarVision-Market-Intelligence
          - TelecomAI-Customer-Intelligence

    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Set up Python ${{ matrix.python-version }}
        uses: actions/setup-python@v5
        with:
          python-version: ${{ matrix.python-version }}
          cache: 'pip'
```

```
      - name: Install dependencies
        working-directory: ${{ matrix.project }}
        run: |
          python -m pip install --upgrade pip
          pip install -r requirements.txt 2>/dev/null || pip install -e .
          pip install pytest pytest-cov flake8 black isort mypy

      - name: Run linting
        working-directory: ${{ matrix.project }}
        run: |
          flake8 src/ --count --select=E9,F63,F7,F82 --show-source --statistics || true
          black --check src/ || true

      - name: Run tests with coverage
        working-directory: ${{ matrix.project }}
        run: |
          # Determinar threshold por proyecto
          if [ "${{ matrix.project }}" = "BankChurn-Predictor" ]; then
            THRESHOLD=79
          else
            THRESHOLD=80
          fi

          pytest -m "not slow" \
            --cov=src/ \
            --cov-report=xml \
            --cov-report=term-missing \
            --cov-fail-under=$THRESHOLD

      - name: Upload coverage
        uses: codecov/codecov-action@v5
        with:
          files: ${{ matrix.project }}/coverage.xml
          flags: ${{ matrix.project }}

  # ═══════════════════════════════════════════════════════════════════════════
  # JOB 2: Quality Gates
  # ═══════════════════════════════════════════════════════════════════════════
  quality-gates:
    name: Quality Gates
    runs-on: ubuntu-latest
    needs: [tests]

    steps:
      - uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: ${{ env.PYTHON_VERSION }}

      - name: Install tools
        run: pip install black flake8 isort

      - name: Check formatting
        run: |
          for project in BankChurn-Predictor CarVision-Market-Intelligence TelecomAI-Customer-Intelligence; do
            echo "Checking $project..."
            black --check "$project/src" "$project/app" 2>/dev/null || true
            isort --check-only "$project/src" 2>/dev/null || true
          done

  # ═══════════════════════════════════════════════════════════════════════════
  # JOB 3: Security Scan
  # ═══════════════════════════════════════════════════════════════════════════
  security-scan:
    name: Security Scan
    runs-on: ubuntu-latest
    needs: [tests]

    steps:
      - uses: actions/checkout@v4
        with:
          fetch-depth: 0
```

```
      - name: Gitleaks
        uses: gitleaks/gitleaks-action@v2
        env:
          GITHUB_TOKEN: ${{ secrets.GITHUB_TOKEN }}

      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: ${{ env.PYTHON_VERSION }}

      - name: Run Bandit
        run: |
          pip install bandit
          bandit -r */src/ -f json -o bandit-report.json || true

      - name: Upload security report
        uses: actions/upload-artifact@v5
        with:
          name: security-reports
          path: bandit-report.json

  # ═══════════════════════════════════════════════════════════════════════
  # JOB 4: Docker Build (solo en main)
  # ═══════════════════════════════════════════════════════════════════════
  docker-build:
    name: Docker Build
    runs-on: ubuntu-latest
    needs: [tests, quality-gates]
    if: github.event_name == 'push' && github.ref == 'refs/heads/main'

    strategy:
      matrix:
        include:
          - project: BankChurn-Predictor
            image: ml-portfolio-bankchurn
          - project: CarVision-Market-Intelligence
            image: ml-portfolio-carvision
          - project: TelecomAI-Customer-Intelligence
            image: ml-portfolio-telecom

    steps:
      - uses: actions/checkout@v4

      - name: Set up Docker Buildx
        uses: docker/setup-buildx-action@v3

      - name: Login to GHCR
        uses: docker/login-action@v3
        with:
          registry: ghcr.io
          username: ${{ github.actor }}
          password: ${{ secrets.GITHUB_TOKEN }}

      - name: Build and push
        uses: docker/build-push-action@v5
        with:
          context: ./${{ matrix.project }}
          push: true
          tags: |
            ghcr.io/${{ github.repository_owner }}/${{ matrix.image }}:latest
            ghcr.io/${{ github.repository_owner }}/${{ matrix.image }}:${{ github.sha }}
          cache-from: type=gha
          cache-to: type=gha,mode=max
```

🧨 Errores habituales y cómo depurarlos en CI/CD

En este módulo los problemas suelen venir de **triggers mal configurados**, **rutas incorrectas** o **jobs mal encadenados**.

1) El workflow no se dispara

Síntomas típicos

- Haces push o abres un PR y GitHub no muestra ningún run nuevo.

Cómo identificarlo

- Revisa la sección `on:` del workflow:
- ¿Incluye las ramas correctas (`main`, `develop`, feature branches)?
- ¿Estás haciendo push a una rama no contemplada?

Cómo corregirlo

- Ajusta los triggers a tu flujo real:
  `yaml on: push: branches: [main, develop, "feature/*"] pull_request: branches: [main]`

2) Falla solo en un proyecto o en una versión de Python

Síntomas típicos

- En la matrix, solo falla `CarVision` en Python 3.12, el resto pasa.

Cómo identificarlo

- Mira los logs filtrando por `matrix.project` y `matrix.python-version`.

Cómo corregirlo

- Ejecuta localmente con la misma versión de Python y el mismo directorio (`working-directory`) que en el job.
- Asegúrate de que los paths (`src/`, `app/`, `requirements.txt`) sean correctos para cada proyecto en la matrix.

3) Coverage o linting no respetan el threshold esperado

Síntomas típicos

- Crees haber configurado `--cov-fail-under`, pero el job pasa aunque el coverage sea bajo.

Cómo identificarlo

- Verifica la línea exacta del comando `pytest` en el workflow.

Cómo corregirlo

- Asegúrate de que el parámetro `--cov-fail-under` se pase realmente al comando que se ejecuta (no a un alias intermedio).
- Diferencia claramente entre thresholds por proyecto usando condiciones `if` en el script del job.

4) Jobs que fallan por falta de dependencias o rutas

Síntomas típicos

- Errores como `ModuleNotFoundError` en CI pero no en local.
- `pip install -r requirements.txt` falla porque el archivo no existe en ese directorio.

Cómo identificarlo

- Verifica el `working-directory` de cada `step`.
- Revisa la estructura real del repo y compara con las rutas usadas en el workflow.

Cómo corregirlo

- Ajusta `working-directory` para que apunte al proyecto correcto (`BankChurn-Predictor`, etc.).
- Si un proyecto no tiene `requirements.txt`, instala en modo editable con `pip install -e .` como fallback.

5) Patrón general de debugging en GitHub Actions

1. Reproduce localmente el comando exacto que falla (`pytest`, `docker build`, etc.).
2. Verifica `on:` y `matrix` para asegurarte de que el job se ejecuta en los contextos esperados.
3. Usa `working-directory` y rutas relativas coherentes con la estructura del repo.
4. Encadena bien los jobs usando `needs` para que la lógica del pipeline sea clara.

Con este enfoque, CI/CD pasa de ser una caja negra “que a veces falla” a un pipeline confiable que te protege al hacer cambios en el portafolio.

✅ Ejercicio: Crear Tu Propio Workflow

Paso 1: Workflow Mínimo

Crea `.github/workflows/ci.yml`:

```
name: CI

on:
  push:
```

```
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: '3.11'
      - run: pip install pytest
      - run: pytest
```

Paso 2: Añadir Coverage

```
      - run: pip install pytest pytest-cov
      - run: pytest --cov=src/ --cov-fail-under=80
```

Paso 3: Añadir Matrix

```
    strategy:
      matrix:
        python-version: ['3.11', '3.12']
```

Paso 4: Añadir Security

Añade un job nuevo con Bandit y Gitleaks.

✅ Checkpoint

- [ ] Tienes un workflow básico que ejecuta tests
- [ ] El workflow usa matrix testing (múltiples versiones Python)
- [ ] Coverage está enforced con threshold
- [ ] Tienes al menos un scan de seguridad
- [ ] Los artifacts se suben correctamente

📦 Cómo se Usó en el Portafolio

El portafolio tiene un workflow CI/CD real en `.github/workflows/ci-mlops.yml`:

Workflow Real del Portafolio

```
# .github/workflows/ci-mlops.yml (extracto)
name: CI/CD MLOps Portfolio

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    strategy:
      matrix:
        project: [BankChurn-Predictor, CarVision-Market-Intelligence, TelecomAI-Customer-Intelligence]
        python-version: ['3.10', '3.11']

    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: ${{ matrix.python-version }}

      - name: Install dependencies
        run: |
          cd ${{ matrix.project }}
          pip install -e ".[dev]"

      - name: Run tests with coverage
```

```
        run: |
          cd ${{ matrix.project }}
          pytest tests/ --cov=src/ --cov-fail-under=79
```

Features del CI/CD

Feature	Implementación
Matrix Testing	3 proyectos × 2 versiones Python
Coverage Gate	`--cov-fail-under=79`
Security Scan	gitleaks en pre-commit
Artifacts	Coverage reports

🔧 Ejercicio: Revisa el CI Real

```
# 1. Ve el workflow real
cat .github/workflows/ci-mlops.yml

# 2. Simula localmente con act (opcional)
act -j test --matrix project:BankChurn-Predictor

# 3. Ve los runs en GitHub
# https://github.com/DuqueOM/ML-MLOps-Portfolio/actions
```

💼 Consejos Profesionales

Recomendaciones para destacar en entrevistas y proyectos reales

Para Entrevistas

1. **CI vs CD**: CI = integrar código frecuentemente, CD = desplegar automáticamente.
2. **GitHub Actions vs Jenkins vs GitLab CI**: Trade-offs de cada uno.
3. **ML-specific CI**: Explica cómo CI para ML incluye validación de datos y modelos.

Para Proyectos Reales

Situación	Consejo
Secrets	Usa GitHub Secrets, nunca hardcodees
Caching	Cachea dependencias y datos para velocidad
Paralelización	Matriz de tests para múltiples versiones
Rollback	Siempre ten estrategia de rollback

Pipeline CI/CD para ML

```
1. Lint + Format (ruff, black)
2. Unit Tests (pytest)
3. Integration Tests
4. Security Scan (gitleaks, bandit)
5. Build Docker Image
6. Model Validation
7. Deploy to Staging
8. Smoke Tests
9. Deploy to Production
```

📺 Recursos Externos Recomendados

Ver RECURSOS_POR_MODULO.md para la lista completa.

🏷️	Recurso	Tipo
🔴	GitHub Actions Tutorial - TechWorld Nana	Video
🟡	CI/CD for ML - Made With ML	Tutorial

Documentación oficial:
- GitHub Actions
- pre-commit

🔗 Referencias del Glosario

Ver 21_GLOSARIO.md para definiciones de:
- **CI/CD**: Integración y despliegue continuo
- **GitHub Actions**: Automatización de workflows
- **pre-commit**: Hooks de validación antes de commit

📋 Plantillas Relacionadas

Ver templates/ para plantillas listas:
- ci_github_actions.yml — Pipeline CI/CD completo
- ci_template.yml — Versión mínima para quick start

✅ Ejercicios

Ver EJERCICIOS.md - Módulo 12:
- 12.1: GitHub Actions workflow básico

DOCKER

GUÍA MLOPS V5.0

PORTFOLIO EDITION

14. Docker Avanzado para ML

🎯 Objetivo del Módulo

Construir imágenes Docker optimizadas, seguras y pequeñas como las del portafolio.

```
╔═══════════════════════════════════════════════════════════════╗
╗                                                               ╗
╗  NIVEL 1: Funcional       NIVEL 2: Optimizado      NIVEL 3: Production  ╗
╗  ────────────────         ───────────────────      ───────────────────  ╗
╗  FROM python:3.11         Multi-stage build        Distroless/Alpine    ╗
╗  COPY . .                 Slim base                Non-root user        ╗
╗  pip install              Layer caching            CVE scanning         ╗
╗                                                                         ╗
╗  ~1.2GB                   ~400MB                   ~150MB               ╗
╗  ⚠️ Básica                 ✅ Mejor                   🛡️ Hardened          ╗
╗                                                                         ╗
╚═══════════════════════════════════════════════════════════════╝
```

📋 Contenido

1. Dockerfile Básico vs Optimizado
2. Multi-Stage Builds
3. Mejores Prácticas
4. Dockerfile Real del Portafolio
5. Docker Compose para ML

14.1 Dockerfile Básico vs Optimizado

❌ Nivel 1: Básico (No usar en producción)

```
# Dockerfile MALO - Solo para demos rápidas
FROM python:3.11

WORKDIR /app
COPY . .

RUN pip install -r requirements.txt

CMD ["python", "main.py"]

# Problemas:
# - Imagen de ~1.2GB
# - Incluye herramientas de desarrollo innecesarias
# - Cache de pip no aprovechado
# - Corre como root (inseguro)
# - Copia archivos innecesarios (.git, tests, etc.)
```

✅ Nivel 2: Optimizado

```
# Dockerfile MEJOR - Para staging/desarrollo

# 1. Usar slim para reducir tamaño
FROM python:3.11-slim

# 2. Establecer directorio de trabajo
WORKDIR /app

# 3. Copiar SOLO requirements primero (aprovecha cache)
COPY requirements.txt .

# 4. Instalar dependencias (capa cacheada si requirements no cambia)
RUN pip install --no-cache-dir -r requirements.txt

# 5. Copiar código fuente
COPY src/ ./src/
COPY app/ ./app/
COPY configs/ ./configs/

# 6. Usuario no-root
RUN useradd -m appuser && chown -R appuser:appuser /app
USER appuser
```

```
# 7. Puerto y comando
EXPOSE 8000
CMD ["uvicorn", "app.fastapi_app:app", "--host", "0.0.0.0", "--port", "8000"]

# Mejoras:
# - ~400MB (slim base)
# - Cache de layers optimizado
# - No corre como root
# - Solo archivos necesarios
```

14.2 Multi-Stage Builds

El Concepto

```
┌──────────────────────────────────────────────────────────────────────────┐
│                        MULTI-STAGE BUILD                                 │
├──────────────────────────────────────────────────────────────────────────┤
│                                                                          │
│  STAGE 1: Builder                    STAGE 2: Runtime                    │
│  ─────────────────                   ────────────────                    │
│  • Imagen completa                   • Imagen mínima                     │
│  • Compila código                    • Solo runtime                      │
│  • Instala dependencias              • Copia solo binarios               │
│  • Genera wheels                     • Sin compiladores                  │
│                                                                          │
│  ┌─────────────────┐                 ┌─────────────────┐                 │
│  │ python:3.11     │                 │ python:3.11-slim │                 │
│  │ + gcc, make     │                 │                 │                 │
│  │ + pip wheel     │   ──COPY──►     │ + wheels only   │                 │
│  │ = 1.2GB         │                 │ = 150-400MB     │                 │
│  └─────────────────┘                 └─────────────────┘                 │
│                                                                          │
│  Se DESCARTA                         Se USA en producción                │
│                                                                          │
└──────────────────────────────────────────────────────────────────────────┘
```

Implementación

```
# Dockerfile Multi-Stage - Nivel 3

# ═══════════════════════════════════════════════════════════════════════════
# STAGE 1: Builder - Compila dependencias
# ═══════════════════════════════════════════════════════════════════════════
FROM python:3.11-slim AS builder

WORKDIR /build

# Instalar herramientas de compilación (temporales)
RUN apt-get update && apt-get install -y --no-install-recommends \
    gcc \
    python3-dev \
    && rm -rf /var/lib/apt/lists/*

# Copiar requirements
COPY requirements.txt .

# Crear wheels (binarios precompilados)
RUN pip wheel --no-cache-dir --wheel-dir /wheels -r requirements.txt

# ═══════════════════════════════════════════════════════════════════════════
# STAGE 2: Runtime - Imagen final mínima
# ═══════════════════════════════════════════════════════════════════════════
FROM python:3.11-slim AS runtime

WORKDIR /app

# Copiar SOLO los wheels del builder
COPY --from=builder /wheels /wheels

# Instalar desde wheels (sin compilación)
RUN pip install --no-cache-dir /wheels/* && rm -rf /wheels

# Copiar código
```

```
COPY src/ ./src/
COPY app/ ./app/
COPY configs/ ./configs/

# Copiar modelo pre-entrenado si existe
COPY artifacts/model.joblib ./artifacts/model.joblib 2>/dev/null || true

# Crear usuario no-root
RUN useradd -m -u 1000 appuser && chown -R appuser:appuser /app
USER appuser

# Healthcheck
HEALTHCHECK --interval=30s --timeout=10s --start-period=5s --retries=3 \
    CMD curl -f http://localhost:8000/health || exit 1

# Exponer puerto
EXPOSE 8000

# Comando de inicio
CMD ["uvicorn", "app.fastapi_app:app", "--host", "0.0.0.0", "--port", "8000"]
```

14.3 Mejores Prácticas

.dockerignore

```
# .dockerignore - Excluir archivos innecesarios

# Git
.git
.gitignore

# Python
__pycache__
*.py[cod]
*.pyo
.pytest_cache
.mypy_cache
.coverage
htmlcov/
.venv/
venv/
*.egg-info/

# IDE
.vscode/
.idea/
*.swp

# Tests (no necesarios en producción)
tests/
*_test.py
test_*.py
conftest.py

# Documentación
docs/
*.md
!README.md

# Datos (montar como volumen, no copiar)
data/
*.csv
*.parquet

# Notebooks
*.ipynb
notebooks/

# Logs y temporales
*.log
logs/
tmp/
```

Layer Caching

```
# ❌ MALO: Cualquier cambio en código invalida cache de pip
COPY . .
RUN pip install -r requirements.txt

# ✅ BUENO: requirements separado para aprovechar cache
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY src/ ./src/  # Cambios aquí NO invalidan pip install
```

Security: Non-Root User

```
# Crear usuario con UID específico (evita conflictos de permisos)
RUN useradd -m -u 1000 appuser

# Dar permisos al directorio de trabajo
RUN chown -R appuser:appuser /app

# Cambiar a usuario no-root ANTES de CMD
USER appuser

# Ahora el proceso corre como appuser, no como root
```

14.4 Dockerfile Real del Portafolio

BankChurn-Predictor/Dockerfile

```
# BankChurn-Predictor Production Dockerfile
# Multi-stage build optimizado para ML

# ═══════════════════════════════════════════════════════════════════════
# Stage 1: Builder
# ═══════════════════════════════════════════════════════════════════════
FROM python:3.11-slim AS builder

WORKDIR /build

# Dependencias de sistema para compilación
RUN apt-get update && apt-get install -y --no-install-recommends \
    gcc \
    python3-dev \
    && rm -rf /var/lib/apt/lists/*

# Copiar requirements
COPY requirements.txt .

# Crear wheels
RUN pip wheel --no-cache-dir --wheel-dir /wheels -r requirements.txt

# ═══════════════════════════════════════════════════════════════════════
# Stage 2: Runtime
# ═══════════════════════════════════════════════════════════════════════
FROM python:3.11-slim

# Labels para metadata
LABEL maintainer="duqueom@example.com"
LABEL version="1.0.0"
LABEL description="BankChurn Predictor API"

WORKDIR /app

# Instalar curl para healthcheck
RUN apt-get update && apt-get install -y --no-install-recommends curl \
    && rm -rf /var/lib/apt/lists/*

# Instalar dependencias desde wheels
COPY --from=builder /wheels /wheels
RUN pip install --no-cache-dir /wheels/* && rm -rf /wheels

# Copiar código fuente
COPY src/ ./src/
COPY app/ ./app/
```

```
COPY configs/ ./configs/

# Copiar modelo (si existe)
COPY models/ ./models/ 2>/dev/null || mkdir -p ./models

# Crear usuario no-root
RUN useradd -m -u 1000 appuser && chown -R appuser:appuser /app
USER appuser

# Variables de entorno
ENV PYTHONUNBUFFERED=1
ENV PYTHONDONTWRITEBYTECODE=1
ENV PORT=8000

# Healthcheck
HEALTHCHECK --interval=30s --timeout=10s --start-period=45s --retries=3 \
    CMD curl -f http://localhost:${PORT}/health || exit 1

EXPOSE ${PORT}

CMD ["uvicorn", "app.fastapi_app:app", "--host", "0.0.0.0", "--port", "8000"]
```

14.5 Docker Compose para ML

docker-compose.demo.yml (Portafolio)

```
# Docker Compose para demo completa del portafolio
version: "3.8"

services:
  # MLflow Server (central)
  mlflow:
    image: ghcr.io/mlflow/mlflow:v2.9.2
    container_name: mlflow-server
    ports:
      - "5000:5000"
    volumes:
      - mlflow-data:/mlflow
    command: >
      mlflow server
      --backend-store-uri sqlite:///mlflow/mlflow.db
      --default-artifact-root /mlflow/artifacts
      --host 0.0.0.0
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:5000/health"]
      interval: 30s
      timeout: 10s
      retries: 3
    networks:
      - ml-network

  # BankChurn API
  bankchurn:
    build:
      context: ./BankChurn-Predictor
      dockerfile: Dockerfile
    container_name: bankchurn-api
    ports:
      - "8001:8000"
    volumes:
      - ./BankChurn-Predictor/models:/app/models:ro
    environment:
      - MLFLOW_TRACKING_URI=http://mlflow:5000
    depends_on:
      mlflow:
        condition: service_healthy
    networks:
      - ml-network

  # CarVision API
  carvision:
    build:
      context: ./CarVision-Market-Intelligence
      dockerfile: Dockerfile
```

```
    container_name: carvision-api
    ports:
      - "8002:8000"
    volumes:
      - ./CarVision-Market-Intelligence/artifacts:/app/artifacts:ro
    environment:
      - MLFLOW_TRACKING_URI=http://mlflow:5000
    depends_on:
      mlflow:
        condition: service_healthy
    networks:
      - ml-network

  # TelecomAI API
  telecom:
    build:
      context: ./TelecomAI-Customer-Intelligence
      dockerfile: Dockerfile
    container_name: telecom-api
    ports:
      - "8003:8000"
    environment:
      - MLFLOW_TRACKING_URI=http://mlflow:5000
    depends_on:
      mlflow:
        condition: service_healthy
    networks:
      - ml-network

volumes:
  mlflow-data:

networks:
  ml-network:
    driver: bridge
```

Comandos Útiles

```
# Construir todas las imágenes
docker compose -f docker-compose.demo.yml build

# Iniciar todos los servicios
docker compose -f docker-compose.demo.yml up -d

# Ver logs
docker compose -f docker-compose.demo.yml logs -f bankchurn

# Parar todo
docker compose -f docker-compose.demo.yml down

# Limpiar volúmenes también
docker compose -f docker-compose.demo.yml down -v
```

🧨 Errores habituales y cómo depurarlos en Docker para ML

En ML es muy común tener imágenes gigantes, problemas de permisos o contenedores que “funcionan en mi máquina pero no en producción”.

1) Imágenes demasiado grandes

Síntomas típicos

- `docker images` muestra tamaños > 1 GB.
- Push/pull al registry tarda mucho o falla por timeout.

Cómo identificarlo

- Compara tu Dockerfile con los ejemplos `python:3.11` vs `python:3.11-slim` del módulo.
- Revisa si estás copiando todo el repo (`COPY . .`) sin `.dockerignore`.

Cómo corregirlo

- Usa bases `slim` y **multi-stage builds**.
- Añade un `.dockerignore` que excluya datos, notebooks, tests y `.venv`.

2) Errores de permisos al correr como non-root

Síntomas típicos

- El contenedor arranca pero falla al leer modelos, logs o escribir en directorios.
- Mensajes tipo `Permission denied: '/app/models/model.joblib'`.

Cómo identificarlo

- Verifica que después de copiar archivos hagas `chown` al usuario de la app.
- Revisa que `USER appuser` aparezca **después** de ajustar permisos.

Cómo corregirlo

- Asegúrate de:
  `dockerfile RUN useradd -m -u 1000 appuser && chown -R appuser:appuser /app USER appuser`
- Monta volúmenes con permisos compatibles (por ejemplo, propiedad UID 1000 en host).

3) Modelo o artefactos no encontrados dentro del contenedor

Síntomas típicos

- La API levanta pero responde 500 porque no encuentra el modelo (`FileNotFoundError`).

Cómo identificarlo

- Revisa las rutas de `COPY` en el Dockerfile y las rutas que tu código usa (`./models`, `./artifacts`).

Cómo corregirlo

- Copia los artefactos a la ruta esperada o monta un volumen de solo lectura:
  ```yaml
  volumes:
    - ./BankChurn-Predictor/models:/app/models:ro
  ```

4) Contenedores que arrancan pero el healthcheck falla

Síntomas típicos

- El servicio aparece como "unhealthy" en `docker ps`.

Cómo identificarlo

- Examina el `HEALTHCHECK` y verifica que la URL y puerto sean correctos.

Cómo corregirlo

- Asegúrate de que el endpoint `/health` exista y escuche en el mismo puerto que expones.
- Ajusta tiempos de `start-period` si el modelo tarda más en cargar.

5) Patrón general de debugging con Docker

1. Inspecciona el contenedor en ejecución con `docker exec -it <container> /bin/bash`.
2. Navega por `/app` para verificar que el código, modelos y configs estén donde esperas.
3. Comprueba permisos (`ls -l`) y usuario actual (`whoami`).
4. Si la imagen es muy grande, revisa el historial de capas con `docker history`.

Con este enfoque, tus imágenes Docker serán reproducibles, ligeras y listas para producción.

💼 Consejos Profesionales

Recomendaciones para destacar en entrevistas y proyectos reales

Para Entrevistas

1. **Multi-stage builds**: Explica cómo reducen tamaño de imagen.
2. **Layer caching**: Por qué el orden de instrucciones importa.
3. **Security**: No correr como root, no incluir secrets en imagen.

Para Proyectos Reales

Situación	Consejo
Imágenes grandes	Multi-stage + slim base images

Situación	Consejo
Secrets	Usa build args o secrets mounting
Debugging	Usa `docker exec -it container bash`
Producción	Healthchecks obligatorios

Dockerfile Optimizado

```
# Stage 1: Build
FROM python:3.11-slim AS builder
COPY requirements.txt .
RUN pip wheel --no-cache-dir -r requirements.txt

# Stage 2: Runtime
FROM python:3.11-slim
COPY --from=builder /wheels /wheels
RUN pip install --no-cache /wheels/*
COPY src/ /app/src/
USER nobody
HEALTHCHECK CMD curl -f http://localhost:8000/health
```

📺 Recursos Externos Recomendados

Ver RECURSOS_POR_MODULO.md para la lista completa.

🏷️	Recurso	Tipo
🔴	Docker Tutorial - TechWorld Nana	Video
🟡	Multi-stage Builds	Video

Documentación oficial:
- Docker Multi-stage Builds
- Dockerfile Best Practices

🔗 Referencias del Glosario

Ver 21_GLOSARIO.md para definiciones de:
- **Multi-stage Build**: Separar build de runtime
- **Docker Compose**: Orquestar múltiples contenedores
- **Non-root user**: Seguridad en contenedores

📋 Plantillas Relacionadas

Ver templates/ para plantillas listas:
- Dockerfile — Multi-stage completo para ML APIs
- Dockerfile_template — Versión simplificada
- docker-compose.yml — Stack con servicios

✅ Ejercicios

Ver EJERCICIOS.md - Módulo 13:
- 13.1: Dockerfile multi-stage
- 13.2: Docker Compose para stack ML

FASTAPI

GUÍA MLOPS V5.0

PORTFOLIO EDITION

15. FastAPI para Producción

🎯 Objetivo del Módulo

Construir APIs de ML robustas, documentadas y production-ready como las del portafolio.

```
╔═══════════════════════════════════════════════════════════════════╗
║                                                                   ║
║  FastAPI = El framework ideal para ML APIs                        ║
║                                                                   ║
║  ✅ Type hints nativos (Pydantic)                                 ║
║  ✅ Documentación automática (Swagger/OpenAPI)                    ║
║  ✅ Async support (alto throughput)                                ║
║  ✅ Validación automática de requests                              ║
║  ✅ Dependency Injection built-in                                  ║
║                                                                   ║
╚═══════════════════════════════════════════════════════════════════╝
```

📋 Contenido

1. Estructura de una API ML
2. Schemas con Pydantic
3. Endpoints de Predicción
4. Error Handling
5. Código Real del Portafolio

15.1 Estructura de una API ML

Anatomía Típica

```
# app/fastapi_app.py - Estructura profesional

from contextlib import asynccontextmanager
from pathlib import Path

import joblib
from fastapi import FastAPI, HTTPException
from fastapi.middleware.cors import CORSMiddleware

from .schemas import PredictionRequest, PredictionResponse, HealthResponse


# ═══════════════════════════════════════════════════════════════════
# LIFECYCLE: Cargar modelo al iniciar
# ═══════════════════════════════════════════════════════════════════

model = None  # Global para acceso en endpoints

@asynccontextmanager
async def lifespan(app: FastAPI):
    """Lifecycle: carga modelo al iniciar, limpia al cerrar."""
    global model

    # Startup: cargar modelo
    model_path = Path("artifacts/model.joblib")
    if model_path.exists():
        model = joblib.load(model_path)
        print(f"✅ Modelo cargado: {model_path}")
    else:
        print(f"⚠️ Modelo no encontrado: {model_path}")

    yield  # App corriendo

    # Shutdown: limpiar recursos
    model = None
    print("🛑 App cerrada")


# ═══════════════════════════════════════════════════════════════════
# APP SETUP
# ═══════════════════════════════════════════════════════════════════
```

```
app = FastAPI(
    title="BankChurn Predictor API",
    description="API para predicción de churn de clientes bancarios",
    version="1.0.0",
    lifespan=lifespan,
)

# CORS para permitir requests desde frontend
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],  # En prod: especificar dominios
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)


# ═══════════════════════════════════════════════════════════════════════
# ENDPOINTS
# ═══════════════════════════════════════════════════════════════════════

@app.get("/health", response_model=HealthResponse)
async def health_check():
    """Health check endpoint para load balancers/k8s."""
    return HealthResponse(
        status="healthy" if model is not None else "degraded",
        model_loaded=model is not None,
        version="1.0.0"
    )


@app.post("/predict", response_model=PredictionResponse)
async def predict(request: PredictionRequest):
    """Predice probabilidad de churn para un cliente."""
    if model is None:
        raise HTTPException(status_code=503, detail="Modelo no disponible")

    # Convertir request a DataFrame
    import pandas as pd
    df = pd.DataFrame([request.dict()])

    # Predecir
    proba = model.predict_proba(df)[0, 1]
    prediction = int(proba >= 0.5)

    return PredictionResponse(
        prediction=prediction,
        probability=round(proba, 4),
        risk_level="high" if proba >= 0.7 else "medium" if proba >= 0.3 else "low"
    )
```

15.2 Schemas con Pydantic

Request/Response Models

```
# app/schemas.py

from typing import Literal, Optional
from pydantic import BaseModel, Field, validator


class PredictionRequest(BaseModel):
    """Schema para request de predicción.

    Pydantic valida automáticamente:
    - Tipos correctos
    - Rangos válidos
    - Valores permitidos
    """

    CreditScore: int = Field(..., ge=300, le=850, description="Credit score del cliente")
    Geography: Literal["France", "Germany", "Spain"] = Field(..., description="País")
    Gender: Literal["Male", "Female"] = Field(..., description="Género")
    Age: int = Field(..., ge=18, le=100, description="Edad")
```

```
    Tenure: int = Field(..., ge=0, le=10, description="Años como cliente")
    Balance: float = Field(..., ge=0, description="Balance en cuenta")
    NumOfProducts: int = Field(..., ge=1, le=4, description="Número de productos")
    HasCrCard: Literal[0, 1] = Field(..., description="Tiene tarjeta de crédito")
    IsActiveMember: Literal[0, 1] = Field(..., description="Es miembro activo")
    EstimatedSalary: float = Field(..., ge=0, description="Salario estimado")

    class Config:
        json_schema_extra = {
            "example": {
                "CreditScore": 650,
                "Geography": "France",
                "Gender": "Female",
                "Age": 40,
                "Tenure": 3,
                "Balance": 60000.0,
                "NumOfProducts": 2,
                "HasCrCard": 1,
                "IsActiveMember": 1,
                "EstimatedSalary": 50000.0
            }
        }


class PredictionResponse(BaseModel):
    """Schema para response de predicción."""

    prediction: Literal[0, 1] = Field(..., description="0=No churn, 1=Churn")
    probability: float = Field(..., ge=0, le=1, description="Probabilidad de churn")
    risk_level: Literal["low", "medium", "high"] = Field(..., description="Nivel de riesgo")


class HealthResponse(BaseModel):
    """Schema para health check."""

    status: Literal["healthy", "degraded", "unhealthy"]
    model_loaded: bool
    version: str


class BatchPredictionRequest(BaseModel):
    """Schema para predicción en batch."""

    customers: list[PredictionRequest] = Field(
        ...,
        min_items=1,
        max_items=1000,
        description="Lista de clientes (máx 1000)"
    )


class BatchPredictionResponse(BaseModel):
    """Schema para response de batch."""

    predictions: list[PredictionResponse]
    processed: int
    errors: int = 0
```

15.3 Endpoints de Predicción

Single Prediction

```
@app.post("/predict", response_model=PredictionResponse)
async def predict(request: PredictionRequest):
    """
    Predice probabilidad de churn para UN cliente.

    - **CreditScore**: Score crediticio (300-850)
    - **Geography**: País (France, Germany, Spain)
    - **Gender**: Género
    - **Age**: Edad (18-100)
    - ... etc

    Returns:
```

```
    - **prediction**: 0 (no churn) o 1 (churn)
    - **probability**: Probabilidad [0, 1]
    - **risk_level**: low/medium/high
    """
    if model is None:
        raise HTTPException(
            status_code=503,
            detail="Modelo no disponible. Reinicie el servicio."
        )

    try:
        import pandas as pd
        df = pd.DataFrame([request.model_dump()])

        proba = model.predict_proba(df)[0, 1]
        prediction = int(proba >= 0.5)

        if proba >= 0.7:
            risk = "high"
        elif proba >= 0.3:
            risk = "medium"
        else:
            risk = "low"

        return PredictionResponse(
            prediction=prediction,
            probability=round(float(proba), 4),
            risk_level=risk
        )

    except Exception as e:
        raise HTTPException(status_code=500, detail=f"Error en predicción: {str(e)}")
```

Batch Prediction

```
@app.post("/predict/batch", response_model=BatchPredictionResponse)
async def predict_batch(request: BatchPredictionRequest):
    """
    Predice churn para múltiples clientes (máx 1000).

    Útil para scoring masivo de cartera.
    """
    if model is None:
        raise HTTPException(status_code=503, detail="Modelo no disponible")

    import pandas as pd

    results = []
    errors = 0

    # Convertir todos los requests a DataFrame (más eficiente)
    data = [c.model_dump() for c in request.customers]
    df = pd.DataFrame(data)

    try:
        probas = model.predict_proba(df)[:, 1]

        for proba in probas:
            prediction = int(proba >= 0.5)
            risk = "high" if proba >= 0.7 else "medium" if proba >= 0.3 else "low"

            results.append(PredictionResponse(
                prediction=prediction,
                probability=round(float(proba), 4),
                risk_level=risk
            ))

    except Exception as e:
        raise HTTPException(status_code=500, detail=f"Error en batch: {str(e)}")

    return BatchPredictionResponse(
        predictions=results,
        processed=len(results),
```

```
        errors=errors
    )
```

15.4 Error Handling

Custom Exception Handlers

```
from fastapi import Request
from fastapi.responses import JSONResponse

class ModelNotLoadedError(Exception):
    """Modelo no cargado."""
    pass

class InvalidInputError(Exception):
    """Input inválido."""
    pass


@app.exception_handler(ModelNotLoadedError)
async def model_not_loaded_handler(request: Request, exc: ModelNotLoadedError):
    return JSONResponse(
        status_code=503,
        content={
            "error": "service_unavailable",
            "message": "El modelo no está cargado. Intente más tarde.",
            "retry_after": 30
        }
    )


@app.exception_handler(InvalidInputError)
async def invalid_input_handler(request: Request, exc: InvalidInputError):
    return JSONResponse(
        status_code=400,
        content={
            "error": "invalid_input",
            "message": str(exc),
            "hint": "Verifique que todos los campos tengan valores válidos"
        }
    )


# Catch-all para errores no manejados
@app.exception_handler(Exception)
async def generic_exception_handler(request: Request, exc: Exception):
    return JSONResponse(
        status_code=500,
        content={
            "error": "internal_error",
            "message": "Error interno del servidor",
            "detail": str(exc) if app.debug else None
        }
    )
```

15.5 Código Real del Portafolio

app/fastapi_app.py (BankChurn - Simplificado)

```
"""FastAPI application for BankChurn prediction service."""

from __future__ import annotations

import logging
import os
from contextlib import asynccontextmanager
from pathlib import Path
from typing import Literal

import joblib
import pandas as pd
from fastapi import FastAPI, HTTPException
from fastapi.middleware.cors import CORSMiddleware
from pydantic import BaseModel, Field
```

```
logger = logging.getLogger(__name__)

# ═══════════════════════════════════════════════════════════════════════
# SCHEMAS
# ═══════════════════════════════════════════════════════════════════════

class CustomerInput(BaseModel):
    CreditScore: int = Field(..., ge=300, le=850)
    Geography: str
    Gender: str
    Age: int = Field(..., ge=18, le=100)
    Tenure: int = Field(..., ge=0, le=10)
    Balance: float = Field(..., ge=0)
    NumOfProducts: int = Field(..., ge=1, le=4)
    HasCrCard: int = Field(..., ge=0, le=1)
    IsActiveMember: int = Field(..., ge=0, le=1)
    EstimatedSalary: float = Field(..., ge=0)


class PredictionOutput(BaseModel):
    prediction: int
    probability: float
    risk_level: str


class HealthOutput(BaseModel):
    status: str
    model_loaded: bool


# ═══════════════════════════════════════════════════════════════════════
# APP
# ═══════════════════════════════════════════════════════════════════════

model = None

@asynccontextmanager
async def lifespan(app: FastAPI):
    global model

    # Buscar modelo en varias ubicaciones
    paths = [
        Path("models/model_v1.0.0.pkl"),
        Path("artifacts/model.joblib"),
        Path(os.getenv("MODEL_PATH", "model.joblib")),
    ]

    for path in paths:
        if path.exists():
            model = joblib.load(path)
            logger.info(f"Modelo cargado: {path}")
            break

    if model is None:
        logger.warning("⚠️ Ningún modelo encontrado")

    yield
    model = None


app = FastAPI(
    title="BankChurn Predictor",
    version="1.0.0",
    lifespan=lifespan
)

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_methods=["*"],
    allow_headers=["*"],
)
```

```
@app.get("/health", response_model=HealthOutput)
async def health():
    return HealthOutput(
        status="healthy" if model else "degraded",
        model_loaded=model is not None
    )


@app.post("/predict", response_model=PredictionOutput)
async def predict(customer: CustomerInput):
    if model is None:
        raise HTTPException(503, "Modelo no disponible")

    df = pd.DataFrame([customer.model_dump()])
    proba = model.predict_proba(df)[0, 1]

    return PredictionOutput(
        prediction=int(proba >= 0.5),
        probability=round(proba, 4),
        risk_level="high" if proba >= 0.7 else "medium" if proba >= 0.3 else "low"
    )


if __name__ == "__main__":
    import uvicorn
    uvicorn.run(app, host="0.0.0.0", port=8000)
```

🧨 Errores habituales y cómo depurarlos en FastAPI para ML

FastAPI te da mucho “gratis”, pero en APIs de ML los fallos suelen venir de modelos no cargados, esquemas desalineados o problemas de tipos/serialización.

1) El modelo no se carga (503 constantes)

Síntomas típicos

- El endpoint `/predict` responde `503 Modelo no disponible`.
- Logs con mensajes tipo `Modelo no encontrado` o `Ningún modelo encontrado`.

Cómo identificarlo

- Revisa la función `lifespan` o código de startup: ¿la ruta del modelo (`models/`, `artifacts/`) existe dentro del contenedor?
- Comprueba variables de entorno como `MODEL_PATH`.

Cómo corregirlo

- Asegura rutas consistentes entre entrenamiento, Dockerfile y FastAPI.
- En local, imprime (`logger.info`) la ruta exacta desde la que intentas cargar y verifica que el archivo esté ahí.

2) Esquema Pydantic desalineado con el pipeline

Síntomas típicos

- Errores `KeyError` o `Column not found` al predecir.
- El modelo espera columnas con ciertos nombres pero el `PredictionRequest` usa otros.

Cómo identificarlo

- Compara los campos del schema (`CreditScore`, `Geography`, etc.) con las columnas que el pipeline de sklearn espera.

Cómo corregirlo

- Usa los **mismos nombres de features** que en el training pipeline.
- Si renombraste columnas en feature engineering, refleja esos cambios en el schema y en la transformación de entrada antes de llamar al modelo.

3) Problemas de tipos y serialización

Síntomas típicos

- Errores `TypeError: Object of type ... is not JSON serializable`.
- Respuestas con valores `NaN` o `Infinity` que rompen el cliente.

Cómo identificarlo

- Revisa el tipo real de lo que devuelves en `PredictionResponse` (por ejemplo, `numpy.float32` en vez de `float`).

Cómo corregirlo

- Convierte explícitamente a tipos nativos de Python (`float`, `int`, `str`).
- Asegúrate de que no devuelves `NaN` o `inf` (redondea o reemplaza por valores válidos).

4) CORS o healthcheck mal configurados

Síntomas típicos

- El frontend no puede llamar al API por errores de CORS.
- Kubernetes/Compose marcan el servicio como unhealthy.

Cómo identificarlo

- Revisa configuración de `CORSMiddleware` y el endpoint `/health`.

Cómo corregirlo

- En desarrollo puedes usar `allow_origins=["*"]`, pero en producción limita a tus dominios.
- Verifica que `/health` no dependa de modelos pesados para responder rápido y con `200`.

5) Patrón general de debugging en APIs de ML

1. Llama al endpoint con `curl` o `httpie` usando el `example` del schema.
2. Mira los logs del servidor (uvicorn) para ver tracebacks completos.
3. Verifica rutas de modelo y variables de entorno que afectan al loading.
4. Asegúrate de que lo que entra/sale del API coincide con lo que tu modelo entrenado espera.

Con esta disciplina, tu API FastAPI pasará de "funciona solo en local" a estar lista para producción.

✅ Ejercicio

1. Implementa `/predict/batch` para procesar múltiples clientes
2. Añade endpoint `/model/info` que retorne metadata del modelo
3. Implementa rate limiting básico

📦 Cómo se Usó en el Portafolio

Cada proyecto tiene una API FastAPI en `app/fastapi_app.py`:

API de BankChurn

```
# BankChurn-Predictor/app/fastapi_app.py (estructura)
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel

app = FastAPI(title="BankChurn Predictor API")

class PredictionRequest(BaseModel):
    CreditScore: int
    Geography: str
    Gender: str
    Age: int
    Balance: float
    # ... más features

class PredictionResponse(BaseModel):
    prediction: int
    probability: float
    risk_level: str

@app.get("/health")
async def health():
    return {"status": "healthy", "model_loaded": model is not None}

@app.post("/predict", response_model=PredictionResponse)
async def predict(request: PredictionRequest):
    features = request.dict()
    df = pd.DataFrame([features])
    prediction = pipeline.predict(df)[0]
    probability = pipeline.predict_proba(df)[0, 1]
    return PredictionResponse(
        prediction=int(prediction),
        probability=float(probability),
```

```
        risk_level="high" if probability > 0.7 else "low"
    )
```

APIs por Proyecto

Proyecto	Endpoint Principal	Tipo
BankChurn	`/predict`	Clasificación binaria
CarVision	`/predict`	Regresión
TelecomAI	`/predict`	Clasificación multiclase

🔧 Ejercicio: Prueba las APIs Reales

```
# 1. Inicia API de BankChurn
cd BankChurn-Predictor
uvicorn app.fastapi_app:app --reload

# 2. Prueba con curl
curl http://localhost:8000/health

curl -X POST http://localhost:8000/predict \
  -H "Content-Type: application/json" \
  -d '{"CreditScore": 650, "Geography": "France", ...}'

# 3. Ve docs interactivos
# http://localhost:8000/docs
```

💼 Consejos Profesionales

Recomendaciones para destacar en entrevistas y proyectos reales

Para Entrevistas

1. **Pydantic + FastAPI**: Explica cómo la validación automática reduce código.
2. **Async vs Sync**: Cuándo usar cada uno (IO-bound vs CPU-bound).
3. **OpenAPI/Swagger**: Documentación automática como feature de FastAPI.

Para Proyectos Reales

Situación	Consejo
ML Serving	Carga modelo en startup, no en cada request
Validación	Usa Pydantic para input/output schemas
Errores	HTTPException con códigos y mensajes claros
Producción	Gunicorn + Uvicorn workers

Endpoints Esenciales para ML

```
/health          → Liveness check
/ready           → Readiness check (modelo cargado)
/predict         → Inferencia principal
/predict/batch   → Inferencia batch
/model/info      → Versión, métricas, metadata
```

📺 Recursos Externos Recomendados

Ver RECURSOS_POR_MODULO.md para la lista completa.

🏷️	Recurso	Tipo
🔴	FastAPI Tutorial - Sebastián Ramírez	Video

🏷️	Recurso	Tipo
🟡	ML APIs with FastAPI	Video

Documentación oficial:
- FastAPI Documentation
- Pydantic v2

🔗 Referencias del Glosario

Ver 21_GLOSARIO.md para definiciones de:
- **FastAPI**: Framework web async para APIs
- **Pydantic**: Validación de datos con type hints
- **OpenAPI**: Especificación de APIs (Swagger)

✅ Ejercicios

Ver EJERCICIOS.md - Módulo 14:
- 14.1: Schemas Pydantic para request/response
- 14.2: Endpoint de predicción completo

STREAMLIT

GUÍA MLOPS V5.0

PORTFOLIO EDITION

16. Streamlit Dashboards para ML

🎯 Objetivo del Módulo

Construir dashboards interactivos profesionales como el de CarVision.

```
╔═══════════════════════════════════════════════════════════════════╗
│                                                                   │
│  Streamlit = La forma más rápida de crear UIs para ML             │
│                                                                   │
│  ✅ Python puro (sin HTML/CSS/JS)                                 │
│  ✅ Reactivo (cambios automáticos)                                │
│  ✅ Widgets interactivos                                          │
│  ✅ Integración con pandas/plotly                                 │
│  ✅ Deploy fácil                                                  │
│                                                                   │
╚═══════════════════════════════════════════════════════════════════╝
```

📋 Contenido

1. Estructura de un Dashboard ML
2. Caching para Performance
3. Tabs y Secciones
4. Visualizaciones con Plotly
5. Predictor Interactivo

16.1 Estructura de un Dashboard ML

Arquitectura del Dashboard CarVision

```
┌─────────────────────────────────────────────────────────────────┐
│                      CarVision Dashboard                        │
├─────────────────────────────────────────────────────────────────┤
│                                                                 │
│  ┌──────────────┐ ┌──────────────┐ ┌──────────────┐ ┌──────────────┐
│  │  Overview    │ │   Market     │ │   Model      │ │    Price     │
│  │   (KPIs)     │ │  Analysis    │ │  Metrics     │ │  Predictor   │
│  └──────────────┘ └──────────────┘ └──────────────┘ └──────────────┘
│                                                                 │
│  TAB 1: Overview                TAB 2: Market Analysis          │
│  • Total vehicles               • Investment recommendations    │
│  • Average price                • Risk assessment               │
│  • Price distribution           • Market trends                 │
│                                                                 │
│  TAB 3: Model Metrics           TAB 4: Price Predictor          │
│  • RMSE, MAE, R², MAPE         • Input form                     │
│  • Bootstrap confidence         • Single prediction             │
│  • Temporal backtest            • Gauge visualization           │
│                                                                 │
└─────────────────────────────────────────────────────────────────┘
```

Código Base

```python
# app/streamlit_app.py - Estructura básica

import streamlit as st
import pandas as pd
import joblib
from pathlib import Path

# ═══════════════════════════════════════════════════════════════════
# PAGE CONFIG (debe ser la primera llamada Streamlit)
# ═══════════════════════════════════════════════════════════════════

st.set_page_config(
    page_title="CarVision Market Intelligence",
    page_icon="🚗",
    layout="wide",
    initial_sidebar_state="expanded"
)
```

```
# ═══════════════════════════════════════════════════════════════════════════
# CACHING: Cargar datos y modelo UNA vez
# ═══════════════════════════════════════════════════════════════════════════

@st.cache_data  # Cache para datos (inmutables)
def load_data():
    """Carga dataset - cached para performance."""
    path = Path("data/raw/vehicles_us.csv")
    if path.exists():
        return pd.read_csv(path)
    return None


@st.cache_resource  # Cache para recursos (modelo, conexiones)
def load_model():
    """Carga modelo - cached para no recargar en cada interacción."""
    path = Path("artifacts/model.joblib")
    if path.exists():
        return joblib.load(path)
    return None


# ═══════════════════════════════════════════════════════════════════════════
# MAIN APP
# ═══════════════════════════════════════════════════════════════════════════

def main():
    st.title("🚗 CarVision Market Intelligence")
    st.markdown("*Análisis de mercado y predicción de precios de vehículos*")

    # Cargar datos
    df = load_data()
    model = load_model()

    if df is None:
        st.error("❌ No se encontró el dataset")
        return

    # Tabs para organizar contenido
    tab1, tab2, tab3, tab4 = st.tabs([
        "📊 Overview",
        "📈 Market Analysis",
        "🎯 Model Metrics",
        "💰 Price Predictor"
    ])

    with tab1:
        render_overview(df)

    with tab2:
        render_market_analysis(df)

    with tab3:
        render_model_metrics()

    with tab4:
        render_price_predictor(model, df)


if __name__ == "__main__":
    main()
```

16.2 Caching para Performance

@st.cache_data vs @st.cache_resource

```
# @st.cache_data: Para DATOS (DataFrames, listas, dicts)
# Se serializa y almacena. Inmutable.

@st.cache_data(ttl=3600)  # TTL: 1 hora
def load_data():
    df = pd.read_csv("data.csv")
    return df
```

```
@st.cache_data
def compute_statistics(df):
    """Cálculos pesados - cached."""
    return {
        "mean": df["price"].mean(),
        "median": df["price"].median(),
        "std": df["price"].std(),
    }


# @st.cache_resource: Para RECURSOS (modelos, conexiones DB)
# No se serializa. Se mantiene la referencia.

@st.cache_resource
def load_model():
    return joblib.load("model.joblib")

@st.cache_resource
def get_db_connection():
    return create_engine("postgresql://...")
```

Patrón: Separar Carga de Visualización

```
# ❌ MALO: Carga datos cada vez que cambia un widget
def main():
    filter_year = st.slider("Año", 2010, 2024)
    df = pd.read_csv("data.csv")  # Se ejecuta en cada interacción!
    filtered = df[df["year"] >= filter_year]
    st.dataframe(filtered)


# ✅ BUENO: Datos cargados una vez, filtrado es rápido
@st.cache_data
def load_data():
    return pd.read_csv("data.csv")

def main():
    df = load_data()  # Cached - instantáneo después de la primera carga

    filter_year = st.slider("Año", 2010, 2024)
    filtered = df[df["year"] >= filter_year]  # Operación rápida en memoria
    st.dataframe(filtered)
```

16.3 Tabs y Secciones

Tab 1: Overview

```
def render_overview(df: pd.DataFrame):
    """Tab de resumen con KPIs principales."""

    st.header("📊 Portfolio Overview")

    # KPIs en columnas
    col1, col2, col3, col4 = st.columns(4)

    with col1:
        st.metric(
            label="Total Vehicles",
            value=f"{len(df):,}",
            delta=None
        )

    with col2:
        avg_price = df["price"].mean()
        st.metric(
            label="Average Price",
            value=f"${avg_price:,.0f}",
            delta=None
        )

    with col3:
        median_price = df["price"].median()
        st.metric(
            label="Median Price",
```

```
            value=f"${median_price:,.0f}",
            delta=f"{((avg_price - median_price) / median_price * 100):+.1f}% vs avg"
        )

    with col4:
        avg_age = 2024 - df["model_year"].mean()
        st.metric(
            label="Avg Vehicle Age",
            value=f"{avg_age:.1f} years",
            delta=None
        )

    st.divider()

    # Distribución de precios
    st.subheader("Price Distribution")

    import plotly.express as px
    fig = px.histogram(
        df,
        x="price",
        nbins=50,
        title="Vehicle Price Distribution"
    )
    fig.update_layout(
        xaxis_title="Price ($)",
        yaxis_title="Count"
    )
    st.plotly_chart(fig, use_container_width=True)
```

Tab 3: Model Metrics

```
def render_model_metrics():
    """Tab de métricas del modelo."""

    st.header("🎯 Model Performance")

    # Cargar métricas
    metrics_path = Path("artifacts/metrics.json")
    if not metrics_path.exists():
        st.warning("⚠️ Métricas no disponibles. Entrene el modelo primero.")
        return

    import json
    metrics = json.loads(metrics_path.read_text())

    # Mostrar métricas principales
    col1, col2, col3, col4 = st.columns(4)

    with col1:
        st.metric("RMSE", f"${metrics['rmse']:,.0f}")
    with col2:
        st.metric("MAE", f"${metrics['mae']:,.0f}")
    with col3:
        st.metric("R²", f"{metrics['r2']:.3f}")
    with col4:
        st.metric("MAPE", f"{metrics['mape']:.1f}%")

    # Explicación de métricas
    with st.expander("ℹ️ ¿Qué significan estas métricas?"):
        st.markdown("""
        - **RMSE** (Root Mean Square Error): Error promedio en dólares. Menor es mejor.
        - **MAE** (Mean Absolute Error): Error absoluto promedio. Más interpretable que RMSE.
        - **R²** (Coefficient of Determination): % de varianza explicada. 1.0 es perfecto.
        - **MAPE** (Mean Absolute Percentage Error): Error porcentual promedio.
        """)
```

16.4 Visualizaciones con Plotly

Gráficos Interactivos

```
import plotly.express as px
import plotly.graph_objects as go
```

```
def create_price_by_brand(df: pd.DataFrame):
    """Box plot de precios por marca."""

    # Top 10 marcas por volumen
    top_brands = df["brand"].value_counts().head(10).index
    df_top = df[df["brand"].isin(top_brands)]

    fig = px.box(
        df_top,
        x="brand",
        y="price",
        title="Price Distribution by Brand (Top 10)",
        color="brand"
    )

    fig.update_layout(
        xaxis_title="Brand",
        yaxis_title="Price ($)",
        showlegend=False
    )

    return fig


def create_price_gauge(predicted_price: float, min_price: float, max_price: float):
    """Gauge para mostrar predicción de precio."""

    fig = go.Figure(go.Indicator(
        mode="gauge+number+delta",
        value=predicted_price,
        domain={"x": [0, 1], "y": [0, 1]},
        title={"text": "Predicted Price", "font": {"size": 24}},
        number={"prefix": "$", "font": {"size": 40}},
        gauge={
            "axis": {"range": [min_price, max_price], "tickprefix": "$"},
            "bar": {"color": "darkblue"},
            "steps": [
                {"range": [min_price, min_price + (max_price-min_price)*0.33], "color": "lightgreen"},
                {"range": [min_price + (max_price-min_price)*0.33, min_price + (max_price-min_price)*0.66], "color":
"yellow"},
                {"range": [min_price + (max_price-min_price)*0.66, max_price], "color": "salmon"},
            ],
            "threshold": {
                "line": {"color": "red", "width": 4},
                "thickness": 0.75,
                "value": predicted_price
            }
        }
    ))

    fig.update_layout(height=300)
    return fig
```

16.5 Predictor Interactivo

Tab 4: Price Predictor

```
def render_price_predictor(model, df: pd.DataFrame):
    """Tab de predicción interactiva de precios."""

    st.header("💰 Price Predictor")

    if model is None:
        st.error("❌ Modelo no cargado. Entrene el modelo primero.")
        return

    st.markdown("Ingrese las características del vehículo para obtener una estimación de precio.")

    # Form para inputs
    with st.form("prediction_form"):
        col1, col2 = st.columns(2)

        with col1:
            model_year = st.number_input(
```

```
                "Model Year",
                min_value=1990,
                max_value=2024,
                value=2018,
                help="Año del modelo del vehículo"
            )

            odometer = st.number_input(
                "Odometer (miles)",
                min_value=0,
                max_value=500000,
                value=50000,
                step=1000,
                help="Millaje del vehículo"
            )

            # Obtener opciones únicas del dataset
            models = sorted(df["model"].dropna().unique())
            selected_model = st.selectbox(
                "Model",
                options=models[:100],  # Limitar para performance
                index=0
            )

        with col2:
            fuel_options = df["fuel"].dropna().unique().tolist()
            fuel = st.selectbox("Fuel Type", options=fuel_options)

            trans_options = df["transmission"].dropna().unique().tolist()
            transmission = st.selectbox("Transmission", options=trans_options)

            condition_options = ["new", "like new", "excellent", "good", "fair", "salvage"]
            condition = st.selectbox("Condition", options=condition_options, index=3)

        submitted = st.form_submit_button("🔮 Predict Price", use_container_width=True)

    # Hacer predicción cuando se envía el form
    if submitted:
        # Preparar datos para predicción
        input_data = pd.DataFrame([{
            "model_year": model_year,
            "odometer": odometer,
            "model": selected_model,
            "fuel": fuel,
            "transmission": transmission,
            "condition": condition,
        }])

        try:
            # Predecir
            prediction = model.predict(input_data)[0]

            # Mostrar resultado
            st.success(f"### 💵 Estimated Price: **${prediction:,.0f}**")

            # Gauge de visualización
            min_price = df["price"].quantile(0.05)
            max_price = df["price"].quantile(0.95)

            fig = create_price_gauge(prediction, min_price, max_price)
            st.plotly_chart(fig, use_container_width=True)

            # Contexto de mercado
            percentile = (df["price"] < prediction).mean() * 100
            st.info(f"📊 Este precio está en el percentil {percentile:.0f} del mercado.")

        except Exception as e:
            st.error(f"Error en predicción: {str(e)}")
---

## 🧨 Errores habituales y cómo depurarlos en Streamlit para ML

En dashboards de ML es fácil mezclar lógica pesada con UI y terminar con apps lentas o que se rompen al mínimo cambio.

### 1) App muy lenta o que recalcula todo en cada interacción
```

```
**Síntomas típicos**

- Cada vez que mueves un slider, tarda varios segundos.
- Ves en logs que se vuelve a leer el CSV o cargar el modelo a cada cambio.

**Cómo identificarlo**

- Busca lecturas de disco o cargas de modelo dentro de la función `main` o dentro de callbacks de widgets.

**Cómo corregirlo**

- Usa `@st.cache_data` para datos y `@st.cache_resource` para el modelo, como en los ejemplos del módulo.
- Separa **carga** (funciones cacheadas) de **visualización** (funciones ligeras que usan los datos ya cargados).

---

### 2) Errores al filtrar o mapear columnas (DataFrame desalineado)

**Síntomas típicos**

- Errores tipo `KeyError: 'price'` o columnas que no existen en ciertos entornos.

**Cómo identificarlo**

- Verifica que el dataset que usas en Streamlit tenga la misma estructura que el usado en entrenamiento.

**Cómo corregirlo**

- Centraliza la carga y preprocesado básico en una función (ej. `load_data`) y reutilízala en todas las tabs.
- Añade checks defensivos (`if 'price' not in df.columns: ...`).

---

### 3) Modelo o artefactos que no se encuentran desde Streamlit

**Síntomas típicos**

- El predictor muestra `Modelo no cargado. Entrene el modelo primero.` aunque sabes que existe un modelo.

**Cómo identificarlo**

- Inspecciona la ruta usada en `load_model` y compárala con la estructura real del proyecto / contenedor.

**Cómo corregirlo**

- Alinea las rutas (`artifacts/`, `models/`) entre training, Docker y Streamlit.
- Si corres en Docker, monta los artefactos en la misma ruta que espera la app.

---

### 4) Comportamiento raro por estado oculto o re-runs

**Síntomas típicos**

- Formularios que se envían varias veces.
- Widgets que vuelven a su valor inicial sin razón aparente.

**Cómo identificarlo**

- Revisa el uso de `st.session_state` y de formularios (`st.form`).

**Cómo corregirlo**

- Usa `st.form` para agrupar inputs y ejecutar lógica solo cuando el usuario pulsa el botón de submit.
- Cuando necesites estado, usa `st.session_state` de forma explícita y documenta qué claves manejas.

---

### 5) Patrón general de debugging en Streamlit

1. Reproduce el problema con un **mínimo ejemplo** (quita tabs/funciones hasta aislar el fallo).
2. Añade logs (`st.write`, `print`) temporales para ver en qué orden se ejecuta el código.
3. Verifica qué funciones deberían estar cacheadas y cuáles no.
4. Asegúrate de que las dependencias clave (datos, modelo) están disponibles antes de renderizar la UI.
```

```
Con este enfoque, tus dashboards serán rápidos, robustos y mantenibles.

---

## 16.6 Dashboard Avanzado: Visualizaciones Profesionales

### Gauge Chart para Predicciones

```python
import plotly.graph_objects as go

def create_price_gauge(predicted_price: float, min_price: float = 0, max_price: float = 100000):
    """Crea un gauge chart para visualizar predicción de precio."""

# Determinar color según rango
    if predicted_price < max_price * 0.3:
        color = "green"
    elif predicted_price < max_price * 0.7:
        color = "orange"
    else:
        color = "red"

    fig = go.Figure(go.Indicator(
        mode="gauge+number+delta",
        value=predicted_price,
        domain={'x': [0, 1], 'y': [0, 1]},
        title={'text': "Predicted Price", 'font': {'size': 24}},
        number={'prefix': "$", 'font': {'size': 40}},
        gauge={
            'axis': {'range': [min_price, max_price], 'tickwidth': 1},
            'bar': {'color': color},
            'bgcolor': "white",
            'borderwidth': 2,
            'steps': [
                {'range': [0, max_price * 0.3], 'color': 'lightgreen'},
                {'range': [max_price * 0.3, max_price * 0.7], 'color': 'lightyellow'},
                {'range': [max_price * 0.7, max_price], 'color': 'lightcoral'}
            ],
            'threshold': {
                'line': {'color': "black", 'width': 4},
                'thickness': 0.75,
                'value': predicted_price
            }
        }
    ))

    fig.update_layout(height=300)
    return fig

# Uso en Streamlit
if prediction is not None:
    gauge = create_price_gauge(prediction, min_price=0, max_price=80000)
    st.plotly_chart(gauge, use_container_width=True)
```

Métricas con Confianza (Bootstrap)

```
def display_model_metrics(metrics: dict):
    """Muestra métricas del modelo con intervalos de confianza."""

    col1, col2, col3, col4 = st.columns(4)

    with col1:
        st.metric(
            label="RMSE",
            value=f"${metrics['rmse']:,.0f}",
            delta=f"±{metrics.get('rmse_ci', 500):,.0f}",
            delta_color="inverse"  # Menor es mejor
        )

    with col2:
        st.metric(
            label="MAE",
            value=f"${metrics['mae']:,.0f}",
```
```

```
            delta=f"±{metrics.get('mae_ci', 300):,.0f}",
            delta_color="inverse"
        )

    with col3:
        st.metric(
            label="R²",
            value=f"{metrics['r2']:.3f}",
            delta=f"{metrics.get('r2_improvement', 0):.1%} vs baseline",
            delta_color="normal"
        )

    with col4:
        st.metric(
            label="MAPE",
            value=f"{metrics['mape']:.1%}",
            delta=f"±{metrics.get('mape_ci', 0.02):.1%}",
            delta_color="inverse"
        )
```

Feature Importance Interactivo

```
import plotly.express as px

def plot_feature_importance(model, feature_names: list, top_n: int = 15):
    """Gráfico interactivo de importancia de features."""

# Extraer importancias (asume RandomForest o similar)
    if hasattr(model, 'feature_importances_'):
        importances = model.feature_importances_
    elif hasattr(model, 'named_steps'):

# Pipeline sklearn
        clf = model.named_steps.get('classifier') or model.named_steps.get('model')
        importances = clf.feature_importances_
    else:
        st.warning("Modelo no soporta feature_importances_")
        return None

# Crear DataFrame y ordenar
    df_imp = pd.DataFrame({
        'feature': feature_names,
        'importance': importances
    }).sort_values('importance', ascending=True).tail(top_n)

# Gráfico horizontal
    fig = px.bar(
        df_imp,
        x='importance',
        y='feature',
        orientation='h',
        title=f'Top {top_n} Feature Importances',
        labels={'importance': 'Importance', 'feature': 'Feature'},
        color='importance',
        color_continuous_scale='Viridis'
    )

    fig.update_layout(height=400, showlegend=False)
    return fig

# Uso
with st.expander("🔍 Feature Importance", expanded=True):
    fig = plot_feature_importance(model, feature_names)
    if fig:
        st.plotly_chart(fig, use_container_width=True)
```

Multi-page App con Navigation

```
# pages/1_📊_Overview.py
import streamlit as st

st.set_page_config(page_title="Overview", page_icon="📊")
```

```
st.title("📊 Dashboard Overview")

# ... contenido de overview

# pages/2_🔮_Predictor.py
import streamlit as st

st.set_page_config(page_title="Predictor", page_icon="🔮")
st.title("🔮 Price Predictor")

# ... contenido de predictor

# Estructura de archivos:

# app/

# ├── streamlit_app.py       # Main entry point

# └── pages/

#     ├── 1_📊_Overview.py

#     ├── 2_📈_Analysis.py

#     └── 3_🔮_Predictor.py
```

📦 Cómo se usó en el Portafolio

El dashboard de CarVision (`CarVision-Market-Intelligence/app/streamlit_app.py`) implementa:

Componente	Líneas	Técnica
4 Tabs navegables	150-600	`st.tabs()`
KPIs ejecutivos	200-250	`st.metric()` con delta
Gauge de predicción	450-500	Plotly `go.Indicator`
Feature importance	350-400	Plotly `px.bar` horizontal
Bootstrap validation	400-430	Métricas con intervalos
Caching de modelo	50-80	`@st.cache_resource`

💼 Consejos Profesionales

Recomendaciones para destacar en entrevistas y proyectos reales

Para Entrevistas

1. **Streamlit vs Gradio vs Dash**: Trade-offs (Streamlit simple, Gradio para ML demos, Dash para dashboards complejos).
2. **Session State**: Explica cómo mantener estado entre reruns.
3. **Caching**: `@st.cache_data` vs `@st.cache_resource`.

Para Proyectos Reales

Situación	Consejo
Modelo pesado	Usa `@st.cache_resource` para cargarlo una vez
Datos grandes	Pagina o muestra samples
Deployment	Streamlit Cloud para demos, Docker para producción
UX	Añade spinners y progress bars

Estructura de App Profesional

```
app/
├── streamlit_app.py   # Entry point limpio
├── pages/             # Multi-page app
├── components/        # Widgets reutilizables
└── utils/             # Lógica de negocio
```

📺 Recursos Externos Recomendados

Ver RECURSOS_POR_MODULO.md para la lista completa.

🏷️	Recurso	Tipo	Duración
🔴	Streamlit Crash Course - Patrick Loeber	Video	45 min
🟡	30 Days of Streamlit	Curso	30 días
🟡	Streamlit Multi-page Apps	Video	20 min
🟢	Streamlit Gallery	Ejemplos	-

🔗 Referencias del Glosario

Ver 21_GLOSARIO.md para definiciones de:
- **Streamlit**: Framework para dashboards en Python
- **@st.cache_resource**: Decorator para cachear modelos
- **Plotly**: Librería de visualizaciones interactivas

✅ Ejercicios

Ver EJERCICIOS.md - Módulo 15:
- 15.1: Dashboard de predicción

Ejercicio completo:
Crea un dashboard Streamlit para BankChurn con:
1. Tab Overview: Distribución de churn, KPIs
2. Tab Analysis: Factores de riesgo por segmento
3. Tab Predictor: Formulario para predecir churn de un cliente

Bonus:
- Añade gauge chart para probabilidad de churn
- Implementa SHAP waterfall plot para explicar predicciones
- Usa multi-page structure

🎤 Checkpoint: Simulacro Mid

🎯 **¡Has completado ML Core + Deploy!** (Módulos 07-15)

Si buscas posiciones **Mid-Level ML Engineer**, ahora es buen momento para practicar:

→ SIMULACRO_ENTREVISTA_MID.md
- 60 preguntas de pipelines, testing, CI/CD, Docker, APIs
- Enfoque en implementación end-to-end y debugging

[← FastAPI Producción](#mod_14_FASTAPI) | [Siguiente: Observabilidad →](#mod_16_OBSERVABILIDAD)

SIMULACRO ENTREVISTA MID

GUÍA MLOPS V5.0

PORTFOLIO EDITION

🎯 Simulacro de Entrevista Mid-Level ML Engineer

Portafolio MLOps — 60 Preguntas Técnicas

Nivel: Mid (2-4 años de experiencia)
Versión: 1.0 | Diciembre 2025

📋 Índice

1. Pipelines y Arquitectura
2. MLOps Práctico
3. Testing y Calidad
4. Deployment y APIs
5. Escenarios Prácticos

🎯 ¿Qué se espera de un Mid-Level?

Sí se espera	No se espera (aún)
Diseñar pipelines end-to-end	Arquitecturas distribuidas complejas
Implementar CI/CD funcional	Optimización de infraestructura a escala
Debugging autónomo	Mentoring de equipos
Code reviews	Decisiones de arquitectura críticas
Escribir tests comprehensivos	Diseño de sistemas desde cero

1. Pipelines y Arquitectura (Preguntas 1-15)

Pregunta 1: Pipeline Unificado

¿Por qué usar un Pipeline unificado en lugar de artefactos separados?

Respuesta:

```
# ❌ Antes: artefactos separados
preprocessor = joblib.load("preprocessor.pkl")
model = joblib.load("model.pkl")
X = preprocessor.transform(X)
pred = model.predict(X)

# ✅ Después: pipeline unificado
pipe = joblib.load("pipeline.joblib")
pred = pipe.predict(X)  # Todo en uno
```

Beneficios:
1. Elimina training-serving skew
2. Single source of truth
3. Versionado simple
4. Deploy más limpio

Pregunta 2: ColumnTransformer

Explica el ColumnTransformer del portafolio.

Respuesta:

```
preprocessor = ColumnTransformer([
    ('num', Pipeline([
        ('imputer', SimpleImputer(strategy='median')),
        ('scaler', StandardScaler())
    ]), numerical_cols),
    ('cat', Pipeline([
        ('imputer', SimpleImputer(strategy='constant', fill_value='Unknown')),
        ('encoder', OneHotEncoder(handle_unknown='ignore'))
    ]), categorical_cols),
], remainder='drop')
```

Procesa columnas en paralelo: numéricas y categóricas tienen transformaciones distintas.

Pregunta 3: Custom Transformer

¿Cuándo crear un transformer personalizado?

Respuesta:

```
class FeatureEngineer(BaseEstimator, TransformerMixin):
    def fit(self, X, y=None):
        return self

    def transform(self, X):
        X = X.copy()
        X['vehicle_age'] = 2024 - X['model_year']
        return X
```

Cuándo usar:
- Lógica de negocio específica
- Features derivadas
- Transformaciones no estándar

Pregunta 4: Estratified Split

¿Por qué stratify=y en train_test_split?

Respuesta:

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42
)
```

Con clases desbalanceadas ($80/20$ churn), `stratify=y` garantiza que train y test mantengan la misma proporción. Sin esto, un split aleatorio podría dar $85/15$ en train y $70/30$ en test.

Pregunta 5: Hyperparameter Tuning

¿Cómo optimizas hiperparámetros?

Respuesta:

```
from sklearn.model_selection import RandomizedSearchCV

param_dist = {
    'model__n_estimators': [50, 100, 200],
    'model__max_depth': [5, 10, 20, None]
}

search = RandomizedSearchCV(
    pipe, param_dist, n_iter=20, cv=5, scoring='f1'
)
search.fit(X_train, y_train)
print(search.best_params_)
```

GridSearch vs RandomizedSearch: Random es más eficiente con muchos parámetros.

Pregunta 6: Métricas de Negocio

¿Cómo traduces métricas ML a valor de negocio?

Respuesta:

```
# Costo de falsos negativos (cliente que churns sin detectar)
cost_fn = 500  # Costo de adquisición de nuevo cliente

# Costo de falsos positivos (retención innecesaria)
cost_fp = 50   # Costo de campaña de retención

# Costo total
total_cost = (FN * cost_fn) + (FP * cost_fp)
```

Optimizar para **minimizar costo total**, no solo accuracy.

Pregunta 7: Ensemble Methods

Explica VotingClassifier con soft voting.

Respuesta:

```
ensemble = VotingClassifier([
    ('lr', LogisticRegression()),
    ('rf', RandomForestClassifier())
], voting='soft', weights=[0.4, 0.6])
```

- **Soft voting**: Promedia probabilidades (mejor que votos binarios)
- **Weights**: RF tiene más peso porque tiene mejor AUC individual
- **Complementariedad**: LR lineal + RF no-lineal = menor varianza

Pregunta 8: Cross-Validation Avanzado

¿Cuándo usar TimeSeriesSplit vs StratifiedKFold?

Respuesta:

Tipo	Usar cuando
StratifiedKFold	Clasificación con clases desbalanceadas
TimeSeriesSplit	Datos temporales (evitar data leakage temporal)
GroupKFold	Datos con grupos (ej: múltiples muestras por paciente)

```
from sklearn.model_selection import TimeSeriesSplit
tscv = TimeSeriesSplit(n_splits=5)
# Train: [1,2,3], Test: [4]
# Train: [1,2,3,4], Test: [5]
```

Pregunta 9: Feature Importance

¿Cómo explicas qué features son importantes?

Respuesta:

```
# 1. Importancia de RF
importances = model.feature_importances_

# 2. Permutation importance (más robusto)
from sklearn.inspection import permutation_importance
perm = permutation_importance(model, X_test, y_test)

# 3. SHAP (más interpretable)
import shap
explainer = shap.TreeExplainer(model)
shap_values = explainer.shap_values(X_test)
```

Pregunta 10: Handling Categorical High Cardinality

¿Cómo manejas categorías con muchos valores únicos?

Respuesta:

```
# 1. Target encoding (con cuidado de leakage)
from category_encoders import TargetEncoder
encoder = TargetEncoder()

# 2. Frequency encoding
X['brand_freq'] = X['brand'].map(X['brand'].value_counts(normalize=True))

# 3. Grouping rare categories
X['brand'] = X['brand'].apply(lambda x: x if freq[x] > 0.01 else 'Other')
```

Pregunta 11: Reproducibilidad

¿Cómo garantizas experimentos reproducibles?

Respuesta:

```
# 1. Seeds
SEED = 42
np.random.seed(SEED)
random.seed(SEED)

# 2. Config versionada
config = BankChurnConfig.from_yaml("configs/config.yaml")

# 3. MLflow tracking
mlflow.log_params(config.model.dict())
mlflow.log_artifact("configs/config.yaml")

# 4. Dependencias fijas
# pyproject.toml con versiones específicas
```

Pregunta 12: Data Validation

¿Cómo validas datos de entrada en producción?

Respuesta:

```
from pydantic import BaseModel, Field, validator

class PredictionInput(BaseModel):
    credit_score: int = Field(ge=300, le=850)
    age: int = Field(ge=18, le=100)
    geography: str

    @validator('geography')
    def validate_geography(cls, v):
        valid = ['France', 'Germany', 'Spain']
        if v not in valid:
            raise ValueError(f'Must be one of {valid}')
        return v
```

Pydantic valida antes de que llegue al modelo.

Pregunta 13: Config Management

¿Por qué Pydantic para configuración?

Respuesta:

```
class ModelConfig(BaseModel):
    model_type: Literal["rf", "lr", "xgb"]
    n_estimators: int = Field(ge=10, le=1000)

    @validator('n_estimators')
    def validate_estimators(cls, v, values):
        if values.get('model_type') == 'lr' and v != 1:
            raise ValueError('LR no usa n_estimators')
        return v
```

Beneficios: Validación automática, tipos claros, errores descriptivos, documentación implícita.

Pregunta 14: Artifact Management

¿Cómo organizas artefactos del modelo?

Respuesta:

```
artifacts/
├── pipeline.joblib       # Modelo + preprocessor
├── training_results.json # Métricas
├── config.yaml          # Config usada
└── feature_names.json   # Features esperadas
```

```
# Guardar
joblib.dump(pipe, 'artifacts/pipeline.joblib')
with open('artifacts/training_results.json', 'w') as f:
    json.dump(metrics, f)
```

Pregunta 15: Model Versioning

¿Cómo versionas modelos?

Respuesta:

```
# 1. MLflow Model Registry
mlflow.sklearn.log_model(pipe, "model")
# Registrar como v1, v2, etc.

# 2. Naming convention
model_name = f"bankchurn_v{version}_{timestamp}.joblib"

# 3. Git tags
git tag -a v1.0.0 -m "Model v1.0.0: AUC 0.85"
```

2. MLOps Práctico (Preguntas 16-30)

Pregunta 16: MLflow Tracking

¿Cómo usas MLflow para tracking?

Respuesta:

```
import mlflow

with mlflow.start_run():
    mlflow.log_params({"n_estimators": 100, "max_depth": 10})
    mlflow.log_metrics({"auc": 0.85, "f1": 0.78})
    mlflow.sklearn.log_model(pipe, "model")
    mlflow.log_artifact("configs/config.yaml")
```

Pregunta 17: DVC

¿Para qué usas DVC?

Respuesta:

```
# Trackear datos
dvc add data/raw/Churn.csv

# Push a remote
dvc push

# Pull datos
dvc pull
```

Beneficio: Versionar datos grandes sin subirlos a Git.

Pregunta 18: GitHub Actions CI

Explica el workflow CI del portafolio.

Respuesta:

```
name: CI
on: [push, pull_request]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: pip install -e ".[dev]"
      - run: pytest tests/ --cov=src
      - run: ruff check src/
```

Flujo: Push → Install → Test → Lint → Pass/Fail badge.

Pregunta 19: Pre-commit Hooks

¿Qué hooks usas?

Respuesta:

```
# .pre-commit-config.yaml
repos:
  - repo: https://github.com/astral-sh/ruff-pre-commit
    hooks:
      - id: ruff
      - id: ruff-format
  - repo: https://github.com/pre-commit/mirrors-mypy
    hooks:
      - id: mypy
```

Ejecutan automáticamente antes de cada commit.

Pregunta 20: Docker Multi-stage

Explica el Dockerfile del portafolio.

Respuesta:

```
# Build stage
FROM python:3.11-slim AS builder
COPY requirements.txt .
RUN pip wheel --no-cache-dir -w /wheels -r requirements.txt

# Runtime stage
FROM python:3.11-slim
COPY --from=builder /wheels /wheels
RUN pip install --no-cache /wheels/*
COPY . /app
USER nonroot
CMD ["uvicorn", "app:app", "--host", "0.0.0.0"]
```

Multi-stage: Build pesado en stage 1 , runtime ligero en stage 2 .

Pregunta 21: Training-Serving Skew

¿Qué es training-serving skew y cómo lo evitas?

Respuesta:

Training-serving skew ocurre cuando el modelo ve datos diferentes en producción vs entrenamiento.

Causas comunes:

```
# ❌ MAL: Preprocesamiento diferente
# Training
X_train['age_normalized'] = (X_train['age'] - X_train['age'].mean()) / X_train['age'].std()

# Serving (usa stats de producción, no de training!)
X_prod['age_normalized'] = (X_prod['age'] - X_prod['age'].mean()) / X_prod['age'].std()
```

Solución: Pipeline unificado:

```
# ✅ BIEN: Todo en un pipeline
pipe = Pipeline([
    ('scaler', StandardScaler()),  # Guarda mean/std de training
    ('model', RandomForestClassifier())
])
pipe.fit(X_train, y_train)
joblib.dump(pipe, 'model.joblib')

# En producción: mismo pipeline
pipe = joblib.load('model.joblib')
pred = pipe.predict(X_new)  # Usa stats de training
```

Pregunta 22: Data Drift Detection

¿Cómo detectas data drift en producción?

Respuesta:

```
from evidently.metrics import DataDriftTable
from evidently.report import Report
```

```
# Comparar distribuciones
report = Report(metrics=[DataDriftTable()])
report.run(reference_data=X_train, current_data=X_prod)
report.save_html("drift_report.html")
```

Métodos estadísticos:

Método	Uso	Umbral típico
PSI (Population Stability Index)	Categóricas	> 0.2 = drift significativo
KS-test (Kolmogorov-Smirnov)	Numéricas	p-value < 0.05
JS Divergence	Distribuciones	> 0.1 = drift

En el portafolio: Configurable en `16_OBSERVABILIDAD.md`.

Pregunta 23: Métricas de Producción

¿Qué métricas monitoreas en producción?

Respuesta:

```
# Prometheus metrics en FastAPI
from prometheus_client import Counter, Histogram

PREDICTIONS = Counter('predictions_total', 'Total predictions', ['model_version'])
LATENCY = Histogram('prediction_latency_seconds', 'Prediction latency')

@app.post("/predict")
async def predict(data: Input):
    with LATENCY.time():
        result = model.predict(data)
    PREDICTIONS.labels(model_version="v1.2").inc()
    return result
```

Métricas clave:

Categoría	Métricas
Rendimiento	Latencia $p50/p95/p99$, throughput
Disponibilidad	Error rate, uptime
ML específicas	Prediction distribution, feature distributions
Negocio	Conversiones, costos evitados

Pregunta 24: Rollback de Modelos

¿Cómo haces rollback si un modelo falla?

Respuesta:

```
# 1. Versionado de modelos
models/
├── v1.0.0/pipeline.joblib  # ← Rollback aquí
├── v1.1.0/pipeline.joblib
└── v1.2.0/pipeline.joblib  # Actual (fallando)

# 2. Blue-Green deployment
# deployment.yaml
spec:
  replicas: 2
  selector:
    matchLabels:
      version: v1.1.0  # Cambiar a versión anterior

# 3. Con MLflow
client = MlflowClient()
```

```
client.transition_model_version_stage(
    name="bankchurn",
    version=3,
    stage="Production"  # Promover versión anterior
)
```

Proceso de rollback:
1. Detectar degradación (alertas de métricas)
2. Cambiar variable de entorno o config
3. Reiniciar pods / recargar modelo
4. Verificar métricas post-rollback

Pregunta 25: A/B Testing en ML

¿Cómo implementas A/B testing para modelos?

Respuesta:

```
import random

@app.post("/predict")
async def predict(data: Input, user_id: str):
    # Asignar bucket consistente por usuario
    bucket = hash(user_id) % 100

    if bucket < 10:  # 10% tráfico
        model = model_v2  # Challenger
        version = "v2"
    else:
        model = model_v1  # Champion
        version = "v1"

    result = model.predict(data)

    # Logging para análisis
    log_prediction(user_id, version, result)

    return {"prediction": result, "model_version": version}
```

Métricas a comparar:
- Accuracy/F1 en cohortes
- Métricas de negocio (conversión, revenue)
- Latencia y error rate

Pregunta 26: Manejo de Secrets

¿Cómo manejas secrets y credenciales?

Respuesta:

```
# ❌ MAL: Hardcoded
API_KEY = "TU_API_KEY_AQUI"  # EJEMPLO, NO USAR EN PRODUCCIÓN

# ✅ BIEN: Variables de entorno
import os
API_KEY = os.getenv("API_KEY")

# ✅ MEJOR: python-dotenv
from dotenv import load_dotenv
load_dotenv()  # Carga .env
API_KEY = os.getenv("API_KEY")
```

.env (nunca en Git):

```
# .env (valores de ejemplo)
API_KEY=REEMPLAZAR_EN_ENTORNO_REAL
DB_PASSWORD=REEMPLAZAR_EN_ENTORNO_REAL
```

.gitignore:

```
.env
.env.*
!.env.example
```

En CI/CD: GitHub Secrets → `${{ secrets.API_KEY }}`

Pregunta 27: Feature Store

¿Qué es un feature store y cuándo usarlo?

Respuesta:

Feature store = repositorio centralizado de features reutilizables.

```
# Sin feature store (problema)
# Equipo A: calcula age_bucket de una forma
# Equipo B: calcula age_bucket de otra forma
# → Inconsistencia

# Con feature store (solución)
from feast import FeatureStore

store = FeatureStore(repo_path=".")
features = store.get_online_features(
    features=["customer:age_bucket", "customer:tenure_months"],
    entity_rows=[{"customer_id": "C123"}]
)
```

Cuándo usar:

Situación	Feature Store
1 - 2 modelos, equipo pequeño	No necesario
Múltiples modelos, features compartidas	Recomendado
Features en tiempo real	Muy recomendado

Pregunta 28: Escalado de Inferencia

¿Cómo escalas inferencia para alto tráfico?

Respuesta:

```
# Kubernetes HPA (Horizontal Pod Autoscaler)
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: bankchurn-api
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: bankchurn-api
  minReplicas: 2
  maxReplicas: 10
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 70
```

Estrategias:

Estrategia	Cuándo
HPA	Tráfico variable, latencia crítica
Batch processing	Alto volumen, latencia flexible

Estrategia	Cuándo
Caching	Inputs repetidos frecuentes
Model optimization	Latencia muy baja requerida

Pregunta 29: Logging en ML

¿Qué información loggeas en producción?

Respuesta:

```
import logging
import json

logger = logging.getLogger(__name__)

@app.post("/predict")
async def predict(data: Input):
    request_id = str(uuid.uuid4())

    # Log de entrada
    logger.info(json.dumps({
        "event": "prediction_request",
        "request_id": request_id,
        "features": data.dict(),
        "timestamp": datetime.utcnow().isoformat()
    }))

    start = time.time()
    result = model.predict(data)
    latency = time.time() - start

    # Log de salida
    logger.info(json.dumps({
        "event": "prediction_response",
        "request_id": request_id,
        "prediction": result,
        "probability": float(proba),
        "latency_ms": latency * 1000,
        "model_version": "v1.2.0"
    }))

    return result
```

Logs esenciales: request_id, inputs, outputs, latencia, versión, errores.

Pregunta 30: Retraining Automático

¿Cómo automatizas el retraining?

Respuesta:

```
# GitHub Actions scheduled workflow
name: Weekly Retrain
on:
  schedule:
    - cron: '0 2 * * 0'  # Domingos 2am
  workflow_dispatch:  # Manual trigger

jobs:
  retrain:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: pip install -e ".[dev]"
      - run: python main.py --config configs/config.yaml
      - run: python scripts/evaluate.py --threshold 0.80
      - run: |
          if [ $? -eq 0 ]; then
            echo "Model passed threshold, deploying..."
            # Deploy logic
          fi
```

Triggers de retraining:

Trigger	Implementación
Scheduled	Cron jobs, Airflow
Data drift	Alerta → trigger workflow
Performance degradation	Métricas bajo umbral
New data volume	X nuevos registros

3. Testing y Calidad (Preguntas 31-40)

Pregunta 31: Tipos de Tests

¿Qué tipos de tests tiene el portafolio?

Respuesta:

```
# Unit test
def test_feature_engineer():
    fe = FeatureEngineer()
    result = fe.transform(sample_df)
    assert 'vehicle_age' in result.columns

# Integration test
def test_training_pipeline():
    trainer = Trainer(config)
    trainer.fit(X, y)
    assert trainer.model_ is not None

# API test
def test_predict_endpoint():
    response = client.post("/predict", json=sample_input)
    assert response.status_code == 200
```

Pregunta 32: Fixtures

¿Cómo usas fixtures en pytest?

Respuesta:

```
@pytest.fixture
def sample_data():
    return pd.DataFrame({
        'CreditScore': [650, 700],
        'Age': [35, 45],
        'Exited': [0, 1]
    })

@pytest.fixture
def trained_model(sample_data):
    trainer = Trainer(config)
    trainer.fit(sample_data)
    return trainer

def test_predict(trained_model, sample_data):
    preds = trained_model.predict(sample_data)
    assert len(preds) == len(sample_data)
```

Pregunta 33: Coverage

¿Cuánto coverage es suficiente?

Respuesta:

```
pytest tests/ --cov=src --cov-report=html
```

Nivel	Coverage	Comentario
Mínimo	70%	Lo básico
Bueno	80%	Estándar industria
Excelente	90%+	Código crítico

El portafolio tiene 79% en BankChurn.

Pregunta 34: Property-Based Testing

¿Qué es property-based testing?

Respuesta:

En lugar de casos específicos, defines **propiedades** que siempre deben cumplirse.

```
from hypothesis import given, strategies as st

@given(
    credit_score=st.integers(min_value=300, max_value=850),
    age=st.integers(min_value=18, max_value=100)
)
def test_prediction_is_valid(credit_score, age):
    """Propiedad: la predicción siempre es 0 o 1."""
    input_data = {"credit_score": credit_score, "age": age}
    pred = model.predict(pd.DataFrame([input_data]))
    assert pred[0] in [0, 1]

@given(df=st.data())
def test_feature_engineer_preserves_rows(df):
    """Propiedad: FeatureEngineer no cambia número de filas."""
    sample = df.draw(st.dataframes(columns=[
        st.column("age", dtype=int),
        st.column("salary", dtype=float)
    ]))
    result = fe.transform(sample)
    assert len(result) == len(sample)
```

Ventaja: Encuentra edge cases que no pensaste.

Pregunta 35: Testing de Modelos ML

¿Cómo testeas que un modelo funciona correctamente?

Respuesta:

```
# 1. Test de smoke: modelo carga y predice
def test_model_loads_and_predicts():
    model = joblib.load("artifacts/pipeline.joblib")
    sample = pd.DataFrame([{"CreditScore": 650, "Age": 35}])
    pred = model.predict(sample)
    assert len(pred) == 1

# 2. Test de formato de salida
def test_prediction_format():
    pred = model.predict(X_test)
    assert pred.shape == (len(X_test),)
    assert set(pred).issubset({0, 1})

# 3. Test de rendimiento mínimo
def test_model_performance():
    y_pred = model.predict(X_test)
    accuracy = accuracy_score(y_test, y_pred)
    assert accuracy >= 0.75, f"Accuracy {accuracy} below threshold"

# 4. Test de invarianza
def test_prediction_deterministic():
    pred1 = model.predict(X_test)
    pred2 = model.predict(X_test)
    assert np.array_equal(pred1, pred2)
```

Pregunta 36: Mocking

¿Qué es mocking y cuándo usarlo?

Respuesta:

Mocking = reemplazar dependencias reales con objetos simulados.

```
from unittest.mock import Mock, patch

# Mockear llamada a API externa
@patch('myapp.external_api.get_customer_data')
def test_predict_with_external_data(mock_api):
    # Configurar mock
    mock_api.return_value = {"credit_score": 700, "age": 45}

    # Test usa el mock en lugar de API real
    result = predict_for_customer("C123")

    # Verificar que se llamó
    mock_api.assert_called_once_with("C123")
    assert result is not None

# Mockear modelo para test de API
@patch('app.fastapi_app.model')
def test_predict_endpoint(mock_model):
    mock_model.predict.return_value = np.array([1])
    mock_model.predict_proba.return_value = np.array([[0.2, 0.8]])

    response = client.post("/predict", json=sample_input)
    assert response.json()["prediction"] == 1
```

Cuándo usar: APIs externas, base de datos, servicios lentos.

Pregunta 37: Testing de APIs

¿Cómo testeas endpoints de FastAPI?

Respuesta:

```
from fastapi.testclient import TestClient
from app.fastapi_app import app

client = TestClient(app)

def test_health_endpoint():
    response = client.get("/health")
    assert response.status_code == 200
    assert response.json()["status"] == "healthy"

def test_predict_valid_input():
    response = client.post("/predict", json={
        "credit_score": 650,
        "age": 35,
        "geography": "France"
    })
    assert response.status_code == 200
    assert "prediction" in response.json()
    assert "probability" in response.json()

def test_predict_invalid_input():
    response = client.post("/predict", json={
        "credit_score": 9999,  # Fuera de rango
        "age": 35
    })
    assert response.status_code == 422  # Validation error

def test_predict_missing_field():
    response = client.post("/predict", json={
        "credit_score": 650
        # Falta age
    })
    assert response.status_code == 422
```

Pregunta 38: Parametrized Tests

¿Cómo evitas duplicación en tests?

Respuesta:

```
import pytest

@pytest.mark.parametrize("credit_score,age,expected", [
    (300, 18, 0),   # Mínimos válidos
    (850, 100, 1),  # Máximos válidos
    (650, 45, 0),   # Caso típico
])
def test_prediction_cases(credit_score, age, expected):
    input_data = {"credit_score": credit_score, "age": age}
    pred = model.predict(pd.DataFrame([input_data]))
    # Solo verificamos que no falla, no el valor exacto
    assert pred[0] in [0, 1]

@pytest.mark.parametrize("invalid_input,expected_error", [
    ({"credit_score": -1}, "greater than or equal to 300"),
    ({"credit_score": 1000}, "less than or equal to 850"),
    ({"age": 5}, "greater than or equal to 18"),
])
def test_validation_errors(invalid_input, expected_error):
    response = client.post("/predict", json=invalid_input)
    assert response.status_code == 422
    assert expected_error in str(response.json())
```

Pregunta 39: Testing de Edge Cases

¿Cómo testeas edge cases en ML?

Respuesta:

```
# 1. Inputs vacíos
def test_empty_dataframe():
    df = pd.DataFrame()
    with pytest.raises(ValueError):
        model.predict(df)

# 2. Nulls
def test_missing_values():
    df = pd.DataFrame([{"CreditScore": None, "Age": 35}])
    # Pipeline debe manejar o fallar graciosamente
    result = model.predict(df)  # O pytest.raises si debe fallar

# 3. Outliers extremos
def test_extreme_values():
    df = pd.DataFrame([{
        "CreditScore": 850,
        "Age": 100,
        "Balance": 1_000_000_000  # Outlier extremo
    }])
    pred = model.predict(df)
    assert pred[0] in [0, 1]  # No falla

# 4. Tipos incorrectos
def test_wrong_types():
    with pytest.raises(Exception):
        model.predict("not a dataframe")

# 5. Columnas faltantes
def test_missing_columns():
    df = pd.DataFrame([{"CreditScore": 650}])  # Falta Age
    with pytest.raises(KeyError):
        model.predict(df)
```

Pregunta 40: Test-Driven Development (TDD)

¿Cómo aplicas TDD en ML?

Respuesta:

TDD: Escribir test → Ver que falla → Implementar → Ver que pasa → Refactorizar.

```
# 1. Escribir test primero
def test_feature_engineer_creates_age_bucket():
    df = pd.DataFrame({"age": [25, 45, 65]})
    fe = FeatureEngineer()
    result = fe.transform(df)

    assert "age_bucket" in result.columns
    assert list(result["age_bucket"]) == ["young", "middle", "senior"]

# 2. Test falla (FeatureEngineer no existe aún)
# 3. Implementar mínimo para pasar
class FeatureEngineer(BaseEstimator, TransformerMixin):
    def transform(self, X):
        X = X.copy()
        X["age_bucket"] = pd.cut(
            X["age"],
            bins=[0, 30, 50, 100],
            labels=["young", "middle", "senior"]
        )
        return X

# 4. Test pasa ✓
# 5. Refactorizar si es necesario
```

En ML, TDD es útil para:
- Feature engineering (definir comportamiento esperado)
- Validación de datos
- APIs

4. Deployment y APIs (Preguntas 41-50)

Pregunta 41: FastAPI Basics

Muestra un endpoint de predicción.

Respuesta:

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class Input(BaseModel):
    credit_score: int
    age: int

@app.post("/predict")
def predict(data: Input):
    X = pd.DataFrame([data.dict()])
    pred = model.predict(X)
    return {"prediction": int(pred[0])}
```

Pregunta 42: Health Checks

¿Por qué tener /health endpoint?

Respuesta:

```
@app.get("/health")
def health():
    return {
        "status": "healthy",
        "model_loaded": model is not None,
        "version": "1.0.0"
    }
```

Kubernetes usa esto para saber si el pod está listo.

Pregunta 43: Uvicorn y ASGI

¿Qué es uvicorn y por qué usarlo?

Respuesta:

Uvicorn = servidor ASGI (Asynchronous Server Gateway Interface) de alto rendimiento.

```
# Desarrollo
uvicorn app.fastapi_app:app --reload --port 8000

# Producción
uvicorn app.fastapi_app:app --host 0.0.0.0 --port 8000 --workers 4
```

Configuración para producción:

```
# Con gunicorn + uvicorn workers
gunicorn app.fastapi_app:app \
    --workers 4 \
    --worker-class uvicorn.workers.UvicornWorker \
    --bind 0.0.0.0:8000
```

ASGI vs WSGI:

WSGI	ASGI
Sync only	Async + Sync
Flask, Django	FastAPI, Starlette
Una request a la vez por worker	Múltiples requests concurrentes

Pregunta 44: CORS Configuration

¿Cómo manejas CORS en FastAPI?

Respuesta:

CORS = Cross-Origin Resource Sharing. Necesario cuando frontend y backend están en dominios distintos.

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware

app = FastAPI()

app.add_middleware(
    CORSMiddleware,
    allow_origins=[
        "http://localhost:3000",      # React dev
        "https://myapp.example.com",  # Production
    ],
    allow_credentials=True,
    allow_methods=["GET", "POST"],
    allow_headers=["*"],
)
```

En producción: Especificar orígenes exactos, no usar `"*"`.

Pregunta 45: Async en FastAPI

¿Cuándo usar async def vs def?

Respuesta:

```
# Sync: operaciones CPU-bound o librerías sync
@app.post("/predict")
def predict(data: Input):
    result = model.predict(data)  # sklearn es sync
    return {"prediction": result}

# Async: operaciones I/O-bound
@app.get("/external-data")
async def get_external():
    async with httpx.AsyncClient() as client:
        response = await client.get("https://api.example.com/data")
    return response.json()
```

Regla general:

Operación	Usar
sklearn, pandas, joblib	`def` (sync)
HTTP requests, DB async	`async def`
File I/O masivo	`async def` con aiofiles

Pregunta 46: Model Caching

¿Cómo evitas cargar el modelo en cada request?

Respuesta:

```
# FastAPI: lru_cache
from functools import lru_cache

@lru_cache()
def get_model():
    return joblib.load("artifacts/pipeline.joblib")

@app.post("/predict")
def predict(data: Input):
    model = get_model()  # Cacheado después del primer call
    return model.predict(data)

# Alternativa: cargar al inicio
model = None

@app.on_event("startup")
async def load_model():
    global model
    model = joblib.load("artifacts/pipeline.joblib")
```

Streamlit:

```
@st.cache_resource
def load_model():
    return joblib.load("artifacts/pipeline.joblib")

model = load_model()  # Cacheado entre reruns
```

Pregunta 47: Streamlit Dashboard

¿Cómo creas un dashboard de predicción?

Respuesta:

```
import streamlit as st
import pandas as pd

st.title("🏦 BankChurn Predictor")

# Sidebar para inputs
st.sidebar.header("Customer Data")
credit_score = st.sidebar.slider("Credit Score", 300, 850, 650)
age = st.sidebar.slider("Age", 18, 100, 35)
geography = st.sidebar.selectbox("Geography", ["France", "Germany", "Spain"])

# Cargar modelo (cacheado)
@st.cache_resource
def load_model():
    return joblib.load("artifacts/pipeline.joblib")

model = load_model()

# Predicción
if st.sidebar.button("Predict"):
    input_df = pd.DataFrame([{
        "CreditScore": credit_score,
```

```
        "Age": age,
        "Geography": geography
    }])

    prediction = model.predict(input_df)[0]
    proba = model.predict_proba(input_df)[0, 1]

    col1, col2 = st.columns(2)
    col1.metric("Prediction", "Churn" if prediction else "Stay")
    col2.metric("Probability", f"{proba:.1%}")
```

Pregunta 48: Docker Compose para ML

¿Cómo orquestas múltiples servicios?

Respuesta:

```
# docker-compose.yml
version: '3.8'

services:
  api:
    build: .
    ports:
      - "8000:8000"
    environment:
      - MODEL_PATH=/app/artifacts/pipeline.joblib
    volumes:
      - ./artifacts:/app/artifacts:ro
    depends_on:
      - mlflow
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:8000/health"]
      interval: 30s
      timeout: 10s
      retries: 3

  mlflow:
    image: python:3.11-slim
    command: mlflow server --host 0.0.0.0
    ports:
      - "5000:5000"
    volumes:
      - ./mlruns:/mlflow/mlruns

  prometheus:
    image: prom/prometheus
    ports:
      - "9090:9090"
    volumes:
      - ./prometheus.yml:/etc/prometheus/prometheus.yml
```

```
docker-compose up -d
docker-compose logs -f api
```

Pregunta 49: Kubernetes Deployment

¿Cómo despliegas en Kubernetes?

Respuesta:

```
# deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: bankchurn-api
spec:
  replicas: 3
  selector:
    matchLabels:
      app: bankchurn-api
  template:
    metadata:
      labels:
```

```
        app: bankchurn-api
    spec:
      containers:
      - name: api
        image: bankchurn-api:v1.0.0
        ports:
        - containerPort: 8000
        resources:
          requests:
            memory: "256Mi"
            cpu: "250m"
          limits:
            memory: "512Mi"
            cpu: "500m"
        livenessProbe:
          httpGet:
            path: /health
            port: 8000
          initialDelaySeconds: 30
          periodSeconds: 10
        readinessProbe:
          httpGet:
            path: /health
            port: 8000
          initialDelaySeconds: 5
          periodSeconds: 5
---
apiVersion: v1
kind: Service
metadata:
  name: bankchurn-api
spec:
  selector:
    app: bankchurn-api
  ports:
  - port: 80
    targetPort: 8000
  type: LoadBalancer
```

```
kubectl apply -f deployment.yaml
kubectl get pods
kubectl logs -f deployment/bankchurn-api
```

Pregunta 50: Horizontal Pod Autoscaler

¿Cómo escalas automáticamente?

Respuesta:

```
# hpa.yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: bankchurn-api-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: bankchurn-api
  minReplicas: 2
  maxReplicas: 10
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 70
  - type: Resource
    resource:
      name: memory
      target:
        type: Utilization
        averageUtilization: 80
```

```
  behavior:
    scaleDown:
      stabilizationWindowSeconds: 300  # Esperar 5 min antes de escalar down
    scaleUp:
      stabilizationWindowSeconds: 60   # Escalar up más rápido
```

```
kubectl apply -f hpa.yaml
kubectl get hpa
# NAME                  REFERENCE              TARGETS   MINPODS   MAXPODS   REPLICAS
# bankchurn-api-hpa     Deployment/bankchurn   45%/70%   2         10        3
```

5. Escenarios Prácticos (Preguntas 51-60)

Pregunta 51: Debug de Producción

El modelo tiene accuracy 85% en dev pero 60% en prod. ¿Por qué?

Respuesta:

1. **Data drift**: Distribución de datos cambió
2. **Feature mismatch**: Features procesadas diferente
3. **Training-serving skew**: Preprocesamiento distinto
4. **Datos de prod con más ruido**: Edge cases no vistos

Acciones: Comparar distribuciones, revisar pipeline, logging de inputs.

Pregunta 52: Code Review

¿Qué buscas en un code review de ML?

Respuesta:

- [ ] Data leakage en split/preprocessing
- [ ] Tests para features y modelo
- [ ] Config externalizada (no hardcoded)
- [ ] Type hints y docstrings
- [ ] Reproducibilidad (seeds, versiones)
- [ ] Logging apropiado

Pregunta 53: Explicabilidad del Modelo

El cliente dice: "No puedo usar tu modelo si no me explicas por qué toma las decisiones".

Respuesta:

```
import shap

# 1. SHAP para explicaciones individuales
explainer = shap.TreeExplainer(model)
shap_values = explainer.shap_values(X_sample)

# Waterfall plot para una predicción
shap.waterfall_plot(shap.Explanation(
    values=shap_values[0],
    base_values=explainer.expected_value,
    data=X_sample.iloc[0]
))

# 2. Feature importance global
shap.summary_plot(shap_values, X_sample)

# 3. En producción: incluir en respuesta
@app.post("/predict")
def predict(data: Input):
    pred = model.predict(X)[0]

    # Top 3 razones
    shap_vals = explainer.shap_values(X)
    top_features = sorted(
        zip(feature_names, shap_vals[0]),
        key=lambda x: abs(x[1]),
        reverse=True
    )[:3]
```

```
    return {
        "prediction": pred,
        "explanation": [
            {"feature": f, "impact": v}
            for f, v in top_features
        ]
    }
```

Pregunta 54: Optimización de Latencia

El modelo tarda 500 ms por predicción. El negocio necesita < 100 ms.

Respuesta:

```
# 1. Profiling: ¿dónde está el cuello de botella?
import cProfile
cProfile.run('model.predict(X_sample)')

# 2. Opciones de optimización:

# a) Modelo más ligero
from sklearn.linear_model import LogisticRegression
# LR es 10x más rápido que RF

# b) Reducir features
from sklearn.feature_selection import SelectKBest
selector = SelectKBest(k=10)  # Solo top 10 features

# c) Batch predictions
@app.post("/predict/batch")
def predict_batch(items: List[Input]):
    X = pd.DataFrame([item.dict() for item in items])
    preds = model.predict(X)  # Una llamada, muchas predicciones
    return {"predictions": preds.tolist()}

# d) Caching de predicciones frecuentes
from functools import lru_cache

@lru_cache(maxsize=1000)
def predict_cached(credit_score: int, age: int):
    return model.predict([[credit_score, age]])[0]

# e) ONNX para inferencia rápida
from skl2onnx import convert_sklearn
onnx_model = convert_sklearn(model, initial_types=[...])
```

Métricas de latencia:

Optimización	Latencia típica
RF sklearn	50-200 ms
LR sklearn	1-5 ms
ONNX	1-10 ms
Caching (hit)	< 1 ms

Pregunta 55: Manejo de PII

El dataset contiene nombres, emails y teléfonos. ¿Cómo lo manejas?

Respuesta:

```
# 1. Identificar columnas PII
pii_columns = ["name", "email", "phone", "ssn", "address"]

# 2. Anonimización
import hashlib

def anonymize_pii(df: pd.DataFrame) -> pd.DataFrame:
    df = df.copy()
```

```
    for col in pii_columns:
        if col in df.columns:
            # Hash irreversible
            df[col] = df[col].apply(
                lambda x: hashlib.sha256(str(x).encode()).hexdigest()[:16]
            )
    return df

# 3. Drop antes de training (mejor opción)
X = df.drop(columns=pii_columns, errors='ignore')

# 4. En logs: nunca loggear PII
logger.info(f"Prediction for customer {customer_id[:4]}***")

# 5. En respuestas de API: mascarar
def mask_email(email: str) -> str:
    parts = email.split("@")
    return f"{parts[0][:2]}***@{parts[1]}"
```

Compliance checklist:
- [ ] PII no está en features del modelo
- [ ] PII no aparece en logs
- [ ] PII no se almacena en MLflow/tracking
- [ ] Acceso a datos restringido

Pregunta 56: Fairness y Bias

Producto detectó que el modelo rechaza más a clientes de cierta región.

Respuesta:

```
from fairlearn.metrics import MetricFrame
from sklearn.metrics import accuracy_score, recall_score

# 1. Calcular métricas por grupo
metrics = MetricFrame(
    metrics={
        "accuracy": accuracy_score,
        "recall": recall_score
    },
    y_true=y_test,
    y_pred=y_pred,
    sensitive_features=df_test["geography"]
)

print(metrics.by_group)
#              accuracy  recall
# geography
# France         0.85     0.80
# Germany        0.83     0.78
# Spain          0.70     0.55  # ← Problema

# 2. Mitigación
from fairlearn.reductions import ExponentiatedGradient
from fairlearn.constraints import DemographicParity

mitigator = ExponentiatedGradient(
    estimator=base_model,
    constraints=DemographicParity()
)
mitigator.fit(X_train, y_train, sensitive_features=train_geography)

# 3. Monitoreo continuo
# Alertar si la diferencia entre grupos > 10%
```

Pregunta 57: Tests Flaky en CI

El CI pasa 80% de las veces y falla 20% sin cambios en código.

Respuesta:

```
# 1. Problema común: Random sin seed
# ❌ Mal
model = RandomForestClassifier()
```

```
# ✅ Bien
model = RandomForestClassifier(random_state=42)

# 2. Problema: Orden de ejecución
# ❌ Mal: test depende de otro
def test_predict():
    assert model.predict(X) == [1]  # model de test anterior

# ✅ Bien: tests aislados
@pytest.fixture
def trained_model():
    m = Model()
    m.fit(X, y)
    return m

def test_predict(trained_model):
    assert trained_model.predict(X)

# 3. Problema: Timeouts en CI
# ❌ Mal
requests.get("https://external-api.com", timeout=5)

# ✅ Bien
@pytest.fixture
def mock_api():
    with patch("myapp.api.get") as mock:
        mock.return_value = {"data": "test"}
        yield mock

# 4. Debug: Correr múltiples veces
pytest tests/ --count=10  # Con pytest-repeat
```

Pregunta 58: Modelo Grande para Deploy

El modelo pesa 2 GB y tarda 30 s en cargar. ¿Cómo optimizas?

Respuesta:

```
# 1. Quantization (reducir precisión)
import onnxruntime as ort
from onnxruntime.quantization import quantize_dynamic

quantize_dynamic(
    "model.onnx",
    "model_quantized.onnx",
    weight_type=ort.QuantType.QInt8
)
# 2GB → ~500MB

# 2. Model distillation (modelo más pequeño que imita al grande)
teacher = load_large_model()
student = SmallModel()

# Entrenar student con outputs del teacher
student_preds = student(X)
teacher_preds = teacher(X)
loss = mse_loss(student_preds, teacher_preds)

# 3. Feature selection (menos features = modelo más pequeño)
from sklearn.feature_selection import SelectFromModel
selector = SelectFromModel(model, threshold="median")
X_reduced = selector.transform(X)  # Menos columnas

# 4. Lazy loading en API
model = None

@app.on_event("startup")
async def load():
    global model
    model = joblib.load("model.joblib")  # Solo una vez
```

Pregunta 59: Muchos Falsos Positivos

El modelo predice churn para clientes que claramente no van a irse.

Respuesta:

```
# 1. Ajustar threshold (default=0.5)
y_proba = model.predict_proba(X_test)[:, 1]

# Encontrar threshold óptimo
from sklearn.metrics import precision_recall_curve

precision, recall, thresholds = precision_recall_curve(y_test, y_proba)

# Threshold que maximiza F1
f1_scores = 2 * (precision * recall) / (precision + recall)
optimal_threshold = thresholds[np.argmax(f1_scores)]
print(f"Optimal threshold: {optimal_threshold}")  # Ej: 0.65

# Usar nuevo threshold
y_pred = (y_proba >= optimal_threshold).astype(int)

# 2. Revisar balance de datos
print(y_train.value_counts(normalize=True))
# Si muy desbalanceado: SMOTE, class_weight

# 3. Verificar data leakage
# ¿Hay features que "predicen perfectamente"?
for col in X.columns:
    corr = X[col].corr(y)
    if abs(corr) > 0.9:
        print(f"⚠️ {col} tiene correlación {corr}")
```

Pregunta 60: Comunicar a Stakeholders No Técnicos

El VP de producto pregunta: "¿Funciona o no funciona tu modelo?"

Respuesta:

```
# 1. Traducir métricas técnicas a impacto de negocio
"""
❌ Mal: "El modelo tiene AUC 0.85 y F1 0.78"

✅ Bien:
"Por cada 100 clientes que van a hacer churn:
- Detectamos 78 antes de que se vayan
- De los que marcamos como riesgo, 82% efectivamente se iban

Impacto: Si cada cliente perdido cuesta $500,
el modelo puede prevenir $31,200 en pérdidas mensuales
(78 clientes × $500 × 80% tasa de retención con intervención)"
"""

# 2. Visualizaciones claras
import plotly.express as px

# Confusion matrix visual
fig = px.imshow(
    [[TN, FP], [FN, TP]],
    labels=dict(x="Predicted", y="Actual"),
    x=["Stay", "Churn"],
    y=["Stay", "Churn"],
    text_auto=True
)
fig.show()

# 3. Dashboard ejecutivo en Streamlit
st.metric("Clientes en Riesgo", "234", delta="-12 vs mes pasado")
st.metric("Precision Retención", "82%", delta="+5%")
st.metric("Ahorro Estimado", "$45,000/mes")
```

Regla de oro: Siempre conectar con dinero o KPIs que el stakeholder ya conoce.

📚 Recursos

Tema	Módulo
Pipelines	07_SKLEARN_PIPELINES.md
Testing	11_TESTING_ML.md
CI/CD	12_CI_CD.md
Docker	13_DOCKER.md
FastAPI	14_FASTAPI.md
MLflow	10_EXPERIMENT_TRACKING.md

¡Éxito en tu entrevista! 🚀

OBSERVABILIDAD

GUÍA MLOPS V5.0

PORTFOLIO EDITION

17. Observabilidad para ML

🎯 Objetivo del Módulo

Implementar monitoreo completo: logs, métricas, y drift detection como en el portafolio.

```
╔═══════════════════════════════════════════════════════════════════╗
│                                                                   │
│  "Si no puedo verlo en un dashboard, no sé si está funcionando."  │
│                                        — Mentalidad Senior        │
│                                                                   │
│  OBSERVABILIDAD = LOGS + METRICS + TRACES + ML MONITORING         │
│                                                                   │
╚═══════════════════════════════════════════════════════════════════╝
```

📋 Contenido

1. Las 4 Señales de Oro
2. Prometheus + Grafana
3. Logging Estructurado
4. Model Monitoring

17.1 Las 4 Señales de Oro

```
┌──────────────────────────────────────────────────────────────────┐
│                    📊 LAS 4 SEÑALES DE ORO (+ ML)                │
├──────────────────────────────────────────────────────────────────┤
│                                                                  │
│  1. LATENCIA          ¿Cuánto tarda una predicción?              │
│     ────────────      Target: P99 < 100ms                        │
│                       Alerta: P99 > 200ms                        │
│                                                                  │
│  2. TRÁFICO           ¿Cuántas requests por segundo?             │
│     ────────          Monitorear: picos, tendencias, anomalías   │
│                                                                  │
│  3. ERRORES           ¿Qué porcentaje de requests falla?         │
│     ───────           Target: Error rate < 0.1%                  │
│                       Alerta: Error rate > 1%                    │
│                                                                  │
│  4. SATURACIÓN        ¿Cuánto recurso queda?                     │
│     ──────────        Alerta: CPU > 80%, Memory > 85%            │
│                                                                  │
│  + ML-ESPECÍFICO:                                                │
│  ────────────────                                                │
│  5. DATA DRIFT        ¿Los datos de entrada cambiaron?           │
│  6. PREDICTION DRIFT  ¿Las predicciones cambiaron distribución?  │
│  7. MODEL DECAY       ¿El accuracy está degradando?              │
│                                                                  │
└──────────────────────────────────────────────────────────────────┘
```

17.2 Prometheus + Grafana

Configuración del Portafolio

```yaml
# infra/prometheus-config.yaml

global:
  scrape_interval: 15s
  evaluation_interval: 15s

scrape_configs:
  - job_name: 'bankchurn-api'
    static_configs:
      - targets: ['bankchurn:8000']
    metrics_path: /metrics

  - job_name: 'carvision-api'
    static_configs:
      - targets: ['carvision:8000']
    metrics_path: /metrics

  - job_name: 'telecom-api'
```

```
    static_configs:
      - targets: ['telecom:8000']
    metrics_path: /metrics
```

Métricas en FastAPI

```
# app/metrics.py

from prometheus_client import Counter, Histogram, Gauge, generate_latest
from fastapi import Response

# Métricas
PREDICTIONS_TOTAL = Counter(
    'predictions_total',
    'Total de predicciones realizadas',
    ['model', 'result']
)

PREDICTION_LATENCY = Histogram(
    'prediction_latency_seconds',
    'Latencia de predicciones',
    ['model'],
    buckets=[0.01, 0.025, 0.05, 0.1, 0.25, 0.5, 1.0]
)

MODEL_LOADED = Gauge(
    'model_loaded',
    'Indica si el modelo está cargado',
    ['model']
)

PREDICTION_PROBABILITY = Histogram(
    'prediction_probability',
    'Distribución de probabilidades predichas',
    ['model'],
    buckets=[0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1.0]
)


# Endpoint de métricas
@app.get("/metrics")
async def metrics():
    return Response(
        content=generate_latest(),
        media_type="text/plain"
    )


# Uso en predicción
import time

@app.post("/predict")
async def predict(request: PredictionRequest):
    start = time.time()

    # ... predicción ...
    proba = model.predict_proba(df)[0, 1]
    prediction = int(proba >= 0.5)

    # Registrar métricas
    latency = time.time() - start
    PREDICTION_LATENCY.labels(model="bankchurn").observe(latency)
    PREDICTIONS_TOTAL.labels(model="bankchurn", result=str(prediction)).inc()
    PREDICTION_PROBABILITY.labels(model="bankchurn").observe(proba)

    return {"prediction": prediction, "probability": proba}
```

17.3 Logging Estructurado

Configuración Profesional

```
# src/logging_config.py

import logging
```

```
import json
import sys
from datetime import datetime


class JSONFormatter(logging.Formatter):
    """Formatter que produce logs en JSON para fácil parsing."""

    def format(self, record):
        log_obj = {
            "timestamp": datetime.utcnow().isoformat(),
            "level": record.levelname,
            "logger": record.name,
            "message": record.getMessage(),
            "module": record.module,
            "function": record.funcName,
            "line": record.lineno,
        }

        # Añadir extras si existen
        if hasattr(record, "request_id"):
            log_obj["request_id"] = record.request_id
        if hasattr(record, "user_id"):
            log_obj["user_id"] = record.user_id
        if hasattr(record, "prediction"):
            log_obj["prediction"] = record.prediction

        # Añadir exception si existe
        if record.exc_info:
            log_obj["exception"] = self.formatException(record.exc_info)

        return json.dumps(log_obj)


def setup_logging(level: str = "INFO", json_format: bool = True):
    """Configura logging para producción."""

    root = logging.getLogger()
    root.setLevel(getattr(logging, level.upper()))

    handler = logging.StreamHandler(sys.stdout)

    if json_format:
        handler.setFormatter(JSONFormatter())
    else:
        handler.setFormatter(logging.Formatter(
            "%(asctime)s | %(levelname)-8s | %(name)s | %(message)s"
        ))

    root.addHandler(handler)

    # Silenciar loggers ruidosos
    logging.getLogger("urllib3").setLevel(logging.WARNING)
    logging.getLogger("uvicorn.access").setLevel(logging.WARNING)
```

Logs con Contexto

```
import logging
import uuid

logger = logging.getLogger(__name__)

@app.post("/predict")
async def predict(request: PredictionRequest):
    request_id = str(uuid.uuid4())[:8]

    # Log con contexto
    logger.info(
        "Prediction request received",
        extra={
            "request_id": request_id,
            "credit_score": request.CreditScore,
            "geography": request.Geography,
        }
```

```
    )

    try:
        prediction = model.predict(...)

        logger.info(
            "Prediction completed",
            extra={
                "request_id": request_id,
                "prediction": prediction,
                "latency_ms": latency * 1000,
            }
        )

        return {"prediction": prediction}

    except Exception as e:
        logger.error(
            f"Prediction failed: {str(e)}",
            extra={"request_id": request_id},
            exc_info=True
        )
        raise
```

17.4 Model Monitoring (Drift Detection)

Script de Drift Detection

```
# monitoring/check_drift.py - Código REAL del portafolio

"""
Detecta drift en datos usando Evidently AI.

Compara datos de referencia (training) con datos actuales (producción).
Genera reporte HTML y métricas JSON.

Uso:
    python monitoring/check_drift.py --reference data/train.csv --current data/recent.csv
"""

import argparse
import json
from pathlib import Path
from datetime import datetime

import pandas as pd

try:
    from evidently import ColumnMapping
    from evidently.report import Report
    from evidently.metric_preset import DataDriftPreset, DataQualityPreset
    EVIDENTLY_AVAILABLE = True
except ImportError:
    EVIDENTLY_AVAILABLE = False


def check_drift(
    reference_data: pd.DataFrame,
    current_data: pd.DataFrame,
    output_dir: Path,
    numerical_features: list = None,
    categorical_features: list = None,
) -> dict:
    """
    Ejecuta análisis de drift entre datos de referencia y actuales.

    Returns
    -------
    dict
        Métricas de drift incluyendo:
        - dataset_drift: bool (True si hay drift significativo)
        - drift_share: float (% de features con drift)
        - drifted_features: list (features con drift detectado)
    """
```

```
    if not EVIDENTLY_AVAILABLE:
        return {"error": "Evidently no instalado", "dataset_drift": None}

    # Column mapping
    column_mapping = ColumnMapping()
    if numerical_features:
        column_mapping.numerical_features = numerical_features
    if categorical_features:
        column_mapping.categorical_features = categorical_features

    # Crear reporte
    report = Report(metrics=[
        DataDriftPreset(),
        DataQualityPreset(),
    ])

    report.run(
        reference_data=reference_data,
        current_data=current_data,
        column_mapping=column_mapping
    )

    # Guardar HTML
    output_dir.mkdir(parents=True, exist_ok=True)
    timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")

    html_path = output_dir / f"drift_report_{timestamp}.html"
    report.save_html(str(html_path))

    # Extraer métricas
    results = report.as_dict()

    drift_metrics = {
        "timestamp": timestamp,
        "reference_rows": len(reference_data),
        "current_rows": len(current_data),
        "dataset_drift": False,
        "drift_share": 0.0,
        "drifted_features": [],
        "report_path": str(html_path),
    }

    # Parsear resultados de Evidently
    for metric in results.get("metrics", []):
        if "DataDriftTable" in str(metric.get("metric", "")):
            result = metric.get("result", {})
            drift_metrics["dataset_drift"] = result.get("dataset_drift", False)
            drift_metrics["drift_share"] = result.get("drift_share", 0.0)

            # Features con drift
            drift_by_columns = result.get("drift_by_columns", {})
            for col, col_data in drift_by_columns.items():
                if col_data.get("drift_detected", False):
                    drift_metrics["drifted_features"].append(col)

    # Guardar métricas JSON
    json_path = output_dir / f"drift_metrics_{timestamp}.json"
    with open(json_path, "w") as f:
        json.dump(drift_metrics, f, indent=2)

    return drift_metrics


def main():
    parser = argparse.ArgumentParser(description="Check data drift")
    parser.add_argument("--reference", required=True, help="Path to reference data CSV")
    parser.add_argument("--current", required=True, help="Path to current data CSV")
    parser.add_argument("--output", default="artifacts", help="Output directory")
    args = parser.parse_args()

    reference = pd.read_csv(args.reference)
    current = pd.read_csv(args.current)

    metrics = check_drift(reference, current, Path(args.output))
```

```
    print(json.dumps(metrics, indent=2))

    # Exit code basado en drift
    if metrics.get("dataset_drift"):
        print("⚠️ DRIFT DETECTADO")
        exit(1)
    else:
        print("✅ No hay drift significativo")
        exit(0)


if __name__ == "__main__":
    main()
```

GitHub Action para Drift Scheduled

```
# .github/workflows/drift-detection.yml

name: Drift Detection

on:
  schedule:
    - cron: '0 2 * * *'  # Diario a las 2am UTC
  workflow_dispatch:

jobs:
  check-drift:
    runs-on: ubuntu-latest

    steps:
      - uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: '3.11'

      - name: Install dependencies
        run: |
          pip install pandas evidently

      - name: Run drift check
        run: |
          python monitoring/check_drift.py \
            --reference data/reference/train.csv \
            --current data/recent/latest.csv \
            --output artifacts/drift

      - name: Upload report
        uses: actions/upload-artifact@v4
        with:
          name: drift-report
          path: artifacts/drift/

      - name: Create issue if drift detected
        if: failure()
        uses: actions/github-script@v7
        with:
          script: |
            github.rest.issues.create({
              owner: context.repo.owner,
              repo: context.repo.repo,
              title: '⚠️ Data Drift Detected',
              body: 'Drift detection workflow failed. Check the artifacts.',
              labels: ['drift', 'monitoring']
            })
```

🧨 Errores habituales y cómo depurarlos en Observabilidad ML

En observabilidad ML es habitual tener dashboards bonitos pero poca señal útil, o scripts de drift que fallan en silencio.

1) Métricas que no aparecen en Prometheus/Grafana

Síntomas típicos

- En Grafana, los paneles muestran `No data`.
- En Prometheus, la métrica `predictions_total` no existe o tiene solo ceros.

Cómo identificarlo

- Verifica que el endpoint `/metrics` responde localmente (`curl http://localhost:8000/metrics`).
- Revisa `prometheus-config.yaml`:
- ¿El `job_name` y `targets` apuntan al host/puerto correctos?
- ¿`metrics_path` es `/metrics`?

Cómo corregirlo

- Asegura que el API exponga `/metrics` y que el contenedor esté accesible desde Prometheus (mismo docker network).
- Usa nombres de servicio (`bankchurn:8000`) coherentes con `docker-compose`.

2) Alertas demasiado ruidosas (alert fatigue)

Síntomas típicos

- Canal de Slack/Email lleno de alertas constantes que el equipo ignora.

Cómo identificarlo

- Revisa las reglas de alerta: thresholds demasiado agresivos (por ejemplo, alertar por cualquier spike puntual).

Cómo corregirlo

- Usa ventanas de tiempo y reglas de severidad (warning vs critical).
- Define claramente métricas **críticas** (latencia P 9 9 , error rate, dataset_drift) y otras solo informativas.

3) Logs JSON imposibles de parsear

Síntomas típicos

- La herramienta de logs (ELK, Loki, etc.) no reconoce campos como `request_id` o `prediction`.
- Aparecen líneas mezcladas de formatos distintos.

Cómo identificarlo

- Revisa `setup_logging`: ¿todos los handlers usan `JSONFormatter` en producción?
- Busca logs que usen `print` en vez de `logger.info`.

Cómo corregirlo

- Centraliza la configuración de logging y evita crear loggers adicionales con otros formatos.
- Usa siempre `extra={...}` en los logs de negocio en vez de concatenar strings.

4) Script de drift que falla en CI o nunca encuentra drift

Síntomas típicos

- El workflow `drift-detection.yml` falla por `ImportError: evidently` o rutas incorrectas.
- El script siempre devuelve "✅ No hay drift" aunque sabes que los datos cambiaron.

Cómo identificarlo

- Revisa los paths `--reference` y `--current` usados en el workflow.
- Comprueba que `EVIDENTLY_AVAILABLE` es `True` y que las columnas de referencia/actual coinciden.

Cómo corregirlo

- Alinea las rutas de datos de referencia y actuales con la estructura de tu repo.
- Asegúrate de instalar `evidently` en el job de CI (`pip install evidently`).
- Revisa el JSON de métricas generado para validar que `drift_share` y `drifted_features` tienen sentido.

5) Patrón general de debugging en observabilidad ML

1. Empieza por el **flujo de datos**: API → `/metrics` → Prometheus → Grafana.
2. Verifica que logs y métricas contengan campos de negocio (no solo técnica básica).
3. Revisa periódicamente los umbrales de alerta según el comportamiento real del sistema.
4. Usa los reports de drift como insumo para decisiones, no como verdad absoluta: combínalos con métricas de negocio.

Con esta mentalidad, la observabilidad deja de ser un "extra" y se convierte en tu principal herramienta para operar modelos en producción.

💼 Consejos Profesionales

Recomendaciones para destacar en entrevistas y proyectos reales

Para Entrevistas

1. **Observability vs Monitoring**: Monitoring = métricas predefinidas, Observability = entender comportamiento inesperado.
2. **Three Pillars**: Logs, Metrics, Traces. Explica cada uno.
3. **ML Monitoring**: Model drift, data drift, concept drift.

Para Proyectos Reales

Situación	Consejo
Alertas	Evita alert fatigue: alerta solo lo accionable
Dashboards	Un dashboard por audiencia (ops, ML, negocio)
On-call	Documenta runbooks para cada alerta
Drift detection	Monitorea distribuciones de features y predictions

Métricas Clave para ML

- **Serving**: Latency p50/p95/p99, error rate, throughput
- **Model**: Prediction distribution, confidence scores
- **Data**: Missing values, schema changes, drift
- **Business**: Conversion, revenue impact

📺 Recursos Externos Recomendados

Ver RECURSOS_POR_MODULO.md para la lista completa.

🏷️	Recurso	Tipo
🔴	Prometheus + Grafana - TechWorld Nana	Video
🟡	ML Monitoring with Evidently	Video

Documentación oficial:
- Prometheus
- Grafana
- Evidently AI

🔗 Referencias del Glosario

Ver 21_GLOSARIO.md para definiciones de:
- **Data Drift**: Cambio en distribución de features
- **Prometheus**: Sistema de monitoreo y alertas
- **PSI**: Population Stability Index

✅ Ejercicios

Ver EJERCICIOS.md - Módulo 16:
- 16.1: Logging estructurado JSON

Checkpoint:
- [ ] Tienes endpoint `/metrics` en tu API
- [ ] Logs en formato JSON estructurado
- [ ] Script de drift detection funcional
- [ ] Alertas configuradas para métricas críticas

DESPLIEGUE

GUÍA MLOPS V5.0

PORTFOLIO EDITION

MÓDULO 12: SERVERLESS VS CONTENEDORES

Cuándo Usar Lambda, ECS o Kubernetes

Guía MLOps v5.0: Senior Edition | DuqueOM | Noviembre 2025

🌐 MÓDULO 12: Serverless vs Contenedores

La Decisión que Define tu Arquitectura

"No hay solución universal. Hay trade-offs que debes entender."

Duración	Teoría	Práctica
4 - 5 horas	40%	60%

🎯 ADR: ¿Dónde Desplegar?

```
╔═══════════════════════════════════════════════════════════════════╗
│  ADR-008: Selección de Plataforma de Despliegue                   │
╠═══════════════════════════════════════════════════════════════════╣
│                                                                   │
│  OPCIONES:                                                        │
│  1. Serverless (AWS Lambda, GCP Cloud Functions)                  │
│  2. Contenedores Managed (ECS, Cloud Run)                         │
│  3. Kubernetes (EKS, GKE, self-managed)                           │
│                                                                   │
│  FACTORES DE DECISIÓN:                                            │
│  • Tráfico esperado (requests/mes)                                │
│  • Requisitos de latencia                                         │
│  • Tamaño del equipo de Ops                                       │
│  • Presupuesto                                                    │
│  • Complejidad del modelo (GPU, memoria)                          │
│                                                                   │
╚═══════════════════════════════════════════════════════════════════╝
```

12.1 Matriz de Decisión

```
╔═══════════════════════════════════════════════════════════════════╗
│                   MATRIZ DE DECISIÓN DE DESPLIEGUE                │
╠═══════════════════════════════════════════════════════════════════╣
│                                                                   │
│   Factor              │ Lambda/Serverless │ ECS/Cloud Run │ Kubernetes │
│   ────────────────────┼───────────────────┼───────────────┼────────────│
│   Tráfico             │ < 1M req/mes      │ 1M-100M       │ > 100M     │
│   Latencia            │ Variable (cold)   │ Consistente   │ Consistente│
│   Costo bajo tráfico  │ 💰 Muy bajo       │ 💰💰 Medio    │ 💰💰💰 Alto │
│   Costo alto tráfico  │ 💰💰💰 Caro       │ 💰💰 Medio    │ 💰 Barato  │
│   Complejidad Ops     │ ⭐ Baja           │ ⭐⭐ Media    │ ⭐⭐⭐⭐ Alta │
│   Equipo necesario    │ 1 persona         │ 2-3 personas  │ 5+ personas │
│   GPU Support         │ ❌                │ ✅            │ ✅          │
│   Max memoria         │ 10GB              │ 120GB+        │ Ilimitado  │
│   Max timeout         │ 15 min            │ Ilimitado     │ Ilimitado  │
│   Modelo size límite  │ ~250MB pkg        │ Sin límite    │ Sin límite │
│   Auto-scaling        │ Automático        │ Automático    │ Configurable│
│   Vendor lock-in      │ Alto              │ Medio         │ Bajo       │
│                                                                   │
╚═══════════════════════════════════════════════════════════════════╝
```

12.2 Opción 1: Serverless (AWS Lambda)

Cuándo Usar

```
✅ USA LAMBDA SI:
• Tráfico bajo o esporádico (< 1M requests/mes)
• Modelo pequeño (< 250MB empaquetado)
• Latencia variable es aceptable
• No tienes equipo de DevOps
• Quieres minimizar costos en bajo tráfico
```

```
❌ NO USES LAMBDA SI:
• Necesitas GPU
• Modelo > 250MB
• Cold starts son inaceptables (< 100ms requerido)
• Tráfico constante y alto
```

Estructura para Lambda

```
lambda_function/
├── handler.py          # Entry point
├── model/
│   └── pipeline.pkl    # Modelo (< 250MB)
├── src/
│   └── inference.py    # Lógica
└── requirements.txt
```

handler.py

```
# handler.py - AWS Lambda Handler
import json
import joblib
import pandas as pd
from pathlib import Path

# Cargar modelo al inicio (fuera del handler para reutilizar)
MODEL_PATH = Path(__file__).parent / "model" / "pipeline.pkl"
model = joblib.load(MODEL_PATH)

def lambda_handler(event, context):
    """AWS Lambda handler."""
    try:
        # Parse input
        if isinstance(event.get("body"), str):
            body = json.loads(event["body"])
        else:
            body = event.get("body", event)

        # Crear DataFrame
        df = pd.DataFrame([body])

        # Predecir
        proba = model.predict_proba(df)[0, 1]
        prediction = "churn" if proba >= 0.5 else "no_churn"

        return {
            "statusCode": 200,
            "headers": {"Content-Type": "application/json"},
            "body": json.dumps({
                "churn_probability": round(proba, 4),
                "prediction": prediction,
            })
        }
    except Exception as e:
        return {
            "statusCode": 500,
            "body": json.dumps({"error": str(e)})
        }
```

serverless.yml (Serverless Framework)

```
# serverless.yml
service: bankchurn-predictor

provider:
  name: aws
  runtime: python3.11
  region: us-east-1
  memorySize: 1024
  timeout: 30

functions:
  predict:
    handler: handler.lambda_handler
```

```
    events:
      - http:
          path: predict
          method: post
          cors: true

plugins:
  - serverless-python-requirements

custom:
  pythonRequirements:
    dockerizePip: true
    slim: true
```

12.3 Opción 2: Contenedores Managed (AWS ECS / GCP Cloud Run)

Cuándo Usar

```
✅ USA ECS/CLOUD RUN SI:
• Tráfico medio-alto (1M-100M requests/mes)
• Necesitas latencia consistente
• Modelo de cualquier tamaño
• Quieres balance entre control y simplicidad
• Equipo pequeño de DevOps (2-3 personas)

❌ NO USES SI:
• Necesitas control granular de networking
• Multi-cloud es requisito
• Tráfico extremadamente alto (> 100M)
```

AWS ECS Task Definition

```
{
  "family": "bankchurn-api",
  "networkMode": "awsvpc",
  "requiresCompatibilities": ["FARGATE"],
  "cpu": "512",
  "memory": "1024",
  "containerDefinitions": [
    {
      "name": "api",
      "image": "123456789.dkr.ecr.us-east-1.amazonaws.com/bankchurn:latest",
      "portMappings": [
        {
          "containerPort": 8000,
          "protocol": "tcp"
        }
      ],
      "environment": [
        {"name": "LOG_LEVEL", "value": "INFO"}
      ],
      "healthCheck": {
        "command": ["CMD-SHELL", "curl -f http://localhost:8000/health || exit 1"],
        "interval": 30,
        "timeout": 5,
        "retries": 3
      },
      "logConfiguration": {
        "logDriver": "awslogs",
        "options": {
          "awslogs-group": "/ecs/bankchurn",
          "awslogs-region": "us-east-1",
          "awslogs-stream-prefix": "api"
        }
      }
    }
  ]
}
```

GCP Cloud Run (más simple)

```
# Deploy a Cloud Run
gcloud run deploy bankchurn-api \
```

```
  --image gcr.io/my-project/bankchurn:latest \
  --platform managed \
  --region us-central1 \
  --allow-unauthenticated \
  --memory 1Gi \
  --cpu 1 \
  --min-instances 0 \
  --max-instances 10 \
  --port 8000
```

12.4 Opción 3: Kubernetes

Cuándo Usar

```
✅ USA KUBERNETES SI:
• Tráfico muy alto (> 100M requests/mes)
• Múltiples servicios ML que escalan diferente
• Necesitas GPU para inferencia
• Multi-cloud o hybrid cloud
• Equipo de Ops experimentado (5+ personas)
• Ya tienes inversión en K8s

❌ NO USES SI:
• Un solo modelo simple
• Equipo pequeño sin experiencia K8s
• Presupuesto limitado para Ops
```

Manifiestos Básicos

```
# k8s/deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: bankchurn-api
  labels:
    app: bankchurn-api
spec:
  replicas: 3
  selector:
    matchLabels:
      app: bankchurn-api
  template:
    metadata:
      labels:
        app: bankchurn-api
    spec:
      containers:
      - name: api
        image: ghcr.io/username/bankchurn:latest
        ports:
        - containerPort: 8000
        resources:
          requests:
            memory: "512Mi"
            cpu: "250m"
          limits:
            memory: "1Gi"
            cpu: "500m"
        readinessProbe:
          httpGet:
            path: /health
            port: 8000
          initialDelaySeconds: 10
          periodSeconds: 5
        livenessProbe:
          httpGet:
            path: /health
            port: 8000
          initialDelaySeconds: 15
          periodSeconds: 10
        env:
        - name: LOG_LEVEL
          value: "INFO"
---
```

```
# k8s/service.yaml
apiVersion: v1
kind: Service
metadata:
  name: bankchurn-api
spec:
  selector:
    app: bankchurn-api
  ports:
  - port: 80
    targetPort: 8000
  type: ClusterIP
---
# k8s/hpa.yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: bankchurn-api
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: bankchurn-api
  minReplicas: 2
  maxReplicas: 10
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 70
```

12.5 Análisis de Costos (FinOps)

```
╔═══════════════════════════════════════════════════════════════════════╗
║                    ANÁLISIS DE COSTOS MENSUAL                         ║
╠═══════════════════════════════════════════════════════════════════════╣
║                                                                       ║
║   ESCENARIO: 1M requests/mes, ~1 req/seg promedio                    ║
║                                                                       ║
║   AWS Lambda:                                                        ║
║   • 1M requests × $0.20/1M = $0.20                                   ║
║   • 1M × 200ms × 1GB = 200K GB-s × $0.0000166 = $3.32                ║
║   • Total: ~$4/mes ✅ (bajo tráfico es barato)                        ║
║                                                                       ║
║   ECS Fargate:                                                       ║
║   • 0.5 vCPU × 730h × $0.04 = $14.60                                 ║
║   • 1GB RAM × 730h × $0.004 = $2.92                                  ║
║   • Total: ~$18/mes (consistente)                                    ║
║                                                                       ║
║   EKS (3 nodos t3.small):                                            ║
║   • 3 × $15/mes (EC2) = $45                                          ║
║   • EKS fee: $72/mes                                                 ║
║   • Total: ~$120/mes (overkill para este volumen)                    ║
║                                                                       ║
╠═══════════════════════════════════════════════════════════════════════╣
║                                                                       ║
║   ESCENARIO: 100M requests/mes, ~40 req/seg promedio                 ║
║                                                                       ║
║   AWS Lambda:                                                        ║
║   • 100M × $0.20/1M = $20                                            ║
║   • 100M × 200ms × 1GB = 20M GB-s × $0.0000166 = $332                ║
║   • Total: ~$350/mes (ya no tan barato)                               ║
║                                                                       ║
║   ECS Fargate (auto-scaling):                                        ║
║   • ~5 tareas promedio                                                ║
║   • Total: ~$90/mes ✅                                                ║
║                                                                       ║
║   EKS (auto-scaling):                                                ║
║   • 5 nodos t3.medium promedio                                       ║
║   • Total: ~$200/mes                                                 ║
```

```
╚═══════════════════════════════════════════════════════════════════╝
```

12.6 Decisión para BankChurn

Recomendación por Fase

Fase	Plataforma	Razón
MVP/Desarrollo	Cloud Run o Lambda	Simplicidad, bajo costo inicial
Producción inicial	ECS/Cloud Run	Balance costo-control
Escala enterprise	Kubernetes	Control total, multi-service

ADR para BankChurn

```
╔═══════════════════════════════════════════════════════════════════╗
  ADR-009: Despliegue de BankChurn en Cloud Run
╠═══════════════════════════════════════════════════════════════════╣
                                                                    
  DECISIÓN: Usar Google Cloud Run para el MVP
                                                                    
  RAZONES:
  • Escala a cero cuando no hay tráfico (costo mínimo)
  • Sin gestión de infraestructura
  • Latencia consistente (mejor que Lambda para ML)
  • Soporta contenedores Docker estándar
  • Fácil migración a GKE si necesario
                                                                    
  TRADE-OFFS ACEPTADOS:
  • Vendor lock-in medio (GCP)
  • Menos control que K8s
                                                                    
╚═══════════════════════════════════════════════════════════════════╝
```

🧨 Errores habituales y cómo depurarlos en despliegue ML

En despliegue ML es muy fácil elegir mal la plataforma o romper detalles como puertos, healthchecks o tamaños de imagen.

1) Elegir la plataforma equivocada (costos o latencia inesperados)

Síntomas típicos

- Con Lambda: facturas altas al subir el tráfico o latencias variables por cold starts.
- Con K8s: infraestructura sobredimensionada para un solo modelo simple.

Cómo identificarlo

- Compara tu caso con la **matriz de decisión** del módulo (tráfico, latencia, equipo Ops, presupuesto).

Cómo corregirlo

- Para MVPs y tráfico moderado, prefiere **Cloud Run/ECS** en lugar de K8s.
- Reserva K8s para escenarios enterprise con múltiples servicios y tráfico muy alto.

2) Lambdas que no despliegan o fallan al importar el modelo

Síntomas típicos

- Errores como `Unable to import module 'handler'`.
- Deployment fallido por paquete demasiado grande (> 250 MB).

Cómo identificarlo

- Revisa el tamaño del zip y la estructura de `lambda_function/`.

Cómo corregirlo

- Empaqueta solo lo necesario (`model/`, `src/`, `handler.py`, `requirements.txt`).
- Usa capas o reduce dependencias pesadas si es posible.

3) Contenedores que arrancan pero nunca pasan el healthcheck

Síntomas típicos

- En ECS/Cloud Run/K8s el servicio queda en estado `UNHEALTHY` o se reinicia en bucle.

Cómo identificarlo

- Compara el `healthCheck`/`readinessProbe` con los endpoints reales (`/health`, puerto `8000`).

Cómo corregirlo

- Asegura que tu API expone exactamente el endpoint y puerto que la plataforma espera.
- Ajusta `initialDelaySeconds`/`timeout` si el modelo tarda en cargar.

4) Puertos y rutas inconsistentes entre Docker y la plataforma

Síntomas típicos

- Funciona en `docker run -p 8000:8000` pero falla al desplegar en Cloud Run/ECS.

Cómo identificarlo

- Verifica que el `EXPOSE` del Dockerfile, el puerto del servidor (uvicorn) y el puerto configurado en la plataforma coincidan.

Cómo corregirlo

- Usa un puerto estándar (`8000`) y mantén el mismo valor en Dockerfile y manifiestos.

5) Patrón general de debugging en despliegue ML

1. Verifica primero que la imagen Docker funciona **en local** (`docker run` + `curl /health`).
2. Revisa logs de la plataforma (Lambda logs, Cloud Run logs, ECS/K8s events) para ver errores reales.
3. Comprueba healthchecks, puertos y variables de entorno.
4. Ajusta la plataforma elegida si tus patrones de tráfico o equipo no encajan con la decisión inicial.

Con esta disciplina, pasar de local a producción se vuelve un proceso repetible y menos doloroso.

12.7 Ejercicio: Deploy a Cloud Run

```
# 1. Build imagen
docker build -t gcr.io/my-project/bankchurn:v1 .

# 2. Push a GCR
docker push gcr.io/my-project/bankchurn:v1

# 3. Deploy
gcloud run deploy bankchurn \
  --image gcr.io/my-project/bankchurn:v1 \
  --platform managed \
  --region us-central1 \
  --memory 1Gi \
  --allow-unauthenticated

# 4. Test
curl -X POST https://bankchurn-xxx.run.app/api/v1/predict \
  -H "Content-Type: application/json" \
  -d '{"credit_score": 650, "age": 35, ...}'
```

📋 Operaciones y Runbooks

Estructura de un Runbook ML

```
# Runbook: [Nombre del Servicio]

## Información del Servicio
- **Propósito**: Predicción de churn
- **Owner**: ML Team
- **Criticality**: Tier 2

## Endpoints
| Endpoint | Descripción | SLO |
|----------|-------------|-----|
| /health | Health check | 99.9% |
| /predict | Predicción | p99 < 200ms |
```

```
## Alertas Comunes

### Alta Latencia (> 500ms)
1. Verificar métricas: `kubectl top pods`
2. Revisar logs: `kubectl logs -f deploy/bankchurn-api`
3. Escalar si es necesario: `kubectl scale deploy/bankchurn-api --replicas=5`

### Error Rate > 5%
1. Verificar modelo: ¿Cambió la distribución de inputs?
2. Revisar logs de errores
3. Rollback si es necesario: `kubectl rollout undo deploy/bankchurn-api`

## Procedimientos de Emergencia
- **Rollback**: `make rollback VERSION=v1.0.0`
- **Escalar**: `kubectl scale deploy/bankchurn-api --replicas=10`
- **Deshabilitar**: `kubectl scale deploy/bankchurn-api --replicas=0`
```

SLOs para APIs ML

```
# Ejemplo de SLOs
slos:
  availability:
    target: 99.5%
    window: 30d

  latency:
    p50: 50ms
    p95: 150ms
    p99: 300ms

  error_rate:
    target: < 1%

  prediction_quality:
    accuracy_drift: < 5%  # vs baseline
```

Checklist de Operaciones

- [ ] **Monitoreo activo**: Dashboards y alertas configurados
- [ ] **Runbook documentado**: Procedimientos de respuesta
- [ ] **Rollback probado**: Capacidad de volver a versión anterior
- [ ] **Escalado automático**: HPA configurado
- [ ] **Logs centralizados**: Acceso a logs históricos
- [ ] **On-call definido**: Rotación y escalamiento

📖 Ver ejemplo completo en el Runbook del Portafolio.

💼 Consejos Profesionales

Recomendaciones para destacar en entrevistas y proyectos reales

Para Entrevistas

1. **Deployment Strategies**: Blue-green, canary, rolling. Pros y cons de cada uno.
2. **A/B Testing**: Cómo evaluar modelos en producción con tráfico real.
3. **Rollback**: Siempre ten plan de rollback automático.

Para Proyectos Reales

Situación	Consejo
Nuevo modelo	Canary deployment con 5 % de tráfico
Alta disponibilidad	Múltiples réplicas + load balancer
Modelo grande	Considera serverless o batch serving
Costos	Autoscaling basado en tráfico real

Checklist Pre-Deployment

- [ ] Tests pasando en staging
- [ ] Métricas baseline documentadas
- [ ] Runbook actualizado
- [ ] Rollback probado
- [ ] Alertas configuradas
- [ ] Comunicación a stakeholders

📺 Recursos Externos Recomendados

Ver RECURSOS_POR_MODULO.md para la lista completa.

🏷️	Recurso	Tipo
🔴	Docker Deploy - TechWorld Nana	Video
🟡	Cloud Run Tutorial	Video

🔗 Referencias del Glosario

Ver 21_GLOSARIO.md para definiciones de:
- **Multi-stage Build**: Optimización de imágenes Docker
- **Cloud Run**: Serverless containers de GCP
- **Non-root user**: Seguridad en contenedores

✅ Ejercicios

Ver EJERCICIOS.md - Módulo 17:
- 17.1: Dockerfile multi-stage
- 17.2: Docker Compose para stack ML

➡️ Siguiente Paso

Con la plataforma elegida, es hora de gestionar **infraestructura como código**.

Ir a Módulo 18: Infraestructura como Código →

INFRAESTRUCTURA

GUÍA MLOPS V5.0

PORTFOLIO EDITION

18. Infraestructura como Código

🎯 Objetivo

Conceptos de IaC (Terraform) y orquestación (Kubernetes) para despliegue ML.

> **Nota**: Este módulo es AVANZADO. Para el portafolio actual, Docker + GitHub Actions es suficiente.

Terraform Básico

Concepto

```
╔═══════════════════════════════════════════════════════════════════╗
║  TERRAFORM = Definir infraestructura en código                    ║
║                                                                   ║
║  En lugar de:                                                     ║
║  "Crear una instancia EC2 manualmente en la consola AWS"          ║
║                                                                   ║
║  Escribes:                                                        ║
║  resource "aws_instance" "ml_server" {                            ║
║    ami           = "ami-12345"                                    ║
║    instance_type = "t3.medium"                                    ║
║  }                                                                ║
║                                                                   ║
║  Beneficios:                                                      ║
║  • Reproducible                                                   ║
║  • Versionado en Git                                              ║
║  • Auditado                                                       ║
║  • Destruir y recrear fácilmente                                  ║
╚═══════════════════════════════════════════════════════════════════╝
```

Estructura Típica

```
# main.tf

terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

provider "aws" {
  region = var.region
}

# ECS para ML API
resource "aws_ecs_cluster" "ml_cluster" {
  name = "ml-portfolio-cluster"
}

resource "aws_ecs_service" "bankchurn_api" {
  name            = "bankchurn-api"
  cluster         = aws_ecs_cluster.ml_cluster.id
  task_definition = aws_ecs_task_definition.bankchurn.arn
  desired_count   = 2

  load_balancer {
    target_group_arn = aws_lb_target_group.bankchurn.arn
    container_name   = "bankchurn"
    container_port   = 8000
  }
}
```

Kubernetes Básico

Concepto

```
╔═══════════════════════════════════════════════════════════════════╗
║  KUBERNETES = Orquestar contenedores a escala                     ║
║                                                                   ║
```

```
│  Pod: Un contenedor corriendo                                          │
│  Deployment: N réplicas de un Pod                                      │
│  Service: Exponer Pods a la red                                        │
│  Ingress: Routing HTTP externo                                         │
│                                                                        │
│  Para ML:                                                              │
│  • Deployment para API de inferencia                                   │
│  • HPA (Horizontal Pod Autoscaler) para escalar con carga              │
│  • Secrets para API keys y credenciales                                │
└────────────────────────────────────────────────────────────────────────┘
```

Deployment YAML

```yaml
# k8s/deployment.yaml

apiVersion: apps/v1
kind: Deployment
metadata:
  name: bankchurn-api
  labels:
    app: bankchurn
spec:
  replicas: 2
  selector:
    matchLabels:
      app: bankchurn
  template:
    metadata:
      labels:
        app: bankchurn
    spec:
      containers:
      - name: bankchurn
        image: ghcr.io/user/bankchurn:latest
        ports:
        - containerPort: 8000
        resources:
          requests:
            memory: "256Mi"
            cpu: "250m"
          limits:
            memory: "512Mi"
            cpu: "500m"
        livenessProbe:
          httpGet:
            path: /health
            port: 8000
          initialDelaySeconds: 30
          periodSeconds: 10
        env:
        - name: MLFLOW_TRACKING_URI
          valueFrom:
            secretKeyRef:
              name: ml-secrets
              key: mlflow-uri
---
apiVersion: v1
kind: Service
metadata:
  name: bankchurn-service
spec:
  selector:
    app: bankchurn
  ports:
  - port: 80
    targetPort: 8000
  type: LoadBalancer
```

¿Cuándo Usar Qué?

Escenario	Solución Recomendada
Proyecto personal/demo	Docker + docker-compose
Startup pequeña	ECS Fargate o Cloud Run
Empresa mediana	EKS/GKE con Terraform
Enterprise	Full K8s + GitOps (ArgoCD)

Para Este Portafolio

Docker + GitHub Actions es suficiente.

Terraform y K8s son skills valiosos, pero no necesarios para demostrar competencia MLOps en proyectos de portafolio.

🧨 Errores habituales y cómo depurarlos en Infraestructura como Código

Aunque este módulo es avanzado, es común cometer errores que dejan tu IaC frágil o inconsistente.

1) Terraform aplicado "a mano" sin estado controlado

Síntomas típicos

- Se ejecuta `terraform apply` desde distintas máquinas sin control del `terraform.tfstate`.
- Recursos que aparecen duplicados o que se destruyen sin querer.

Cómo identificarlo

- Verifica dónde se guarda el estado: local vs backend remoto (S3, GCS, etc.).

Cómo corregirlo

- Para proyectos serios, usa un **backend remoto** para el estado y controla quién puede aplicar cambios.

2) Manifiestos de K8s que funcionan en minikube pero no en cloud

Síntomas típicos

- Deployment correcto en local, pero en EKS/GKE los Pods quedan `CrashLoopBackOff` o `ImagePullBackOff`.

Cómo identificarlo

- Revisa la imagen referenciada (`image:`) y las credenciales de registry.

Cómo corregirlo

- Asegura que la imagen esté en un registry accesible desde el cluster (ECR/GCR/GHCR) y que el cluster tenga permisos para leerla.

3) Resources/limits mal configurados en K8s

Síntomas típicos

- Pods que se matan por OOMKilled o throttling excesivo de CPU.

Cómo identificarlo

- Observa eventos del Pod y métricas de consumo real.

Cómo corregirlo

- Ajusta `requests` y `limits` según el perfil real de uso de tu API ML, empezando conservador y ajustando con métricas.

4) ¿Cuándo escalar más allá de Docker?

Síntomas típicos

- Intentar introducir Terraform/K8s en un proyecto de portafolio cuando aún no dominas Docker + CI/CD.

Cómo identificarlo

- Si todavía no tienes un flujo sólido con Docker + GitHub Actions, probablemente es pronto para meter K8s.

Cómo corregirlo

- Sigue la recomendación del módulo: primero domina Docker + CI/CD. Usa IaC/K8s solo si tu contexto profesional lo exige.

5) Patrón general de debugging en IaC

1. Aplica primero en entornos de prueba pequeños (playgrounds, sandbox).
2. Revisa siempre el **plan** (`terraform plan`, `kubectl diff`) antes de aplicar.
3. Usa métricas y eventos del cluster para ajustar configuración en lugar de adivinar.

Con este enfoque, IaC y K8s se vuelven herramientas que suman, no otra fuente de problemas.

Horizontal Pod Autoscaler (HPA)

El HPA escala automáticamente los pods basándose en métricas como CPU o memoria.

```
# k8s/hpa.yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: bankchurn-hpa
  namespace: mlops
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: bankchurn-api
  minReplicas: 2
  maxReplicas: 10
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 70
  behavior:
    scaleDown:
      stabilizationWindowSeconds: 300  # Esperar 5 min antes de escalar abajo
    scaleUp:
      stabilizationWindowSeconds: 0    # Escalar arriba inmediatamente
```

¿Por qué 70% CPU? Es un balance entre eficiencia (no desperdiciar recursos) y capacidad de respuesta (tener margen para picos).

ConfigMaps y Secrets

ConfigMap (configuración no sensible)

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: bankchurn-config
  namespace: mlops
data:
  LOG_LEVEL: "INFO"
  MODEL_PATH: "/app/artifacts/model.joblib"
  MLFLOW_TRACKING_URI: "http://mlflow-service:5000"
```

Secret (ejemplo didáctico, no usar en producción)

```
apiVersion: v1
kind: Secret
metadata:
  name: ml-secrets
  namespace: mlops
type: Opaque
data:
  # Valores de ejemplo. En un entorno real se inyectan desde el sistema de secretos.
  database-password: REEMPLAZAR_EN_ENTORNO_REAL
  api-key: REEMPLAZAR_EN_ENTORNO_REAL
```

Uso en Deployment

```
spec:
  containers:
  - name: bankchurn
```

```
    envFrom:
    - configMapRef:
        name: bankchurn-config
    - secretRef:
        name: ml-secrets
```

Ingress para Routing HTTP

```
# k8s/ingress.yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: mlops-ingress
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
spec:
  ingressClassName: nginx
  rules:
  - host: api.mlops.example.com
    http:
      paths:
      - path: /bankchurn
        pathType: Prefix
        backend:
          service:
            name: bankchurn-service
            port:
              number: 80
      - path: /carvision
        pathType: Prefix
        backend:
          service:
            name: carvision-service
            port:
              number: 80
```

📦 Cómo se usó en el Portafolio

El directorio `k8s/` del portafolio contiene 8 manifests production-ready:

Archivo	Propósito
`namespace.yaml`	Namespace `mlops` aislado
`bankchurn-deployment.yaml`	Deployment + Service + HPA
`carvision-deployment.yaml`	Deployment + Service
`telecom-deployment.yaml`	Deployment + Service
`prometheus-deployment.yaml`	Monitoreo
`grafana-deployment.yaml`	Dashboards
`ingress.yaml`	Routing HTTP
`storage.yaml`	PersistentVolumeClaims

Comandos útiles:

```
# Aplicar todos los manifests
kubectl apply -f k8s/

# Ver estado de pods
kubectl get pods -n mlops

# Ver logs de un pod
kubectl logs -f deployment/bankchurn-api -n mlops

# Escalar manualmente (si no usas HPA)
kubectl scale deployment bankchurn-api --replicas=3 -n mlops
```

```
# Port-forward para testing local
kubectl port-forward svc/bankchurn-service 8001:80 -n mlops
```

💼 Consejos Profesionales

Recomendaciones para destacar en entrevistas y proyectos reales

Para Entrevistas

1. **IaC (Infrastructure as Code)**: Por qué Terraform/Pulumi sobre click-ops.
2. **Kubernetes basics**: Pods, Deployments, Services, ConfigMaps.
3. **Cloud agnostic**: Diseña para portabilidad cuando sea posible.

Para Proyectos Reales

Situación	Consejo
Multi-environment	Usa Terraform workspaces o directorios
Secrets	External Secrets Operator o cloud-native solutions
Costos	Tagging obligatorio para cost allocation
DR (Disaster Recovery)	Documenta y prueba regularmente

Stack Recomendado

```
IaC:        Terraform + Terragrunt
Containers: Docker + Kubernetes
CI/CD:      GitHub Actions + ArgoCD
Secrets:    Vault o AWS Secrets Manager
Monitoring: Prometheus + Grafana
```

📺 Recursos Externos Recomendados

Ver RECURSOS_POR_MODULO.md para la lista completa.

🏷️	Recurso	Tipo	Duración
🔴	Kubernetes Tutorial - TechWorld Nana	Video	4 h
🟡	Terraform Tutorial - freeCodeCamp	Video	2.5 h
🟢	Kubernetes Fundamentals - LF	Curso	3.5 h

🔗 Referencias del Glosario

Ver 21_GLOSARIO.md para definiciones detalladas de:
- **Kubernetes**: Orquestación de contenedores
- **HPA**: Horizontal Pod Autoscaler
- **ConfigMap/Secret**: Configuración en K8s
- **Terraform**: Infrastructure as Code

✅ Checkpoint

Para este nivel:
- [ ] Entiendes el concepto de IaC (infraestructura como código)
- [ ] Puedes leer un deployment.yaml de K8s
- [ ] Sabes qué hace un HPA y cuándo usarlo
- [ ] Entiendes la diferencia entre ConfigMap y Secret
- [ ] Sabes cuándo escalar más allá de Docker

Ejercicios: Ver EJERCICIOS.md - Módulos 17-18

DOCUMENTACIÓN

GUÍA MLOPS V5.0

PORTFOLIO EDITION

MÓDULO 16: DOCUMENTACIÓN Y ÉTICA

MkDocs, Model Cards y Responsible AI

Guía MLOps v5.0: Senior Edition | DuqueOM | Noviembre 2025

📚 MÓDULO 16: Documentación y Ética

Tu Trabajo No Existe Si No Está Documentado

"La documentación es el regalo que le haces a tu yo del futuro."

Duración	Teoría	Práctica
4 - 5 horas	30%	70%

🎯 Lo Que Lograrás

1. **Crear** documentación técnica con MkDocs
2. **Escribir** Model Cards profesionales
3. **Implementar** prácticas de Responsible AI
4. **Publicar** docs en GitHub Pages

16.1 MkDocs con Material Theme

Estructura de Docs

```
docs/
├── index.md                # Home
├── getting-started/
│   ├── installation.md
│   ├── quickstart.md
│   └── configuration.md
├── architecture/
│   ├── overview.md
│   ├── data-flow.md
│   └── decisions.md
├── api/
│   ├── endpoints.md
│   └── schemas.md
├── development/
│   ├── contributing.md
│   └── testing.md
└── model/
    └── model-card.md

mkdocs.yml                  # Configuración
```

mkdocs.yml

```
site_name: BankChurn Predictor
site_description: API para predicción de churn bancario
site_author: Tu Nombre
site_url: https://username.github.io/bankchurn

theme:
  name: material
  language: es
  palette:
    - scheme: default
      primary: indigo
      accent: indigo
      toggle:
        icon: material/brightness-7
        name: Cambiar a modo oscuro
    - scheme: slate
      primary: indigo
      accent: indigo
      toggle:
        icon: material/brightness-4
        name: Cambiar a modo claro
```

```
  features:
    - navigation.tabs
    - navigation.sections
    - navigation.expand
    - search.suggest
    - content.code.copy
    - content.tabs.link

nav:
  - Home: index.md
  - Getting Started:
    - Instalación: getting-started/installation.md
    - Quick Start: getting-started/quickstart.md
    - Configuración: getting-started/configuration.md
  - Arquitectura:
    - Overview: architecture/overview.md
    - Flujo de Datos: architecture/data-flow.md
    - Decisiones: architecture/decisions.md
  - API Reference:
    - Endpoints: api/endpoints.md
    - Schemas: api/schemas.md
  - Desarrollo:
    - Contribuir: development/contributing.md
    - Testing: development/testing.md
  - Model Card: model/model-card.md

markdown_extensions:
  - pymdownx.highlight:
      anchor_linenums: true
  - pymdownx.superfences:
      custom_fences:
        - name: mermaid
          class: mermaid
          format: !!python/name:pymdownx.superfences.fence_code_format
  - pymdownx.tabbed:
      alternate_style: true
  - admonition
  - pymdownx.details
  - attr_list
  - md_in_html
  - tables

plugins:
  - search
  - mkdocstrings:
      handlers:
        python:
          options:
            show_source: true

extra:
  social:
    - icon: fontawesome/brands/github
      link: https://github.com/username/bankchurn
```

Comandos MkDocs

```
# Instalar
pip install mkdocs mkdocs-material mkdocstrings[python]

# Desarrollo local
mkdocs serve

# Build
mkdocs build

# Deploy a GitHub Pages
mkdocs gh-deploy
```

16.2 Model Card Profesional

```
# Model Card: BankChurn Predictor
```

```
## Información del Modelo

| Campo | Valor |
|-------|-------|
| **Nombre** | BankChurn Predictor |
| **Versión** | 1.2.3 |
| **Tipo** | Clasificación Binaria |
| **Framework** | Scikit-learn 1.3.0 |
| **Fecha de Entrenamiento** | 2024-01-15 |
| **Autor** | Tu Nombre |

## Propósito

### Uso Previsto
- **Caso de uso principal**: Identificar clientes con alta probabilidad de abandonar el banco
- **Usuarios objetivo**: Equipo de Retención de Clientes
- **Decisiones habilitadas**: Campañas de retención personalizadas

### Uso No Previsto
- ❌ No usar para decisiones crediticias
- ❌ No usar como único criterio para cancelar servicios
- ❌ No usar en mercados fuera de Europa (entrenado solo con datos de FR/DE/ES)

## Datos de Entrenamiento

### Dataset
- **Fuente**: Sistema CRM interno
- **Período**: 2022-01-01 a 2023-12-31
- **Tamaño**: 10,000 registros
- **Split**: 80% train, 20% test (estratificado)

### Features
| Feature | Tipo | Descripción |
|---------|------|-------------|
| CreditScore | Numérica | Score crediticio (300-850) |
| Age | Numérica | Edad del cliente |
| Geography | Categórica | País (France, Germany, Spain) |
| ... | ... | ... |

### Distribución del Target
- **Churn (1)**: 20%
- **No Churn (0)**: 80%
- **Estrategia**: class_weight='balanced'

## Métricas de Performance

### Métricas Globales
| Métrica | Train | Test | Threshold |
|---------|-------|------|-----------|
| AUC-ROC | 0.89 | 0.87 | > 0.85 ✅ |
| Precision | 0.72 | 0.68 | > 0.60 ✅ |
| Recall | 0.78 | 0.74 | > 0.70 ✅ |
| F1 | 0.75 | 0.71 | > 0.65 ✅ |

### Métricas por Subgrupo (Fairness)
| Subgrupo | AUC-ROC | Precision | Recall |
|----------|---------|-----------|--------|
| Gender: Male | 0.86 | 0.67 | 0.73 |
| Gender: Female | 0.88 | 0.69 | 0.75 |
| Geography: France | 0.87 | 0.68 | 0.74 |
| Geography: Germany | 0.85 | 0.66 | 0.72 |
| Geography: Spain | 0.88 | 0.70 | 0.76 |

**Nota**: La diferencia máxima de AUC entre subgrupos es 0.03 (< 0.05 threshold).

## Limitaciones

### Limitaciones Conocidas
1. **Temporal**: Modelo entrenado con datos hasta 2023. Puede degradarse con cambios económicos.
2. **Geográfico**: Solo válido para Francia, Alemania y España.
3. **Demográfico**: Menos preciso para clientes < 25 años (pocos datos).

### Cuándo NO Usar
- Datos con > 30% de valores faltantes
- Clientes corporativos (solo entrenado con personas físicas)
```

```
- Períodos de crisis económica (cambio de distribución)

## Consideraciones Éticas

### Fairness
- Se monitorean métricas por género y geografía
- Diferencias de performance < 5% entre grupos
- No se usan features protegidas directamente (pero Geography correlaciona con cultura)

### Privacidad
- Datos pseudonimizados (no PII en features)
- Cumple con GDPR (Art. 22 - derecho a explicación)
- Retención de datos: 24 meses

### Transparencia
- SHAP values disponibles para explicabilidad
- Documentación de limitaciones pública
- Proceso de feedback habilitado

## Mantenimiento

### Monitoreo
- Data drift monitoreado diariamente (Evidently)
- Alerta si drift > 10%
- Performance evaluada mensualmente con ground truth

### Retraining
- **Frecuencia**: Trimestral o si drift detectado
- **Proceso**: Automatizado vía GitHub Actions
- **Aprobación**: Requiere validación de Data Science Lead

## Historial de Versiones

| Versión | Fecha | Cambios | AUC |
|---------|-------|---------|-----|
| 1.0.0 | 2023-06-01 | Versión inicial | 0.82 |
| 1.1.0 | 2023-09-01 | Feature engineering | 0.85 |
| 1.2.0 | 2024-01-01 | Retraining con datos 2023 | 0.87 |
| 1.2.3 | 2024-01-15 | Fix en preprocessing | 0.87 |

## Contacto

- **Responsable**: tu.email@company.com
- **Equipo**: ML Platform Team
- **Escalación**: data-ethics@company.com
```

16.3 Responsible AI Checklist

```
╔═══════════════════════════════════════════════════════════════════════╗
║                    CHECKLIST DE RESPONSIBLE AI                        ║
╟═══════════════════════════════════════════════════════════════════════╢
║                                                                       ║
║   FAIRNESS:                                                          ║
║   [ ] Métricas calculadas por subgrupos demográficos                 ║
║   [ ] Diferencias de performance < 5% entre grupos                   ║
║   [ ] Features sensibles identificadas y documentadas                ║
║   [ ] Estrategia de mitigación si hay sesgo                          ║
║                                                                       ║
║   TRANSPARENCIA:                                                     ║
║   [ ] Model Card completo y público                                   ║
║   [ ] Explicabilidad disponible (SHAP/LIME)                           ║
║   [ ] Limitaciones claramente documentadas                            ║
║   [ ] Usuarios saben que interactúan con ML                           ║
║                                                                       ║
║   PRIVACIDAD:                                                        ║
║   [ ] No PII en features                                              ║
║   [ ] Cumplimiento GDPR/CCPA documentado                              ║
║   [ ] Política de retención de datos                                  ║
║   [ ] Proceso de eliminación de datos                                 ║
║                                                                       ║
║   ACCOUNTABILITY:                                                    ║
║   [ ] Responsable del modelo identificado                             ║
║   [ ] Proceso de escalación definido                                  ║
║   [ ] Auditoría periódica programada                                  ║
```

```
│   [ ] Canal de feedback para usuarios                                    │
│                                                                          │
└──────────────────────────────────────────────────────────────────────────┘
```

16.4 README Profesional

```
# 🏦 BankChurn Predictor

[![CI](https://github.com/username/bankchurn/actions/workflows/ci.yml/badge.svg)](https://github.com/username/bankchurn/
actions)
[![Coverage](https://codecov.io/gh/username/bankchurn/branch/main/graph/badge.svg)](https://codecov.io/gh/username/
bankchurn)
[![License](https://img.shields.io/badge/License-MIT-blue.svg)](LICENSE)
[![Python](https://img.shields.io/badge/Python-3.10+-blue.svg)](https://python.org)

> API para predicción de churn bancario con MLOps completo.

## 🚀 Quick Start

```bash

# Clonar
git clone https://github.com/username/bankchurn.git
cd bankchurn

# Instalar
pip install -e ".[dev]"

# Ejecutar tests
pytest

# Iniciar API
uvicorn app.main:app --reload
```

📖 Documentation

- Documentación Completa
- API Reference
- Model Card

🏗️ Architecture

```
flowchart LR
    Client --> API --> Model --> Response
```

📊 Metrics

Metric	Value
AUC-ROC	0.87
Latency P99	45ms
Coverage	85%

📜 License

MIT © Tu Nombre

```
## 🧨 Errores habituales y cómo depurarlos en documentación ML

La documentación suele quedarse para el final, y eso genera READMEs desactualizados y Model Cards incompletas.

### 1) README que no refleja el estado real del proyecto

**Síntomas típicos**

- Instrucciones de instalación que no funcionan.
```
```

```
- Comandos de entrenamiento/serve diferentes a los reales.

**Cómo identificarlo**

- Intenta seguir tu propio `Quick Start` desde cero en una máquina limpia.

**Cómo corregirlo**

- Actualiza el README cada vez que cambies la CLI, el Makefile o los entornos.
- Copia los comandos reales desde tu `Makefile` o scripts, no los escribas de memoria.


### 2) Model Card incompleta o decorativa

**Síntomas típicos**

- Hay secciones de plantilla sin rellenar o con texto genérico.
- No hay detalles de datos, métricas por subgrupo ni limitaciones claras.

**Cómo identificarlo**

- Compara tu Model Card con el ejemplo de este módulo: ¿faltan tablas clave o secciones enteras?

**Cómo corregirlo**

- Completa al menos: propósito, datos, métricas principales, métricas por subgrupos, limitaciones y plan de
mantenimiento.


### 3) MkDocs que compila pero no se integra en el flujo

**Síntomas típicos**

- `mkdocs serve` funciona, pero nadie sabe la URL de docs en el README o en el repo.

**Cómo identificarlo**

- Revisa si tu README enlaza a la documentación generada.

**Cómo corregirlo**

- Añade enlaces claros en el README (sección Documentation) y en la descripción del repositorio.


### 4) Responsible AI checklist ignorada

**Síntomas típicos**

- El checklist de Responsible AI está en el repo, pero nunca se usa en revisiones.

**Cómo identificarlo**

- Pregunta: ¿se revisan fairness, privacidad y accountability en los PRs importantes?

**Cómo corregirlo**

- Integra partes del checklist en tu proceso de revisión (por ejemplo, una sección en la PR template).


### 5) Patrón general de debugging en documentación

1. Usa tu propia documentación como si fueras un usuario nuevo (installation, quick start).
2. Mantén un lugar único de verdad para comandos y rutas (Makefile, docs técnicas) y enlázalo desde el README.
3. Considera la Model Card como parte del contrato del modelo, no como adorno.

Con esta mentalidad, tu documentación pasa de ser un "nice to have" a convertirse en una parte crítica de la calidad de
tu sistema ML.


## 16.5 Ejercicio: Crea Tu Documentación

### Checklist
```

MKDOCS:
[ ] mkdocs.yml configurado
[ ] Home page con overview
[ ] Getting started completo
[ ] API documentada

MODEL CARD:
[ ] Información del modelo
[ ] Datos de entrenamiento
[ ] Métricas de performance
[ ] Limitaciones y ética

README:
[ ] Badges de CI/Coverage
[ ] Quick Start
[ ] Links a docs
[ ] Arquitectura visual
```

## 📹 Material Audiovisual

Para crear demos profesionales (GIFs, screenshots, videos) de tu documentación y portafolio, consulta:

**→ Guía Audiovisual Completa**

Incluye:
- Cómo grabar GIFs demostrativos de APIs y dashboards
- Screenshots profesionales para README
- Video de 5 minutos explicando el portafolio
- Scripts y comandos para levantar el stack demo

## 💼 Consejos Profesionales

**Recomendaciones para destacar en entrevistas y proyectos reales**

### Para Entrevistas

1. **Model Cards**: Explica por qué documentar limitaciones y sesgos es crítico.
2. **Documentation as Code**: Docs versionados junto al código.
3. **Audience-aware**: Diferentes docs para diferentes audiencias.

### Para Proyectos Reales

| Documento | Audiencia | Contenido |
|---|---|---|
| README.md | Todos | Quick start, overview |
| Model Card | ML team, stakeholders | Métricas, limitaciones, ética |
| API Docs | Developers | Endpoints, schemas, ejemplos |
| Runbook | Ops | Troubleshooting, alertas |

### Documentación que Diferencia

- **ADRs**: Decisiones arquitectónicas con contexto
- **Changelogs**: Generados automáticamente desde commits
- **Diagramas**: Mermaid/PlantUML versionados
- **Ejemplos**: Notebooks con casos de uso reales

## 📺 Recursos Externos Recomendados

Ver RECURSOS_POR_MODULO.md para la lista completa.

| 🏷️ | Recurso | Tipo |
|---|---|---|
| 🔴 | Model Cards - Google | Documentación |
```

🏷️	Recurso	Tipo
🟡	MkDocs Tutorial	Video

🔗 Referencias del Glosario

Ver 21_GLOSARIO.md para definiciones de:
- **Model Card**: Documentación estandarizada de modelos
- **ADR**: Architecture Decision Record
- **MkDocs**: Generador de documentación

📋 Plantillas Relacionadas

Ver templates/ para plantillas listas:
- model_card_template.md — Documentación completa de modelos
- dataset_card_template.md — Documentación de datasets

🎬 Material Audiovisual

Documentar tu trabajo también incluye crear demos visuales profesionales.

> 📺 **Ver GUIA_AUDIOVISUAL.md** para:
> - Crear GIFs demostrativos de cada proyecto
> - Capturar screenshots profesionales
> - Producir video principal del portafolio
> - Scripts y comandos para demos

Material	Propósito
GIFs (3-5 por proyecto)	README, documentación rápida
Screenshots	Issues, PRs, documentación
Video principal (5-7 min)	LinkedIn, presentaciones

✅ Ejercicios

Ver EJERCICIOS.md - Módulo 19:
- 19.1: Crear Model Card
- 19.2: Crear Dataset Card

➡️ Siguiente Paso

Con documentación lista, es hora del **Proyecto Integrador**.

Ir a Módulo 20: Proyecto Integrador →

PROYECTO INTEGRADOR

GUÍA MLOPS V5.0

PORTFOLIO EDITION

20. Proyecto Integrador

🎯 Objetivo

Construir un proyecto ML completo desde cero, aplicando TODO lo aprendido.

```
╔═══════════════════════════════════════════════════════════════╗
║                                                               ║
║  🏆 EL RETO FINAL                                             ║
║                                                               ║
║  Has aprendido los conceptos. Has estudiado el código del portafolio.  ║
║  Ahora es momento de DEMOSTRAR que puedes construirlo desde cero.      ║
║                                                               ║
║  TIEMPO: 1-2 semanas                                          ║
║  RESULTADO: Un 4to proyecto digno del portafolio               ║
║                                                               ║
╚═══════════════════════════════════════════════════════════════╝
```

📋 El Proyecto: Sistema de Recomendación de Planes

Contexto: Una empresa de telecomunicaciones quiere recomendar planes móviles basándose en el comportamiento del usuario.

Dataset sugerido: Telecom Users Dataset o similar.

✅ Checklist de Entrega (100 puntos)

Fase 1: Estructura y Configuración (20 puntos)

Requisito	Puntos	Archivo
Estructura src/ layout	3	`src/planrec/`
pyproject.toml completo	3	`pyproject.toml`
Makefile con comandos básicos	2	`Makefile`
Config Pydantic con validación	4	`src/planrec/config.py`
Config YAML externo	2	`configs/config.yaml`
.gitignore apropiado	2	`.gitignore`
README profesional	4	`README.md`

Fase 2: Pipeline ML (25 puntos)

Requisito	Puntos	Archivo
Carga y validación de datos	3	`src/planrec/data.py`
Feature Engineering como Transformer	5	`src/planrec/features.py`
sklearn Pipeline unificado	5	`src/planrec/training.py`
Cross-validation estratificada	3	`src/planrec/training.py`
Métricas apropiadas (F 1 , AUC)	3	`src/planrec/evaluation.py`
Guardado de artefactos	3	`artifacts/`
Prevención de data leakage	3	`drop_columns` en config

Fase 3: Testing (20 puntos)

Requisito	Puntos	Archivo
conftest.py con fixtures	4	`tests/conftest.py`

Requisito	Puntos	Archivo
Tests unitarios (features)	4	`tests/test_features.py`
Tests de datos	3	`tests/test_data.py`
Tests de modelo	3	`tests/test_model.py`
Tests de integración	3	`tests/test_training.py`
Coverage ≥ 80%	3	`pytest --cov`

Fase 4: API y Serving (15 puntos)

Requisito	Puntos	Archivo
FastAPI con Pydantic schemas	4	`app/fastapi_app.py`
Endpoint /health	2	
Endpoint /predict	4	
Dockerfile multi-stage	3	`Dockerfile`
Non-root user	2	

Fase 5: CI/CD y Calidad (15 puntos)

Requisito	Puntos	Archivo
GitHub Actions workflow	5	`.github/workflows/ci.yml`
Tests automáticos	3	
Coverage enforcement	3	
Linting (ruff/black)	2	
Pre-commit hooks	2	`.pre-commit-config.yaml`

Fase 6: Documentación (5 puntos)

Requisito	Puntos	Archivo
Model Card	3	`docs/model_card.md`
Data Card	2	`docs/data_card.md`

📝 Plantilla de README

```
# 📱 PlanRec: Mobile Plan Recommender

[![CI](https://github.com/USER/planrec/actions/workflows/ci.yml/badge.svg)](...)
[![Coverage](https://img.shields.io/badge/Coverage-85%25-brightgreen)](...)
[![Python](https://img.shields.io/badge/Python-3.11-blue)](...)

> Sistema de recomendación de planes móviles basado en comportamiento de usuarios.

## 🎯 Resumen del Proyecto

| Métrica | Valor |
|---------|-------|
| **Accuracy** | 85% |
| **F1-Score** | 0.82 |
| **Coverage** | 85% |
```

```
## 🚀 Quick Start

\`\`\`bash
# Instalar
pip install -e ".[dev]"

# Entrenar
make train

# Servir API
make serve

# Tests
make test
\`\`\`

## 📁 Estructura

\`\`\`
planrec/
├── src/planrec/       # Código fuente
├── app/               # FastAPI
├── tests/             # Tests
├── configs/           # Configuración
└── artifacts/         # Modelos (gitignored)
\`\`\`

## 📊 Arquitectura

[Diagrama de arquitectura]

## 🛠️ Stack Tecnológico

- **ML**: scikit-learn, pandas, numpy
- **API**: FastAPI, uvicorn
- **Config**: Pydantic, PyYAML
- **Testing**: pytest, pytest-cov
- **CI/CD**: GitHub Actions
- **Container**: Docker

## 📖 Documentación

- [Model Card](#mod_model_card)
- [Data Card](#mod_data_card)
```

🎯 Rúbrica de Evaluación

Nivel Junior (50-69 puntos)

- Funciona pero con estructura básica
- Tests mínimos
- Sin CI/CD

Nivel Mid (70-84 puntos)

- Estructura correcta
- Tests con coverage > 70%
- CI básico

Nivel Senior (85-94 puntos)

- Custom Transformer funcionando
- Coverage > 80%
- CI/CD completo
- Documentación profesional

Nivel Staff (95-100 puntos)

- Todo lo anterior
- Drift detection
- MLflow integration
- Model Card completo
- Code review pasable en FAANG

🧨 Errores habituales y cómo depurarlos en el Proyecto Integrador

En el proyecto integrador el mayor reto no es una tecnología concreta, sino **coordinar todas las piezas** sin romper nada en el camino.

1) Empezar por el modelo y olvidar la estructura

Síntomas típicos

- Tienes notebooks y scripts sueltos, pero no un paquete `src/planrec` ni `pyproject.toml` claros.
- Es difícil correr el proyecto en otra máquina o en CI.

Cómo identificarlo

- Pregúntate: ¿puedo ejecutar `pip install -e .` y luego `python -m planrec.cli` o similar?

Cómo corregirlo

- Copia la estructura de BankChurn/CarVision: `src/`, `configs/`, `app/`, `tests/`, `artifacts/`.
- Define desde el inicio `pyproject.toml`, `Makefile` y `.gitignore`.

2) Config dispersa o duplicada

Síntomas típicos

- Rutas de datos, thresholds o hiperparámetros hardcodeados en varios archivos.
- Cambias algo en un sitio y se rompe otra parte.

Cómo identificarlo

- Busca valores repetidos (por ejemplo, paths o columnas) en múltiples módulos.

Cómo corregirlo

- Centraliza configuración en `configs/config.yaml` y una clase Pydantic (`Config`) que valide todo.
- Haz que training, API y scripts lean SIEMPRE desde esa fuente de verdad.

3) Tests que no cubren el flujo completo

Síntomas típicos

- Coverage aceptable, pero sin tests de integración ni de API.
- El pipeline entero falla cuando intentas ejecutar `make train` o el endpoint `/predict`.

Cómo identificarlo

- Revisa si tienes al menos:
- Tests de features (`test_features.py`).
- Tests de datos (`test_data.py`).
- Tests de entrenamiento/integración (`test_training.py`).

Cómo corregirlo

- Añade al menos un test que recorra el flujo E 2 E con datos pequeños, similar a los de CarVision.
- Usa fixtures y `tmp_path` para no depender de rutas reales.

4) CI/CD que solo corre en local

Síntomas típicos

- Tienes un archivo `.github/workflows/ci.yml` pero los jobs fallan siempre en GitHub.

Cómo identificarlo

- Compara el workflow con el del portafolio: ¿coinciden `working-directory`, versiones de Python y comandos?

Cómo corregirlo

- Simplifica primero: un job que haga `pip install -e .` y `pytest`.
- Añade coverage y linting cuando el flujo básico sea estable.

5) Patrón general de debugging del proyecto integrador

1 . Valida la **base**: estructura, instalación (`pip install -e .`), `make test`.
2 . Asegúrate de que el **pipeline de training** funciona de principio a fin con datos pequeños.
3 . Solo entonces añade API, Docker y CI/CD, verificando cada capa con su propio conjunto de tests.

Con este enfoque, reduces la frustración y aumentas la probabilidad de tener un 4 º **proyecto sólido de portafolio**.

💡 Tips para Éxito

1. **Empieza por la estructura** - No escribas código sin tener pyproject.toml y Makefile
2. **Tests primero** - TDD te ahorra tiempo a largo plazo
3. **Commits pequeños** - Un commit por feature, mensajes claros
4. **README actualizado** - Actualízalo mientras avanzas, no al final
5. **Copia patrones** - Usa el código de BankChurn/CarVision como referencia

💼 Consejos Profesionales

Recomendaciones para destacar en entrevistas y proyectos reales

Para Entrevistas

1. **Cuenta una historia**: Tu portafolio debe mostrar progresión y aprendizaje.
2. **Explica decisiones**: "¿Por qué elegiste X?" es la pregunta más común.
3. **Muestra métricas**: Impacto cuantificable impresiona más que features.

Para tu Portafolio

Elemento	Por qué Importa
README profesional	Primera impresión, 30 segundos para captar atención
Demo en vivo	Muestra que funciona, no solo que existe
Código limpio	Los revisores leen tu código
Documentación	Demuestra comunicación técnica

Checklist Final del Portafolio

- [ ] Cada proyecto tiene problema claro y solución
- [ ] Métricas de performance documentadas
- [ ] CI/CD funcionando con badges
- [ ] Docker para reproducibilidad
- [ ] README con GIFs o screenshots
- [ ] Deployed y accesible (demo link)

📺 Recursos Externos Recomendados

Ver RECURSOS_POR_MODULO.md para la lista completa.

🏷️	Recurso	Tipo
🔴	End-to-End ML Project - Krish Naik	Video
🟡	MLOps Zoomcamp - DataTalks	Curso

🔗 Referencias del Glosario

Ver 21_GLOSARIO.md para definiciones de:
- **E2E Pipeline**: Flujo completo de datos a predicción
- **Integration Test**: Tests que verifican componentes juntos
- **CI/CD**: Integración y despliegue continuo

✅ Ejercicios

Ver EJERCICIOS.md - Módulo 20:
- 20.1: Script E2E completo
- 20.2: Health Check Script

🏁 Entrega

1. Repositorio público en GitHub
2. CI pasando (verde)
3. README con badges actualizados

4. Self-assessment del checklist completado

🎤 Checkpoint: Simulacro Senior/Lead

🎯 **¡Has completado la guía completa!** (Módulos 01-20)

Si buscas posiciones **Senior/Lead ML Engineer**, es momento del simulacro completo:

→ **SIMULACRO_ENTREVISTA_SENIOR_PARTE1.md** — 70 preguntas técnicas avanzadas
→ **SIMULACRO_ENTREVISTA_SENIOR_PARTE2.md** — System design, liderazgo, trade-offs

Material complementario:
- APENDICE_A_SPEECH_PORTAFOLIO.md — Guión de presentación 5-7 min
- APENDICE_B_TALKING_POINTS.md — Puntos clave concisos

¡Éxito en tu proyecto! 🚀

SIMULACRO SENIOR (PARTE 1)

GUÍA MLOPS V5.0

PORTFOLIO EDITION

🎯 Simulacro de Entrevista Lead/Senior ML Engineer

Portafolio MLOps — 100+ Preguntas Técnicas y Conceptuales

Autor del Portafolio: Daniel Duque (DuqueOM)
Versión: 1.0
Fecha: Noviembre 2025

📋 Índice

1. Fundamentos de ML
2. MLOps y Ciclo de Vida
3. BankChurn-Predictor
4. CarVision-Market-Intelligence
5. TelecomAI-Customer-Intelligence
6. Arquitectura y Diseño
7. CI/CD y DevOps
8. Infraestructura K8s/Docker
9. Ética y Fairness
10. Liderazgo y Escenarios

1. Fundamentos de Machine Learning (Preguntas 1-15)

Pregunta 1: Data Leakage

¿Qué es el data leakage y cómo lo preveniste?

Respuesta:

El data leakage ocurre cuando información del test set o target filtra al training.

Prevención en BankChurn (`training.py:278-293`):

```
# CRITICAL: Split BEFORE fitting preprocessor
X_train, X_test, y_train, y_test = train_test_split(X, y, ...)
self.preprocessor_ = self.build_preprocessor(X_train)  # Solo training
X_train = self.preprocessor_.fit_transform(X_train)
X_test = self.preprocessor_.transform(X_test)  # Solo transform
```

En CarVision (`config.yaml`):

```
drop_columns: ["price_per_mile", "price_category"]  # Dependen del target
```

Justificación: El preprocesador (StandardScaler, OneHotEncoder) debe ajustarse **exclusivamente** con datos de training.

Pregunta 2: Class Imbalance

¿Cómo manejaste el desbalance 80/20 en churn?

Respuesta:

Implementé múltiples estrategias en `ResampleClassifier` (`models.py`):

```
class ResampleClassifier(BaseEstimator, ClassifierMixin):
    # strategy: {"none", "oversample", "undersample", "class_weight"}
    def _apply_resampling(self, X, y):
        if self.strategy == "oversample":
            return SMOTE(random_state=self.random_state).fit_resample(X, y)
        elif self.strategy == "undersample":
            return RandomUnderSampler().fit_resample(X, y)
```

Estrategias adicionales:
- `LogisticRegression(class_weight="balanced")`
- `RandomForestClassifier(class_weight="balanced_subsample")`
- **F1-Score** como métrica primaria (equilibra precision-recall)

Pregunta 3: Ensemble VotingClassifier

Explica el VotingClassifier con pesos [0.4, 0.6].

Respuesta:

```
ensemble = VotingClassifier(
    estimators=[("lr", lr), ("rf", rf)],
    voting="soft",      # Promedia probabilidades
    weights=[0.4, 0.6]  # RF tiene mayor peso
)
```

Razones:
1. **Soft voting**: Aprovecha confianza del modelo (probabilidades)
2. **RF (0.6)**: Mejor AUC individual, captura no-linealidades
3. **LR (0.4)**: Regularización, interpretabilidad, buen baseline

Complementariedad: LR asume linealidad, RF captura interacciones → combinación reduce varianza.

Pregunta 4: FeatureEngineer Centralizado

¿Cómo garantizas consistencia entre training e inference?

Respuesta:

Clase `FeatureEngineer` como sklearn transformer (`features.py`):

```
class FeatureEngineer(BaseEstimator, TransformerMixin):
    def transform(self, X):
        X = X.copy()
        if "model_year" in X.columns:
            X["vehicle_age"] = self.current_year - X["model_year"]
        if "model" in X.columns:
            X["brand"] = X["model"].str.split().str[0]
        return X
```

Pipeline integrado:

```
pipe = Pipeline([("features", fe), ("pre", pre), ("model", model)])
joblib.dump(pipe, "model.joblib")  # TODO serializado
```

Beneficio: Single Source of Truth para training, API, dashboard.

Pregunta 5: StratifiedKFold

¿Por qué usaste StratifiedKFold?

Respuesta:

```
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
```

Razones:
1. **Preserva proporción**: Con 80/20, KFold simple podría generar folds 90/10
2. **Representatividad**: Cada fold simula distribución de producción
3. **Estabilidad**: Reduce varianza entre folds

Pregunta 6: Métricas de Regresión

¿Cuándo priorizas RMSE vs MAE vs MAPE?

Respuesta:

Métrica	Priorizar cuando
RMSE	Errores grandes son costosos (penaliza outliers)
MAE	Robustez a outliers, interpretación directa
MAPE	Comparación entre escalas diferentes
R^2	Benchmarking vs baseline

En CarVision priorizo RMSE: Un error de $10K en auto de $5K es crítico.

Pregunta 7: Imputation Strategies

¿Por qué median para numéricos y "constant" para categóricos?

Respuesta:

```
# Numéricos: robusta a outliers
("imputer", SimpleImputer(strategy="median"))
# Categóricos: categoría explícita permite aprender patrón de missing
("imputer", SimpleImputer(strategy="constant", fill_value="missing"))
```

Justificación: `most_frequent` sesga hacia categoría dominante; "missing" es más informativo.

Pregunta 8: Regularización L2

¿Qué significa C= 0.1 en LogisticRegression?

Respuesta:

```
LogisticRegression(C=0.1, solver="liblinear")
```

C = $1/\lambda$ (inverso de regularización L 2):
- C bajo (0.1) = regularización fuerte → modelo simple, menos overfitting
- C alto (10) = regularización débil → más flexibilidad

Selección: Incluido en espacio de búsqueda Optuna `[0.01, 10.0]` log scale.

Pregunta 9: Random Forest Hyperparameters

¿Por qué max_depth= 10 y min_samples_leaf= 5 ?

Respuesta:

Parámetro	Valor	Efecto
`max_depth=10`	Límite profundidad	Previene memorización
`min_samples_leaf=5`	Mínimo en hojas	Evita hojas con 1 - 2 samples
`n_estimators=100`	Árboles	Balance varianza/costo

Validación: ~ 0.85 AUC con gap train-test < 3 %.

Pregunta 10: GradientBoosting vs RandomForest

¿Por qué GB en TelecomAI y RF en BankChurn?

Respuesta:

Aspecto	Random Forest	Gradient Boosting
Enfoque	Bagging (paralelo)	Boosting (secuencial)
Dataset	Grande (10 K+)	Pequeño (~ 2 K)
Features	Mix cat/num	Numéricas simples

TelecomAI usa GB con `max_depth=2, lr=0.05`: muchos árboles simples → regularización fuerte.

Pregunta 11: Cross-Validation con Logging

¿Cómo reportas resultados de CV?

Respuesta:

```
for fold, (train_idx, val_idx) in enumerate(cv.split(X_train, y_train), 1):
    fold_f1 = f1_score(y_fold_val, y_pred, average="weighted")
    cv_scores.append(fold_f1)
    logger.info(f"Fold {fold}/{n_folds}: F1 = {fold_f1:.4f}")

logger.info(f"CV Mean F1: {np.mean(cv_scores):.4f} (+/- {np.std(cv_scores):.4f})")
```

Métricas reportadas: Media ± desviación estándar para evaluar estabilidad.

Pregunta 12: Threshold Selection

¿Cómo elegiste el threshold de clasificación?

Respuesta:

```
# prediction.py
def predict(self, X, threshold=0.5):
    results["prediction"] = (results["probability"] >= threshold).astype(int)
    results["risk_level"] = pd.cut(results["probability"],
        bins=[0, 0.3, 0.7, 1.0], labels=["low", "medium", "high"])
```

Default 0.5, pero configurable vía API. En producción:
- Si costo de FN alto → bajar threshold (más recall)
- Si costo de FP alto → subir threshold (más precision)

Pregunta 13: Feature Importance

¿Cómo calculas y usas feature importance?

Respuesta:

```
# evaluation.py - error_analysis
feature_importance: true  # En config

# RandomForest tiene .feature_importances_
# Para ensemble, extraemos de sub-modelos
```

Uso: Identificar features dominantes, detectar posibles data leakage, explicabilidad al negocio.

Pregunta 14: Calibration

¿Qué es la calibración de probabilidades?

Respuesta:
Del Model Card:

```
## Calibration
- Platt (sigmoid) sobre el modelo final (cv="prefit")
```

Problema: `predict_proba()` puede dar probabilidades que no reflejan frecuencia real.
Solución: `CalibratedClassifierCV` ajusta probabilidades para que 0.7 signifique ~ 70% de positivos reales.

Pregunta 15: Baseline Comparison

¿Cómo comparas tu modelo contra un baseline?

Respuesta:

```
# evaluation.py (CarVision)
dummy = DummyRegressor(strategy="median")
dummy.fit(X_train, y_train)
baseline_metrics = {"rmse": rmse(y_test, dummy.predict(X_test))}
```

Bootstrap para significancia estadística:

```
delta_rmse_mean = modelo - baseline
p_value_two_sided  # Si < 0.05, diferencia significativa
```

2. MLOps y Ciclo de Vida (Preguntas 16-30)

Pregunta 16: MLflow Integration

¿Cómo integraste MLflow?

Respuesta:

```
# training.py
if self.config.mlflow.enabled:
    mlflow.set_tracking_uri(self.config.mlflow.tracking_uri)
    mlflow.set_experiment(self.config.mlflow.experiment_name)
```

```
with mlflow.start_run():
    mlflow.log_params(self.config.model.dict())
    mlflow.log_metrics(metrics)
```

Config (`config.yaml`):

```
mlflow:
  tracking_uri: "file:./mlruns"  # Local
  experiment_name: "bankchurn-experiment"
```

Pregunta 17: Reproducibilidad con Seeds

¿Cómo garantizas reproducibilidad?

Respuesta:

```
# common_utils/seed.py
def set_seed(seed=None):
    seed = seed or os.getenv("SEED") or 42
    os.environ["PYTHONHASHSEED"] = str(seed)
    random.seed(seed)
    np.random.seed(seed)
    # PyTorch/TensorFlow si instalados
    return seed
```

Uso en todos los proyectos:

```
seed_used = set_seed(args.seed)
logger.info(f"Using seed: {seed_used}")
```

Pregunta 18: Configuration con Pydantic

Explica tu sistema de configuración.

Respuesta:

```
# config.py
class ModelConfig(BaseModel):
    test_size: float = Field(0.2, ge=0.0, le=1.0)
    cv_folds: int = Field(5, ge=2)

class BankChurnConfig(BaseModel):
    model: ModelConfig
    data: DataConfig
    mlflow: MLflowConfig

    @classmethod
    def from_yaml(cls, path):
        with open(path) as f:
            return cls(**yaml.safe_load(f))
```

Beneficios: Validación automática, valores por defecto, type hints, serialización.

Pregunta 19: Model Versioning

¿Cómo versionas modelos?

Respuesta:

1. **Naming**: `models/model_v1.0.0.pkl`
2. **Metadata JSON**: commit_sha, métricas, timestamp
3. **K 8 s ConfigMap**: `MODEL_VERSION: "v2.0.0"`
4. **Docker tags**: `bankchurn:v1.0.0`
5. **MLflow Registry**: Staging → Production

Pregunta 20: DVC Data Versioning

¿Por qué DVC?

Respuesta:

```
dvc add data/raw/Churn.csv
git add data/raw/Churn.csv.dvc .gitignore
dvc push  # A storage remoto
```

Beneficios: Cada commit Git tiene snapshot exacto de datos, sin almacenarlos en Git.

Pregunta 21: Pipeline como Artefacto Único

¿Por qué empaquetar preprocessor + model?

Respuesta:

```
pipeline = Pipeline([("preprocess", pre), ("clf", clf)])
joblib.dump(pipeline, "model.joblib")
```

Deployment simplificado:

```
pipe = joblib.load("model.joblib")
pred = pipe.predict(raw_df)  # Sin pasos intermedios
```

Pregunta 22: Health Checks

¿Cómo implementaste health checks?

Respuesta:

```
@app.get("/health")
async def health_check():
    return {"status": "healthy" if predictor else "degraded",
            "model_loaded": predictor is not None}
```

K 8 s probes:

```
livenessProbe:
  httpGet: {path: /health, port: 8000}
  initialDelaySeconds: 30
readinessProbe:
  httpGet: {path: /health, port: 8000}
  initialDelaySeconds: 10
```

Pregunta 23: Model Loading Strategy

¿Cómo cargas el modelo en FastAPI?

Respuesta:

```
@contextlib.asynccontextmanager
async def lifespan(app: FastAPI):
    global predictor
    predictor = load_model_logic()  # Una vez al iniciar
    yield

app = FastAPI(lifespan=lifespan)
```

Beneficio: Carga única, no por request. Fail-fast friendly.

Pregunta 24: Batch Prediction

¿Cómo manejas predicciones batch?

Respuesta:

```
@app.post("/predict_batch")
async def predict_batch(batch_data: BatchCustomerData):
    if len(batch_data.customers) > 1000:
        raise HTTPException(400, "Max 1000 per batch")

    df = pd.DataFrame([c.dict() for c in batch_data.customers])
    results = predictor.predict(df)
    return {"predictions": results, "processing_time": time}
```

Pregunta 25: Metrics Collection

¿Qué métricas operacionales colectas?

Respuesta:

```
@app.get("/metrics")
async def get_metrics():
    return {
        "total_predictions": request_count,
        "average_prediction_time_ms": avg_time,
        "model_accuracy": model_metadata.get("test_accuracy")
    }
```

Prometheus-ready: Annotations en K 8 s para scraping automático.

Pregunta 26: Error Handling en API

¿Cómo manejas errores?

Respuesta:

```
@app.post("/predict")
async def predict(customer: CustomerData):
    if predictor is None:
        raise HTTPException(503, "Model not available")
    try:
        return predictor.predict(df)
    except Exception as e:
        logger.error(f"Prediction error: {e}")
        raise HTTPException(500, str(e))
```

Códigos apropiados: 5 0 3 servicio no disponible, 5 0 0 error interno.

Pregunta 27: Logging Strategy

¿Cómo estructuraste el logging?

Respuesta:

```
logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s - %(name)s - %(levelname)s - %(message)s",
    handlers=[FileHandler("app.log"), StreamHandler()]
)
logger = logging.getLogger(__name__)
```

Niveles usados: INFO para operaciones normales, WARNING para fallbacks, ERROR para fallos.

Pregunta 28: API Validation con Pydantic

¿Cómo validas inputs en la API?

Respuesta:

```
class CustomerData(BaseModel):
    CreditScore: int = Field(..., ge=300, le=850)
    Age: int = Field(..., ge=18, le=100)
    Geography: str

    @validator("Geography")
    def validate_geo(cls, v):
        if v not in ["France", "Spain", "Germany"]:
            raise ValueError("Invalid geography")
        return v
```

Pregunta 29: Response Enrichment

¿Qué incluyes en la respuesta de predicción?

Respuesta:

```
class PredictionResponse(BaseModel):
    churn_probability: float
    churn_prediction: int
```

```
    risk_level: str         # LOW/MEDIUM/HIGH
    confidence: float       # abs(prob - 0.5) * 2
    model_version: str
    prediction_timestamp: str
```

Risk level: Categorización para decisiones de negocio.

Pregunta 30: Graceful Degradation

¿Qué pasa si el modelo no carga?

Respuesta:

```
async def lifespan(app):
    success = load_model_logic()
    if not success:
        logger.warning("Started without model")
    yield  # App sigue corriendo

# Health check reporta estado degradado
{"status": "degraded", "model_loaded": false}
```

Beneficio: Health checks funcionan, ops pueden diagnosticar.

3. BankChurn-Predictor (Preguntas 31-45)

Pregunta 31: Arquitectura Modular

Describe la estructura del proyecto.

Respuesta:

```
src/bankchurn/
├── cli.py        # Entry point CLI
├── config.py     # Pydantic validation
├── models.py     # ResampleClassifier
├── training.py   # ChurnTrainer class
├── evaluation.py # ModelEvaluator class
└── prediction.py # ChurnPredictor class
```

Principio: Single Responsibility - cada módulo una función.

Pregunta 32: CLI Design

¿Cómo diseñaste la CLI?

Respuesta:

```
# cli.py
parser.add_subparsers(dest="command")
train_parser = subparsers.add_parser("train")
train_parser.add_argument("--config", required=True)

def cli_main(argv=None):
    args = parser.parse_args(argv)
    if args.command == "train":
        return train_command(args)
```

Uso: `python -m bankchurn train --config configs/config.yaml`

Pregunta 33: ResampleClassifier

Explica tu clasificador custom.

Respuesta:

```
class ResampleClassifier(BaseEstimator, ClassifierMixin):
    def __init__(self, estimator=None, strategy="none"):
        self.estimator = estimator
        self.strategy = strategy

    def fit(self, X, y):
        X_res, y_res = self._apply_resampling(X, y)
```

```
        self.estimator_.fit(X_res, y_res)
        return self
```

Implementa interfaz sklearn: `fit`, `predict`, `predict_proba` → compatible con Pipeline.

Pregunta 34: Fairness Evaluation

¿Cómo evalúas sesgo por grupos?

Respuesta:

```
def compute_fairness_metrics(self, X, y, sensitive_features):
    for feature in sensitive_features:  # ["Geography", "Gender"]
        for group in X[feature].unique():
            mask = X[feature] == group
            group_f1 = f1_score(y[mask], y_pred[mask])

    # Disparate Impact
    disparate_impact = min(positive_rates) / max(positive_rates)
```

Threshold: Disparate Impact < 0.8 indica discriminación potencial.

Pregunta 35: Model Card

¿Qué incluyes en tu Model Card?

Respuesta:

```
# Model Card - BankChurn
- Model: VotingClassifier (LR + RF)
- Target: Exited (1=churn)
## Intended Use: Priorizar retención, no decisión automática
## Limitations: Desbalance 80/20, posible drift temporal
## Fairness: Tests por geografía/género, gap recall < 0.3
## SLO: Disponibilidad 99.5%, Latencia P95 < 50ms
```

Pregunta 36: Testing Strategy

¿Cómo organizaste los tests?

Respuesta:

```
tests/
├── conftest.py       # Fixtures compartidos
├── test_training.py  # ChurnTrainer
├── test_evaluation.py# ModelEvaluator
├── test_prediction.py# ChurnPredictor
├── test_config.py    # Pydantic validation
└── test_integration.py# E2E workflow
```

Coverage: 77% con pytest-cov.

Pregunta 37: Fixture Pattern

¿Cómo usas fixtures en pytest?

Respuesta:

```
@pytest.fixture
def config():
    return BankChurnConfig.from_yaml("configs/config.yaml")

@pytest.fixture
def sample_data():
    return pd.DataFrame({
        "CreditScore": np.random.randint(300, 850, 200),
        "Exited": np.random.choice([0, 1], 200, p=[0.8, 0.2])
    })

def test_training(config, sample_data, tmp_path):
    trainer = ChurnTrainer(config)
    # ...
```

Pregunta 38: Integration Test

¿Qué cubre tu test de integración?

Respuesta:

```
def test_full_training_workflow(config, sample_data, tmp_path):
    trainer = ChurnTrainer(config)
    data = trainer.load_data(data_path)
    X, y = trainer.prepare_features(data)
    model, metrics = trainer.train(X, y)
    trainer.save_model(model_path)

    # Verify artifacts
    assert model_path.exists()
    assert metrics["test_f1"] > 0.5
```

Pregunta 39: Dockerfile Multi-Stage

Explica tu Dockerfile.

Respuesta:

```
# Stage 1: Builder
FROM python:3.13-slim AS builder
RUN pip install -r requirements.txt
# Stage 2: Runtime
FROM python:3.13-slim AS runtime
COPY --from=builder /opt/venv /opt/venv
USER appuser  # Non-root
HEALTHCHECK CMD curl -f http://localhost:8000/health
CMD ["uvicorn", "app.fastapi_app:app", "--host", "0.0.0.0"]
```

Beneficios: Imagen final sin build tools, usuario non-root, healthcheck.

Pregunta 40: CORS Configuration

¿Cómo configuraste CORS?

Respuesta:

```
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],  # Producción: restringir
    allow_methods=["GET", "POST"],
    allow_headers=["*"],
)
```

Nota: `allow_origins=["*"]` solo para desarrollo. Producción especifica dominios.

Pregunta 41-45: [Ver archivo parte 2]

4. CarVision-Market-Intelligence (Preguntas 46-60)

Pregunta 46: Pipeline [features, pre, model]

Explica el pipeline de CarVision.

Respuesta:

```
pipe = Pipeline([
    ("features", FeatureEngineer(current_year=2024)),
    ("pre", ColumnTransformer([
        ("num", numeric_pipeline, num_cols),
        ("cat", categorical_pipeline, cat_cols)
    ])),
    ("model", RandomForestRegressor())
])
```

Flujo: Raw DF → Feature Engineering → Preprocessing → Model.

Pregunta 47: Data Filtering

¿Por qué filtras precios entre $ 1 K-$ 5 0 0 K?

Respuesta:

```
def clean_data(df, filters):
    df = df[(df["price"] > 1000) & (df["price"] < 500000)]
    df = df[df["model_year"] >= 1990]
    df = df[df["odometer"] < 500000]
```

Razón: < $ 1 K son errores/donaciones, > $ 5 0 0 K son coleccionables (mercado diferente).

Pregunta 48: Bootstrap Confidence Intervals

¿Cómo calculas intervalos de confianza?

Respuesta:

```
for _ in range(200):
    idx = rng.choice(n, size=n, replace=True)
    deltas.append(rmse(y[idx], y_model[idx]) - rmse(y[idx], y_base[idx]))

ci_low, ci_high = np.percentile(deltas, [2.5, 97.5])
```

Interpretación: CI 9 5 no incluye 0 → diferencia significativa.

Pregunta 49: Temporal Backtesting

¿Cómo validas con datos temporales?

Respuesta:

```
df_sorted = df.sort_values("model_year")
df_test = df_sorted.tail(int(len(df) * 0.2))  # Más recientes
metrics_temporal = evaluate(model, df_test)
```

Simula producción: Siempre predecimos el "futuro".

Pregunta 50: Streamlit Caching

¿Cómo optimizas el dashboard?

Respuesta:

```
@st.cache_data  # DataFrames (serializable)
def load_data():
    return FeatureEngineer().transform(load_raw_data())

@st.cache_resource  # Modelos (no serializable)
def load_model():
    return joblib.load("model.joblib")
```

Preguntas 51-60: [Ver archivo parte 2]

5. TelecomAI (Preguntas 61-70)

Pregunta 61: Simple Feature Set

¿Por qué solo 4 features?

Respuesta:

```
features: [calls, minutes, messages, mb_used]
```

Razón: ~ 2 K muestras, más features = overfitting. AUC 0 . 8 4 indica señal suficiente.

Pregunta 62: GradientBoosting Shallow

¿Por qué max_depth= 2 ?

Respuesta:

```
model:
  name: gradient_boosting
  params: {n_estimators: 200, max_depth: 2, learning_rate: 0.05}
```

Muchos árboles simples: Regularización fuerte, reduce overfitting en dataset pequeño.

Pregunta 63-70: [Ver archivo parte 2]

6. Arquitectura y Diseño (Preguntas 71-80)

Pregunta 71: Design Patterns

¿Qué patrones aplicaste?

Respuesta:

1. **Strategy**: `ResampleClassifier(strategy="oversample")`
2. **Factory**: `build_model(cfg)` crea diferentes clasificadores
3. **Template Method**: `ChurnTrainer.train()` con pasos customizables
4. **Dependency Injection**: `ChurnTrainer(config)`

Pregunta 72: SOLID Principles

¿Cómo aplicaste SOLID?

Respuesta:

- **S**: Un módulo, una responsabilidad (training.py solo entrena)
- **O**: ResampleClassifier abierto a nuevas estrategias
- **L**: Hereda de sklearn, sustituible donde se espere classifier
- **I**: Interfaces pequeñas (Predictor, Evaluator, Trainer)
- **D**: Depende de config, no de valores hardcodeados

Preguntas 73-80: [Ver archivo parte 2]

7. CI/CD y DevOps (Preguntas 81-90)

Pregunta 81: Unified CI Pipeline

Explica tu workflow unificado.

Respuesta:

```
# ci-mlops.yml
jobs:
  tests:           # pytest + coverage por proyecto
  quality-gates:   # black, flake8, mypy
  security:        # bandit, gitleaks, pip-audit
  docker:          # build + trivy scan
  integration-test: # docker-compose E2E
```

Matrix: Python $3.11/3.12 \times 3$ proyectos.

Pregunta 82: Security Scanning

¿Qué herramientas de seguridad usas?

Respuesta:

- **Gitleaks**: Detecta secrets en código
- **Bandit**: Análisis estático Python
- **Trivy**: Vulnerabilidades en containers
- **pip-audit**: Dependencias con CVEs

Preguntas 83-90: [Ver archivo parte 2]

8. Infraestructura (Preguntas 91-100)

Pregunta 91: Kubernetes Deployment

Explica tu deployment de K8s.

Respuesta:

```
apiVersion: apps/v1
kind: Deployment
spec:
  replicas: 3
  strategy: {type: RollingUpdate}
  template:
    spec:
```

```
      containers:
      - name: bankchurn-api
        resources:
          requests: {memory: "512Mi", cpu: "250m"}
          limits: {memory: "1Gi", cpu: "1000m"}
        livenessProbe: ...
        readinessProbe: ...
```

Pregunta 92: HorizontalPodAutoscaler

¿Cómo configuras autoscaling?

Respuesta:

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
spec:
  minReplicas: 2
  maxReplicas: 10
  metrics:
  - type: Resource
    resource: {name: cpu, target: {averageUtilization: 70}}
```

Preguntas 93-100: [Ver archivo parte 2]

9. Ética y Fairness (Preguntas 101-105)

Pregunta 101: Responsible AI

¿Cómo abordas responsabilidad en ML?

Respuesta:

1. **Model Cards**: Documentan limitaciones y riesgos
2. **Fairness metrics**: Disparate Impact por grupo
3. **Transparencia**: Logs de predicciones, versiones
4. **Human-in-loop**: Modelo como apoyo, no decisión final

Pregunta 102-105: [Ver archivo parte 2]

10. Liderazgo (Preguntas 106-115)

Pregunta 106: Technical Decision Making

¿Cómo decides entre RF y GB?

Respuesta:

Factores: Tamaño dataset, tipo de features, requerimientos de latencia, interpretabilidad.
Proceso: Experimento con baseline → comparar métricas → considerar trade-offs.

Preguntas 107-115: [Ver archivo Parte 2]

[Continúa en SIMULACRO_ENTREVISTA_PARTE 2 .md]

SIMULACRO SENIOR (PARTE 2)

GUÍA MLOPS V5.0

PORTFOLIO EDITION

🎯 Simulacro Entrevista Lead/Senior ML Engineer - Parte 2

Continuación de preguntas 41-115

Preguntas 41-45: BankChurn (continuación)

Pregunta 41: Risk Level Classification

¿Cómo categorizas el riesgo de churn?

Respuesta:

```
# prediction.py
def _assign_risk_level(self, probability: float) -> str:
    if probability < 0.3:
        return "low"
    elif probability < 0.7:
        return "medium"
    else:
        return "high"
```

Uso en negocio:
- **High**: Contacto proactivo inmediato
- **Medium**: Ofertas de retención
- **Low**: Monitoreo estándar

Pregunta 42: Feature Contribution Explanation

¿Cómo explicas las predicciones?

Respuesta:

```
# prediction.py
def _generate_explanation(self, X_row) -> Dict[str, float]:
    # Enfoque simplificado: distancia a media por feature
    contributions = {}
    for col in self.feature_names:
        mean_val = self.feature_means.get(col, 0)
        diff = (X_row[col] - mean_val) / (self.feature_stds.get(col, 1) + 1e-8)
        contributions[col] = float(diff)
    return contributions
```

Limitación reconocida: No es SHAP values real, pero da intuición inicial sin dependencia adicional.

Pregunta 43: Model Metadata

¿Qué metadata guardas con el modelo?

Respuesta:

```
{
  "version": "1.0.0",
  "trained_at": "2024-11-20T10:30:00Z",
  "config_hash": "abc123def456",
  "test_metrics": {
    "f1_score": 0.82,
    "auc_roc": 0.853
  },
  "feature_names": ["CreditScore", "Age", ...],
  "training_samples": 8000,
  "git_commit": "abc123"
}
```

Uso: Trazabilidad, comparación entre versiones, debugging.

Pregunta 44: Lifespan vs On-Event Loading

¿Por qué usas lifespan en lugar de startup event?

Respuesta:

```
# FastAPI >= 0.95 depreca @app.on_event
@asynccontextmanager
async def lifespan(app: FastAPI):
```

```
    # Startup
    load_model()
    yield
    # Shutdown (cleanup)

app = FastAPI(lifespan=lifespan)
```

Beneficios:
- Patrón moderno recomendado
- Context manager claro para setup/teardown
- Mejor manejo de recursos async

Pregunta 45: Test Coverage Strategy

¿Cómo alcanzaste 77% coverage?

Respuesta:

```
# pytest.ini
[pytest]
addopts = --cov=src/bankchurn --cov-report=term-missing

# Estrategia:
# 1. Unit tests por módulo
# 2. Integration tests para workflows completos
# 3. Edge cases: datos vacíos, missing columns, tipos inválidos
# 4. Error paths: excepciones esperadas
```

Cobertura por módulo:
- training.py: 85%
- evaluation.py: 80%
- config.py: 90%
- cli.py: 60% (I/O difícil de testear)

Preguntas 51-60: CarVision (continuación)

Pregunta 51: Feature Type Inference

¿Cómo detectas tipos de features automáticamente?

Respuesta:

```
# data.py
def infer_feature_types(df, cfg):
    num_cfg = cfg.get("numeric_features", [])
    cat_cfg = cfg.get("categorical_features", [])

    if not num_cfg:
        num_cfg = df.select_dtypes(include=["int64", "float64"]).columns.tolist()
    if not cat_cfg:
        cat_cfg = df.select_dtypes(include=["object", "category"]).columns.tolist()

    return num_cfg, cat_cfg
```

Beneficio: Config minimalista (`[]`) usa inferencia automática.

Pregunta 52: Segment Error Analysis

¿Cómo analizas errores por segmento?

Respuesta:

```
def _analyse_errors_by_segment(df, y_true, y_pred, segment_cols):
    results = []
    for col in segment_cols:  # ["condition", "type", "model_year"]
        for val, group in df.groupby(col):
            if len(group) < 30:
                continue
            results.append({
                "segment": f"{col}={val}",
                "rmse": rmse(group[target], y_pred[group.index]),
                "n_samples": len(group)
```

```
            })
    return pd.DataFrame(results)
```

Hallazgo típico: Mayor error en autos muy nuevos (pocos datos) o muy viejos (alta variabilidad).

Pregunta 53: Dual Application (API + Dashboard)

¿Cómo manejas API y Streamlit juntos?

Respuesta:

```
app/
├── fastapi_app.py     # REST API (port 8002)
└── streamlit_app.py   # Dashboard (port 8501)
```

Ambos usan:

```
from src.carvision.features import FeatureEngineer
from src.carvision.data import clean_data

# Mismo pipeline, diferentes interfaces
model = joblib.load(MODEL_PATH)
```

Docker Compose:

```
services:
  carvision-api:
    command: uvicorn app.fastapi_app:app --port 8002
  carvision-dashboard:
    command: streamlit run app/streamlit_app.py --server.port 8501
```

Pregunta 54: Price Category Leakage

¿Por qué price_category causa leakage?

Respuesta:

```
# features.py - ANTES (bug)
X["price_category"] = pd.cut(X["price"], bins=[0, 5000, 15000, 50000, np.inf])
```

Problema: `price_category` depende de `price` (el target).
- En training: tiene el valor correcto
- En inference: `price` no existe → error o leakage si se imputa

Solución: `drop_columns: ["price_per_mile", "price_category"]` en config.

Pregunta 55: Split Indices Persistence

¿Por qué guardas los índices del split?

Respuesta:

```
def save_split_indices(indices, path):
    with open(path, "w") as f:
        json.dump({k: v.tolist() for k, v in indices.items()}, f)
```

Uso:

```
# Reproducir exactamente el mismo split
indices = load_split_indices("split_indices.json")
X_train = X.iloc[indices["train"]]
X_test = X.iloc[indices["test"]]
```

Beneficio: Comparar modelos con EXACTAMENTE los mismos datos de test.

Pregunta 56: Dummy Baseline

¿Por qué comparas con DummyRegressor?

Respuesta:

```
dummy = DummyRegressor(strategy="median")
baseline_rmse = rmse(y_test, dummy.fit(X_train, y_train).predict(X_test))
```

Estrategias disponibles:
- `mean`: Predice promedio
- `median`: Predice mediana (robusta a outliers)
- `constant`: Valor fijo

Interpretación: Si tu modelo no supera dummy, no agrega valor.

Pregunta 57: Random Forest para Regresión

¿Por qué RF y no XGBoost para CarVision?

Respuesta:

Criterio	RandomForest	XGBoost
Simplicidad	Menos hiperparámetros	Muchos HP críticos
Robustez	Menos sensible a HP	Requiere tuning fino
Interpretabilidad	Feature importance directa	Más complejo
Performance	Competitivo en tabular	Marginalmente mejor

Decisión: RF es "good enough" con menor riesgo de overfitting y configuración más simple.

Pregunta 58: MAPE Calculation

¿Por qué sumas epsilon en MAPE?

Respuesta:

```
mape = np.mean(np.abs((y_true - y_pred) / (y_true + 1e-8))) * 100
```

Problema: Si `y_true = 0`, división por cero.
Solución: `+ 1e-8` evita división por cero.

Alternativa mejor para precios:

```
# Symmetric MAPE
smape = np.mean(2 * np.abs(y_true - y_pred) / (np.abs(y_true) + np.abs(y_pred)))
```

Pregunta 59: Streamlit Sections

¿Cómo organizas el dashboard?

Respuesta:

```
# streamlit_app.py
tab1, tab2, tab3, tab4 = st.tabs([
    "📊 Overview",
    "📈 Market Analysis",
    "🎯 Model Metrics",
    "💰 Price Predictor"
])

with tab1:
    display_overview_kpis()
with tab4:
    # Formulario de predicción
    brand = st.selectbox("Brand", brands)
    year = st.slider("Year", 1990, 2024)
    if st.button("Predict"):
        pred = model.predict(features)
        st.success(f"Estimated Price: ${pred:,.0f}")
```

Pregunta 60: API vs Dashboard Trade-offs

¿Cuándo recomiendas API vs Dashboard?

Respuesta:

Caso de Uso	Recomendación
Integración con otros sistemas	API
Usuarios no técnicos	Dashboard
Alto volumen	API
Exploración ad-hoc	Dashboard
Automatización	API
Demos/POC	Dashboard

CarVision ofrece ambos: API para integración con sistemas de dealers, Dashboard para analistas de mercado.

Preguntas 63-70: TelecomAI (continuación)

Pregunta 63: Unified Pipeline Benefit

¿Por qué un solo artefacto pipeline?

Respuesta:

```
# ANTES (2 archivos)
preprocessor = joblib.load("preprocessor.pkl")
model = joblib.load("model.pkl")
X_proc = preprocessor.transform(X)
pred = model.predict(X_proc)

# AHORA (1 archivo)
pipeline = joblib.load("model.joblib")
pred = pipeline.predict(X)
```

Beneficios:
1. Deployment más simple
2. Imposible desincronizar preprocessor/model
3. Una sola versión para auditar

Pregunta 64: Config YAML Structure

¿Cómo estructuras el config de TelecomAI?

Respuesta:

```
project_name: TelecomAI-Customer-Intelligence
random_seed: 42

paths:
  data_csv: users_behavior.csv
  model_path: artifacts/model.joblib

features: [calls, minutes, messages, mb_used]
target: is_ultra

split:
  test_size: 0.2
  stratify: true

model:
  name: gradient_boosting
  params:
    n_estimators: 200
    max_depth: 2
```

Principio: Toda configuración externalizada, ningún valor hardcodeado.

Pregunta 65: Stratify con Clasificación Binaria

¿Cuándo es crítica la estratificación?

Respuesta:

Crítica cuando:
- Clases desbalanceadas (< 70/30)
- Dataset pequeño (< 5K samples)
- Métrica sensible a distribución (precision, recall)

```
# Sin estratificación en 2K samples con 30% positivos:
# Test set podría tener 20% o 40% positivos → métricas no comparables
```

Pregunta 66: Simple Model Debugging

¿Cómo debuggeas un modelo simple?

Respuesta:

```
# 1. Check feature distributions
print(X_train.describe())
print(X_test.describe())  # Debería ser similar

# 2. Check target distribution
print(y_train.value_counts(normalize=True))
print(y_test.value_counts(normalize=True))

# 3. Learning curve
from sklearn.model_selection import learning_curve
train_sizes, train_scores, val_scores = learning_curve(
    model, X, y, cv=5, scoring="roc_auc"
)
```

Pregunta 67: Gradient Boosting Learning Rate

¿Qué pasa si lr es muy alto o muy bajo?

Respuesta:

lr	Efecto
0.5+ **(alto)**	Convergencia rápida, riesgo de overfitting
0.01-0.1 **(medio)**	Balance típico
< 0.01 **(bajo)**	Necesita muchos estimators, más robusto

```
# TelecomAI usa lr=0.05, n_estimators=200
# Conservador pero estable
```

Regla práctica: `lr * n_estimators ≈ 10-20` para convergencia.

Pregunta 68: Evaluation Metrics Save

¿Cómo persistes métricas?

Respuesta:

```
# evaluation.py
def evaluate_model(pipeline, X_test, y_test, cfg):
    metrics = compute_classification_metrics(y_test, y_pred, y_proba)

    # Save to YAML
    with open(cfg.paths["metrics_path"], "w") as f:
        yaml.safe_dump(metrics, f)

    return metrics
```

Formato YAML para legibilidad humana:

```
accuracy: 0.812
precision: 0.785
recall: 0.743
f1: 0.763
roc_auc: 0.840
```

Pregunta 69: FastAPI App TelecomAI

¿Cómo estructuras la API?

Respuesta:

```
# app/fastapi_app.py
class UserBehavior(BaseModel):
    calls: int
    minutes: float
    messages: int
    mb_used: float

@app.post("/predict")
async def predict_plan(user: UserBehavior):
    features = pd.DataFrame([user.dict()])
    pred = pipeline.predict(features)[0]
    proba = pipeline.predict_proba(features)[0][1]
    return {
        "recommended_plan": "Ultra" if pred == 1 else "Basic",
        "confidence": float(proba)
    }
```

Pregunta 70: TelecomAI Business Context

¿Cómo se usa el modelo en el negocio?

Respuesta:

Contexto: Recomendar plan de datos (Basic vs Ultra) basado en patrones de uso.

Flujo:
1. Cliente usa servicio por periodo de prueba
2. Sistema recolecta métricas: calls, minutes, messages, mb_used
3. Modelo predice plan óptimo
4. Ventas contacta con oferta personalizada

Valor:
- Reduce churn por plan inadecuado
- Aumenta ARPU en usuarios infraservidos
- Mejora satisfacción del cliente

Preguntas 73-80: Arquitectura (continuación)

Pregunta 73: Error Handling Philosophy

¿Cuál es tu filosofía de manejo de errores?

Respuesta:

1. **Fail fast**: Validar inputs al inicio
2. **Errores específicos**: `ValueError` vs `FileNotFoundError` vs genérico
3. **Logs contextuales**: Incluir datos relevantes en el error
4. **Graceful degradation**: Servicio funciona con capacidad reducida

```
# Malo
try:
    do_something()
except Exception:
    pass

# Bueno
try:
    do_something(input_data)
except ValueError as e:
    logger.error(f"Invalid input {input_data}: {e}")
    raise HTTPException(400, f"Invalid input: {e}")
```

```
except FileNotFoundError as e:
    logger.error(f"Model file missing: {e}")
    raise HTTPException(503, "Model not available")
```

Pregunta 74: Dependency Management

¿Cómo manejas dependencias entre proyectos?

Respuesta:

```
Projects Tripe Ten/
├── common_utils/         # Shared code
│   ├── __init__.py
│   ├── logger.py
│   └── seed.py
├── BankChurn-Predictor/
│   └── requirements.txt  # Project-specific deps
├── CarVision-Market-Intelligence/
│   └── requirements.txt
└── TelecomAI-Customer-Intelligence/
    └── requirements.txt
```

common_utils en cada project:

```
import sys
sys.path.insert(0, str(Path(__file__).parent.parent.parent))
from common_utils.seed import set_seed
```

Pregunta 75: API Versioning

¿Cómo versionarías la API?

Respuesta:

```
# Opción 1: Path versioning
@app.post("/v1/predict")
@app.post("/v2/predict")

# Opción 2: Header versioning
@app.post("/predict")
async def predict(request: Request):
    version = request.headers.get("API-Version", "1")

# Opción 3: Query param
@app.post("/predict")
async def predict(version: str = Query("1")):
```

Recomendación: Path versioning es más explícito y cacheable.

Pregunta 76: Configuration Hierarchy

¿Cómo manejas diferentes environments?

Respuesta:

```
# configs/config.yaml (base)
mlflow:
  tracking_uri: "file:./mlruns"  # Default: local

# Override via environment variables
# MLFLOW_TRACKING_URI=http://mlflow.prod:5000

# O archivos separados
# configs/config.dev.yaml
# configs/config.prod.yaml
```

Carga con override:

```
config = BankChurnConfig.from_yaml("configs/config.yaml")
config.mlflow.tracking_uri = os.getenv("MLFLOW_URI", config.mlflow.tracking_uri)
```

Pregunta 77: Async vs Sync en FastAPI

¿Cuándo usas async?

Respuesta:

```
# Sync - operaciones CPU-bound (ML inference)
@app.post("/predict")
def predict_sync(data: CustomerData):
    return model.predict(data)  # CPU bound

# Async - operaciones I/O bound
@app.get("/health")
async def health_async():
    await check_external_service()  # Network call
```

ML inference es CPU-bound: `def` es preferible para evitar bloquear event loop.

Pregunta 78: Testing Pyramid

¿Cómo estructuras tus tests?

Respuesta:

```
Tests/
├── Unit (70%)
│   ├── test_config.py
│   ├── test_models.py
│   └── test_evaluation.py
├── Integration (20%)
│   └── test_training.py (E2E workflow)
└── E2E (10%)
    └── test_api.py (requests reales)
```

Pirámide: Muchos unit tests (rápidos), pocos E 2 E (lentos pero valiosos).

Pregunta 79: Code Review Checklist

¿Qué revisas en un PR de ML?

Respuesta:

1 . **Data leakage**: ¿Split antes de fit?
2 . **Reproducibilidad**: ¿Seeds configurados?
3 . **Tests**: ¿Cubren happy path y edge cases?
4 . **Config**: ¿Valores hardcodeados?
5 . **Métricas**: ¿Apropiadas para el problema?
6 . **Logging**: ¿Suficiente para debugging?
7 . **Documentation**: ¿Model card actualizado?

Pregunta 80: Technical Debt Management

¿Cómo manejas deuda técnica?

Respuesta:

Categorización:
- **Critical**: Bugs de seguridad, data leakage → Sprint actual
- **High**: Tests faltantes, logging pobre → Próximo sprint
- **Medium**: Refactoring, documentación → Backlog priorizado
- **Low**: Nice-to-have → Tech debt day mensual

Tracking: Issues en GitHub con label `tech-debt` y estimación de impacto.

Preguntas 83-90: CI/CD (continuación)

Pregunta 83: Matrix Testing Strategy

¿Por qué matrix Python 3 . 1 1 / 3 . 1 2 ?

Respuesta:

```
strategy:
  matrix:
```

```
    python-version: ['3.11', '3.12']
    project: [BankChurn, CarVision, TelecomAI]
```

Razones:
- 3.11: Versión estable ampliamente usada
- 3.12: Versión más reciente, validar compatibilidad
- 3 **proyectos**: Detectar regresiones cross-project

Total jobs: 2 × 3 = 6 combinaciones paralelas.

Pregunta 84: Fail-Fast Strategy

¿Cuándo usar fail-fast: false?

Respuesta:

```
strategy:
  fail-fast: false  # Continúa aunque un job falle
```

Usar `false` cuando:
- Quieres ver TODOS los errores, no solo el primero
- Jobs son independientes (matriz de versiones)
- Debugging de problemas de compatibilidad

Usar `true` cuando:
- Jobs dependientes
- Quieres feedback rápido
- Recursos de CI limitados

Pregunta 85: Docker Build Caching

¿Cómo optimizas builds de Docker?

Respuesta:

```
# Orden óptimo de COPY
COPY requirements.txt .
RUN pip install -r requirements.txt  # Cached si requirements no cambia
COPY src/ ./src/                     # Solo copia código
```

CI caching:

```
- uses: docker/build-push-action@v5
  with:
    cache-from: type=gha
    cache-to: type=gha,mode=max
```

Pregunta 86: Security Scanning Results

¿Qué haces con findings de seguridad?

Respuesta:

Proceso:
1. **Critical/High**: Bloquea PR, fix inmediato
2. **Medium**: Documenta, fix en siguiente sprint
3. **Low**: Backlog, evaluar riesgo/esfuerzo
4. **False positives**: Añadir a `.gitleaksignore` con justificación

```
# .gitleaksignore
# False positive: example API key in documentation
docs/examples/api_usage.md:15
```

Pregunta 87: Coverage Thresholds

¿Por qué 70% coverage mínimo?

Respuesta:

```
- name: Check coverage threshold
  run: |
    coverage report --fail-under=70
```

Razón del 70%:
- **<60%**: Riesgo de bugs no detectados
- **70-80%**: Balance costo/beneficio típico
- **>90%**: Diminishing returns, tests frágiles

No todo necesita tests: I/O, logging, error messages triviales.

Pregunta 88: Integration Test Strategy

¿Qué cubren tus integration tests?

Respuesta:

```
integration-test:
  steps:
    - run: docker-compose -f docker-compose.demo.yml up -d
    - run: |
        # Wait for services
        sleep 30
        # Test each API
        curl http://localhost:8001/health
        curl http://localhost:8002/health
        curl -X POST http://localhost:8001/predict -d '...'
```

Cobertura: Health checks, predicciones básicas, formato de respuesta.

Pregunta 89: Documentation Validation

¿Cómo validas documentación?

Respuesta:

```
doc-validation:
  steps:
    - name: Check markdown links
      run: |
        npm install -g markdown-link-check
        find . -name "*.md" -exec markdown-link-check {} \;

    - name: Build mkdocs
      run: |
        pip install mkdocs
        mkdocs build --strict
```

Valida: Links rotos, sintaxis markdown, build de docs.

Pregunta 90: Deployment Strategy

¿Cómo desplegarías a producción?

Respuesta:

```
deploy-prod:
  needs: [tests, security, docker-build]
  if: github.ref == 'refs/heads/main'
  steps:
    - name: Push to registry
      run: docker push ghcr.io/${{ github.repository }}:${{ github.sha }}

    - name: Update K8s deployment
      run: |
        kubectl set image deployment/bankchurn \
          bankchurn=ghcr.io/${{ github.repository }}:${{ github.sha }}
```

Estrategia: Rolling update con readiness probes.

Preguntas 93-100: Infraestructura (continuación)

Pregunta 93: Resource Requests vs Limits

Explica requests vs limits en K8s.

Respuesta:

```
resources:
  requests:
    memory: "512Mi"  # Garantizado
    cpu: "250m"      # Scheduler usa esto para placement
  limits:
    memory: "1Gi"    # Máximo (OOMKill si excede)
    cpu: "1000m"     # Throttling si excede
```

Best practice:
- `requests`: Uso típico (P50)
- `limits`: Uso pico aceptable (P99)

Pregunta 94: Prometheus Scraping

¿Cómo configuras Prometheus para scraping?

Respuesta:

```
# prometheus-config.yaml
scrape_configs:
  - job_name: 'bankchurn-predictor'
    kubernetes_sd_configs:
    - role: pod
      namespaces:
        names: [ml-portfolio]
    relabel_configs:
    - source_labels: [__meta_kubernetes_pod_label_app]
      action: keep
      regex: bankchurn-predictor
```

Annotations en Deployment:

```
annotations:
  prometheus.io/scrape: "true"
  prometheus.io/port: "8000"
  prometheus.io/path: "/metrics"
```

Pregunta 95: ConfigMap Usage

¿Cuándo usas ConfigMap vs Secret?

Respuesta:

Dato	Usar
MODEL_VERSION	ConfigMap
LOG_LEVEL	ConfigMap
API_KEY	Secret
DB_PASSWORD	Secret

```
# ConfigMap
apiVersion: v1
kind: ConfigMap
data:
  MODEL_VERSION: "v2.0.0"
  LOG_LEVEL: "INFO"
---
# Montaje en pod
envFrom:
```

```
  - configMapRef:
      name: bankchurn-config
```

Pregunta 96: Rolling Update Strategy

Explica tu estrategia de actualización.

Respuesta:

```
spec:
  replicas: 3
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxUnavailable: 1  # Máximo 1 pod down
      maxSurge: 1        # Máximo 1 pod extra temporal
```

Flujo con 3 replicas:
1. Crear 1 nuevo pod (4 total)
2. Cuando nuevo está Ready, terminar 1 viejo (3 total)
3. Repetir hasta todos actualizados

Zero downtime con readiness probes correctos.

Pregunta 97: Volume Mounts para Modelos

¿Cómo montas modelos en K8s?

Respuesta:

```
spec:
  containers:
  - name: bankchurn-api
    volumeMounts:
    - name: models
      mountPath: /app/models
      readOnly: true
  volumes:
  - name: models
    persistentVolumeClaim:
      claimName: ml-models-pvc
```

Alternativas:
- **PVC**: Modelos compartidos entre pods
- **S3/GCS**: Descarga al inicio
- **ConfigMap**: Solo para configs pequeños

Pregunta 98: Ingress Configuration

¿Cómo expones servicios externamente?

Respuesta:

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: ml-portfolio-ingress
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
spec:
  rules:
  - host: bankchurn.example.com
    http:
      paths:
      - path: /
        backend:
          service:
            name: bankchurn-service
            port:
              number: 8000
```

Pregunta 99: Terraform Overview

¿Qué infraestructura defines con Terraform?

Respuesta:

```
infra/terraform/
├── aws/
│   ├── main.tf      # EKS cluster, S3, RDS
│   ├── variables.tf
│   └── outputs.tf
└── gcp/
    ├── main.tf      # GKE, GCS, CloudSQL
    └── ...
```

Recursos típicos:
- Container registry (ECR/GCR)
- Kubernetes cluster (EKS/GKE)
- Object storage (S3/GCS) para modelos
- Database (RDS/CloudSQL) para MLflow

Pregunta 100: Disaster Recovery

¿Cómo manejas DR para ML systems?

Respuesta:

1. **Model artifacts**:
- S3 versioning + cross-region replication
- Git LFS / DVC como backup secundario

2. **MLflow database**:
- RDS multi-AZ + daily snapshots
- Point-in-time recovery

3. **Kubernetes**:
- Declarative configs en Git (GitOps)
- Multi-region deployment para HA

RTO/RPO objetivos:
- RTO (Recovery Time): < 1 hora
- RPO (Recovery Point): < 1 día de experimentos

Preguntas 102-105: Ética (continuación)

Pregunta 102: Bias Detection

¿Cómo detectas bias en producción?

Respuesta:

```
# Monitoreo continuo
for segment in ["Geography", "Gender", "Age_bucket"]:
    pred_rate = predictions.groupby(segment)["churn_pred"].mean()
    log_metric(f"pred_rate_{segment}", pred_rate.to_dict())

    # Alerta si disparate impact < 0.8
    di = pred_rate.min() / pred_rate.max()
    if di < 0.8:
        alert(f"Potential bias in {segment}: DI={di}")
```

Pregunta 103: Model Card Maintenance

¿Con qué frecuencia actualizas Model Cards?

Respuesta:

Actualizar cuando:
- Nueva versión del modelo
- Cambio en datos de entrenamiento
- Descubrimiento de limitación/bias
- Cambio en uso previsto

Versionado: Model Card versión = Model versión (`v1.0.0`).

Pregunta 104: Human-in-the-Loop

¿Cómo integras supervisión humana?

Respuesta:

BankChurn:
- Modelo sugiere clientes en riesgo
- Equipo de retención revisa lista
- Decisión final es humana (llamar o no)

No automatizar: Ofertas, descuentos, cierre de cuentas.

Pregunta 105: GDPR Compliance

¿Cómo manejas datos personales?

Respuesta:

1. **Minimización**: Solo features necesarias (no nombre, email)
2. **Pseudonimización**: CustomerID sin mapeo a identidad real
3. **Right to erasure**: Pipeline para eliminar datos de un cliente
4. **Audit trail**: Logs de predicciones (sin PII) para auditoría

Preguntas 107-115: Liderazgo (continuación)

Pregunta 107: Team Communication

¿Cómo comunicas decisiones técnicas al equipo?

Respuesta:

1. **ADRs (Architecture Decision Records)**: Documentar why, not just what
2. **Tech talks**: Sesiones de 30 min sobre decisiones importantes
3. **PR descriptions**: Contexto suficiente para reviewers
4. **Diagrams**: Mermaid/Lucidchart para arquitectura

Pregunta 108: Mentoring Junior Engineers

¿Cómo mentoras a juniors en ML?

Respuesta:

1. **Pair programming**: En primeros PRs de ML
2. **Code review detallado**: Explicar el "por qué"
3. **Recursos curados**: Pointing to best practices
4. **Proyectos graduales**: Simple → Complejo

Errores comunes a prevenir:
- Data leakage (el más crítico)
- Overfitting sin validación
- Métricas incorrectas para el problema

Pregunta 109: Stakeholder Management

¿Cómo manejas expectativas de stakeholders?

Respuesta:

1. **Baseline comparison**: "El modelo mejora X% sobre regla actual"
2. **Confidence intervals**: "Precisión entre 75-85%"
3. **Limitations explícitas**: "No funciona bien para casos X"
4. **Iterative delivery**: MVP → Mejoras incrementales

Evitar: Prometer 99% accuracy, plazos imposibles, omitir limitaciones.

Pregunta 110: Technical Debt Negotiation

¿Cómo negocias tiempo para tech debt?

Respuesta:

Argumentos efectivos:
1. **Riesgo cuantificado**: "Sin tests, bugs llegan a prod"
2. **Velocity impact**: "Refactor ahora ahorra X horas/semana"
3. **Costo de delay**: "Cada mes aumenta esfuerzo 20%"

Estrategia: 20% del sprint para tech debt (negociado upfront).

Pregunta 111: Production Incident Response

¿Cómo manejas un incidente en producción?

Respuesta:

Playbook:
1. **Detect**: Alertas de Prometheus/PagerDuty
2. **Triage**: Severity assessment (P1-P4)
3. **Communicate**: Status page update
4. **Mitigate**: Rollback if needed
5. **Fix**: Root cause resolution
6. **Postmortem**: Blameless análisis

Para ML específico: Rollback = deploy versión anterior del modelo.

Pregunta 112: Cross-functional Collaboration

¿Cómo trabajas con Data Scientists vs ML Engineers?

Respuesta:

Rol	Responsabilidad
Data Scientist	Exploración, feature engineering, model selection
ML Engineer	Productionization, CI/CD, monitoring
Overlap	Evaluación, experiments

Handoff: DS entrega notebook + requirements, MLE convierte a pipeline.

Pregunta 113: Prioritization Framework

¿Cómo priorizas features de ML?

Respuesta:

RICE Score:
- **R**each: ¿Cuántos usuarios afecta?
- **I**mpact: ¿Cuánto mejora métricas?
- **C**onfidence: ¿Qué tan seguros estamos?
- **E**ffort: ¿Cuánto trabajo requiere?

Score = (R × I × C) / E

Pregunta 114: Remote Team Leadership

¿Cómo lideras equipos remotos?

Respuesta:

1. **Async by default**: Documentación > meetings
2. **Overlap hours**: 2-3 horas para sync
3. **Clear ownership**: Cada task tiene responsable
4. **Over-communication**: Status updates frecuentes
5. **Trust + accountability**: Medir outcomes, no horas

Pregunta 115: Career Growth Path

¿Cómo defines el growth path para ML Engineers?

Respuesta:

```
Junior ML Engineer
  ↓ (1-2 años)
ML Engineer
  ↓ (2-3 años)
Senior ML Engineer
  ↓ (2-4 años)
  ├→ Staff ML Engineer (IC track)
  └→ ML Engineering Manager (Management track)
```

Skills por nivel:
- **Junior**: Implementar pipelines existentes
- **Mid**: Diseñar nuevos pipelines
- **Senior**: Arquitectura end-to-end, mentoring
- **Staff**: Cross-team influence, technical vision

📚 Resumen y Recursos

Skills Demostrados en Este Portafolio

Área	Evidencia
ML Fundamentals	3 proyectos: clasificación, regresión, ensemble
MLOps	MLflow, DVC, CI/CD, monitoring
Software Engineering	Pydantic, tests, modular design
DevOps	Docker, K 8 s, Terraform
Leadership	Documentation, decisions, trade-offs

Preparación Adicional Recomendada

1. **System Design**: Diseñar ML system de principio a fin
2. **Coding Interview**: LeetCode medium (estructuras de datos)
3. **Behavioral**: STAR method para experiencias pasadas
4. **Deep Dive**: Estar listo para explicar CUALQUIER línea de código

Fin del Simulacro de Entrevista

Generado basado en análisis exhaustivo del portafolio ML-MLOps

GLOSARIO

GUÍA MLOPS V5.0

PORTFOLIO EDITION

MÓDULO 21: GLOSARIO COMPLETO MLOps

Diccionario Exhaustivo de A-Z con Explicaciones Profundas, Analogías y Ejemplos

Guía MLOps v5.0: Senior Edition | DuqueOM | Diciembre 2025

📖 MÓDULO 21: Glosario Completo MLOps

Diccionario Exhaustivo con Explicaciones Profundas, Analogías y Ejemplos del Portafolio

"Dominar el vocabulario técnico es el primer paso para comunicarte como Senior."

Nivel	Duración
📚 Referencia	Consulta continua

📚 Introducción

Este glosario define **todos** los términos técnicos utilizados en la Guía MLOps v5.0 y en los proyectos del portafolio (BankChurn, CarVision, TelecomAI). Cada término incluye:

- **Definición técnica** precisa y completa
- **Explicación conceptual** para entender el "por qué"
- **Analogía desarrollada** para facilitar comprensión intuitiva
- **Ejemplo del portafolio** cuando aplica
- **Términos relacionados** para profundizar

Cómo usar este glosario

1. **Primera lectura**: Lee las analogías para captar la intuición
2. **Profundización**: Lee la explicación conceptual completa
3. **Aplicación**: Revisa los ejemplos del portafolio
4. **Conexión**: Explora los términos relacionados

A

Accuracy (Exactitud)

Definición técnica: Métrica de clasificación que mide el porcentaje de predicciones correctas sobre el total. Se calcula como `(TP + TN) / (TP + TN + FP + FN)` donde TP=True Positives, TN=True Negatives, FP=False Positives, FN=False Negatives.

Explicación conceptual: Accuracy responde a la pregunta "¿qué porcentaje de mis predicciones fueron correctas?". Es intuitiva pero **peligrosamente engañosa** con clases desbalanceadas. Si el 95% de tus clientes NO abandonan (no-churn), un modelo que siempre predice "no-churn" tiene 95% accuracy pero es completamente inútil para detectar churners.

Analogía desarrollada: Imagina un arquero que dispara 100 flechas a un blanco. Si 85 dan en el blanco, su accuracy es 85%. Pero si el blanco ocupa el 95% del muro, incluso disparando con los ojos cerrados acertarías 95%. Por eso en ML usamos métricas adicionales (Precision, Recall) que nos dicen qué tan bien acertamos a cada zona específica.

En el portafolio: BankChurn tiene ~20% de churners. Un modelo "dummy" que siempre predice "no-churn" tendría 80% accuracy. Por eso usamos ROC-AUC (86%) y Recall como métricas principales.

Relacionados: Precision, Recall, F1 Score, ROC-AUC, Class Imbalance

ADR (Architecture Decision Record)

Definición técnica: Documento estructurado que registra una decisión de arquitectura significativa junto con su contexto, las alternativas consideradas, la decisión tomada y sus consecuencias (positivas y negativas).

Explicación conceptual: En proyectos de software, tomamos cientos de decisiones técnicas. Meses después, nadie recuerda por qué se eligió PostgreSQL en vez de MongoDB, o por qué el modelo usa RandomForest y no XGBoost. Los ADRs resuelven esto: son la "memoria institucional" del proyecto. Siguen un formato estándar (Estado, Contexto, Decisión, Consecuencias) que facilita la lectura y búsqueda.

Analogía desarrollada: Piensa en un ADR como el acta de una reunión de arquitectos. Años después de construir un edificio, si alguien pregunta "¿por qué las vigas son de acero y no de madera?", el acta explica: "En 2020, consideramos madera (más barata) y acero (más resistente). Elegimos acero porque el edificio está en zona sísmica. Consecuencia: costo 20% mayor pero certificación antisísmica garantizada."

Ejemplo del portafolio:

```
# ADR-001: Uso de RandomForest sobre XGBoost

## Estado: Aceptado

## Contexto
Necesitamos un modelo de clasificación para churn que sea interpretable
para el equipo de negocio y robusto sin tuning extensivo.

## Decisión
Usamos RandomForestClassifier con class_weight='balanced'.

## Consecuencias
+ Feature importances nativas (explicabilidad)
+ Robusto sin hiperparámetro tuning complejo
- Puede perder 1-2% AUC vs XGBoost optimizado
```

Relacionados: ML Canvas, C4 Model, Documentación, DECISIONES_TECH.md

API (Application Programming Interface)

Definición técnica: Contrato que define cómo dos sistemas de software se comunican. Especifica los endpoints disponibles, los formatos de entrada/salida, los métodos HTTP soportados y los códigos de respuesta. En MLOps, las APIs REST son el mecanismo principal para exponer modelos ML como servicios consumibles.

Explicación conceptual: Un modelo ML entrenado es solo un archivo (.pkl, .joblib). Para que sea útil, otros sistemas deben poder enviarle datos y recibir predicciones. Una API actúa como la "ventana al mundo" del modelo: recibe requests HTTP con datos del cliente, los valida, los pasa al modelo, y devuelve la predicción en formato estructurado (JSON). Esto desacopla el modelo de los consumidores: la app móvil, el dashboard, el sistema de CRM pueden todos usar la misma API sin conocer los detalles internos del modelo.

Analogía desarrollada: Una API es como el mesero de un restaurante. Tú (el cliente) no entras a la cocina a preparar tu comida (no cargas el modelo en tu código). En su lugar, le dices al mesero qué quieres (envías un request), él lleva el pedido a la cocina (la API invoca al modelo), y te trae el plato preparado (la API devuelve la predicción). El menú es la documentación de la API: te dice qué puedes pedir y cómo.

Ejemplo del portafolio (BankChurn FastAPI):

```
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel, Field

class PredictionRequest(BaseModel):
    CreditScore: int = Field(..., ge=300, le=850)
    Age: int = Field(..., ge=18, le=100)
    Balance: float = Field(..., ge=0)
    # ... más features

class PredictionResponse(BaseModel):
    prediction: int
    probability: float
    risk_level: str

@app.post("/predict", response_model=PredictionResponse)
async def predict(request: PredictionRequest):
    df = pd.DataFrame([request.model_dump()])
    proba = model.predict_proba(df)[0, 1]
    return PredictionResponse(
        prediction=int(proba > 0.5),
        probability=proba,
        risk_level="high" if proba > 0.7 else "medium" if proba > 0.3 else "low"
    )
```

Relacionados: REST, FastAPI, Endpoint, HTTP, Pydantic, OpenAPI/Swagger

Artefacto (Artifact)

Definición técnica: Cualquier archivo generado durante el ciclo de vida de ML que necesita ser versionado, almacenado y potencialmente reproducido. Incluye: modelos serializados (.pkl, .joblib, .onnx), datasets procesados, gráficos de evaluación, reportes de métricas, logs de entrenamiento, y configuraciones.

Explicación conceptual: Un proyecto ML no es solo código—genera "productos intermedios" en cada etapa. El dataset limpio es un artefacto. El modelo entrenado es un artefacto. El reporte de métricas es un artefacto. La gestión

profesional de artefactos permite: (1) reproducibilidad—volver a cualquier versión anterior, (2) trazabilidad—saber qué datos y código produjeron qué modelo, (3) colaboración—compartir resultados entre equipos.

Analogía desarrollada: Piensa en una fábrica de autos. Los planos son artefactos (código). Las piezas moldeadas son artefactos (datasets procesados). El motor ensamblado es un artefacto (modelo entrenado). El auto terminado es un artefacto (pipeline completo). Cada pieza tiene un número de serie y registro de qué máquina la produjo, cuándo, con qué materiales. Si un auto tiene un defecto, puedes rastrear hacia atrás hasta encontrar la pieza defectuosa y qué lote de materiales causó el problema.

Ejemplo del portafolio:

```
artifacts/
├── model.joblib          # Modelo serializado (pipeline completo)
├── metrics.json          # {"roc_auc": 0.86, "recall": 0.75}
├── feature_importance.png # Gráfico de importancia
└── training_config.yaml  # Configuración usada
```

Relacionados: MLflow, Model Registry, DVC, Reproducibilidad

ASGI (Asynchronous Server Gateway Interface)

Definición: Especificación para servidores web async en Python. Maneja múltiples requests concurrentemente.

Analogía: Mesero que anota pedido mesa 1, mientras espera va a mesa 2, etc. Maneja conversaciones "en paralelo".

Relacionados: Uvicorn, FastAPI, Async/Await

AUC-ROC

Definición: Área bajo curva ROC. Mide capacidad de distinguir clases. 1.0 = perfecto, 0.5 = aleatorio.

Analogía: Separando manzanas buenas de malas. AUC 0.9 = 90% de las veces asigna mayor score a la manzana buena.

Relacionados: ROC Curve, Precision, Recall, Threshold

Auto-scaling

Definición: Sistema que aumenta/disminuye recursos automáticamente según demanda.

Analogía: Restaurante contratando meseros temporales cuando hay mucha gente.

Relacionados: HPA, Kubernetes, Load Balancer

B

Backpropagation

Definición: Algoritmo de entrenamiento de redes neuronales que propaga el error hacia atrás calculando gradientes.

Analogía: Equipo de relevos donde analizas hacia atrás quién contribuyó al fallo.

Relacionados: Gradient Descent, Learning Rate, Neural Network

Baseline

Definición: Modelo simple como referencia. Si tu modelo complejo no lo supera, algo está mal.

Analogía: Antes de comprar auto deportivo, verifica que sea más rápido que tu bicicleta.

Relacionados: Benchmark, Model Evaluation

BaseEstimator

Definición: Clase base sklearn con `get_params()` y `set_params()`. Todos los estimadores heredan de ella.

Analogía: Contrato estándar que todos los constructores deben seguir para que el sistema funcione.

Relacionados: TransformerMixin, Custom Transformer, Pipeline

Batch Prediction

Definición: Procesar múltiples muestras a la vez, programadamente. Contrasta con online/real-time.

Analogía: Catering (cocinas todo de antemano) vs restaurante a la carta (cocinas cada plato al pedirlo).

Relacionados: Online Prediction, Latencia

Black

Definición: Formateador Python opinionado. Aplica estilo consistente automáticamente.

Analogía: Corrector que arregla gramática y estilo sin preguntarte.

```
black src/
```

Relacionados: Linting, Flake 8 , isort

Branch (Rama)

Definición: Línea de desarrollo paralela en Git.

Analogía: Fotocopia del manuscrito para probar final alternativo sin afectar original.

```
git checkout -b feature/add-mlflow
```

Relacionados: Git, Merge, Pull Request

C

C4 Model

Definición: Visualización de arquitectura en 4 niveles: Context, Container, Component, Code.

Analogía: Google Maps con zoom. Mundo → País → Ciudad → Calle.

Relacionados: ADR, Arquitectura

CI/CD

Definición: Continuous Integration (tests automáticos) + Continuous Deployment (deploy automático).

Analogía: Fábrica con control de calidad automatizado que envía autos aprobados al concesionario.

Relacionados: GitHub Actions, Pipeline, DevOps

Classification

Definición: Problema ML supervisado para predecir categorías discretas.

Analogía: Doctor diagnosticando enfermedades (multiclase) o decidiendo operar/no operar (binaria).

Relacionados: Regression, Supervised Learning

Class Imbalance (Desbalance de Clases)

Definición técnica: Situación donde una o más clases están significativamente subrepresentadas en el dataset de entrenamiento. Ratios como 95:5, 99:1 o peores son comunes en problemas reales (fraude, churn, enfermedades raras).

Explicación conceptual: Los algoritmos de ML optimizan métricas globales. Si el 95% de tus datos son "no-fraude", el modelo aprende que la estrategia más "segura" es predecir siempre "no-fraude"—obtiene 95% accuracy haciendo nada útil. El desbalance es quizás el problema más común y subestimado en ML aplicado. Afecta tanto al entrenamiento (el modelo no ve suficientes ejemplos de la clase minoritaria) como a la evaluación (accuracy es engañosa).

Analogía desarrollada: Imagina entrenar un perro buscador de trufas dándole 1000 piedras y solo 10 trufas. El perro aprende rápidamente que decir "piedra" le da premio el 99% de las veces. Nunca aprende realmente a oler trufas. Para entrenarlo bien, necesitas: (1) darle más trufas (oversampling), (2) penalizarlo más cuando falla una trufa (class weights), o (3) medir su éxito por trufas encontradas, no por piedras correctamente ignoradas (métricas apropiadas).

Soluciones técnicas:

```
# 1. Class weights (penaliza más errores en clase minoritaria)
RandomForestClassifier(class_weight='balanced')

# 2. SMOTE (genera ejemplos sintéticos de clase minoritaria)
from imblearn.over_sampling import SMOTE
X_resampled, y_resampled = SMOTE().fit_resample(X, y)

# 3. Threshold adjustment (bajar umbral de decisión)
proba = model.predict_proba(X)[:, 1]
predictions = (proba > 0.3).astype(int)  # En vez de 0.5

# 4. Métricas apropiadas
from sklearn.metrics import recall_score, roc_auc_score
# NO usar accuracy como métrica principal
```

En el portafolio: BankChurn tiene ~20% churners. Usamos `class_weight='balanced'` y priorizamos Recall sobre Accuracy.

Relacionados: class_weight, SMOTE, Recall, Precision, ROC-AUC, Threshold

class_weight

Definición técnica: Parámetro de sklearn que asigna pesos diferentes a las clases durante el entrenamiento. Con `class_weight='balanced'`, los pesos se calculan automáticamente como inversamente proporcionales a la frecuencia de cada clase.

Explicación conceptual: Es la forma más simple de manejar desbalance. En lugar de modificar los datos (oversampling/undersampling), modificamos cómo el modelo "valora" los errores. Un error en la clase minoritaria "cuenta más" que un error en la clase mayoritaria. Matemáticamente, es como si tuviéramos más ejemplos de la clase minoritaria sin realmente duplicarlos.

Fórmula: `weight[i] = n_samples / (n_classes * n_samples_i)`

Ejemplo del portafolio:

```
# BankChurn: 80% no-churn, 20% churn
# Sin class_weight: modelo ignora churners
# Con class_weight='balanced': churners valen 4x más

from sklearn.ensemble import RandomForestClassifier

model = RandomForestClassifier(
    n_estimators=100,
    class_weight='balanced',  # Crítico para churn
    random_state=42
)
```

Relacionados: Class Imbalance, SMOTE, RandomForest

Cold Start

Definición: Tiempo para que servicio esté listo tras iniciarse. Incluye cargar modelo en memoria.

Analogía: Encender auto en invierno. Debes esperar que el motor se caliente.

Relacionados: Serverless, Lambda, Latencia

ColumnTransformer

Definición: Sklearn: aplica diferentes transformaciones a diferentes columnas.

Analogía: Lavandería con máquinas diferentes: color→encoder, blanca→scaler, delicados→passthrough.

```
preprocessor = ColumnTransformer([
    ('num', StandardScaler(), num_cols),
    ('cat', OneHotEncoder(), cat_cols),
])
```

Relacionados: Pipeline, Transformer

Commit

Definición: Snapshot de cambios en Git con hash único y mensaje.

Analogía: Foto de tu escritorio. Puedes volver a cualquier foto anterior.

```
git commit -m "feat: add probability calibration"
```

Relacionados: Git, Branch, Push, Conventional Commits

conftest.py

Definición técnica: Archivo especial de pytest que contiene fixtures (funciones que proveen datos/recursos) compartidas entre todos los tests del directorio y subdirectorios. pytest lo descubre automáticamente sin necesidad de imports.

Explicación conceptual: Los tests necesitan datos de prueba, conexiones a bases de datos mock, modelos pre-entrenados, etc. Sin conftest.py, cada archivo de tests tendría que definir o importar estos recursos. conftest.py centraliza esta lógica: defines las fixtures una vez, y están disponibles automáticamente en todos los tests. Es el "almacén central de recursos de testing".

Analogía desarrollada: Imagina un set de filmación. Antes de cada escena, alguien prepara el escenario: pone las luces, coloca los props, prepara el vestuario. conftest.py es ese equipo de preparación. Los actores (tests) llegan y todo está listo. No tienen que traer sus propios props—solo los piden por nombre y aparecen.

Ejemplo del portafolio (CarVision):

```
# tests/conftest.py
import pytest
import pandas as pd
import numpy as np

@pytest.fixture
def sample_data():
    """Datos sintéticos para tests."""
    np.random.seed(42)
    return pd.DataFrame({
        'year': np.random.randint(2010, 2023, 100),
        'mileage': np.random.randint(10000, 150000, 100),
        'price': np.random.uniform(5000, 50000, 100),
    })

@pytest.fixture
def trained_pipeline(sample_data):
    """Pipeline entrenado para tests de inferencia."""
    from carvision.pipeline import build_pipeline
    pipe = build_pipeline()
    X = sample_data.drop('price', axis=1)
    y = sample_data['price']
    return pipe.fit(X, y)

@pytest.fixture
def config():
    """Configuración de test."""
    return {'model': {'n_estimators': 10}, 'random_state': 42}
```

Relacionados: pytest, Fixture, Unit Test, Integration Test

Conventional Commits

Definición técnica: Especificación para escribir mensajes de commit estandarizados. Formato: `<type>(<scope>): <description>`. Types incluyen: feat, fix, docs, style, refactor, test, chore.

Explicación conceptual: Los mensajes de commit son la historia del proyecto. "fixed bug" o "updates" no dicen nada útil. Conventional Commits impone estructura: el tipo indica qué cambió (feature nueva, bug fix, documentación), el scope indica dónde (api, pipeline, tests), la descripción explica qué. Esto permite: (1) generar CHANGELOGs automáticamente, (2) determinar versiones semánticas, (3) entender la historia del proyecto rápidamente.

Analogía desarrollada: Imagina un libro de bitácora de un barco. "Navegamos" no ayuda. "2024-01-15 14:00 - Cambio de rumbo: de Norte a Noroeste para evitar tormenta detectada a 50 km" es útil. Conventional Commits son esa bitácora estructurada para código.

Ejemplos del portafolio:

```
# Formato: <type>(<scope>): <description>

feat(api): add batch prediction endpoint
fix(pipeline): handle NaN values in categorical columns
docs(readme): add quick start guide and badges
test(training): add integration tests for cross-validation
refactor(features): extract FeatureEngineer to separate module
chore(deps): update scikit-learn to 1.3.0
```

Relacionados: Git, pre-commit, Semantic Versioning

Concept Drift

Definición: Cambio en relación features-target. Patrones aprendidos ya no son válidos.

Analogía: Modelo entrenado pre-pandemia predice gustos de películas post-pandemia incorrectamente.

vs Data Drift: Data Drift = cambia X. Concept Drift = cambia P(Y|X).

ConfigMap

Definición: Kubernetes: almacena configuración no sensible como pares clave-valor.

Analogía: Tablón de anuncios de oficina. Información pública que todos necesitan.

Container (Contenedor)

Definición: Software empaquetado con código y dependencias. Ejecuta igual en cualquier ambiente.

Analogía: Contenedor de barco. Funciona igual en cualquier puerto.

Relacionados: Docker, Image, Kubernetes

Coverage

Definición: Porcentaje de código ejecutado por tests. No garantiza corrección.

Analogía: Inspector que revisó 80% de habitaciones. No significa que encontró todos los problemas.

```
pytest --cov=src
```

Target: > 80% para código crítico

Cross-Validation

Definición: Evaluar modelo dividiendo datos en K folds. Entrena K veces con diferentes splits.

Analogía: 5 estudiantes, 5 rondas. En cada ronda, diferente estudiante es evaluado.

```
scores = cross_val_score(model, X, y, cv=5, scoring='roc_auc')
```

Relacionados: K-Fold, Overfitting

Custom Transformer

Definición: Clase sklearn personalizada que hereda BaseEstimator + TransformerMixin.

Analogía: Pieza LEGO personalizada con conexiones estándar (fit/transform).

```
class RatioFeatures(BaseEstimator, TransformerMixin):
    def fit(self, X, y=None): return self
    def transform(self, X):
        X['ratio'] = X['Balance'] / (X['Products'] + 1)
        return X
```

D

DAG (Directed Acyclic Graph)

Definición: Grafo dirigido sin ciclos. Representa dependencias entre tareas.

Analogía: Instrucciones de receta. No puedes hornear antes de mezclar.

Relacionados: DVC, Pipeline, Airflow

Data Drift

Definición técnica: Cambio en la distribución estadística de las features (P(X)) entre el momento del entrenamiento y la inferencia en producción. No implica necesariamente que la relación feature-target haya cambiado, solo que los datos de entrada son diferentes.

Explicación conceptual: Tu modelo fue entrenado con datos de 2023. Llega 2025 y los patrones de los clientes han cambiado: son más jóvenes, usan más canales digitales, tienen balances diferentes. Aunque la "lógica" de qué causa churn no haya cambiado, tu modelo recibe inputs que nunca vio y puede fallar. Data drift es como un médico entrenado solo con pacientes adultos intentando diagnosticar niños—la anatomía es diferente aunque las enfermedades sean las mismas.

Analogía desarrollada: Imagina un modelo que predice si lloverá basándose en la presión atmosférica. Fue entrenado en Madrid. Lo despliegas en Ciudad de México (altitud muy diferente). La presión "normal" en CDMX es mucho menor que en Madrid. El modelo ve presiones que interpreta como "muy baja" y siempre predice lluvia. No es que el modelo esté roto—es que los datos de entrada son muy diferentes a los de entrenamiento.

Tipos de drift:
- **Covariate shift**: Cambia P(X), pero P(Y|X) permanece igual
- **Prior probability shift**: Cambia P(Y), la proporción de clases
- **Concept drift**: Cambia P(Y|X), la relación misma

Detección técnica:

```
# Kolmogorov-Smirnov test para cada feature
from scipy.stats import ks_2samp

for col in features:
    stat, pvalue = ks_2samp(train_data[col], prod_data[col])
    if pvalue < 0.05:
        print(f"Drift detectado en {col}: KS={stat:.3f}, p={pvalue:.4f}")
```

```
# Population Stability Index (PSI)
# PSI < 0.1: No drift
# PSI 0.1-0.2: Drift moderado
# PSI > 0.2: Drift significativo
```

Herramientas: Evidently, NannyML, Great Expectations

Relacionados: Concept Drift, Model Monitoring, Evidently, Retraining

Data Leakage

Definición: Información del futuro o test filtra al entrenamiento. Métricas infladas.

Analogía: Estudiar con las respuestas del mismo examen. 100% en práctica, 0% en real.

Ejemplos: `price_per_mile = price / miles`, normalizar antes de split.

Dependency Injection

Definición: Dependencias se pasan desde afuera en lugar de crearse internamente.

Analogía: Cafetería recibe leche de proveedor en vez de tener vacas propias.

```
# Con DI: fácil de testear
class Predictor:
    def __init__(self, model: BaseEstimator):
        self.model = model  # Inyectado
```

Relacionados: SOLID, Testing

Deployment

Definición: Poner modelo/aplicación en ambiente donde usuarios reales lo usan.

Analogía: Abrir restaurante al público después de cocinar en casa y probar con amigos.

Tipos: Batch, REST API, Edge, Streaming

Docker

Definición: Plataforma para aplicaciones en contenedores. Código + dependencias portables.

Analogía: Máquina del tiempo para código. Congelas ambiente exacto.

```
FROM python:3.11-slim
COPY . /app
RUN pip install -r requirements.txt
CMD ["uvicorn", "app:app"]
```

Docstring

Definición: String de documentación al inicio de funciones/clases.

Analogía: Instrucciones en la caja de un producto.

```
def predict(data: pd.DataFrame) -> np.ndarray:
    """
    Genera predicciones de churn.

    Args:
        data: DataFrame con features.
    Returns:
        Array de probabilidades 0-1.
    """
```

DVC (Data Version Control)

Definición: Versiona datasets y pipelines ML. Datos grandes en storage remoto, metadatos en Git.

Analogía: Git = álbum con miniaturas. DVC = almacén con fotos originales grandes.

```
dvc add data/dataset.csv
dvc push
git add data/dataset.csv.dvc
```

E

E2E Test

Definición: Test del sistema completo, desde entrada hasta salida final.

Analogía: Test drive de auto completo, no motor aislado.

Early Stopping

Definición: Detiene entrenamiento cuando validación deja de mejorar. Evita overfitting.

Analogía: Sacar galletas del horno cuando están doradas, antes de que se quemen.

```
EarlyStopping(monitor='val_loss', patience=5)
```

Embedding

Definición: Representación vectorial densa de datos de alta dimensionalidad.

Analogía: Mapear ciudades del mundo en papel 2D. Similares quedan cerca.

Uso: Word2Vec, Entity embeddings

Endpoint

Definición: URL específica de API que realiza operación particular.

Analogía: Ventanillas de banco. Cada una hace algo diferente.

```
@app.get("/health")
@app.post("/predict")
```

Ensemble

Definición: Combina múltiples modelos para mejores predicciones.

Analogía: 100 doctores opinando en vez de 1. Opinión agregada suele ser mejor.

Tipos: Bagging (Random Forest), Boosting (XGBoost), Stacking

Environment

Definición: Conjunto aislado de dependencias donde ejecuta código.

Analogía: Diferentes cocinas para diferentes tipos de comida.

Tipos: Desarrollo, Staging, Producción

Evidently

Definición técnica: Librería open-source de Python para monitoreo de modelos ML en producción. Genera reportes interactivos de data drift, target drift, data quality, y performance del modelo comparando datasets de referencia con datasets actuales.

Explicación conceptual: Cuando despliegas un modelo, necesitas saber si sigue funcionando bien. Evidently automatiza esta vigilancia: compara los datos que ve el modelo en producción con los datos de entrenamiento, detecta cambios estadísticos (drift), genera alertas, y produce reportes visuales. Es como tener un "chequeo médico" continuo para tu modelo.

Analogía desarrollada: Imagina que tienes un carro. Evidently es el tablero de instrumentos que te dice si la presión de las llantas bajó, si el aceite necesita cambio, si el motor está sobrecalentando. No esperas a que el carro se descomponga—el tablero te avisa antes de que el problema sea grave.

Ejemplo práctico:

```
from evidently.report import Report
from evidently.metric_preset import DataDriftPreset, DataQualityPreset

# Comparar datos de training vs producción
report = Report(metrics=[
    DataDriftPreset(),
    DataQualityPreset(),
])

report.run(
    reference_data=train_df,
    current_data=production_df
)

# Generar reporte HTML interactivo
```

```
report.save_html("drift_report.html")

# O extraer métricas programáticamente
drift_results = report.as_dict()
if drift_results['metrics'][0]['result']['dataset_drift']:
    print("⚠️ Drift significativo detectado!")
```

Capacidades:
- Data Drift: Detecta cambios en distribuciones de features
- Target Drift: Detecta cambios en distribución del target
- Data Quality: Valores faltantes, outliers, correlaciones
- Model Performance: Accuracy, precision, recall en producción
- Regression Performance: MAE, RMSE, error distribution

Relacionados: Data Drift, Model Monitoring, Observabilidad, NannyML

Experiment Tracking

Definición: Registrar parámetros, métricas, artefactos de cada experimento ML.

Analogía: Cuaderno de laboratorio de científico.

Herramientas: MLflow, W&B, Neptune

F

F1 Score

Definición: Media armónica de Precision y Recall. Balance entre ambas.

Fórmula: `F1 = 2 × (Precision × Recall) / (Precision + Recall)`

Analogía: Buscador de trufas. No sirve encontrar pocas muy precisamente ni todas con muchas falsas.

FastAPI

Definición: Framework Python para APIs de alto rendimiento con validación automática.

Analogía: Mesero eficiente que valida pedidos, da menú descriptivo, atiende muchas mesas.

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

@app.post("/predict")
async def predict(data: PredictionInput):
    return {"probability": model.predict_proba([data])[0, 1]}
```

Relacionados: Pydantic, Uvicorn, REST

Feature

Definición: Variable de entrada para predicciones.

Analogía: Ingredientes de receta. Para predecir si pastel sale bien: harina, azúcar, temperatura.

Tipos: Numéricas, Categóricas, Binarias, Derivadas

Feature Engineering

Definición: Crear/transformar/seleccionar features para mejorar modelo.

Analogía: Chef preparando ingredientes. Ingredientes crudos se transforman en algo digerible.

```
df['balance_per_product'] = df['Balance'] / (df['NumOfProducts'] + 1)
```

Feature Store

Definición: Sistema centralizado para almacenar y servir features consistentemente.

Analogía: Almacén central de ingredientes preparados para cadena de restaurantes.

Herramientas: Feast, Tecton

Fixture (pytest)

Definición: Función que provee datos/recursos reutilizables para tests.

Analogía: Setup de set de filmación antes de cada escena.

```
@pytest.fixture
def sample_customer():
    return {"Age": 35, "Balance": 50000}
```

Flake8

Definición: Linting para Python: errores lógicos, estilo PEP 8 , complejidad.

Analogía: Corrector de estilo de periódico.

```
flake8 src/
```

G

Git

Definición: Control de versiones distribuido.

Analogía: "Deshacer" infinito. Volver a cualquier momento, ver qué cambió y por qué.

```
git add . && git commit -m "mensaje" && git push
```

GitHub Actions

Definición: CI/CD integrado en GitHub.

Analogía: Mayordomo robot que ejecuta instrucciones automáticamente.

```
on: [push]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - run: pytest tests/
```

Gitleaks

Definición: Detecta secrets accidentalmente commiteados.

Analogía: Detector de metales en aeropuerto para código.

Gradient Descent

Definición: Algoritmo que encuentra parámetros que minimizan pérdida.

Analogía: En montaña con niebla, das pasos pequeños siempre cuesta abajo.

Relacionados: Learning Rate, Loss Function

Grafana

Definición: Visualización y dashboards para métricas.

Analogía: Tablero de instrumentos de avión.

Relacionados: Prometheus, Observabilidad

H

Health Check

Definición: Endpoint que verifica si servicio funciona.

Analogía: Médico preguntando "¿cómo te sientes?".

```
@app.get("/health")
def health():
    return {"status": "healthy"}
```

HPA (Horizontal Pod Autoscaler)

Definición: Kubernetes: escala pods automáticamente según métricas.

Analogía: Gerente de restaurante que llama más meseros si hay muchas mesas ocupadas.

Hyperparameter

Definición: Parámetro configurado ANTES del entrenamiento.

Analogía: Decisiones antes de hornear: temperatura, tiempo, tamaño de molde.

Ejemplos: n_estimators, learning_rate, max_depth

Hyperparameter Tuning

Definición: Encontrar combinación óptima de hiperparámetros.

Analogía: Afinar guitarra. Probar perillas hasta mejor sonido.

Técnicas: Grid Search, Random Search, Bayesian Optimization

I

Image (Docker)

Definición: Template inmutable para crear contenedores.

Analogía: Receta + ingredientes pre-empaquetados. Imagen es el kit, contenedor es el pastel horneado.

Imputer

Definición: Rellena valores faltantes (NaN).

Analogía: Restaurador de pinturas rellenando huecos.

```
SimpleImputer(strategy='median')
```

Inference

Definición: Usar modelo entrenado para predicciones sobre datos nuevos.

Analogía: Entrenamiento = estudiar. Inferencia = tomar el examen.

Ingress

Definición: Kubernetes: gestiona acceso HTTP externo al cluster.

Analogía: Recepción de edificio que dirige tráfico.

Integration Test

Definición: Verifica que múltiples componentes funcionan juntos.

Analogía: Probar que motor, transmisión y ruedas funcionan juntos.

isort

Definición: Ordena imports de Python automáticamente.

Analogía: Organizador de armario que siempre pone ropa en mismo orden.

J

Job (GitHub Actions)

Definición: Conjunto de steps en mismo runner.

Relacionados: Workflow, Step, Runner

Joblib

Definición: Serializa objetos Python, especialmente modelos sklearn.

```
joblib.dump(model, "model.pkl")
model = joblib.load("model.pkl")
```

K

Kubernetes (K8s)

Definición: Orquestador de contenedores para automatizar despliegue y escalado.

Analogía: Director de orquesta coordinando muchos músicos (contenedores).

Recursos: Pod, Deployment, Service, Ingress

K-Fold

Definición: Dividir datos en K partes para cross-validation.

Relacionados: Cross-Validation, Stratified

L

Latency (Latencia)

Definición: Tiempo de respuesta del sistema. En APIs ML: milisegundos.

Analogía: Tiempo entre pedir comida y que llegue.

P_{95}: El 95% de requests responden en menos de X ms.

Learning Rate

Definición: Tamaño de paso en gradient descent.

Analogía: Paso grande = llegas rápido pero puedes pasar el mínimo. Paso pequeño = lento pero preciso.

Linting

Definición: Análisis estático para detectar errores y violaciones de estilo.

Herramientas: Flake8, pylint, mypy

Load Balancer

Definición: Distribuye tráfico entre múltiples servidores.

Analogía: Hostess de restaurante que asigna mesas equitativamente.

Loss Function (Función de Pérdida)

Definición: Mide qué tan mal son las predicciones. El entrenamiento la minimiza.

Ejemplos: MSE (regresión), Cross-Entropy (clasificación)

M

Makefile

Definición: Archivo con comandos abreviados para tareas comunes.

```
test:
    pytest tests/ -v
lint:
    black src/ && flake8 src/
```

Matrix (GitHub Actions)

Definición: Ejecutar job con múltiples combinaciones de parámetros.

```
strategy:
  matrix:
    python-version: [3.10, 3.11]
```

Metric (Métrica)

Definición: Valor numérico que mide rendimiento del modelo.

Clasificación: Accuracy, Precision, Recall, F_1, AUC
Regresión: MSE, RMSE, MAE, R^2

Middleware

Definición: Código que intercepta requests/responses entre cliente y aplicación.

Analogía: Portero que revisa credenciales antes de dejarte pasar.

MLflow

Definición: Plataforma open-source para gestionar ciclo de vida ML.

Componentes: Tracking, Projects, Models, Registry

```
with mlflow.start_run():
    mlflow.log_params(params)
    mlflow.log_metrics(metrics)
    mlflow.sklearn.log_model(model, "model")
```

MLOps

Definición técnica: Conjunto de prácticas que unifican Machine Learning, DevOps y Data Engineering para automatizar y estandarizar el ciclo de vida completo de modelos ML: desde experimentación hasta producción, incluyendo monitoreo, reentrenamiento y gobernanza.

Explicación conceptual: Data Scientists saben entrenar modelos. DevOps sabe desplegar aplicaciones. Data Engineers saben mover datos. MLOps es el puente que conecta estos tres mundos. Sin MLOps, tienes "modelos en notebooks" que nunca llegan a producción, o modelos desplegados que nadie monitorea y se degradan silenciosamente. MLOps trae madurez industrial al ML.

Analogía desarrollada: Imagina que los Data Scientists son chefs que crean recetas increíbles en su cocina experimental. DevOps es el equipo que opera restaurantes a escala. MLOps es el proceso que convierte esa receta

experimental en un menú estandarizado, con control de calidad, ingredientes versionados, y alertas si la calidad baja. Sin MLOps, tienes un chef genial cuyas recetas nadie puede reproducir consistentemente.

Pilares de MLOps:
1. **Versionado**: Código (Git), Datos (DVC), Modelos (MLflow)
2. **Automatización**: CI/CD, pipelines de entrenamiento
3. **Testing**: Datos, modelos, APIs, integración
4. **Monitoreo**: Drift, performance, latencia
5. **Reproducibilidad**: Ambientes, seeds, configuraciones

Relacionados: DevOps, CI/CD, MLflow, DVC, Model Monitoring

Multi-stage Build (Docker)

Definición técnica: Técnica de construcción de imágenes Docker que usa múltiples `FROM` statements, permitiendo separar el ambiente de compilación/build del ambiente de ejecución. El resultado es una imagen final más pequeña y segura que solo contiene lo necesario para ejecutar la aplicación.

Explicación conceptual: Cuando construyes una aplicación, necesitas herramientas de compilación, tests, dependencias de desarrollo. Pero en producción, solo necesitas el binario final y las dependencias runtime. Multi-stage te permite "cocinar" en una cocina completa y luego servir solo el plato terminado, sin llevar todos los utensilios al comedor.

Analogía desarrollada: Imagina construir un mueble IKEA. Necesitas martillo, destornillador, nivel, instrucciones, embalaje. Pero una vez terminado, solo quieres el mueble en tu sala—no el taller completo. Multi-stage es exactamente eso: usas un container "taller" con todas las herramientas, construyes, y luego copias solo el resultado final a un container "sala" limpio y minimalista.

Ejemplo del portafolio:

```
# ═══════════════════════════════════════════════════════
# STAGE 1: Builder - Tiene todas las herramientas
# ═══════════════════════════════════════════════════════
FROM python:3.11-slim AS builder

WORKDIR /app

# Instalar dependencias de compilación (solo en builder)
RUN apt-get update && apt-get install -y --no-install-recommends \
    build-essential \
    && rm -rf /var/lib/apt/lists/*

# Instalar dependencias Python en directorio aislado
COPY requirements.txt .
RUN pip install --no-cache-dir --target=/app/deps -r requirements.txt

# ═══════════════════════════════════════════════════════
# STAGE 2: Runtime - Solo lo necesario para ejecutar
# ═══════════════════════════════════════════════════════
FROM python:3.11-slim

# Usuario no-root (seguridad)
RUN useradd --create-home appuser

WORKDIR /app

# Copiar SOLO las dependencias instaladas (no el toolchain)
COPY --from=builder /app/deps /usr/local/lib/python3.11/site-packages/

# Copiar código de aplicación
COPY --chown=appuser:appuser src/ ./src/
COPY --chown=appuser:appuser app/ ./app/
COPY --chown=appuser:appuser artifacts/ ./artifacts/

USER appuser

EXPOSE 8000
CMD ["uvicorn", "app.fastapi_app:app", "--host", "0.0.0.0", "--port", "8000"]
```

Beneficios:
- **Imagen más pequeña**: De ~1.5 GB a ~500 MB
- **Más segura**: Sin compiladores ni herramientas de ataque
- **Más rápida de desplegar**: Menos bytes que transferir

Relacionados: Docker, Container, Dockerfile, Non-root User

Model Card

Definición: Documento describiendo modelo: propósito, datos, métricas, limitaciones, ética.

Analogía: Prospecto de medicamento. Información completa sobre qué hace y sus efectos.

Model Registry

Definición: Sistema para versionar y gestionar modelos ML.

Estados: Staging → Production → Archived

mypy

Definición: Type checking estático para Python.

```
mypy src/
```

Relacionados: Type Hints, Pydantic

N

NaN (Not a Number)

Definición: Valor especial para datos faltantes o indefinidos.

```
import numpy as np
np.nan
```

Namespace

Definición: Kubernetes: división lógica del cluster para aislamiento.

Analogía: Departamentos en una empresa. Cada uno tiene sus recursos.

O

Observability (Observabilidad)

Definición: Capacidad de entender estado interno de sistema desde outputs externos.

3 **Pilares:** Logs, Metrics, Traces

Analogía: Instrumentos de avión. Si no puedes ver, no puedes arreglar.

One-Hot Encoding

Definición: Convierte variables categóricas en vectores binarios.

```
Country: [France, Spain, Germany]
France → [1, 0, 0]
Spain  → [0, 1, 0]
```

Overfitting (Sobreajuste)

Definición: Modelo memoriza datos de entrenamiento, no generaliza.

Analogía: Estudiante que memoriza respuestas exactas pero no entiende conceptos.

Señales: Train accuracy muy alta, validation accuracy baja.

Soluciones: Más datos, regularización, early stopping, dropout

P

Pipeline (sklearn)

Definición: Secuencia de transformaciones y estimador final encadenados.

Analogía: Línea de ensamblaje. Cada estación hace una transformación.

```
pipeline = Pipeline([
    ('scaler', StandardScaler()),
    ('model', RandomForestClassifier())
])
```

Pod

Definición: Kubernetes: unidad de deployment más pequeña. Uno o más contenedores.

Analogía: Apartamento en edificio. Contenedores son habitaciones del apartamento.

Precision (Precisión)

Definición: De predicciones positivas, ¿cuántas son correctas? TP / (TP + FP)

Analogía: De las personas que detuviste como sospechosas, ¿cuántas eran realmente criminales?

Pre-commit Hook

Definición: Script que se ejecuta automáticamente antes de cada commit.

Analogía: Control de calidad que revisa tu trabajo antes de entregarlo.

```
# .pre-commit-config.yaml
repos:
  - repo: https://github.com/psf/black
    hooks:
      - id: black
```

Prometheus

Definición: Sistema de monitoreo y alertas. Recolecta métricas de servicios.

Relacionados: Grafana, Metrics, Observabilidad

Pull Request (PR)

Definición: Solicitud para integrar cambios con revisión de código.

Analogía: Propuesta formal que requiere aprobación antes de aceptarse.

Pydantic

Definición: Validación de datos en Python usando type hints.

```
class Customer(BaseModel):
    age: int = Field(ge=18, le=100)
    name: str
```

Relacionados: Type Hints, FastAPI, Validation

pytest

Definición: Framework de testing para Python.

```
def test_prediction():
    result = model.predict([[35, 50000]])
    assert result[0] in [0, 1]
```

R

Random Forest

Definición técnica: Algoritmo de ensemble learning que construye múltiples árboles de decisión durante el entrenamiento y combina sus predicciones (votación mayoritaria para clasificación, promedio para regresión). Cada árbol se entrena con un subconjunto aleatorio de datos (bagging) y features (random subspace).

Explicación conceptual: Un solo árbol de decisión puede sobreajustarse fácilmente y es muy sensible a pequeños cambios en los datos. Random Forest resuelve esto con la "sabiduría de las multitudes": entrena cientos de árboles "diversos" (cada uno ve datos diferentes) y promedia sus opiniones. Los errores individuales se cancelan, produciendo un modelo robusto y estable.

Analogía desarrollada: Imagina 100 doctores, cada uno especializado en diferentes aspectos (algunos ven más casos de ciertas enfermedades, otros atienden diferentes demografías). Si cada doctor da su diagnóstico individualmente, algunos acertarán y otros fallarán. Pero si los 100 votan y tomas la opinión mayoritaria, casi siempre aciertas. Eso es Random Forest: democracia de árboles donde los errores individuales se cancelan.

Por qué es popular en MLOps:
- **Interpretabilidad**: Feature importances nativas
- **Robustez**: Funciona bien "out of the box" sin tuning extensivo
- **Versatilidad**: Clasificación y regresión
- **Sin normalización**: No requiere escalar features

Ejemplo del portafolio (BankChurn):

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.pipeline import Pipeline

# Parámetros clave:
```

```
# - n_estimators: Número de árboles (más = más estable, más lento)
# - max_depth: Profundidad máxima (controla overfitting)
# - class_weight: Manejo de desbalance

pipeline = Pipeline([
    ('preprocessor', ColumnTransformer([
        ('num', SimpleImputer(strategy='median'), num_cols),
        ('cat', OneHotEncoder(handle_unknown='ignore'), cat_cols),
    ])),
    ('classifier', RandomForestClassifier(
        n_estimators=100,
        max_depth=10,
        class_weight='balanced',  # Crítico para churn
        random_state=42,
        n_jobs=-1  # Paralelizar
    ))
])

# Feature importance después de entrenar
importances = pipeline.named_steps['classifier'].feature_importances_
```

Hiperparámetros importantes:

Parámetro	Default	Efecto
n_estimators	100	Más árboles = más estable pero más lento
max_depth	None	Limitar previene overfitting
min_samples_split	2	Mayor valor = árboles más pequeños
class_weight	None	'balanced' para clases desbalanceadas

Relacionados: Ensemble, Bagging, Decision Tree, class_weight, Feature Importance

Recall (Sensibilidad)

Definición técnica: Métrica que mide qué proporción de los casos positivos reales fueron correctamente identificados. Fórmula: `TP / (TP + FN)`. También llamada Sensibilidad o True Positive Rate.

Explicación conceptual: Recall responde: "De todos los casos positivos reales, ¿cuántos logré detectar?". Es crítica cuando el costo de **no detectar** un positivo es alto: diagnóstico de cáncer (no detectar = paciente sin tratamiento), detección de fraude (no detectar = pérdida financiera), predicción de churn (no detectar = cliente perdido).

Analogía desarrollada: Imagina un detector de metales en un aeropuerto. Recall es: "De todas las armas reales que pasaron, ¿cuántas detectó?". Un Recall del 100% significa que detectó todas las armas (aunque haya generado muchas falsas alarmas con llaves y monedas). En seguridad, preferimos alta sensibilidad aunque suene más veces innecesariamente.

En el portafolio: BankChurn prioriza Recall porque el costo de no detectar un churner (perderlo) es mayor que el costo de ofrecerle retención a alguien que no iba a irse.

Trade-off Precision vs Recall:

```
                    Predicción
                    Positivo    Negativo
Realidad Positivo   TP          FN (Recall falla aquí)
         Negativo   FP          TN

Recall = TP / (TP + FN) → Maximizar TP, minimizar FN
```

Relacionados: Precision, F1 Score, Threshold, ROC-AUC

Regression (Regresión)

Definición: Problema ML para predecir valor numérico continuo.

Ejemplos: Precio de casa, temperatura, ventas

Regularization (Regularización)

Definición: Técnicas para prevenir overfitting penalizando complejidad.

Tipos: L1 (Lasso), L2 (Ridge), Dropout, Early Stopping

Replica

Definición: Copia de un pod/servicio para alta disponibilidad.

Relacionados: Deployment, ReplicaSet

Reproducibility (Reproducibilidad)

Definición: Obtener mismos resultados con mismo código y datos.

Clave: Seeds, versionado de datos/código/ambiente

REST API

Definición: Estilo arquitectónico con HTTP methods: GET, POST, PUT, DELETE.

Relacionados: API, HTTP, Endpoint

Runbook

Definición técnica: Documento operacional que contiene procedimientos paso a paso para manejar incidentes, alertas o tareas de mantenimiento de un sistema en producción. Incluye información del servicio, alertas comunes, y procedimientos de emergencia.

Contenido típico:
- Información del servicio (owner, criticidad, endpoints)
- Procedimientos para alertas comunes
- Comandos de diagnóstico y recuperación
- Escalamiento y contactos

En el portafolio: Ver 17_DESPLIEGUE.md → Operaciones y Runbooks.

Relacionados: SLO, SLA, Incident Response, On-call

Ruff

Definición técnica: Linter y formateador de código Python extremadamente rápido, escrito en Rust. Reemplaza múltiples herramientas (Flake8, Black, isort, pyupgrade, etc.) con una sola herramienta 10-100x más rápida.

Explicación conceptual: Tradicionalmente, un proyecto Python necesitaba múltiples herramientas para mantener la calidad del código: Black para formatear, Flake8 para detectar errores, isort para ordenar imports, pyupgrade para sintaxis moderna. Cada herramienta tenía su configuración, versión, y tiempo de ejecución. Ruff unifica todo esto: un solo binario que hace todo, instantáneamente. Es la herramienta moderna que está reemplazando al stack tradicional.

Analogía desarrollada: Imagina tener una navaja suiza en vez de cargar tijeras, destornillador, cuchillo y abridor por separado. Ruff es esa navaja suiza: todas las herramientas de calidad de código en una, y además es más ligera y rápida que cualquiera de las individuales.

Por qué importa:
- **Velocidad**: 10-100x más rápido que Flake8+Black+isort
- **Unificación**: Una herramienta, una configuración
- **Compatibilidad**: Entiende las reglas de Flake8, Black, isort
- **Moderno**: Soporta Python 3.12+, type hints, f-strings

Ejemplo de configuración (pyproject.toml):

```
[tool.ruff]
line-length = 88
target-version = "py311"

[tool.ruff.lint]
select = [
    "E",    # pycodestyle errors
    "W",    # pycodestyle warnings
    "F",    # Pyflakes
    "I",    # isort
    "B",    # flake8-bugbear
    "C4",   # flake8-comprehensions
    "UP",   # pyupgrade
]
ignore = ["E501"]  # Line too long (handled by formatter)

[tool.ruff.lint.isort]
known-first-party = ["bankchurn", "carvision", "telecomai"]
```

Uso:

```
# Lint (detectar errores)
ruff check src/

# Lint con auto-fix
ruff check --fix src/

# Format (como Black)
ruff format src/

# Pre-commit hook
# .pre-commit-config.yaml
repos:
  - repo: https://github.com/astral-sh/ruff-pre-commit
    rev: v0.4.4
    hooks:
      - id: ruff
        args: [--fix]
      - id: ruff-format
```

Relacionados: Linting, Black, Flake 8 , isort, pre-commit, Code Quality

S

Scaling (Escalado de Features)

Definición: Normalizar features a rango similar.

Técnicas: StandardScaler (z-score), MinMaxScaler (0 - 1)

Scikit-learn (sklearn)

Definición: Librería Python para ML clásico.

Módulos: preprocessing, model_selection, ensemble, metrics

Secret

Definición: Valor sensible (contraseña, API key) que no debe estar en código.

Kubernetes: Objeto Secret para almacenar datos sensibles encriptados.

Seed (Random State)

Definición: Valor para inicializar generadores aleatorios. Garantiza reproducibilidad.

```
np.random.seed(42)
RandomForestClassifier(random_state=42)
```

Service (Kubernetes)

Definición: Abstracción que expone pods como servicio de red.

Tipos: ClusterIP, NodePort, LoadBalancer

SHAP

Definición técnica: SHapley Additive exPlanations. Framework de interpretabilidad basado en teoría de juegos que asigna a cada feature su contribución marginal a una predicción específica. Funciona con cualquier modelo (model-agnostic).

Explicación conceptual: Cuando un modelo predice que un cliente va a abandonar, quieres saber por qué. SHAP descompone la predicción en contribuciones de cada feature: "El balance alto contribuyó + 0 . 1 5 a la probabilidad de churn, la edad joven contribuyó - 0 . 0 8 , el número de productos contribuyó + 0 . 1 2 ...". Esto permite explicar cada predicción individual, no solo el modelo en general.

Analogía desarrollada: Imagina un jurado de 1 0 personas que decide un veredicto. SHAP es como analizar cuánto influyó cada jurado en la decisión final. "María estaba muy convencida (+ 0 . 3), Juan estaba indeciso (+ 0 . 0 5), Pedro iba en contra (- 0 . 2)...". Sumando todas las contribuciones, obtienes el veredicto final.

Relacionados: Interpretabilidad, Feature Importance, Explainability

SMOTE (Synthetic Minority Over-sampling Technique)

Definición técnica: Técnica de oversampling que genera ejemplos sintéticos de la clase minoritaria interpolando entre ejemplos existentes y sus k vecinos más cercanos. No duplica ejemplos—crea nuevos puntos en el espacio de features.

Explicación conceptual: Cuando tienes 9 5 % de una clase y 5 % de otra, el modelo aprende a ignorar la minoritaria. SMOTE resuelve esto generando ejemplos sintéticos "plausibles" de la clase minoritaria. Toma un ejemplo

real, encuentra sus vecinos más cercanos (también de la clase minoritaria), y crea nuevos puntos en la línea que los conecta. Así el modelo ve más variedad de la clase minoritaria sin simplemente copiar los mismos ejemplos.

Analogía desarrollada: Imagina que tienes 10 fotos de gatos negros y 1000 de perros. Duplicar la foto del gato 100 veces no ayuda—el modelo memoriza esa única foto. SMOTE es como un artista que mira tus 10 fotos de gatos negros y pinta 90 fotos nuevas de gatos negros "plausibles" interpolando características: "este tiene los ojos del gato 1, las orejas del gato 3, el tamaño del gato 7 ...".

Ejemplo:

```
from imblearn.over_sampling import SMOTE
from sklearn.model_selection import train_test_split

# Siempre aplicar DESPUÉS del split (evitar data leakage)
X_train, X_test, y_train, y_test = train_test_split(X, y, stratify=y)

# SMOTE solo en training
smote = SMOTE(random_state=42, k_neighbors=5)
X_train_resampled, y_train_resampled = smote.fit_resample(X_train, y_train)

# Ahora las clases están balanceadas en training
print(f"Original: {y_train.value_counts().to_dict()}")
print(f"Resampled: {pd.Series(y_train_resampled).value_counts().to_dict()}")
```

Cuándo usar SMOTE vs class_weight:
- **SMOTE**: Cuando quieres más variedad en ejemplos minoritarios
- **class_weight**: Más simple, no modifica datos, funciona bien en la mayoría de casos

Relacionados: Class Imbalance, Oversampling, class_weight, imblearn

SOLID

Definición técnica: Cinco principios de diseño de software orientado a objetos que promueven código mantenible, extensible y testeable.

Los 5 principios:

1. **S - Single Responsibility**: Una clase debe tener una sola razón para cambiar
   ```python
   # ❌ Mal: Clase hace demasiado
   class ChurnPredictor:
   def load_data(self): ...
   def clean_data(self): ...
   def train(self): ...
   def save_to_s3(self): ...

# ✅ Bien: Responsabilidades separadas
class DataLoader: ...
class FeatureEngineer: ...
class ChurnTrainer: ...
class S3Uploader: ...
```

1. **O - Open/Closed**: Abierto para extensión, cerrado para modificación
   ```python
   # Puedes añadir nuevos modelos sin modificar código existente
   class BaseTrainer(ABC):
   @abstractmethod
   def train(self, X, y): ...

class RandomForestTrainer(BaseTrainer): ...
class XGBoostTrainer(BaseTrainer): ... # Extensión, no modificación
```

1. **L - Liskov Substitution**: Subclases deben ser substituibles por sus padres
2. **I - Interface Segregation**: Interfaces pequeñas y específicas
3. **D - Dependency Inversion**: Depender de abstracciones, no de implementaciones

En el portafolio: `FeatureEngineer`, `ChurnTrainer` siguen Single Responsibility. El uso de sklearn Pipeline permite Open/Closed (cambiar modelo sin modificar pipeline).

Relacionados: Clean Code, Design Patterns, Testing

src/ Layout

Definición técnica: Estructura de proyecto Python donde el código fuente reside en un subdirectorio `src/` en lugar de la raíz. El paquete se instala con `pip install -e .` para desarrollo.

Explicación conceptual: La estructura "flat" (código en raíz) causa problemas: Python puede importar archivos locales en vez del paquete instalado, tests pueden pasar localmente pero fallar en CI, y es difícil distinguir código de proyecto de configuración. `src/` layout resuelve esto forzando que el código solo sea accesible como paquete instalado.

Analogía desarrollada: Imagina una tienda donde los productos están tanto en el almacén como en el piso de venta. Confusión garantizada: ¿el cliente está comprando del almacén o del piso? src/ layout es como tener una puerta clara entre almacén (desarrollo) y piso de venta (paquete instalado).

Estructura del portafolio:

```
BankChurn-Predictor/
├── src/
│   └── bankchurn/           # Paquete principal
│       ├── __init__.py
│       ├── config.py        # Configuración Pydantic
│       ├── pipeline.py      # Pipeline sklearn
│       └── trainer.py       # Clase de entrenamiento
├── tests/                   # Tests (fuera de src/)
├── app/                     # APIs (fuera de src/)
├── configs/                 # Configuraciones YAML
├── artifacts/               # Modelos entrenados
└── pyproject.toml          # Configuración de paquete
```

Configuración en pyproject.toml:

```
[build-system]
requires = ["setuptools>=61.0"]
build-backend = "setuptools.build_meta"

[project]
name = "bankchurn"
version = "0.1.0"

[tool.setuptools.packages.find]
where = ["src"]
```

Relacionados: pyproject.toml, Package, Import, Project Structure

Staging

Definición: Ambiente que replica producción para testing final.

Analogía: Ensayo general antes del estreno.

Stratified Split

Definición: División que mantiene proporción de clases en train y test.

```
train_test_split(X, y, stratify=y)
```

Streamlit

Definición técnica: Framework Python para crear aplicaciones web interactivas con código puro Python. Convierte scripts de análisis de datos en dashboards web sin necesidad de conocimientos de HTML, CSS o JavaScript.

Explicación conceptual: Data Scientists crean análisis increíbles en notebooks, pero compartirlos requiere que el receptor tenga Python instalado y sepa ejecutar notebooks. Streamlit permite convertir ese análisis en una aplicación web que cualquiera puede usar: añades decoradores como `st.title()`, `st.button()`, `st.dataframe()` y Streamlit genera una UI web automáticamente. Es la forma más rápida de pasar de "script de análisis" a "aplicación interactiva".

Analogía desarrollada: Imagina que eres un chef que crea recetas increíbles. Jupyter notebooks es como escribir la receta en un cuaderno técnico—otros chefs pueden seguirla, pero no el público general. Streamlit es como montar un food truck donde la gente puede probar tus platos sin saber cocinar. Tu código Python sigue siendo la "cocina", pero ahora tiene una ventana de servicio bonita.

Ejemplo del portafolio (CarVision Dashboard):

```
import streamlit as st
import pandas as pd
import joblib

st.set_page_config(page_title="CarVision Predictor", page_icon="🚗")
st.title("🚗 CarVision Price Predictor")

# Sidebar para inputs
with st.sidebar:
    st.header("Vehicle Features")
    year = st.slider("Year", 2000, 2024, 2018)
    mileage = st.number_input("Mileage", 0, 300000, 50000)
    brand = st.selectbox("Brand", ["Toyota", "Honda", "Ford"])

# Cargar modelo (con cache para no recargar)
@st.cache_resource
def load_model():
    return joblib.load("artifacts/model.joblib")

model = load_model()

# Botón de predicción
if st.button("🔮 Predict Price"):
    input_df = pd.DataFrame([{"year": year, "mileage": mileage, "brand": brand}])
    prediction = model.predict(input_df)[0]

    st.success(f"Estimated Price: ${prediction:,.0f}")

    # Métricas visuales
    col1, col2 = st.columns(2)
    col1.metric("Predicted Price", f"${prediction:,.0f}")
    col2.metric("Confidence", "High" if prediction > 10000 else "Medium")
```

Componentes clave:
- `st.title()`, `st.header()`: Títulos
- `st.slider()`, `st.number_input()`, `st.selectbox()`: Inputs
- `st.button()`: Acciones
- `st.dataframe()`, `st.plotly_chart()`: Visualización
- `@st.cache_resource`: Cache de modelos/datos pesados

Relacionados: Dashboard, FastAPI, Gradio, Panel

T

Target

Definición: Variable que queremos predecir. También llamada "label" o "y".

Terraform

Definición: Infrastructure as Code. Provisiona recursos en cloud con código.

```
resource "aws_instance" "ml_server" {
  instance_type = "t3.medium"
}
```

Test Coverage

Definición: Porcentaje de código ejecutado durante tests.

Relacionados: Coverage, pytest

Threshold (Umbral)

Definición: Punto de corte para convertir probabilidades en clases.

Default: `0.5`, pero ajustable según necesidades de negocio.

Throughput

Definición: Cantidad de predicciones/requests por unidad de tiempo.

Analogía: Cuántos platos puede servir el restaurante por hora.

Traces

Definición: Seguimiento de requests a través de sistema distribuido.

Herramientas: Jaeger, OpenTelemetry

Relacionados: Observabilidad, Logs, Metrics

TransformerMixin

Definición: Mixin sklearn que añade `fit_transform()` automáticamente.

Relacionados: BaseEstimator, Custom Transformer

Trivy

Definición: Escáner de vulnerabilidades para contenedores.

```
trivy image my-app:latest
```

Type Hints

Definición: Anotaciones en Python que indican tipos esperados.

```
def predict(data: pd.DataFrame) -> np.ndarray:
    pass
```

Relacionados: mypy, Pydantic

U

Underfitting (Subajuste)

Definición: Modelo demasiado simple. No captura patrones.

Señales: Train y validation accuracy bajas.

Unit Test

Definición: Test de función/método individual en aislamiento.

```
def test_feature_ratio():
    result = compute_ratio(100, 2)
    assert result == 50
```

Uvicorn

Definición: Servidor ASGI de alto rendimiento para FastAPI.

```
uvicorn app:app --host 0.0.0.0 --port 8000
```

V

Validation Set

Definición: Datos para ajustar hiperparámetros, separado de train y test.

Split típico: 60% train, 20% validation, 20% test

Vendor Lock-in

Definición: Dependencia de proveedor específico que dificulta migración.

Analogía: Comprar auto donde repuestos solo existen en una tienda.

Version Control

Definición: Sistema para rastrear cambios en archivos.

Herramientas: Git (código), DVC (datos), MLflow (modelos)

Voting Classifier

Definición: Ensemble que combina predicciones por votación.

```
VotingClassifier([
    ('rf', RandomForestClassifier()),
    ('xgb', XGBClassifier())
], voting='soft')
```

W

Weights & Biases (W&B)

Definición: Plataforma SaaS para experiment tracking con visualizaciones avanzadas.

Relacionados: MLflow, Experiment Tracking

Workflow (GitHub Actions)

Definición: Proceso automatizado definido en archivo YAML.

Relacionados: Job, Step, CI/CD

X

XGBoost

Definición: Implementación optimizada de gradient boosting. Muy popular en competencias.

```
from xgboost import XGBClassifier
model = XGBClassifier(n_estimators=100, learning_rate=0.1)
```

Y

YAML

Definición: Formato de serialización legible para configuración.

```
model:
  type: ensemble
  n_estimators: 100
```

Z

Zero-Downtime Deployment

Definición: Actualizar aplicación sin interrumpir servicio.

Técnicas: Rolling update, Blue-green deployment

Símbolos y Abreviaciones

Símbolo	Significado
TP	True Positive
TN	True Negative
FP	False Positive
FN	False Negative
P 9 5	Percentil 9 5
GHCR	GitHub Container Registry
IaC	Infrastructure as Code
DAG	Directed Acyclic Graph
OOM	Out of Memory
CRUD	Create, Read, Update, Delete
SLA	Service Level Agreement
SLO	Service Level Objective
TTL	Time To Live

📊 Tablas de Referencia Rápida

Métricas de Clasificación

Métrica	Fórmula	Uso
Accuracy	(TP+TN)/(Total)	Balance general (clases balanceadas)
Precision	TP/(TP+FP)	Minimizar falsos positivos

Métrica	Fórmula	Uso
Recall	TP/(TP+FN)	Minimizar falsos negativos
F1	2×P×R/(P+R)	Balance P y R
AUC-ROC	Área bajo curva	Capacidad discriminatoria

Tipos de Testing

Tipo	Alcance	Ejemplo
Unit	Función individual	`test_compute_ratio()`
Integration	Múltiples componentes	`test_pipeline_fit()`
E2E	Sistema completo	`test_api_predict_flow()`

Ambientes

Ambiente	Propósito	Datos
Development	Desarrollo	Sintéticos/muestra
Staging	Testing final	Réplica producción
Production	Usuarios reales	Reales

Navegación

⏪ Anterior	📑 Índice	⏩ Siguiente
20_PROYECTO_INTEGRADOR.md	Índice	22_CHECKLIST.md

© 2025 DuqueOM - Guía MLOps v5.0: Senior Edition

Módulo 21 Completado ✅

CHECKLIST

GUÍA MLOPS V5.0

PORTFOLIO EDITION

MÓDULO 22: CHECKLIST FINAL

Verificación del Portafolio

Guía MLOps v2.0 | DuqueOM | Noviembre 2025

✅ MÓDULO 22: Checklist Final

Verificación del Portafolio

"La calidad se verifica, no se asume."

Nivel	Duración
📚 Referencia	1 hora

🎯 Objetivo

Lista de verificación completa para asegurar que tu portafolio MLOps está listo para presentar.

✅ Checklist Pre-Release

1. Repositorio y Estructura

```
## Estructura del Repo
- [ ] README.md completo con badges
- [ ] LICENSE presente (MIT recomendado)
- [ ] .gitignore apropiado para Python/ML
- [ ] Estructura de carpetas profesional
- [ ] Cada proyecto tiene su propio README

## Versionado
- [ ] Commits con formato convencional
- [ ] Branches organizados (main, develop)
- [ ] Tags para releases
- [ ] CHANGELOG.md actualizado
```

2. Código y Calidad

```
## Código
- [ ] Código modular (src/proyecto/)
- [ ] Configuración con Pydantic (config.py)
- [ ] Logging implementado
- [ ] Type hints en funciones principales
- [ ] Docstrings en clases y funciones públicas

## Estilo
- [ ] Formateado con Black
- [ ] Imports ordenados con isort
- [ ] Sin errores críticos de flake8
- [ ] Sin warnings de mypy (o justificados)
```

3. Datos

```
## Versionado de Datos
- [ ] DVC inicializado
- [ ] Datasets versionados
- [ ] Remote configurado (local o cloud)
- [ ] .dvc files commiteados
- [ ] dvc.yaml con pipeline definido

## Documentación de Datos
- [ ] Data Card presente
- [ ] Descripción de features
- [ ] Distribución de target documentada
```

4. Modelo y Pipeline

```
## Pipeline ML
- [ ] Pipeline sklearn unificado
```

```
- [ ] Preprocessor incluido en pipeline
- [ ] Modelo serializable con joblib
- [ ] Reproducibilidad verificada (seeds)

## Tracking
- [ ] MLflow configurado
- [ ] Parámetros logueados
- [ ] Métricas logueadas
- [ ] Modelos registrados como artefactos
```

5. Testing

```
## Tests
- [ ] tests/conftest.py con fixtures
- [ ] Tests unitarios (test_*.py)
- [ ] Tests de integración
- [ ] Tests de API
- [ ] Coverage ≥ 70%

## Ejecución
- [ ] pytest ejecuta sin errores
- [ ] pytest --cov reporta coverage
- [ ] Tests son independientes (no orden)
- [ ] Tests rápidos (<30s total)
```

6. CI/CD

```
## GitHub Actions
- [ ] .github/workflows/ci.yml presente
- [ ] Jobs: lint, test, build
- [ ] Matrix testing (Python 3.11, 3.12)
- [ ] Badge de CI en README
- [ ] Pipeline pasa en verde

## Seguridad
- [ ] Bandit scan sin HIGH severity
- [ ] Gitleaks configurado
- [ ] No secrets en código
- [ ] .env.example presente (no .env)
```

7. Docker

```
## Dockerfile
- [ ] Multi-stage build
- [ ] Usuario no-root
- [ ] .dockerignore presente
- [ ] Imagen < 1GB (idealmente < 500MB)
- [ ] HEALTHCHECK configurado

## Compose
- [ ] docker-compose.yml funcional
- [ ] Servicios se levantan correctamente
- [ ] Health checks pasan
- [ ] Puertos documentados
```

8. API

```
## FastAPI
- [ ] /health endpoint
- [ ] /predict endpoint
- [ ] Validación con Pydantic
- [ ] Documentación en /docs
- [ ] CORS configurado

## Funcionamiento
- [ ] API responde correctamente
- [ ] Predicciones son válidas
- [ ] Errores tienen mensajes claros
- [ ] Latencia < 100ms
```

9. Documentación

```
## README Principal
- [ ] Descripción clara del proyecto
- [ ] Quick Start funcional
- [ ] Instrucciones de instalación
- [ ] Ejemplos de uso
- [ ] Badges de CI, coverage, etc.

## Model Card
- [ ] Model Details completos
- [ ] Intended Use documentado
- [ ] Métricas de performance
- [ ] Limitaciones explícitas
- [ ] Consideraciones éticas
```

10. Demo

```
## Video Demo
- [ ] Duración 3-5 minutos
- [ ] Introducción del problema
- [ ] Muestra estructura del código
- [ ] Demo en vivo funcionando
- [ ] CI/CD pipeline visible
- [ ] Cierre con call-to-action
```

🔍 Verificación Final

Comandos de Validación

```
# 1. Clone limpio
cd /tmp
git clone https://github.com/USUARIO/REPO.git
cd REPO

# 2. Instalar y testear
pip install -e ".[dev]"
pytest tests/ -v --cov=src

# 3. Lint
black --check src/
flake8 src/ --select=E9,F63,F7,F82

# 4. Docker
docker build -t test-image .
docker run --rm -d -p 8000:8000 test-image
sleep 10
curl http://localhost:8000/health

# 5. DVC
dvc status
dvc repro --dry

# 6. MLflow
mlflow ui --port 5000 &
# Verificar UI en http://localhost:5000
```

Criterios de Aprobación

Criterio	Mínimo	Ideal
Coverage	70%	85%+
Latencia API	< 200ms	< 50ms
Tamaño Docker	< 1GB	< 500MB
Tests	10+	30+
CI tiempo	< 10min	< 5min

📋 Checklist por Proyecto

BankChurn-Predictor

```
- [ ] src/bankchurn/ modular
- [ ] VotingClassifier implementado
- [ ] Calibración de probabilidades
- [ ] Tests de fairness
- [ ] API /predict funcional
- [ ] Model Card con métricas
```

CarVision-Market-Intelligence

```
- [ ] FeatureEngineer centralizado
- [ ] Pipeline [features, pre, model]
- [ ] Streamlit dashboard
- [ ] FastAPI backend
- [ ] Data leakage prevenido
- [ ] Bootstrap confidence intervals
```

TelecomAI-Customer-Intelligence

```
- [ ] Clasificación multi-estrategia
- [ ] VotingClassifier configurado
- [ ] Pipeline end-to-end
- [ ] Tests de integración
- [ ] API documentada
```

🚀 Pre-Push Checklist

Antes de cada push importante:

```
## Quick Check
- [ ] `pytest` pasa
- [ ] `black --check .` pasa
- [ ] `flake8 src/` sin errores críticos
- [ ] `docker build` funciona
- [ ] Commit message es descriptivo
- [ ] No hay archivos sensibles staged
```

📊 Scorecard de Portafolio

Auto-evaluación (0-10 por ítem)

Categoría	Puntuación	Notas
Código	/ 1 0	Modularidad, estilo, documentación
Testing	/ 1 0	Coverage, variedad de tests
CI/CD	/ 1 0	Automatización, velocidad
Docker	/ 1 0	Optimización, seguridad
API	/ 1 0	Funcionalidad, documentación
Datos	/ 1 0	Versionado, documentación
ML	/ 1 0	Pipeline, métricas, tracking
Docs	/ 1 0	README, Model Cards
Demo	/ 1 0	Video, presentación
Profesionalismo	/ 1 0	Consistencia, atención al detalle

Total: / 1 0 0

Niveles

Puntuación	Nivel
90-100	🏆 Excepcional - Listo para entrevistas senior
80-89	⭐ Excelente - Muy competitivo
70-79	✅ Bueno - Sólido para aplicar
60-69	📈 Aceptable - Necesita mejoras menores
<60	🔄 En progreso - Continuar trabajando

🎯 Acciones Post-Checklist

Si pasas todo:

1. ✅ Publicar en GitHub
2. ✅ Añadir a LinkedIn
3. ✅ Incluir en CV
4. ✅ Compartir en comunidades

Si faltan items:

1. Priorizar items críticos (tests, CI, docs)
2. Crear issues para items faltantes
3. Planificar sprints de mejora
4. Re-evaluar en 1 semana

Navegación

⏪ Anterior	📑 Índice	⏩ Siguiente
21_GLOSARIO.md	Índice	23_RECURSOS.md

© 2025 DuqueOM - Guía MLOps v3.0

Módulo 22 Completado ✅

RECURSOS

GUÍA MLOPS V5.0

PORTFOLIO EDITION

MÓDULO 23: RECURSOS Y REFERENCIAS

Links, Cursos y Comunidades

Guía MLOps v2.0 | DuqueOM | Noviembre 2025

🔗 MÓDULO 23: Recursos y Referencias

Links, Cursos y Comunidades

"El aprendizaje nunca termina."

Nivel	Duración
📚 Referencia	Consulta

📚 Recursos Recomendados

Esta lista contiene los recursos más valiosos para profundizar en cada área de MLOps.

> 📺 **¿Buscas recursos específicos por módulo?** Ver RECURSOS_POR_MODULO.md para videos, cursos y documentación curados para cada módulo con sistema de clasificación 🔴🟡🟢.
>
> Además de estos recursos externos, revisa también los recursos internos de esta guía: PLANTILLAS.md, GUIA_AUDIOVISUAL.md y los scripts `generate_pdfs.py` / `generate_audio.py` para exportar la guía a PDF y audio.

🎓 Cursos y Programas

MLOps Fundamentales

Recurso	Tipo	Nivel	Costo
Made With ML	Curso	Intermedio	Gratis
Full Stack Deep Learning	Curso	Avanzado	Gratis
MLOps Zoomcamp	Bootcamp	Intermedio	Gratis
Coursera MLOps Specialization	Especialización	Intermedio	Pagado

Herramientas Específicas

Herramienta	Recurso Oficial
MLflow	Docs
DVC	Docs
FastAPI	Tutorial
Docker	Docs
Kubernetes	Docs
GitHub Actions	Docs

📖 Libros

Esenciales

Título	Autor	Tema
Designing Machine Learning Systems	Chip Huyen	MLOps end-to-end
Building Machine Learning Pipelines	Hapke & Nelson	Pipelines con TFX

Título	Autor	Tema
Machine Learning Engineering	Andriy Burkov	Prácticas de ML
Reliable Machine Learning	Cathy Chen et al.	ML en producción

Python y Software Engineering

Título	Autor	Tema
Fluent Python	Luciano Ramalho	Python avanzado
Clean Code	Robert C. Martin	Código limpio
The Pragmatic Programmer	Hunt & Thomas	Buenas prácticas

🔗 Blogs y Artículos

Blogs de Empresas

Empresa	Blog	Foco
Netflix	Tech Blog	ML a escala
Uber	Engineering Blog	ML en tiempo real
Airbnb	Tech Blog	Feature stores
Spotify	Engineering Blog	Recomendaciones
Google AI	Blog	Investigación aplicada

Artículos Fundamentales

1. **Model Cards**
2. Model Cards for Model Reporting - Mitchell et al.
3. Hugging Face Model Cards Guide
4. **MLOps**
5. Hidden Technical Debt in ML Systems - Google
6. Continuous Delivery for ML
7. **Testing ML**
8. Testing ML Systems - Jeremy Jordan
9. Effective Testing for ML

🛠️ Herramientas por Categoría

Tracking de Experimentos

Herramienta	Tipo	URL
MLflow	Open Source	mlflow.org
Weights & Biases	SaaS	wandb.ai
Neptune	SaaS	neptune.ai
Comet	SaaS	comet.ml
DVC	Open Source	dvc.org

Feature Stores

Herramienta	Tipo	URL
Feast	Open Source	feast.dev
Tecton	SaaS	tecton.ai
Hopsworks	Híbrido	hopsworks.ai

Monitoreo de Modelos

Herramienta	Tipo	URL
Evidently	Open Source	evidentlyai.com
WhyLabs	SaaS	whylabs.ai
Arize	SaaS	arize.com
NannyML	Open Source	nannyml.com

CI/CD para ML

Herramienta	Tipo	URL
GitHub Actions	SaaS	github.com/features/actions
GitLab CI	SaaS/Self	docs.gitlab.com/ee/ci/
CML	Open Source	cml.dev
Kubeflow Pipelines	Open Source	kubeflow.org

Serving de Modelos

Herramienta	Tipo	URL
FastAPI	Open Source	fastapi.tiangolo.com
BentoML	Open Source	bentoml.com
Seldon	Open Source	seldon.io
TensorFlow Serving	Open Source	tensorflow.org/tfx/guide/serving
Triton	Open Source	developer.nvidia.com/triton-inference-server

📺 Videos y Charlas

Conferencias

Conferencia	Foco	URL
MLOps World	MLOps	mlopsworld.com
ML Conf	ML Engineering	mlconf.com
PyData	Python + Data	pydata.org
KubeCon	Kubernetes	kubecon.io

Charlas Recomendadas

1. **MLOps Overview**
2. Practical MLOps - Stanford

3. ML Engineering at Google
4. **Testing ML**
5. Testing ML Systems - PyCon
6. **Production ML**
7. ML at Scale - Netflix

🐙 Repositorios de Referencia

Plantillas y Ejemplos

Repositorio	Descripción
cookiecutter-data-science	Plantilla de proyectos DS
mlops-python-package	Plantilla MLOps
made-with-ml	Curso completo
mlops-zoomcamp	Bootcamp MLOps

Proyectos de Referencia

Repositorio	Destacado
DuqueOM/ML-MLOps-Portfolio	Este portafolio
kedro	Framework de pipelines
feast	Feature store
evidently	Monitoreo de drift

📋 Cheat Sheets

Git

- Git Cheat Sheet (GitHub)
- Git Flow

Docker

- Docker Cheat Sheet
- Docker Compose

Kubernetes

- kubectl Cheat Sheet

Python

- Python Cheat Sheet
- Pandas Cheat Sheet

🎯 Rutas de Aprendizaje

Ruta Principiante (2-3 meses)

```
1. Python básico → intermedio
2. Git y GitHub
3. pandas y sklearn
4. pytest básico
5. Docker fundamentals
```

Ruta Intermedia (3-4 meses)

```
1. Pipelines sklearn avanzados
2. MLflow tracking
3. DVC para datos
4. FastAPI
```

```
5. GitHub Actions CI/CD
6. Testing completo
```

Ruta Avanzada (4-6 meses)

```
1. Kubernetes
2. Terraform / IaC
3. Feature stores
4. Monitoreo de drift
5. A/B testing
6. Auto-retraining
```

📬 Comunidades

Discord/Slack

Comunidad	Foco	URL
MLOps Community	MLOps	mlops.community
DataTalks.Club	Data Engineering	datatalks.club
Locally Optimistic	Data Teams	locallyoptimistic.com

Reddit

- r/MachineLearning
- r/learnmachinelearning
- r/datascience
- r/MLOps

Twitter/X

Cuentas recomendadas:
- @ChipHuyen
- @kaboroevich
- @eugeneyan
- @sh_reya

🔧 Herramientas de Productividad

Herramienta	Uso	URL
Notion	Documentación	notion.so
Obsidian	Notas	obsidian.md
Excalidraw	Diagramas	excalidraw.com
Mermaid	Diagramas en código	mermaid.js.org

📊 Datasets para Practicar

Dataset	Tipo	URL
Kaggle	Variado	kaggle.com/datasets
UCI ML Repository	Clásicos	archive.ics.uci.edu/ml
Hugging Face Datasets	NLP/CV	huggingface.co/datasets
Google Dataset Search	Búsqueda	datasetsearch.research.google.com

📅 Mantente Actualizado

Newsletters

Newsletter	Foco	Frecuencia
The Batch	AI General	Semanal
MLOps Community	MLOps	Semanal
Data Elixir	Data Science	Semanal
Python Weekly	Python	Semanal

Podcasts

Podcast	Foco
MLOps Coffee Sessions	MLOps
Practical AI	ML aplicado
Data Engineering Podcast	Data infra
Talk Python To Me	Python

Recursos Relacionados

Recurso	Descripción
📺 RECURSOS_POR_MODULO.md	Videos y cursos específicos para cada módulo
📋 PLANTILLAS.md	Templates reutilizables

Navegación

⏪ Anterior	📑 Índice	🎯 Final
22_CHECKLIST.md	Índice	🎉 ¡Guía Completada!

© 2025 DuqueOM - Guía MLOps v3.0

Módulo 23 Completado ✅

🎉 **¡FELICITACIONES! Has completado toda la guía MLOps.**

RECURSOS POR MÓDULO

GUÍA MLOPS V5.0

PORTFOLIO EDITION

📚 Recursos Externos por Módulo — Guía MLOps

Videos, cursos y materiales externos curados e integrados con cada módulo

Última actualización: Diciembre 2025
Sistema de clasificación: 🔴 Obligatorio | 🟡 Recomendado | 🟢 Complementario

🏷️ Sistema de Etiquetas

Etiqueta	Significado	Cuándo Usarlo
🔴 **Obligatorio**	Esencial para entender el módulo	Si tienes dudas conceptuales después de leer el módulo
🟡 **Recomendado**	Profundiza significativamente el tema	Si quieres dominar el tema, no solo entenderlo
🟢 **Complementario**	Perspectiva adicional o avanzada	Si ya dominas lo básico y quieres más

📋 Índice Rápido

Módulo	Tema	Videos	Cursos	Docs
01	Python Moderno	3	1	2
02	Diseño de Sistemas	2	1	3
03	Estructura de Proyecto	2	-	2
04	Entornos Virtuales	2	-	3
05	Git Profesional	3	1	2
06	Versionado de Datos	3	1	2
07	sklearn Pipelines	4	2	2
08	Feature Engineering	3	1	2
09	Training Profesional	3	1	2
10	Experiment Tracking	4	1	3
11	Testing ML	4	1	3
12	CI/CD	4	1	2
13	Docker	5	2	3
14	FastAPI	4	1	2
15	Streamlit	4	1	2
16	Observabilidad	3	1	3
17	Despliegue	3	2	3
18	Infraestructura	3	2	3

📖 Módulo 01: Python Moderno

🎬 Videos

🏷️	Título	Canal	Duración	Link
🔴	**Type Hints in Python**	ArjanCodes	18 min	YouTube
🔴	**Pydantic V2: The Complete Guide**	ArjanCodes	25 min	YouTube

🏷️	Título	Canal	Duración	Link
🟡	**Python OOP - Classes and Objects**	Corey Schafer	15 min	YouTube

📚 Cursos

🏷️	Título	Plataforma	Duración	Link
🟢	Modern Python 3 Bootcamp	Udemy	30 h	Udemy

📄 Documentación

🏷️	Recurso	Descripción
🔴	Pydantic Docs	Documentación oficial de Pydantic V2
🟡	typing Module	Documentación de type hints

💡 Cuándo Consultar

- **Antes de empezar**: Ve el video de Type Hints si no los usas regularmente
- **Durante el módulo**: Consulta Pydantic docs cuando trabajes con `config.py`
- **Después**: El curso complementario si quieres profundizar en POO

📖 Módulo 02: Diseño de Sistemas ML

🎬 Videos

🏷️	Título	Canal	Duración	Link
🔴	**ML System Design Interview**	Exponent	22 min	YouTube
🟡	**Designing ML Systems (Chip Huyen)**	Stanford MLSys	55 min	YouTube

📚 Cursos

🏷️	Título	Plataforma	Duración	Link
🟡	Machine Learning System Design	Educative.io	8 h	Educative

📄 Documentación

🏷️	Recurso	Descripción
🔴	ML Canvas Template	Canvas para diseño ML
🟡	ADR GitHub	Guía de Architecture Decision Records
🟢	C4 Model	Modelo de diagramas de arquitectura

📖 Módulo 05: Git Profesional

🎬 Videos

🏷️	Título	Canal	Duración	Link
🔴	**Git for Professionals**	freeCodeCamp	41 min	YouTube
🟡	**Git Branching Strategies**	DevOps Toolkit	18 min	YouTube
🟢	**Pre-commit Hooks Tutorial**	mCoding	12 min	YouTube

📚 Cursos

🏷️	Título	Plataforma	Duración	Link
🟢	Version Control with Git	Coursera (Atlassian)	13 h	Coursera

📖 Módulo 06: Versionado de Datos (DVC)

🎬 Videos

🏷️	Título	Canal	Duración	Link
🔴	**DVC Tutorial - Data Version Control**	DVCorg	12 min	YouTube
🔴	**DVC Pipelines Deep Dive**	DVCorg	18 min	YouTube
🟡	**MLOps with DVC and CML**	DataTalksClub	45 min	YouTube

📚 Cursos

🏷️	Título	Plataforma	Duración	Link
🟡	Iterative Tools for ML	DVCorg	4 h	Learn.iterative.ai

📄 Documentación

🏷️	Recurso	Descripción
🔴	DVC Get Started	Tutorial oficial paso a paso
🟡	DVC with Remote Storage	Configuración de remotes S3/GCS

📖 Módulo 07: sklearn Pipelines

🎬 Videos

🏷️	Título	Canal	Duración	Link
🔴	**Sklearn Pipeline Tutorial**	Data School	28 min	YouTube
🔴	**ColumnTransformer Explained**	Data School	35 min	YouTube
🟡	**Custom Transformers in Sklearn**	PyData	32 min	YouTube
🟢	**Sklearn Pipeline Best Practices**	PyData Berlin	45 min	YouTube

📚 Cursos

🏷️	Título	Plataforma	Duración	Link
🔴	ML Pipelines with scikit-learn	DataCamp	4 h	DataCamp
🟡	Feature Engineering for ML	Coursera (Google)	5 weeks	Coursera

📄 Documentación

🏷️	Recurso	Descripción
🔴	sklearn Pipeline User Guide	Guía oficial de pipelines
🟡	Custom Transformers	Cómo crear transformers custom

📖 Módulo 10: Experiment Tracking (MLflow)

🎬 Videos

🏷️	Título	Canal	Duración	Link
🔴	**MLflow Tutorial (2024)**	DataTalksClub	35 min	YouTube
🔴	**MLflow Model Registry**	Databricks	22 min	YouTube
🟡	**MLflow Projects and Models**	MLflow	28 min	YouTube
🟢	**MLflow + Docker Integration**	Databricks	18 min	YouTube

📚 Cursos

🏷️	Título	Plataforma	Duración	Link
🟡	End-to-End ML with MLflow	Databricks Academy	3 h	Databricks

📄 Documentación

🏷️	Recurso	Descripción
🔴	MLflow Quickstart	Inicio rápido oficial
🔴	MLflow Tracking Guide	Guía completa de tracking
🟡	MLflow Model Registry	Registro de modelos

📖 Módulo 11: Testing para ML

🎬 Videos

🏷️	Título	Canal	Duración	Link
🔴	**pytest Tutorial - Full Course**	freeCodeCamp	90 min	YouTube
🔴	**Testing ML Models**	Made With ML	25 min	YouTube
🟡	**pytest Fixtures Deep Dive**	ArjanCodes	20 min	YouTube
🟢	**Property-Based Testing with Hypothesis**	PyCon	35 min	YouTube

📚 Cursos

🏷️	Título	Plataforma	Duración	Link
🟡	Testing ML Models	Made With ML	2 h	MadeWithML

📄 Documentación

🏷️	Recurso	Descripción
🔴	pytest Documentation	Documentación oficial
🟡	pytest-cov	Plugin de coverage
🟢	Great Expectations	Testing de datos

📖 Módulo 12: CI/CD con GitHub Actions

🎬 Videos

🏷️	Título	Canal	Duración	Link
🔴	**GitHub Actions Tutorial**	TechWorld with Nana	32 min	YouTube
🔴	**CI/CD for ML Projects**	DataTalksClub	40 min	YouTube
🟡	**Matrix Strategy Deep Dive**	GitHub	15 min	YouTube
🟢	**Security Scanning in CI**	GitHub Universe	25 min	YouTube

📚 Cursos

🏷️	Título	Plataforma	Duración	Link
🟡	GitHub Actions	GitHub Learning Lab	4 h	GitHub

📄 Documentación

🏷️	Recurso	Descripción
🔴	GitHub Actions Docs	Documentación oficial
🟡	Reusable Workflows	Workflows reutilizables

📖 Módulo 13: Docker para ML

🎬 Videos

🏷️	Título	Canal	Duración	Link
🔴	**Docker Tutorial for Beginners**	TechWorld with Nana	60 min	YouTube
🔴	**Multi-stage Docker Builds**	Docker	15 min	YouTube
🟡	**Docker for Data Science**	DataTalksClub	35 min	YouTube
🟡	**Docker Compose Tutorial**	TechWorld with Nana	45 min	YouTube
🟢	**Docker Security Best Practices**	Aqua Security	28 min	YouTube

📚 Cursos

🏷️	Título	Plataforma	Duración	Link
🟡	Docker Mastery	Udemy	20 h	Udemy
🟢	Containerization for ML	Coursera	4 weeks	Coursera

📄 Documentación

🏷️	Recurso	Descripción
🔴	Docker Get Started	Tutorial oficial
🟡	Dockerfile Best Practices	Mejores prácticas
🟢	Docker Security	Guía de seguridad

📖 Módulo 14: FastAPI para ML

🎬 Videos

🏷️	Título	Canal	Duración	Link
🔴	**FastAPI Full Course**	freeCodeCamp	5 h 30 m	YouTube
🔴	**Deploy ML Models with FastAPI**	Patrick Loeber	25 min	YouTube
🟡	**FastAPI Best Practices**	ArjanCodes	22 min	YouTube
🟢	**Async FastAPI with ML**	Python Engineer	18 min	YouTube

📚 Cursos

🏷️	Título	Plataforma	Duración	Link
🟡	FastAPI Fundamentals	TestDriven.io	8 h	TestDriven.io

📄 Documentación

🏷️	Recurso	Descripción
🔴	FastAPI Tutorial	Tutorial oficial completo
🟡	Pydantic with FastAPI	Schemas y validación

📖 Módulo 15: Streamlit Dashboards

🎬 Videos

🏷️	Título	Canal	Duración	Link
🔴	**Streamlit Crash Course**	Patrick Loeber	45 min	YouTube
🔴	**Build ML Apps with Streamlit**	Data Professor	35 min	YouTube
🟡	**Streamlit Multi-page Apps**	Streamlit	20 min	YouTube
🟢	**Advanced Streamlit - Caching**	Streamlit	15 min	YouTube

📚 Cursos

🏷️	Título	Plataforma	Duración	Link
🟡	30 Days of Streamlit	Streamlit	30 days	Streamlit

📄 Documentación

🏷️	Recurso	Descripción
🔴	Streamlit Docs	Documentación oficial
🟡	Streamlit Gallery	Ejemplos de apps

📖 Módulo 16: Observabilidad

🎬 Videos

🏷️	Título	Canal	Duración	Link
🔴	**Prometheus + Grafana Tutorial**	TechWorld with Nana	50 min	YouTube

🏷️	Título	Canal	Duración	Link
🟡	**ML Model Monitoring with Evidently**	Evidently AI	30 min	YouTube
🟢	**Drift Detection Explained**	NannyML	25 min	YouTube

📚 Cursos

🏷️	Título	Plataforma	Duración	Link
🟡	ML Monitoring	Made With ML	3 h	MadeWithML

📄 Documentación

🏷️	Recurso	Descripción
🔴	Evidently Docs	Documentación oficial
🟡	Prometheus Docs	Documentación de Prometheus
🟢	Grafana Dashboards	Creación de dashboards

📖 Módulo 17: Despliegue

🎬 Videos

🏷️	Título	Canal	Duración	Link
🔴	**Deploy ML Models to Production**	Made With ML	40 min	YouTube
🟡	**AWS for ML Deployment**	AWS	35 min	YouTube
🟢	**GCP Cloud Run for ML**	Google Cloud	25 min	YouTube

📚 Cursos

🏷️	Título	Plataforma	Duración	Link
🟡	MLOps on AWS	Coursera (AWS)	4 weeks	Coursera
🟢	MLOps on GCP	Coursera (Google)	5 weeks	Coursera

📖 Módulo 18: Infraestructura (Kubernetes/Terraform)

🎬 Videos

🏷️	Título	Canal	Duración	Link
🔴	**Kubernetes Tutorial**	TechWorld with Nana	4 h	YouTube
🟡	**Terraform Tutorial**	freeCodeCamp	2 h 30 m	YouTube
🟢	**Kubeflow for ML**	Google Cloud	40 min	YouTube

📚 Cursos

🏷️	Título	Plataforma	Duración	Link
🟡	Kubernetes Fundamentals	Linux Foundation	35 h	LF Training
🟢	Terraform Associate	HashiCorp	15 h	HashiCorp Learn

📌 Cursos Integrales Recomendados

Estos cursos cubren **múltiples módulos** y son excelentes para una visión completa:

🏷️	Curso	Plataforma	Cubre Módulos	Link
🔴	**MLOps Zoomcamp**	DataTalksClub	06, 07, 10, 11, 12, 13, 14, 16, 17	GitHub
🔴	**Made With ML**	Made With ML	07, 08, 09, 11, 14, 16	MadeWithML
🟡	**Full Stack Deep Learning**	FSDL	02, 07, 10, 12, 13, 16, 17	FSDL
🟡	**MLOps Specialization**	Coursera (DeepLearning.AI)	07, 08, 10, 12, 13, 17	Coursera

📺 Canales de YouTube Recomendados

Canal	Foco	Por qué seguirlo
ArjanCodes	Python profesional	Mejores prácticas, patrones de diseño
TechWorld with Nana	DevOps/MLOps	Docker, K8s, CI/CD explicados claramente
Data School	sklearn, pandas	Tutoriales detallados de ML
Patrick Loeber	ML deployment	FastAPI, PyTorch, deployment
DVCorg	DVC, CML	Versionado de datos
DataTalksClub	MLOps completo	MLOps Zoomcamp, entrevistas

🔗 Cómo Integrar los Recursos en tu Estudio

Flujo Recomendado por Módulo

```
1. 📖 LEE el módulo de la guía (completo)
          ↓
2. 🎬 VE el video 🔴 Obligatorio si hay dudas conceptuales
          ↓
3. 💻 HAZ los ejercicios del módulo
          ↓
4. 🔍 COMPARA con el código real del portafolio
          ↓
5. 🎬 VE videos 🟡 Recomendados para profundizar
          ↓
6. 📚 EXPLORA cursos 🟢 Complementarios si quieres dominar el tema
```

Tiempo Estimado por Módulo

Profundidad	Solo Guía	+ Videos 🔴	+ Videos 🟡	+ Cursos
Básico	2 - 3 h	+ 1 - 2 h	-	-
Intermedio	2 - 3 h	+ 1 - 2 h	+ 2 - 3 h	-
Avanzado	2 - 3 h	+ 1 - 2 h	+ 2 - 3 h	+ 10 - 20 h

EJERCICIOS

GUÍA MLOPS V5.0

PORTFOLIO EDITION

🔧 Ejercicios Prácticos — Guía MLOps

Ejercicios organizados por módulo para practicar cada concepto

📋 Índice de Ejercicios

Módulo	Tema	Dificultad	Ejercicios
01	Python Moderno	⭐⭐	3
02	Diseño de Sistemas	⭐⭐	2
03	Estructura de Proyecto	⭐	2
04	Entornos	⭐	2
05	Git Profesional	⭐⭐	2
06	Versionado de Datos	⭐⭐	2
07	sklearn Pipelines	⭐⭐⭐	3
08	Feature Engineering	⭐⭐⭐	2
09	Training Profesional	⭐⭐⭐	2
10	Experiment Tracking	⭐⭐	2
11	Testing ML	⭐⭐⭐	3
12	CI/CD	⭐⭐	1
13	Docker	⭐⭐	2
14	FastAPI	⭐⭐⭐	2
15	Streamlit	⭐⭐	1
16	Observabilidad	⭐⭐	1
17	Despliegue	⭐⭐⭐	2
18	Infraestructura	⭐⭐⭐	2
19	Documentación	⭐⭐	2
20	Proyecto Integrador	⭐⭐⭐	2
21	Glosario	⭐	1
22	Checklist	⭐⭐	1
23	Recursos	⭐	1

Total: 43 **ejercicios**

📺 **Recursos externos**: Ver RECURSOS_POR_MODULO.md para videos y cursos complementarios organizados por módulo.

📝 Módulo 01: Python Moderno

Ejercicio 1.1: Type Hints

Objetivo: Añadir type hints a funciones existentes.

```
# ANTES (sin tipos)
def load_data(path):
    return pd.read_csv(path)
```

```
def train_model(X, y, params):
    model = RandomForestClassifier(**params)
    return model.fit(X, y)

# TU TAREA: Añadir type hints completos
# Hint: usa pd.DataFrame, np.ndarray, dict[str, Any]
```

Ejercicio 1.2: Pydantic Config

Objetivo: Crear configuración validada con Pydantic.

```
# Crear una clase ModelConfig que valide:
# - n_estimators: int entre 10 y 500
# - max_depth: int opcional, entre 1 y 50
# - random_state: int, default 42

# TU CÓDIGO AQUÍ
from pydantic import BaseModel, Field

class ModelConfig(BaseModel):
    # ...
    pass
```

Ejercicio 1.3: src/ Layout

Objetivo: Reorganizar código en estructura profesional.

```
# Dado este código en un solo archivo main.py:
# - load_data()
# - preprocess()
# - train()
# - predict()
# - FastAPI app

# TU TAREA: Crear estructura src/ con:
# src/myproject/data.py
# src/myproject/training.py
# src/myproject/prediction.py
# app/fastapi_app.py
```

📝 Módulo 02: Diseño de Sistemas

Ejercicio 2.1: ML Canvas

Objetivo: Completar un ML Canvas para un proyecto.

```
# ML Canvas: [Tu Proyecto]

## 1. Propuesta de Valor
- ¿Qué problema de negocio resuelve?
- ¿Cuál es el impacto medible?

## 2. Datos
- ¿De dónde vienen los datos?
- ¿Qué features son necesarias?

## 3. Modelo
- ¿Qué tipo de modelo (clasificación/regresión)?
- ¿Métricas de éxito?

# TU TAREA: Completar para BankChurn-Predictor
```

Ejercicio 2.2: ADR (Architecture Decision Record)

Objetivo: Documentar una decisión técnica.

```
# ADR-001: Elección de Framework de API

## Estado
Propuesto | Aceptado | Deprecado

## Contexto
¿Por qué necesitamos tomar esta decisión?
```

```
## Opciones Consideradas
1. FastAPI
2. Flask
3. Django REST

## Decisión
¿Cuál elegimos y por qué?

## Consecuencias
- Positivas:
- Negativas:

# TU TAREA: Crear ADR para decisión de modelo en TelecomAI
```

📝 Módulo 03: Estructura de Proyecto

Ejercicio 3.1: Crear src/ Layout

Objetivo: Organizar código en estructura profesional.

```
# TU TAREA: Crear la siguiente estructura para un proyecto "fraud_detector"

fraud-detector/
├── src/
│   └── frauddetector/
│       ├── __init__.py
│       ├── config.py      # Crear con Pydantic
│       ├── data.py        # load_data()
│       ├── features.py    # FeatureEngineer
│       ├── training.py    # FraudTrainer
│       └── pipeline.py    # build_pipeline()
├── tests/
│   └── conftest.py
├── configs/
│   └── config.yaml
├── pyproject.toml
└── Makefile
```

Ejercicio 3.2: pyproject.toml

Objetivo: Configurar proyecto Python moderno.

```
# TU TAREA: Completar pyproject.toml

[build-system]
requires = ["setuptools>=61.0"]
build-backend = "setuptools.build_meta"

[project]
name = "fraud-detector"
version = "0.1.0"
# ... completar dependencies, optional-dependencies, scripts
```

📝 Módulo 04: Entornos

Ejercicio 4.1: Makefile

Objetivo: Crear Makefile con comandos esenciales.

```
# TU TAREA: Crear Makefile con estos targets

.PHONY: install test lint format train serve clean

install:
    # Instalar dependencias

test:
    # Ejecutar tests con coverage

lint:
    # Ejecutar ruff check

format:
    # Formatear con ruff format
```

```
train:
    # Entrenar modelo

serve:
    # Iniciar API

clean:
    # Limpiar artefactos
```

Ejercicio 4.2: Entorno Reproducible

Objetivo: Configurar entorno desde cero.

```
# TU TAREA: Ejecutar estos comandos y documentar cualquier problema

# 1. Crear entorno virtual
python -m venv .venv
source .venv/bin/activate

# 2. Instalar proyecto en modo editable
pip install -e ".[dev]"

# 3. Verificar instalación
python -c "import frauddetector; print(frauddetector.__version__)"

# 4. Ejecutar tests
pytest tests/ -v
```

📝 Módulo 05: Git Profesional

Ejercicio 5.1: Conventional Commits

Objetivo: Escribir commits con formato profesional.

```
# TU TAREA: Convertir estos mensajes a Conventional Commits

# MAL: "fixed bug"
# BIEN: ???

# MAL: "added tests"
# BIEN: ???

# MAL: "updated readme"
# BIEN: ???

# MAL: "refactored code"
# BIEN: ???

# Formato: <type>(<scope>): <description>
# Types: feat, fix, docs, style, refactor, test, chore
```

Ejercicio 5.2: Pre-commit Hooks

Objetivo: Configurar hooks de calidad.

```
# .pre-commit-config.yaml
# TU TAREA: Completar configuración

repos:
  - repo: https://github.com/astral-sh/ruff-pre-commit
    rev: v0.4.4
    hooks:
      # ¿Qué hooks de ruff necesitas?

  - repo: https://github.com/pre-commit/pre-commit-hooks
    rev: v4.6.0
    hooks:
      # ¿Qué hooks básicos necesitas?

# Después: pre-commit install && pre-commit run --all-files
```

📝 Módulo 06: Versionado de Datos

Ejercicio 6.1: Inicializar DVC

Objetivo: Configurar DVC en un proyecto.

```
# TU TAREA: Ejecutar y documentar cada paso

# 1. Inicializar DVC
dvc init

# 2. Añadir remote local (para práctica)
dvc remote add -d localremote /tmp/dvc-storage

# 3. Trackear datos
dvc add data/raw/dataset.csv

# 4. Commitear archivos .dvc
git add data/raw/dataset.csv.dvc data/raw/.gitignore
git commit -m "chore(data): track dataset with DVC"

# PREGUNTA: ¿Qué archivos se crean? ¿Qué contiene el .dvc?
```

Ejercicio 6.2: Pipeline DVC

Objetivo: Definir pipeline reproducible.

```
# dvc.yaml
# TU TAREA: Definir pipeline de 3 stages

stages:
  prepare:
    cmd: python src/data.py
    deps:
      # ¿Qué dependencias?
    outs:
      # ¿Qué outputs?

  train:
    cmd: python src/training.py
    deps:
      # ???
    outs:
      # ???
    metrics:
      # ???

  evaluate:
    cmd: python src/evaluate.py
    deps:
      # ???
    metrics:
      # ???
```

📝 Módulo 07: sklearn Pipelines

Ejercicio 7.1: Pipeline Básico

Objetivo: Crear un pipeline con preprocesamiento.

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier

# TU TAREA: Crear pipeline con:
# 1. StandardScaler para features numéricas
# 2. RandomForestClassifier

pipe = Pipeline([
    # TU CÓDIGO
])
```

Ejercicio 7.2: ColumnTransformer

Objetivo: Procesar columnas numéricas y categóricas por separado.

```
# Dado un DataFrame con:
# - numeric_cols = ['age', 'balance', 'salary']
# - categorical_cols = ['geography', 'gender']

# TU TAREA: Crear ColumnTransformer que:
# - Aplique StandardScaler a numéricas
# - Aplique OneHotEncoder a categóricas

from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder

preprocessor = ColumnTransformer([
    # TU CÓDIGO
])
```

Ejercicio 7.3: Custom Transformer

Objetivo: Crear un transformer personalizado.

```
from sklearn.base import BaseEstimator, TransformerMixin

# TU TAREA: Crear AgeGroupTransformer que:
# - Añada columna 'age_group' basada en rangos de edad
# - 0-30: 'young', 31-50: 'middle', 51+: 'senior'

class AgeGroupTransformer(BaseEstimator, TransformerMixin):
    def fit(self, X, y=None):
        # TU CÓDIGO
        return self

    def transform(self, X):
        # TU CÓDIGO
        pass
```

📝 Módulo 08: Feature Engineering

Ejercicio 8.1: Detectar Data Leakage

Objetivo: Identificar leakage en un pipeline.

```
# CÓDIGO CON LEAKAGE - Encuentra los 3 errores:

df = pd.read_csv('data.csv')

# Error 1: ¿Dónde está?
df['price_category'] = pd.cut(df['price'], bins=3, labels=['low', 'mid', 'high'])

# Error 2: ¿Dónde está?
scaler = StandardScaler()
df[numeric_cols] = scaler.fit_transform(df[numeric_cols])

X_train, X_test, y_train, y_test = train_test_split(df.drop('target', axis=1), df['target'])

# Error 3: ¿Dónde está?
model = RandomForestClassifier()
model.fit(X_train, y_train)
```

Ejercicio 8.2: Pipeline Sin Leakage

Objetivo: Reescribir el código anterior sin leakage.

```
# TU TAREA: Reescribir el ejercicio 8.1 sin data leakage
# Hint: El scaler debe estar DENTRO del pipeline
# Hint: No crear features basadas en el target antes del split
```

📝 Módulo 09: Training Profesional

Ejercicio 9.1: Trainer Class

Objetivo: Implementar clase de entrenamiento profesional.

```
# TU TAREA: Completar la clase FraudTrainer

from pathlib import Path
```

```
import pandas as pd
from sklearn.model_selection import cross_val_score

class FraudTrainer:
    """Entrenador profesional siguiendo patrón del portafolio."""

    def __init__(self, config: dict):
        self.config = config
        self.model_ = None
        self.metrics_ = {}

    def run(self, input_path: Path, output_dir: Path) -> dict:
        """Flujo completo de entrenamiento."""
        # TODO: Implementar:
        # 1. load_data()
        # 2. split_data()
        # 3. cross_validate()
        # 4. fit modelo final
        # 5. evaluate()
        # 6. save_artifacts()
        pass

    def cross_validate(self, X, y) -> dict:
        """Cross-validation con métricas."""
        # TODO: Implementar CV estratificada
        pass
```

Ejercicio 9.2: Reproducibilidad

Objetivo: Garantizar resultados reproducibles.

```
# TU TAREA: Hacer este código 100% reproducible

import numpy as np
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split

# ¿Qué seeds faltan?
X_train, X_test, y_train, y_test = train_test_split(X, y)
model = RandomForestClassifier()
model.fit(X_train, y_train)

# PISTA: np.random.seed, random_state en split, random_state en modelo
```

📝 Módulo 10: Experiment Tracking

Ejercicio 10.1: MLflow Básico

Objetivo: Loggear experimento con MLflow.

```
import mlflow

# TU TAREA: Completar el tracking

mlflow.set_tracking_uri("file:./mlruns")
mlflow.set_experiment("fraud-detection")

with mlflow.start_run():
    # TODO: Log parámetros
    mlflow.log_params({
        # ???
    })

    # TODO: Entrenar modelo
    model.fit(X_train, y_train)

    # TODO: Log métricas
    mlflow.log_metrics({
        # ???
    })

    # TODO: Log modelo
    mlflow.sklearn.log_model(???)
```

Ejercicio 10.2: Comparar Experimentos

Objetivo: Ejecutar y comparar múltiples runs.

```
# TU TAREA: Crear script que entrene 3 modelos diferentes
# y los compare en MLflow UI

models = [
    ("rf", RandomForestClassifier(n_estimators=100)),
    ("gb", GradientBoostingClassifier(n_estimators=100)),
    ("lr", LogisticRegression()),
]

for name, model in models:
    with mlflow.start_run(run_name=name):
        # TODO: Entrenar, evaluar, loggear
        pass

# Después: mlflow ui -> comparar runs
```

📝 Módulo 11: Testing ML

Ejercicio 11.1: Test de Datos

Objetivo: Escribir tests para validar datos.

```
# TU TAREA: Escribir tests que verifiquen:
# 1. No hay valores nulos en columnas críticas
# 2. Valores de 'age' están entre 18 y 100
# 3. 'target' solo contiene 0 y 1

import pytest

def test_no_nulls(sample_data):
    # TU CÓDIGO
    pass

def test_age_range(sample_data):
    # TU CÓDIGO
    pass

def test_target_binary(sample_data):
    # TU CÓDIGO
    pass
```

Ejercicio 11.2: Test de Modelo

Objetivo: Testear que el modelo funciona correctamente.

```
# TU TAREA: Escribir tests que verifiquen:
# 1. El modelo puede hacer fit sin errores
# 2. Las predicciones tienen el shape correcto
# 3. El accuracy es mayor que un baseline (ej: 0.5)

def test_model_fit(trained_model, sample_data):
    # TU CÓDIGO
    pass

def test_predictions_shape(trained_model, sample_data):
    # TU CÓDIGO
    pass

def test_accuracy_above_baseline(trained_model, sample_data):
    # TU CÓDIGO
    pass
```

Ejercicio 11.3: Fixture con conftest.py

Objetivo: Crear fixtures reutilizables.

```
# tests/conftest.py

import pytest
import pandas as pd
```

```
# TU TAREA: Crear fixtures para:
# 1. sample_data: DataFrame con datos de prueba
# 2. trained_model: Modelo ya entrenado
# 3. config: Configuración de prueba

@pytest.fixture
def sample_data():
    # TU CÓDIGO
    pass

@pytest.fixture
def trained_model(sample_data):
    # TU CÓDIGO
    pass
```

📝 Módulo 12: CI/CD

Ejercicio 12.1: GitHub Actions Básico

Objetivo: Crear workflow de CI.

```
# .github/workflows/ci.yml

# TU TAREA: Crear workflow que:
# 1. Se ejecute en push y PR a main
# 2. Use Python 3.11
# 3. Instale dependencias
# 4. Ejecute tests con coverage
# 5. Falle si coverage < 80%

name: CI

on:
  # TU CÓDIGO

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      # TU CÓDIGO
```

📝 Módulo 13: Docker

Ejercicio 13.1: Dockerfile Multi-stage

Objetivo: Crear Dockerfile optimizado.

```
# TU TAREA: Crear Dockerfile que:
# 1. Use multi-stage build
# 2. Stage 1: instalar dependencias
# 3. Stage 2: copiar solo lo necesario
# 4. Use usuario non-root
# 5. Exponga puerto 8000

# Stage 1: Builder
FROM python:3.11-slim AS builder
# TU CÓDIGO

# Stage 2: Runtime
FROM python:3.11-slim
# TU CÓDIGO
```

Ejercicio 13.2: Docker Compose Avanzado

Objetivo: Orquestar stack ML completo.

```
# docker-compose.ml.yml
# TU TAREA: Crear compose que levante:
# 1. API de ML (tu Dockerfile)
# 2. MLflow server
# 3. Prometheus para métricas
# 4. Red compartida entre servicios
# 5. Volúmenes para persistencia

version: '3.8'
```

```
services:
  mlflow:
    image: ghcr.io/mlflow/mlflow:v2.9.0
    # TODO: ports, volumes, environment, healthcheck

  api:
    build: .
    # TODO: ports, depends_on, environment, healthcheck

  prometheus:
    image: prom/prometheus:v2.47.0
    # TODO: ports, volumes, command

networks:
  # TODO: Red compartida

volumes:
  # TODO: Volúmenes persistentes
```

📺 Ver RECURSOS_POR_MODULO.md - Módulo 13 para videos de Docker Compose

📝 Módulo 14: FastAPI

Ejercicio 14.1: Schemas Pydantic

Objetivo: Crear schemas de request/response.

```
# TU TAREA: Crear schemas para API de predicción

from pydantic import BaseModel, Field

class PredictionRequest(BaseModel):
    # Features del modelo
    # TU CÓDIGO
    pass

class PredictionResponse(BaseModel):
    # prediction: int
    # probability: float
    # TU CÓDIGO
    pass
```

Ejercicio 14.2: Endpoint de Predicción

Objetivo: Implementar /predict endpoint.

```
from fastapi import FastAPI, HTTPException

app = FastAPI()

# TU TAREA: Implementar endpoint que:
# 1. Reciba PredictionRequest
# 2. Valide los datos
# 3. Haga predicción con modelo cargado
# 4. Retorne PredictionResponse
# 5. Maneje errores con HTTPException

@app.post("/predict", response_model=PredictionResponse)
async def predict(request: PredictionRequest):
    # TU CÓDIGO
    pass
```

📝 Módulo 15: Streamlit

Ejercicio 15.1: Dashboard de Predicción

Objetivo: Crear dashboard interactivo.

```
# app/streamlit_app.py
# TU TAREA: Crear dashboard con:
# 1. Sidebar con inputs para features
# 2. Botón de predicción
# 3. Mostrar resultado con probabilidad
```

```
# 4. Visualización de importancia de features

import streamlit as st
import pandas as pd
import joblib

st.title("🔮 Fraud Predictor")

# TODO: Sidebar con inputs
with st.sidebar:
    # st.number_input, st.selectbox, etc.
    pass

# TODO: Cargar modelo
@st.cache_resource
def load_model():
    # ???
    pass

# TODO: Botón y predicción
if st.button("Predecir"):
    # ???
    pass

# TODO: Mostrar resultado
# st.success(), st.error(), st.metric()
```

📝 Módulo 16: Observabilidad

Ejercicio 16.1: Logging Estructurado

Objetivo: Implementar logging profesional.

```
# TU TAREA: Configurar logging estructurado

import logging
import json
from datetime import datetime

class JSONFormatter(logging.Formatter):
    """Formatter que produce logs en JSON."""

    def format(self, record):
        log_data = {
            "timestamp": datetime.utcnow().isoformat(),
            "level": record.levelname,
            "message": record.getMessage(),
            "module": record.module,
            # TODO: Añadir más campos útiles
        }
        return json.dumps(log_data)

# TODO: Configurar logger
logger = logging.getLogger(__name__)
# ???

# Uso:
logger.info("Prediction made", extra={"customer_id": 123, "prediction": 1})
```

📝 Módulo 17: Despliegue

Ejercicio 17.1: Dockerfile Multi-stage

Objetivo: Crear un Dockerfile optimizado para producción.

```
# TU TAREA: Crear Dockerfile multi-stage para una API de ML
# Requisitos:
# 1. Stage "builder": Instalar dependencias
# 2. Stage "runtime": Imagen mínima para producción
# 3. Usuario no-root
# 4. Health check

# STAGE 1: Builder
FROM python:3.11-slim AS builder
```

```
# TODO: Instalar dependencias de compilación
# TODO: Copiar requirements e instalar

# STAGE 2: Runtime
FROM python:3.11-slim

# TODO: Crear usuario no-root
# TODO: Copiar solo lo necesario del builder
# TODO: Configurar HEALTHCHECK
# TODO: CMD para ejecutar uvicorn
```

Ejercicio 17.2: Docker Compose para Stack ML

Objetivo: Orquestar múltiples servicios ML.

```
# docker-compose.yml
# TU TAREA: Crear stack con:
# 1. API de ML (FastAPI)
# 2. MLflow server
# 3. Prometheus
# 4. Red compartida
# 5. Volúmenes persistentes

version: '3.8'

services:
  mlflow:
    # TODO: Configurar MLflow server

  api:
    # TODO: Configurar API con dependencia de MLflow

  prometheus:
    # TODO: Configurar Prometheus con config file

networks:
  # TODO: Red compartida

volumes:
  # TODO: Volúmenes persistentes
```

📝 Módulo 18: Infraestructura

Ejercicio 18.1: Kubernetes Deployment

Objetivo: Crear manifests K8s para una API ML.

```
# k8s/deployment.yaml
# TU TAREA: Crear Deployment con:
# 1. 2 replicas
# 2. Resource limits (CPU, memoria)
# 3. Liveness y readiness probes
# 4. Environment variables desde ConfigMap

apiVersion: apps/v1
kind: Deployment
metadata:
  name: bankchurn-api
  # TODO: labels
spec:
  replicas: # ???
  selector:
    # TODO: matchLabels
  template:
    spec:
      containers:
      - name: api
        image: # ???
        ports:
        - containerPort: 8000
        resources:
          # TODO: limits y requests
        livenessProbe:
          # TODO: Configurar probe
        readinessProbe:
```

```
          # TODO: Configurar probe
        envFrom:
          # TODO: ConfigMap reference
```

Ejercicio 18.2: Horizontal Pod Autoscaler

Objetivo: Configurar autoscaling basado en CPU.

```
# k8s/hpa.yaml
# TU TAREA: Crear HPA que:
# 1. Escale entre 2 y 10 pods
# 2. Target CPU utilization: 70%
# 3. Scale down stabilization: 5 minutos

apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: bankchurn-api-hpa
spec:
  scaleTargetRef:
    # TODO: Referencia al Deployment
  minReplicas: # ???
  maxReplicas: # ???
  metrics:
  - type: Resource
    resource:
      # TODO: Configurar métrica de CPU
  behavior:
    scaleDown:
      # TODO: Configurar estabilización
```

📝 Módulo 19: Documentación

Ejercicio 19.1: Model Card

Objetivo: Documentar un modelo siguiendo estándares.

```
<!-- model_card.md -->
<!-- TU TAREA: Completar Model Card para BankChurn -->

# Model Card: BankChurn Predictor

## Model Details
- **Developed by**: ???
- **Model type**: ???
- **Language**: ???
- **License**: ???

## Intended Use
- **Primary use case**: ???
- **Primary users**: ???
- **Out-of-scope uses**: ???

## Training Data
- **Dataset**: ???
- **Size**: ???
- **Features**: ???

## Evaluation Metrics
| Metric | Value |
|--------|-------|
| ROC-AUC | ??? |
| Recall | ??? |
| Precision | ??? |

## Ethical Considerations
- ???

## Limitations
- ???
```

Ejercicio 19.2: Dataset Card

Objetivo: Documentar un dataset siguiendo estándares.

```
<!-- dataset_card.md -->
<!-- TU TAREA: Completar Dataset Card -->

# Dataset Card: Bank Customer Churn

## Dataset Description
- **Source**: ???
- **Size**: ??? rows, ??? columns
- **Time period**: ???

## Features
| Feature | Type | Description |
|---------|------|-------------|
| CreditScore | int | ??? |
| Geography | str | ??? |
| ... | ... | ... |

## Data Quality
- **Missing values**: ???
- **Class balance**: ???

## Ethical Considerations
- **Sensitive attributes**: ???
- **Potential biases**: ???
```

📝 Módulo 20: Proyecto Integrador

Ejercicio 20.1: Integración End-to-End

Objetivo: Crear script que ejecute pipeline completo.

```
# scripts/run_e2e.py
# TU TAREA: Script que:
# 1. Cargue datos
# 2. Entrene modelo
# 3. Evalúe métricas
# 4. Guarde artefactos
# 5. Registre en MLflow
# 6. Pruebe API localmente

import subprocess
import requests
from pathlib import Path

def run_e2e_pipeline():
    """Ejecuta pipeline completo de ML."""

    # 1. Verificar datos
    data_path = Path("data/raw/dataset.csv")
    assert data_path.exists(), "Dataset no encontrado"

    # 2. Entrenar modelo
    # TODO: Ejecutar training script

    # 3. Verificar artefactos
    # TODO: Verificar que model.joblib existe

    # 4. Levantar API
    # TODO: subprocess.Popen para uvicorn

    # 5. Test de integración
    # TODO: requests.post("/predict", ...)

    # 6. Cleanup
    # TODO: Terminar proceso de API

if __name__ == "__main__":
    run_e2e_pipeline()
```

Ejercicio 20.2: Health Check Script

Objetivo: Verificar salud de todos los servicios.

```
# scripts/health_check.py
# TU TAREA: Script que verifique:
# 1. Todos los servicios responden
# 2. Modelos están cargados
# 3. MLflow está accesible
# 4. Genera reporte de estado

import requests
from dataclasses import dataclass
from typing import List

@dataclass
class ServiceHealth:
    name: str
    url: str
    healthy: bool
    details: str

def check_service(name: str, url: str) -> ServiceHealth:
    """Verifica salud de un servicio."""
    # TODO: Implementar
    pass

def check_all_services() -> List[ServiceHealth]:
    """Verifica todos los servicios del stack."""
    services = [
        ("MLflow", "http://localhost:5000/health"),
        ("BankChurn API", "http://localhost:8001/health"),
        ("CarVision API", "http://localhost:8002/health"),
        ("TelecomAI API", "http://localhost:8003/health"),
    ]

    results = []
    for name, url in services:
        # TODO: Verificar cada servicio
        pass

    return results

def generate_report(results: List[ServiceHealth]) -> str:
    """Genera reporte de salud."""
    # TODO: Implementar
    pass

if __name__ == "__main__":
    results = check_all_services()
    print(generate_report(results))
```

📝 Módulo 21: Glosario (Autoestudio)

Ejercicio 21.1: Flashcards de Términos

Objetivo: Crear set de flashcards para memorizar términos clave.

```
<!-- Formato: Pregunta | Respuesta -->

¿Qué es Data Drift? | Cambio en la distribución de features entre training y producción

¿Qué es Concept Drift? | Cambio en la relación P(Y|X) entre features y target

¿Diferencia entre Precision y Recall? | Precision=TP/(TP+FP), Recall=TP/(TP+FN)

¿Qué resuelve class_weight='balanced'? | Penaliza más los errores en la clase minoritaria

<!-- TU TAREA: Añadir 20 flashcards más con términos del glosario -->
```

📝 Módulo 22: Checklist de Producción

Ejercicio 22.1: Auditoría de Proyecto

Objetivo: Aplicar checklist a un proyecto existente.

```
# Auditoría: CarVision-Market-Intelligence
```

```
## Código
- [ ] Type hints en todas las funciones
- [ ] Docstrings en clases y métodos públicos
- [ ] Sin secretos hardcodeados
- [ ] Linting pasando (ruff check)

## Testing
- [ ] Coverage >= 80%
- [ ] Tests de datos (schema, rangos)
- [ ] Tests de modelo (predicciones válidas)
- [ ] Tests de integración (API funcional)

## CI/CD
- [ ] Pipeline ejecuta en cada PR
- [ ] Tests ejecutan en múltiples versiones Python
- [ ] Security scanning configurado
- [ ] Docker build automatizado

## Documentación
- [ ] README actualizado
- [ ] Model Card completado
- [ ] API documentada (Swagger)

## Observabilidad
- [ ] Logging estructurado
- [ ] Métricas de Prometheus
- [ ] Health endpoint implementado

<!-- TU TAREA: Ejecutar esta auditoría en el proyecto real -->
<!-- Marcar items completados y crear issues para pendientes -->
```

📝 Módulo 23: Recursos (Práctica)

Ejercicio 23.1: Plan de Estudio Personalizado

Objetivo: Crear plan basado en gaps identificados.

```
# Mi Plan de Estudio MLOps

## Autoevaluación (1-5)
| Área | Nivel | Prioridad |
|------|:-----:|:---------:|
| Python moderno | ??? | ??? |
| sklearn Pipelines | ??? | ??? |
| Docker | ??? | ??? |
| CI/CD | ??? | ??? |
| Testing | ??? | ??? |
| Observabilidad | ??? | ??? |

## Gaps Identificados
1. ???
2. ???
3. ???

## Recursos Seleccionados
| Gap | Recurso | Tipo | Tiempo |
|-----|---------|------|:------:|
| ??? | ??? | Video/Curso/Doc | ???h |

## Timeline
| Semana | Módulos | Recursos Externos | Entregable |
|:------:|---------|-------------------|------------|
| 1 | ??? | ??? | ??? |
| 2 | ??? | ??? | ??? |

<!-- TU TAREA: Completar basándote en tu autoevaluación honesta -->
```

✅ Soluciones

Ver EJERCICIOS_SOLUCIONES.md para las soluciones detalladas.

📌 Cómo Usar los Ejercicios

1. **Lee el módulo** correspondiente primero
2. **Intenta resolver** sin mirar la solución

3 . **Compara** con el código real del portafolio
4 . **Valida** ejecutando tests cuando sea posible

Referencia Rápida al Portafolio

Módulo	Archivo de Referencia
01	`BankChurn-Predictor/src/bankchurn/config.py`
03	Estructura de cualquier proyecto
07	`BankChurn-Predictor/src/bankchurn/training.py`
08	`CarVision-Market-Intelligence/src/carvision/features.py`
09	`BankChurn-Predictor/src/bankchurn/training.py`
11	`CarVision-Market-Intelligence/tests/conftest.py`
12	`.github/workflows/ci-mlops.yml`
13	`BankChurn-Predictor/Dockerfile`
14	`BankChurn-Predictor/app/fastapi_app.py`
15	`CarVision-Market-Intelligence/app/streamlit_app.py`
16	`infra/prometheus-rules.yaml`
17	`docker-compose.demo.yml`
18	`k8s/bankchurn-deployment.yaml`
19	`BankChurn-Predictor/docs/model_card.md`
20	`scripts/run_demo_tests.sh`
22	`CHECKLIST_RELEASE.md`

SOLUCIONES DE EJERCICIOS

GUÍA MLOPS V5.0

PORTFOLIO EDITION

✅ Soluciones — Ejercicios Guía MLOps

Soluciones detalladas con explicaciones

Módulo 01: Python Moderno

Solución 1.1: Type Hints

```
from typing import Any
import pandas as pd
import numpy as np
from sklearn.ensemble import RandomForestClassifier

def load_data(path: str) -> pd.DataFrame:
    """Load CSV data from path."""
    return pd.read_csv(path)

def train_model(
    X: pd.DataFrame,
    y: np.ndarray,
    params: dict[str, Any]
) -> RandomForestClassifier:
    """Train RandomForest with given parameters."""
    model = RandomForestClassifier(**params)
    return model.fit(X, y)
```

Explicación: Los type hints documentan qué tipos espera y retorna cada función, facilitando el mantenimiento y detectando errores con mypy.

Solución 1.2: Pydantic Config

```
from pydantic import BaseModel, Field

class ModelConfig(BaseModel):
    """Configuration for RandomForest model."""

    n_estimators: int = Field(
        default=100,
        ge=10,
        le=500,
        description="Number of trees in the forest"
    )
    max_depth: int | None = Field(
        default=None,
        ge=1,
        le=50,
        description="Maximum depth of trees"
    )
    random_state: int = Field(
        default=42,
        description="Random seed for reproducibility"
    )

# Uso:
config = ModelConfig(n_estimators=200, max_depth=10)
# Validación automática - esto fallaría:
# config = ModelConfig(n_estimators=1000)  # Error: > 500
```

Módulo 07: sklearn Pipelines

Solución 7.1: Pipeline Básico

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier

pipe = Pipeline([
    ("scaler", StandardScaler()),
    ("model", RandomForestClassifier(n_estimators=100, random_state=42))
])

# Uso:
```

```
pipe.fit(X_train, y_train)
predictions = pipe.predict(X_test)
```

Solución 7.2: ColumnTransformer

```
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.pipeline import Pipeline
from sklearn.ensemble import RandomForestClassifier

numeric_cols = ['age', 'balance', 'salary']
categorical_cols = ['geography', 'gender']

preprocessor = ColumnTransformer([
    ("num", StandardScaler(), numeric_cols),
    ("cat", OneHotEncoder(handle_unknown='ignore'), categorical_cols)
])

# Pipeline completo:
pipe = Pipeline([
    ("preprocess", preprocessor),
    ("model", RandomForestClassifier())
])
```

Solución 7.3: Custom Transformer

```
import pandas as pd
from sklearn.base import BaseEstimator, TransformerMixin

class AgeGroupTransformer(BaseEstimator, TransformerMixin):
    """Transform age into categorical groups."""

    def fit(self, X, y=None):
        return self  # Stateless transformer

    def transform(self, X):
        X = X.copy()
        X['age_group'] = pd.cut(
            X['age'],
            bins=[0, 30, 50, 100],
            labels=['young', 'middle', 'senior']
        )
        return X

# Uso en pipeline:
pipe = Pipeline([
    ("age_groups", AgeGroupTransformer()),
    ("preprocess", preprocessor),
    ("model", RandomForestClassifier())
])
```

Módulo 08: Feature Engineering

Solución 8.1: Detectar Data Leakage

```
# ERROR 1: price_category usa el target (price) antes del split
# Esto causa TARGET LEAKAGE

# ERROR 2: StandardScaler se ajusta a TODO el dataset
# Esto causa TRAIN-TEST CONTAMINATION

# ERROR 3: No hay error aquí, pero los errores anteriores
# ya contaminaron los datos
```

Solución 8.2: Pipeline Sin Leakage

```
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
```

```
df = pd.read_csv('data.csv')

# CORRECCIÓN 1: NO crear price_category (depende del target)
# Si necesitas esta feature, créala SOLO con datos de training

# CORRECCIÓN 2: Split ANTES de cualquier transformación
X = df.drop('target', axis=1)
y = df['target']
X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=42)

# CORRECCIÓN 3: Scaler DENTRO del pipeline
pipe = Pipeline([
    ("scaler", StandardScaler()),  # Se ajusta solo en fit()
    ("model", RandomForestClassifier())
])

# El scaler solo ve X_train durante fit
pipe.fit(X_train, y_train)
score = pipe.score(X_test, y_test)
```

Módulo 11: Testing ML

Solución 11.1: Test de Datos

```
import pytest
import pandas as pd

def test_no_nulls(sample_data):
    """Critical columns should have no null values."""
    critical_cols = ['age', 'balance', 'target']
    for col in critical_cols:
        assert sample_data[col].isnull().sum() == 0, f"Nulls found in {col}"

def test_age_range(sample_data):
    """Age should be between 18 and 100."""
    assert sample_data['age'].min() >= 18, "Age below 18 found"
    assert sample_data['age'].max() <= 100, "Age above 100 found"

def test_target_binary(sample_data):
    """Target should only contain 0 and 1."""
    unique_values = set(sample_data['target'].unique())
    assert unique_values <= {0, 1}, f"Invalid target values: {unique_values}"
```

Solución 11.2: Test de Modelo

```
import numpy as np
from sklearn.metrics import accuracy_score

def test_model_fit(sample_data):
    """Model should fit without errors."""
    X = sample_data.drop('target', axis=1)
    y = sample_data['target']

    model = RandomForestClassifier(n_estimators=10, random_state=42)
    model.fit(X, y)  # Should not raise

    assert hasattr(model, 'classes_'), "Model not fitted"

def test_predictions_shape(trained_model, sample_data):
    """Predictions should match input shape."""
    X = sample_data.drop('target', axis=1)
    predictions = trained_model.predict(X)

    assert predictions.shape[0] == len(X), "Prediction count mismatch"

def test_accuracy_above_baseline(trained_model, sample_data):
    """Model should beat random baseline."""
    X = sample_data.drop('target', axis=1)
    y = sample_data['target']

    predictions = trained_model.predict(X)
    accuracy = accuracy_score(y, predictions)
```

```
    assert accuracy > 0.5, f"Accuracy {accuracy} below baseline 0.5"
```

Solución 11.3: conftest.py

```
# tests/conftest.py

import pytest
import pandas as pd
import numpy as np
from sklearn.ensemble import RandomForestClassifier

@pytest.fixture
def sample_data():
    """Generate sample data for testing."""
    np.random.seed(42)
    n_samples = 100

    return pd.DataFrame({
        'age': np.random.randint(18, 70, n_samples),
        'balance': np.random.uniform(0, 100000, n_samples),
        'salary': np.random.uniform(30000, 150000, n_samples),
        'target': np.random.randint(0, 2, n_samples)
    })

@pytest.fixture
def trained_model(sample_data):
    """Return a trained model."""
    X = sample_data.drop('target', axis=1)
    y = sample_data['target']

    model = RandomForestClassifier(n_estimators=10, random_state=42)
    model.fit(X, y)
    return model

@pytest.fixture
def config():
    """Return test configuration."""
    return {
        'model': {'n_estimators': 10, 'random_state': 42},
        'data': {'test_size': 0.2}
    }
```

Módulo 12: CI/CD

Solución 12.1: GitHub Actions

```
name: CI

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest

    steps:
      - uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: '3.11'

      - name: Install dependencies
        run: |
          pip install --upgrade pip
          pip install -e ".[dev]"

      - name: Run tests with coverage
        run: |
```

```
          pytest --cov=src/ --cov-report=xml --cov-fail-under=80

      - name: Upload coverage
        uses: codecov/codecov-action@v3
        with:
          files: ./coverage.xml
```

Módulo 13: Docker

Solución 13.1: Dockerfile Multi-stage

```
# Stage 1: Builder
FROM python:3.11-slim AS builder

WORKDIR /app

# Install build dependencies
RUN pip install --no-cache-dir --upgrade pip

# Copy and install requirements
COPY requirements.txt .
RUN pip install --no-cache-dir --target=/app/deps -r requirements.txt

# Stage 2: Runtime
FROM python:3.11-slim

# Create non-root user
RUN useradd --create-home --shell /bin/bash appuser

WORKDIR /app

# Copy dependencies from builder
COPY --from=builder /app/deps /usr/local/lib/python3.11/site-packages/

# Copy application code
COPY --chown=appuser:appuser src/ ./src/
COPY --chown=appuser:appuser app/ ./app/
COPY --chown=appuser:appuser artifacts/ ./artifacts/

# Switch to non-root user
USER appuser

EXPOSE 8000

CMD ["python", "-m", "uvicorn", "app.fastapi_app:app", "--host", "0.0.0.0", "--port", "8000"]
```

Módulo 14: FastAPI

Solución 14.1 y 14.2: Schemas y Endpoint

```
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel, Field
import joblib
import pandas as pd

# Schemas
class PredictionRequest(BaseModel):
    age: int = Field(..., ge=18, le=100)
    balance: float = Field(..., ge=0)
    salary: float = Field(..., ge=0)
    geography: str = Field(..., pattern="^(France|Germany|Spain)$")
    gender: str = Field(..., pattern="^(Male|Female)$")

class PredictionResponse(BaseModel):
    prediction: int = Field(..., ge=0, le=1)
    probability: float = Field(..., ge=0, le=1)
    status: str = "success"

# App
app = FastAPI(title="ML Prediction API")

# Load model at startup
model = None
```

```
@app.on_event("startup")
async def load_model():
    global model
    model = joblib.load("artifacts/model.joblib")

@app.get("/health")
async def health():
    return {"status": "healthy", "model_loaded": model is not None}

@app.post("/predict", response_model=PredictionResponse)
async def predict(request: PredictionRequest):
    if model is None:
        raise HTTPException(status_code=503, detail="Model not loaded")

    try:
        # Convert to DataFrame
        df = pd.DataFrame([request.model_dump()])

        # Get prediction and probability
        prediction = int(model.predict(df)[0])
        probability = float(model.predict_proba(df)[0, 1])

        return PredictionResponse(
            prediction=prediction,
            probability=probability
        )
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))
```

Módulo 02: Diseño de Sistemas

Solución 2.1: ML Canvas

```
# ML Canvas: BankChurn-Predictor

## 1. Propuesta de Valor
- Reducir churn de clientes bancarios
- Impacto: Retener 15% más clientes = $2M/año ahorro estimado

## 2. Datos
- Fuente: Base de datos CRM bancaria (10K clientes)
- Features: CreditScore, Age, Balance, Geography, NumOfProducts, IsActiveMember

## 3. Modelo
- Tipo: Clasificación binaria (churn: 0/1)
- Métricas: ROC-AUC > 0.85, Recall > 0.75 (priorizar capturar churners)
- Baseline: 79.6% (predecir siempre "no churn")
```

Solución 2.2: ADR

```
# ADR-001: Elección de RandomForest sobre XGBoost

## Estado
Aceptado

## Contexto
Necesitamos un modelo para predicción de churn que sea interpretable y robusto.

## Opciones Consideradas
1. RandomForest - Interpretable, feature importances nativas
2. XGBoost - Mejor performance en benchmarks
3. LogisticRegression - Muy interpretable pero menos potente

## Decisión
RandomForest porque:
- Feature importances integradas (útil para equipo de negocio)
- No requiere tuning extensivo
- Buen balance accuracy/interpretabilidad

## Consecuencias
- Positivas: Fácil de explicar a stakeholders, pipeline simple
- Negativas: Puede perder 1-2% accuracy vs XGBoost tuneado
```

Módulo 03: Estructura de Proyecto

Solución 3.1: src/ Layout

```
mkdir -p fraud-detector/{src/frauddetector,tests,configs,app,artifacts}

# Crear archivos base
touch fraud-detector/src/frauddetector/{__init__,config,data,features,training,pipeline}.py
touch fraud-detector/tests/conftest.py
touch fraud-detector/configs/config.yaml
touch fraud-detector/{pyproject.toml,Makefile,README.md}
```

Solución 3.2: pyproject.toml

```
[build-system]
requires = ["setuptools>=61.0"]
build-backend = "setuptools.build_meta"

[project]
name = "fraud-detector"
version = "0.1.0"
requires-python = ">=3.10"
dependencies = [
    "pandas>=2.0",
    "scikit-learn>=1.3",
    "pydantic>=2.0",
    "pyyaml>=6.0",
]

[project.optional-dependencies]
dev = ["pytest>=7.0", "pytest-cov>=4.0", "ruff>=0.1"]

[project.scripts]
train = "frauddetector.training:main"
```

Módulo 04: Entornos

Solución 4.1: Makefile

```
.PHONY: install test lint format train serve clean

install:
    pip install -e ".[dev]"

test:
    pytest tests/ --cov=src/frauddetector --cov-report=term-missing --cov-fail-under=80

lint:
    ruff check src/ tests/

format:
    ruff format src/ tests/

train:
    python -m frauddetector.training --config configs/config.yaml

serve:
    uvicorn app.fastapi_app:app --reload --port 8000

clean:
    rm -rf __pycache__ .pytest_cache .ruff_cache htmlcov .coverage
    find . -type d -name "__pycache__" -exec rm -rf {} +
```

Módulo 05: Git Profesional

Solución 5.1: Conventional Commits

```
# "fixed bug" →
fix(pipeline): handle NaN values in categorical columns

# "added tests" →
test(training): add integration tests for cross-validation
```

```
# "updated readme" →
docs(readme): add quick start guide and badges

# "refactored code" →
refactor(features): extract FeatureEngineer to separate module
```

Solución 5.2: Pre-commit Hooks

```yaml
repos:
  - repo: https://github.com/astral-sh/ruff-pre-commit
    rev: v0.4.4
    hooks:
      - id: ruff
        args: [--fix]
      - id: ruff-format

  - repo: https://github.com/pre-commit/pre-commit-hooks
    rev: v4.6.0
    hooks:
      - id: trailing-whitespace
      - id: end-of-file-fixer
      - id: check-yaml
      - id: check-added-large-files
        args: ['--maxkb=1000']
```

Módulo 06: Versionado de Datos

Solución 6.1: Inicializar DVC

```
# Los archivos creados:
# - .dvc/ (directorio de configuración)
# - .dvcignore (gitignore para DVC)
# - data/raw/dataset.csv.dvc (archivo de tracking)
# - data/raw/.gitignore (ignora el CSV real)

# Contenido de dataset.csv.dvc:
# outs:
# - md5: abc123def456...
#   size: 1234567
#   path: dataset.csv
```

Solución 6.2: Pipeline DVC

```yaml
stages:
  prepare:
    cmd: python src/data.py
    deps:
      - data/raw/dataset.csv
      - src/data.py
    outs:
      - data/processed/train.csv
      - data/processed/test.csv

  train:
    cmd: python src/training.py
    deps:
      - data/processed/train.csv
      - src/training.py
      - configs/config.yaml
    outs:
      - artifacts/model.joblib
    metrics:
      - artifacts/metrics.json:
          cache: false

  evaluate:
    cmd: python src/evaluate.py
    deps:
      - data/processed/test.csv
      - artifacts/model.joblib
    metrics:
```

```
      - artifacts/evaluation.json:
          cache: false
```

Módulo 09: Training Profesional

Solución 9.1: Trainer Class

```
from pathlib import Path
import pandas as pd
import joblib
from sklearn.model_selection import cross_val_score, train_test_split
from sklearn.metrics import accuracy_score, f1_score

class FraudTrainer:
    def __init__(self, config: dict):
        self.config = config
        self.model_ = None
        self.metrics_ = {}

    def run(self, input_path: Path, output_dir: Path) -> dict:
        df = self.load_data(input_path)
        X, y = df.drop('target', axis=1), df['target']
        X_train, X_test, y_train, y_test = train_test_split(
            X, y, test_size=0.2, random_state=42, stratify=y
        )

        self.metrics_['cv_scores'] = self.cross_validate(X_train, y_train)

        self.model_ = self.build_pipeline()
        self.model_.fit(X_train, y_train)

        self.metrics_.update(self.evaluate(X_test, y_test))
        self.save_artifacts(output_dir)

        return self.metrics_

    def cross_validate(self, X, y) -> dict:
        scores = cross_val_score(self.build_pipeline(), X, y, cv=5, scoring='f1')
        return {'mean': scores.mean(), 'std': scores.std()}
```

Solución 9.2: Reproducibilidad

```
import numpy as np
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split

SEED = 42

# 1. Seed global de numpy
np.random.seed(SEED)

# 2. random_state en train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=SEED
)

# 3. random_state en modelo
model = RandomForestClassifier(n_estimators=100, random_state=SEED)
model.fit(X_train, y_train)
```

Módulo 10: Experiment Tracking

Solución 10.1: MLflow Básico

```
import mlflow
from sklearn.metrics import accuracy_score, f1_score, roc_auc_score

mlflow.set_tracking_uri("file:./mlruns")
mlflow.set_experiment("fraud-detection")

with mlflow.start_run():
    mlflow.log_params({
        "model_type": "RandomForest",
```

```
        "n_estimators": 100,
        "max_depth": 10,
        "random_state": 42
    })

    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    y_proba = model.predict_proba(X_test)[:, 1]

    mlflow.log_metrics({
        "accuracy": accuracy_score(y_test, y_pred),
        "f1_score": f1_score(y_test, y_pred),
        "roc_auc": roc_auc_score(y_test, y_proba)
    })

    mlflow.sklearn.log_model(model, "model")
```

Módulo 15: Streamlit

Solución 15.1: Dashboard de Predicción

```
import streamlit as st
import pandas as pd
import joblib

st.title("🔮 Fraud Predictor")

with st.sidebar:
    st.header("Input Features")
    age = st.number_input("Age", 18, 100, 35)
    balance = st.number_input("Balance", 0.0, 250000.0, 50000.0)
    salary = st.number_input("Salary", 0.0, 200000.0, 60000.0)
    geography = st.selectbox("Geography", ["France", "Germany", "Spain"])

@st.cache_resource
def load_model():
    return joblib.load("artifacts/model.joblib")

model = load_model()

if st.button("🔮 Predecir"):
    input_df = pd.DataFrame([{
        "age": age, "balance": balance,
        "salary": salary, "geography": geography
    }])

    prediction = model.predict(input_df)[0]
    probability = model.predict_proba(input_df)[0, 1]

    col1, col2 = st.columns(2)
    col1.metric("Predicción", "Fraude" if prediction else "No Fraude")
    col2.metric("Probabilidad", f"{probability:.1%}")

    if prediction:
        st.error("⚠️ Alto riesgo de fraude detectado")
    else:
        st.success("✅ Transacción normal")
```

Módulo 16: Observabilidad

Solución 16.1: Logging Estructurado

```
import logging
import json
from datetime import datetime

class JSONFormatter(logging.Formatter):
    def format(self, record):
        log_data = {
            "timestamp": datetime.utcnow().isoformat(),
            "level": record.levelname,
            "message": record.getMessage(),
            "module": record.module,
            "function": record.funcName,
```

```
            "line": record.lineno,
        }
        # Add extra fields if present
        if hasattr(record, 'customer_id'):
            log_data['customer_id'] = record.customer_id
        if hasattr(record, 'prediction'):
            log_data['prediction'] = record.prediction
        return json.dumps(log_data)

# Configurar logger
logger = logging.getLogger(__name__)
handler = logging.StreamHandler()
handler.setFormatter(JSONFormatter())
logger.addHandler(handler)
logger.setLevel(logging.INFO)

# Uso:
logger.info("Prediction made", extra={"customer_id": 123, "prediction": 1})
# Output: {"timestamp": "2024-12-04T...", "level": "INFO", "message": "Prediction made", "customer_id": 123,
"prediction": 1}
```

Módulo 17: Despliegue

Solución 17.1: Dockerfile Multi-stage

```
# ═══════════════════════════════════════════════════════════════════════
# STAGE 1: Builder - Todas las herramientas de compilación
# ═══════════════════════════════════════════════════════════════════════
FROM python:3.11-slim AS builder

WORKDIR /app

# Dependencias de compilación
RUN apt-get update && apt-get install -y --no-install-recommends \
    build-essential \
    && rm -rf /var/lib/apt/lists/*

# Instalar dependencias Python en directorio aislado
COPY requirements.txt .
RUN pip install --no-cache-dir --target=/app/deps -r requirements.txt

# ═══════════════════════════════════════════════════════════════════════
# STAGE 2: Runtime - Solo lo necesario
# ═══════════════════════════════════════════════════════════════════════
FROM python:3.11-slim

# Usuario no-root (seguridad)
RUN useradd --create-home --shell /bin/bash appuser

WORKDIR /app

# Copiar dependencias del builder
COPY --from=builder /app/deps /usr/local/lib/python3.11/site-packages/

# Copiar código y artefactos
COPY --chown=appuser:appuser src/ ./src/
COPY --chown=appuser:appuser app/ ./app/
COPY --chown=appuser:appuser artifacts/ ./artifacts/

# Cambiar a usuario no-root
USER appuser

# Health check
HEALTHCHECK --interval=30s --timeout=10s --start-period=5s --retries=3 \
    CMD python -c "import requests; requests.get('http://localhost:8000/health')" || exit 1

EXPOSE 8000

CMD ["uvicorn", "app.fastapi_app:app", "--host", "0.0.0.0", "--port", "8000"]
```

Explicación:
- Stage 1 tiene todas las herramientas de build (compiladores, headers)
- Stage 2 solo tiene Python runtime + dependencias instaladas

- Usuario non-root previene escalación de privilegios
- HEALTHCHECK permite orquestadores (K8s, Docker Swarm) verificar salud

Solución 17.2: Docker Compose para Stack ML

```
version: '3.8'

services:
  mlflow:
    image: ghcr.io/mlflow/mlflow:v2.9.0
    container_name: mlflow-server
    ports:
      - "5000:5000"
    volumes:
      - mlflow-data:/mlflow
    environment:
      - MLFLOW_TRACKING_URI=sqlite:///mlflow/mlflow.db
      - MLFLOW_DEFAULT_ARTIFACT_ROOT=/mlflow/artifacts
    command: >
      mlflow server
      --host 0.0.0.0
      --port 5000
      --backend-store-uri sqlite:///mlflow/mlflow.db
      --default-artifact-root /mlflow/artifacts
    networks:
      - mlops-network
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:5000/health"]
      interval: 30s
      timeout: 10s
      retries: 3

  api:
    build:
      context: .
      dockerfile: Dockerfile
    container_name: bankchurn-api
    ports:
      - "8001:8000"
    environment:
      - MLFLOW_TRACKING_URI=http://mlflow:5000
      - MODEL_PATH=/app/artifacts/model.joblib
    volumes:
      - ./artifacts:/app/artifacts:ro
    depends_on:
      mlflow:
        condition: service_healthy
    networks:
      - mlops-network
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:8000/health"]
      interval: 30s
      timeout: 10s
      retries: 3

  prometheus:
    image: prom/prometheus:v2.47.0
    container_name: prometheus
    ports:
      - "9090:9090"
    volumes:
      - ./infra/prometheus-config.yaml:/etc/prometheus/prometheus.yml:ro
      - prometheus-data:/prometheus
    command:
      - '--config.file=/etc/prometheus/prometheus.yml'
      - '--storage.tsdb.path=/prometheus'
    networks:
      - mlops-network

networks:
  mlops-network:
    driver: bridge

volumes:
```

```
  mlflow-data:
  prometheus-data:
```

Explicación:
- `depends_on` con `condition: service_healthy` asegura orden de inicio
- Volúmenes nombrados persisten datos entre reinicios
- Red compartida permite comunicación por nombre de servicio
- Health checks permiten auto-recuperación

Módulo 18: Infraestructura

Solución 18.1: Kubernetes Deployment

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: bankchurn-api
  namespace: mlops
  labels:
    app: bankchurn
    tier: api
spec:
  replicas: 2
  selector:
    matchLabels:
      app: bankchurn
  template:
    metadata:
      labels:
        app: bankchurn
        tier: api
    spec:
      containers:
      - name: api
        image: ghcr.io/duqueom/bankchurn-api:latest
        ports:
        - containerPort: 8000
          name: http
        resources:
          requests:
            memory: "256Mi"
            cpu: "100m"
          limits:
            memory: "512Mi"
            cpu: "500m"
        livenessProbe:
          httpGet:
            path: /health
            port: 8000
          initialDelaySeconds: 10
          periodSeconds: 30
          timeoutSeconds: 5
          failureThreshold: 3
        readinessProbe:
          httpGet:
            path: /health
            port: 8000
          initialDelaySeconds: 5
          periodSeconds: 10
          timeoutSeconds: 3
          failureThreshold: 2
        envFrom:
        - configMapRef:
            name: bankchurn-config
---
apiVersion: v1
kind: ConfigMap
metadata:
  name: bankchurn-config
  namespace: mlops
data:
  MLFLOW_TRACKING_URI: "http://mlflow-service:5000"
  LOG_LEVEL: "INFO"
  MODEL_PATH: "/app/artifacts/model.joblib"
```

```yaml
---
apiVersion: v1
kind: Service
metadata:
  name: bankchurn-service
  namespace: mlops
spec:
  selector:
    app: bankchurn
  ports:
  - port: 80
    targetPort: 8000
  type: ClusterIP
```

Explicación:
- `livenessProbe`: K8s reinicia el pod si falla (container muerto)
- `readinessProbe`: K8s deja de enviar tráfico si falla (container sobrecargado)
- `resources.requests`: Mínimo garantizado
- `resources.limits`: Máximo permitido (OOMKilled si excede)
- ConfigMap externaliza configuración del código

Solución 18.2: Horizontal Pod Autoscaler

```yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: bankchurn-api-hpa
  namespace: mlops
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: bankchurn-api
  minReplicas: 2
  maxReplicas: 10
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 70
  - type: Resource
    resource:
      name: memory
      target:
        type: Utilization
        averageUtilization: 80
  behavior:
    scaleDown:
      stabilizationWindowSeconds: 300  # 5 minutos
      policies:
      - type: Percent
        value: 10
        periodSeconds: 60
    scaleUp:
      stabilizationWindowSeconds: 0
      policies:
      - type: Percent
        value: 100
        periodSeconds: 15
      - type: Pods
        value: 4
        periodSeconds: 15
      selectPolicy: Max
```

Explicación:
- Escala entre 2 y 10 pods basado en CPU (70%) y memoria (80%)
- `scaleDown.stabilizationWindowSeconds: 300`: Espera 5 min antes de escalar hacia abajo (evita flapping)
- `scaleUp`: Agresivo (0s estabilización, hasta 4 pods o 100% más por cada 15s)
- `selectPolicy: Max`: Usa la política que escale más rápido

Módulo 19: Documentación

Solución 19.1: Model Card

```
# Model Card: BankChurn Predictor

## Model Details
- **Developed by**: DuqueOM
- **Model type**: Random Forest Classifier
- **Language**: Python 3.11
- **License**: MIT
- **Version**: 1.0.0
- **Last updated**: 2024-12-04

## Intended Use
- **Primary use case**: Predecir probabilidad de abandono de clientes bancarios
- **Primary users**: Equipos de retención, analistas de negocio
- **Out-of-scope uses**:
  - Decisiones de crédito automatizadas sin supervisión humana
  - Discriminación basada en atributos protegidos

## Training Data
- **Dataset**: Bank Customer Churn Dataset
- **Size**: 10,000 registros
- **Features**: 11 (CreditScore, Age, Geography, Gender, Tenure, Balance, NumOfProducts, HasCrCard, IsActiveMember,
EstimatedSalary)
- **Target**: Exited (0/1)
- **Class distribution**: 80% no-churn, 20% churn

## Evaluation Metrics
| Metric | Value | Confidence Interval |
|--------|-------|---------------------|
| ROC-AUC | 0.86 | [0.84, 0.88] |
| Recall | 0.75 | [0.72, 0.78] |
| Precision | 0.62 | [0.58, 0.66] |
| F1 Score | 0.68 | [0.64, 0.72] |

## Ethical Considerations
- **Fairness**: El modelo usa Geography y Gender. Se recomienda auditar disparidad de tasas de predicción entre grupos.
- **Bias potencial**: El dataset es de una región específica; puede no generalizar a otros mercados.
- **Transparencia**: Feature importances disponibles; se recomienda usar SHAP para explicaciones individuales.

## Limitations
- Entrenado solo con datos históricos 2019-2023
- No considera factores macroeconómicos externos
- Performance puede degradar con drift significativo en distribución de clientes
- Umbral de decisión (0.5) puede necesitar ajuste según costo de FP vs FN

## How to Use
```python
import joblib
model = joblib.load("artifacts/model.joblib")
prediction = model.predict(customer_df)
probability = model.predict_proba(customer_df)[:, 1]
```
```

Solución 19.2: Dataset Card

```markdown
# Dataset Card: Bank Customer Churn

## Dataset Description
- **Source**: Kaggle Bank Customer Churn Dataset
- **Size**: 10,000 rows, 14 columns
- **Time period**: 2019-2023 (simulado)
- **License**: CC0 Public Domain

## Features
| Feature | Type | Description | Range/Values |
|---------|------|-------------|--------------|
| CustomerId | int | Identificador único | - |
| Surname | str | Apellido (no usar en modelo) | - |
| CreditScore | int | Puntuación crediticia | 300-850 |
```

```
| Geography | str | País del cliente | France, Germany, Spain |
| Gender | str | Género | Male, Female |
| Age | int | Edad en años | 18-92 |
| Tenure | int | Años como cliente | 0-10 |
| Balance | float | Saldo en cuenta | 0-250K |
| NumOfProducts | int | Productos contratados | 1-4 |
| HasCrCard | int | Tiene tarjeta de crédito | 0, 1 |
| IsActiveMember | int | Cliente activo | 0, 1 |
| EstimatedSalary | float | Salario estimado | 10K-200K |
| Exited | int | **TARGET**: Abandonó el banco | 0, 1 |

## Data Quality
- **Missing values**: 0%
- **Duplicates**: 0 (verificado por CustomerId)
- **Class balance**: 79.6% no-churn, 20.4% churn

## Preprocessing Applied
1. Drop: CustomerId, Surname (no predictivos)
2. One-hot encoding: Geography, Gender
3. StandardScaler: Features numéricas
4. No se aplica SMOTE (usamos class_weight='balanced')

## Ethical Considerations
- **Sensitive attributes**: Geography, Gender pueden introducir sesgo
- **Potential biases**: Dataset europeo, puede no representar otros mercados
- **Recommendations**: Auditar equidad de predicciones por grupo demográfico

## Citation
```bibtex
@misc{bank_churn_dataset,
  title={Bank Customer Churn Dataset},
  author={Kaggle},
  year={2023},
  url={https://www.kaggle.com/datasets/...}
}
```
```

```
## Módulo 20: Proyecto Integrador

### Solución 20.1: Integración End-to-End

```python
#!/usr/bin/env python3
"""
scripts/run_e2e.py
Pipeline end-to-end: train → evaluate → serve → test API
"""

import subprocess
import requests
import time
import sys
from pathlib import Path

def run_e2e_pipeline():
    """Ejecuta pipeline completo de ML."""

    print("=" * 60)
    print("🚀 PIPELINE E2E - BankChurn Predictor")
    print("=" * 60)

    # 1. Verificar datos
    print("\n📊 [1/6] Verificando datos...")
    data_path = Path("data/raw/churn.csv")
    if not data_path.exists():
        print(f"❌ Dataset no encontrado: {data_path}")
        return False
    print(f"✅ Dataset encontrado: {data_path}")

    # 2. Entrenar modelo
    print("\n🎯 [2/6] Entrenando modelo...")
    result = subprocess.run(
        ["python", "-m", "bankchurn.train"],
```
```

```
        capture_output=True, text=True
    )
    if result.returncode != 0:
        print(f"❌ Training falló: {result.stderr}")
        return False
    print("✅ Modelo entrenado")

    # 3. Verificar artefactos
    print("\n📦 [3/6] Verificando artefactos...")
    model_path = Path("artifacts/model.joblib")
    if not model_path.exists():
        print(f"❌ Modelo no encontrado: {model_path}")
        return False
    print(f"✅ Modelo guardado: {model_path}")

    # 4. Levantar API
    print("\n🌐 [4/6] Iniciando API...")
    api_process = subprocess.Popen(
        ["uvicorn", "app.fastapi_app:app", "--host", "0.0.0.0", "--port", "8000"],
        stdout=subprocess.PIPE, stderr=subprocess.PIPE
    )
    time.sleep(5)  # Esperar inicio

    # 5. Test de integración
    print("\n🧪 [5/6] Ejecutando test de integración...")
    try:
        # Health check
        health = requests.get("http://localhost:8000/health", timeout=5)
        assert health.status_code == 200, f"Health check falló: {health.status_code}"
        print("✅ Health check OK")

        # Prediction test
        test_data = {
            "CreditScore": 650,
            "Geography": "France",
            "Gender": "Male",
            "Age": 35,
            "Tenure": 5,
            "Balance": 50000.0,
            "NumOfProducts": 2,
            "HasCrCard": 1,
            "IsActiveMember": 1,
            "EstimatedSalary": 60000.0
        }
        pred = requests.post("http://localhost:8000/predict", json=test_data, timeout=5)
        assert pred.status_code == 200, f"Prediction falló: {pred.status_code}"
        result = pred.json()
        assert "probability" in result, "Response sin probability"
        print(f"✅ Prediction OK: {result}")

    except Exception as e:
        print(f"❌ Test falló: {e}")
        api_process.terminate()
        return False

    # 6. Cleanup
    print("\n🧹 [6/6] Limpiando...")
    api_process.terminate()
    api_process.wait()
    print("✅ API terminada")

    print("\n" + "=" * 60)
    print("🎉 PIPELINE E2E COMPLETADO EXITOSAMENTE")
    print("=" * 60)
    return True

if __name__ == "__main__":
    success = run_e2e_pipeline()
    sys.exit(0 if success else 1)
```

Solución 20.2: Health Check Script

```
#!/usr/bin/env python3
"""
```

```
scripts/health_check.py
Verifica salud de todos los servicios del stack MLOps
"""

import requests
from dataclasses import dataclass
from typing import List
from datetime import datetime

@dataclass
class ServiceHealth:
    name: str
    url: str
    healthy: bool
    status_code: int | None
    response_time_ms: float
    details: str

def check_service(name: str, url: str, timeout: int = 5) -> ServiceHealth:
    """Verifica salud de un servicio."""
    start = datetime.now()
    try:
        response = requests.get(url, timeout=timeout)
        response_time = (datetime.now() - start).total_seconds() * 1000

        return ServiceHealth(
            name=name,
            url=url,
            healthy=response.status_code == 200,
            status_code=response.status_code,
            response_time_ms=response_time,
            details=response.json() if response.headers.get('content-type', '').startswith('application/json') else
response.text[:100]
        )
    except requests.exceptions.ConnectionError:
        return ServiceHealth(name, url, False, None, 0, "Connection refused")
    except requests.exceptions.Timeout:
        return ServiceHealth(name, url, False, None, timeout * 1000, "Timeout")
    except Exception as e:
        return ServiceHealth(name, url, False, None, 0, str(e))

def check_all_services() -> List[ServiceHealth]:
    """Verifica todos los servicios del stack."""
    services = [
        ("MLflow", "http://localhost:5000/health"),
        ("BankChurn API", "http://localhost:8001/health"),
        ("CarVision API", "http://localhost:8002/health"),
        ("CarVision Dashboard", "http://localhost:8501/healthz"),
        ("TelecomAI API", "http://localhost:8003/health"),
        ("Prometheus", "http://localhost:9090/-/healthy"),
        ("Grafana", "http://localhost:3000/api/health"),
    ]

    return [check_service(name, url) for name, url in services]

def generate_report(results: List[ServiceHealth]) -> str:
    """Genera reporte de salud."""
    healthy_count = sum(1 for r in results if r.healthy)
    total = len(results)

    lines = [
        "=" * 70,
        f"🏥 HEALTH CHECK REPORT - {datetime.now().strftime('%Y-%m-%d %H:%M:%S')}",
        "=" * 70,
        f"Status: {healthy_count}/{total} services healthy",
        "",
        f"{'Service':<20} {'Status':<10} {'Code':<6} {'Time':<10} {'Details'}",
        "-" * 70,
    ]

    for r in results:
        status = "✅ UP" if r.healthy else "❌ DOWN"
        code = str(r.status_code) if r.status_code else "N/A"
        time_str = f"{r.response_time_ms:.0f}ms" if r.response_time_ms > 0 else "N/A"
        details = r.details[:30] + "..." if len(r.details) > 30 else r.details
```

```
        lines.append(f"{r.name:<20} {status:<10} {code:<6} {time_str:<10} {details}")

    lines.extend([
        "-" * 70,
        f"Overall: {'🟢 ALL HEALTHY' if healthy_count == total else '🔴 DEGRADED'}",
        "=" * 70,
    ])

    return "\n".join(lines)

if __name__ == "__main__":
    results = check_all_services()
    print(generate_report(results))

    # Exit code para CI/CD
    import sys
    sys.exit(0 if all(r.healthy for r in results) else 1)
```

Módulo 21: Glosario (Autoestudio)

Solución 21.1: Flashcards de Términos

```
<!-- Formato: Pregunta | Respuesta -->

# Flashcards MLOps - 25 Términos Clave

¿Qué es Data Drift? | Cambio en la distribución de features P(X) entre training y producción

¿Qué es Concept Drift? | Cambio en la relación P(Y|X) entre features y target

¿Diferencia entre Precision y Recall? | Precision=TP/(TP+FP) "de los que predije positivos, cuántos eran reales".
Recall=TP/(TP+FN) "de los positivos reales, cuántos detecté"

¿Qué resuelve class_weight='balanced'? | Penaliza más los errores en la clase minoritaria, inversamente proporcional a
su frecuencia

¿Para qué sirve ColumnTransformer? | Aplicar diferentes transformaciones a diferentes columnas en paralelo dentro de un
Pipeline sklearn

¿Qué es training-serving skew? | Diferencia entre el preprocesamiento en training vs producción que causa predicciones
incorrectas

¿Por qué usar Pipeline en vez de pasos separados? | Evita data leakage, garantiza misma transformación en train/test/
prod, serializa todo junto

¿Qué hace @st.cache_resource? | Cachea recursos pesados (modelos, conexiones DB) que no deben recargarse en cada
interacción de Streamlit

¿Qué es un ADR? | Architecture Decision Record - documento que registra una decisión técnica con contexto, alternativas
y consecuencias

¿Diferencia entre liveness y readiness probe? | Liveness: reinicia pod si falla (muerto). Readiness: deja de enviar
tráfico si falla (sobrecargado)

¿Qué es multi-stage build en Docker? | Separar build (con compiladores) de runtime (solo binarios) para imágenes más
pequeñas y seguras

¿Por qué usuario non-root en Docker? | Seguridad: si el container es comprometido, el atacante no tiene privilegios root

¿Qué es PSI (Population Stability Index)? | Métrica para detectar drift: <0.1 no drift, 0.1-0.2 moderado, >0.2
significativo

¿Qué hace DVC? | Versiona datasets y modelos grandes junto con Git, sin guardarlos en el repo

¿Qué es un Model Card? | Documentación estandarizada de un modelo: propósito, métricas, limitaciones, consideraciones
éticas

¿Para qué sirve HPA en Kubernetes? | Horizontal Pod Autoscaler: escala réplicas automáticamente basado en métricas (CPU,
memoria)

¿Qué es Feature Store? | Repositorio centralizado de features procesadas, reutilizables entre modelos y equipos

¿Qué mide ROC-AUC? | Capacidad discriminatoria del modelo: probabilidad de rankear un positivo por encima de un negativo
```

```
¿Por qué pytest fixtures? | Reutilizar setup de tests (datos, modelos) sin duplicar código, con scope controlado

¿Qué hace mlflow.log_artifact()? | Guarda archivos (gráficos, modelos, CSVs) asociados a un experimento para
reproducibilidad

¿Qué es SMOTE? | Synthetic Minority Over-sampling: genera ejemplos sintéticos de clase minoritaria interpolando entre
vecinos

¿Por qué src/ layout? | Evita importar código local en vez del paquete instalado, estructura profesional instalable

¿Qué son Conventional Commits? | Formato estandarizado de mensajes: type(scope): description. Permite generar CHANGELOGs
automáticos

¿Qué hace conftest.py? | Define fixtures compartidas para todos los tests del directorio, pytest lo descubre
automáticamente

¿Qué es Pydantic? | Librería de validación de datos usando type hints. Valida, serializa y documenta esquemas
automáticamente
```

Módulo 22: Checklist de Producción

Solución 22.1: Auditoría de Proyecto

```
# Auditoría: CarVision-Market-Intelligence

## Código ✅
- [x] Type hints en todas las funciones (verificado: 100% en src/)
- [x] Docstrings en clases y métodos públicos
- [x] Sin secretos hardcodeados (verificado con gitleaks)
- [x] Linting pasando (ruff check src/ → 0 errores)

## Testing ✅
- [x] Coverage >= 80% (actual: 97%)
- [x] Tests de datos (schema, rangos) → test_data.py
- [x] Tests de modelo (predicciones válidas) → test_pipeline.py
- [x] Tests de integración (API funcional) → test_api.py

## CI/CD ✅
- [x] Pipeline ejecuta en cada PR (ci-mlops.yml)
- [x] Tests ejecutan en múltiples versiones Python (3.10, 3.11)
- [x] Security scanning configurado (Bandit, pip-audit, Trivy)
- [x] Docker build automatizado (GHCR push en main)

## Documentación ⚠️
- [x] README actualizado (badges, quickstart, arquitectura)
- [ ] Model Card completado → PENDIENTE: crear docs/model_card.md
- [x] API documentada (Swagger en /docs)

## Observabilidad ⚠️
- [x] Logging estructurado (common_utils.logger)
- [ ] Métricas de Prometheus → PENDIENTE: añadir prometheus_client
- [x] Health endpoint implementado (/health)

## Issues Creados
1. #42: Crear Model Card para CarVision
2. #43: Añadir métricas Prometheus a FastAPI

## Puntuación Estimada
- Código: 18/20
- Pipeline: 18/20
- Testing/CI: 19/20
- APIs: 13/15
- Tracking: 8/10
- Docs: 12/15

**TOTAL: 88/100 → Senior Level 🥈**
```

Módulo 23: Recursos (Práctica)

Solución 23.1: Plan de Estudio Personalizado

```
# Mi Plan de Estudio MLOps

## Autoevaluación (1-5)
```

```
| Área | Nivel | Prioridad |
|------|:-----:|:---------:|
| Python moderno | 4 | Baja |
| sklearn Pipelines | 3 | Alta |
| Docker | 2 | Alta |
| CI/CD | 2 | Alta |
| Testing | 4 | Media |
| Observabilidad | 1 | Alta |

## Gaps Identificados
1. **Docker**: Solo uso básico, no multi-stage ni compose
2. **CI/CD**: No he configurado GitHub Actions desde cero
3. **Observabilidad**: No conozco Prometheus/Grafana

## Recursos Seleccionados
| Gap | Recurso | Tipo | Tiempo |
|-----|---------|------|:------:|
| Docker | [Docker Tutorial - Nana](https://youtube.com/...) 🔴 | Video | 1h |
| Docker | Docker Mastery (Udemy) 🟡 | Curso | 10h |
| CI/CD | [GitHub Actions Tutorial](https://youtube.com/...) 🔴 | Video | 30m |
| CI/CD | Módulo 12 + Ejercicio 12.1 | Guía | 3h |
| Observabilidad | [Prometheus+Grafana - Nana](https://youtube.com/...) 🔴 | Video | 50m |
| Observabilidad | Módulo 16 + prometheus-rules.yaml | Guía | 4h |

## Timeline
| Semana | Módulos | Recursos Externos | Entregable |
|:------:|---------|-------------------|------------|
| 1 | 07, 08 | sklearn Pipeline video | FeatureEngineer class |
| 2 | 11, 12 | CI/CD video + ejercicio | GitHub Actions workflow |
| 3 | 13 | Docker tutorial + compose | Multi-stage Dockerfile |
| 4 | 14, 15 | FastAPI video | API + Streamlit |
| 5 | 16 | Prometheus/Grafana video | Métricas + alertas |
| 6 | 17, 18 | K8s tutorial | Deployment manifest |
| 7 | 19, 20 | - | Model Card + E2E script |
| 8 | 21-23 + Simulacros | - | Speech preparado |

## Checkpoints
- [ ] Semana 2: Primer proyecto con CI verde
- [ ] Semana 4: API funcionando con Docker
- [ ] Semana 6: Stack completo en Kubernetes local
- [ ] Semana 8: Portafolio completo, simulacros hechos
```

PLANTILLAS

GUÍA MLOPS V5.0

PORTFOLIO EDITION

📋 Plantillas Reutilizables — Guía MLOps

Templates listos para usar en tus proyectos ML

📁 **Ver también**: templates/ contiene las plantillas como archivos individuales descargables.

📄 1. Template README.md

```
# 🚀 [Nombre del Proyecto]

[![CI](https://github.com/USER/REPO/actions/workflows/ci.yml/badge.svg)](https://github.com/USER/REPO/actions)
[![Coverage](https://img.shields.io/badge/Coverage-XX%25-brightgreen)](reports/)
[![Python](https://img.shields.io/badge/Python-3.11-blue)](https://python.org)

> Breve descripción del proyecto en una línea.

## 🎯 Métricas del Modelo

| Métrica | Valor |
|---------|-------|
| Accuracy | XX% |
| F1-Score | X.XX |
| Coverage | XX% |

## ⚡ Quick Start

```bash

# Clonar
git clone https://github.com/USER/REPO.git
cd REPO

# Instalar
pip install -e ".[dev]"

# Entrenar
make train

# Tests
make test

# Servir API
make serve
\`\`\`

## 📁 Estructura

\`\`\`
proyecto/
├── src/proyecto/      # Código fuente
├── app/               # APIs
├── tests/             # Tests
├── configs/           # Configuración
└── artifacts/         # Modelos (gitignored)
\`\`\`

## 📖 Documentación

- [Model Card](#mod_model_card)
- [API Reference](#mod_api)
```

📄 2. Template pyproject.toml

```
[build-system]
requires = ["setuptools>=61.0"]
build-backend = "setuptools.build_meta"

[project]
name = "mi-proyecto"
version = "1.0.0"
description = "Descripción del proyecto"
readme = "README.md"
```
```

```
requires-python = ">=3.10"
license = {text = "MIT"}

dependencies = [
    "pandas>=2.0.0",
    "numpy>=1.24.0",
    "scikit-learn>=1.3.0",
    "pydantic>=2.0.0",
    "pyyaml>=6.0",
]

[project.optional-dependencies]
api = ["fastapi>=0.104.0", "uvicorn>=0.24.0"]
dev = ["pytest>=7.4.0", "pytest-cov>=4.1.0", "black>=23.0.0", "ruff>=0.1.0"]
all = ["mi-proyecto[api,dev]"]

[tool.setuptools.packages.find]
where = ["src"]

[tool.pytest.ini_options]
testpaths = ["tests"]
addopts = "-v --cov=src/"

[tool.coverage.run]
source = ["src"]

[tool.coverage.report]
fail_under = 80
```

📄 3. Template Dockerfile

```
# Stage 1: Builder
FROM python:3.11-slim AS builder
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir --target=/deps -r requirements.txt

# Stage 2: Runtime
FROM python:3.11-slim
RUN useradd --create-home appuser
WORKDIR /app
COPY --from=builder /deps /usr/local/lib/python3.11/site-packages/
COPY --chown=appuser:appuser src/ ./src/
COPY --chown=appuser:appuser app/ ./app/
COPY --chown=appuser:appuser artifacts/ ./artifacts/
USER appuser
EXPOSE 8000
HEALTHCHECK --interval=30s --timeout=3s CMD curl -f http://localhost:8000/health || exit 1
CMD ["uvicorn", "app.fastapi_app:app", "--host", "0.0.0.0", "--port", "8000"]
```

📄 4. Template GitHub Actions

```
name: CI

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    strategy:
      matrix:
        python-version: ['3.11', '3.12']

    steps:
      - uses: actions/checkout@v4
```

```
      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: ${{ matrix.python-version }}

      - name: Install dependencies
        run: pip install -e ".[dev]"

      - name: Lint
        run: ruff check src/ tests/

      - name: Test
        run: pytest --cov=src/ --cov-fail-under=80
```

📄 5. Template conftest.py

```
import pytest
import pandas as pd
import numpy as np

@pytest.fixture
def sample_data():
    np.random.seed(42)
    return pd.DataFrame({
        'feature1': np.random.randn(100),
        'feature2': np.random.randn(100),
        'target': np.random.randint(0, 2, 100)
    })

@pytest.fixture
def config():
    return {
        'model': {'n_estimators': 10, 'random_state': 42},
        'data': {'test_size': 0.2}
    }
```

📄 6. Template Model Card

```
# Model Card: [Nombre del Modelo]

## Información General
- **Desarrollador**: [Tu nombre]
- **Fecha**: [Fecha]
- **Versión**: 1.0.0
- **Tipo**: Clasificación binaria

## Uso Previsto
- **Usuarios**: [Quién usará el modelo]
- **Casos de uso**: [Para qué se usará]

## Métricas

| Métrica | Train | Test |
|---------|-------|------|
| Accuracy | X% | X% |
| Precision | X% | X% |
| Recall | X% | X% |
| F1 | X% | X% |

## Limitaciones
- [Limitación 1]
- [Limitación 2]

## Consideraciones Éticas
- [Consideración 1]
```

📄 7. Template .gitignore

```
# Python
__pycache__/
*.py[cod]
*.egg-info/
dist/
build/

# Environments
.venv/
venv/

# Data & Models
data/
artifacts/
*.joblib
mlruns/

# IDE
.vscode/
.idea/

# Coverage
.coverage
htmlcov/

# Env
.env
```

📄 8. Template Makefile

```
.PHONY: install test lint train serve clean

install:
pip install -e ".[dev]"

test:
pytest --cov=src/ --cov-fail-under=80

lint:
ruff check src/ tests/
black --check src/ tests/

format:
black src/ tests/
ruff check --fix src/ tests/

train:
python -m src.proyecto.training

serve:
uvicorn app.fastapi_app:app --reload --port 8000

clean:
rm -rf __pycache__ .pytest_cache .coverage htmlcov
```

📚 Módulos que Usan Estas Plantillas

Plantilla	Módulo
README, pyproject, Makefile	03_ESTRUCTURA_PROYECTO.md
GitHub Actions	12_CI_CD.md
Dockerfile	13_DOCKER.md
Model Card	19_DOCUMENTACION.md

[← Volver al Índice](# mod_ 0 0 _INDICE) | [templates/](# mod_index)

DECISIONES TÉCNICAS

GUÍA MLOPS V5.0

PORTFOLIO EDITION

⚖️ Decisiones Técnicas — ADRs del Portafolio

Por qué elegimos cada herramienta y tecnología

Última actualización: Diciembre 2025

📋 Índice de ADRs

#	Decisión	Alternativas	Módulo	Estado
001	Python 3.11+	R, Julia	01	✅ Aceptada
002	scikit-learn	XGBoost, LightGBM	07	✅ Aceptada
003	Pydantic v2	dataclasses, attrs	01	✅ Aceptada
004	FastAPI	Flask, Django	14	✅ Aceptada
005	pytest	unittest	11	✅ Aceptada
006	GitHub Actions	Jenkins, GitLab CI	12	✅ Aceptada
007	MLflow	W&B, Neptune	10	✅ Aceptada
008	Docker	Conda, Poetry	13, 17	✅ Aceptada
009	DVC	Git LFS, S3 directo	06	✅ Aceptada
010	Streamlit	Gradio, Panel	15	✅ Aceptada
011	Prometheus + Grafana	Datadog, New Relic	16	✅ Aceptada
012	Kubernetes	Docker Swarm, ECS	18	✅ Aceptada
013	Ruff	Flake8 + Black + isort	01	✅ Aceptada
014	src/ Layout	Flat layout	03	✅ Aceptada

ADR-001: Python 3.11+

Contexto

Necesitamos un lenguaje para todo el stack ML.

Decisión

Usar Python 3.11+ como lenguaje principal.

Alternativas Consideradas

- **R**: Mejor para estadística, peor para APIs y producción
- **Julia**: Más rápido, ecosistema menos maduro

Consecuencias

- ✅ Ecosistema ML más completo
- ✅ FastAPI, Pydantic nativos
- ✅ Mayor pool de talento
- ❌ Más lento que lenguajes compilados

ADR-002: scikit-learn para Modelos

Contexto

Necesitamos un framework ML para clasificación/regresión tabular.

Decisión

Usar scikit-learn como framework principal.

Alternativas Consideradas

- **XGBoost/LightGBM**: Más performance, menos integración con pipelines
- **PyTorch**: Overkill para datos tabulares

Consecuencias

- ✅ Pipelines unificados con `Pipeline` y `ColumnTransformer`
- ✅ Fácil de testear y serializar
- ✅ Documentación excelente
- ❌ Menos performance que gradient boosting dedicado

ADR-003: Pydantic v2 para Configuración

Contexto

Necesitamos validar configuración de forma robusta.

Decisión

Usar Pydantic v 2 para todas las configuraciones.

Alternativas Consideradas

- **dataclasses**: Sin validación built-in
- **attrs**: Menos popular, similar funcionalidad
- **Dict/YAML directo**: Sin validación

Consecuencias

- ✅ Validación automática de tipos
- ✅ Errores claros en config inválida
- ✅ Integración perfecta con FastAPI
- ❌ Dependencia adicional

Ejemplo:

```
class ModelConfig(BaseModel):
    n_estimators: int = Field(ge=10, le=500)
    max_depth: int | None = Field(default=None, ge=1)
```

ADR-004: FastAPI para APIs

Contexto

Necesitamos servir modelos via HTTP.

Decisión

Usar FastAPI para todas las APIs.

Alternativas Consideradas

- **Flask**: Más simple, sin async, sin docs automáticas
- **Django**: Overkill para APIs ML
- **gRPC**: Más complejo, mejor para microservicios internos

Consecuencias

- ✅ Async por defecto
- ✅ Docs OpenAPI automáticas
- ✅ Validación con Pydantic integrada
- ✅ Rendimiento excelente
- ❌ Menos tutoriales que Flask

ADR-005: pytest para Testing

Contexto

Necesitamos un framework de testing.

Decisión

Usar pytest con pytest-cov.

Alternativas Consideradas

- **unittest**: Más verboso, menos features
- **nose** 2 : Abandonado

Consecuencias

- ✅ Fixtures potentes
- ✅ Plugins (pytest-cov, pytest-mock)

- ✅ Sintaxis simple con assert
- ✅ Parametrización fácil

ADR-006: GitHub Actions para CI/CD

Contexto

Necesitamos CI/CD automatizado.

Decisión

Usar GitHub Actions.

Alternativas Consideradas

- **Jenkins**: Self-hosted, más mantenimiento
- **GitLab CI**: Requiere migrar repos
- **CircleCI**: Costo adicional

Consecuencias

- ✅ Integrado con GitHub
- ✅ Gratis para repos públicos
- ✅ Matrix testing fácil
- ✅ Marketplace de actions
- ❌ Vendor lock-in con GitHub

ADR-007: MLflow para Tracking

Contexto

Necesitamos tracking de experimentos y registry de modelos.

Decisión

Usar MLflow (local + server).

Alternativas Consideradas

- **W&B (Weights & Biases)**: Mejor UI, costo para equipos
- **Neptune**: Similar a W&B
- **DVC**: Más para datos que experimentos

Consecuencias

- ✅ Open source, sin vendor lock-in
- ✅ Model Registry integrado
- ✅ Funciona local sin servidor
- ❌ UI menos moderna que W&B

ADR-008: Docker para Empaquetado

Contexto

Necesitamos empaquetar aplicaciones para deploy.

Decisión

Usar Docker con multi-stage builds.

Alternativas Consideradas

- **Conda pack**: Solo Python, sin proceso completo
- **Poetry**: Solo dependencias, no containerización

Consecuencias

- ✅ Reproducibilidad total
- ✅ Funciona en cualquier cloud
- ✅ Compose para desarrollo local
- ❌ Overhead de imagen

ADR-009: DVC para Versionado de Datos

Contexto

Necesitamos versionar datasets grandes sin guardarlos en Git.

Decisión

Usar DVC (Data Version Control).

Alternativas Consideradas

- **Git LFS**: Pago por storage, menos features
- **S 3 directo**: Sin versionado semántico
- **Delta Lake**: Overkill para nuestro tamaño

Consecuencias

- ✅ Versionado semántico de datos
- ✅ Pipelines reproducibles
- ✅ Integración con Git
- ❌ Curva de aprendizaje adicional

📖 Ver Módulo 0 6

ADR-010: Streamlit para Dashboards

Contexto

Necesitamos crear dashboards interactivos para stakeholders.

Decisión

Usar Streamlit para dashboards ML.

Alternativas Consideradas

- **Gradio**: Más simple, menos personalizable
- **Panel**: Menos popular, más verboso
- **Dash**: Más complejo, mejor para apps empresariales

Consecuencias

- ✅ Python puro, sin HTML/CSS/JS
- ✅ Reactivo por defecto
- ✅ Caching de modelos integrado
- ✅ Deploy fácil (Streamlit Cloud)
- ❌ Menos control sobre UI que frameworks web

📖 Ver Módulo 1 5

ADR-011: Prometheus + Grafana para Observabilidad

Contexto

Necesitamos monitorear modelos en producción y detectar drift.

Decisión

Usar Prometheus para métricas y Grafana para dashboards.

Alternativas Consideradas

- **Datadog**: Excelente pero costoso
- **New Relic**: Similar a Datadog
- **CloudWatch/Stackdriver**: Vendor lock-in

Consecuencias

- ✅ Open source, sin costo
- ✅ Estándar de la industria
- ✅ Alertas configurables
- ✅ Integración con K 8 s nativa
- ❌ Más setup que SaaS

📖 Ver Módulo 1 6

ADR-012: Kubernetes para Orquestación

Contexto

Necesitamos orquestar contenedores en producción con auto-scaling.

Decisión

Usar Kubernetes para deployment.

Alternativas Consideradas

- **Docker Swarm**: Más simple, menos features
- **ECS/Fargate**: Vendor lock-in AWS
- **Nomad**: Menos adopción

Consecuencias

- ✅ Estándar de la industria
- ✅ Auto-scaling (HPA)
- ✅ Self-healing (probes)
- ✅ Portable entre clouds
- ❌ Curva de aprendizaje alta

📖 Ver Módulo 18

ADR-013: Ruff para Linting

Contexto

Necesitamos herramientas de calidad de código rápidas.

Decisión

Usar Ruff como linter y formateador unificado.

Alternativas Consideradas

- **Flake8 + Black + isort**: Múltiples herramientas, más lento
- **Pylint**: Muy lento, muchos false positives

Consecuencias

- ✅ 10-100x más rápido que alternativas
- ✅ Una herramienta = una config
- ✅ Compatible con reglas de Flake8
- ✅ Formateador incluido
- ❌ Herramienta relativamente nueva

📖 Ver Módulo 01 - Glosario: Ruff

ADR-014: src/ Layout para Proyectos

Contexto

Necesitamos una estructura de proyecto profesional e instalable.

Decisión

Usar src/ layout en todos los proyectos.

Alternativas Consideradas

- **Flat layout**: Más simple pero problemático con imports
- **Monorepo**: Más complejo para este tamaño

Consecuencias

- ✅ Evita importar código local en vez del paquete
- ✅ Estructura profesional estándar
- ✅ Instalable con `pip install -e .`
- ❌ Un nivel más de directorios

📖 Ver Módulo 03

📊 Matriz de Decisiones Resumen

Área	Herramienta	Por qué	Módulo
Lenguaje	Python 3.11+	Ecosistema ML	01
ML Framework	scikit-learn	Pipelines unificados	07
Config	Pydantic v2	Validación + FastAPI	01

Área	Herramienta	Por qué	Módulo
API	FastAPI	Async + docs auto	14
Dashboard	Streamlit	Python puro, reactivo	15
Testing	pytest	Fixtures + plugins	11
CI/CD	GitHub Actions	Integración nativa	12
Tracking	MLflow	Open source + local	10
Versionado datos	DVC	Git + datos grandes	06
Container	Docker	Reproducibilidad	13, 17
Orquestación	Kubernetes	Auto-scaling, probes	18
Monitoreo	Prometheus + Grafana	Open source, estándar	16
Linting	Ruff	Rápido, unificado	01
Estructura	src/ layout	Profesional, instalable	03

🔗 Referencias

- RECURSOS_POR_MODULO.md - Videos y cursos por herramienta
- 21_GLOSARIO.md - Definiciones detalladas de cada herramienta

RÚBRICA DE EVALUACIÓN

GUÍA MLOPS V5.0

PORTFOLIO EDITION

📊 Rúbrica de Evaluación — Portfolio MLOps

Criterios profesionales para evaluar proyectos ML

🎯 Puntuación Total: 100 puntos

Rango	Nivel	Descripción
90-100	**Staff/Principal**	Listo para liderar equipos ML
80-89	**Senior**	Production-ready, contratación inmediata
70-79	**Mid-Level**	Sólido, necesita pulir detalles
60-69	**Junior+**	Funcional, falta madurez
<60	**En desarrollo**	Requiere más trabajo

📋 Criterios de Evaluación

1. Calidad del Código (20 puntos)

Aspecto	Puntos	Criterio
Type hints	5	100% de funciones públicas tipadas
Docstrings	3	Todas las clases y funciones documentadas
Pydantic configs	4	Configuración validada, no dicts crudos
src/ layout	4	Estructura profesional instalable
SOLID principles	4	Código modular y extensible

Ejemplo 5/5 **en type hints:**

```
def predict(self, features: pd.DataFrame) -> np.ndarray:
    """Generate predictions for input features."""
    return self.model.predict(features)
```

2. Pipeline ML (20 puntos)

Aspecto	Puntos	Criterio
sklearn Pipeline	6	Pipeline unificado (no pasos sueltos)
ColumnTransformer	4	Preprocessing organizado
Custom Transformer	4	Al menos 1 transformer propio
Data leakage prevention	4	drop_columns correcto, sin target leak
Reproducibilidad	2	random_state fijado

Ejemplo 6/6 **en Pipeline:**

```
pipe = Pipeline([
    ("features", FeatureEngineer()),
    ("preprocess", ColumnTransformer([...])),
    ("model", RandomForestClassifier())
])
```

3. Testing y CI/CD (20 puntos)

Aspecto	Puntos	Criterio
Coverage ≥ 80%	6	Medido con pytest-cov
Unit tests	4	Tests de funciones individuales
Integration tests	4	Tests de pipeline completo
GitHub Actions	4	CI automático en cada push
Security scanning	2	Bandit, pip-audit, o similar

Ejemplo 6/6 **en Coverage:**

```
# ci.yml
- name: Test with coverage
  run: pytest --cov=src/ --cov-fail-under=80
```

4. Containerización y APIs (15 puntos)

Aspecto	Puntos	Criterio
Dockerfile multi-stage	4	Build y runtime separados
Non-root user	2	Seguridad básica
FastAPI schemas	4	Pydantic request/response
Health endpoint	2	/health funcional
Error handling	3	Respuestas HTTP correctas

5. Experiment Tracking (10 puntos)

Aspecto	Puntos	Criterio
MLflow logging	4	Params, metrics, artifacts
Model Registry	3	Modelo registrado con versión
Comparación experimentos	3	Múltiples runs comparables

6. Documentación (15 puntos)

Aspecto	Puntos	Criterio
README profesional	5	Badges, quickstart, arquitectura
Model Card	4	Performance, limitaciones, uso
Docstrings	3	Código autodocumentado
ADRs	3	Decisiones técnicas explicadas

📊 Checklist Rápido por Proyecto

BankChurn-Predictor

- [ ] Pipeline con ResampleClassifier
- [ ] Coverage ≥ 79%
- [ ] MLflow tracking
- [ ] FastAPI /predict endpoint
- [ ] Dockerfile funcional

CarVision-Market-Intelligence

- [ ] FeatureEngineer custom transformer
- [ ] Coverage ≥ 80%
- [ ] Streamlit dashboard
- [ ] drop_columns para evitar leakage

TelecomAI-Customer-Intelligence

- [ ] Pipeline sklearn completo
- [ ] Coverage ≥ 80%
- [ ] Múltiples modelos comparados
- [ ] API funcional

🏆 Niveles de Certificación

Nivel	Puntuación	Badge
MLOps Practitioner	70-79	🥉
MLOps Engineer	80-89	🥈
Senior MLOps Engineer	90-94	🥇
Staff MLOps Engineer	95-100	💎

📚 Evaluación por Módulo

Sistema de autoevaluación para cada fase del programa.

Fase 1: Fundamentos (Módulos 01-06)

Módulo	Criterio de Aprobación	Ejercicio Requerido
01	Type hints en 100% funciones, config Pydantic	1.1, 1.2
02	Diagrama C4 de un proyecto, ADR documentado	2.1
03	Proyecto con src/ layout instalable	3.1
04	requirements.txt + lockfile, .env funcional	4.1
05	pre-commit configurado, commits convencionales	5.1
06	DVC pipeline funcional, remote configurado	6.1

Checkpoint Fase 1: Proyecto con estructura profesional, versionado con DVC

Fase 2: ML Engineering (Módulos 07-10)

Módulo	Criterio de Aprobación	Ejercicio Requerido
07	Pipeline sklearn unificado, ColumnTransformer	7.1, 7.2
08	Custom Transformer (FeatureEngineer o similar)	8.1
09	Clase Trainer con fit/predict, cross-validation	9.1
10	MLflow tracking: params, metrics, artifacts	10.1

Checkpoint Fase 2: Modelo entrenado con pipeline unificado, experimentos en MLflow

Fase 3: MLOps Core (Módulos 11-16)

Módulo	Criterio de Aprobación	Ejercicio Requerido
11	Tests con ≥ 80% coverage, conftest.py	11.1, 11.2

Módulo	Criterio de Aprobación	Ejercicio Requerido
12	GitHub Actions CI funcionando en cada push	12.1
13	Dockerfile multi-stage, non-root user	13.1 (→17.1)
14	FastAPI /predict + /health, schemas Pydantic	14.1, 14.2
15	Dashboard Streamlit funcional	15.1
16	Logging JSON estructurado	16.1

Checkpoint Fase 3: API dockerizada con CI/CD verde, ≥ 80% coverage

Fase 4: Producción (Módulos 17-18)

Módulo	Criterio de Aprobación	Ejercicio Requerido
17	Docker Compose con API + MLflow + Prometheus	17.2
18	K8s Deployment con probes, HPA configurado	18.1, 18.2

Checkpoint Fase 4: Stack completo desplegable en K8s local

Fase 5: Especialización (Módulos 19-23)

Módulo	Criterio de Aprobación	Ejercicio Requerido
19	Model Card + Dataset Card completados	19.1, 19.2
20	Script E2E funcionando	20.1
21	Flashcards revisadas, términos dominados	21.1
22	Auditoría de proyecto completada	22.1
23	Plan de estudio personalizado	23.1

Checkpoint Fase 5: Portafolio documentado, listo para entrevistas

🎯 Autoevaluación Rápida

Completa esta tabla honestamente para identificar tus gaps:

```
| Competencia | 1-5 | Gap? | Recurso |
|-------------|:---:|:----:|---------|
| Type hints + Pydantic | _ | | Módulo 01 |
| sklearn Pipeline | _ | | Módulo 07 |
| Testing (pytest) | _ | | Módulo 11 |
| GitHub Actions | _ | | Módulo 12 |
| Docker | _ | | Módulo 13, 17 |
| FastAPI | _ | | Módulo 14 |
| MLflow | _ | | Módulo 10 |
| Observabilidad | _ | | Módulo 16 |
| Kubernetes | _ | | Módulo 18 |
```

📺 Ver RECURSOS_POR_MODULO.md para videos y cursos según tus gaps

APÉNDICE A: SPEECH PORTAFOLIO

GUÍA MLOPS V5.0

PORTFOLIO EDITION

Speech de entrevista – Portafolio ML/MLOps (5–7 minutos)

0. Objetivo del speech

Este guion está pensado para una respuesta de **“Cuéntame de tu portafolio / experiencia en ML/MLOps”** en 5–7 **minutos**.
Te posiciona explícitamente como **Senior / Staff ML(MLOps) Engineer** que diseña y opera **sistemas ML multi-proyecto**.

1. Apertura (0:00 – 1:00)

Mensaje clave: Quién eres, qué rol buscas, y una frase muy clara sobre el portafolio.

- Preséntate con rol y foco:
- "Soy Daniel Duque, ML/MLOps Engineer, y mi foco es llevar modelos de ML a producción con buenas prácticas de ingeniería de software, observabilidad y automatización."
- En una frase, resume el portfolio:
- "Para demostrarlo construí un portafolio compuesto por 3 **proyectos ML de negocio reales** (churn bancario, pricing de vehículos y recomendación de planes de telecom), todos integrados en un **ecosistema MLOps unificado**."
- Aclara el nivel que quieres transmitir:
- "Más que tres demos aisladas, es **un sistema multi-servicio**, con CI/CD compartido, MLflow centralizado, Docker/Kubernetes, monitoreo con Prometheus/Grafana y workflows de drift + retraining."

2. Vista global del portfolio (1:00 – 2:30)

Mensaje clave: No son proyectos sueltos; hay una plataforma y decisiones de arquitectura.

- Estructura:
- "El repo tiene tres servicios: **BankChurn-Predictor**, **CarVision-Market-Intelligence** y **TelecomAI-Customer-Intelligence**."
- "Comparten librerías comunes, una **pipeline de CI unificada** en GitHub Actions y un **stack de observabilidad e infraestructura** coherente."
- CI/CD:
- "La CI se define en un único workflow con **matrices de Python** y **umbrales de cobertura por proyecto** (≈ 79–97%), integración con **GHCR** para las imágenes Docker y **tests de integración** con Docker Compose."
- Plataforma MLOps:
- "Uso **MLflow** como capa de experiment tracking y model registry común."
- "Para producción describo despliegue con **Docker + Kubernetes**, IaC con **Terraform**, y monitoreo con **Prometheus + Grafana**."
- Temas avanzados:
- "Incluí **explainability con SHAP**, **data drift monitoring con Evidently** y un workflow de **retrain orquestado por GitHub Actions** en el caso de churn."
- "También documento **FinOps** y el diseño de un **Feature Store** compartido como extensiones futuras."

Frase puente:

> "Sobre esa plataforma, cada proyecto resuelve un problema de negocio distinto. Te resumo cada uno muy rápido."

3. Proyecto 1 – BankChurn-Predictor (2:30 – 3:45)

Mensaje clave: Proyecto “estrella” de churn con MLOps profundo.

- Problema y valor:
- "BankChurn-Predictor es un sistema para **predecir churn de clientes bancarios**, priorizando recall para reducir fuga de clientes."
- Modelado:
- "Uso un **ensemble** con modelos clásicos y técnicas de balanceo de clases, con métricas sólidas documentadas en la Model Card."
- "Todo está encapsulado en pipelines de sklearn para evitar discrepancias entre entrenamiento y serving."
- MLOps:
- "Tiene **alta cobertura de tests**, incluyendo rutas complejas como `explainability.py` con SHAP y fallbacks cuando la librería no está disponible."
- "Está integrado con **MLflow** para registrar experimentos, métricas y versiones de modelo."

- Explainability:
- "Implementé una clase `ModelExplainer` basada en **SHAP** con mecanismos de fallback (coeficientes / feature_importances) y pruebas unitarias que cubren esos caminos."
- Drift + Retrain:
- "Para este proyecto añadí un flujo de **data drift monitoring** con **Evidently** que genera reportes y, de forma manual controlada, puede disparar el workflow de **retrain** en GitHub Actions."
- Documentación:
- "El proyecto tiene **ARCHITECTURE.md**, README muy detallado y Model Card, lo que permite entender decisiones de diseño, trade-offs y roadmap (Feature Store, explainability avanzada, etc.)."

Frase de cierre:

> "BankChurn muestra mi capacidad de diseñar un sistema ML **end-to-end** con explainability, drift y retraining, no sólo un modelo."

4. Proyecto 2 – CarVision-Market-Intelligence (3:45 – 5:00)

Mensaje clave: Pricing de vehículos con API + dashboard y foco fuerte en experiencia de usuario y calidad de código.

- Problema:
- "CarVision es un proyecto de **pricing de vehículos**: predice el precio de mercado a partir de características del coche, ayudando a tomar decisiones de compra/venta."
- Modelado:
- "Uso modelos de regresión (por ejemplo, RandomForest Regressor) comparados con alternativas lineales, con métricas como **R^2** y **RMSE** documentadas."
- "La parte interesante es el diseño de un **FeatureEngineer** como transformer de sklearn para mantener alineados entrenamiento y serving."
- Producto / UX:
- "Además de la **API de inferencia**, hay un **dashboard en Streamlit** con 4 secciones (overview, análisis de mercado, métricas del modelo y predictor interactivo), pensado para stakeholders no técnicos."
- MLOps:
- "La cobertura de tests es muy alta (alrededor del 97%), y se integra en la misma CI compartida usando MLflow, Docker, GHCR, etc."
- Valor:
- "Este proyecto demuestra que puedo llevar un caso de uso tabular a un producto usable con API, UI, tracking de experimentos y buenas prácticas de ingeniería."

Frase de cierre:

> "CarVision refuerza mi perfil **producto + plataforma**, mostrando cómo empaqueto modelos en APIs y dashboards operables."

5. Proyecto 3 – TelecomAI-Customer-Intelligence (5:00 – 6:00)

Mensaje clave: Recomendador de planes con métricas claras y buen diseño operando como microservicio robusto.

- Problema:
- "TelecomAI es un sistema de **recomendación de planes de telecom** (Standard vs Ultra) basado en el comportamiento de uso."
- Métricas:
- "El modelo consigue aproximadamente $\mathbf{0.84}$ **de AUC-ROC**, 82**% de accuracy** y $\mathbf{0.63}$ **de F**1, con una **cobertura de tests ~** 97**%**."
- Servicio:
- "Está implementado como una **API FastAPI** con endpoints `/predict` y `/health`, y con **métricas de Prometheus** expuestas en `/metrics` para observabilidad."
- "El health check maneja estados degraded (por ejemplo cuando el modelo no está disponible en CI), lo que es muy realista a nivel de producción."
- Plataforma:
- "Comparte el mismo pipeline de CI/CD, prácticas de Docker y experiment tracking del resto del portfolio."
- Documentación:
- "Tiene un **ONE-PAGER de negocio**, README completo, Model Card y un **ARCHITECTURE.md** alineado con los otros dos proyectos."

Frase de cierre:

> "TelecomAI es un microservicio ML muy limpio y con excelente coverage, que demuestra consistencia en mis patrones de diseño."

6. Cierre (6:00 – 7:00)

Mensaje clave: Reforzar nivel, madurez y cómo esto se traduce en valor para el equipo.

- Nivel:
- "En conjunto, el portafolio está diseñado para demostrar un nivel **Senior / Staff**: no sólo modelos, sino **sistemas completos**, con CI/CD multi-proyecto, observabilidad, workflows de retraining y documentación estilo empresa."
- Aprendizajes:
- "En el proceso he trabajado temas como explainability, drift, diseño de Feature Store, y **FinOps** para estimar y optimizar costes en cloud."
- Cómo encaja en la empresa:
- "Mi objetivo es traer este enfoque al equipo: **estandarizar pipelines**, mejorar **confiabilidad y observabilidad** de los modelos en producción y reducir el tiempo desde prototipo hasta valor en negocio."

Cierre corto:

> "Si queréis, puedo profundizar en cualquiera de los tres proyectos, en la parte de CI/CD, en explainability o en cómo diseñaría el Feature Store y el coste en vuestra infraestructura actual."

Apéndice A — Material de Preparación para Entrevistas

APÉNDICE B: TALKING POINTS

GUÍA MLOPS V5.0

PORTFOLIO EDITION

Talking Points – Portafolio ML/MLOps

Guía rápida para responder preguntas de entrevista sobre el portafolio y cada proyecto.

1. Mensaje global del portafolio

- **Rol objetivo**
- Apunta a **Senior / Staff ML(MLOps) Engineer**.
- Demuestra capacidad de diseñar y operar un **ecosistema multi-servicio**.
- **Arquitectura global**
- 3 proyectos: churn bancario, pricing vehículos, recomendación de planes telecom.
- Librerías comunes y estructura homogénea de proyectos.
- **MLflow centralizado**, **Docker + Kubernetes**, **Terraform** descrito para IaC.
- **CI/CD y calidad**
- Un único workflow (`ci-mlops.yml`) con **matrices de Python** y **umbrales de cobertura por proyecto**.
- Coverage ≈ 79**%+** en BankChurn y ≈ 97**%** en CarVision y TelecomAI.
- Jobs de **integration tests**, build y push de imágenes a **GHCR**, escaneos de seguridad.
- **Observabilidad y operaciones**
- APIs con **/metrics** vía Prometheus.
- Integración con **Grafana** descrita en docs.
- **RUNBOOK** y `OPERATIONS_PORTFOLIO.md` para procedimientos operativos.
- **Temas avanzados**
- **Explainability** (SHAP + fallbacks) en BankChurn.
- **Data drift monitoring** con Evidently + reporte y recomendaciones de **retrain**.
- Diseño de **Feature Store** compartido como futura extensión.
- Sección específica de **FinOps** (costes, right-sizing, buenas prácticas de infra).

2. BankChurn-Predictor – Talking Points

- **Problema de negocio**
- Reducir **churn de clientes bancarios** identificando quién probablemente se marchará.
- Prioriza recall y estabilidad del modelo; impacto directo en retención.
- **Modeling & ML**
- Pipelines de sklearn con **preprocesamiento + modelo**.
- **Ensemble** y técnicas de desbalanceo (SMOTE / resampling).
- Métricas sólidas (AUC, F1) documentadas en Model Card.
- **MLOps / Plataforma**
- **Alta cobertura de tests** (incluyendo rutas complejas como explainability).
- Seguimiento de experimentos en **MLflow**.
- Makefile con comandos estándar (`train`, `evaluate`, `check-drift`, `serve`, etc.).
- **Explainability**
- Clase `ModelExplainer` con:
  - SHAP (Tree/KernelExplainer).
  - Fallbacks (coeficientes o `feature_importances_`).
  - Pruebas unitarias para cubrir escenarios sin SHAP o sin explainer.
- **Drift & Retrain**
- Script de drift basado en **Evidently**.
- Workflow `drift-bankchurn.yml`: genera reportes y recomendaciones.
- Workflow `retrain-bankchurn.yml`: reentrena y registra modelos en MLflow, con opción de promoción.
- **Diferenciadores para entrevista**
- Caso donde puedes hablar de:
  - trade-offs de **automatizar o no** el retraining,
  - relación entre drift y **métricas de negocio**,

- cómo integrar esto con un equipo real (alertas, SLOs).

3. CarVision-Market-Intelligence – Talking Points

- **Problema de negocio**
- Predicción del **precio de vehículos** para soporte a decisiones de compra/venta.
- **Modeling & ML**
- Modelos de regresión (ej. RandomForest) comparados con alternativas.
- Métricas como **R^2** y **RMSE** (documentadas en la Model Card).
- Diseño de **FeatureEngineer** reutilizable como transformer sklearn.
- **Producto y UX**
- **API de inferencia** para integrarse con otros sistemas.
- **Dashboard Streamlit** con:
  - overview del dataset,
  - análisis de mercado,
  - métricas de modelo,
  - predictor interactivo.
- **MLOps / Calidad**
- Coverage ≈ 97%.
- Integración con CI unificada, MLflow, Docker, GHCR.
- Buen manejo de configuración, logging y estructura de paquetes.
- **Diferenciadores para entrevista**
- Ejemplo claro de pasar de un **modelo en notebook** a un **producto utilizable**.
- Permite hablar de experiencia colaborando con negocio (dashboard, explicaciones visuales).

4. TelecomAI-Customer-Intelligence – Talking Points

- **Problema de negocio**
- **Recomendar el plan de telecom óptimo** (Standard vs Ultra) según uso.
- Minimiza tanto la fuga por sobre-coste como la pérdida de ingresos por infra-venta.
- **Modeling & ML**
- Modelos tipo GradientBoosting / RandomForest.
- Métricas (documentadas y fáciles de citar):
  - **AUC-ROC ≈** 0.84
  - **Accuracy ≈** 82%
  - **F**1 ≈ 0.63
  - **Cobertura de tests ≈** 97%
- **Servicio y operaciones**
- API **FastAPI** con:
  - `/predict` (plan recomendado + probabilidad),
  - `/health` (incluye estado **degraded** para escenarios sin modelo).
- Exposición de métricas en `/metrics` para Prometheus.
- Arquitectura y operaciones documentadas en `ARCHITECTURE.md` y ONE-PAGER.
- **MLOps / Plataforma**
- Comparte CI, MLflow, Docker, GHCR del resto.
- Casos de test e 2 e para validar comportamiento end-to-end.
- **Diferenciadores para entrevista**
- Excelente ejemplo de **microservicio ML** con:
  - health checks realistas,
  - alta cobertura,
  - integración en un ecosistema mayor.

5. Preguntas frecuentes y ángulos de respuesta

- **“¿En qué se diferencia esto de un proyecto típico de Kaggle?”**
- Enfatizar:
  - sistemas multi-servicio,
  - CI/CD, cobertura, observabilidad,
  - documentación de arquitectura y operaciones.
- **“¿Qué harías si tuvieras más tiempo?”**
- Implementar el **Feature Store real**, extender drift monitoring a los 3 proyectos, añadir **auth, RBAC y multi-tenant** en las APIs, y pipelines de datos en streaming.
- **“¿Cómo aplicarías esto en nuestra empresa?”**
- Responder con:
  - estandarizar pipelines y CI/CD,
  - mejorar visibilidad de modelos en producción (logs, métricas, alertas),
  - diseñar **runbooks** y **playbooks de incidentes**,
  - definir una estrategia de **FinOps** básica para workloads de ML.

6. Uso recomendado de esta guía

- Imprimir o tener abierta esta hoja durante simulacros de entrevista.
- Practicar el **speech de** 5 – 7 **minutos** del archivo APENDICE_A_SPEECH_PORTAFOLIO.md.
- Usar estas bullets para responder preguntas de seguimiento sin perder el mensaje principal:
- soluciones de negocio,
- calidad técnica,
- visión de plataforma.

Apéndice B — Material de Preparación para Entrevistas

GUÍA AUDIOVISUAL

GUÍA MLOPS V5.0

PORTFOLIO EDITION

🎬 Guía de Material Audiovisual — ML-MLOps Portfolio

Guía completa para crear demos profesionales de tu portafolio

Última actualización: Diciembre 2025
Versión: 5.1 — Portfolio Edition
Repositorio: github.com/DuqueOM/ML-MLOps-Portfolio

📋 Índice

1. Estado Actual del Portafolio
2. Servicios del Stack Demo
3. Material Audiovisual Requerido
4. Herramientas Recomendadas
5. Guía de GIFs Demostrativos
6. Guía de Screenshots
7. Guía de Video Principal
8. Comandos y Scripts Útiles
9. Checklist Final

🎯 Estado Actual del Portafolio

Proyectos del Portafolio

Proyecto	Descripción	Tecnologías Clave
BankChurn-Predictor	Predicción de abandono bancario	sklearn Pipeline, ResampleClassifier, MLflow
CarVision-Market-Intelligence	Predicción de precios de vehículos	FeatureEngineer transformer, Streamlit dashboard
TelecomAI-Customer-Intelligence	Clasificación de planes móviles	Pipeline ML unificado

Métricas Actuales

Proyecto	Coverage	Métrica Principal	CI Status
BankChurn	79%	86% AUC	✅ Passing
CarVision	80%	0.87 R^2	✅ Passing
TelecomAI	80%	82% Accuracy	✅ Passing

🖥️ Servicios del Stack Demo

Comando para Levantar

```
docker-compose -f docker-compose.demo.yml up -d
```

5 Servicios Principales

```
┌─────────────────────────────────────────────────────────────────────┐
│                    SERVICIOS DEL STACK DEMO                         │
├─────────────────────────────────────────────────────────────────────┤
│                                                                     │
│   🔹 MLFLOW TRACKING SERVER                                         │
│      URL: http://localhost:5000                                     │
│      Función: Tracking de experimentos, Model Registry              │
│      Mostrar: Lista de experimentos, métricas, modelos registrados  │
│                                                                     │
│   🔹 BANKCHURN API (FastAPI)                                        │
│      URL: http://localhost:8001/docs                                │
│      Función: Predicción de abandono de clientes                    │
│      Mostrar: Swagger UI, endpoint /predict, respuesta JSON         │
│                                                                     │
```

```
│   🔹 CARVISION API (FastAPI)                                          │
│      URL: http://localhost:8002/docs                                   │
│      Función: Predicción de precios de vehículos                       │
│      Mostrar: Swagger UI, endpoint /predict                            │
│                                                                        │
│   🔹 CARVISION STREAMLIT DASHBOARD                                     │
│      URL: http://localhost:8501                                        │
│      Función: Dashboard interactivo para análisis y predicción         │
│      Mostrar: Gráficos, formulario de predicción, resultados           │
│                                                                        │
│   🔹 TELECOMAI API (FastAPI)                                           │
│      URL: http://localhost:8003/docs                                   │
│      Función: Clasificación de planes móviles                          │
│      Mostrar: Swagger UI, endpoint /predict                            │
│                                                                        │
├────────────────────────────────────────────────────────────────────────┤
│  SERVICIOS OPCIONALES (con --profile monitoring)                       │
├────────────────────────────────────────────────────────────────────────┤
│                                                                        │
│   🔸 PROMETHEUS: http://localhost:9090                                 │
│   🔸 GRAFANA:    http://localhost:3000 (admin/admin)                   │
│                                                                        │
└────────────────────────────────────────────────────────────────────────┘
```

Resumen de URLs

Servicio	Puerto	URL Completa	Tipo
MLflow UI	5000	http://localhost:5000	Dashboard
BankChurn API	8001	http://localhost:8001/docs	Swagger
CarVision API	8002	http://localhost:8002/docs	Swagger
CarVision Dashboard	8501	**http://localhost:8501**	**Streamlit**
TelecomAI API	8003	http://localhost:8003/docs	Swagger
Prometheus	9090	http://localhost:9090	Monitoring
Grafana	3000	http://localhost:3000	Dashboards

📊 Material Audiovisual Requerido

Resumen de Elementos

Categoría	Cantidad	Prioridad	Descripción
GIFs Demostrativos	5	🔴 Alta	Portfolio, 3 APIs, Streamlit
Screenshots	8	🟡 Media	UIs, dashboards, CI
Video Demo Principal	1	🔴 Alta	3-5 min completo
Thumbnails	4	🟢 Baja	Para YouTube/docs

Mapa de Archivos → Referencias

Archivo	Ubicación	Se usa en
`portfolio-demo.gif`	`media/gifs/`	README.md principal
`bankchurn-preview.gif`	`media/gifs/`	README.md, BankChurn/README.md
`carvision-preview.gif`	`media/gifs/`	README.md, CarVision/README.md
`streamlit-carvision.gif`	`media/gifs/`	CarVision/README.md

Archivo	Ubicación	Se usa en
`telecom-preview.gif`	`media/gifs/`	README.md, TelecomAI/README.md
`mlflow-experiments.png`	`media/screenshots/`	docs/, READMEs
`mlflow-model-registry.png`	`media/screenshots/`	docs/
`swagger-bankchurn.png`	`media/screenshots/`	BankChurn/README.md
`swagger-carvision.png`	`media/screenshots/`	CarVision/README.md
`swagger-telecom.png`	`media/screenshots/`	TelecomAI/README.md
`streamlit-dashboard.png`	`media/screenshots/`	CarVision/README.md
`github-actions-ci.png`	`media/screenshots/`	README.md principal

🛠️ Herramientas Recomendadas

Para Windows

Herramienta	Uso	Instalación
OBS Studio	Grabar pantalla	`winget install OBSProject.OBSStudio`
Greenshot	Screenshots	`winget install Greenshot.Greenshot`
ffmpeg	Convertir video→GIF	`winget install ffmpeg`
ShareX	GIFs directos	`winget install ShareX.ShareX`

Para Linux

Herramienta	Uso	Instalación
OBS Studio	Grabar pantalla	`sudo apt install obs-studio`
Flameshot	Screenshots	`sudo apt install flameshot`
ffmpeg	Convertir video→GIF	`sudo apt install ffmpeg`
Peek	GIFs directos	`sudo apt install peek`

Para macOS

Herramienta	Uso	Instalación
OBS Studio	Grabar pantalla	`brew install obs`
Screenshot nativo	Screenshots	Cmd+Shift+4
ffmpeg	Convertir video→GIF	`brew install ffmpeg`
Gifski	GIFs de alta calidad	`brew install gifski`

🎞️ Guía de GIFs Demostrativos

GIF 1: Portfolio Demo Principal (TODOS los servicios)

Archivo: `media/gifs/portfolio-demo.gif`
Duración: 20-25 segundos
Resolución: 800x600

Guion Detallado

```
┌──────────────────────────────────────────────────────────────────────┐
│              PORTFOLIO DEMO PRINCIPAL (20-25 segundos)                 │
├──────────────────────────────────────────────────────────────────────┤
│                                                                        │
│  0:00-0:03  ESCENA 1: Levantar servicios                              │
│  ──────────────────────────────────────────────────────────────────── │
│  • Terminal con comando:                                               │
│    docker-compose -f docker-compose.demo.yml up -d                     │
│  • Mostrar output: "Creating mlflow-server...",                        │
│    "Creating bankchurn-api...", "Creating carvision-api...",           │
│    "Creating carvision-dashboard...", "Creating telecom-api..."        │
│                                                                        │
│  0:03-0:08  ESCENA 2: Los 5 servicios funcionando                     │
│  ──────────────────────────────────────────────────────────────────── │
│  • Abrir 5 pestañas del navegador (split screen o en secuencia):       │
│    1. http://localhost:5000 (MLflow)                                   │
│    2. http://localhost:8001/docs (BankChurn Swagger)                   │
│    3. http://localhost:8002/docs (CarVision Swagger)                   │
│    4. http://localhost:8501 (CarVision Streamlit) ← IMPORTANTE         │
│    5. http://localhost:8003/docs (TelecomAI Swagger)                   │
│  • Pausar 2 segundos en cada una                                       │
│                                                                        │
│  0:08-0:13  ESCENA 3: Demo Streamlit Dashboard                        │
│  ──────────────────────────────────────────────────────────────────── │
│  • Enfocar en localhost:8501                                           │
│  • Mostrar gráficos de análisis de datos                               │
│  • Llenar formulario de predicción rápido                              │
│  • Mostrar resultado de precio estimado                                │
│                                                                        │
│  0:13-0:18  ESCENA 4: Predicción en API                               │
│  ──────────────────────────────────────────────────────────────────── │
│  • Cambiar a BankChurn Swagger (localhost:8001/docs)                   │
│  • Click en POST /predict → "Try it out"                               │
│  • Ejecutar y mostrar respuesta JSON                                   │
│                                                                        │
│  0:18-0:22  ESCENA 5: MLflow Experiments                              │
│  ──────────────────────────────────────────────────────────────────── │
│  • Cambiar a MLflow (localhost:5000)                                   │
│  • Mostrar lista de experimentos                                       │
│  • Click en un experimento para ver métricas                           │
│                                                                        │
│  0:22-0:25  ESCENA 6: Cierre                                          │
│  ──────────────────────────────────────────────────────────────────── │
│  • Volver a vista general con las 5 pestañas                           │
│  • O mostrar terminal con "docker-compose ps" (5 running)              │
│                                                                        │
└──────────────────────────────────────────────────────────────────────┘
```

Preparación Completa

```
# 1. Levantar todos los servicios
cd /path/to/ML-MLOps-Portfolio
docker-compose -f docker-compose.demo.yml up -d

# 2. Esperar a que estén listos (importante!)
echo "Esperando 45 segundos para que todos los servicios inicien..."
sleep 45

# 3. Verificar TODOS los servicios
echo "=== Verificando 5 servicios ==="
echo "MLflow:" && curl -s http://localhost:5000/health 2>/dev/null || echo "OK"
echo "BankChurn:" && curl -s http://localhost:8001/health
echo "CarVision API:" && curl -s http://localhost:8002/health
echo "CarVision Streamlit:" && curl -s http://localhost:8501 >/dev/null && echo '{"status":"healthy"}'
echo "TelecomAI:" && curl -s http://localhost:8003/health

# 4. Ver estado de contenedores
docker-compose -f docker-compose.demo.yml ps

# 5. Abrir TODAS las pestañas
# Linux:
xdg-open http://localhost:5000      # MLflow
```

```
xdg-open http://localhost:8001/docs # BankChurn
xdg-open http://localhost:8002/docs # CarVision API
xdg-open http://localhost:8501      # CarVision Streamlit ← NO OLVIDAR
xdg-open http://localhost:8003/docs # TelecomAI

# Windows (PowerShell):
# Start-Process http://localhost:5000
# Start-Process http://localhost:8001/docs
# Start-Process http://localhost:8002/docs
# Start-Process http://localhost:8501
# Start-Process http://localhost:8003/docs
```

GIF 2: BankChurn API Demo

Archivo: `media/gifs/bankchurn-preview.gif`
Duración: 8-10 segundos
Resolución: 800x600

Guion

```
┌──────────────────────────────────────────────────────────────────────┐
│                    BANKCHURN DEMO (8-10 segundos)                    │
├──────────────────────────────────────────────────────────────────────┤
│                                                                      │
│  0:00-0:02  Swagger UI de BankChurn                                 │
│  • Mostrar http://localhost:8001/docs                                │
│  • Título visible: "BankChurn Predictor API"                         │
│                                                                      │
│  0:02-0:05  Expandir /predict                                       │
│  • Click en POST /predict                                            │
│  • Click "Try it out"                                                │
│  • Llenar con datos de ejemplo (ver abajo)                           │
│                                                                      │
│  0:05-0:08  Ejecutar y ver resultado                                │
│  • Click "Execute"                                                   │
│  • Scroll para ver respuesta:                                        │
│    {                                                                 │
│      "prediction": 0,                                                │
│      "probability": 0.23,                                            │
│      "label": "No Churn"                                             │
│    }                                                                 │
│                                                                      │
└──────────────────────────────────────────────────────────────────────┘
```

Datos de Ejemplo para BankChurn

```
{
  "credit_score": 650,
  "age": 45,
  "tenure": 5,
  "balance": 50000,
  "num_of_products": 2,
  "has_cr_card": 1,
  "is_active_member": 1,
  "estimated_salary": 75000,
  "geography": "France",
  "gender": "Male"
}
```

GIF 3: CarVision API Demo

Archivo: `media/gifs/carvision-preview.gif`
Duración: 8-10 segundos
Resolución: 800x600

Guion

```
┌──────────────────────────────────────────────────────────────────────┐
│                    CARVISION API DEMO (8-10 segundos)                │
├──────────────────────────────────────────────────────────────────────┤
│                                                                      │
│  0:00-0:02  Swagger UI de CarVision                                 │
│  • Mostrar http://localhost:8002/docs                                │
│  • Título: "CarVision Market Intelligence API"                       │
```

```
│                                                                    │
│  0:02-0:05  Expandir /predict                                      │
│  • Click en POST /predict                                          │
│  • "Try it out"                                                    │
│  • Llenar datos de vehículo                                        │
│                                                                    │
│  0:05-0:08  Resultado                                              │
│  • Ejecutar predicción                                             │
│  • Mostrar precio estimado: {"predicted_price": 25430.50}          │
│                                                                    │
└────────────────────────────────────────────────────────────────────┘
```

Datos de Ejemplo para CarVision

```
{
  "model_year": 2020,
  "model": "toyota camry",
  "condition": "good",
  "odometer": 35000,
  "fuel": "gas",
  "transmission": "automatic",
  "type": "sedan",
  "paint_color": "white"
}
```

GIF 4: CarVision Streamlit Dashboard (NUEVO - IMPORTANTE)

Archivo: `media/gifs/streamlit-carvision.gif`
Duración: 12-15 segundos
Resolución: 800x600

Guion Detallado

```
┌────────────────────────────────────────────────────────────────────┐
│              CARVISION STREAMLIT DEMO (12-15 segundos)             │
├────────────────────────────────────────────────────────────────────┤
│                                                                    │
│  0:00-0:03  Dashboard Principal                                    │
│  ──────────────────────────────────────────────────────────────    │
│  • Mostrar http://localhost:8501                                   │
│  • Vista inicial del dashboard con título                          │
│  • Sidebar visible con opciones                                    │
│                                                                    │
│  0:03-0:06  Sección de Análisis de Datos                           │
│  ──────────────────────────────────────────────────────────────    │
│  • Mostrar gráficos de distribución de precios                     │
│  • Gráfico de precios por marca/año                                │
│  • Estadísticas descriptivas                                       │
│                                                                    │
│  0:06-0:10  Formulario de Predicción                               │
│  ──────────────────────────────────────────────────────────────    │
│  • Navegar a sección de predicción                                 │
│  • Seleccionar marca: Toyota                                       │
│  • Seleccionar modelo: Camry                                       │
│  • Año: 2020                                                       │
│  • Kilometraje: 35,000                                             │
│  • Condición: Good                                                 │
│                                                                    │
│  0:10-0:13  Resultado de Predicción                                │
│  ──────────────────────────────────────────────────────────────    │
│  • Click en botón "Predecir Precio"                                │
│  • Mostrar resultado: "$25,430" (grande, visible)                  │
│  • Mostrar intervalo de confianza si existe                        │
│                                                                    │
│  0:13-0:15  Vista Final                                            │
│  ──────────────────────────────────────────────────────────────    │
│  • Scroll up para mostrar todo el dashboard                        │
│  • O cambiar a otra sección brevemente                             │
│                                                                    │
└────────────────────────────────────────────────────────────────────┘
```

GIF 5: TelecomAI API Demo

Archivo: `media/gifs/telecom-preview.gif`
Duración: 8 segundos
Resolución: 800x600

Guion

```
┌──────────────────────────────────────────────────────────────────┐
│                    TELECOMAI DEMO (8 segundos)                   │
├──────────────────────────────────────────────────────────────────┤
│                                                                  │
│  0:00-0:02  Swagger UI de TelecomAI                              │
│  • Mostrar http://localhost:8003/docs                            │
│  • Título: "TelecomAI Customer Intelligence API"                 │
│                                                                  │
│  0:02-0:05  Expandir /predict                                    │
│  • Llenar datos de uso del cliente                               │
│                                                                  │
│  0:05-0:08  Resultado                                            │
│  • Ejecutar predicción                                           │
│  • Mostrar plan recomendado                                      │
│                                                                  │
└──────────────────────────────────────────────────────────────────┘
```

GIF 6: MLflow Dashboard (NUEVO - Recomendado)

Archivo: `media/gifs/mlflow-demo.gif`
Duración: 10-12 segundos
Resolución: 800x600

Guion

```
┌──────────────────────────────────────────────────────────────────┐
│                    MLFLOW DEMO (10-12 segundos)                  │
├──────────────────────────────────────────────────────────────────┤
│                                                                  │
│  0:00-0:03  MLflow UI Principal                                  │
│  • Mostrar http://localhost:5000                                 │
│  • Lista de experimentos visible                                 │
│                                                                  │
│  0:03-0:06  Seleccionar Experimento                              │
│  • Click en experimento "bankchurn" o "carvision"                │
│  • Mostrar lista de runs                                         │
│                                                                  │
│  0:06-0:09  Ver Métricas                                         │
│  • Click en un run específico                                    │
│  • Mostrar métricas: AUC, F1, Accuracy                           │
│  • Mostrar parámetros logueados                                  │
│                                                                  │
│  0:09-0:12  Model Artifacts                                      │
│  • Mostrar sección de artifacts                                  │
│  • Modelo guardado visible                                       │
│                                                                  │
└──────────────────────────────────────────────────────────────────┘
```

📸 Guía de Screenshots

Screenshots Requeridos (8 total)

#	Nombre	Qué capturar	URL
1	`mlflow-experiments.png`	Lista de experimentos en MLflow	localhost:5000
2	`mlflow-metrics.png`	Gráficos de métricas de un run	localhost:5000
3	`swagger-bankchurn.png`	Swagger UI de BankChurn	localhost:8001/docs
4	`swagger-carvision.png`	Swagger UI de CarVision	localhost:8002/docs
5	`swagger-telecom.png`	Swagger UI de TelecomAI	localhost:8003/docs

#	Nombre	Qué capturar	URL
6	`streamlit-dashboard.png`	Dashboard Streamlit completo	localhost: 8501
7	`streamlit-prediction.png`	Resultado de predicción en Streamlit	localhost: 8501
8	`github-actions-ci.png`	CI pipeline pasando	GitHub

Cómo Tomar Buenos Screenshots

1. **Usa zoom al** 100% en el navegador
2. **Limpia la URL bar** (quita extensiones visibles)
3. **Usa modo claro** para mejor legibilidad en docs
4. **Resolución mínima**: 1200×800
5. **Comprime** después con `pngquant`

```
# Comprimir todos los screenshots
for f in media/screenshots/*.png; do
  pngquant --quality=65-80 "$f" --output "${f%.png}-opt.png"
done
```

🎥 Guía de Video Principal

Especificaciones

Campo	Valor
Duración	4-6 minutos
Resolución	1080p (1920×1080)
Formato	MP4
Audio	Narración clara
Plataforma	YouTube (unlisted) o Google Drive

Estructura del Video (Actualizada)

```
┌──────────────────────────────────────────────────────────────────────┐
│                  VIDEO DEMO PRINCIPAL (4-6 min)                      │
├──────────────────────────────────────────────────────────────────────┤
│                                                                      │
│  0:00-0:30  INTRODUCCIÓN                                            │
│  ──────────────────────────────────────────────────────────────────│
│  • "Hola, soy [nombre] y este es mi portafolio MLOps"               │
│  • Mostrar GitHub repo                                              │
│  • "3 proyectos ML end-to-end con CI/CD y 5 servicios dockerizados"  │
│                                                                      │
│  0:30-1:00  LEVANTAR EL STACK                                       │
│  ──────────────────────────────────────────────────────────────────│
│  • Mostrar terminal: docker-compose up                              │
│  • Explicar: "Con un solo comando levanto 5 servicios"              │
│  • Mostrar docker ps con los 5 contenedores                         │
│                                                                      │
│  1:00-2:00  TOUR POR LOS 5 SERVICIOS                                │
│  ──────────────────────────────────────────────────────────────────│
│  • MLflow (5000): "Aquí trackeo todos los experimentos"             │
│  • BankChurn API (8001): "API de predicción de churn"               │
│  • CarVision API (8002): "API de precios de vehículos"              │
│  • Streamlit (8501): "Dashboard interactivo para CarVision"         │
│  • TelecomAI API (8003): "Clasificación de planes móviles"          │
│                                                                      │
│  2:00-3:00  DEMO BANKCHURN                                          │
│  ──────────────────────────────────────────────────────────────────│
│  • Mostrar código del pipeline sklearn                              │
│  • Ejecutar predicción en Swagger UI                                │
│  • Mostrar métricas en MLflow                                       │
│                                                                      │
```

```
│  3:00-4:00  DEMO CARVISION + STREAMLIT                                  │
│  ──────────────────────────────────────────────────────────────────────│
│  • Mostrar FeatureEngineer custom transformer                           │
│  • Demo en Streamlit Dashboard (gráficos + predicción)                  │
│  • Mostrar API también funcionando                                      │
│                                                                         │
│  4:00-4:30  CI/CD Y TESTING                                             │
│  ──────────────────────────────────────────────────────────────────────│
│  • Mostrar GitHub Actions                                               │
│  • Tests con 80%+ coverage                                              │
│  • Badge de CI passing                                                  │
│                                                                         │
│  4:30-5:00  ARQUITECTURA                                                │
│  ──────────────────────────────────────────────────────────────────────│
│  • Mostrar diagrama de arquitectura                                     │
│  • Stack: sklearn, MLflow, FastAPI, Streamlit, Docker                   │
│  • Configuración con Pydantic                                           │
│                                                                         │
│  5:00-5:30  CIERRE                                                      │
│  ──────────────────────────────────────────────────────────────────────│
│  • Resumen: "3 proyectos, 5 servicios, 80%+ coverage"                   │
│  • "Todo el código está en GitHub"                                      │
│  • Mostrar URL del repositorio                                          │
│                                                                         │
└─────────────────────────────────────────────────────────────────────────┘
```

Script de Narración

INTRO: "Hola, soy [nombre]. Este es mi portafolio de Machine Learning y MLOps. Incluye tres proyectos completos que se ejecutan como cinco servicios dockerizados: tres APIs con FastAPI, un dashboard con Streamlit, y tracking centralizado con MLflow."

STACK: "Con docker-compose levanto todo el stack. Mira, aquí puedes ver los cinco contenedores corriendo: el servidor de MLflow, las tres APIs de predicción, y el dashboard de Streamlit para CarVision."

TOUR: "Déjame mostrarte cada servicio. En el puerto 5000 tenemos MLflow donde trackeo todos los experimentos. En 8001 está BankChurn para predicción de abandono de clientes. En 8002 CarVision para precios de vehículos. En 8501, que es muy importante, tenemos el dashboard de Streamlit con visualizaciones interactivas. Y en 8003 TelecomAI para clasificación de planes."

BANKCHURN: "Veamos BankChurn. El modelo usa un pipeline unificado de sklearn con ColumnTransformer para preprocesamiento. Aquí hago una predicción en la API... y mira, el cliente tiene 23% de probabilidad de abandonar."

CARVISION: "CarVision tiene algo especial: un custom transformer llamado FeatureEngineer que calcula features como la edad del vehículo. Pero lo mejor es el dashboard de Streamlit... aquí puedo ver análisis de datos y hacer predicciones de forma interactiva. Mira, este Toyota Camry 2020 tiene un precio estimado de $25,000."

CIERRE: "Todo pasa por CI con GitHub Actions y tiene más de 80% de coverage. El código completo está en GitHub. Gracias por ver."

💻 Comandos y Scripts Útiles

Levantar el Stack Completo

```
# Clonar repositorio
git clone https://github.com/DuqueOM/ML-MLOps-Portfolio.git
cd ML-MLOps-Portfolio

# Levantar los 5 servicios principales
docker-compose -f docker-compose.demo.yml up -d

# Esperar a que estén listos
sleep 45

# Verificar TODOS los servicios (5)
echo "=== Estado de los 5 servicios ==="
echo "1. MLflow (5000):"
curl -s http://localhost:5000 >/dev/null && echo "   ✅ Running" || echo "   ❌ Down"

echo "2. BankChurn API (8001):"
```

```
curl -s http://localhost:8001/health && echo ""

echo "3. CarVision API (8002):"
curl -s http://localhost:8002/health && echo ""

echo "4. CarVision Streamlit (8501):"
curl -s http://localhost:8501 >/dev/null && echo '   ✅ {"status":"healthy"}' || echo "   ❌ Down"

echo "5. TelecomAI API (8003):"
curl -s http://localhost:8003/health && echo ""

# Ver contenedores
docker-compose -f docker-compose.demo.yml ps
```

Abrir Todas las URLs

```
# Linux
xdg-open http://localhost:5000 &      # MLflow
xdg-open http://localhost:8001/docs & # BankChurn
xdg-open http://localhost:8002/docs & # CarVision API
xdg-open http://localhost:8501 &      # CarVision Streamlit
xdg-open http://localhost:8003/docs & # TelecomAI

# macOS
open http://localhost:5000
open http://localhost:8001/docs
open http://localhost:8002/docs
open http://localhost:8501
open http://localhost:8003/docs
```

PowerShell (Windows)

```
# Abrir todas las URLs
Start-Process "http://localhost:5000"      # MLflow
Start-Process "http://localhost:8001/docs" # BankChurn
Start-Process "http://localhost:8002/docs" # CarVision API
Start-Process "http://localhost:8501"      # CarVision Streamlit
Start-Process "http://localhost:8003/docs" # TelecomAI
```

Convertir Video a GIF

```
# Método con paleta (mejor calidad)
ffmpeg -i video.mp4 -vf "fps=12,scale=800:-1:flags=lanczos,palettegen" palette.png
ffmpeg -i video.mp4 -i palette.png -filter_complex "fps=12,scale=800:-1:flags=lanczos[x];[x][1:v]paletteuse" output.gif
rm palette.png

# Optimizar tamaño
gifsicle -O3 --colors 128 output.gif -o output-optimized.gif
```

Ejemplos de Predicción para Demo

```
# BankChurn - Cliente que NO abandonará
curl -X POST http://localhost:8001/predict \
  -H "Content-Type: application/json" \
  -d '{
    "credit_score": 750,
    "age": 35,
    "tenure": 8,
    "balance": 125000,
    "num_of_products": 2,
    "has_cr_card": 1,
    "is_active_member": 1,
    "estimated_salary": 95000,
    "geography": "France",
    "gender": "Female"
  }' | jq

# CarVision - Predecir precio
curl -X POST http://localhost:8002/predict \
  -H "Content-Type: application/json" \
  -d '{
    "model_year": 2020,
```

```
    "model": "toyota camry",
    "condition": "good",
    "odometer": 35000,
    "fuel": "gas",
    "transmission": "automatic",
    "type": "sedan"
  }' | jq
```

✅ Checklist Final

Material de Alta Prioridad (5 GIFs)

- [ ] `media/gifs/portfolio-demo.gif` — Demo completo (5 servicios)
- [ ] `media/gifs/bankchurn-preview.gif` — Demo API BankChurn
- [ ] `media/gifs/carvision-preview.gif` — Demo API CarVision
- [ ] `media/gifs/streamlit-carvision.gif` — Demo Streamlit Dashboard ← NUEVO
- [ ] `media/gifs/telecom-preview.gif` — Demo API TelecomAI

Material de Alta Prioridad (Video)

- [ ] Video principal grabado (4 - 6 min)
- [ ] Video subido a YouTube/Drive
- [ ] Link actualizado en README.md

Material de Media Prioridad (Screenshots)

- [ ] `mlflow-experiments.png` — Lista de experimentos
- [ ] `mlflow-metrics.png` — Métricas de un run
- [ ] `swagger-bankchurn.png` — Swagger BankChurn
- [ ] `swagger-carvision.png` — Swagger CarVision
- [ ] `swagger-telecom.png` — Swagger TelecomAI
- [ ] `streamlit-dashboard.png` — Dashboard completo
- [ ] `streamlit-prediction.png` — Resultado de predicción
- [ ] `github-actions-ci.png` — CI pasando

Verificación Final

- [ ] Todos los GIFs pesan < 5 MB
- [ ] Screenshots optimizados
- [ ] Video tiene audio claro
- [ ] READMEs actualizados con GIFs
- [ ] Links funcionan correctamente
- [ ] Git push realizado

📚 Recursos Adicionales

Tutoriales Recomendados

- OBS Studio Quickstart
- ffmpeg GIF Guide
- Streamlit Deployment

Ejemplos de Portafolios con Buenos Demos

- made-with-ml
- mlops-zoomcamp

🔗 Links del Portafolio

Recurso	URL
Repositorio	https://github.com/DuqueOM/ML-MLOps-Portfolio
BankChurn	/BankChurn-Predictor
CarVision	/CarVision-Market-Intelligence
TelecomAI	/TelecomAI-Customer-Intelligence

URLs Locales (con Docker)

Servicio	URL
MLflow	http://localhost:5000
BankChurn API	http://localhost:8001/docs
CarVision API	http://localhost:8002/docs
CarVision Streamlit	http://localhost:8501
TelecomAI API	http://localhost:8003/docs

¡Tu portafolio tiene 5 servicios listos para demostrar! 🚀

GUÍA DE MANTENIMIENTO

GUÍA MLOPS V5.0

PORTFOLIO EDITION

🔧 Guía de Mantenimiento — guia_mlops v5

Meta-documento: Esta guía describe cómo mantener **la guía MLOps en sí misma** actualizada y funcional.

⚠️ **¿Buscas contenido sobre operaciones de sistemas ML en producción?** Ver:
- 17_DESPLIEGUE.md → Operaciones y Runbooks
- 16_OBSERVABILIDAD.md — Monitoreo y alertas
- Runbook del Portafolio — Operaciones end-to-end del portafolio

Última actualización: Diciembre 2025

📊 Estado Actual de la Guía

Componente	Cantidad	Estado
Módulos principales	23	✅ Completos
Ejercicios	42	✅ Con soluciones
ADRs	14	✅ Actualizados
Recursos externos	50+ videos	✅ Curados
Glosario	100+ términos	✅ Expandido

📅 Calendario de Mantenimiento

Mensual

- [ ] Verificar que todos los links funcionan (`./scripts/check_links.sh`)
- [ ] Actualizar versiones de dependencias en `requirements.txt`
- [ ] Ejecutar tests de todos los módulos
- [ ] Verificar que videos de RECURSOS_POR_MODULO.md siguen disponibles

Trimestral

- [ ] Revisar y actualizar ejemplos de código con mejores prácticas
- [ ] Regenerar `requirements.txt` con versiones actuales
- [ ] Verificar compatibilidad con Python más reciente (actualmente 3.11+)
- [ ] Actualizar templates con mejores prácticas
- [ ] Revisar y actualizar RECURSOS_POR_MODULO.md con nuevos videos/cursos

Semestral

- [ ] Revisar estructura completa de la guía (23 módulos)
- [ ] Actualizar referencias y recursos externos
- [ ] Incorporar feedback de usuarios
- [ ] Evaluar nuevas herramientas del ecosistema MLOps
- [ ] Actualizar DECISIONES_TECH.md con nuevas herramientas
- [ ] Revisar que el glosario cubre todos los términos usados en módulos

🔍 Validación de la Guía

Script de Validación

Ejecutar para verificar la integridad de la guía:

```
# Dar permisos de ejecución
chmod +x scripts/validate_guide.sh

# Ejecutar validación
./scripts/validate_guide.sh
```

El script verifica:
1. **Estructura de directorios**: Todos los módulos existen
2. **Archivos requeridos**: mkdocs.yml, requirements.txt, etc.
3. **Links en Markdown**: No hay links rotos

4. **Sintaxis YAML**: Archivos de configuración válidos
5. **Tests por módulo**: Cada módulo tiene tests
6. **Notebooks**: Son JSON válidos

Ejecutar Tests Completos

```
# Activar entorno
source .venv/bin/activate

# Ejecutar todos los tests
make check-all

# O módulo por módulo
make check-01
make check-02
# ...
```

📦 Actualización de Dependencias

Verificar Desactualizadas

```
pip list --outdated
```

Proceso de Actualización

1. **Crear branch de actualización**
   `bash git checkout -b chore/update-deps-YYYY-MM`
2. **Actualizar dependencias**
   `bash pip install --upgrade package-name`
3. **Ejecutar tests**
   `bash pytest docs/ -v`
4. **Si pasan, regenerar lockfile**
   `bash pip freeze > requirements.lock`
5. **Commit y PR**
   `bash git add requirements.txt requirements.lock git commit -m "chore: update dependencies YYYY-MM"`

Auditoría de Seguridad

```
# Instalar herramientas
pip install pip-audit safety

# Verificar vulnerabilidades
pip-audit
safety check
```

🐛 Resolución de Problemas

Tests Fallando

1. Verificar que el entorno está activado
2. Reinstalar dependencias: `pip install -r requirements.txt`
3. Verificar versión de Python: `python --version` (3.10+)
4. Ejecutar test individual para más detalles

Links Rotos

1. Ejecutar `./scripts/validate_guide.sh`
2. Revisar output para links específicos
3. Actualizar o eliminar links rotos

MkDocs No Funciona

1. Verificar instalación: `mkdocs --version`
2. Reinstalar: `pip install mkdocs mkdocs-material`
3. Verificar sintaxis de `mkdocs.yml`

📝 Contribuir a la Guía

Agregar Nuevo Contenido

1. Crear branch: `git checkout -b feat/new-content`
2. Agregar contenido en el módulo correspondiente
3. Agregar tests si aplica
4. Actualizar `mkdocs.yml` si es necesario
5. Ejecutar validación: `./scripts/validate_guide.sh`
6. Crear PR con descripción clara

Estructura de un Módulo

```
docs/XX_nombre_modulo/
├── index.md           # Contenido principal
├── tests/
│   └── test_*.py      # Tests del módulo
└── solutions/
    └── *.py           # Soluciones de ejercicios
```

Convenciones

- Usar **Markdown** estándar
- Incluir **ejemplos de código** ejecutables
- Agregar **ejercicios prácticos** con tests
- Mantener **links relativos** entre módulos

📊 Métricas de Calidad

Objetivos

Métrica	Objetivo	Actual
Tests pasando	100%	✅
Links rotos	0	✅
Módulos completos	23/23	✅
Ejercicios con solución	42/42	✅
ADRs documentados	14/14	✅
Glosario términos	100+	✅
Recursos externos	50+	✅

Monitoreo

Ejecutar semanalmente:

```
./scripts/validate_guide.sh > reports/validation_$(date +%Y%m%d).log
```

📁 Estructura de Archivos de la Guía

```
docs/guia_mlops/
├── 00_INDICE.md              # Índice principal
├── 01-23_*.md                # 23 módulos temáticos
├── EJERCICIOS.md             # 42 ejercicios prácticos
├── EJERCICIOS_SOLUCIONES.md  # Soluciones detalladas
├── RUBRICA_EVALUACION.md     # Sistema de evaluación (100 puntos)
├── RECURSOS_POR_MODULO.md    # 📺 Videos y cursos externos
├── DECISIONES_TECH.md        # 14 ADRs de herramientas
├── 21_GLOSARIO.md            # 100+ términos con ejemplos
├── SIMULACRO_*.md            # Entrevistas técnicas
├── APENDICE_A_SPEECH.md      # Speech de portafolio
├── APENDICE_B_TALKING.md     # Puntos clave
├── SYLLABUS.md               # Programa de 8 semanas
├── PLAN_ESTUDIOS.md          # Cronograma día a día
├── GUIA_AUDIOVISUAL.md       # Crear demos y videos
```

```
├── MAINTENANCE_GUIDE.md      # Esta guía
├── templates/                # 13 plantillas reutilizables
└── mkdocs.yml                # Configuración MkDocs
```

🔗 Recursos Internos

Archivo	Propósito	Actualización
RECURSOS_POR_MODULO.md	Videos y cursos externos	Trimestral
DECISIONES_TECH.md	ADRs de herramientas	Semestral
2 1 _GLOSARIO.md	Definiciones de términos	Mensual
RUBRICA_EVALUACION.md	Sistema de puntuación	Semestral

Recursos Externos

- MkDocs Documentation
- Material for MkDocs
- pytest Documentation

👥 Contacto

- **Mantenedor**: DuqueOM
- **Repositorio**: ML-MLOps-Portfolio